UPCT · GRADO EN INGENIERÍA BIOMÉDICA · 3.º CURSO

Última actualización: 9 de noviembre de 2026

Electrónica Digital

Guía para aprobar

14	11	18	1101
temas (T0–T13)	problemas de examen resueltos	problemas de hojas resueltos	comprobaciones por código

PORTADA

Empieza aquí

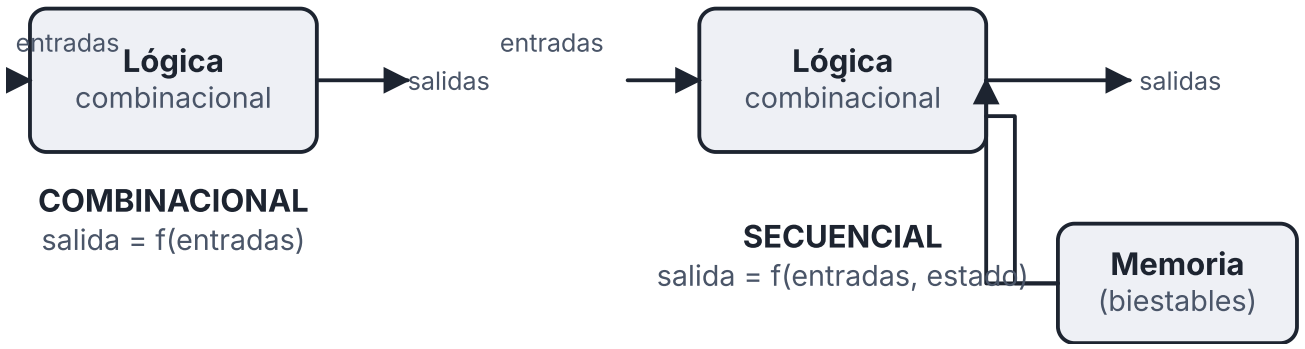
Qué es la asignatura, lo que necesitas saber antes de empezar y cómo usar esta guía.

Qué vas a aprender

Electrónica Digital es una asignatura obligatoria de **3.º de Ingeniería Biomédica** (4,5 ECTS, primer cuatrimestre). Aprendes a **diseñar circuitos que trabajan con unos y ceros**: desde una puerta que decide si se abre un toldo hasta una máquina que reconoce la combinación de una caja fuerte.

Todo lo que se construye en electrónica digital se reduce a dos tipos de circuitos:

- **Combinacionales**: la salida depende *solo* de las entradas de ahora. No tienen memoria. Ejemplo: un sumador.
- **Secuenciales**: la salida depende de las entradas de ahora *y de lo que pasó antes*. Tienen memoria. Ejemplo: un contador.



Todo circuito digital es una de estas dos cosas, o una mezcla: lógica combinacional sola, o lógica combinacional más memoria (biestables) realimentada.

El temario oficial se reparte en cuatro unidades:

Unidad	Temas	Qué trabajas
UD1 · Introducción	0 a 3	Numeración, códigos, álgebra de Boole
UD2 · Combinacional	4 y 5	Familias lógicas, sumadores, multiplexores, decodificadores, comparadores
UD3 · Secuencial	6 a 11	Biestables, memorias, contadores, registros, autómatas
UD4 · Lógica programable	12 y 13	FPGA y lenguaje VHDL

La evaluación se reparte en **80 % examen escrito** y **20 % prácticas**, y hay que sacar **al menos un 4,0 en cada parte** para aprobar. Todos los detalles están en El examen y cómo estudiar.

Repaso desde cero

No hace falta saber electrónica. Hace falta tener claras cinco ideas.

1. Un bit es un interruptor

Un **bit** vale 0 o 1. En un circuito real, 0 y 1 son dos rangos de tensión (por ejemplo, cerca de 0 V y cerca de 5 V). Con n bits puedes escribir 2^n combinaciones distintas.

$$N = 2^n \qquad \Longleftrightarrow \qquad n = \lceil \log_2 N \rceil$$

Con 3 bits hay 8 combinaciones; con 4 bits, 16. Si necesitas distinguir 10 cosas (los dígitos del 0 al 9), hacen falta 4 bits, porque con 3 solo llegas a 8.

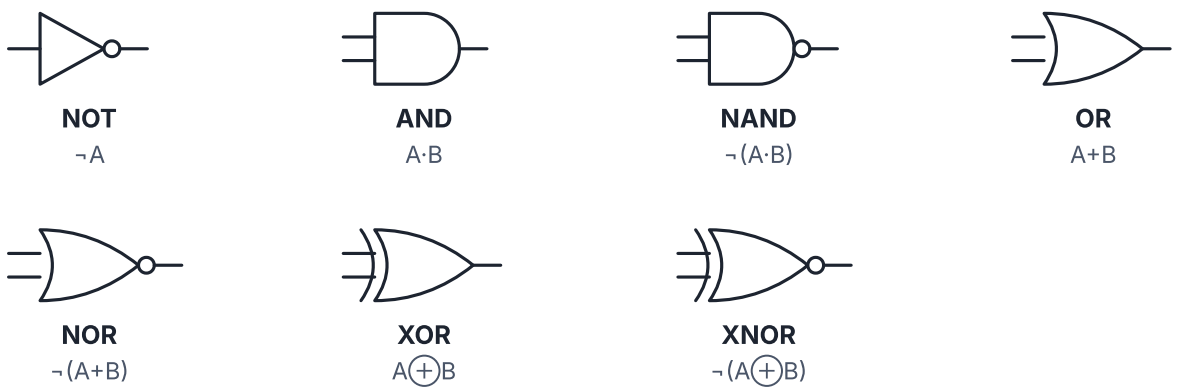
2. Contar en binario es contar con dos dedos

En decimal cada posición vale 10 veces la anterior. En binario, 2 veces.

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

3. Las tres puertas básicas

- **AND** ($A \cdot B$): vale 1 solo si *todas* las entradas valen 1.
- **OR** ($A + B$): vale 1 si *alguna* entrada vale 1.
- **NOT** ($\overline{A}$): invierte.



Las siete puertas que vas a usar. El círculo pequeño a la salida significa «negado».

4. Tabla de verdad

Una tabla de verdad es la lista de **todas** las combinaciones de entradas con la salida que quieres para cada una. Es el punto de partida de casi cualquier diseño combinacional: si no sabes qué circuito hacer, escribe la tabla.

5. El reloj

En los circuitos secuenciales hay una señal de **reloj** que sube y baja siempre igual. Los biestables solo miran sus entradas en un instante: el **flanco** (el momento en que el reloj sube, o baja). Entre flancos, lo que pasa en las entradas no cambia la memoria.

Cómo usar esta guía

1. **Empieza por la teoría de cada tema.** Está escrita con la intuición primero y las fórmulas después. Cada tema acaba con un recuadro de *Qué suele fallar* y otro de *Fórmulas clave*.
2. **Juega con los simuladores.** Si no entiendes algo, muévelo: cambia un bit, una secuencia, una tensión. Verás el resultado al momento.
3. **Haz los ejercicios sin mirar.** Cada ejercicio tiene el enunciado, un recuadro *Antes de empezar* y la resolución en pasos plegados. Abre un paso solo cuando estés atascado.
4. **Practica con exámenes reales.** En Exámenes resueltos tienes los que han salido, con búsqueda por tema.
5. **Cierra el círculo** con la autoevaluación, los simulacros y el plan de estudio.

> **⚠️ Cómo se ha comprobado todo**
>
> Cada resultado numérico, cada ecuación minimizada y cada secuencia de estados de esta guía se ha **comprobado con un programa** que simula el circuito entrada por entrada. Los ejercicios con la etiqueta «✔ verificado» pasan esa comprobación. Cuando una solución de referencia tenía un fallo, no se ha copiado: hay una nota **Ojo** que dice dónde está y por qué.

> **📘 Notación que usa la guía**
>
> - Decimales con coma en el texto ($0{,}5$) y con punto en el código (`0.5`).
> - $\overline{A}$ es «A negada». $A \cdot B$ se escribe a veces AB. $\oplus$ es la XOR.
> - Q_n es el estado actual y Q_{n+1} el estado siguiente.
> - Los números en base 2 llevan subíndice (1011_2); sin subíndice, son decimales.

EMPIEZA AQUÍ

El examen y cómo estudiar

Cómo se evalúa, qué cae más según los exámenes reales y una estrategia para sacar nota.

Cómo se evalúa la asignatura

Parte	Peso	Qué es
Exámenes escritos	**80 %**	Cuestiones conceptuales, análisis y síntesis de circuitos digitales, interpretación de diseños y resolución de casos prácticos.
Prácticas	**20 %**	Memorias e informes, y defensa del trabajo de laboratorio (individual o en grupo).

Reglas que conviene tener presentes:

- Para aprobar hace falta **un 4,0 como mínimo tanto en la parte teórica como en la práctica**.
- La asistencia a las prácticas es obligatoria: hace falta al menos un **80 %** para que cuente como hecha.
- Si ya superaste una actividad en la evaluación continua y quieres presentarte a esa misma actividad en la evaluación final, **debes renunciar a la nota que tenías**.
- Las prácticas 4 a 7 permiten usar inteligencia artificial como ayuda, pero tienes que **entender, revisar y justificar** el código. En la defensa te preguntarán por él.
- También hay pruebas cortas en clase, individuales o por parejas (evaluación formativa).

> ⚠️ **Fechas y ponderaciones**
>
> Esta guía no inventa fechas ni notas. Mira en tu calendario académico cuándo es la convocatoria. Lo que sí sabes: el examen escrito vale el 80 % y debes sacar un 4,0 como mínimo.

Cómo son los exámenes de otros años

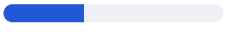
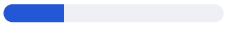
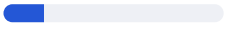
Los exámenes reales que han servido para preparar esta guía son de la **misma asignatura en la titulación de Ingeniería Electrónica Industrial y Automática** de la misma escuela. El contenido es esencialmente el mismo. La organización exacta del examen puede variar de un curso a otro, así que comprueba siempre la de tu convocatoria.

Lo que se repite en todos ellos:

Dato	Lo que dicen los exámenes
Número de problemas	4
Valor de cada problema	2,5 puntos (examen de febrero de 2023)
Tiempo	2 horas (examen de diciembre de 2024)
Cómo se corrige	«Se tendrá en cuenta la simplicidad de la solución propuesta, así como la claridad de realización del mismo» (febrero de 2023 y febrero de 2025)
Cronogramas	«No se contabilizarán los ejercicios que sólo dibujen las formas de onda sin explicar nada» (diciembre de 2024)
Notas de aprobado (febrero de 2023)	Examen escrito ≥ 5 y prácticas ≥ 1 sobre 2; nota mínima de la asignatura 6 puntos. En tu titulación rige el 80 %/20 % de arriba.

La última fila es una norma de *esa* convocatoria. Se incluye solo para que veas qué tipo de normas aparecen.

Qué cae más

Tipo de problema	Problemas	Peso	Aparece en
Bloques MSI y displays	6 de 11		3 de 3 exámenes
Autómatas / síntesis secuencial	5 de 11		3 de 3 exámenes
Aritmética y códigos (BCD, C2)	4 de 11		3 de 3 exámenes
Combinacional (puertas, Karnaugh)	3 de 11		3 de 3 exámenes
Contadores y biestables	2 de 11		1 de 3 exámenes

Recuento propio sobre los 11 problemas de los 3 exámenes resueltos aquí (un problema puede tener varias etiquetas). Son pocos exámenes: úsalo como orientación, no como regla.

Mira la forma que tiene un examen típico. Casi siempre hay:

1. **Un problema de diseño combinacional con bloques estándar** (sumadores, comparadores, multiplexores, displays): «haz la media de…», «calcula $A \cdot B + C$ y muéstralo en displays», «un recinto con dos puertas».
2. **Un problema pequeño de lógica o de contadores**: simplificar una función con Karnaugh y pasarla a NAND, o analizar un circuito dado y dibujar su cronograma.

3. **Un problema de síntesis de un contador** con biestables JK y puertas, con una entrada de control (sube/baja).
4. **Un autómata**: la caja fuerte, la noria, la máquina expendedora... Diagrama de estados, tabla de transiciones, ecuaciones y circuito.

El problema 4 suele ser el más largo. Reserva tiempo.

Estrategia para el examen

> 💡 **Reparto de las 2 horas**
>
> - **5 min**: lee los cuatro enunciados completos y marca qué te sabes mejor.
> - **25-30 min por problema**, empezando por el que tengas más claro.
> - **10 min al final** para revisar: ¿cada problema tiene diagrama/tabla, ecuaciones y circuito?
>
> Un problema empezado con buena idea y un esquema claro puntúa más que uno en blanco.

Reglas prácticas que se repiten en las correcciones:

- **Escribe los pasos.** Un cronograma sin explicar cómo se obtuvo no puntúa. Una tabla de verdad sin la ecuación tampoco.
- **Define tus entradas, salidas y estados** antes de dibujar. Escribe qué significa cada estado con palabras.
- **Si el enunciado es ambiguo, di qué supones** y sigue. Es lo que se espera.
- **Simplifica**: la solución más sencilla puntúa mejor. Usa los indiferentes (×) en Karnaugh.
- **Comprueba con un caso**: antes de entregar, mete una entrada concreta en tu circuito y mira que sale lo que debe.

Tu seguimiento

Marca lo que vas haciendo. Se guarda en este navegador.

> 📈 **Tu progreso** (se guarda en este navegador)
>
> 0 de 47 tareas (0 %)

Tema 0 · Introducción a la asignatura

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 1 · Introducción a los sistemas digitales

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 2 · Sistemas de numeración, operaciones y códigos

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 3 · Álgebra de Boole y funciones lógicas

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 4 · Familias lógicas

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 5 · Funciones de la lógica combinacional

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 7 · Memorias y circuitos lógicos programables

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 8 · Contadores

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 9 · Registros de desplazamiento

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 11 · Descripciones funcionales y estructurales

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 12 · Introducción a los dispositivos lógicos programables

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Tema 13 · Introducción al diseño digital con VHDL

- [ ] Leer la teoría y entender las figuras
- [ ] Hacer los ejemplos sin mirar la solución
- [ ] Test de autoevaluación del tema

Bloques de examen

- [ ] Resolver 2 exámenes completos en 2 horas (sin mirar la teoría)
- [ ] Rehacer los problemas que fallaste
- [ ] Hojas de problemas: combinacional
- [ ] Hojas de problemas: autómatas
- [ ] Repasar las prácticas y el código VHDL

TEORÍA POR TEMAS

Tema 0 · Introducción a la asignatura

Qué se pide, con qué herramientas y cómo encaja cada tema.

El mapa en una página

La asignatura recorre este camino. Cada paso usa lo del anterior.

combinacional → secuencial → lógica programable

De los números al diseño con FPGA. Los temas de la izquierda son la base; los de la derecha son lo que se ve en el laboratorio.

Cuatro resultados de aprendizaje te dicen qué se espera de ti:

- **Explicar** cómo funcionan los sistemas digitales.
- **Analizar y sintetizar** circuitos digitales. *Analizar* es entender uno que ya existe. *Sintetizar* es diseñar uno a partir de una descripción.
- **Usar** instrumentación básica y software de simulación.
- **Programar** una FPGA con esquemáticos y con VHDL.

Dos verbos que lo resumen todo

💡 Analizar frente a sintetizar

- **Analizar**: te dan el circuito y te piden la función, la tabla o el cronograma. Camino: circuito → ecuaciones → tabla → comportamiento.
- **Sintetizar**: te dan una descripción en palabras y te piden el circuito. Camino: palabras → tabla o diagrama de estados → ecuaciones → circuito.

Son el mismo camino recorrido en sentidos opuestos. Si dominas uno, el otro es el mismo mapa al revés.

Herramientas del laboratorio

Se trabaja con placas de desarrollo **Basys** con una **FPGA de Xilinx** y el entorno **Xilinx ISE**. Una FPGA es un chip que puedes «reconfigurar» para que se comporte como el circuito digital que has diseñado. Se describe el circuito de tres maneras equivalentes:

1. **Esquemático**: dibujas las puertas y los bloques.
2. **Diagrama de estados**: dibujas los estados y las transiciones.
3. **VHDL**: lo escribes como texto.

El proyecto final es un sistema de monitorización de una **incubadora neonatal** usada como entorno de simulación. Lo verás en Prácticas.

> **🚧 Qué suele fallar**
>
> - Empezar a dibujar circuitos antes de escribir la **tabla de verdad o el diagrama de estados**. Todo diseño bueno empieza ahí.
> - Confundir *analizar* con *sintetizar* y calcular al revés.
> - No decir las suposiciones cuando el enunciado es ambiguo.

TEORÍA POR TEMAS

Tema 1 · Introducción a los sistemas digitales

Qué significa «digital», por qué se usa y cómo se representan 0 y 1.

Analógico frente a digital

Una señal **analógica** puede tomar cualquier valor entre dos extremos (la tensión de un micrófono). Una señal **digital** solo toma un conjunto finito de valores, casi siempre dos: 0 y 1.

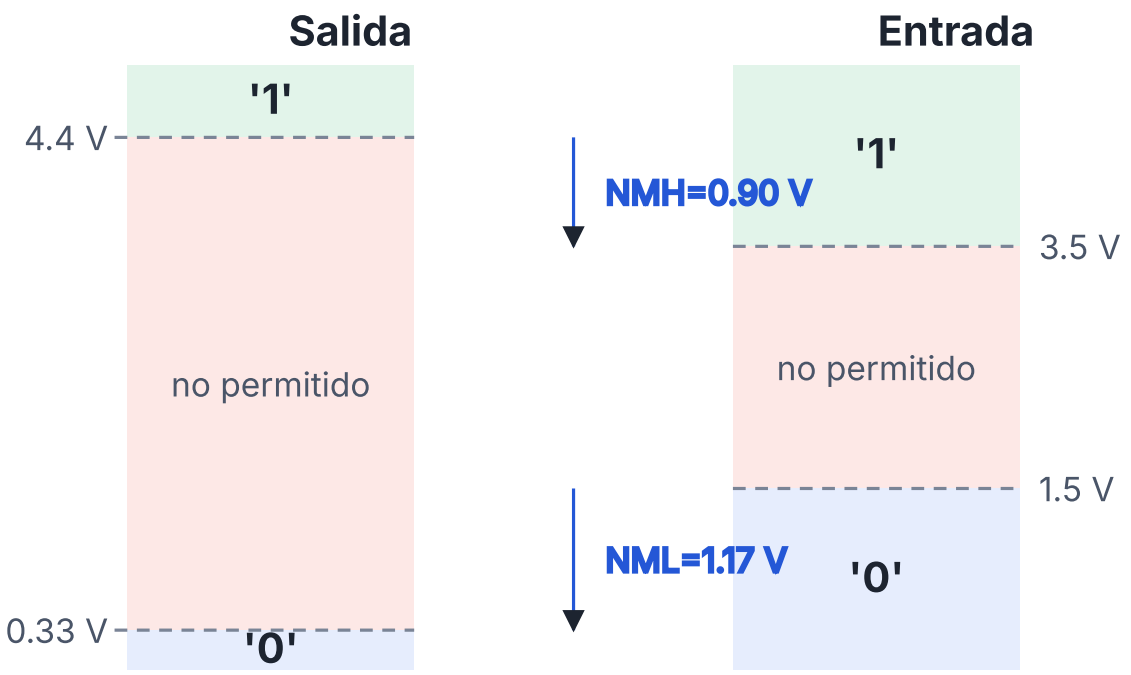
¿Por qué se prefiere lo digital?

- **Aguanta el ruido.** Si un 1 es «unos 5 V» y le llegan 4,6 V, sigue siendo un 1. El circuito siguiente lo limpia.
- **Se puede copiar sin pérdida.** Un fichero se duplica bit a bit.
- **Es programable.** El mismo hardware hace cosas distintas según el código.

El precio: hay que convertir lo real (temperatura, presión, señal ECG) en números. Eso lo hacen los convertidores A/D.

Representar 0 y 1 con tensiones

Un 1 y un 0 son **rangos** de tensión, no valores exactos. Entre ellos hay una zona prohibida. Lo verás en detalle en el Tema 4.



Niveles lógicos de una familia CMOS de 5 V

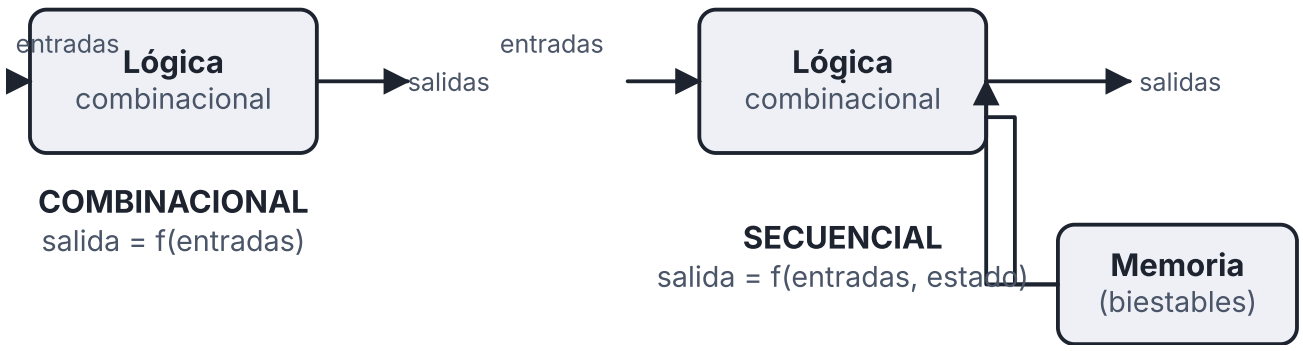
A la salida, un 1 es cualquier tensión por encima de 4,4 V. A la entrada, el circuito acepta como 1 todo lo que supere 3,5 V. La diferencia (0,9 V) es el margen de ruido.

Lógica positiva y negativa

En **lógica positiva** el nivel alto significa 1. En **lógica negativa**, el nivel alto significa 0. Casi todo lo que verás es lógica positiva. Pero en las placas Basys los segmentos de los displays y algunos pulsadores son *activos a nivel bajo*: se encienden con un 0.

> 📘 **Señal activa a nivel alto o bajo**
>
> Una señal está **activa a nivel alto** si hace su trabajo cuando vale 1, y **activa a nivel bajo** si lo hace cuando vale 0. En los esquemas, la activa a nivel bajo lleva un círculo en su pin o una barra encima ($\overline{CLR}$, $\overline{LD}$).

Tipos de sistemas digitales



Un **sistema combinacional** no recuerda nada. Un **sistema secuencial** tiene memoria y un reloj. Esta división organiza todo el temario:

	Combinacional	Secuencial
Salida depende de	Entradas actuales	Entradas actuales y pasado
Elemento básico	Puerta lógica	Biestable
Se describe con	Tabla de verdad, ecuación	Diagrama de estados, tabla de transiciones
Ejemplos	Sumador, multiplexor, decodificador	Contador, registro, autómata

🚧 Qué suele fallar

- Pensar que una puerta «recuerda». No: una puerta solo calcula.
- Olvidar que un 0 o un 1 reales son *rangos*. Dos circuitos solo se pueden conectar directamente si sus rangos encajan (Tema 4).

TEORÍA POR TEMAS

Tema 2 · Sistemas de numeración, operaciones y códigos

Cómo escribe números un circuito: binario, complemento a dos, BCD, Gray y detección de errores.

💡 **Desde cero**

Un circuito solo guarda ceros y unos. Para guardar el número 156, el 37 negativo o la letra «A», hace falta un **acuerdo** sobre qué significa cada combinación de bits. Ese acuerdo es un **código**. Este tema recoge los códigos que vas a usar siempre: binario natural, hexadecimal, complemento a dos, BCD, Gray y paridad.

Binario, octal y hexadecimal

En decimal cada posición vale una potencia de 10. En binario, una potencia de 2:

$$1011_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11_{10}$$

Del decimal al binario: divide entre 2 una y otra vez y apunta los restos. Lee los restos de abajo arriba.

Hexadecimal: agrupa los bits de 4 en 4 desde la derecha. Cada grupo es una cifra (0-9, A=10, B=11, … F=15). Es la forma corta de escribir binario. **Octal:** agrupa de 3 en 3.

Ejemplo T2 **De decimal a binario, hexadecimal y octal** ✔ verificado

Pasa 156_{10} a binario, hexadecimal y octal.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Usas la división sucesiva entre 2 y la agrupación de bits en bloques de 4 (hex) y de 3 (octal).

1 **Divisiones entre 2**

$156 \to 78 \to 39 \to 19 \to 9 \to 4 \to 2 \to 1 \to 0$ con restos $0, 0, 1, 1, 1, 0, 0, 1$. Leídos de abajo arriba: 10011100_2.

2 **Agrupa de 4 en 4**

$1001\ 1100$ = $9\ C$. Es decir, $9C_{16}$.

3 **Agrupa de 3 en 3**

$010\ 011\ 100$ = $2\ 3\ 4$. Es decir, 234_8. (Se rellena con un 0 a la izquierda.)

✅ **Resultado final**

$156_{10} = 10011100_2 = 9C_{16} = 234_8$. Comprobación: $9 \cdot 16 + 12 = 156$.

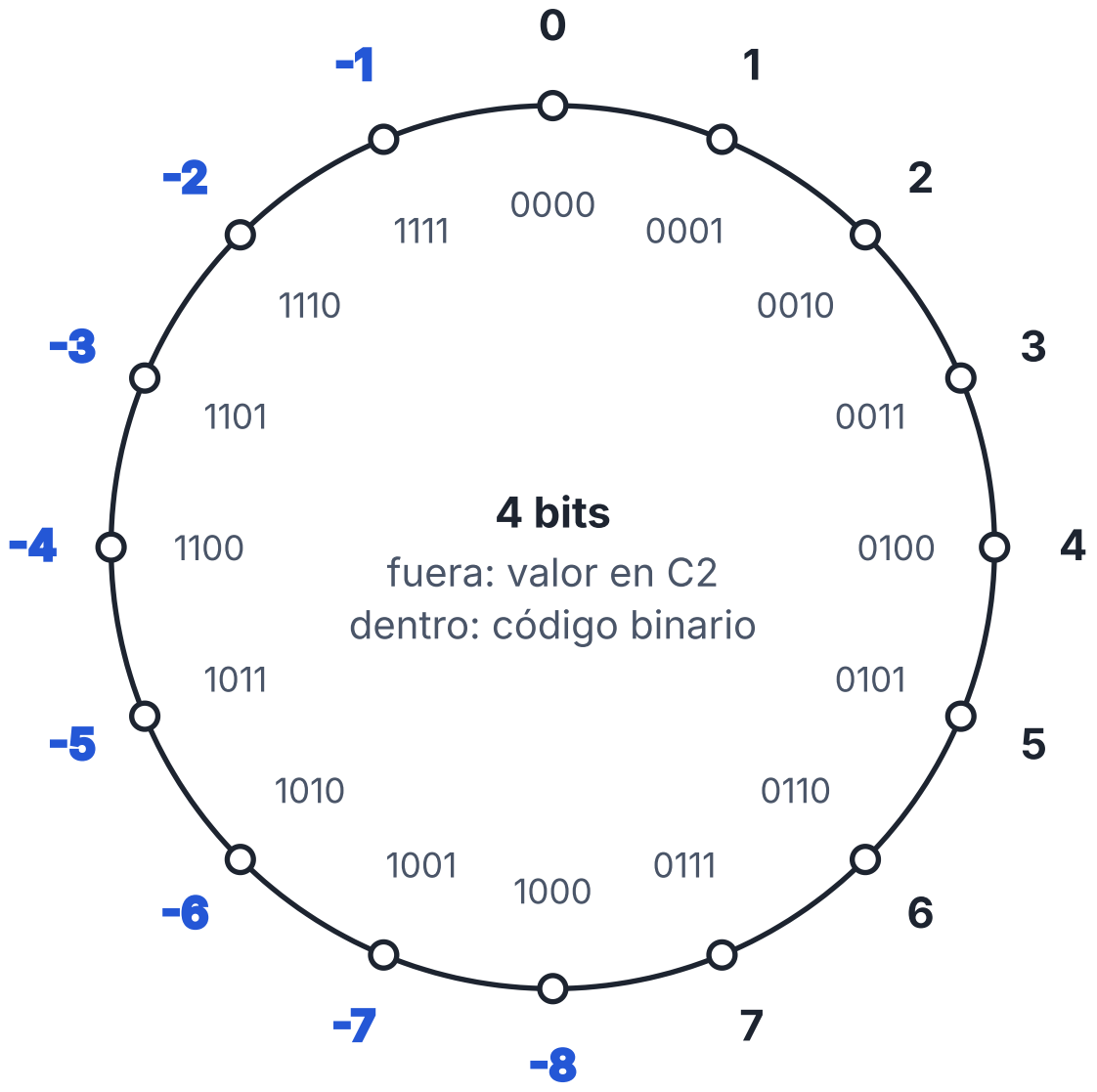
Parte fraccionaria: multiplica por 2 y apunta la parte entera cada vez. Ejemplo: $0{,}6875 \to 1{,}375 \to 0{,}75 \to 1{,}5 \to 1{,}0$ da $0{,}1011_2$.

⚠️ **Muchos decimales no caben en binario**

$0{,}1_{10}$ en binario es periódico, igual que $1/3$ en decimal. Por eso un circuito nunca guarda $0{,}1$ exacto: guarda una aproximación.

Números con signo: complemento a dos

Con n bits hay 2^n combinaciones. Para repartirlas entre positivos y negativos se usa el **complemento a dos (C2)**. El bit más significativo (MSB) es el **signo**: 0 positivo, 1 negativo.



La rueda del complemento a 2 con 4 bits: tras el 7 viene el −8. Sumar es girar en el sentido de las agujas del reloj.

Rango con n bits:

$$-2^{n-1} \leq N \leq 2^{n-1} - 1$$

Con 8 bits: de -128 a $+127$. Hay un negativo más que positivos.

Cambiar de signo: invierte todos los bits y suma 1. Ejemplo con $37 = 00100101$: invierto, 11011010; sumo 1, $11011011 = -37$.

Intuición: en C2 el peso del MSB es -2^{n-1} (negativo). Por eso $11011011 = -128 + 64 + 16 + 8 + 2 + 1 = -37$.

Ejemplo T2 Sumar en complemento a dos y detectar desbordamiento

✔ verificado

Con 8 bits en C2: (a) escribe -37; (b) calcula $45 + (-20)$; (c) calcula $100 + 50$ y di qué pasa.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: C2 = invertir y sumar 1; la suma de C2 es la suma binaria normal; hay **desbordamiento** si el resultado no cabe en el rango $[-128, 127]$.

1 **(a)** -37

$37 = 00100101$. Invierto: 11011010. Sumo 1: 11011011.

2 **(b)** $45 + (-20)$

$45 = 00101101$, $-20 = 11101100$. Suma: $1\,00011001$. Se descarta el acarreo que sale del byte: queda $00011001 = 25$. Correcto.

3 **(c)** $100 + 50$

$100 = 01100100$, $50 = 00110010$. Suma: 10010110. El MSB vale 1, así que se lee como negativo: -106. Es **incorrecto**: $150 > 127$. Hay desbordamiento.

✅ Resultado final

(a) 11011011. (b) $00011001 = 25$. (c) 10010110 (se leería -106) y hay **desbordamiento**.

∑ Cómo detectar el desbordamiento en C2

- Dos positivos que suman un negativo, o dos negativos que suman un positivo: hay desbordamiento.
- Equivale a: el acarreo que entra al bit de signo es distinto del acarreo que sale de él.
- Un positivo más un negativo **nunca** desborda.

SIMULADOR · SUMA EN COMPLEMENTO A 2 Y DESBORDAMIENTO

Número de bits

8

Sumando A (decimal con signo)

100

Sumando B (decimal con signo)

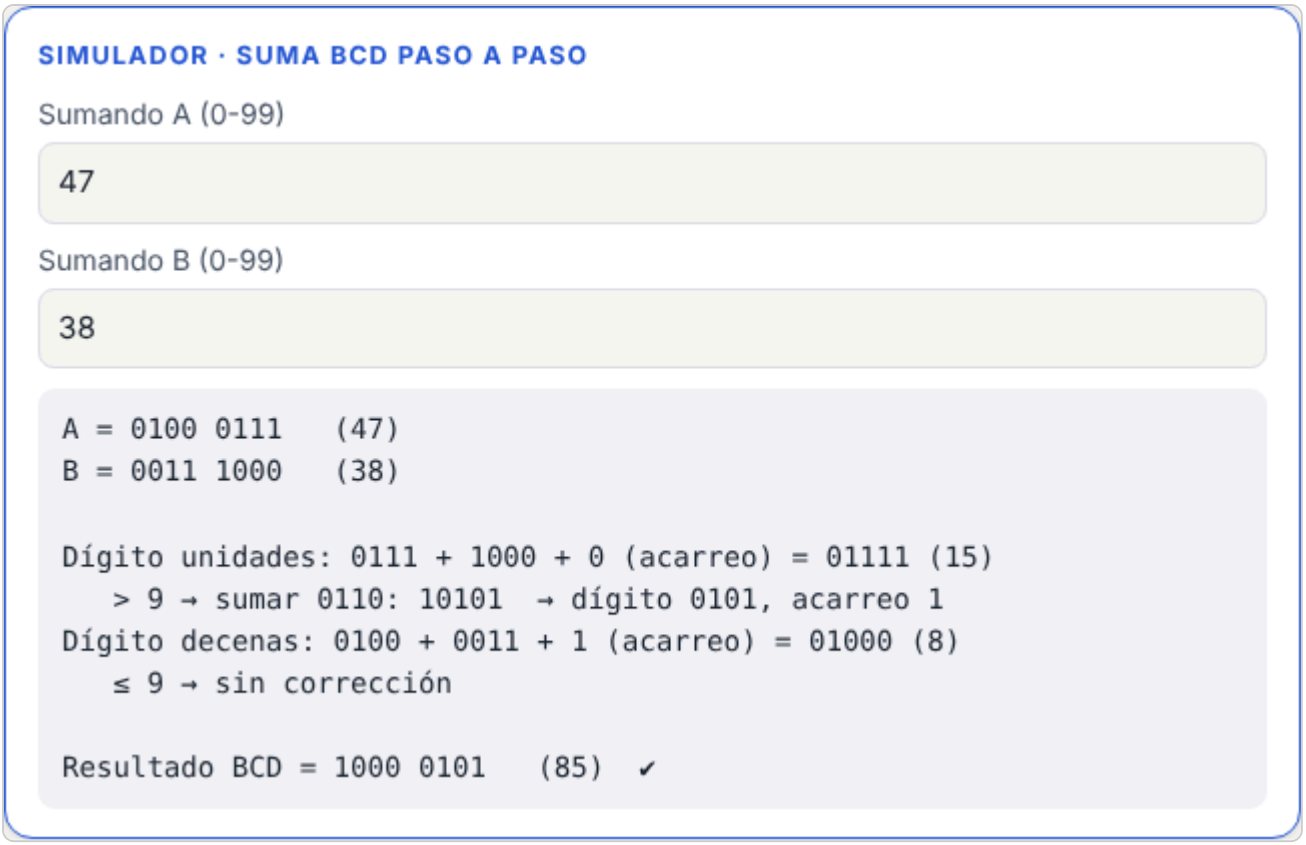
50

```
A = 01100100  (100)
B = 00110010  (50)
A+B = 10010110  → se lee -106

Acarreo que entra al bit de signo: 1
Acarreo que sale del bit de signo: 0
Desbordamiento V = 1 ⊕ 0 = 1

✘ DESBORDAMIENTO: la suma real es 150 y no cabe en 8 bits (-128 a
127).
```

Captura del simulador interactivo. Pruébalo en la versión web.



Captura del simulador interactivo. Pruébalo en la versión web.

BCD: decimal en cuatro bits

En **BCD** (decimal codificado en binario) cada cifra decimal se escribe con 4 bits, y solo se usan las combinaciones 0000 a 1001. Así $47_{10} = 0100\ 0111_{BCD}$. Es cómodo para mostrar números en displays, pero ocupa más bits y sumar es más difícil.

Suma en BCD: suma cada cifra en binario. Si el resultado de una cifra supera 9 (o hay acarreo), **suma 6** para saltarte las 6 combinaciones prohibidas (1010 a 1111).

Ejemplo T2 — Suma en BCD: 38 + 45

✔ verificado

Suma $38 + 45$ en BCD.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: BCD, y la corrección «+6» cuando una cifra pasa de 9.

1 Escribe los sumandos

$38 = 0011\ 1000$ y $45 = 0100\ 0101$.

2 **Suma binaria**

$0011\,1000 + 0100\,0101 = 0111\,1101$.

3 **Corrige la cifra de abajo**

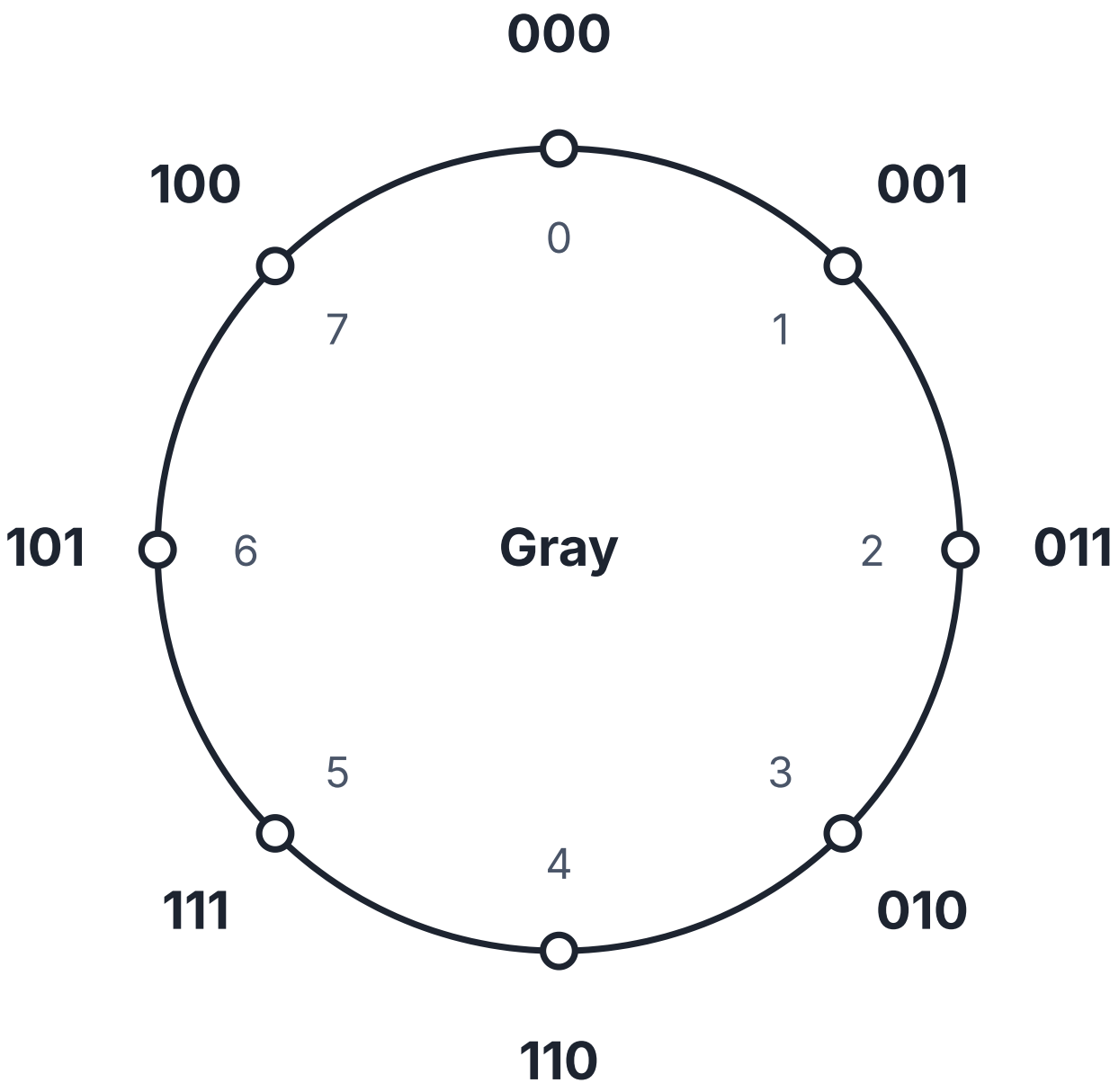
Las cifras bajas suman $1101 = 13 > 9$: es inválido. Suma 0110: $0111\,1101 + 0110 = 1000\,0011$.

✅ **Resultado final**

$1000\ 0011_{BCD} = 83$. Comprobación: $38 + 45 = 83$.

Código Gray y detección de errores

Gray: una secuencia de códigos donde de uno al siguiente **cambia un solo bit**. Para 3 bits: $000, 001, 011, 010, 110, 111, 101, 100$. Se usa en codificadores de posición: si dos bits cambiaran a la vez y uno llegara antes, leerías una posición falsa. Con Gray el error máximo es una posición.



En el código Gray de 3 bits, dos vecinos cualesquiera de la rueda se diferencian en un solo bit (también el último con el primero).

Paso de binario a Gray: $g_i = b_i \oplus b_{i+1}$ (el MSB se copia). Es decir, $g = b \oplus (b \gg 1)$.

Paridad: se añade un bit para que el número total de unos sea par (paridad par) o impar. Detecta **un** error, pero no sabe dónde está. Para 1011001 hay cuatro unos: el bit de paridad par es 0.

Hamming (7,4): añade 3 bits de paridad a 4 de datos de forma que se **corrige** un error. Los bits p_1, p_2, p_3 están en las posiciones 1, 2 y 4. Para los datos 1011 la palabra es 0110011. Si falla un bit, los tres chequeos de paridad forman un número binario que **es la posición del bit erróneo**.

🔭 Más a fondo: distancia de Hamming

La **distancia** entre dos palabras es el número de bits en que difieren. Un código con distancia mínima d detecta $d-1$ errores y corrige $\lfloor (d-1)/2 \rfloor$. La paridad simple tiene $d=2$ (detecta 1, corrige 0). Hamming (7,4) tiene $d=3$ (corrige 1). Se ha comprobado por código que corrige cualquiera de los 7 posibles errores de un bit en las 16 palabras.

🚧 Qué suele fallar

- Olvidar que en C2 el rango **no es simétrico**: con 8 bits el mínimo es -128, y no tiene opuesto positivo.
- Descartar el acarreo final y creer que ya está: hay que mirar el **desbordamiento** aparte.
- En BCD, sumar 6 a todas las cifras en vez de solo a las que pasan de 9.
- Mezclar «mayor que 9» con «hay acarreo»: ambas condiciones obligan a corregir.

∑ Fórmulas clave

- N combinaciones $\Rightarrow n = \lceil \log_2 N \rceil$ bits.
- C2 de n bits: $-2^{n-1} \leq N \leq 2^{n-1}-1$.
- $-N = \overline{N} + 1$ (invertir y sumar 1).
- Gray: $g = b \oplus (b \gg 1)$.
- Suma BCD: si cifra > 9, sumar 0110.

¿Rango de un número en C2 de n bits?

-2^{n-1} a $2^{n-1}-1$.

¿Cómo se niega un número en C2?

Invertir todos los bits y sumar 1.

¿Cuándo hay desbordamiento en C2?

Cuando sumas dos números del mismo signo y el resultado tiene signo contrario.

¿Corrección en la suma BCD?

Si la cifra pasa de 9 (o hay acarreo), sumar 0110.

¿Qué caracteriza al código Gray?

Entre dos códigos consecutivos cambia un solo bit.

BCD

Decimal codificado en binario: cada cifra decimal en 4 bits.

Complemento a dos

Representación de enteros con signo: el MSB pesa -2^{n-1}.

Desbordamiento

El resultado de una operación no cabe en el número de bits disponible.

Código Gray

Código en el que dos palabras consecutivas difieren en un solo bit.

Distancia de Hamming

Número de bits en que difieren dos palabras de un código.

Paridad

Bit extra que hace par (o impar) el número de unos de una palabra.

¿Cuál es el rango de un entero en complemento a dos con 6 bits?

A de -31 a $+32$

B de -32 a $+32$

C de 0 a 63

D de -32 a $+31$

Con $n = 6$: $-2^5 = -32$ hasta $2^5 - 1 = 31$.

En C2 de 8 bits sumas $01100100 + 00110010$. ¿Qué ocurre?

A Hay un error de paridad

B Se produce acarreo y el resultado es correcto

C Hay desbordamiento: dos positivos dan un resultado con MSB igual a 1

D No hay problema: 150 cabe en 8 bits

$100 + 50 = 150 > 127$. El resultado 10010110 se lee como -106.

$0100\,0111$ en BCD representa:

A 47

B 0x47 en decimal, es decir, 71

C 71

D 4,7

Cada grupo de 4 bits es una cifra decimal: 4 y 7.

Para 3 bits, ¿qué código de Gray sigue a 011?

A 000

B 100

C 010

D 111

La secuencia es 000, 001, 011, 010, 110, 111, 101, 100.

TEORÍA POR TEMAS

Tema 3 · Álgebra de Boole y funciones lógicas

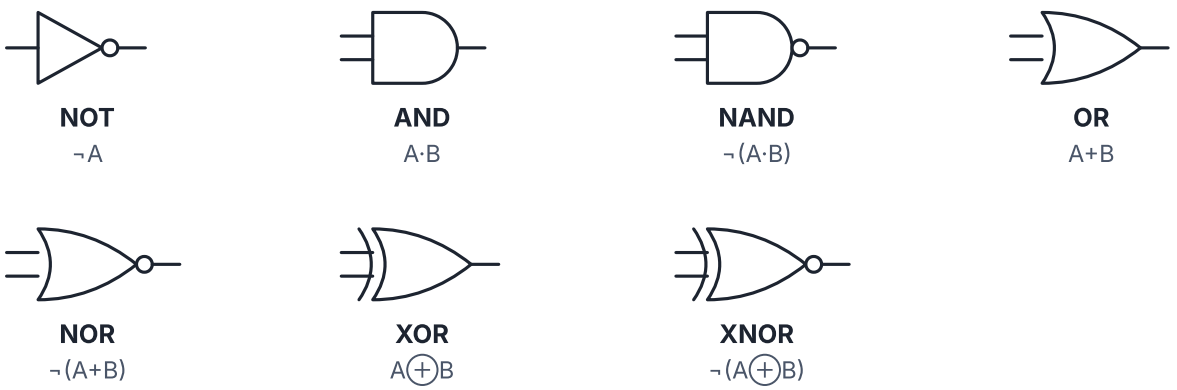
Puertas, tablas de verdad, formas canónicas y simplificación con mapas de Karnaugh.

💡 Desde cero

Una **función lógica** es una regla que dice, para cada combinación de entradas (0/1), si la salida es 0 o 1. Se puede escribir de tres formas equivalentes: **tabla de verdad**, **ecuación** y **circuito de puertas**. El álgebra de Boole es el conjunto de reglas para pasar de una a otra y, sobre todo, para **simplificar**: menos puertas es menos coste, menos consumo y menos retardo.

Las operaciones básicas

Operación	Símbolo	Se lee	Vale 1 cuando
NOT	$\overline{A}$	no A	$A = 0$
AND	$A \cdot B$	A y B	las dos entradas valen 1
OR	$A + B$	A o B	al menos una vale 1
XOR	$A \oplus B$	A o exclusivo B	las entradas **son distintas**
NAND	$\overline{A \cdot B}$		no las dos a 1
NOR	$\overline{A + B}$		las dos a 0



Símbolos de las puertas básicas. El círculo a la salida significa «negado».

Captura del simulador interactivo. Pruébalo en la versión web.

Las puertas NAND y NOR son universales: con solo una de ellas puedes construir cualquier circuito. Se ha comprobado por código que con NAND se obtienen NOT, AND y OR.

Reglas que debes saber de memoria

Regla	Forma 1	Forma 2
Identidad	$A + 0 = A$	$A \cdot 1 = A$
Nulo	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotencia	$A + A = A$	$A \cdot A = A$
Complemento	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
Absorción	$A + AB = A$	$A(A + B) = A$

Regla	Forma 1	Forma 2
Adyacencia	$AB + A\overline{B} = A$	$(A + B)(A + \overline{B}) = A$
Consenso	$AB + \overline{A}C + BC = AB + \overline{A}C$	
De Morgan	$\overline{A + B} = \overline{A} \cdot \overline{B}$	$\overline{A \cdot B} = \overline{A} + \overline{B}$

Todas estas identidades se han comprobado por código con las tablas de verdad completas.

> 💡 **De Morgan en una frase**
>
> Para negar una expresión: **cambia cada AND por OR (y al revés) y niega cada variable**. Por eso una puerta NAND es lo mismo que una OR con las entradas negadas.

Formas canónicas

Un **minitérmino** es un AND que contiene todas las variables (cada una negada o no) y vale 1 en **una sola** fila de la tabla. Un **maxitérmino** es un OR con todas las variables que vale 0 en una sola fila.

- **Suma de minitérminos** (Σm): suma (OR) de los minitérminos de las filas donde $f = 1$.
- **Producto de maxitérminos** (ΠM): producto (AND) de los maxitérminos de las filas donde $f = 0$.

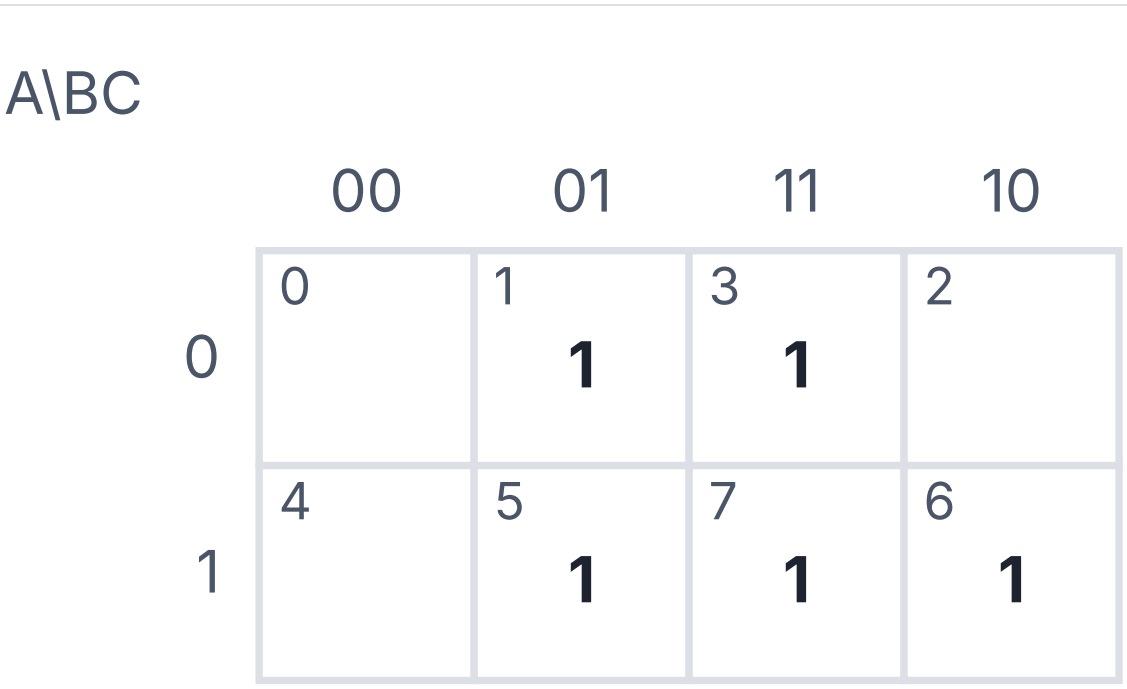
Ejemplo: $f(A, B, C) = \Sigma m(1, 3, 5, 6, 7)$ significa que $f = 1$ en las filas $001, 011, 101, 110, 111$.

Simplificar con mapas de Karnaugh

Un **mapa de Karnaugh** es la tabla de verdad colocada en una rejilla donde las casillas vecinas difieren en **un solo bit** (por eso las columnas van en orden Gray: 00, 01, 11, 10). Dos casillas vecinas con un 1 se pueden juntar y desaparece la variable que cambia.

Reglas para agrupar:

1. Grupos de 1, 2, 4, 8... casillas (potencias de 2), rectangulares.
2. Cuanto **más grandes**, mejor: eliminan más variables.
3. El mapa es **cíclico**: el borde izquierdo es vecino del derecho, y el de arriba del de abajo.
4. Cubre todos los 1 con el mínimo número de grupos. Un 1 puede estar en varios grupos.
5. Las casillas **indiferentes** (X, *don't care*) se usan solo si ayudan a hacer el grupo mayor.



$$f = \Sigma m(1,3,5,6,7)$$

Dos grupos de cuatro y de dos casillas cubren los cinco unos.

Ejemplo T3 **Simplificar $f = \Sigma m(1,3,5,6,7)$** ✔ verificado

Simplifica $f(A,B,C) = \Sigma m(1,3,5,6,7)$ como suma de productos y como producto de sumas.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: mapa de Karnaugh de 3 variables, agrupación en potencias de 2, y la dualidad: para el producto de sumas se agrupan los **ceros** y se niega el resultado.

1 Rellena el mapa

Pon un 1 en las casillas $1, 3, 5, 6, 7$. Los ceros están en $0, 2, 4$.

2 Agrupa los unos

Grupo de 4: m_1, m_3, m_5, m_7 (la columna $C = 1$): queda solo C. El 1 que falta, $m_6 = 110$, se agrupa con m_7: queda AB.

3 Escribe la suma de productos

$f = C + AB.$

4 **Producto de sumas**

Ahora agrupa los **ceros** (m_0, m_2, m_4). m_0 y m_2 (la fila $A = 0, C = 0$) dan $\overline{A}\,\overline{C}$. m_0 y m_4 ($B = 0, C = 0$) dan $\overline{B}\,\overline{C}$. Así $\overline{f} = \overline{A}\,\overline{C} + \overline{B}\,\overline{C}$.

5 **Niega con De Morgan**

$f = \overline{\overline{A}\,\overline{C} + \overline{B}\,\overline{C}} = (A + C)(B + C)$. Es la misma función: $(A + C)(B + C) = AB + C$.

✅ **Resultado final**

$f = C + AB = (A + C)(B + C).$

SIMULADOR · MINIMIZADOR DE FUNCIONES (2-4 VARIABLES)

Nº de variables · clic en una salida: 0 → 1 → × (indiferente)

4

— ejemplos —

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

```
Minterms (1): —
Indiferentes: —

Suma de productos : F = 0
Producto de sumas : F = ()

Puertas: 0 producto(s) + 1 OR  ·  0 literales
NAND-NAND: se obtiene negando dos veces la SOP (De Morgan);
inversores solo para las variables que aparezcan negadas.
```

Captura del simulador interactivo. Pruébalo en la versión web.

🔭 Más a fondo: método de Quine–McCluskey

Cuando hay más de 4 variables el mapa se vuelve incómodo. El algoritmo de **Quine–McCluskey** hace lo mismo con tablas: (1) agrupa minitérminos que difieren en un bit para obtener los **implicantes primos**; (2) elige un conjunto mínimo que cubra todos los minitérminos, empezando por los **implicantes primos esenciales** (los únicos que cubren algún minitérmino). Todas las simplificaciones de esta guía se han obtenido con una implementación propia de este método y se han comprobado contra la tabla de verdad.

🚧 Qué suele fallar

- Hacer grupos de 3, 5 o 6 casillas. Solo valen 1, 2, 4, 8, 16.
- Olvidar que el mapa es cíclico y no ver el grupo de las esquinas.
- Usar una casilla X como si fuera un 1 obligatorio: las X solo ayudan, nunca hay que cubrirlas.
- Dejar un grupo redundante (todos sus unos ya están cubiertos por otros).
- Negar mal con De Morgan: hay que cambiar la operación **y** negar las variables.

∑ Fórmulas clave

- $\overline{A+B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.
- $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$.
- $A \oplus B = A\overline{B} + \overline{A}B$.
- n variables: 2^n filas y 2^{2^n} funciones distintas (con 4 variables, 65536).

¿Enuncia De Morgan?

$\overline{A+B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.

¿Cuántas funciones lógicas distintas hay de n variables?

2^{2^n}.

¿Qué son puertas universales?

NAND y NOR: con una sola se construye cualquier función.

¿Por qué las columnas de Karnaugh van en orden 00,01,11,10?

Para que las casillas vecinas difieran en un solo bit (código Gray).

¿Cómo se usan las casillas indiferentes?

Solo si permiten hacer un grupo mayor; no hay que cubrirlas.

Minitérmino

AND con todas las variables que vale 1 en una sola fila de la tabla.

Maxitérmino

OR con todas las variables que vale 0 en una sola fila de la tabla.

Implicante primo

Grupo de unos que no se puede ampliar más en el mapa de Karnaugh.

Indiferente (X)

Combinación de entradas que no ocurre o cuya salida da igual.

Puerta universal

Puerta con la que se puede construir cualquier función lógica (NAND o NOR).

Según De Morgan, $\overline{A \cdot B + C}$ es igual a:

A $\overline{A} + \overline{B} + C$

B $(\overline{A} + \overline{B})\,\overline{C}$

C $(\overline{A} + \overline{B}) + \overline{C}$

D $\overline{A}\,\overline{B} + \overline{C}$

Se cambia + por · y se niega cada término: $\overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B})\overline{C}$.

En un mapa de Karnaugh, ¿cuántas casillas puede tener un grupo válido?

A 1, 2, 4, 8...

B Solo números pares

C Cualquier número

D Hasta un máximo de 4

Los grupos son potencias de 2.

¿Cuál es la forma simplificada de $AB + A\overline{B} + \overline{A}B$?

A $\overline{A} + \overline{B}$

B $A \oplus B$

C $A + B$

D AB

$AB + A\overline{B} = A$, y queda $A + \overline{A}B = A + B$.

TEORÍA POR TEMAS

Tema 4 · Familias lógicas

Qué hay dentro de una puerta real: niveles de tensión, márgenes de ruido, fan-out, consumo y retardo.

💡 **Desde cero**

Hasta ahora una puerta era un símbolo perfecto. En la realidad es un circuito con transistores, con **tensiones** que no son exactamente 0 y 5 V, que **consume** energía y que **tarda** un poco en responder. Una **familia lógica** es un conjunto de puertas fabricadas con la misma tecnología (TTL, CMOS…) y diseñadas para conectarse entre sí. Este tema responde: ¿se pueden conectar estas dos puertas? ¿Cuántas? ¿Qué velocidad y consumo tendrá el conjunto?

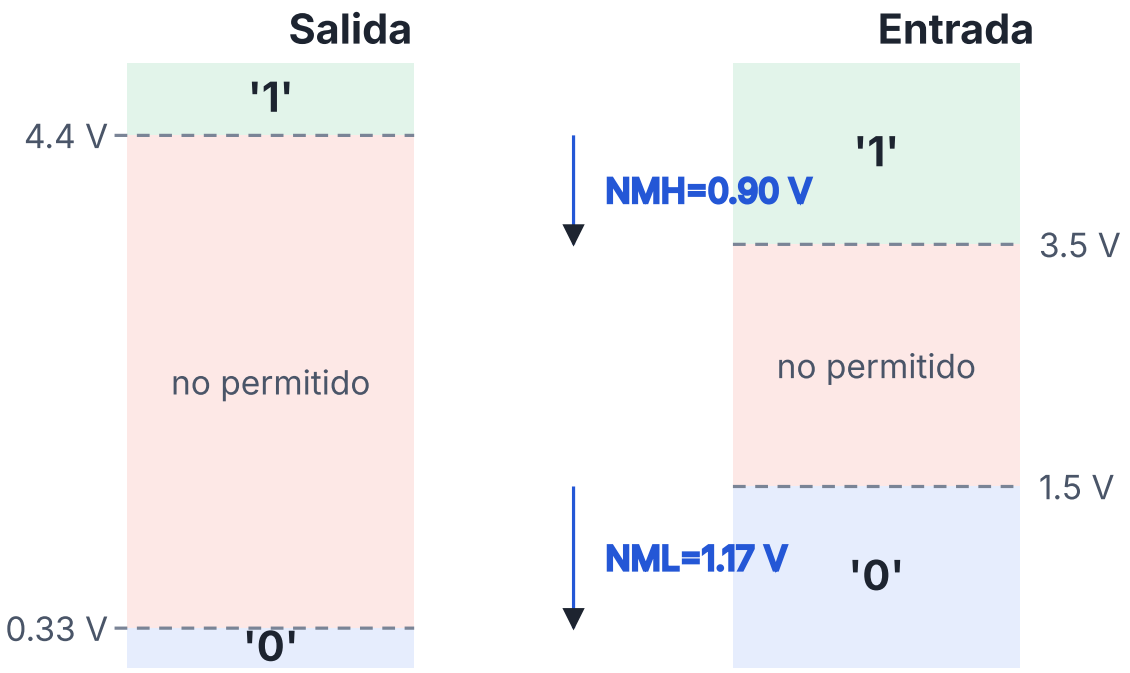
Niveles lógicos y margen de ruido

Un 0 o un 1 es un **rango** de tensión. Cada familia da cuatro números (valores típicos de un 74LS de TTL y de un 74HC de CMOS a 5 V):

Parámetro	Significa	TTL (74LS)	CMOS (74HC)
V_{OH}	salida mínima en nivel alto	2,7 V	4,4 V
V_{OL}	salida máxima en nivel bajo	0,5 V	0,33 V
V_{IH}	entrada mínima que se reconoce como alta	2,0 V	3,5 V
V_{IL}	entrada máxima que se reconoce como baja	0,8 V	1,5 V

La salida de una puerta alimenta la entrada de la siguiente. Para que funcione, lo que **garantiza** la salida debe cumplir lo que **exige** la entrada. La diferencia es el **margen de ruido**: cuánto ruido puede sumarse al cable sin que se estropee el dato.

$$NM_H = V_{OH} - V_{IH} \qquad NM_L = V_{IL} - V_{OL}$$



74HC a 5 V

Márgenes de ruido de CMOS: 0,9 V en alto y 1,17 V en bajo.

Ejemplo T4 **Márgenes de ruido de TTL y CMOS** ✔ verificado

Calcula los márgenes de ruido de la familia TTL (74LS) y de la CMOS (74HC) de la tabla anterior. ¿Cuál tolera más ruido?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: $NM_H = V_{OH} - V_{IH}$ y $NM_L = V_{IL} - V_{OL}$. El margen **efectivo** de una conexión es el menor de los dos.

1 TTL

$NM_H = 2{,}7 - 2{,}0 = 0{,}7$ V y $NM_L = 0{,}8 - 0{,}5 = 0{,}3$ V.

2 CMOS

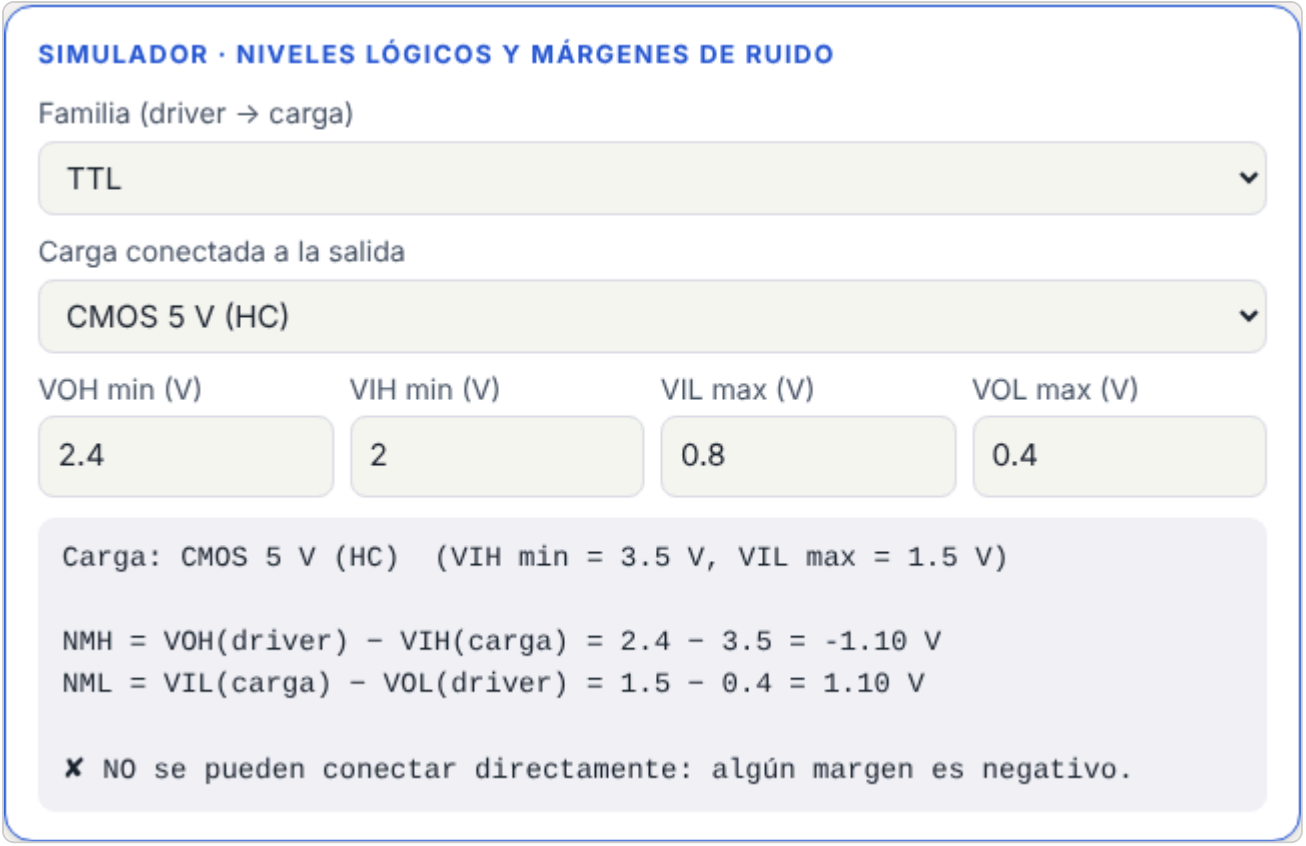
$NM_H = 4{,}4 - 3{,}5 = 0{,}9$ V y $NM_L = 1{,}5 - 0{,}33 = 1{,}17$ V.

3 Compara

El peor caso de TTL es $0{,}3$ V; el de CMOS, $0{,}9$ V. CMOS aguanta el triple de ruido en el peor caso.

✅ **Resultado final**

TTL: $0{,}7$ V y $0{,}3$ V. CMOS: $0{,}9$ V y $1{,}17$ V. Gana CMOS.



Captura del simulador interactivo. Pruébalo en la versión web.

Fan-out: cuántas puertas puede mover una salida

Cada entrada **absorbe** o **suministra** algo de corriente. Una salida solo puede dar una corriente máxima. El **fan-out** es el número de entradas que una salida puede alimentar sin salirse de los niveles garantizados. Se calcula para cada nivel y vale el **menor**:

$$\text{fan-out} = \min\left(\frac{I_{OL}}{I_{IL}},\ \frac{I_{OH}}{I_{IH}}\right)$$

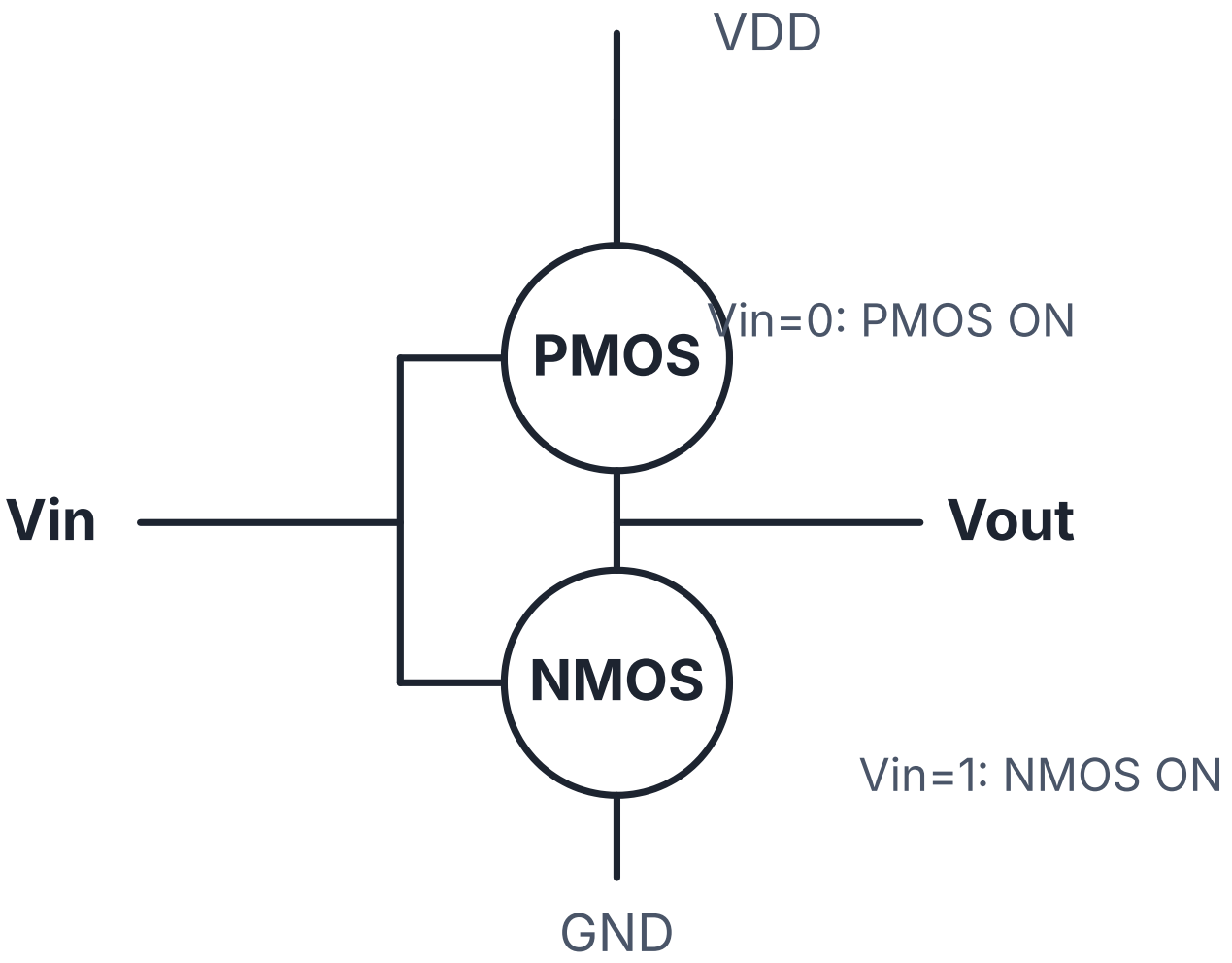
Para un 74LS: $I_{OL} = 8$ mA e $I_{IL} = 0{,}4$ mA dan 20; $I_{OH} = 0{,}4$ mA e $I_{IH} = 20$ μA dan 20. El fan-out es **20**. En CMOS las entradas casi no toman corriente, así que el límite real lo pone la capacidad (retardo), no la corriente.

Retardo y potencia

- **Retardo de propagación t_{pd}**: lo que tarda la salida en reaccionar cuando cambia la entrada. En una cadena de puertas, **los retardos se suman**: 5 puertas de 10 ns dan 50 ns.
- **Potencia en CMOS:** casi no consume en reposo, pero **cada conmutación** carga y descarga una capacidad:

$$P_{din} = C\, V_{DD}^2\, f$$

Con $C = 20$ pF, $V_{DD} = 5$ V y $f = 1$ MHz: $P = 20 \cdot 10^{-12} \cdot 25 \cdot 10^{6} = 0{,}5$ mW. Doblar la frecuencia dobla la potencia; doblar la tensión la multiplica por cuatro.



Inversor CMOS: con la entrada a 0 conduce el PMOS y la salida va a VDD; con la entrada a 1 conduce el NMOS y va a tierra. Nunca conducen los dos a la vez, por eso casi no consume en reposo.

Salidas especiales

- **Colector abierto / drenador abierto:** la salida solo sabe llevar a 0; para el 1 hace falta una resistencia externa (*pull-up*). Permite conectar varias salidas al mismo cable (AND cableada).
- **Tres estados (*tri-state*):** la salida puede estar en 0, 1 o **alta impedancia** (Z, «desconectada»). Permite que muchos dispositivos compartan un bus: solo uno habilita su salida a la vez.
- **Entradas sin conectar:** en CMOS una entrada al aire **flota** y puede oscilar y consumir. Conéctala siempre a 0 o a 1.

🚧 Qué suele fallar

- Comparar V_{OH} con V_{OL} en vez de V_{OH} con V_{IH} para el margen.
- Dar como fan-out el cociente de un solo nivel en lugar del **mínimo** de los dos.
- Sumar retardos en paralelo: solo se suman los de la **ruta más larga**.
- Creer que CMOS no consume: no consume en reposo, pero sí al conmutar, y crece con la frecuencia.
- Conectar dos salidas totem-pole directamente: si una da 1 y otra 0, hay cortocircuito. Para eso existen colector abierto y tres estados.

∑ Fórmulas clave

- $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
- fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$.
- $P_{din} = C\,V_{DD}^2\,f$.
- Retardo de una ruta: suma de los t_{pd} de sus puertas.

¿Margen de ruido en nivel alto?

$NM_H = V_{OH} - V_{IH}$.

¿Cómo se calcula el fan-out?

El menor de I_{OL}/I_{IL} y I_{OH}/I_{IH}.

¿Para qué sirve el colector abierto?

Permite unir varias salidas en un cable (AND cableada) usando una resistencia pull-up.

¿Qué es el estado Z?

Alta impedancia: la salida queda desconectada. Permite compartir un bus.

¿De qué depende la potencia dinámica de CMOS?

De la capacidad, de la tensión al cuadrado y de la frecuencia.

Margen de ruido

Cuánto ruido tolera una conexión sin que se interprete mal el nivel lógico.

Fan-out

Número máximo de entradas que una salida puede alimentar.

Retardo de propagación

Tiempo que tarda la salida en reaccionar a un cambio de la entrada.

Colector abierto

Salida que solo fuerza el 0; el 1 lo pone una resistencia externa.

Alta impedancia (Z)

Estado de una salida tri-state en el que está eléctricamente desconectada.

Una puerta tiene $V_{OH} = 2{,}7$ V y la siguiente exige $V_{IH} = 2{,}0$ V. El margen de ruido en alto es:

- **A** 0,7 V
- **B** 4,7 V
- **C** 0,3 V
- **D** 2,0 V

$NM_H = V_{OH} - V_{IH} = 0{,}7$ V.

Una salida da $I_{OL} = 16$ mA y $I_{OH} = 0{,}8$ mA; cada entrada pide $I_{IL} = 0{,}8$ mA e $I_{IH} = 40$ μA. El fan-out es:

- **A** 40
- **B** 1
- **C** 16
- **D** 20

$16/0{,}8 = 20$ y $0{,}8\ \text{mA}/40\ \mu\text{A} = 20$. El mínimo es 20.

Una puerta CMOS conmuta a 2 MHz con $C = 10$ pF y $V_{DD} = 3$ V. Su potencia dinámica es:

- **A** 0,18 mW
- **B** 1,8 mW
- **C** 0,9 mW

D 0,06 mW

$P = 10 \cdot 10^{-12} \cdot 9 \cdot 2 \cdot 10^{6} = 1{,}8 \cdot 10^{-4}$ W $= 0{,}18$ mW.

TEORÍA POR TEMAS

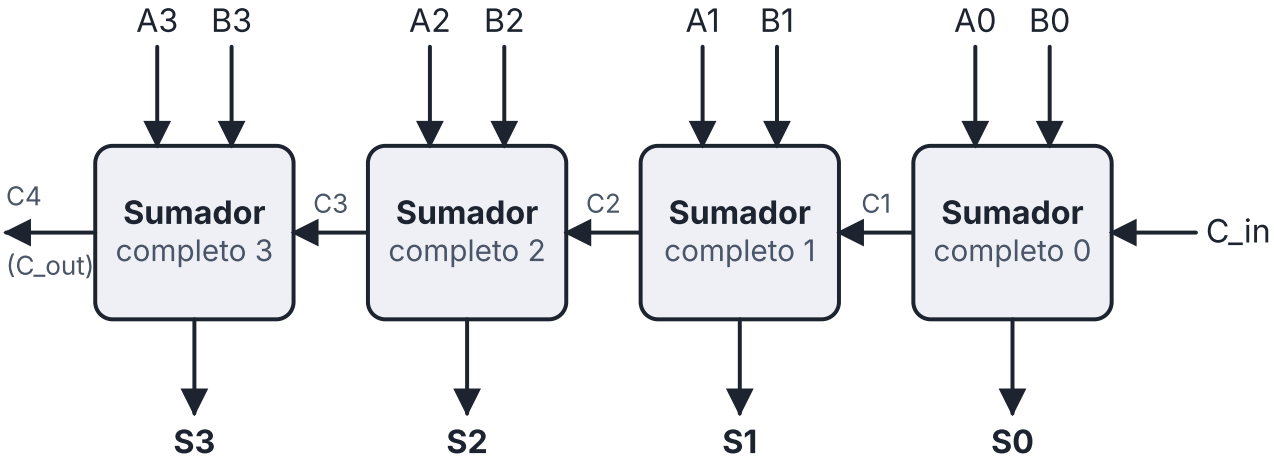
Tema 5 · Funciones de la lógica combinacional

Sumadores, comparadores, multiplexores, decodificadores, codificadores y displays de 7 segmentos.

> 💡 **Desde cero**
>
> Los mismos circuitos se necesitan una y otra vez: sumar, comparar, elegir una entrada entre varias, convertir un número en una línea activa. Los fabricantes los venden ya hechos (bloques **MSI**). Diseñar «a nivel de subsistema» es **usar estos bloques como piezas** en lugar de dibujar puertas. Saber qué hace cada pieza y cómo conectarla es la mitad del examen de combinacional.

Sumadores

- **Semisumador** (suma 2 bits): $S = A \oplus B$, $C_{out} = A \cdot B$.
- **Sumador completo** (suma 2 bits y el acarreo de entrada C_{in}): $S = A \oplus B \oplus C_{in}$ y $C_{out} = AB + C_{in}(A \oplus B)$.



Sumador de 4 bits con acarreo en cascada: el acarreo de salida de cada bit entra en el siguiente.

Se ha comprobado por código con las 8 combinaciones que da la suma binaria correcta.

Sumador de n bits con acarreo en cascada (*ripple carry*): encadenas n sumadores completos, el C_{out} de uno es el C_{in} del siguiente. Es barato, pero el acarreo tiene que **viajar** por todos: con 4 etapas y 2 puertas de 10 ns por etapa, el retardo es $4 \cdot 2 \cdot 10 = 80$ ns. Los **sumadores con acarreo anticipado** (*carry look-ahead*) calculan los acarreos en paralelo y son más rápidos.

Restar con un sumador: $A - B = A + \overline{B} + 1$ (invertir B y poner $C_{in} = 1$). Con XOR en la entrada B el mismo circuito suma o resta según una señal de control.

Ejemplo T5 **Sumador de 4 bits** ✔ verificado

Dibuja cómo se construye un sumador de 4 bits con sumadores completos y calcula su retardo en el peor caso si cada puerta tarda 10 ns y cada sumador completo tiene dos niveles de puertas en el camino del acarreo.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: sumador completo, cascada de acarreo, retardo = suma de retardos de la ruta más larga.

1 Cascada

Los bits $A_3A_2A_1A_0$ y $B_3B_2B_1B_0$ entran por pares en cuatro sumadores completos. C_{in} del primero es 0 y cada C_{out} va al C_{in} siguiente.

2 Ruta crítica

El peor caso es cuando el acarreo nace en el bit 0 y viaja hasta el bit 3: pasa por 4 sumadores.

3 Retardo

$4\ \text{etapas} \times 2\ \text{puertas} \times 10\ \text{ns} = 80\ \text{ns}$.

✅ **Resultado final**

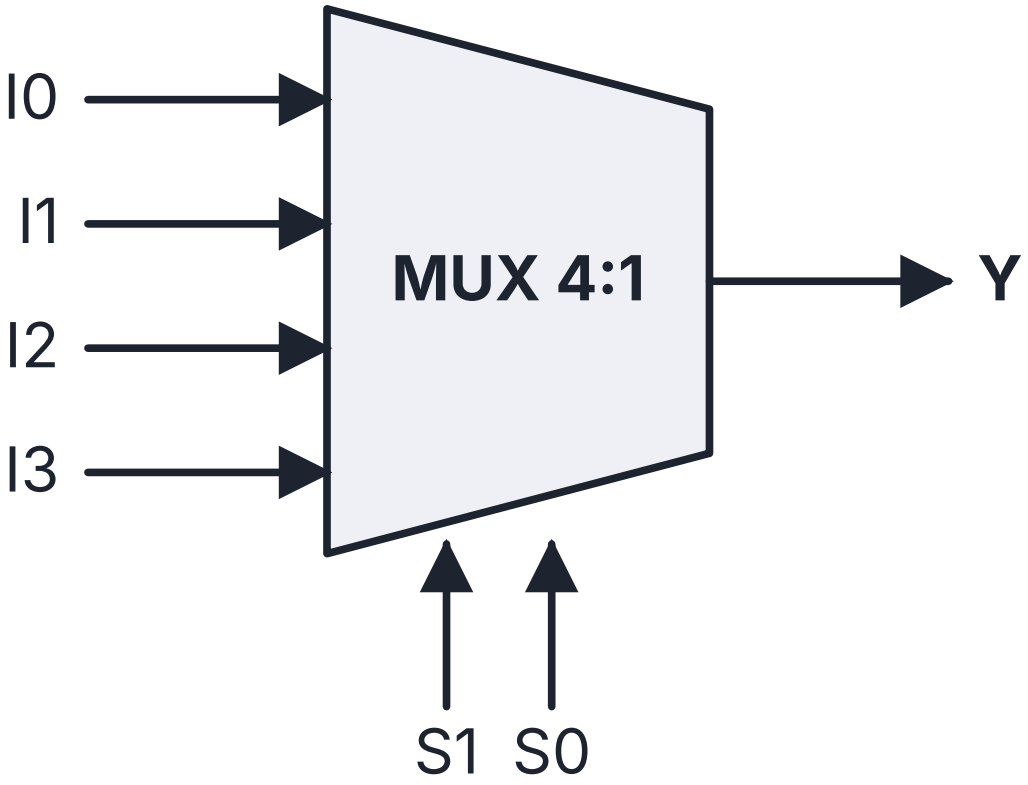
Cuatro sumadores completos en cascada. Retardo del acarreo: **80 ns**.

Comparadores

Un comparador de 1 bit da tres salidas: $A > B = A\overline{B}$, $A = B = \overline{A \oplus B}$ y $A < B = \overline{A}B$. Para n bits se compara **de más a menos significativo**: manda el primer bit que difiere. Los circuitos integrados tienen entradas de cascada para unir varios.

Multiplexores

Un **multiplexor** (MUX) de 2^k a 1 tiene 2^k entradas de datos, k entradas de **selección** y una salida: copia a la salida la entrada que indica el número de selección. Es un «selector de vía».



Multiplexor 4:1: las entradas S1S0 eligen cuál de las cuatro entradas llega a Y.

$$Y = \sum_{i=0}^{2^k-1} m_i(S) \cdot D_i$$

Truco de examen: implementar una función con un MUX. Una función de n variables cabe en un MUX de 2^{n-1} a 1. Las $n-1$ primeras variables van a la selección y la última variable (o su negada, 0 o 1) a cada entrada de datos.

Ejemplo T5 **Implementar una función con un MUX 4:1** ✔ verificado

Implementa $f(A,B,C) = \Sigma m(1,2,4,7)$ con un multiplexor 4:1. Usa A y B como selección.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: para cada valor de (A, B), mira cómo depende f de C en las dos filas correspondientes: da 0, 1, C o $\overline{C}$. Eso es lo que va a la entrada D_i de ese valor.

1 **Mira las parejas de filas**

$AB = 00$: filas 0 y 1. $f(0) = 0$, $f(1) = 1 \Rightarrow D_0 = C$. $AB = 01$: filas 2 y 3. $f = 1, 0 \Rightarrow D_1 = \overline{C}$. $AB = 10$: filas 4 y 5. $f = 1, 0 \Rightarrow D_2 = \overline{C}$. $AB = 11$: filas 6 y 7. $f = 0, 1 \Rightarrow D_3 = C$.

2 **Conecta**

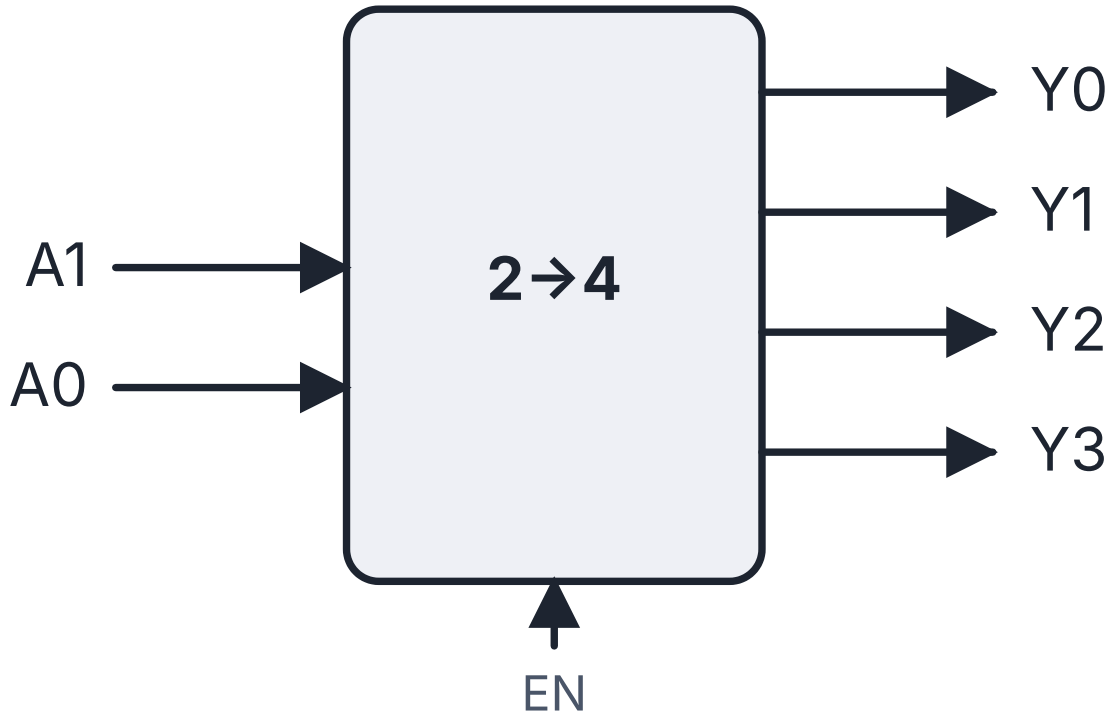
Selección: $S_1 = A$, $S_0 = B$. Entradas: $D_0 = C$, $D_1 = \overline{C}$, $D_2 = \overline{C}$, $D_3 = C$. Solo hace falta un inversor.

✅ **Resultado final**

$Y = \mathrm{MUX}(A, B;\ C, \overline{C}, \overline{C}, C)$. Comprobado con las 8 combinaciones.

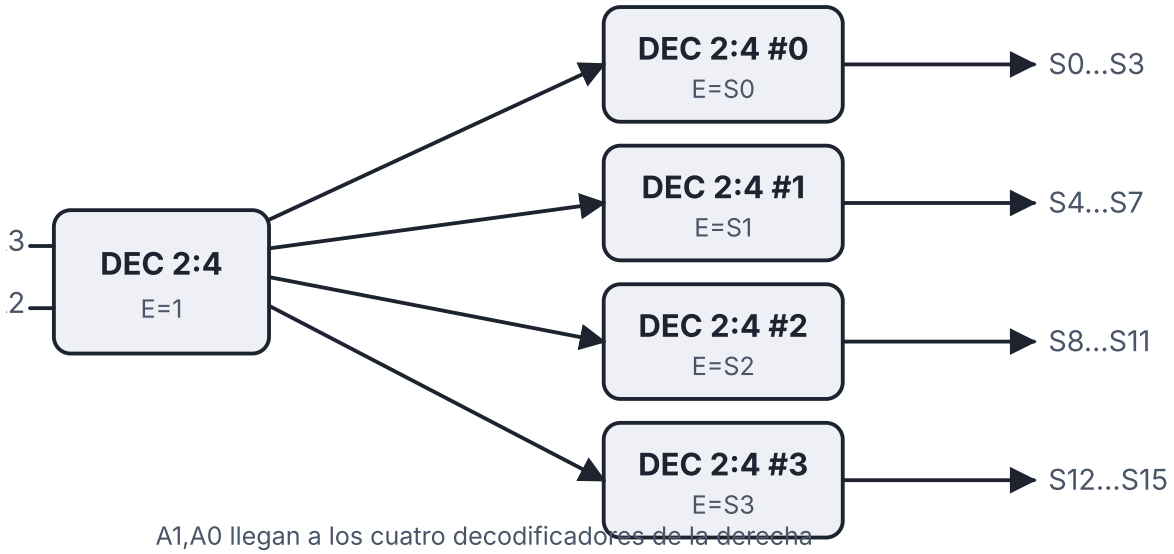
Decodificadores y codificadores

- **Decodificador** k a 2^k: activa **una** salida según el número de entrada. Cada salida es un **minitérmino** de las entradas. Por eso con un decodificador y una puerta OR puedes implementar cualquier función: juntas las salidas de los minitérminos que valen 1. Tiene entrada de habilitación (*enable*).
- **Codificador:** hace lo inverso: de 2^k líneas, activa el código de la línea activa. El **codificador de prioridad** decide cuando hay varias líneas a 1: gana la de mayor índice. Para las entradas 0110 da 2 y para 1010 da 3.



Decodificador 2:4 con habilitación EN: se activa una sola salida.

Ampliar: con decodificadores 2:4 puedes montar uno de 4:16 en árbol: uno de 2:4 habilita a cuatro más.



Un decodificador 4:16 hecho con cinco 2:4. Los dos bits altos eligen cuál de los cuatro de la derecha se habilita; los dos bajos eligen la salida dentro de él.

Captura del simulador interactivo. Pruébalo en la versión web.

Display de 7 segmentos

Un display tiene 7 LED (a a g). Un **decodificador BCD a 7 segmentos** recibe 4 bits (cifra de 0 a 9) y da 7 salidas. Las combinaciones 1010 a 1111 no existen en BCD: son **indiferentes**, y se aprovechan para simplificar. En las placas Basys los segmentos son **activos a nivel bajo**: se encienden con 0.

🚧 Qué suele fallar

- Montar el sumador sin poner $C_{in} = 0$ en el primer bit.
- Confundir qué es selección y qué es dato en un MUX.
- Olvidar la habilitación (*enable*) de un decodificador o mezclar su polaridad (activa a nivel bajo).
- En el codificador de prioridad, ignorar que **la entrada de mayor índice gana**.
- En el display, olvidar que en la Basys se enciende con 0.

∑ Fórmulas clave

- $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$.
- $A - B = A + \overline{B} + 1$.
- MUX $2^k : 1$ implementa cualquier función de $k + 1$ variables.
- Salida i de un decodificador $= m_i$ (minitérmino).

¿Ecuación del acarreo de un sumador completo?

$C_{out} = AB + C_{in}(A \oplus B)$.

¿Cómo se resta con un sumador?

$A + \overline{B} + 1$: se invierte B y se pone $C_{in} = 1$.

¿Qué número de selección necesita un MUX de 8 a 1?

3 bits ($2^3 = 8$).

¿Qué es cada salida de un decodificador?

Un minitérmino de las entradas.

¿Qué hace un codificador de prioridad?

Entrega el código de la entrada activa de mayor índice cuando hay varias.

MSI

Integración de media escala: bloques como sumadores, multiplexores o decodificadores.

Multiplexor

Selector que copia a la salida una de varias entradas según un código de selección.

Decodificador

Activa una única salida según el código binario de entrada.

Codificador de prioridad

Da el código de la línea activa de mayor prioridad.

Acarreo anticipado

Técnica que calcula los acarreos en paralelo para acelerar un sumador.

Para implementar una función de 4 variables con un solo multiplexor, ¿cuál es el mínimo?

A Imposible con un solo MUX

B MUX 4:1

C MUX 16:1 y nada más

D MUX 8:1

Con $2^{n-1} = 8$ entradas de datos y 3 variables en selección; la cuarta va a los datos.

Un decodificador 3:8 tiene la entrada 101. ¿Qué salida se activa?

A S_6

B S_1

C S_5

D S_3

$101_2 = 5$. Se activa la salida S_5.

Un sumador de 8 bits con acarreo en cascada tiene un retardo de 15 ns por etapa. El retardo del acarreo es:

A 120 ns

B 23 ns

C 15 ns

D 8 ns

$8 \cdot 15 = 120$ ns: el acarreo atraviesa las 8 etapas.

TEORÍA POR TEMAS

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

La memoria de 1 bit: latch, flip-flop SR, JK, D y T, tablas características y de excitación.

> 💡 **Desde cero**
>
> Un **biestable** es un circuito con dos estados estables que **recuerda un bit**. Un **flip-flop** cambia de estado solo cuando llega un **flanco** del reloj (el instante en que la señal de reloj pasa de 0 a 1, o de 1 a 0). Entre flanco y flanco no escucha a las entradas. Esa disciplina es lo que permite que un circuito con miles de biestables funcione ordenado: todos cambian a la vez, en el mismo instante.

Latch frente a flip-flop

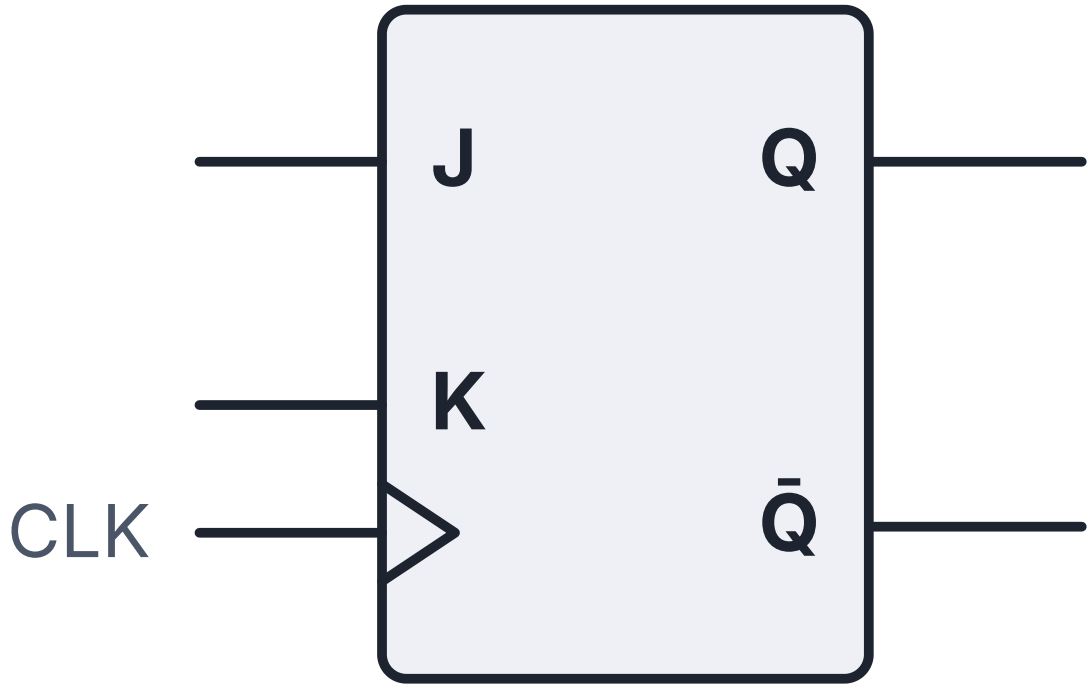
- **Latch** (cerrojo): es **transparente** mientras el reloj (o *enable*) está activo: la salida sigue a la entrada todo ese tiempo.
- **Flip-flop disparado por flanco:** muestrea las entradas **solo en el flanco** y mantiene la salida el resto del ciclo.

Usa flip-flops para diseñar secuenciales síncronos. Los latches dan problemas de temporización.

Los cuatro tipos y sus tablas

Q es el estado actual y Q^+ el estado después del flanco.

Tipo	Ecuación característica	Qué hace
SR	$Q^+ = S + \overline{R}\,Q$ (con $SR = 0$)	$S = 1$ pone a 1; $R = 1$ pone a 0; $S = R = 0$ mantiene; $S = R = 1$ **prohibido**
JK	$Q^+ = J\overline{Q} + \overline{K}\,Q$	Como SR, pero $J = K = 1$ **conmuta**
D	$Q^+ = D$	Copia D en cada flanco
T	$Q^+ = T \oplus Q$	$T = 1$ conmuta; $T = 0$ mantiene



Símbolo del flip-flop JK. El triángulo en la entrada de reloj indica que se dispara por flanco.

Se ha comprobado por código que la tabla del JK, la del D y la del T cumplen estas ecuaciones para todas las combinaciones.

Tabla de excitación: lo que necesitas para diseñar

La tabla característica dice «con estas entradas, ¿qué pasa?». Al **diseñar** preguntas lo contrario: «quiero pasar de Q a Q^+, ¿qué entradas pongo?». Es la **tabla de excitación**:

$Q \to Q^+$	J	K	D	T
$0 \to 0$	0	X	0	0
$0 \to 1$	1	X	1	1
$1 \to 0$	X	1	0	1
$1 \to 1$	X	0	1	0

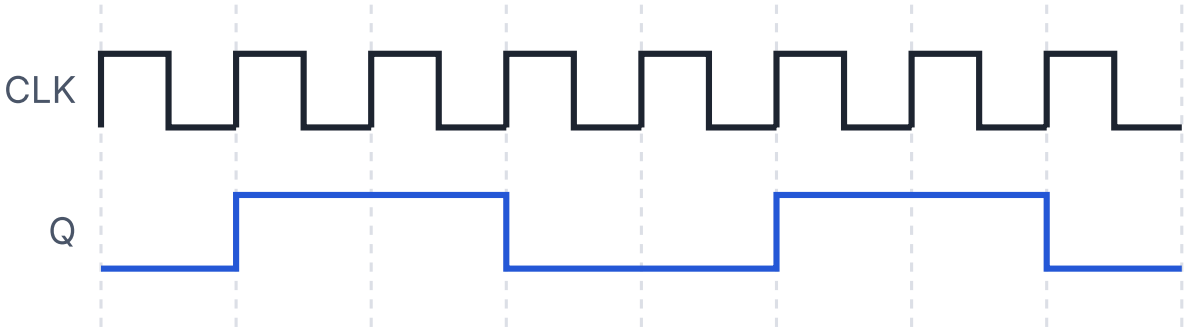
El JK tiene muchos **indiferentes (X)**: eso suele dar ecuaciones más simples que con D, a costa de dos entradas por biestable.

> 💡 **Por qué hay una X en el JK**
>
> Para pasar de 0 a 1 puedes poner $J = 1$ y $K = 0$ (poner a 1) **o** $J = 1$ y $K = 1$ (conmutar). El valor de K da igual. Pasa lo mismo en los otros tres casos.

Captura del simulador interactivo. Pruébalo en la versión web.

Biestables como divisores de frecuencia

Un JK con $J = K = 1$ (o un T con $T = 1$, o un D con $D = \overline{Q}$) **conmuta en cada flanco**: la salida cambia cada ciclo, así que su frecuencia es la **mitad** de la del reloj. Encadenar n así da una división por 2^n. Es la base de los contadores.



Salida de un biestable T con T=1: periodo doble

Q cambia solo en los flancos de subida del reloj y completa un ciclo cada dos del reloj.

Temporización

- **Tiempo de preparación** (*setup*, t_{su}): la entrada debe estar estable **antes** del flanco.
- **Tiempo de mantenimiento** (*hold*, t_h): debe seguir estable **después** del flanco.
- **Retardo de salida** (t_{cq}): tarda un poco en reflejarse la salida tras el flanco.

Si no se respetan, el biestable cae en un estado **metaestable** (ni 0 ni 1, durante un tiempo impredecible). Por eso las entradas asíncronas (un pulsador) se sincronizan con dos biestables en serie.

Frecuencia máxima: el periodo debe cubrir el viaje completo entre dos flancos:

$$T_{clk} \geq t_{cq} + t_{pd,\,comb} + t_{su}$$

🚧 Qué suele fallar

- Usar la tabla característica para diseñar (hay que usar la de excitación).
- Escribir 0 en una X: las X son libertad, no valores.
- Confundir latch (transparente) con flip-flop (flanco).
- Olvidar que $S = R = 1$ está prohibido en un SR.
- No distinguir **flanco de subida** de **bajada** al dibujar un cronograma.

∑ Fórmulas clave

- $Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$.
- $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
- Divisor por 2: $f_{out} = f_{clk}/2$.

¿Ecuación característica del JK?

$Q^+ = J\overline{Q} + \overline{K}Q.$

¿Qué hace un JK con J=K=1?

Conmuta en cada flanco.

¿Entradas del JK para pasar de 1 a 0?

$J = X$ (cualquiera) y $K = 1$.

¿Qué es el tiempo de setup?

Tiempo que la entrada debe estar estable antes del flanco de reloj.

¿Qué diferencia hay entre latch y flip-flop?

El latch es transparente mientras está habilitado; el flip-flop muestrea solo en el flanco.

Flanco

Instante en que la señal de reloj pasa de 0 a 1 (subida) o de 1 a 0 (bajada).

Biestable

Circuito de dos estados estables que almacena un bit.

Tabla de excitación

Dice qué entradas del biestable hacen falta para ir de Q a Q^+.

Metaestabilidad

Estado intermedio e impredecible tras violar los tiempos de setup o hold.

Setup / hold

Tiempos de preparación y mantenimiento de las entradas alrededor del flanco.

Un biestable JK tiene $Q = 1$ y recibe $J = 0$, $K = 0$. Tras el flanco:

A $Q = 1$ y luego conmuta

B $Q = 1$ (mantiene)

C $Q = 0$

D Estado prohibido

$J = K = 0$ mantiene. Con $Q^+ = J\overline{Q} + \overline{K}Q = 1$.

Quieres pasar de $Q = 1$ a $Q^+ = 0$ con un biestable T. ¿Qué pones en T?

A $T = X$

B $T = 1$

C $T = 0$

D No es posible

Para cambiar de estado, $T = 1$.

Un biestable T con $T = 1$ recibe un reloj de 8 MHz. La salida tiene frecuencia:

A 2 MHz

B 8 MHz

C 4 MHz

D 16 MHz

Conmuta en cada flanco: la frecuencia se divide por 2.

TEORÍA POR TEMAS

Tema 7 · Memorias y circuitos lógicos programables

ROM, RAM, cómo se calcula su tamaño y cómo una memoria o una PLA implementan funciones.

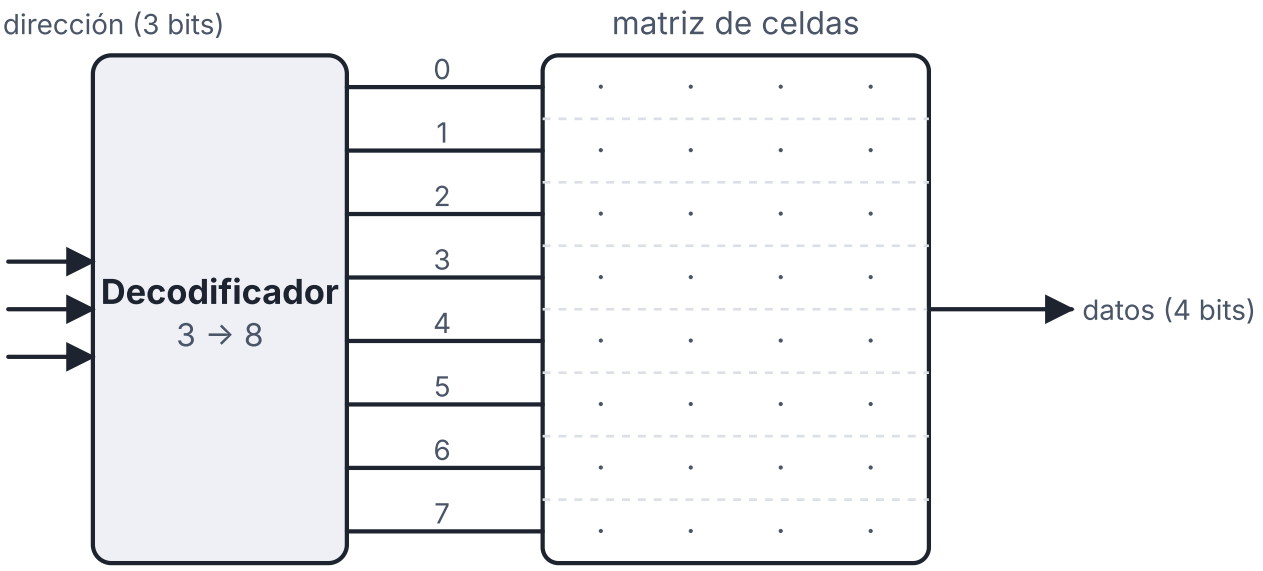
> 💡 **Desde cero**
>
> Una **memoria** es una tabla: das una **dirección** y te devuelve el **dato** guardado ahí. Con k líneas de dirección hay 2^k posiciones, y cada una guarda m bits. Una memoria $2^k \times m$ guarda $m \cdot 2^k$ bits. Ese mismo hecho (una tabla indexada por la entrada) significa que **una ROM es una tabla de verdad con otra forma**: cualquier función combinacional cabe en una ROM.

Tipos de memoria

Tipo	Se puede escribir	Se pierde al apagar	Uso típico
ROM	No (se programa en fábrica)	No	Código fijo
PROM / EPROM / EEPROM / Flash	Sí, pocas veces y despacio	No	Firmware, pendrive
SRAM	Sí, rápido	Sí	Cachés, registros
DRAM	Sí, necesita refresco	Sí	Memoria principal

Tamaño y organización

$$\text{bits totales} = 2^k \cdot m \qquad k = \log_2(\text{posiciones})$$



Memoria de 8 palabras de 4 bits: el decodificador activa una fila y la matriz entrega los 4 bits de esa palabra.

- Una memoria de $8\text{K}\times 8$ necesita $k = 13$ líneas de dirección ($2^{13} = 8192$).
- Una de $64\text{K}\times 16$ guarda $64 \cdot 1024 \cdot 16 = 1\,048\,576$ bits.

Ampliar la memoria:

- **Más ancho** (más bits por dato): pon chips en paralelo con las mismas direcciones.
- **Más largo** (más posiciones): pon chips con las líneas altas de dirección pasando por un **decodificador** que habilita un solo chip cada vez.

Ejemplo T7 Ampliar una memoria

✔ verificado

Con chips de $2\text{K}\times 8$ construye una memoria de $8\text{K}\times 16$. ¿Cuántos chips necesitas y cuántas líneas de dirección tiene el conjunto?

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: ampliar a lo ancho (bits) y a lo largo (posiciones), y el número de líneas de dirección $k = \log_2 N$.

1 Direcciones

$8\text{K} = 2^{13}$ posiciones: **13** líneas de dirección. Cada chip usa 11 ($2^{11} = 2\text{K}$).

2 Ancho

Cada chip da 8 bits y necesito 16: hacen falta **2 chips** en paralelo.

3 Largo

Cada chip tiene 2K posiciones y necesito 8K: hacen falta $8/2 = 4$ filas de chips. Las 2 líneas altas (A_{12}, A_{11}) entran en un decodificador 2:4 que habilita una fila.

4 Total

$4 \times 2 = 8$ chips.

✅ Resultado final

8 chips y **13** líneas de dirección.

3

Dato de entrada (0 a 15, 4 bits)

10

Escribir (WE = 1) Leer (WE = 0)

Dirección	Contenido	Hex
000 (0)	0000	0
001 (1)	0000	0
010 (2)	0000	0
011 (3)	0000	0
100 (4)	0000	0
101 (5)	0000	0
110 (6)	0000	0
111 (7)	0000	0

CÁLCULO DE CHIPS

Palabras de la memoria que se quiere

16384

Bits por palabra que se quiere

16

Palabras de cada chip

4096

Bits por palabra de cada chip

8

```
Direcciones de la memoria: 14 bits  (cada chip usa 12)
Bloques en profundidad: 4 → decodificador de 2 → 4
Chips en paralelo por bloque: 2
Total: 4 × 2 = 8 chips
Capacidad: 262144 bits
```

Captura del simulador interactivo. Pruébalo en la versión web.

La memoria como circuito combinacional

Una ROM de 2^k palabras de m bits **es** un circuito de k entradas y m salidas: cada dirección es una fila de la tabla de verdad. Ejemplo: un **sumador completo** (C_{in}, A, B como dirección) cabe en una ROM de 8×2 cuyo contenido es $0, 1, 1, 2, 1, 2, 2, 3$ (el número de unos de la dirección, en binario).

Lógica programable: PLA, PAL

- **PLA** (*programmable logic array*): un plano **AND programable** genera términos producto y un plano **OR programable** los suma.
- **PAL:** plano AND programable y plano OR **fijo**. Más barata y rápida.
- **Ventaja sobre la ROM:** una ROM genera **todos** los minitérminos (2^n), la PLA solo los **términos producto simplificados** que necesitas. Por eso, si simplificas antes (Karnaugh), cabe mucho más en una PLA.

🔭 **Más a fondo: PLD, CPLD y FPGA**

Las **PLD** simples (PAL, GAL) tienen decenas de funciones. Las **CPLD** juntan varias PLD con una red de interconexión. Las **FPGA** (Tema 12) cambian el plano AND-OR por miles de pequeñas tablas (*LUT*) y registros, con interconexión programable: son las más flexibles y las que se usan en el laboratorio.

🚧 **Qué suele fallar**

- Calcular las líneas de dirección con el número de bits del dato en vez del número de posiciones.
- Olvidar que 1K = 1024, no 1000.
- Al ampliar «a lo largo», conectar todos los chips habilitados a la vez: solo debe estar activo uno.
- Pensar que la ROM guarda datos «que cambian»: solo se lee.

∑ **Fórmulas clave**

- bits totales $= 2^k \cdot m$; $k = \log_2 N$.
- Chips necesarios $= \dfrac{N_{total}}{N_{chip}} \cdot \dfrac{m_{total}}{m_{chip}}$.
- Una ROM $2^n \times m$ implementa m funciones de n variables.

¿Cuántos bits tiene una memoria de $2^k \times m$?

$m \cdot 2^k$ bits.

¿Cuántas líneas de dirección tiene una memoria de 16K posiciones?

14 ($2^{14} = 16384$).

¿Qué diferencia hay entre ROM y RAM?

La ROM solo se lee y no se borra al apagar; la RAM se lee y escribe y (volátil) se borra.

¿Qué plano es fijo en una PAL?

El plano OR.

ROM

Memoria de solo lectura: guarda una tabla fija.

RAM

Memoria de lectura y escritura, volátil.

PLA

Matriz lógica programable con plano AND y plano OR programables.

PAL

Matriz con plano AND programable y plano OR fijo.

Volátil

Que pierde su contenido al quitar la alimentación.

¿Cuántas líneas de dirección tiene una memoria de 32K×8?

- **A** 15
- **B** 32
- **C** 5
- **D** 8

$32\text{K} = 2^5 \cdot 2^{10} = 2^{15}$.

¿Cuántos chips de 4K×4 hacen falta para una memoria de 8K×8?

A 2

B 16

C 4

D 8

A lo ancho, $8/4 = 2$; a lo largo, $8\text{K}/4\text{K} = 2$. Total $2 \cdot 2 = 4$.

Una ROM de 8 posiciones y 2 bits puede implementar:

A Un sumador completo (3 entradas, 2 salidas)

B Un contador de 8 bits

C Un biestable

D Cualquier función de 4 entradas

Tiene 3 líneas de dirección (entradas) y 2 bits de dato (salidas).

TEORÍA POR TEMAS

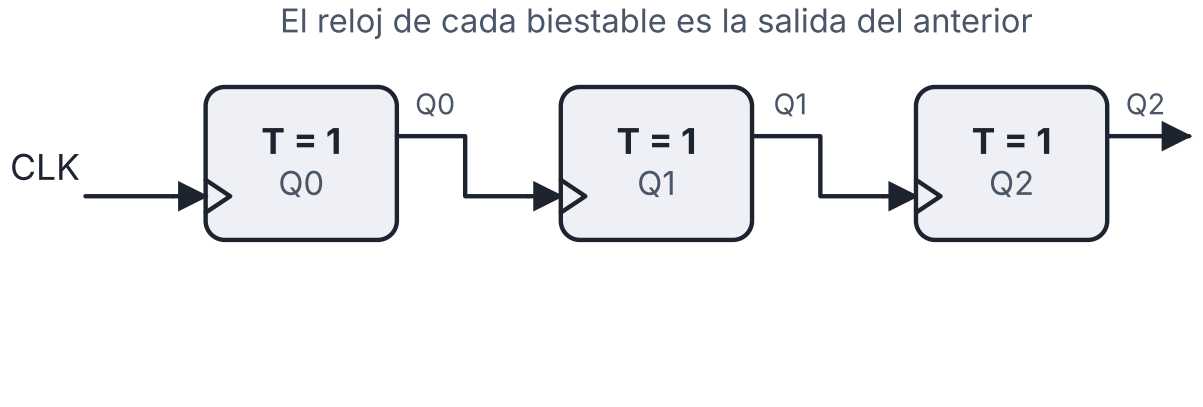
Tema 8 · Contadores

Contadores asíncronos y síncronos, módulo N, carga y puesta a cero, divisores de frecuencia.

> 💡 **Desde cero**
>
> Un **contador** es un circuito secuencial cuyo estado **avanza una cuenta** con cada pulso de reloj: 0, 1, 2, 3… y vuelve a empezar. Se usa para medir tiempo, dividir frecuencias, generar direcciones y controlar secuencias. Un contador de **módulo** N recorre N estados distintos y necesita $n = \lceil \log_2 N \rceil$ biestables.

Contador binario asíncrono (*ripple*)

Es una cadena de biestables T (o JK con $J = K = 1$) donde la salida de cada uno es el **reloj del siguiente**. Solo el primero recibe el reloj externo. Cada etapa divide la frecuencia por 2.

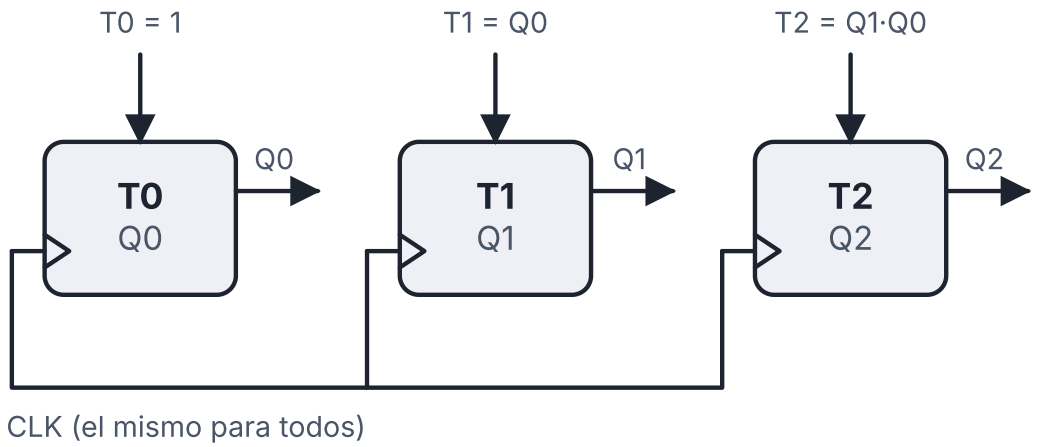


Contador asíncrono de 3 bits: cada biestable se dispara con la salida del anterior.

- Ventaja: muy simple.
- Inconveniente: los cambios **se propagan en cadena** (de ahí *ripple*). Un contador de 4 bits con $t_{pd} = 12$ ns por etapa tarda $4 \cdot 12 = 48$ ns en estabilizarse, así que el reloj máximo es unos $1/48\ \text{ns} \approx 20{,}8$ MHz, y mientras se propaga hay valores intermedios erróneos (*glitches*).

Contador síncrono

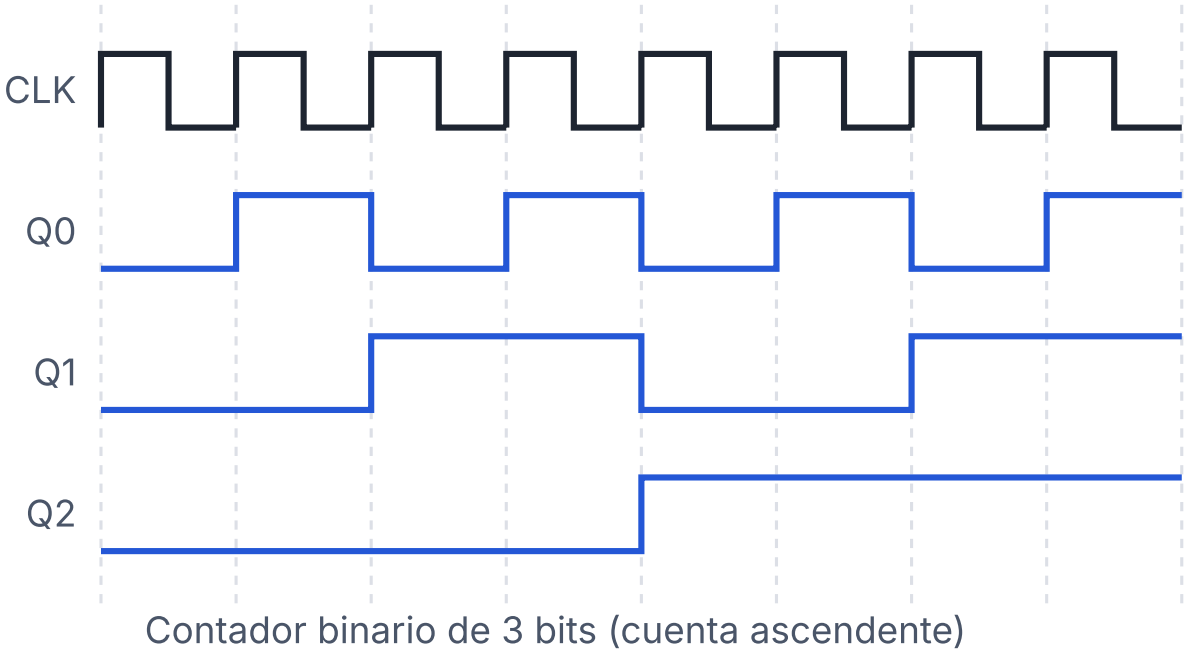
Todos los biestables comparten el mismo reloj. El biestable i debe conmutar cuando **todos los bits menores valen 1**:



Contador síncrono de 3 bits: todos comparten el reloj y cada T es el producto de las salidas de menor peso.

$$T_i = Q_{i-1} \cdot Q_{i-2} \cdots Q_0$$

Es más rápido (no hay propagación en cadena) y no tiene glitches. Se ha comprobado por código que con $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_1 Q_0$ la secuencia de 3 bits es $0, 1, 2, 3, 4, 5, 6, 7, 0, \ldots$



Q0 divide por 2, Q1 por 4 y Q2 por 8. Cada bit conmuta cuando todos los de menor peso valen 1.

SIMULADOR · DISEÑO DE CONTADORES SÍNCRONOS CON BIESTABLES JK

Secuencia de cuenta (estados en decimal, separados por comas)

0,2,4,6,1,3,5,7

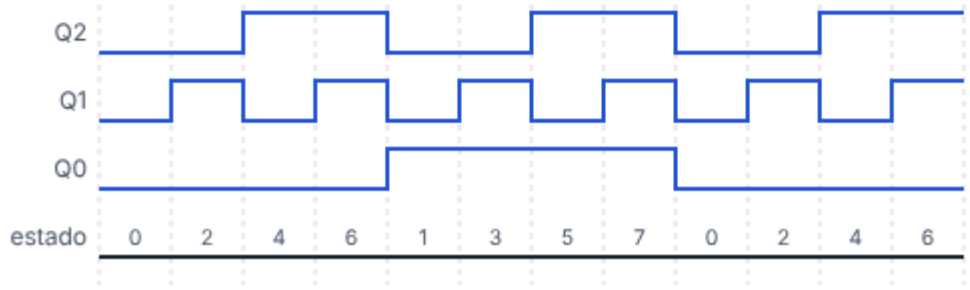
☐ Añadir entrada X (X=0 sube, X=1 baja por la secuencia inversa)

Ciclos de reloj a dibujar

12

```
Estados: 8 → 3 biestables JK (2^3 = 8 ≥ 8)
Estados no usados: 0 (indiferentes en los mapas)

J2 = Q1
K2 = Q1
J1 = 1
K1 = 1
J0 = Q2Q1
K0 = Q2Q1
```



Captura del simulador interactivo. Pruébalo en la versión web.

Contadores de módulo distinto de 2^n

Para contar N estados que no sean potencia de 2, hay tres maneras:

1. **Puesta a cero cuando se alcanza N:** detecta el estado N con una puerta y activa el *reset* (asíncrono: el estado N dura un instante y es un *glitch*; síncrono: el reset actúa en el siguiente flanco).
2. **Carga paralela:** al llegar al último estado, carga el valor inicial.
3. **Diseño directo como autómata** (Tema 10): tabla de transiciones y ecuaciones de entrada de los biestables. Es lo que se pide en examen.

Contador módulo 10 (BCD): $0 \to 1 \to \cdots \to 9 \to 0$. Módulo 12: $0 \to \cdots \to 11 \to 0$ y a 1 MHz la salida más lenta sale a $1\ \text{MHz}/12 \approx 83{,}33$ kHz.

Ejemplo T8 **Contador módulo 12 y su frecuencia de salida** ✔ verificado

Se quiere un contador síncrono de módulo 12 con reloj de 1 MHz. (a) ¿Cuántos biestables hacen falta? (b) ¿Cuál es el último estado? (c) ¿A qué frecuencia se repite el ciclo completo?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: $n = \lceil \log_2 N \rceil$ biestables; un contador de módulo N repite su ciclo cada N pulsos; $f_{out} = f_{clk}/N$.

1 Número de biestables

$2^3 = 8 < 12 \leq 2^4 = 16 \Rightarrow$ **4 biestables**.

2 Último estado

Cuenta $0, 1, \ldots, 11$. El último es $11 = 1011_2$ y el siguiente es 0000. Los estados 12 a 15 **no se usan**.

3 Frecuencia del ciclo

$$f = \frac{1\ \text{MHz}}{12} = 83{,}33\ \text{kHz}.$$

✅ **Resultado final**

(a) 4. (b) 1011_2 (11). (c) $83{,}33$ kHz.

Contadores en circuito integrado

Los contadores comerciales (74×161, 74×163…) incorporan: **carga paralela** (*LOAD*), **puesta a cero** (*CLR*), habilitación de cuenta (*ENP, ENT*), sentido ascendente/descendente (*UP/DOWN*) y una salida de **fin de cuenta** (*RCO/carry*) para **cascadear** varios. Lee siempre en la hoja si cada entrada es **síncrona o asíncrona** y si es **activa a nivel bajo**.

🔭 Más a fondo: contador reversible

Un contador ascendente/descendente con una señal X ($X = 1$ sube, $X = 0$ baja) tiene $T_i = \overline{X} \cdot \overline{Q_{i-1}} \cdots + X \cdot Q_{i-1} \cdots$. Una versión útil es la de los **contadores en código no binario**, como el de secuencia $0, 2, 4, 6, 1, 3, 5, 7$.

🚧 Qué suele fallar

- Contar mal los biestables: para módulo N son $\lceil \log_2 N \rceil$, no N.
- Olvidar que el reset **asíncrono** produce un estado intermedio fugaz.
- En un contador asíncrono, calcular la frecuencia máxima con el retardo de **una** etapa en vez de **todas**.
- Cascadear sin usar la salida de fin de cuenta o la habilitación.
- Confundir *carga síncrona* (espera al flanco) con *carga asíncrona* (inmediata).

∑ Fórmulas clave

- $n = \lceil \log_2 N \rceil$; estados no usados: $2^n - N$.
- $f_{out} = f_{clk} / N$.
- Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$.
- Asíncrono: $t_{total} = n \cdot t_{pd}$.

¿Cuántos biestables necesita un contador de módulo 60?

6 ($2^5 = 32 < 60 \leq 64$).

¿Ecuación de T_2 en un contador síncrono binario?

$T_2 = Q_1 Q_0$.

¿Ventaja del contador síncrono?

No acumula retardos de propagación y no genera glitches.

¿Qué es un glitch?

Un valor erróneo fugaz mientras los biestables de un contador asíncrono cambian en cadena.

¿Frecuencia de salida de un divisor por N?

f_{clk} / N.

Contador módulo N

Contador que recorre N estados y vuelve al inicial.

Contador síncrono

Contador en el que todos los biestables reciben el mismo reloj.

Contador asíncrono

Contador en el que cada biestable usa como reloj la salida del anterior.

Carga paralela

Entrada que fuerza al contador a un valor dado.

Fin de cuenta

Salida que avisa de que el contador llega a su último estado, para encadenar contadores.

¿Cuántos biestables necesita un contador de módulo 100?

A 100

B 10

C 6

D 7

$2^6 = 64 < 100 \leq 2^7 = 128$.

Un contador binario síncrono de 4 bits tiene $Q_3Q_2Q_1Q_0 = 0111$. Tras el siguiente flanco:

A 1111

B 0110

C 1000

D 0000

$T_3 = Q_2Q_1Q_0 = 1, T_2 = Q_1Q_0 = 1, T_1 = Q_0 = 1, T_0 = 1$: todos conmutan.

Un contador asíncrono de 5 bits con $t_{pd} = 10$ ns por etapa: ¿cuánto tarda en estabilizarse como máximo?

A 5 ns

B 32 ns

C 10 ns

D 50 ns

$5 \cdot 10 = 50$ ns.

TEORÍA POR TEMAS

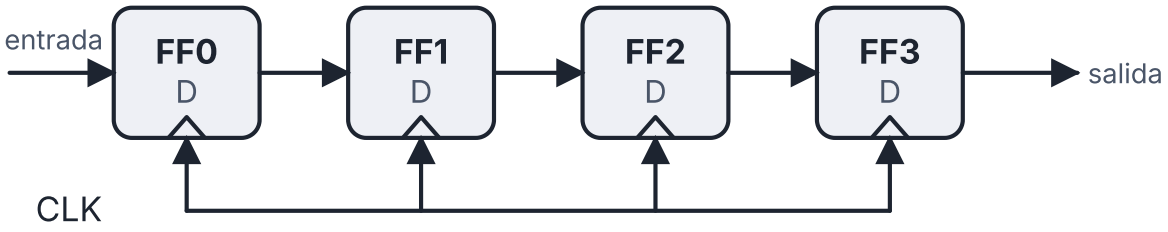
Tema 9 · Registros de desplazamiento

Registros serie/paralelo, contadores en anillo y Johnson, y generadores pseudoaleatorios (LFSR).

> 💡 **Desde cero**
>
> Un **registro** es un grupo de biestables D que guardan una palabra. Un **registro de desplazamiento** además puede **mover** cada bit un lugar a la derecha o a la izquierda en cada flanco, como una cola. Sirve para pasar datos de serie a paralelo (y al revés), para retardar señales, y para construir contadores y generadores de secuencias.

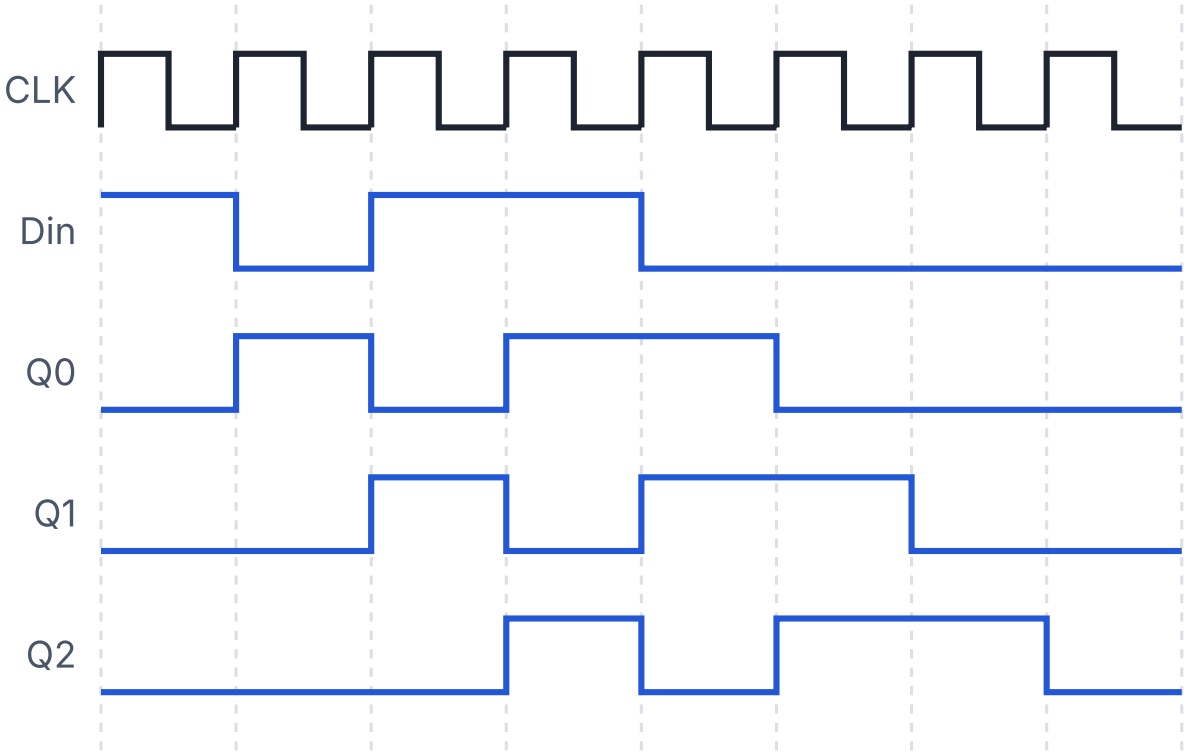
Cómo funciona

Cada biestable toma como entrada la salida del anterior: $D_i = Q_{i-1}$. En cada flanco, todo se mueve una posición y entra un bit nuevo por un extremo.



Registro de desplazamiento de 4 bits: la entrada serie avanza una etapa por cada flanco.

Si metes en serie los bits $1, 0, 1, 1$ en un registro de 4 bits, tras 4 flancos tiene 1011 en paralelo. Se ha comprobado por código.



Registro de desplazamiento de 3 bits: el dato avanza una posición por flanco

Cada bit tarda un ciclo en pasar de una etapa a la siguiente.

Los cuatro modos

Entrada → Salida	Nombre	Uso
Serie → serie	SISO	Línea de retardo
Serie → paralelo	SIPO	Recibir datos de un puerto serie
Paralelo → serie	PISO	Transmitir una palabra por un solo cable
Paralelo → paralelo	PIPO	Registro de almacenamiento

Los registros universales (74×194) tienen entradas de **modo** que eligen entre cargar en paralelo, desplazar a la izquierda, a la derecha o mantener.

Contadores basados en registros

- **Contador en anillo:** se realimenta la salida del último en la entrada del primero con **un solo 1** circulando: $1000 \to 0100 \to 0010 \to 0001 \to 1000$. Con n biestables da n **estados**. Es un

decodificador gratis: cada salida es un estado.

- **Contador Johnson (anillo retorcido):** se realimenta la salida **negada**. Para 4 bits: $0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001$. Da $2n$ **estados** y cada transición cambia un solo bit (como un Gray).

Ejemplo T9 **Secuencias de anillo y de Johnson** ✔ verificado

Un registro de 4 bits parte de 0000 (Johnson) y de 1000 (anillo). Escribe la secuencia y cuántos estados distintos tiene cada uno.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: anillo: $D_3 = Q_0$ (se desplaza a la derecha y el LSB vuelve al MSB). Johnson: $D_3 = \overline{Q_0}$.

1 Anillo

$1000 \to 0100 \to 0010 \to 0001 \to 1000$. **4 estados** para 4 biestables.

2 Johnson

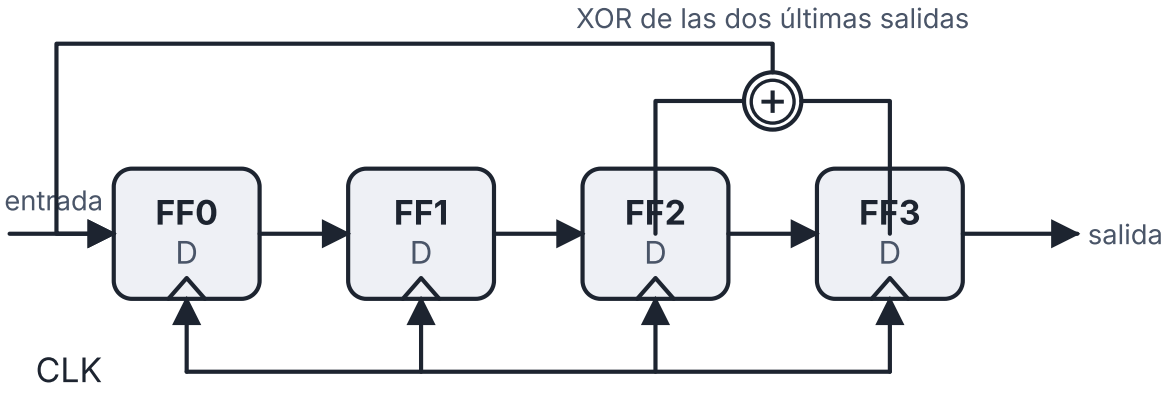
$0000 \to 1000 \to 1100 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 0000$. **8 estados** para 4 biestables.

✅ **Resultado final**

Anillo: 4 estados. Johnson: 8 estados ($2n$).

LFSR: registro con realimentación lineal

Si se realimenta la entrada con un **XOR** de algunas salidas, el registro recorre una secuencia pseudoaleatoria. Con 4 bits y realimentación $Q_3 \oplus Q_2$ se recorren los **15** estados no nulos antes de repetir (periodo $2^4 - 1$). Se usa en comunicaciones, cifrado sencillo y generadores de pruebas. El estado todo ceros es un **atasco**: nunca sale de él.



LFSR de 4 bits: la realimentación es el XOR de las dos últimas salidas.

Captura del simulador interactivo. Pruébalo en la versión web.

🚧 Qué suele fallar

- Dirección del desplazamiento: dibuja siempre qué bit entra y cuál sale.
- Confundir anillo (n estados) con Johnson ($2n$ estados).
- Olvidar poner el estado inicial (un solo 1 en el anillo): si arranca en 0000 no hay nada que circule.
- En un LFSR, arrancar en todo ceros.

∑ Fórmulas clave

- Anillo: n estados; Johnson: $2n$; LFSR máximo: $2^n - 1$.
- Serie → paralelo de n bits: n ciclos de reloj.

¿Cuántos estados tiene un contador Johnson de n bits?

$2n$.

¿Qué entra en el primer biestable de un contador en anillo?

La salida del último.

¿Qué es SIPO?

Registro serie-entrada, paralelo-salida.

¿Periodo máximo de un LFSR de n bits?

$2^n - 1$.

Registro de desplazamiento

Cadena de biestables en la que cada uno toma el valor del anterior en cada flanco.

Contador en anillo

Registro de desplazamiento realimentado con un único 1 que circula.

Contador Johnson

Registro de desplazamiento que realimenta la salida negada; $2n$ estados.

LFSR

Registro de desplazamiento con realimentación lineal (XOR) que genera secuencias pseudoaleatorias.

Un contador Johnson de 5 bits tiene:

A 5 estados

B 10 estados

C 32 estados

D 25 estados

$2n = 10.$

Un registro de desplazamiento de 8 bits recibe un dato serie. ¿Cuántos flancos de reloj hacen falta para cargar una palabra completa?

A 3

B 8

C 16

D 1

Entra un bit por flanco.

TEORÍA POR TEMAS

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

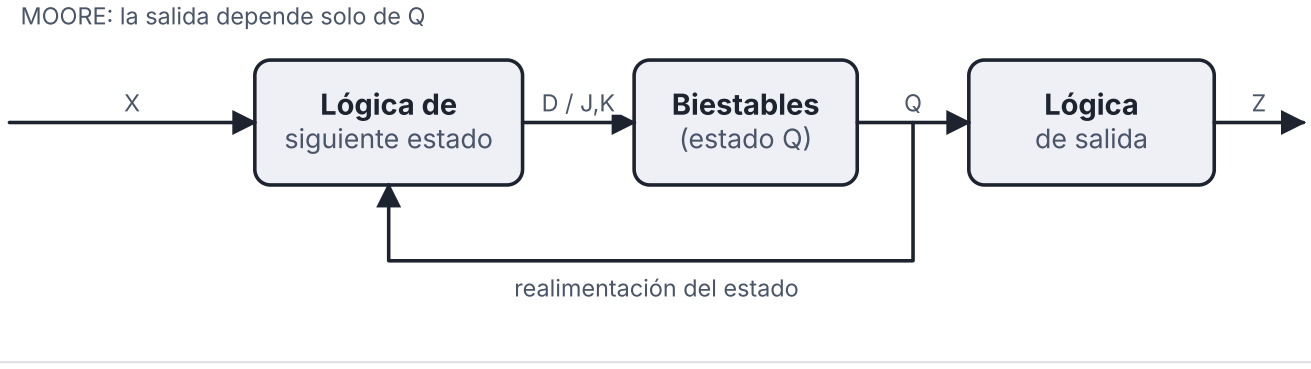
Máquinas de Moore y de Mealy: del enunciado al diagrama de estados, a la tabla y al circuito. El tema que más cae.

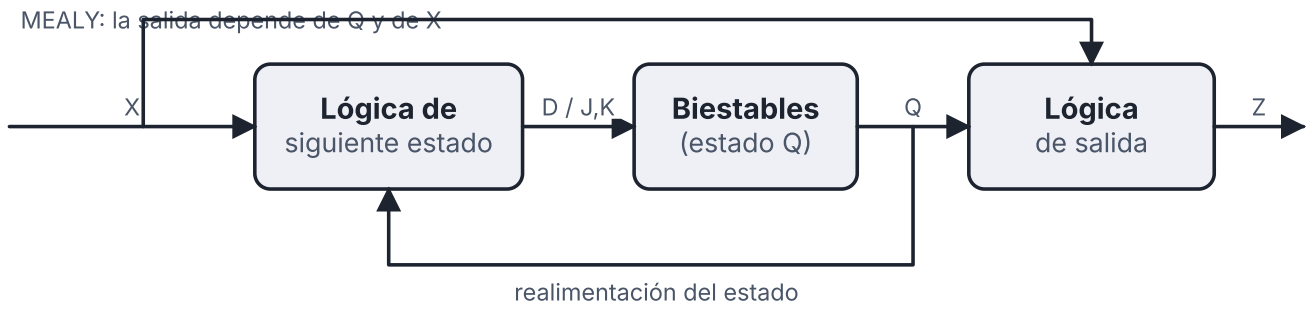
💡 **Desde cero**

Un **autómata** (máquina de estados finitos) es un circuito que **recuerda en qué situación está** (su *estado*) y, según esa situación y las entradas, decide la salida y a qué estado pasa en el siguiente flanco. Un semáforo, una caja fuerte, un ascensor o un detector de secuencias son autómatas. Diseñar uno es siempre el mismo recorrido:

palabras → diagrama de estados → tabla de transiciones → ecuaciones → circuito.

Moore y Mealy





Los dos modelos comparten la parte secuencial (biestables). Se diferencian en de qué depende la salida.

	Moore	Mealy
Salida	$Z = f(Q)$	$Z = f(Q, X)$

	Moore	Mealy
En el diagrama	Dentro del estado	En la flecha (entrada/salida)
Cambia	Solo en el flanco	En cuanto cambia la entrada
Estados	Suele necesitar más	Suele necesitar menos
Peligro	Sin glitches de entrada	La salida hereda ruidos de la entrada

El método de diseño, paso a paso

1. **Entiende el enunciado:** qué entradas hay, qué salidas, cuándo se activa cada una. Decide Moore o Mealy.
2. **Diagrama de estados:** un estado por cada «situación que hay que recordar». Para cada estado y cada valor de la entrada, una flecha.
3. **Número de biestables:** $n = \lceil \log_2(\text{estados}) \rceil$.
4. **Codificación de estados:** asigna un código binario a cada estado. Afecta al tamaño del circuito.
5. **Tabla de transiciones:** para cada (Q, entradas) escribe Q^+ y la salida.
6. **Tabla de excitación:** con la tabla del tipo de biestable (JK, D, T) escribe qué entradas hacen falta.
7. **Simplifica** con Karnaugh. Los estados sin usar y las entradas imposibles son **indiferentes**.
8. **Dibuja** el circuito y **comprueba** con una secuencia de prueba.

Ejemplo completo: detector de la secuencia 101 (Mealy)

Una entrada serie X y una salida Z. Debe dar $Z = 1$ cuando los **tres últimos** bits recibidos sean $1, 0, 1$ (las secuencias pueden solaparse: 10101 contiene dos).

SIMULADOR · DETECTOR DE SECUENCIA SERIE (AUTÓMATA DE MEALY)

Secuencia que se detecta

1011

☑ Con solapamiento

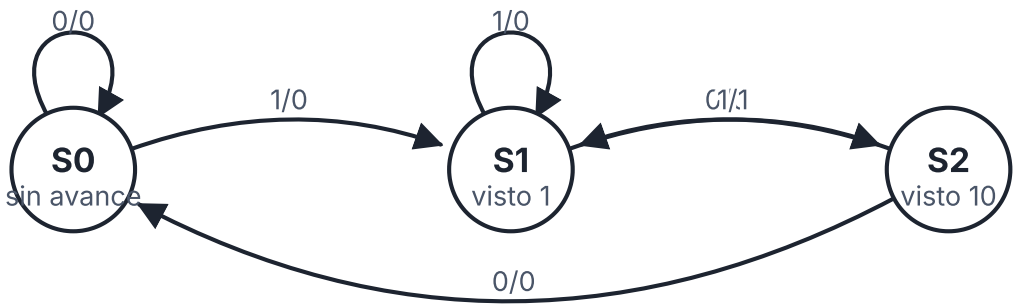
Estado	Recuerda	E=0	E=1
S0 (00)	nada	S0 / 0	S1 / 0
S1 (01)	«1»	S2 / 0	S1 / 0
S2 (10)	«10»	S0 / 0	S3 / 0
S3 (11)	«101»	S2 / 0	S1 / 1

Enviar 0 | Enviar 1 | 16 bits aleatorios | Reiniciar

Estado actual: S0 (00) 4 estados → 2 biestables

Entrada: –
Salida : –
Envía bits hasta formar «1011».

Captura del simulador interactivo. Pruébalo en la versión web.



Detector 101 (Mealy): etiqueta entrada/salida

Tres estados. En S2 (hemos visto 10), un 1 completa la secuencia (Z=1) y además sirve como primer 1 de la siguiente.

Ejemplo T10

Detector de la secuencia 101 con biestables D

✔ verificado

Diseña un detector Mealy de la secuencia 101 con solape y biestables D. Comprueba con la entrada $X = 1, 0, 1, 0, 1, 1, 0, 1$.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: Mealy, tabla de transiciones, biestable D ($D = Q^+$), Karnaugh con la combinación $Q_1Q_0 = 11$ sin usar como indiferente.

1 Estados y codificación

S_0: ningún avance. S_1: último bit 1. S_2: últimos bits 10. Son 3 estados: $n = 2$ biestables. Codifico $S_0 = 00$, $S_1 = 01$, $S_2 = 10$. La combinación 11 queda sin usar.

2 Tabla de transiciones

Q_1Q_0	X	$Q_1^+Q_0^+$	Z
00	0	00	0
00	1	01	0
01	0	10	0
01	1	01	0
10	0	00	0
10	1	01	1

3 Ecuaciones

Con D, $D = Q^+$. $Q_0^+ = 1$ siempre que $X = 1$: $D_0 = X$. $Q_1^+ = 1$ solo en (01, $X = 0$): $D_1 = Q_0\overline{X}$. La salida vale 1 solo en (10, $X = 1$): $Z = Q_1X$.

4 Comprueba con la secuencia

Estado inicial 00. $X = 1$: 01, $Z = 0$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$. $X = 1$: 01, $Z = 0$. $X = 0$: 10, $Z = 0$. $X = 1$: 01, $Z = 1$.

✅ **Resultado final**

$D_1 = Q_0\overline{X}$, $D_0 = X$, $Z = Q_1 X$. Para $X = 1,0,1,0,1,1,0,1$ la salida es $0,0,1,0,1,0,0,1$. Verificado contra una especificación independiente con las 2^{10} secuencias de hasta 10 bits.

💡 **Moore equivalente**

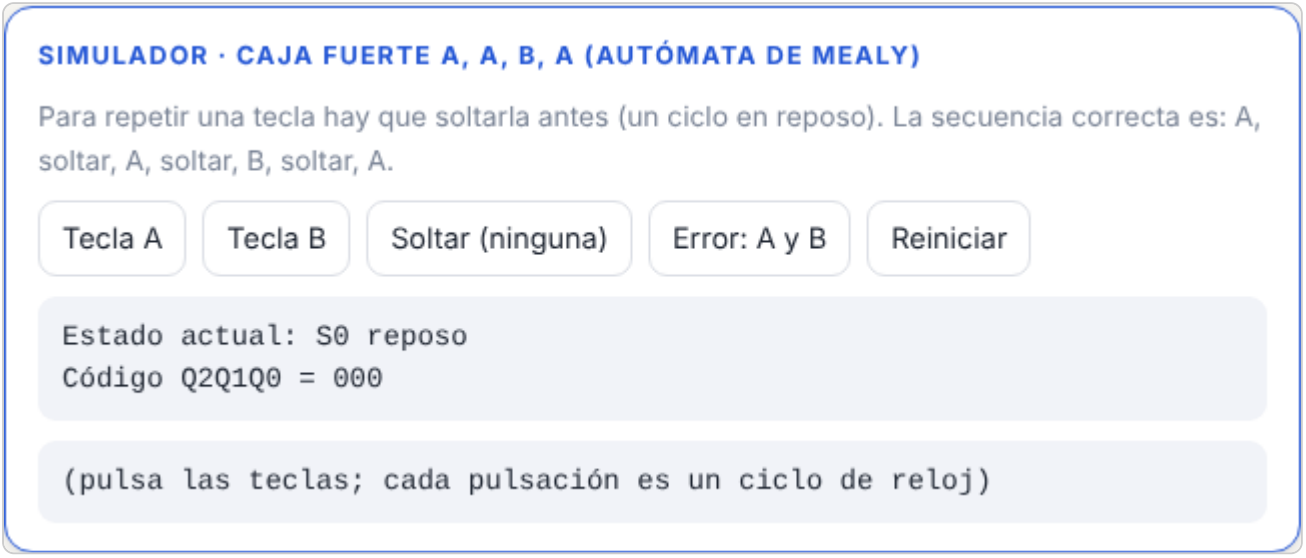
El mismo detector en Moore necesita **4 estados** (hay que añadir un estado «visto 101» donde la salida vale 1), pero la salida solo cambia en el flanco. Se ha verificado por código que es equivalente a la versión Mealy.

Análisis: de un circuito a su comportamiento

Es el camino inverso, y suele ser más corto:

1. Escribe las **ecuaciones de entrada** de los biestables y de la salida a partir del circuito.
2. Construye la **tabla de transiciones** con la tabla característica del biestable.
3. Dibuja el **diagrama de estados**.
4. Con una secuencia de entradas, dibuja el **cronograma**.

Ejemplo de análisis con T: si $T_2 = Q_1 \oplus Q_0$, $T_1 = Q_0$ y $T_0 = \overline{Q_2}$, desde 000 la secuencia de estados es $0, 1, 6, 2, 7, 5, 3, 0$ (período 7). Verificado por código.



Captura del simulador interactivo. Pruébalo en la versión web.

Antirrebote y sincronización de entradas

Un pulsador real **rebota** unos milisegundos al pulsarse. Si lo conectas directamente a un contador, cuenta decenas de pulsaciones. Soluciones: (a) un **filtro**: esperar a que la entrada permanezca estable, por ejemplo, 20 ms; a 50 MHz son $50 \cdot 10^6 \cdot 0{,}02 = 10^6$ ciclos, es decir, un contador de **20 bits**. (b) Pasar la señal por **dos biestables D** para sincronizarla con el reloj y evitar la metaestabilidad.

🔭 Más a fondo: reducción de estados

Dos estados son **equivalentes** si dan las mismas salidas ante todas las secuencias posibles. Fusionarlos reduce biestables y puertas. Se usa la **tabla de implicación** o el método de particiones. Antes de codificar, revisa si hay estados que se pueden unir.

🚧 Qué suele fallar

- Poner la salida de Mealy dentro del estado (o la de Moore en la flecha).
- Olvidar el estado inicial y un modo de forzarlo (reset).
- Contar mal los biestables: con 5 estados hacen falta 3 (no 5).
- Usar la tabla característica en lugar de la de excitación al derivar J, K.
- Olvidar que las combinaciones de código sin usar y las entradas imposibles son **indiferentes**.
- No comprobar el circuito con una secuencia: es la forma más rápida de pillar un error.

∑ Fórmulas clave

- $n = \lceil \log_2(\#\text{estados}) \rceil$.
- Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$.
- D: $D = Q^+$. T: $T = Q \oplus Q^+$. JK: usar tabla de excitación.

¿Diferencia entre Moore y Mealy?

En Moore la salida solo depende del estado; en Mealy, del estado y de la entrada.

¿Cuántos biestables para 9 estados?

4 ($2^3 = 8 < 9 \leq 16$).

¿Entradas D para un estado destino Q^+?

$D = Q^+$.

¿Entrada T para pasar de Q a Q^+?

$T = Q \oplus Q^+$.

¿Qué haces con las combinaciones de código sin usar?

Las marcas como indiferentes (X) en Karnaugh.

Autómata síncrono

Circuito secuencial cuyos biestables cambian todos con el mismo reloj.

Estado

Resumen de la historia del circuito que basta para decidir qué hará.

Diagrama de estados

Grafo con un nodo por estado y una flecha por transición.

Codificación de estados

Asignación de un código binario a cada estado.

Rebote

Oscilaciones breves de un contacto mecánico al abrir o cerrar.

Un autómata tiene 6 estados. ¿Cuántos biestables necesita como mínimo?

A 3

B 6

C 4

D 2

$2^2 = 4 < 6 \leq 2^3 = 8.$

En una máquina de Mealy, la salida:

A Depende solo del estado

B Depende solo de la entrada

C Depende del estado y de la entrada actual

D Es siempre síncrona

Esa es la definición; en Moore depende solo del estado.

Con biestables D, ¿cuál es la entrada D de un biestable que debe pasar de 0 a 1?

A $D = \overline{Q}$

B $D = 0$

C $D = X$

D $D = 1$

En un biestable D, $D = Q^{+}$.

Un pulsador rebota 10 ms. Con reloj de 100 MHz, ¿cuántos ciclos debe esperar el filtro para ser fiable?

A Al menos 10^{6}

B 10^{9}

C 100

D 10^{3}

$10 \cdot 10^{-3}\ \mathrm{s} \cdot 100 \cdot 10^{6}\ \mathrm{Hz} = 10^{6}$ ciclos.

TEORÍA POR TEMAS

Tema 11 · Descripciones funcionales y estructurales

Distintas formas de describir un mismo circuito: ecuaciones, tablas, diagramas, bloques y código.

💡 **Desde cero**

Un mismo circuito se puede **describir** de muchas formas. Una **descripción estructural** dice **de qué piezas** está hecho y cómo se conectan (un esquemático). Una **descripción funcional** dice **qué hace** sin decir cómo (una tabla, unas ecuaciones, un diagrama de estados, un programa). Saber pasar de una a otra es lo que permite usar herramientas de diseño: tú describes el comportamiento y la herramienta fabrica la estructura.

Las formas de describir

Descripción	Tipo	Cuándo usarla
Tabla de verdad	Funcional	Pocas entradas
Ecuaciones booleanas	Funcional	Lógica combinacional pequeña
Diagrama de estados / tabla de transiciones	Funcional	Autómatas
Esquemático de puertas	Estructural	Ver el circuito, depurar
Diagrama de bloques	Estructural (alto nivel)	Sistemas grandes
Lenguaje HDL (VHDL)	Ambas	Todo; es lo que usa el laboratorio

Niveles de abstracción

De más bajo a más alto: **transistor → puerta → bloque MSI (sumador, contador…) → sistema**. El diseño **jerárquico** usa piezas ya hechas y probadas. Primero se diseña y simula cada bloque; luego se unen. Es la forma de abordar el proyecto integrador.

Flujo de diseño con herramientas EDA

Flujo de diseño. Si falla un paso, se vuelve a la descripción.

Del comportamiento a la estructura

El flujo de diseño con herramientas EDA:

1. **Descripción** (esquemático, diagrama de estados o VHDL).
2. **Simulación funcional:** comprueba que *hace lo que debe*, sin retardos.
3. **Síntesis:** la herramienta convierte la descripción en puertas y biestables.
4. **Implementación:** coloca y conecta los recursos de la FPGA.
5. **Simulación temporal y prueba en placa.**

🚧 Qué suele fallar

- Confundir «estructural» con «funcional»: estructural = piezas y cables; funcional = comportamiento.
- Saltarse la simulación funcional y depurar directamente en la placa.
- Describir en un bloque todo el sistema en lugar de diseñar por partes.

¿Qué es una descripción estructural?

Dice de qué componentes consta el circuito y cómo se conectan.

¿Qué es una descripción funcional?

Dice qué hace el circuito sin indicar su estructura interna.

¿Qué hace la síntesis?

Convierte la descripción en puertas y biestables.

EDA

Herramientas de diseño electrónico asistido por ordenador.

Síntesis

Transformación de una descripción de alto nivel en una red de puertas y biestables.

Descripción estructural

Descripción por componentes e interconexiones.

Descripción funcional

Descripción por comportamiento.

Un archivo VHDL que describe un sumador con la expresión `s <= a + b;` es una descripción:

- **A** Física
- **B** De transistores
- **C** Estructural
- **D** Funcional (de comportamiento) ✓

Dice qué calcula, no con qué puertas.

¿Qué paso del flujo de diseño comprueba que el circuito hace lo que debe, sin tener en cuenta los retardos?

- **A** La simulación funcional ✓
- **B** La síntesis
- **C** La implementación
- **D** La programación de la placa

La simulación funcional ignora los retardos reales; la temporal los incluye.

TEORÍA POR TEMAS

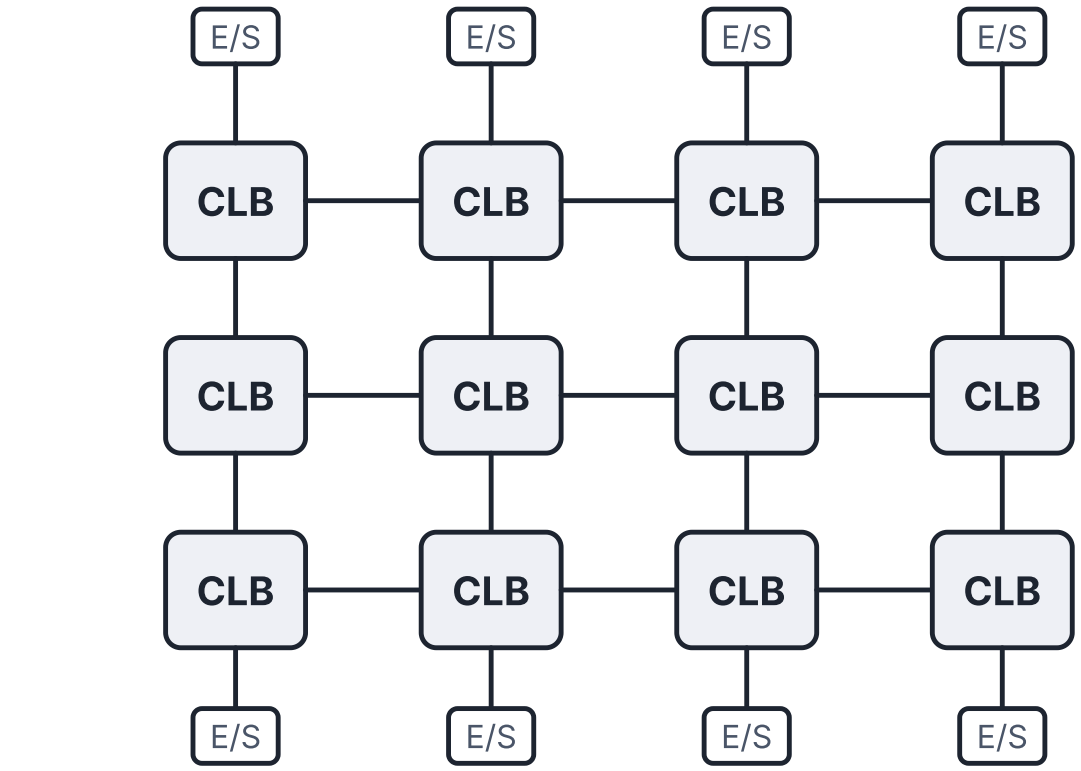
Tema 12 · Introducción a los dispositivos lógicos programables

Qué es una FPGA, cómo está hecha por dentro (LUT, biestables, interconexión) y cómo se programa.

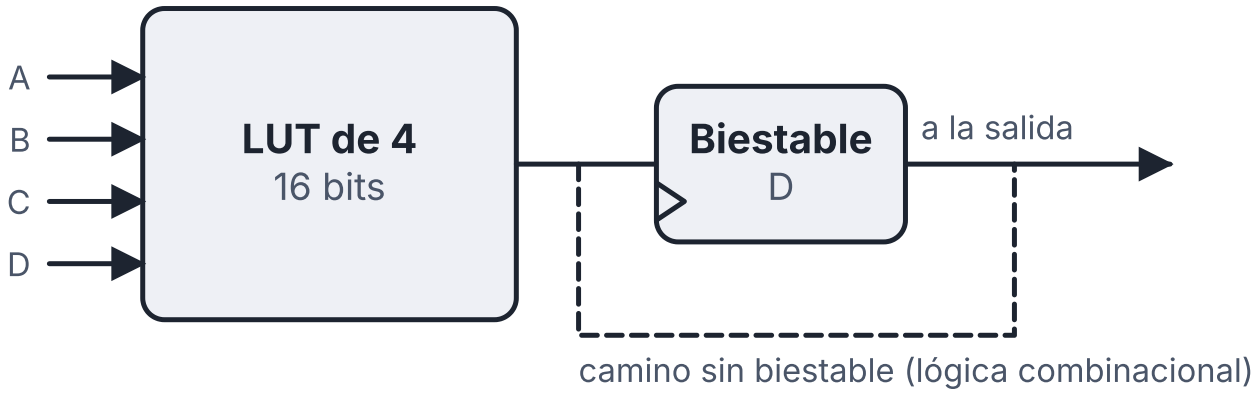
> 💡 **Desde cero**
>
> Un circuito digital normal tiene su función **fija** al fabricarlo. Un **dispositivo lógico programable** se fabrica «en blanco» y tú decides **después** qué circuito es, cargándole un fichero de configuración. Una **FPGA** (*field-programmable gate array*) es el más flexible: contiene miles de bloques lógicos pequeños y una red de cables que se puede reconectar. Es como una caja de piezas de electrónica digital que se recoloca con un fichero.

Qué hay dentro de una FPGA

- **CLB / slice** (bloque lógico configurable): la unidad básica. Contiene **tablas de consulta** (LUT) y **biestables**.
- **LUT de 4 entradas:** una memoria de $2^4 = 16$ bits. Almacena la tabla de verdad de **cualquier** función de 4 variables ($2^{16} = 65\,536$ funciones posibles).
- **Interconexión programable:** cables y conmutadores que unen los bloques.
- **Bloques de E/S:** adaptan los pines a distintos niveles de tensión.
- **Bloques extra:** RAM en bloque, multiplicadores, gestores de reloj.



Estructura de una FPGA: una malla de bloques lógicos unida por conexiones programables.



Un bloque lógico: una LUT de 4 entradas seguida, opcionalmente, de un biestable.

SIMULADOR · TABLA DE BÚSQUEDA (LUT) DE 4 ENTRADAS

XOR de 4 entradas (paridad)

☐ A = 1 ☑ B = 1 ☑ C = 1 ☐ D = 1

ABCD	Valor guardado
0000	0
0001	1
0010	1
0011	0
0100	1
0101	0
0110	0
0111	1
1000	1
1001	0
1010	0
1011	1
1100	0
1101	1
1110	1
1111	0

Entradas ABCD = 0110 → la LUT lee la posición 6 → salida = 0
F = Σm(1, 2, 4, 7, 8, 11, 13, 14)
Forma mínima: F = A'B'C'D + A'B'CD' + A'BC'D' + A'BCD + AB'C'D' + AB'CD + ABC'D + ABCD'

Haz clic en un valor de la tabla para cambiarlo: así se "programa" la LUT. Una LUT de 4 entradas guarda 16 bits y sirve para cualquier función de 4 variables.

Captura del simulador interactivo. Pruébalo en la versión web.

Una función de **más de 4 variables** se reparte entre varias LUT conectadas.

¿Por qué FPGA y no un microcontrolador?

	Microcontrolador	FPGA
Ejecuta	Instrucciones, una tras otra	Hardware real, **todo en paralelo**
Velocidad por tarea	Limitada por el programa	Muy alta (cada módulo es un circuito)
Cambiar la función	Cambiar el programa	Reconfigurar el hardware
Facilidad	Alta	Más trabajo de diseño

Flujo de trabajo en el laboratorio

1. Describes el diseño (esquemático, diagrama de estados o VHDL).
2. **Simulas** para comprobar la función.
3. **Sintetizas** y **implementas** (colocar y conectar).
4. Asignas **pines** (fichero de restricciones) a interruptores, LEDs, displays y reloj.
5. Generas el fichero de configuración y **programas la placa**.

La placa **Basys** tiene una FPGA de Xilinx, interruptores, pulsadores, LEDs, displays de 7 segmentos y un reloj de **50 MHz**.

Ejemplo T12 **Un reloj lento a partir de los 50 MHz de la placa** ✔ verificado

La placa da un reloj de 50 MHz. (a) ¿Hasta cuánto debe contar un divisor para generar 1 Hz y cuántos bits necesita? (b) Para refrescar un display de 4 dígitos con un contador de 18 bits cuyos 2 bits altos eligen el dígito, ¿a qué frecuencia cambia el dígito y a qué frecuencia se refresca cada uno?

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: divisor de frecuencia $f_{out} = f_{in}/DIV$; la salida conmuta cada $DIV/2$ ciclos; bits $= \lceil \log_2(\text{cuenta}) \rceil$.

1 **(a) Divisor a 1 Hz**

$DIV = 50 \cdot 10^6/1 = 50\,000\,000$. La salida debe conmutar cada medio periodo: $DIV/2 = 25\,000\,000$ ciclos. El contador cuenta de 0 a $24\,999\,999$. Como $2^{24} = 16{,}8 \cdot 10^6 < 25 \cdot 10^6 \leq 2^{25}$, hacen falta **25 bits**.

2 **(b) Cambio de dígito**

Los dos bits altos de un contador de 18 bits cambian cada 2^{16} ciclos: $50 \cdot 10^6 / 2^{16} = 762{,}9$ Hz.

3 **(b) Refresco de cada dígito**

Hay 4 dígitos por barrido: $762{,}9 / 4 = 190{,}7$ Hz por dígito. Por encima de unos 60 Hz el ojo no ve parpadeo.

✅ **Resultado final**

(a) Cuenta hasta $24\,999\,999$ con 25 bits. (b) Cambio de dígito a $762{,}9$ Hz; refresco de cada dígito a $190{,}7$ Hz.

🔭 **Más a fondo: CPLD frente a FPGA y memoria de configuración**

Las **CPLD** guardan su configuración en memoria no volátil (el circuito está listo al encender) y tienen retardos predecibles. Las **FPGA** de Xilinx usan **SRAM**: pierden la configuración al apagar y la cargan de una memoria externa (o del PC) al encender. Por eso al reiniciar la placa hay que volver a programarla si no usas memoria flash.

🚧 **Qué suele fallar**

- Olvidar asignar los **pines**: el diseño compila pero no hace nada en la placa.
- Suponer que la placa tiene los displays «activos a 1»: en la Basys se encienden con 0.
- Usar un reloj dividido como si fuera un reloj de la FPGA: es mejor usar **un solo reloj** y una señal de habilitación.
- Pensar que una FPGA «ejecuta» el código VHDL. El código **describe hardware** que existe a la vez.

¿Qué almacena una LUT de 4 entradas?

16 bits: la tabla de verdad de una función de 4 variables.

¿Qué ventaja tiene la FPGA sobre un microcontrolador?

Paralelismo real: cada bloque es hardware que funciona a la vez.

¿Qué reloj trae la placa Basys?

50 MHz.

¿Qué pasa con una FPGA SRAM al apagarse?

Pierde la configuración y hay que volver a cargarla.

FPGA

Dispositivo lógico reprogramable formado por bloques lógicos y una red de interconexión.

LUT

Tabla de consulta: memoria que almacena la tabla de verdad de una función de pocas variables.

CLB

Bloque lógico configurable de una FPGA (LUT y biestables).

Restricciones (UCF)

Fichero que asocia las señales del diseño con los pines físicos de la FPGA.

Síntesis

Conversión de la descripción en puertas y biestables.

Una LUT de 5 entradas almacena:

A 16 bits

B 32 bits

C 5 bits

D 25 bits

$2^5 = 32$ posiciones de un bit.

¿Qué hace falta para que un diseño aparezca en los LEDs de la placa?

A Solo simularlo

B Asignar las señales a los pines de la FPGA

C Cambiar el reloj de la placa

D Nada más que sintetizarlo

Sin las restricciones de pines, la herramienta coloca las señales donde quiere.

TEORÍA POR TEMAS

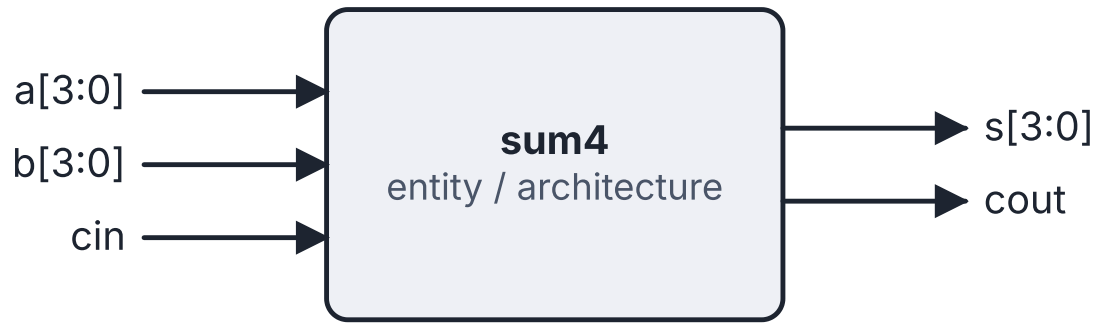
Tema 13 · Introducción al diseño digital con VHDL

Entidad y arquitectura, descripciones de flujo, de comportamiento y estructurales, procesos y máquinas de estados.

> 💡 **Desde cero**
>
> **VHDL** es un lenguaje para **describir hardware**, no para programar un procesador. Cuando escribes `y <= a and b;` no le dices «calcula» a un ordenador: describes que **existe una puerta AND** entre `a`, `b` e `y`. Dos líneas de VHDL a la vez son dos circuitos funcionando a la vez. Esta idea es la que más cuesta y la que más acaba cayendo en examen.

Las dos piezas de todo diseño

- `entity` (entidad): la **caja negra**. Dice el nombre y los puertos (entradas y salidas).
- `architecture` (arquitectura): lo que hay **dentro** de la caja.



La entidad es la caja negra: solo se ven los puertos.

vhdl · Plantilla mínima

```vhdl
 library ieee;
use ieee.std_logic_1164.all;

entity puerta_and is
  port ( a, b : in  std_logic;
         y    : out std_logic );
end puerta_and;

architecture flujo of puerta_and is
begin
```

```
  y <= a and b;
end flujo;
```

Tipos que vas a usar: `std_logic` (un bit, con valores `'0'`, `'1'`, `'Z'`, `'X'`...), `std_logic_vector(3 downto 0)` (un bus de 4 bits), `unsigned` / `signed` (para aritmética, en `ieee.numeric_std`) e `integer`.

Tres estilos de descripción

1. **Flujo de datos:** ecuaciones con asignaciones concurrentes (`<=`). Incluye `when ... else` y `with ... select`.
2. **Comportamiento:** dentro de un `process` con `if`, `case`... Se ejecuta en secuencia **en la simulación**, pero describe hardware.
3. **Estructural:** instancias de otros componentes conectadas con `port map`. Es el equivalente a un esquemático.

vhdl · Flujo de datos: multiplexor (p2_mux4.vhd)

```
 -- Práctica 2: multiplexor 4 a 1 (dos descripciones equivalentes).
library ieee;
use ieee.std_logic_1164.all;

entity p2_mux4 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    sel : in  std_logic_vector(1 downto 0);
    y   : out std_logic
  );
end p2_mux4;

architecture rtl of p2_mux4 is
begin
  with sel select
    y <= d(0) when "00",
         d(1) when "01",
         d(2) when "10",
         d(3) when others;
end rtl;
```

Combinacional y secuencial en VHDL

- **Combinacional:** proceso con **todas** las entradas en la lista de sensibilidad (o asignaciones concurrentes). Si olvidas asignar una salida en alguna rama del `if`, la herramienta infiere un **latch** (memoria no deseada). Regla de oro: **toda salida se asigna en todos los casos**.
- **Secuencial (síncrono):** proceso sensible al reloj con `rising_edge(clk)`. Lo que se asigna ahí se convierte en **biestables**.

vhdl · Secuencial: contador con reset y habilitación (p3_contador.vhd)

```vhdl
 -- Práctica 3: contador síncrono de módulo N con reset asíncrono y habilitación.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p3_contador is
  generic ( N : positive := 10 );
  port (
    clk, rst, en : in  std_logic;
    q            : out std_logic_vector(3 downto 0);
    fin          : out std_logic                       -- vale 1 en el último estado
  );
end p3_contador;

architecture rtl of p3_contador is
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= (others => '0');
    elsif rising_edge(clk) then
      if en = '1' then
        if cnt = N - 1 then
          cnt <= (others => '0');
        else
          cnt <= cnt + 1;
        end if;
      end if;
    end if;
  end process;

  q   <= std_logic_vector(cnt);
  fin <= '1' when cnt = N - 1 else '0';
end rtl;
```

Señales y variables

- Una **señal** (`signal`) es un cable (o un biestable): la asignación `<=` toma efecto **al terminar el proceso**.
- Una **variable** (`variable`) solo existe dentro de un proceso y se asigna al instante con `:=` .

Si lees una señal que acabas de asignar en el mismo proceso, ves el **valor antiguo**. Es una de las trampas más típicas.

Máquinas de estados en VHDL

La forma recomendada son **dos procesos**: uno secuencial que guarda el estado, otro combinacional con el `case` que calcula el siguiente estado y la salida. Es la traducción directa del diagrama de estados

de Tema 10.

vhdl · Máquina de estados: detector 101 (p3_detector101.vhd)

```vhdl
 -- Práctica 3: máquina de estados. Detector Mealy de la secuencia 101 con solape.
library ieee;
use ieee.std_logic_1164.all;

entity p3_detector101 is
  port (
    clk, rst, x : in  std_logic;
    z           : out std_logic
  );
end p3_detector101;

architecture rtl of p3_detector101 is
  type estado_t is (S0, S1, S2);
  signal est, sig : estado_t := S0;
begin
  -- Proceso 1: registro de estado (parte secuencial)
  process (clk, rst)
  begin
    if rst = '1' then
      est <= S0;
    elsif rising_edge(clk) then
      est <= sig;
    end if;
  end process;

  -- Proceso 2: lógica de siguiente estado y de salida (parte combinacional)
  process (est, x)
  begin
    z <= '0';
    case est is
      when S0 =>
        if x = '1' then sig <= S1; else sig <= S0; end if;
      when S1 =>
        if x = '0' then sig <= S2; else sig <= S1; end if;
      when S2 =>
        if x = '1' then sig <= S1; z <= '1'; else sig <= S0; end if;
    end case;
  end process;
end rtl;
```

Ejemplo T13 ## Leer código VHDL: ¿qué circuito es?

✔ verificado

Dado el contador `p3_contador` con $N = 10$, ¿qué secuencia genera `q` si `rst` se activa en el sexto ciclo? ¿Cuántos biestables se sintetizan y qué hace `fin`?

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: proceso síncrono, reset asíncrono, módulo N, número de bits $= \lceil \log_2 N \rceil$.

1 Cuenta normal

Con `en='1'` incrementa en cada flanco: $0, 1, 2, \ldots, 9$ y al llegar a $N - 1 = 9$ vuelve a 0.

2 Efecto del reset

`rst='1'` pone `cnt` a 0 **al instante** (es asíncrono), sin esperar al flanco. En el ejemplo la cuenta reinicia: $\ldots, 4, 5, 0, 1, 2 \ldots$

3 Recursos

`cnt` tiene 4 bits ($2^3 < 10 \leq 2^4$): **4 biestables**, más la lógica de comparación con 9 y de incremento.

4 La salida `fin`

Es combinacional: `fin='1'` cuando `cnt=9`. Sirve para encadenar otro contador.

✅ Resultado final

Secuencia $0, 1, \ldots, 9, 0, \ldots$; 4 biestables; `fin` vale 1 solo en el estado 9.

🔭 Más a fondo: genéricos y jerarquía

Un `generic` es un parámetro de la entidad (`N`, `DIV`...) que se fija al instanciar el componente. Permite reutilizar el mismo código para un contador de módulo 10 o 60. El **diseño jerárquico** crea módulos pequeños y probados (contador, divisor, display) y los une en un módulo superior con `port map`.

🚧 Qué suele fallar

- Pensar que el código se ejecuta «línea a línea» como en C: las asignaciones concurrentes funcionan **a la vez**.
- No asignar una salida en alguna rama: aparece un **latch** inesperado.
- Mezclar `std_logic_vector` con aritmética sin convertir a `unsigned`.
- Olvidar `rising_edge(clk)` y escribir procesos sensibles a todo.
- Leer una señal justo después de asignarla y esperar el valor nuevo.
- Usar un reloj generado por un divisor como si fuera reloj de la FPGA.

∑ Fórmulas clave

- Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos.
- Bits de un contador: $\lceil \log_2 N \rceil$.
- Una `case` completa en un proceso combinatorio: añade siempre `when others`.

¿Qué es una entity?

La interfaz del módulo: nombre y puertos.

¿Qué es una architecture?

La descripción de lo que hay dentro del módulo.

¿Qué infiere VHDL si no asignas una salida en todas las ramas?

Un latch.

¿Cuándo toma efecto una asignación a una signal?

Al terminar el proceso (no al instante).

¿Qué sentencia detecta un flanco de subida?

`rising_edge(clk)`.

VHDL

Lenguaje de descripción de hardware.

Entity

Parte del código VHDL que define las entradas y salidas de un módulo.

Architecture

Parte del código VHDL que define el funcionamiento interno.

Proceso

Bloque `process` de VHDL que describe comportamiento.

Latch inferido

Memoria no deseada que aparece cuando una señal no se asigna en todos los casos.

En VHDL, `y <= a and b;` describe:

- **A** Una puerta AND siempre presente
- **B** Un proceso secuencial
- **C** Una variable temporal
- **D** Una instrucción que se ejecuta una sola vez

Es una asignación concurrente: hardware que existe de forma permanente.

¿Qué provoca un `if` sin `else` dentro de un proceso combinacional?

- **A** Una puerta NOT
- **B** Un error de sintaxis siempre
- **C** Un latch inferido
- **D** Un contador

La salida mantiene su valor en la rama no cubierta, y eso es memoria.

Un contador módulo 10 en VHDL necesita como mínimo una señal de:

- **A** 5 bits
- **B** 10 bits
- **C** 3 bits
- **D** 4 bits

$\lceil \log_2 10 \rceil = 4$.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 1 · Introducción al diseño con herramientas CAD/EDA

Crear un proyecto en Xilinx ISE, describir un circuito y probarlo en la placa Basys.

💡 **Cómo se evalúa**

Las prácticas pesan el **20 %**. Se evalúan con memorias e informes y, y, según se decida en el curso, con pruebas prácticas. Hay que **defender** el trabajo: explicar cómo funciona cada diseño y por qué tomaste cada decisión. Es obligatorio asistir (mínimo 80 %) y sacar **al menos un 4,0** en teoría y en prácticas para aprobar. Si usas IA para generar código (permitido en la Práctica 4), tienes que **entenderlo, revisarlo y justificarlo**.

Simulador de las prácticas

Elige una función, cambia los valores y mira el código VHDL equivalente, la salida tipo consola y el cronograma. Los botones «Abrir en el simulador» de cada práctica cargan aquí el ejemplo correspondiente.

SIMULADOR DE PRÁCTICAS · CÓDIGO EQUIVALENTE, SALIDA DE CONSOLA Y CRONOGRAMA

Elige la función

Sumador de 4 bits (Práctica 2)

a (0 – 15)

9

b (0 – 15)

8

cin (0 – 1)

0

Código VHDL equivalente

```
entity sum4 is
  port(a,b: in std_logic_vector(3 downto 0); cin: in std_logic;
       s: out std_logic_vector(3 downto 0); cout: out std_logic);
end sum4;
architecture rtl of sum4 is
  signal t: unsigned(4 downto 0);
begin
  t    <= ('0'&unsigned(a)) + unsigned(b) + ("0000"&cin);
  s    <= std_logic_vector(t(3 downto 0));
  cout <= t(4);
end rtl;
```

Salida de consola

```
a=1001  b=1000  cin=0
s=0001  cout=1   (9 + 8 + 0 = 17)
```

Captura del simulador interactivo. Pruébalo en la versión web.

Objetivo

Aprender el flujo completo con una función trivial: describir, simular, sintetizar, asignar pines y programar la placa.

Pasos

Práctica 1 Crear el proyecto y describir el circuito

Crea un proyecto en Xilinx Foundation/ISE para la FPGA de la placa Basys y describe un circuito que enlace interruptores con LEDs.

🧭 Antes de empezar: qué piden y qué teoría usas

Herramientas: entorno ISE, captura de esquemáticos, diagramas de estados y VHDL (los tres modos de entrada de diseño). Ya conoces `entity` / `architecture` del Tema 13.

1 Nuevo proyecto

En ISE: *File → New Project*. Elige el nombre, la familia de la FPGA y el dispositivo que indica la placa Basys (consulta el manual de tu placa). Indica lenguaje **VHDL** y simulador **ISim**.

2 Nueva fuente

Project → New Source → VHDL Module. Pega el código de abajo (o dibuja el esquemático equivalente con puertas AND, OR, XOR).

3 Comprueba la sintaxis

Haz doble clic en *Check Syntax*. Debe terminar sin errores.

✅ Resultado final

Un módulo `p1_leds` con 8 interruptores, 4 pulsadores y 8 LEDs, listo para simular.

vhdl · p1_leds.vhd

```vhdl
 -- Práctica 1: entradas y salidas de la placa Basys.
-- Cada interruptor enciende el LED de su mismo número.
library ieee;
use ieee.std_logic_1164.all;

entity p1_leds is
  port (
    sw  : in  std_logic_vector(7 downto 0);   -- interruptores
    btn : in  std_logic_vector(3 downto 0);   -- pulsadores
    led : out std_logic_vector(7 downto 0)    -- LEDs
```

```
  );
end p1_leds;

architecture flujo of p1_leds is
begin
  -- Los 4 LEDs bajos copian los interruptores; los 4 altos muestran un AND y un OR de ejemplo.
  led(3 downto 0) <= sw(3 downto 0);
  led(4) <= sw(0) and sw(1);          -- AND
  led(5) <= sw(0) or  sw(1);          -- OR
  led(6) <= sw(0) xor sw(1);          -- XOR
  led(7) <= btn(0);                   -- el pulsador 0 enciende el LED 7
end flujo;
```

Práctica 1 Simular, asignar pines y programar

Simula el módulo, asigna las señales a los pines de la placa y programa la FPGA.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Flujo: simulación funcional → síntesis → implementación → asignación de pines (UCF) → fichero de configuración → programación.

1 Simulación

Crea un *test bench* o usa el simulador de ISE: pon `sw=00000011`. Espera `led=00110011` (AND=1, OR=1, XOR=0).

2 Asignación de pines

Crea un fichero de restricciones (UCF) que una cada señal con su pin físico. Ejemplo (**comprueba los pines con el manual de tu placa**):

3 Síntesis, implementación y bitstream

Synthesize → *Implement Design* → *Generate Programming File*. Si hay avisos de «latch» o de señales sin pin, léelos.

4 Programar

Con la placa conectada, ejecuta iMPACT y carga el fichero `.bit`. Mueve los interruptores y comprueba los LEDs.

✅ **Resultado final**

Los LEDs siguen los interruptores. Si no se enciende ninguno, lo primero que hay que mirar es la asignación de pines y que el fichero se haya cargado.

text · p1_basys.ucf

```
 # Restricciones de ejemplo. COMPRUEBA los pines con el manual de TU placa Basys antes de usarlos.
NET "sw<0>"  LOC = "P11";
NET "sw<1>"  LOC = "L3";
NET "sw<2>"  LOC = "K3";
NET "sw<3>"  LOC = "B4";
NET "led<0>" LOC = "M5";
NET "led<1>" LOC = "M11";
NET "led<2>" LOC = "P7";
NET "led<3>" LOC = "P6";
# (resto de pines igual: un NET por señal)
```

Salida esperada

text · Resultado del modelo Python (salida esperada en la placa)

```
 sw        btn0  led
00000000  0     00000000
00000001  0     01100001
00000011  0     00110011
00000010  1     11100010
00001111  1     10111111
```

Qué escribir en la memoria

- Un esquema del circuito y su tabla de verdad.
- Una captura de la simulación con $sw = 00000011$ y la explicación del valor de cada LED.
- Una frase sobre qué hace cada paso del flujo (síntesis, implementación, programación).
- Cuál fue el error más típico que cometiste (por ejemplo, pines mal asignados) y cómo lo resolviste.

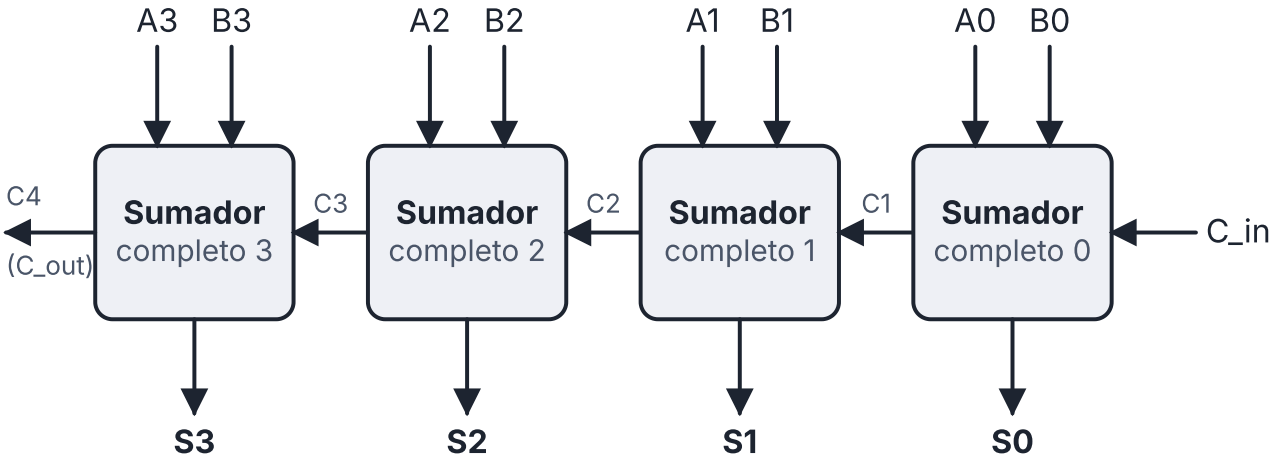
PRÁCTICAS (20 % DE LA NOTA)

Práctica 2 · Diseño e implementación de sistemas combinacionales

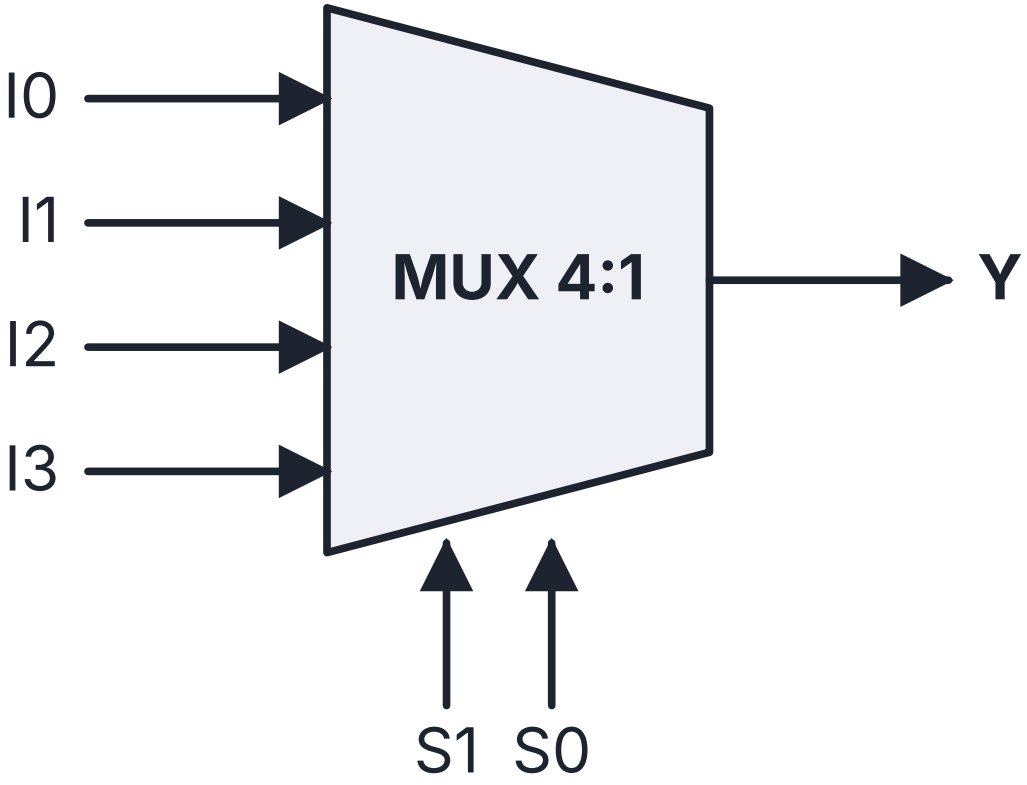
Sumador, comparador, multiplexor y display de 7 segmentos, mostrados en LEDs y displays.

Objetivo

Diseñar con esquemático y con VHDL los bloques combinacionales del Tema 5 y verlos funcionar en LEDs y displays.



Estructura del sumador de 4 bits de la práctica.



Multiplexor 4:1 que se pide en la práctica.

Práctica 2 Sumador de 4 bits

✔ verificado

Describe un sumador de 4 bits con acarreo de entrada y salida. Muestra la suma en 4 LEDs y el acarreo en un quinto.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> Teoría: sumador completo y en cascada (T5); aritmética con `unsigned` (T13).

1 Qué se pide

Entradas `a`, `b` (4 bits) y `cin`; salidas `s` (4 bits) y `cout`.

2 Idea

Para no perder el acarreo, se hace la suma en **5 bits**: se amplían `a` y `b` con un `'0'` por la izquierda. El bit más significativo es `cout`.

3 Código

Ver abajo.

✅ **Resultado final**

$s = (a + b + cin) \bmod 16$ y $cout = \lfloor (a + b + cin)/16 \rfloor$. Comprobado con las 512 combinaciones del modelo.

vhdl · p2_sum4.vhd

```vhdl
 -- Práctica 2: sumador de 4 bits con acarreo de entrada y de salida.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p2_sum4 is
  port (
    a, b : in  std_logic_vector(3 downto 0);
    cin  : in  std_logic;
    s    : out std_logic_vector(3 downto 0);
    cout : out std_logic
  );
end p2_sum4;

architecture rtl of p2_sum4 is
  signal t : unsigned(4 downto 0);        -- un bit más para el acarreo
begin
  t    <= ('0' & unsigned(a)) + ('0' & unsigned(b)) + ("0000" & cin);
  s    <= std_logic_vector(t(3 downto 0));
  cout <= t(4);
end rtl;
```

Práctica 2 Comparador y multiplexor

✔ verificado

Diseña un comparador de 4 bits con salidas $A > B$, $A = B$, $A < B$ y un multiplexor 4:1.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: comparador y multiplexor (T5); `when ... else` y `with ... select` (T13).

1 Comparador

Con `unsigned` el comparador es una línea por salida. **Exactamente una** de las tres salidas vale 1: es una buena comprobación.

2 Multiplexor

Un `with sel select` enumera las cuatro posibilidades de `sel`; la última usa `others` para cubrir los valores no binarios de `std_logic`.

✅ Resultado final

Comparador: una y solo una salida a 1. Mux: $y = d_{sel}$.

vhdl · p2_cmp4.vhd

```vhdl
-- Práctica 2: comparador de 4 bits.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p2_cmp4 is
  port (
    a, b       : in  std_logic_vector(3 downto 0);
    gt, eq, lt : out std_logic
  );
end p2_cmp4;

architecture rtl of p2_cmp4 is
begin
  gt <= '1' when unsigned(a) >  unsigned(b) else '0';
  eq <= '1' when unsigned(a) =  unsigned(b) else '0';
  lt <= '1' when unsigned(a) <  unsigned(b) else '0';
end rtl;
```

vhdl · p2_mux4.vhd

```vhdl
-- Práctica 2: multiplexor 4 a 1 (dos descripciones equivalentes).
library ieee;
use ieee.std_logic_1164.all;

entity p2_mux4 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    sel : in  std_logic_vector(1 downto 0);
    y   : out std_logic
  );
end p2_mux4;

architecture rtl of p2_mux4 is
begin
  with sel select
    y <= d(0) when "00",
         d(1) when "01",
         d(2) when "10",
```

```
        d(3) when others;
end rtl;
```

Práctica 2 Display de 7 segmentos

Muestra en un display de 7 segmentos de la Basys una cifra BCD recibida por los interruptores.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: decodificador BCD a 7 segmentos con indiferentes en 1010–1111 (T5); en la Basys los segmentos son **activos a nivel bajo**.

1 Tabla

Para cada cifra, qué segmentos (a a g) deben encenderse. En lógica positiva el 8 enciende los siete; en la Basys hay que **invertir**.

2 Código

Un `with d select` con las 10 cifras, y `"1111111"` (apagado) para el resto.

3 Activa un dígito

La Basys tiene 4 dígitos con ánodo común: para ver una cifra hay que **activar el ánodo** de ese dígito (a nivel bajo).

✅ Resultado final

Con `d=0111` el display muestra un 7: segmentos a, b, c a 0 (encendidos), el resto a 1.

vhdl · p2_seg7.vhd

```
 -- Práctica 2: decodificador BCD a 7 segmentos para la Basys (segmentos activos a nivel bajo).
-- seg = (a, b, c, d, e, f, g). Un 0 enciende el segmento.
library ieee;
use ieee.std_logic_1164.all;

entity p2_seg7 is
  port (
    d   : in  std_logic_vector(3 downto 0);
    seg : out std_logic_vector(6 downto 0)
  );
end p2_seg7;
```

```
architecture rtl of p2_seg7 is
begin
  with d select
    seg <= "0000001" when "0000",  -- 0
           "1001111" when "0001",  -- 1
           "0010010" when "0010",  -- 2
           "0000110" when "0011",  -- 3
           "1001100" when "0100",  -- 4
           "0100100" when "0101",  -- 5
           "0100000" when "0110",  -- 6
           "0001111" when "0111",  -- 7
           "0000000" when "1000",  -- 8
           "0000100" when "1001",  -- 9
           "1111111" when others;  -- apagado (1010 a 1111 no son BCD)
end rtl;
```

Salida esperada

text · Resultado del modelo Python

```
 --- sumador de 4 bits ---
 9 +  8 + 0 = s 0001 ( 1), cout 1
15 +  1 + 0 = s 0000 ( 0), cout 1
 7 +  8 + 1 = s 0000 ( 0), cout 1
15 + 15 + 1 = s 1111 (15), cout 1
--- comparador ---
a= 5 b= 9 -> gt,eq,lt = (0, 0, 1)
a= 9 b= 9 -> gt,eq,lt = (0, 1, 0)
a=12 b= 3 -> gt,eq,lt = (1, 0, 0)
--- mux 4:1 (d=0110) ---
sel=00 -> y=0
sel=01 -> y=1
sel=10 -> y=1
sel=11 -> y=0
--- display (activo a 0, abcdefg) ---
 0 -> 0000001
 1 -> 1001111
 7 -> 0001111
 9 -> 0000100
12 -> 1111111
Comprobaciones exhaustivas: OK
```

Qué escribir en la memoria

- La tabla de verdad del decodificador y las ecuaciones simplificadas (con indiferentes).
- Una comparación del esquemático y del VHDL: cuántos recursos usa cada uno.
- Capturas de la simulación del sumador con un caso con acarreo ($15 + 1$).

- Por qué en el display se usa lógica negativa.

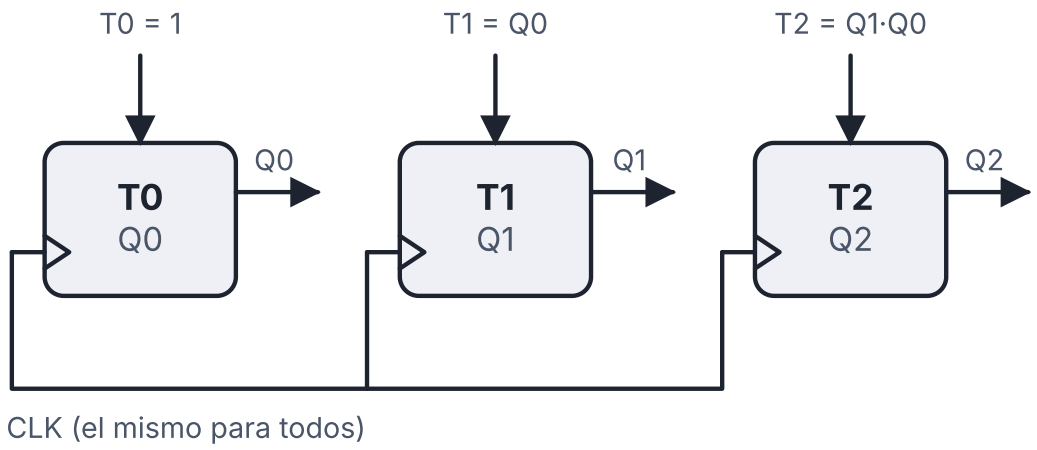
PRÁCTICAS (20 % DE LA NOTA)

Práctica 3 · Diseño de sistemas secuenciales

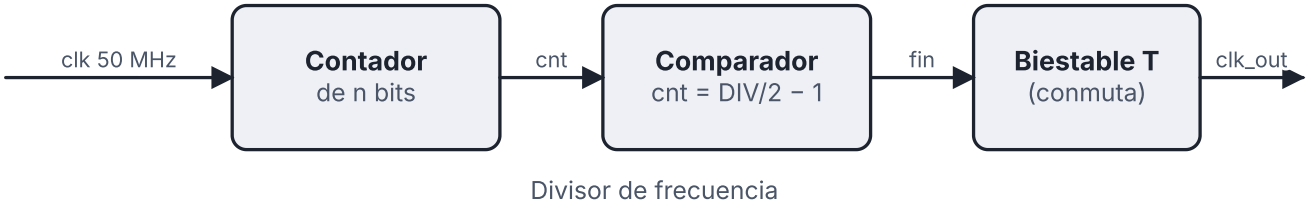
Contadores, divisor de frecuencia, antirrebote y máquinas de estados con reloj.

Objetivo

Diseñar sistemas sincronizados con reloj: contadores, registros y divisores de frecuencia, máquinas de estados y tratamiento del rebote de los pulsadores (T8, T9, T10).



Contador síncrono, base de los divisores de frecuencia.



Divisor de frecuencia: cuenta medio periodo de la salida y entonces conmuta.

Captura del simulador interactivo. Pruébalo en la versión web.

Práctica 3

Contador módulo N

✔ verificado

Diseña un contador síncrono de módulo N configurable, con reset asíncrono, habilitación y una salida de fin de cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: contador de módulo N, biestables $\lceil \log_2 N \rceil$ (T8); proceso síncrono con reset asíncrono (T13).

1 Estructura

Un único proceso con `clk` y `rst` en la lista de sensibilidad. El reset va **antes** que el flanco.

2 Módulo N

Si `cnt = N-1` vuelve a 0; si no, incrementa. El `generic N` hace el diseño reutilizable.

3 Fin de cuenta

`fin` vale 1 cuando `cnt = N-1` : sirve para encadenar contadores.

✅ Resultado final

Secuencia $0, 1, \ldots, N-1, 0, \ldots$. Para $N = 10$: 4 biestables.

vhdl · p3_contador.vhd

```
 -- Práctica 3: contador síncrono de módulo N con reset asíncrono y habilitación.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p3_contador is
  generic ( N : positive := 10 );
  port (
    clk, rst, en : in  std_logic;
    q            : out std_logic_vector(3 downto 0);
    fin          : out std_logic                       -- vale 1 en el último estado
  );
end p3_contador;

architecture rtl of p3_contador is
  signal cnt : unsigned(3 downto 0) := (others => '0');
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= (others => '0');
    elsif rising_edge(clk) then
      if en = '1' then
        if cnt = N - 1 then
          cnt <= (others => '0');
        else
          cnt <= cnt + 1;
        end if;
      end if;
    end if;
```

```
  end process;

  q   <= std_logic_vector(cnt);
  fin <= '1' when cnt = N - 1 else '0';
end rtl;
```

Práctica 3 Divisor de frecuencia

✔ verificado

Genera una señal de 1 Hz a partir del reloj de 50 MHz de la placa y úsala para hacer parpadear un LED.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: contador como divisor (T8); $DIV = f_{in}/f_{out}$.

1 Cálculo

$DIV = 50\,000\,000$. La salida conmuta cada $DIV/2 = 25\,000\,000$ ciclos: se cuenta de 0 a $24\,999\,999$ (25 bits).

2 Código

Contador con comparación y una señal `s` que se invierte al llegar al final.

3 Prueba

En simulación no esperes 25 millones de ciclos: baja `DIV` (por ejemplo, a 10) en el *test bench* y comprueba la forma de onda.

✅ Resultado final

LED parpadeando a 1 Hz (medio segundo encendido y medio apagado).

vhdl · p3_divisor.vhd

```
 -- Práctica 3: divisor de frecuencia. Con reloj de 50 MHz y DIV = 50_000_000 da una salida de 1 Hz.
library ieee;
use ieee.std_logic_1164.all;

entity p3_divisor is
  generic ( DIV : positive := 50_000_000 );     -- relación f_in / f_out (número par)
  port (
    clk, rst : in  std_logic;
```

```
    clk_out  : out std_logic
  );
end p3_divisor;

architecture rtl of p3_divisor is
  signal cnt : integer range 0 to DIV/2 - 1 := 0;
  signal s   : std_logic := '0';
begin
  process (clk, rst)
  begin
    if rst = '1' then
      cnt <= 0; s <= '0';
    elsif rising_edge(clk) then
      if cnt = DIV/2 - 1 then
        cnt <= 0;
        s   <= not s;                          -- conmuta cada medio periodo
      else
        cnt <= cnt + 1;
      end if;
    end if;
  end process;
  clk_out <= s;
end rtl;
```

Práctica 3 ## Antirrebote de un pulsador

✔ verificado

Un pulsador mecánico rebota unos milisegundos. Diseña un circuito que entregue un nivel limpio.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: sincronizar la entrada con 2 biestables para evitar la metaestabilidad (T6) y aceptarla solo si permanece estable un tiempo (T10).

1 Sincroniza

`s1` y `s2` son dos biestables D en serie: la entrada asíncrona se alinea con el reloj.

2 Cuenta de estabilidad

Si la entrada cambia, reinicia el contador. Si llega a `LIM` sin cambiar, acepta el valor.

3 Calcula LIM

20 ms a 50 MHz son 10^6 ciclos: contador de 20 bits.

✅ Resultado final

Un solo pulso limpio por pulsación, aunque la señal original rebote.

vhdl · p3_antirrebote.vhd

```vhdl
 -- Práctica 3: antirrebote. La salida sigue a la entrada solo si esta permanece estable LIM ciclos.
-- Con reloj de 50 MHz y LIM = 1_000_000 el filtro espera 20 ms.
library ieee;
use ieee.std_logic_1164.all;

entity p3_antirrebote is
  generic ( LIM : positive := 1_000_000 );
  port (
    clk, btn : in  std_logic;
    limpio   : out std_logic
  );
end p3_antirrebote;

architecture rtl of p3_antirrebote is
  signal s1, s2 : std_logic := '0';          -- sincronizador de dos biestables
  signal prev   : std_logic := '0';
  signal cnt    : integer range 0 to LIM := 0;
  signal sal    : std_logic := '0';
begin
  process (clk)
  begin
    if rising_edge(clk) then
      s1 <= btn;  s2 <= s1;                  -- evita la metaestabilidad
      if s2 /= prev then                     -- la entrada ha cambiado: reinicia la cuenta
        prev <= s2;
        cnt  <= 0;
      elsif cnt = LIM then
        sal  <= prev;                        -- estable el tiempo suficiente: se acepta
      else
        cnt  <= cnt + 1;
      end if;
    end if;
  end process;
  limpio <= sal;
end rtl;
```

Práctica 3 Máquina de estados: detector de secuencia

✔ verificado

Implementa en VHDL el detector Mealy de la secuencia 101 del Tema 10.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: diagrama de estados de Mealy y su traducción a dos procesos.

1 Tipo enumerado

`type estado_t is (S0, S1, S2);` deja que la herramienta elija la codificación.

2 Dos procesos

El primero guarda el estado en el flanco. El segundo, combinacional, calcula `sig` y `z` con un `case`.

3 Evita latches

Asigna `z <= '0'` al principio del proceso combinacional: así `z` se asigna en **todos** los casos.

✅ Resultado final

Para $x = 1, 0, 1, 0, 1, 1, 0, 1$ da $z = 0, 0, 1, 0, 1, 0, 0, 1$.

vhdl · p3_detector101.vhd

```vhdl
 -- Práctica 3: máquina de estados. Detector Mealy de la secuencia 101 con solape.
library ieee;
use ieee.std_logic_1164.all;

entity p3_detector101 is
  port (
    clk, rst, x : in  std_logic;
    z           : out std_logic
  );
end p3_detector101;

architecture rtl of p3_detector101 is
  type estado_t is (S0, S1, S2);
  signal est, sig : estado_t := S0;
begin
  -- Proceso 1: registro de estado (parte secuencial)
  process (clk, rst)
  begin
    if rst = '1' then
      est <= S0;
    elsif rising_edge(clk) then
      est <= sig;
    end if;
  end process;

  -- Proceso 2: lógica de siguiente estado y de salida (parte combinacional)
  process (est, x)
  begin
    z <= '0';
    case est is
      when S0 =>
```

```
        if x = '1' then sig <= S1; else sig <= S0; end if;
      when S1 =>
        if x = '0' then sig <= S2; else sig <= S1; end if;
      when S2 =>
        if x = '1' then sig <= S1; z <= '1'; else sig <= S0; end if;
    end case;
  end process;
end rtl;
```

Salida esperada

```text · Resultado del modelo Python
 contador mod 10: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2]
contador mod 12: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 0, 1]
divisor 50 MHz -> 1 Hz: DIV=50000000, cuenta hasta 24999999, 25 bits
divisor 50 MHz -> 1 kHz: DIV=50000, cuenta hasta 24999, 15 bits
entrada : 001010111111110100000000000
salida  : 000000000000011111111110000
detector 101: x = [1, 0, 1, 0, 1, 1, 0, 1]  z = [0, 0, 1, 0, 1, 0, 0, 1]
Comprobaciones: OK
```

Qué escribir en la memoria

- El diagrama de estados del detector y la tabla de transiciones.
- El cálculo de DIV y del número de bits del contador del divisor.
- Una captura de la señal del pulsador **con rebotes** y la salida del filtro.
- Una discusión de por qué el reset asíncrono puede dar problemas y cuándo conviene el síncrono.

PRÁCTICAS (20 % DE LA NOTA)

Práctica 4 · Desarrollo de sistemas digitales mediante VHDL

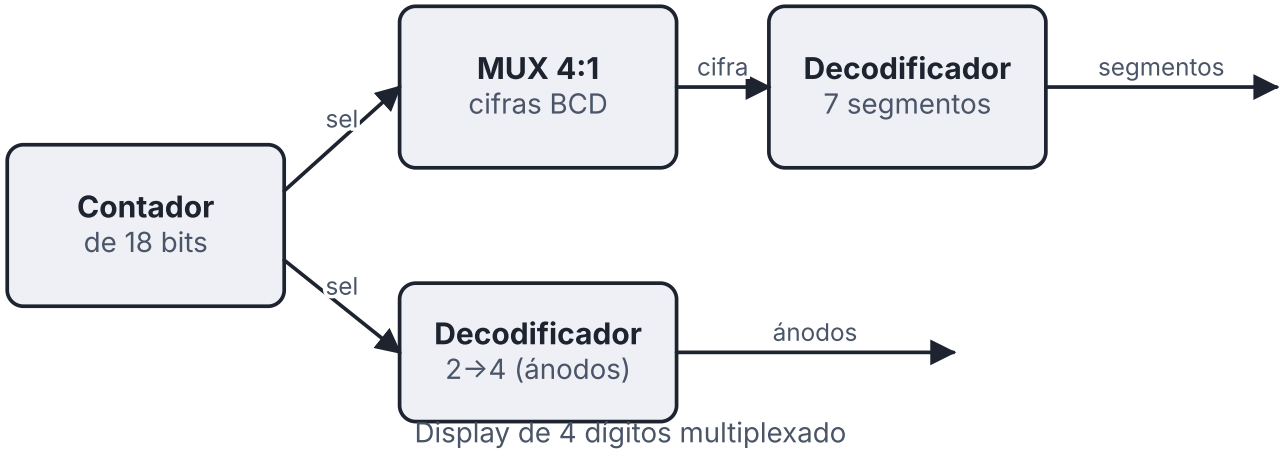
Un sistema completo en VHDL: se permite usar IA, pero hay que entender, revisar y justificar el código.

⚠️ Sobre el uso de IA

Se permite usar IA como apoyo, pero el código debe ser **comprendido, revisado y justificado por ti**. En la defensa pueden pedirte que expliques una línea, que encuentres un error o que lo modifiques. La forma segura de prepararte: pide el código, **simúlalo**, tradúcelo a un diagrama y **busca los fallos típicos** (latches, reset, reloj dividido, señales no asignadas).

Objetivo

Construir un sistema completo combinando bloques: el ejemplo de la guía muestra cuatro cifras en el display multiplexado de la Basys.



Display de 4 dígitos multiplexado

El contador de refresco elige a la vez la cifra y el dígito que se enciende.

Práctica 4 **Display de 4 dígitos multiplexado** ✔ verificado

Muestra cuatro cifras BCD en el display de 4 dígitos de la Basys. Calcula la frecuencia de refresco.

🧭 Antes de empezar: qué piden y qué teoría usas

Teoría: multiplexación en el tiempo, contador de refresco, decodificador de 7 segmentos activo a nivel bajo, múltiples dígitos con ánodo común.

1 Idea

Los 4 dígitos comparten los 7 segmentos. Solo uno está activo cada vez. Se alternan tan rápido que el ojo los ve todos encendidos.

2 Contador de refresco

Un contador libre de 18 bits. Sus dos bits altos eligen el dígito: cada dígito dura 2^{16} ciclos.

3 Selección

Un `with sel select` elige la cifra y activa el ánodo correspondiente (activo a 0).

4 Frecuencias

Cambio de dígito: $50\,\mathrm{MHz}/2^{16} = 762{,}9$ Hz. Refresco de cada dígito: $762{,}9/4 = 190{,}7$ Hz.

✅ Resultado final

Cuatro cifras estables, sin parpadeo visible, con refresco de $190{,}7$ Hz por dígito.

vhdl · p4_display4.vhd

```vhdl
 -- Práctica 4: manejo multiplexado de un display de 4 dígitos (Basys). Muestra 4 cifras BCD.
-- Se enciende un dígito cada vez. Con reloj de 50 MHz se cambia de dígito a 762,9 Hz (50e6/2^16)
-- y cada dígito se refresca a 190,7 Hz: el ojo los ve todos encendidos y sin parpadeo.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p4_display4 is
  port (
    clk    : in  std_logic;                       -- 50 MHz
    d3, d2, d1, d0 : in std_logic_vector(3 downto 0);
    seg    : out std_logic_vector(6 downto 0);    -- activos a nivel bajo
    an     : out std_logic_vector(3 downto 0)     -- ánodos, activos a nivel bajo
  );
end p4_display4;
```

```
architecture rtl of p4_display4 is
  signal cnt : unsigned(17 downto 0) := (others => '0');   -- 2^16 ciclos por dígito
  signal sel : std_logic_vector(1 downto 0);
  signal dig : std_logic_vector(3 downto 0);
begin
  process (clk)
  begin
    if rising_edge(clk) then cnt <= cnt + 1; end if;
  end process;
  sel <= std_logic_vector(cnt(17 downto 16));

  with sel select dig <= d0 when "00", d1 when "01", d2 when "10", d3 when others;
  with sel select an  <= "1110" when "00", "1101" when "01", "1011" when "10", "0111" when others;

  with dig select
    seg <= "0000001" when "0000", "1001111" when "0001", "0010010" when "0010",
           "0000110" when "0011", "1001100" when "0100", "0100100" when "0101",
           "0100000" when "0110", "0001111" when "0111", "0000000" when "1000",
           "0000100" when "1001", "1111111" when others;
end rtl;
```

text · Resultado del modelo Python

```
 cambio de dígito: 762.9 Hz; refresco de cada dígito: 190.7 Hz
barrido para mostrar 1 2 3 4:
  an=1110  cifra=4
  an=1101  cifra=3
  an=1011  cifra=2
  an=0111  cifra=1
Comprobaciones: OK
```

Qué escribir en la memoria

- Las prompts que usaste y qué cambiaste del código generado, y por qué.
- Un diagrama de bloques jerárquico del sistema.
- Los cálculos de frecuencias y tiempos.
- Una lista de **tres errores** que detectaste al revisar el código.

PRÁCTICAS (20 % DE LA NOTA)

Prácticas 5 a 7 · Proyecto integrador

Especificación, implementación e integración de un sistema distribuido que monitoriza una incubadora neonatal.

En qué consisten

Es un **proyecto colaborativo** basado en la monitorización de una incubadora neonatal (Dräger Caleo) usada como **entorno de simulación**.

Práctica	Qué se hace
P5	Especificación y diseño del proyecto integrador
P6	Cada grupo implementa y depura el **módulo asignado** del sistema distribuido, con pruebas individuales y documentación técnica
P7	Integración de todos los módulos, validación y **defensa pública** de la solución

> ⚠️ **Qué no sé de tu proyecto**
>
> El programa solo describe el proyecto en general. El módulo concreto que te asignen y los datos exactos de la incubadora se fijan en el curso. El ejemplo de abajo (una alarma de temperatura con histéresis) **no es tu módulo asignado**: está para mostrar cómo se diseña, se prueba y se documenta un módulo típico de un sistema de monitorización.

Método para cada módulo

1. **Especifica** entradas, salidas, rangos y qué debe hacer, con una tabla.
2. **Diseña** en bloques: qué es combinacional, qué es secuencial.
3. **Escribe** el código (VHDL) y **simula** con casos límite.
4. **Prueba** el módulo aislado en la placa.
5. **Documenta**: interfaz, funcionamiento, pruebas y resultados.
6. **Integra**: conecta con los demás módulos y comprueba temporización y niveles.

Ejemplo: alarma de temperatura con histéresis

Un sensor da la temperatura en décimas de grado. Se quiere una alarma de alta ($\geq 37{,}5$ °C) y una de baja ($< 36{,}0$ °C). Sin histéresis, una medida que oscile alrededor del umbral haría **parpadear** la alarma. Con histéresis, la alarma se **desactiva** solo cuando la medida baja del umbral menos un margen (0,5 °C).

Prácticas 5-7 **Alarma con histéresis**

Diseña un módulo `p6_alarma` que reciba `temp` (10 bits, décimas de grado) y active `al_alta` si $T \geq 375$ y `al_baja` si $T < 360$, con histéresis de 5 décimas.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Teoría: comparadores (T5), biestable que recuerda el estado de la alarma (T6), proceso síncrono (T13).

1 Qué hay que recordar

La alarma depende del **pasado** (si ya estaba activa o no) y no solo de la entrada: es un circuito **secuencial** con un biestable por alarma.

2 Reglas

Alta: se activa con $T \geq T_{ALTA}$ y se desactiva con $T < T_{ALTA} - H$. Baja: se activa con $T < T_{BAJA}$ y se desactiva con $T \geq T_{BAJA} + H$. Entre ambos valores **mantiene** el estado.

3 Código

Ver abajo.

4 Casos límite que hay que probar

$T = 374$ (no activa), 375 (activa), 373 (sigue activa por la histéresis), 369 (se desactiva), y los equivalentes para la baja.

✅ **Resultado final**

Para las medidas $365, 370, 374, 375, 376, 373, 371, 369, \ldots$ la alarma de alta se activa en 375

y se desactiva en 369.

vhdl · p6_alarma_incubadora.vhd

```
 -- Prácticas 5 a 7 (ejemplo de módulo): alarma de temperatura con histéresis para un sistema de
monitorización.
-- Entrada: temperatura en décimas de grado (0 a 1023). Salidas: alarma de alta y de baja.
-- La histéresis evita que la alarma parpadee cuando la medida oscila alrededor del umbral.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity p6_alarma is
  generic (
    T_ALTA  : natural := 375;    -- 37,5 °C: se activa la alarma de alta
    T_BAJA  : natural := 360;    -- 36,0 °C: se activa la alarma de baja
    HIST    : natural := 5       -- 0,5 °C de histéresis
  );
  port (
    clk, rst : in  std_logic;
    temp     : in  std_logic_vector(9 downto 0);
    al_alta  : out std_logic;
    al_baja  : out std_logic
  );
end p6_alarma;

architecture rtl of p6_alarma is
  signal t : unsigned(9 downto 0);
  signal a, b : std_logic := '0';
begin
  t <= unsigned(temp);
  process (clk, rst)
  begin
    if rst = '1' then
      a <= '0'; b <= '0';
    elsif rising_edge(clk) then
      if    t >= T_ALTA        then a <= '1';
      elsif t <  T_ALTA - HIST then a <= '0';
      end if;
      if    t <  T_BAJA        then b <= '1';
      elsif t >= T_BAJA + HIST then b <= '0';
      end if;
    end if;
  end process;
  al_alta <= a;
  al_baja <= b;
end rtl;
```

text · Resultado del modelo Python

```
 temp(0,1 °C)  al_alta  al_baja
   365        0        0
   370        0        0
   374        0        0
   375        1        0
   376        1        0
```

```
   373        1        0
   371        1        0
   369        0        0
   366        0        0
   362        0        0
   359        0        1
   361        0        1
   364        0        1
   366        0        0
Comprobaciones: OK
```

Qué debe contener la documentación y la defensa

- Una **tabla de interfaz** (señales, ancho, sentido, significado).
- Un **diagrama de bloques** del módulo y de cómo encaja en el sistema.
- La **estrategia de pruebas** (casos normales y límite) y los resultados.
- La **justificación** de cada decisión de diseño (por qué histéresis, por qué ese ancho de bits).
- Si usaste IA: qué pediste, qué cambiaste y cómo comprobaste que es correcto.

🚧 Qué suele fallar

- Integrar sin haber probado cada módulo por separado.
- Interfaces mal acordadas (anchos, polaridad activo-alto/bajo, qué unidad lleva cada dato).
- No documentar las suposiciones.
- Defender código que no entiendes.

EJERCICIOS RESUELTOS

Exámenes resueltos

Tres entradas: empieza por aquí, practica por tema o busca un examen concreto. Todos los problemas están resueltos paso a paso y verificados.

Antes de empezar

Cada problema tiene el **enunciado completo**, una caja «Antes de empezar: qué piden y qué teoría usas» y los pasos numerados. Los pasos están plegados: intenta resolver el problema tú primero y abre un paso solo cuando te atasques. El resultado final también está plegado.

Tipo de problema	Problemas	Peso	Aparece en
Bloques MSI y displays	6 de 11		3 de 3 exámenes
Autómatas / síntesis secuencial	5 de 11		3 de 3 exámenes
Aritmética y códigos (BCD, C2)	4 de 11		3 de 3 exámenes
Combinacional (puertas, Karnaugh)	3 de 11		3 de 3 exámenes
Contadores y biestables	2 de 11		1 de 3 exámenes

Recuento propio sobre los 11 problemas de los 3 exámenes resueltos aquí (un problema puede tener varias etiquetas). Son pocos exámenes: úsalo como orientación, no como regla.

⚠️ **Procedencia**

Los exámenes son de la misma asignatura en otra titulación de la misma escuela (véase examen). Al final de esta página se dice qué problemas no están incluidos y por qué.

1 · Empieza por aquí

Los exámenes completos más recientes y más representativos. Haz uno entero en 2 horas, sin mirar la solución.

Febrero 2025 (convocatoria del 23/01/2025)

4 problemas: aritmética con display, decodificador, expendedora (Mealy) y noria

Febrero 2023 (convocatoria del 22/01/2023)

Problemas 1, 2 y 4: sensores con signo, toldo y caja fuerte. El problema 3 (contador de la

(Moore).
4 problemas

figura) no se resuelve aquí.
3 problemas

Diciembre (convocatoria sin año en el enunciado)
4 problemas: recinto con comparadores, media de cuatro números BCD, contador JK+NAND y alarma de ventanas.
4 problemas

2 · Practicar por tema

Elige el tipo de problema que más te cuesta. Verás las listas de problemas de exámenes y de las hojas, con su origen.

Combinacional (puertas, Karnaugh) (13)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Circuito de control de un toldo · Feb 2023 · P2
- P2 · Decodificador 4:16 con decodificadores 2:4 · Feb 2025 · P2
- K1 · Mayoría de tres entradas · Hoja Karnaugh · K1
- K2 · Divisor de dos números de 2 bits · Hoja Karnaugh · K2
- K4 · Multiplicador de 2 × 2 bits · Hoja Karnaugh · K4
- K5 · Cuatro máquinas en serie: estado de la producción · Hoja Karnaugh · K5
- K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos) · Hoja Karnaugh · K6
- K7 · Decodificador BCD a 7 segmentos · Hoja Karnaugh · K7
- K8 · Comparador de dos números de 2 bits · Hoja Karnaugh · K8
- K9 · Máquina de monedas con cambio · Hoja Karnaugh · K9
- K10 · Nota de 0 a 10 en un display de barras · Hoja Karnaugh · K10
- K17 · Binario de 4 bits a dos dígitos BCD · Hoja Karnaugh · K17

Bloques MSI y displays (10)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Media de cuatro números de dos cifras en BCD · Diciembre · P2
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- P1 · Dos sensores en C2: diferencia y media · Feb 2023 · P1
- P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display · Feb 2025 · P1

- P2 · Decodificador 4:16 con decodificadores 2:4 · Feb 2025 · P2
- K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos) · Hoja Karnaugh · K6
- K7 · Decodificador BCD a 7 segmentos · Hoja Karnaugh · K7
- K8 · Comparador de dos números de 2 bits · Hoja Karnaugh · K8
- K10 · Nota de 0 a 10 en un display de barras · Hoja Karnaugh · K10

Aritmética y códigos (BCD, C2) (8)

- P1 · Recinto con dos puertas (comparadores y sumadores) · Diciembre · P1
- P2 · Media de cuatro números de dos cifras en BCD · Diciembre · P2
- P1 · Dos sensores en C2: diferencia y media · Feb 2023 · P1
- P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display · Feb 2025 · P1
- A9 · Sumador serie · Hoja autómatas · A9
- K2 · Divisor de dos números de 2 bits · Hoja Karnaugh · K2
- K4 · Multiplicador de 2 × 2 bits · Hoja Karnaugh · K4
- K17 · Binario de 4 bits a dos dígitos BCD · Hoja Karnaugh · K17

Autómatas / síntesis secuencial (13)

- P3 · Contador síncrono reversible con JK y NAND · Diciembre · P3
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- P4 · Caja fuerte: secuencia A, A, B, A (Mealy con JK) · Feb 2023 · P4
- P3 · Expendedora de tabaco (Mealy, biestables T y JK) · Feb 2025 · P3
- P4 · Noria con dos motores (Moore, biestables JK) · Feb 2025 · P4
- A1 · Tres o más unos seguidos · Hoja autómatas · A1
- A2 · Tres o más ceros seguidos · Hoja autómatas · A2
- A3 · Detector de la secuencia 1011 (con solapamiento) · Hoja autómatas · A3
- A5a · Detector de cuatro unos (1111) · Hoja autómatas · A5a
- A5b · Detector de la secuencia 11011 · Hoja autómatas · A5b
- A7 · Detector de 1010 o de 110 · Hoja autómatas · A7
- A9 · Sumador serie · Hoja autómatas · A9
- A11 · Contador reversible módulo 4 · Hoja autómatas · A11

Contadores y biestables (10)

- P3 · Contador síncrono reversible con JK y NAND · Diciembre · P3
- P4 · Alarma de ventanas con barrido secuencial · Diciembre · P4
- A1 · Tres o más unos seguidos · Hoja autómatas · A1
- A2 · Tres o más ceros seguidos · Hoja autómatas · A2

- A3 · Detector de la secuencia 1011 (con solapamiento) · Hoja autómatas · A3
- A5a · Detector de cuatro unos (1111) · Hoja autómatas · A5a
- A5b · Detector de la secuencia 11011 · Hoja autómatas · A5b
- A7 · Detector de 1010 o de 110 · Hoja autómatas · A7
- A9 · Sumador serie · Hoja autómatas · A9
- A11 · Contador reversible módulo 4 · Hoja autómatas · A11

3 · Buscar un examen

Filtra por tipo de problema o escribe una palabra. Los exámenes están plegados y ordenados por curso.

Sin año

Diciembre (convocatoria sin año en el enunciado)

4 problemas: recinto con comparadores, media de cuatro números BCD, contador JK+NAND y alarma de ventanas.

`Diciembre · P1` **P1 · Recinto con dos puertas (comparadores y sumadores)**

✔ verificado

Para entrar en un recinto hay que pasar por dos puertas, P1 y P2. Para entrar por P1 hay que introducir un dígito BCD mayor o igual a 6 y para pasar por la puerta P2 hay que introducir un número que esté tres unidades por arriba o por debajo de la mitad del número introducido en la puerta P1. Por ejemplo, si en la puerta P1 introducimos el 8, entramos por P1 y deberemos introducir en P2 un número entre el 1 $[(8/2) - 3]$ y el 7 $[(8/2) + 3]$. Diseñar el circuito lógico correspondiente.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un circuito combinacional con dos entradas BCD (P_1, P_2) y dos salidas «abre P1» y «abre P2». **Teoría:** comparadores y sumadores (T5), dividir entre 2 como desplazamiento (T2), C2 para el -3 (T2).

1 **Abrir P1**

$P_1 \geq 6$ para $P_1 \in \{0, \ldots, 9\}$ es $P_1 > 5$. Se usa un comparador de 4 bits con $B = 0101$ y la salida $A > B$.

2 La mitad de P_1

Dividir entre 2 es **desplazar a la derecha**: se toman los bits $P_1(3{:}1)$ y se completan con un 0 en el MSB. El bit menos significativo se pierde (mitad entera): $9/2 \rightarrow 4$.

3 Los límites $h+3$ y $h-3$

Con $h = P_1/2$ (3 bits, ampliado a 4): un sumador de 4 bits con la constante 0011 da $h+3$. Otro con la constante 1101 (que es -3 en C2 de 4 bits) da $h-3$. Como $P_1 \geq 6$ implica $h \geq 3$, el resultado $h-3$ nunca es negativo y se puede comparar **sin signo**. Se ha comprobado con las 40 combinaciones válidas de (P_1, P_2).

4 Abrir P2

Dos comparadores de 4 bits: $P_2 \leq h+3$ (salidas $A<B$ o $A=B$ unidas con una OR) y $P_2 \geq h-3$ ($A>B$ o $A=B$). «Abre P2» es la AND de ambos con «abre P1».

5 Tabla de valores permitidos

P_1	$h = P_1/2$	P_2 permitido
6	3	0 a 6
7	3	0 a 6
8	4	1 a 7
9	4	1 a 7

El caso $P_1 = 8$ coincide con el ejemplo del enunciado (1 a 7).

✅ Resultado final

Abre P1 $= (P_1 > 0101)$; Abre P2 $=$ Abre P1 $\cdot (P_2 \leq h+3) \cdot (P_2 \geq h-3)$ con $h = P_1(3{:}1)$. 2 comparadores de 4 bits para P2, 1 para P1 y 2 sumadores de 4 bits.

Diciembre · P2 — P2 · Media de cuatro números de dos cifras en BCD

✔ verificado

Realice el diseño de un circuito que haga la media de cuatro números de dos cifras dado en BCD.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar los cuatro números de BCD a binario, sumarlos, dividir entre 4 y presentar el resultado. **Teoría:** BCD ↔ binario (T2), sumadores en cascada (T5), dividir entre 2^k como desplazamiento.

1 De BCD a binario

Un número de dos cifras $d_1 d_0$ vale $10d_1 + d_0 = 8d_1 + 2d_1 + d_0$. Multiplicar por 8 y por 2 son **desplazamientos** (cableado), así que basta un sumador: $(d_1 \ll 3) + (d_1 \ll 1) + d_0$. Cada número sale en binario de 7 bits (máximo $99 < 128$).

2 Sumar los cuatro

La suma máxima es $4 \cdot 99 = 396$. Como $2^8 = 256 < 396 < 512 = 2^9$, hacen falta **9 bits**. Se suman con tres sumadores (o dos niveles: $A + B$, $C + D$, y la suma de ambos).

3 Dividir entre 4

Dividir entre 4 es **descartar los dos bits menos significativos** de la suma de 9 bits. El resultado tiene 7 bits ($396/4 = 99$). Es la mitad entera de la mitad entera, calculada **una sola vez**: $\lfloor (A + B + C + D)/4 \rfloor$.

4 Presentar en BCD

Para mostrar la media en dos displays hay que volver de binario a BCD (decenas y unidades). Se hace con comparadores y restas de 10, o con una ROM de 128 posiciones (ver T7).

🛠️ Ojo: la cascada de medias pierde precisión

Una solución típica hace $((A + B)/2 + (C + D)/2)/2$ con dos desplazamientos. **No es equivalente**: cada desplazamiento descarta un bit que el siguiente necesitaba. Con $A = 0, B = 1, C = 1, D = 2$ la media entera correcta es $\lfloor 4/4 \rfloor = 1$, pero la cascada da $(0 + 1) \gg 1 = 0$. Sobre las $199 \times 199 = 39\,601$ combinaciones de $(A + B,\ C + D)$, la cascada falla en $4\,900$ (un 12.4 %), con error máximo de 1 unidad. Para una media exacta: suma primero **todo** y desplaza **una vez**.

✅ Resultado final

BCD→binario con $8d_1 + 2d_1 + d_0$; suma de los cuatro en 9 bits; media = suma$(8{:}2)$ (7 bits).

La versión en cascada de desplazamientos es incorrecta en 12.4 % de los casos.

Diciembre · P3

P3 · Contador síncrono reversible con JK y NAND

✔ verificado

Utilizando biestables J-K y puertas NAND, diseñar un contador síncrono que posea una entrada de control X y que cumpla: (a) si $X = 0$, el contador funciona en modo ascendente con la secuencia $0 \to 2 \to 4 \to 6 \to 1 \to 3 \to 5 \to 7\ [\to 0 \ldots]$; (b) si $X = 1$, funciona en modo descendente, siendo la secuencia de conteo $0 \to 7 \to 5 \to 3 \to 1 \to 6 \to 4 \to 2\ [\to 0 \ldots]$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador de 3 bits con una secuencia no binaria y dos sentidos, con JK y solo NAND. **Teoría:** diseño de contadores como autómata (T8, T10); tabla de excitación del JK (T6); NAND universal (T3).

1 Estados y entradas

8 estados (000 a 111), 3 biestables, una entrada X. Cada estado tiene dos sucesores: el siguiente en la lista (si $X = 0$) o el anterior (si $X = 1$). Es la misma secuencia recorrida en uno u otro sentido. La descendente enunciada es justo la ascendente al revés: $0, 7, 5, 3, 1, 6, 4, 2$ ✓.

2 Tabla de transiciones

$Q_2Q_1Q_0$	$X = 0$	$X = 1$
000	010	111
001	011	110
010	100	000
011	101	001
100	110	010
101	111	011
110	001	100
111	000	101

3 Ecuaciones con la tabla de excitación del JK

Se rellena J y K de cada biestable según $Q \to Q^+$ y se simplifica con Karnaugh de 4 variables (Q_2, Q_1, Q_0, X):

$$J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X} \qquad J_1 = K_1 = 1$$

$$J_0 = K_0 = \overline{Q_2 Q_1}X + Q_2 Q_1 \overline{X}$$

Intuición: Q_1 conmuta en cada ciclo (en ambas secuencias alterna 0,1,0,1...), por eso $J_1 = K_1 = 1$. Q_2 conmuta cuando $Q_1 \neq X$.

4 Pasarlo a NAND

Cada suma de productos se implementa en **dos niveles de NAND** (NAND-NAND). Para $J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X}$: $J_2 = \mathrm{NAND}\big(\mathrm{NAND}(\overline{Q_1}, X),\ \mathrm{NAND}(Q_1, \overline{X})\big)$. Las variables negadas salen de la salida $\overline{Q}$ de los biestables y de una NAND con las dos entradas unidas para $\overline{X}$. $J_1 = K_1$ se conectan a nivel alto.

5 Comprobación

Las ecuaciones minimizadas, simuladas a nivel de puerta, reproducen la tabla en los 16 casos (Q, X) y generan las secuencias ascendente [0, 2, 4, 6, 1, 3, 5, 7] y descendente [0, 7, 5, 3, 1, 6, 4, 2].

✅ Resultado final

$J_2 = K_2 = \overline{Q_1}X + Q_1\overline{X}$; $J_1 = K_1 = 1$; $J_0 = K_0 = \overline{Q_2 Q_1}X + Q_2 Q_1 \overline{X}$. Todo con NAND en dos niveles.

Diciembre · P4

P4 · Alarma de ventanas con barrido secuencial

✔ verificado

Un edificio de cuatro plantas tiene cuatro ventanas por planta. En cada ventana se coloca un sensor que detecta la apertura de la misma (0 cerrada, 1 abierta). Realice el diseño de un sistema de alarma que funcione así: dos displays (planta y número de ventana vigilada); un LED junto a los displays que se enciende si la ventana indicada está abierta; barrido secuencial cíclico, desde planta 1-ventana 1 hasta planta 4-ventana 4, de frecuencia 1 o 2 segundos seleccionable con un interruptor; en el momento en que se encuentre una ventana abierta el sistema se detendrá hasta que la ventana en cuestión se cierre.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador que recorra las 16 ventanas, un selector que mire el sensor de la ventana actual y que **congele** el contador si está abierta. **Teoría:** contador binario (T8), multiplexor 16:1 (T5), sumador para mostrar «1 a 4» en lugar de «0 a 3», decodificador BCD a 7 segmentos (T5).

1 Contador de ventanas

Un contador binario de 4 bits $Q_3Q_2Q_1Q_0$. Los dos bits altos Q_3Q_2 son la **planta** (0 a 3) y los dos bajos Q_1Q_0 la **ventana** (0 a 3). Cuenta 16 estados y vuelve a 0: barrido cíclico.

2 Mirar el sensor de la ventana actual

Un multiplexor 16:1 con los 16 sensores en las entradas de datos y $Q_3Q_2Q_1Q_0$ en la selección da la señal $ABIERTA$ del sensor vigilado. Esa misma señal enciende el LED.

3 Detener el barrido

El contador **solo avanza si** $ABIERTA = 0$: se usa $ABIERTA$ negada como habilitación del contador (o se hace AND de la señal de reloj con $\overline{ABIERTA}$). Mientras la ventana siga abierta, la selección no cambia y el LED sigue encendido. En cuanto se cierra, el barrido continúa. Mejor usar una **habilitación** que cortar el reloj: así no se generan pulsos cortos.

4 Frecuencia seleccionable

Se parte de un reloj de 1 Hz (periodo 1 s). Un biestable T con $T = 1$ lo divide por 2 (periodo 2 s). Un multiplexor 2:1 controlado por el interruptor elige uno de los dos relojes.

5 Displays

Planta $= Q_3Q_2 + 1$ y ventana $= Q_1Q_0 + 1$ (valores de 1 a 4): un sumador de 2 bits con la constante 01 para cada uno, y un decodificador BCD a 7 segmentos para cada display.

6 Comprobación

Con la ventana 5 (planta 2, ventana 2) abierta durante 10 ciclos, el contador se queda en ella los 10 ciclos y luego pasa a la siguiente. Los valores mostrados están siempre entre 1 y 4.

✅ Resultado final

Contador de 4 bits (planta Q_3Q_2, ventana Q_1Q_0) + MUX 16:1 + habilitación $\overline{ABIERTA}$ + selector de reloj (1 Hz o 0,5 Hz) + dos sumadores «+1» y dos decodificadores 7 segmentos.

2024-25

Febrero 2025 (convocatoria del 23/01/2025)

4 problemas: aritmética con display, decodificador, expendedora (Mealy) y noria (Moore).

Feb 2025 · P1 **P1 · Función $(A \cdot B) + C$ con números de 3 bits en C2 y display**

✔ verificado

Realice el diseño de un circuito que realice la función $(A \cdot B) + C$, siendo A, B y C números de 2 bits de valor más 1 de signo, es decir, si el bit de signo es 1 el número es negativo y está representado en complemento a 2. El resultado deberá visualizarse en los displays de 7 segmentos que consideres necesarios.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un circuito combinacional con entradas de 3 bits en C2 (-4 a 3) que multiplique, sume y muestre el resultado con signo en decimal. **Teoría que usas:** C2 y su rango (T2), sumadores y multiplicadores (T5), BCD y display de 7 segmentos (T5). Idea clave: multiplicar **valores absolutos** (más fácil: sin signo) y corregir el signo al final.

1 Rango del resultado (decide cuántos bits y cuántos displays)

$A, B \in [-4, 3]$, luego $A \cdot B \in [-12, 16]$ y $A \cdot B + C \in [-16, 19]$. El módulo llega a 19: **dos dígitos** (decenas 0 o 1, y unidades) más un indicador de signo. Cabe en 6 bits en C2 ($[-32, 31]$). Rango comprobado por código.

2 Valor absoluto de A y de B

Con s el bit de signo: $|X| = X \oplus s$ (XOR de cada bit con s) **más** s (acarreo de entrada de un sumador). Es el «invertir y sumar 1» del C2, que solo se aplica si es negativo. Resultado de 3 bits sin signo (el $-4 = 100$ da $|-4| = 4 = 100$).

3 Producto sin signo

Un multiplicador 3×3 de AND y sumadores da $P = |A| \cdot |B| \leq 4 \cdot 4 = 16$: **5 bits**.

4 Signo del producto y paso a C2

El producto es negativo si los signos son distintos: $s_P = s_A \oplus s_B$. Se amplía P a 6 bits y se convierte a C2 con el mismo truco XOR + s_P como acarreo.

5 Sumar C

Se extiende el signo de C a 6 bits (se repite su MSB) y se suma con un sumador de 6 bits: $R = P_{C2} + C$.

6 Presentar con signo y en decimal

El signo de R es su bit 5: va a un segmento «−» (o a un LED). El módulo $M = |R|$ se obtiene otra vez con XOR + s_R. Para pasar a BCD: **decenas** $= 1$ si $M \geq 10$ (comparador); **unidades** $= M - 10$ si hay decenas, es decir $M + 6$ quedándose con 4 bits. Dos decodificadores BCD a 7 segmentos (T5).

7 Comprobación

El diseño se ha simulado con las $8^3 = 512$ combinaciones y siempre coincide con $A \cdot B + C$ en signo, decenas y unidades.

✅ **Resultado final**

Valor absoluto (XOR + sumador) → multiplicador 3×3 → signo $s_A \oplus s_B$ y paso a C2 en 6 bits → suma de C extendido → signo y módulo → BCD (decenas $= M \geq 10$, unidades $= M + 6$ módulo 16) → 2 displays y un segmento de signo. Rango $[-16, 19]$.

Feb 2025 · P2

P2 · Decodificador 4:16 con decodificadores 2:4

✔ verificado

Realizar un decodificador de 4 entradas y 16 salidas a partir de decodificadores de 2 entradas y 4 salidas (con entrada de habilitación E activa a 1; si $E = 0$ todas las salidas valen 0).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: construir un 4:16 con piezas 2:4. **Teoría:** cada salida de un decodificador es un minitérmino; la entrada E permite **habilitar** un decodificador sin tocar los demás (T5).

1 Cuántas piezas hacen falta

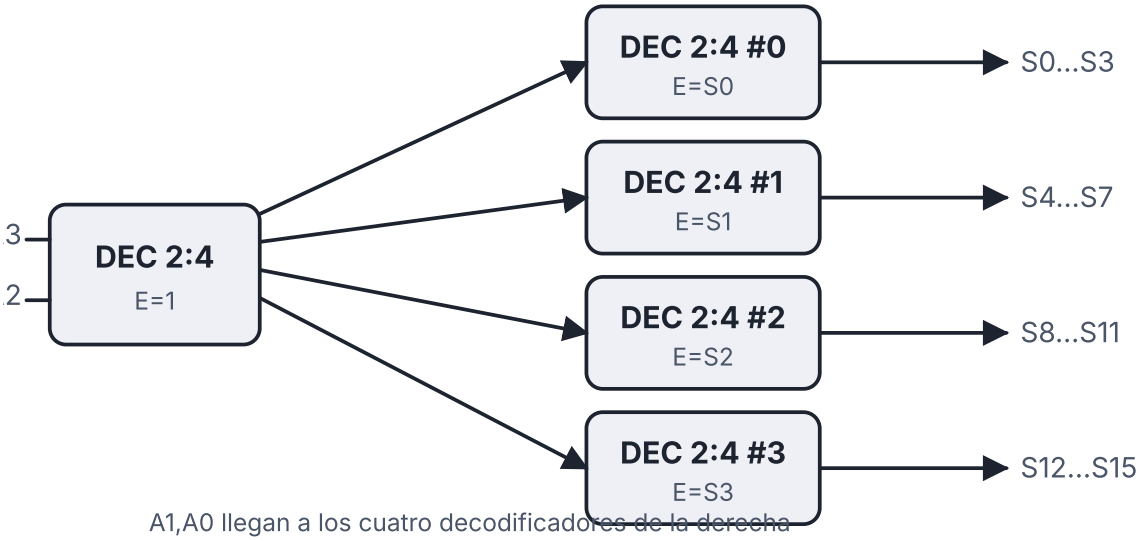
Son 16 salidas y cada 2:4 da 4: $16/4 = 4$ decodificadores para las salidas, más **1** para decidir cuál de ellos trabaja: 5 en total.

2 Reparto de entradas

Los bits altos $A_3 A_2$ entran en el decodificador de arriba (con $E = 1$ siempre). Sus cuatro salidas $S_0 \ldots S_3$ van a la entrada E de cada uno de los cuatro decodificadores de abajo. Los bits bajos $A_1 A_0$ entran en **los cuatro** a la vez.

3 Por qué funciona

Para $A = 1011_2 = 11$: arriba $A_3 A_2 = 10$ activa S_2, que habilita al tercer decodificador; este recibe $A_1 A_0 = 11$ y activa su salida 3. La salida activa es $4 \cdot 2 + 3 = 11$ ✓.



Esquema de la solución.

✅ **Resultado final**

Cinco decodificadores 2:4. Salida $4k + j$ activa $\iff$ $A_3A_2 = k$ y $A_1A_0 = j$. Verificado para las 16 entradas.

Feb 2025 · P3

P3 · Expendedora de tabaco (Mealy, biestables T y JK)

✔ verificado

Una máquina expendedora de tabaco está controlada por un circuito secuencial. Recibe como entradas el código de dos bits de las monedas que se introducen (M_1M_0: 00 no hay moneda, 01 moneda de 50 cts, 10 moneda de 1 €, 11 error) y activa dos señales: si se debe entregar el tabaco (T) y la cantidad de cambio a reintegrar (C_0: 00 no cambio y no tabaco; 01 no cambio y da tabaco; 10 no procede; 11 da cambio y da tabaco). El usuario introduce monedas hasta alcanzar 2 €, que es el precio. Al alcanzar o superar el importe, la máquina sirve el tabaco y devuelve el cambio. Diseña el circuito (Mealy o Moore) indicando: número y codificación de estados, diagrama de estados, tabla de transiciones y salidas con un biestable T para el bit más significativo del estado y JK para el resto, e implementación con mapas de Karnaugh.

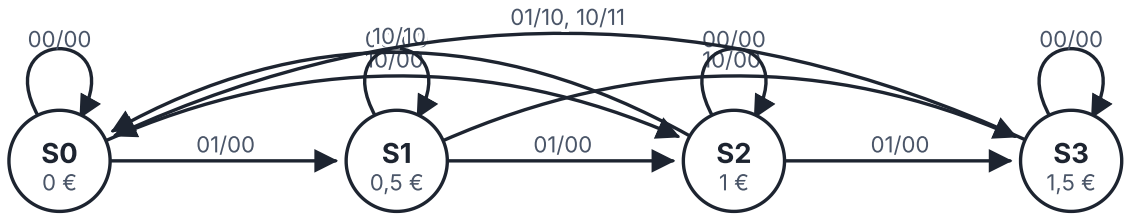
🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que acumule dinero y dispare T (y C_0) al llegar a 2 €. **Teoría:** método de diseño de autómatas (T10): estados → diagrama → tabla → tabla de excitación de T y JK (T6) → Karnaugh (T3).

> ⚠️ **Ojo con el enunciado**
>
> El ejemplo del enunciado habla de «monedas de 25», pero la Tabla 1 solo tiene monedas de **50 cts y 1 €**. Aquí se resuelve con esas dos monedas: el cambio posible es siempre una moneda de 50 cts (si el total llega a 2,5 €). El código 11 de entrada (error) se trata como «sin moneda».

1 Estados

Lo único que hay que recordar es **cuánto dinero hay**: 0 €, 0,5 €, 1 € o 1,5 € (al llegar a 2 € se sirve y se vuelve a 0). **4 estados**, 2 biestables: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$ (el código es el dinero en medios euros). Es Mealy: la salida T, C_0 depende del estado **y de la moneda** que cae.



Etiquetas: M1M0 / T C0

Al llegar a 2 € (T=1) se vuelve a S0. Con 1,5 € y moneda de 1 € se pasa a 2,5 € y se devuelve 50 cts (C0=1).

2 Tabla de transiciones

Q_1Q_0	M_1M_0	$Q_1^+Q_0^+$	T	C_0
00	00/11	00	0	0
00	01	01	0	0
00	10	10	0	0
01	00/11	01	0	0
01	01	10	0	0

Q_1Q_0	M_1M_0	$Q_1^+Q_0^+$	T	C_0
01	10	11	0	0
10	00/11	10	0	0
10	01	11	0	0
10	10	00	1	0
11	00/11	11	0	0
11	01	00	1	0
11	10	00	1	1

3 Tabla de excitación

Para el biestable T del bit alto: $T_1 = Q_1 \oplus Q_1^+$ (1 si cambia). Para el JK del bit bajo se usa la tabla de excitación de T6: $0 \to 0$: $J = 0, K = X$; $0 \to 1$: $J = 1, K = X$; $1 \to 0$: $J = X, K = 1$; $1 \to 1$: $J = X, K = 0$.

4 Ecuaciones (Karnaugh de 4 variables Q_1, Q_0, M_1, M_0)

$$T_1 = M_1\overline{M_0} + Q_0\overline{M_1}M_0$$

$$J_0 = \overline{M_1}M_0 \qquad K_0 = \overline{M_1}M_0 + Q_1M_1\overline{M_0}$$

$$T = Q_1M_1\overline{M_0} + Q_1Q_0\overline{M_1}M_0 \qquad C_0 = Q_1Q_0M_1\overline{M_0}$$

Se simplifica cada una con un mapa de Karnaugh de 4 variables, con las filas $M_1M_0 = 11$ tratadas como moneda inexistente.

5 Comprobación con el ejemplo

Tres monedas: 50 cts, 50 cts, 1 €. Estados: $S_1 \to S_2 \to S_0$; en la última, $T = 1$, $C_0 = 0$ (2 € justos, sin cambio). **Cuatro monedas: 50, 50, 50, 1 €.** $S_1 \to S_2 \to S_3 \to S_0$ y en la última, $T = 1$ y $C_0 = 1$ (2,5 € ⇒ 50 cts de cambio).

✅ **Resultado final**

4 estados (dinero acumulado), 2 biestables. $T_1 = M_1\overline{M_0} + Q_0\overline{M_1}M_0$; $J_0 = \overline{M_1}M_0$, $K_0 = \overline{M_1}M_0 + Q_1M_1\overline{M_0}$; $T = Q_1M_1\overline{M_0} + Q_1Q_0\overline{M_1}M_0$; $C_0 = Q_1Q_0M_1\overline{M_0}$. Verificado con todas las secuencias de hasta 7 monedas contra una especificación aritmética independiente.

Feb 2025 · P4

P4 · Noria con dos motores (Moore, biestables JK)

✔ verificado

El mecanismo de una noria de un parque de atracciones dispone de dos motores que permiten que la noria gire en un sentido o en el inverso. Estos dos motores se gobiernan por medio de dos señales (M_A y M_B) de tal manera que cuando una de ellas se pone a «1» se pone en funcionamiento el motor correspondiente. El empleado de controlar la atracción dispone de dos pulsadores (A y B), de manera que al pulsar uno de ellos se acciona el motor correspondiente. Si se acciona dos veces seguidas el mismo pulsador, la noria se detiene. Además, en el puesto de control central se dispone de otro pulsador (P) que en caso de ser accionado detiene la noria. (Los pulsadores están normalmente a nivel bajo, pasando a nivel alto cuando se accionan.) Diseña el autómata.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que recuerde qué motor está en marcha y si el último pulsador fue el mismo. **Teoría:** Moore (salida solo del estado) y diseño con JK (T10, T6). Hay que distinguir «sigo con el dedo en el botón» de «he vuelto a pulsar».

⚠️ **Suposiciones que hay que declarar**

- A y B no se pulsan a la vez ($AB = 11$ no ocurre: indiferente).
- P tiene prioridad sobre todo: si $P = 1$ se va a parada.
- «Dos veces seguidas el mismo pulsador» = soltar y volver a pulsar el mismo. Mantener pulsado **no** cuenta como dos pulsaciones.
- Pulsar el **otro** botón cambia el sentido sin pararse.

1 Estados (6) y codificación

Estado	Código $Q_2Q_1Q_0$	Significado	M_A	M_B
S_0	000	Parada	0	0

Estado	Código $Q_2Q_1Q_0$	Significado	M_A	M_B
S_1	001	Gira A, botón A aún pulsado	1	0
S_2	010	Gira A, botón soltado (otra pulsación de A parará)	1	0
S_3	011	Gira B, botón B aún pulsado	0	1
S_4	100	Gira B, botón soltado	0	1
S_5	101	Parada, esperando a soltar el botón	0	0

Los códigos 110 y 111 no se usan (indiferentes). Necesitas 3 biestables: $2^2 < 6 \leq 2^3$.

2 Transiciones (entradas A, B, P)

- S_0: $P \to S_0$; $A \to S_1$; $B \to S_3$; nada $\to S_0$.
- S_1: $P \to S_0$; $A \to S_1$ (sigue pulsado); $B \to S_3$; nada $\to S_2$.
- S_2: $P \to S_0$; $A \to S_5$ (segunda pulsación de A: para); $B \to S_3$; nada $\to S_2$.
- S_3: $P \to S_0$; $B \to S_3$; $A \to S_1$; nada $\to S_4$.
- S_4: $P \to S_0$; $B \to S_5$ (para); $A \to S_1$; nada $\to S_4$.
- S_5: si nada y $P = 0 \to S_0$; si no, se queda en S_5 (evita que mantener el botón rearranque).

3 Ecuaciones de entrada de los JK (Karnaugh de 6 variables)

$$J_2 = Q_1\overline{Q_0}A\overline{P} + Q_1Q_0\overline{ABP} \qquad K_2 = \overline{Q_0}P + \overline{Q_0}A + Q_0\overline{ABP}$$

$$J_1 = \overline{Q_2}B\overline{P} + \overline{Q_2}Q_0\overline{AP} \qquad K_1 = P + A + Q_0\overline{B}$$

$$J_0 = B\overline{P} + A\overline{P} \qquad K_0 = \overline{ABP} + \overline{Q_2}P$$

Cada una sale de aplicar la tabla de excitación del JK a las 6 variables Q_2, Q_1, Q_0, A, B, P con indiferentes en los estados 6 y 7 y en $AB = 11$.

4 Salidas (Moore)

$$M_A = \overline{Q_2 Q_1} Q_0 + Q_1 \overline{Q_0} \qquad M_B = Q_1 Q_0 + Q_2 \overline{Q_0}$$

Dependen solo de Q: $M_A = 1$ en S_1, S_2 y $M_B = 1$ en S_3, S_4.

5 Comprobación con escenarios

Pulsación larga de A: gira A y sigue. A, soltar, A: para. A, soltar, B: cambia a giro B. B y luego P: para. A, A (para), A de nuevo: arranca otra vez. Todos coinciden con el circuito de puertas.

✅ **Resultado final**

6 estados, 3 JK, Moore. $M_A = \overline{Q_2 Q_1} Q_0 + Q_1 \overline{Q_0}$, $M_B = Q_1 Q_0 + Q_2 \overline{Q_0}$. Ecuaciones de entrada: ver pasos. Comprobado con las ecuaciones minimizadas simuladas a nivel de puerta frente a la tabla.

2022-23

Febrero 2023 (convocatoria del 22/01/2023)

Problemas 1, 2 y 4: sensores con signo, toldo y caja fuerte. El problema 3 (contador de la figura) no se resuelve aquí.

`Feb 2023 · P1` **P1 · Dos sensores en C2: diferencia y media** ✔ verificado

Un dispositivo industrial utiliza dos sensores para la medida. Estos sensores son idénticos y están midiendo la misma variable, aunque su medida puede diferir. Las dos medidas se entregan a un sistema digital codificadas cada una con 4 bits en complemento a 2 (A_3A_0 para el sensor 1 y B_3B_0 para el sensor 2; A_3 y B_3 son los bits de signo). Para visualizar la medida se dispone de dos displays de 7 segmentos. En uno de ellos se visualizará la diferencia de valores y en el otro la media de las dos medidas. El signo se representará mediante un led. Diseñar mediante bloques combinacionales estándares (multiplexores, decodificadores, sumadores, comparadores...) el esquema del circuito combinacional.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: restar y promediar dos números en C2, mostrar el módulo en un display y el signo en un LED. **Teoría:** C2, extensión de signo y desbordamiento (T2); sumador/restador y valor absoluto (T5).

1 Rango de la diferencia

Cada medida va de -8 a 7. La diferencia $A - B$ va de -15 a 15: **5 bits** en C2. Para no desbordar, primero se **extiende el signo** de A y B a 5 bits (se repite el MSB).

2 Restar con un sumador

$A - B = A + \overline{B} + 1$: un sumador de 5 bits, con B negada y $C_{in} = 1$. El bit 4 del resultado es el **signo**: va al LED.

3 Módulo de la diferencia

$|D| = D \oplus s$ más s como C_{in} de un segundo sumador. El módulo va de 0 a 15 (4 bits).

4 Media

Se suman $A + B$ en 5 bits (rango -16 a 14) y se divide entre 2 con un **desplazamiento aritmético** (se descarta el LSB y se repite el signo). La media va de -8 a 7. El signo (otro LED o el mismo) es el MSB del desplazado.

5 Display

Un módulo de 0 a 15 en un solo display solo se puede mostrar en **hexadecimal** (0–F). Si se quiere decimal hace falta un segundo dígito. Para cada uno: decodificador a 7 segmentos.

⚠️ Redondeo de la media

El desplazamiento aritmético da $\lfloor (A + B)/2 \rfloor$ (redondea hacia $-\infty$: $-1/2 \to -1$). Truncar hacia cero daría 0. Ambas son válidas, pero **hay que decir cuál** se usa. Coinciden salvo en 72 de las 256 combinaciones (suma impar y negativa).

✅ Resultado final

Diferencia: sumador de 5 bits con B invertida y $C_{in} = 1$; signo = bit 4; módulo = XOR + sumador. Media: $A + B$ en 5 bits desplazado 1 a la derecha. Verificado con las 256 combinaciones.

Feb 2023 · P2

P2 · Circuito de control de un toldo

✔ verificado

Se desea realizar un circuito de control para el toldo de una terraza. El toldo tiene la función tanto de dar sombra como de proteger del viento y de la lluvia. Entradas: S (hay sol), L (llueve), V (hay mucho viento), F (hace frío en el interior de la casa). Salida BT: si $BT = 1$ el toldo debe estar extendido (bajado), si $BT = 0$ recogido. Características: (1) siempre que llueva se extiende el toldo para evitar que moje la terraza (no se considera posible que simultáneamente llueva y haga sol); (2) si hace viento se debe extender el toldo, salvo si el día está soleado y hace frío en la casa: entonces se recoge para que el sol caliente la casa; (3) si no hace viento ni llueve, solo se baja el toldo en los días de sol y cuando haga calor en el interior. Se pide: 1. Tabla de verdad de $BT(S, L, V, F)$. 2. Expresión reducida en suma de productos y en producto de sumas. 3. Esquema en puertas. 4. Variante con un interruptor manual/automático: en automático funciona como se ha descrito; en manual, un pulsador da la orden de recoger y otro de bajar, y el circuito debe mantener la orden tras soltarlos; si se pulsan ambos a la vez, el comportamiento debe ser predecible.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar un texto a tabla de verdad, simplificar (SOP y POS) y añadir memoria en el modo manual. **Teoría:** tabla de verdad y Karnaugh con indiferentes (T3); biestables (T6).

1 Traducir el texto a reglas

- Llueve ($L = 1$) $\Rightarrow$ $BT = 1$.
- Viento ($V = 1$) $\Rightarrow$ $BT = 1$, **salvo** $S = 1$ y $F = 1$ ($BT = 0$).
- Sin lluvia ni viento: $BT = 1$ solo si $S = 1$ y $F = 0$.
- $S = L = 1$ **no ocurre**: son 4 combinaciones indiferentes.

2 Tabla de verdad (orden S, L, V, F)

$BT = 1$ en las filas $2, 3, 4, 5, 6, 7, 8, 10$ (con $S = L = 1$, filas 12 a 15, indiferentes). En el resto, $BT = 0$.

BT con indiferentes

Mapa de Karnaugh con las filas $S = L = 1$ como indiferentes.

3 Suma de productos

Tres grupos: L (cuatro casillas con indiferentes), $\overline{S}V$ y $S\overline{F}$:

$$BT = \overline{S}V + L + S\overline{F}$$

4 Producto de sumas

Se agrupan los ceros y se niega: $BT = (S + L + V)(\overline{S} + \overline{F})$. Comprobación: ambas coinciden en las 12 filas posibles.

5 Modo manual con memoria

En manual hay que **recordar** la orden tras soltar los pulsadores: un biestable Q. Sea U «recoger» y D «bajar». $Q^+ = \overline{U}\,(D + Q)$: U tiene prioridad (si se pulsan ambos, se recoge: comportamiento predecible y **seguro**), D pone $Q = 1$ y se mantiene hasta que se pulse U. La salida es $BT = M \cdot Q + \overline{M} \cdot BT_{auto}$ (un multiplexor 2:1 controlado por M).

✅ **Resultado final**

$BT = \overline{S}V + L + S\overline{F} = (S + L + V)(\overline{S} + \overline{F})$. Manual: biestable con $Q^+ = \overline{U}(D + Q)$ y un MUX para elegir manual o automático. Verificado con las 12 combinaciones posibles.

Feb 2023 · P4

P4 · Caja fuerte: secuencia A, A, B, A (Mealy con JK)

✔ verificado

El sistema de apertura de una caja fuerte está compuesto por dos teclas A y B, un circuito secuencial a diseñar y un temporizador que mantiene la caja fuerte abierta durante 5 minutos cuando recibe un nivel lógico 1 desde el circuito secuencial. Este temporizador vuelve a cerrar la caja fuerte pasado dicho tiempo. Mientras no se pulse ninguna tecla no se genera ningún nivel lógico de entrada al circuito secuencial. Para abrir la caja fuerte, la combinación secreta es: pulsar dos veces seguidas la tecla A, a continuación pulsar una vez la tecla B, y finalmente pulsar una vez la tecla A. Si se hace de esta manera, el circuito secuencial dará una salida a nivel lógico 1 que actuará sobre el temporizador. Si en cualquier momento se introdujera un error al pulsar la secuencia secreta, en el siguiente ciclo de reloj todos los biestables se pondrán a cero (el sistema pasará al estado inicial) y la secuencia debe volver a introducirse desde el principio. 1. Dibujar el diagrama de estados, explicando claramente en qué consiste cada estado. 2. Implementar el circuito secuencial usando biestables JK y las puertas necesarias.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un detector de la secuencia A-A-B-A, con las pulsaciones separadas, que reinicia ante cualquier error. **Teoría:** diseño de autómatas (T10), tabla de excitación del JK (T6), Karnaugh de 5 variables.

⚠️ **Suposiciones que hay que declarar**

El enunciado no fija cómo llegan las teclas. Se supone **dos líneas**: $a = 1$ si se pulsa A y $b = 1$ si se pulsa B ($ab = 00$: ninguna tecla; $ab = 11$ no ocurre: indiferente). «Pulsar dos veces seguidas A» = pulsar, soltar y pulsar de nuevo A (dos pulsaciones). Mantener la tecla varios ciclos cuenta como **una** pulsación. Después de abrir se espera a soltar A antes de volver a empezar.

1 Estados (8)

Estado	Significado
S_0	Reposo, esperando la primera A
S_1	A nº1 pulsada (se mantiene)
S_2	A nº1 soltada, espera A nº2
S_3	A nº2 pulsada (se mantiene)
S_4	A nº2 soltada, espera B
S_5	B pulsada (se mantiene)
S_6	B soltada, espera A nº3
S_7	A final pulsada: caja abierta, espera soltar

Son 8 estados: **3 biestables**, código $Q_2Q_1Q_0$ igual al número del estado.

2 Transiciones

- «Esperando una tecla» (S_2, S_4, S_6): sin tecla se queda; con la tecla correcta avanza; con la incorrecta, vuelve a S_0.
- «Tecla pulsada» (S_1, S_3, S_5): si se mantiene, se queda; si se suelta, avanza; si se pulsa otra, error y vuelve a S_0.
- S_0: con A va a S_1; con B va a S_0 (error).
- S_6 con A: pasa a S_7 y $Z = 1$.
- S_7: mientras A siga pulsada se queda; si no, vuelve a S_0.

3 Ecuaciones (Karnaugh de 5 variables Q_2, Q_1, Q_0, a, b con $ab = 11$ indiferente)

$$J_2 = Q_1Q_0\overline{a}\overline{b} \qquad K_2 = \overline{Q_1}a + Q_1b + Q_1Q_0\overline{a}$$

$$J_1 = Q_0\overline{a}\overline{b} \qquad K_1 = b + Q_0\overline{a}$$

$$J_0 = \overline{Q_2}a + Q_1a + Q_2\overline{Q_1}b \qquad K_0 = \overline{Q_2}\overline{a} + Q_1\overline{a} + Q_2\overline{Q_1}b$$

$$Z = Q_2 Q_1 \overline{Q_0} a$$

4 **Comprobación**

Las ecuaciones, simuladas a nivel de puerta, reproducen el diagrama en todas las transiciones y generan $Z = 1$ solo con la secuencia correcta. Se han comparado con una especificación independiente en las 9840 secuencias de entrada de hasta 8 ciclos: sin discrepancias. Si en la secuencia hay un error, no se abre; si A se mantiene dos ciclos, cuenta como una sola pulsación.

✅ **Resultado final**

8 estados, 3 JK. $Z = Q_2 Q_1 \overline{Q_0} a$. $J_2 = Q_1 Q_0 \overline{a}\,\overline{b}$, $K_2 = \overline{Q_1} a + Q_1 b + Q_1 Q_0 \overline{a}$, $J_1 = Q_0 \overline{a}\,\overline{b}$, $K_1 = b + Q_0 \overline{a}$, $J_0 = \overline{Q_2} a + Q_1 a + Q_2 \overline{Q_1} b$, $K_0 = \overline{Q_2}\,\overline{a} + Q_1 \overline{a} + Q_2 \overline{Q_1}\,\overline{b}$ (en total 14 términos y 35 literales).

Qué no se ha incluido

⚠️ **Problemas que no están resueltos aquí**

- **Febrero de 2023, problema 3** (contador de la figura): el esquema no se ha podido interpretar con seguridad y no se inventa.
- **Invernadero** (diciembre de 2024) y **cronograma con biestables T**: no se ha incluido un enunciado completo y fiable, así que no se reproducen. La teoría para resolverlos está en T3 y T6.
- Los exámenes de 2014 y 2015 no se han incluido por la misma razón.

La teoría para resolver los que faltan está en los temas indicados en cada caja.

EJERCICIOS RESUELTOS

Hojas de problemas resueltas

Diseño combinacional con Karnaugh y autómatas síncronos con biestables JK: 18 problemas resueltos paso a paso.

Dos hojas, en el mismo formato que los exámenes: enunciado, caja «Antes de empezar», pasos numerados plegados y resultado. Todas las ecuaciones se han comprobado con simulación a nivel de puerta contra una especificación independiente: las de **Karnaugh** con las tablas de verdad completas y las de **autómatas** con todas las secuencias de entrada hasta 12 bits.

⚠️ **Enunciados**

Los enunciados de las hojas están planteados en esta guía con sus condiciones explícitas (por ejemplo, qué hace cada indicador o cómo se codifican los estados). Si tu enunciado cambia alguna condición, el método es el mismo y solo cambia la tabla.

Hoja de problemas · Diseño combinacional con Karnaugh

10 problemas: mayoría, divisor, multiplicador, producción en serie, barco, decodificadores a 7 segmentos, comparador, máquina de monedas y binario a BCD.

`Hoja Karnaugh · K1` **K1 · Mayoría de tres entradas** ✔ verificado

Diseña un circuito con tres entradas S_2, S_1, S_0 cuya salida S vale 1 cuando **al menos dos** de las entradas valen 1 (circuito de mayoría, por ejemplo para un votador de tres sensores).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tabla de verdad, ecuación mínima y esquema. **Teoría:** minterms, mapa de Karnaugh de 3 variables y grupos de 2 y 4 casillas (T3).

1 **Tabla de verdad**

$S = 1$ en las combinaciones con dos o tres unos: $011, 101, 110, 111$, es decir $S = \sum m(3, 5, 6, 7)$.

2 **Mapa de Karnaugh**

S2\S1S0	00	01	11	10
0	0	1	3 **1**	2
1	4	5 **1**	7 **1**	6 **1**

Mayoría de tres

Tres grupos de dos casillas. Cada par de entradas forma un producto.

Los tres grupos son S_1S_0, S_2S_0 y S_2S_1.

3 Ecuación y puertas

$S = S_1S_0 + S_2S_0 + S_2S_1$: tres AND de dos entradas y una OR de tres. En NAND-NAND: tres NAND de dos entradas y una NAND de tres.

✅ Resultado final

$S = S_1S_0 + S_2S_0 + S_2S_1$ (verificada con las 8 combinaciones).

`Hoja Karnaugh · K2` **K2 · Divisor de dos números de 2 bits** ✔ verificado

Diseña un circuito combinacional que divida el número $A = A_1A_0$ entre $B = B_1B_0$ (ambos de 2 bits, sin signo). Salidas: cociente C_1C_0 y resto R_1R_0. El caso $B = 0$ no puede darse (indiferente).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cuatro funciones de cuatro variables con indiferentes. **Teoría:** Karnaugh de 4 variables con indiferentes (T3).

1 Qué hace el circuito

Es la división entera: $A = B \cdot C + R$ con $0 \le R < B$. Como $A \le 3$, el cociente cabe en 2 bits. Ejemplos: $3/2 \to C = 1, R = 1$; $3/1 \to C = 3, R = 0$; $2/3 \to C = 0, R = 2$.

2 Un mapa por salida

Las cuatro filas con $B_1 B_0 = 00$ son indiferentes (4 casillas «X» en cada mapa). El mapa de C_1:

A1A0\B1B0

	00	01	11	10
00	0 ×	1	3	2
01	4 ×	5	7	6
11	12 ×	13 **1**	15	14
10	8 ×	9 **1**	11	10

C1 (cociente, bit alto)

$C_1 = 1$ solo si $A \ge 2B$ con cociente ≥ 2: $A = 2$ o 3 y $B = 1$.

3 Ecuaciones

$$C_1 = A_1 \overline{B_1} \qquad C_0 = A_0 \overline{B_1} + A_1 \overline{B_0} + A_1 A_0$$

$$R_1 = A_1\overline{A_0}B_1B_0 \qquad R_0 = A_0\overline{B_0} + \overline{A_1}A_0B_1$$

✅ **Resultado final**

Las cuatro ecuaciones anteriores. Comprobadas contra la división entera en las 12 combinaciones válidas.

Hoja Karnaugh · K4

K4 · Multiplicador de 2 × 2 bits

✔ verificado

Diseña un multiplicador combinacional de dos números de 2 bits $A = A_1A_0$ y $B = B_1B_0$. El producto tiene 4 bits $S_3S_2S_1S_0$.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: cuatro funciones de cuatro variables, sin indiferentes. **Teoría:** Karnaugh de 4 variables (T3); producto binario como suma de productos parciales (T2).

1 Rango del producto

$3 \times 3 = 9$ cabe en 4 bits. Todas las combinaciones son válidas.

2 Mapas

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4	5	7 **1**	6 **1**
11	12	13 **1**	15	14 **1**
10	8	9 **1**	11 **1**	10

S1 del producto

S_1 es la función más complicada: cuatro grupos de dos casillas.

3 Ecuaciones

$$S_3 = A_1A_0B_1B_0 \qquad S_2 = A_1\overline{A_0}B_1 + A_1B_1\overline{B_0}$$

$$S_1 = \overline{A_1}A_0B_1 + A_0B_1\overline{B_0} + A_1\overline{A_0}B_0 + A_1\overline{B_1}B_0 \qquad S_0 = A_0B_0$$

$S_0 = A_0B_0$ es la AND de los bits menos significativos. $S_3 = 1$ solo con $3 \times 3 = 9$.

✅ **Resultado final**

Las cuatro ecuaciones anteriores, verificadas con las 16 combinaciones. Alternativa sin Karnaugh: cuatro AND de partida y dos semisumadores.

Hoja Karnaugh · K5

K5 · Cuatro máquinas en serie: estado de la producción

✔ verificado

Una línea de producción tiene cuatro máquinas $S_3S_2S_1S_0$ (1 = funcionando). Un panel tiene cuatro indicadores: **TA** (las cuatro funcionan), **MA** (al menos tres funcionan), **I** (funcionan exactamente dos) y **MR** (funciona una o ninguna). Diseña el circuito que los activa. Este es el reparto que se usa aquí.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cuatro funciones de cuatro variables según el **número de unos**. **Teoría:** Karnaugh de 4 variables y grupos de 2, 4 y 8 casillas (T3).

1 Contar los unos

Sea n el número de máquinas que funcionan. TA: $n = 4$. MA: $n \geq 3$. I: $n = 2$. MR: $n \leq 1$. Los indicadores MA y TA se solapan (con $n = 4$ ambos valen 1) pero I y MR no.

2 Ecuaciones

$$TA = S_3S_2S_1S_0$$

$$MA = S_2S_1S_0 + S_3S_1S_0 + S_3S_2S_0 + S_3S_2S_1$$

$$MR = \overline{S_3S_2S_1} + \overline{S_3S_2S_0} + \overline{S_3S_1S_0} + \overline{S_2S_1S_0}$$

$$I = \overline{S_3}S_2S_1S_0 + \overline{S_3}S_2\overline{S_1}S_0 + \overline{S_3}S_2S_1\overline{S_0} + S_3\overline{S_2S_1}S_0 + S_3\overline{S_2}S_1\overline{S_0} + S_3S_2\overline{S_1S_0}$$

I necesita seis productos de cuatro variables: los seis unos del mapa son todas las casillas aisladas, ninguna se puede agrupar con otra. Es el caso típico donde **no** simplifica el mapa.

⚠️ I (exactamente 2) no se simplifica

Los unos de «exactamente dos» están en las casillas de peso 2: cada una tiene como vecinas casillas de peso 1 o 3, que valen 0. Por eso ningún grupo mayor que 1 es posible. Tras Karnaugh, la ecuación sigue siendo la suma de seis minterms.

✅ Resultado final

TA, MA, MR e I con las ecuaciones anteriores. Verificadas con las 16 combinaciones.

`Hoja Karnaugh · K6`

K6 · Indicador de rumbo de un barco (4 entradas, 7 segmentos)

✔ verificado

Una brújula electrónica da cuatro señales N, S, E, O (1 = el barco apunta en esa dirección). Se muestra en un display de 7 segmentos: N enciende a; S enciende d; E enciende b, c; O enciende e, f; y las direcciones intermedias combinan las dos (NE: a, b; NO: a, f; SE: c, d; SO: d, e). No pueden valer 1 a la vez N y S, ni E y O. Si todas valen 0, no se encuentra nada (indiferente).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: siete funciones (a–g) con muchos indiferentes. **Teoría:** indiferentes en Karnaugh (T3), displays de 7 segmentos (T5).

1 Indiferentes

Las combinaciones con $N = S = 1$ o $E = O = 1$ y la combinación 0000 son indiferentes: 8 de las 16.

2 Ecuaciones

$$a = N \qquad b = \overline{S}E \qquad c = \overline{N}E \qquad d = S$$

$$e = \overline{N}O \qquad f = \overline{S}O \qquad g = 0$$

Con tantos indiferentes cada segmento queda con **un solo literal** o dos: $a = N$ (el segmento de arriba se enciende siempre que hay norte), $d = S$.

💡 **Por qué sale tan simple**

Una combinación indiferente se usa como 1 si ayuda a hacer un grupo mayor. Aquí las prohibidas ($N = S = 1$, $E = O = 1$) permiten agrupar casillas lejanas y el segmento g no se enciende nunca.

✅ **Resultado final**

$$a = N;\ b = \overline{S}E;\ c = \overline{N}E;\ d = S;\ e = \overline{N}O;\ f = \overline{S}O;\ g = 0.$$

`Hoja Karnaugh · K7`

K7 · Decodificador BCD a 7 segmentos

✔ verificado

Diseña el decodificador de un dígito BCD $ABCD$ (A es el bit de más peso) a las siete señales $a, \ldots, g$ de un display de ánodo/cátodo común activas a 1. Las combinaciones 10 a 15 no existen (indiferentes). Los dígitos se dibujan con la forma habitual: el 6 lleva el segmento a, el 7 solo a, b, c y el 9 lleva el segmento d.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: siete funciones de cuatro variables con 6 indiferentes. **Teoría:** Karnaugh con indiferentes (T3), display de 7 segmentos (T5).

1 Tabla de segmentos

Dígito	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1

Dígito	a	b	c	d	e	f	g
9	1	1	1	1	0	1	1

2 Un mapa por segmento

Segmento e

El segmento e solo se enciende en 0, 2, 6 y 8. Con los indiferentes se agrupa en dos grupos.

3 Ecuaciones

$$a = \overline{BD} + C + BD + A \qquad b = \overline{B} + \overline{CD} + CD \qquad c = \overline{C} + D + B$$

$$d = \overline{BD} + \overline{B}C + C\overline{D} + B\overline{C}D + A$$

$$e = \overline{BD} + C\overline{D} \qquad f = \overline{CD} + B\overline{C} + B\overline{D} + A \qquad g = \overline{B}C + C\overline{D} + B\overline{C} + A$$

✅ **Resultado final**

Las siete ecuaciones anteriores, comprobadas con los 10 dígitos válidos (los indiferentes no se comprueban).

`Hoja Karnaugh · K8` **K8 · Comparador de dos números de 2 bits** ✔ verificado

Diseña un comparador de $A = A_1A_0$ y $B = B_1B_0$ con tres salidas: GT ($A > B$), EQ ($A = B$) y LT ($A < B$).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tres funciones de cuatro variables. **Teoría:** comparadores y Karnaugh (T3, T5).

1 **Mapas**

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7	6
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11	10

GT: A > B

Seis casillas en escalera: tres grupos de dos.

EQ vale 1 en la diagonal: cuatro casillas que no se pueden agrupar. LT es simétrica a GT.

2 **Ecuaciones**

$$GT = A_0\overline{B_1}\,\overline{B_0} + A_1\overline{B_1} + A_1A_0\overline{B_0}$$

$$LT = \overline{A_1}\,\overline{A_0}B_0 + \overline{A_1}B_1 + \overline{A_0}B_1B_0$$

$$EQ = \overline{A_1}\,\overline{A_0}\,\overline{B_1}\,\overline{B_0} + \overline{A_1}A_0\overline{B_1}B_0 + A_1\overline{A_0}B_1\overline{B_0} + A_1A_0B_1B_0$$

💡 **Atajo para EQ**

$EQ = \overline{(A_1 \oplus B_1) + (A_0 \oplus B_0)}$: dos XOR y una NOR. Es lo mismo que la suma de cuatro minterms, con menos puertas. También se obtiene como $EQ = \overline{GT + LT}$.

✅ **Resultado final**

Las tres ecuaciones anteriores (con EQ en minterms o con XOR), verificadas con las 16 combinaciones.

`Hoja Karnaugh · K9` **K9 · Máquina de monedas con cambio** ✔ verificado

Una máquina vende un producto de 40 céntimos. Admite hasta dos monedas de 25 (E_{25A}, E_{25B}), una de 10 (E_{10}) y una de 5 (E_5): cada entrada vale 1 si esa moneda se ha introducido. La salida P entrega el producto si la suma llega a 40 o más. El cambio se devuelve con dos dispensadores de 10 (C_{10A}, C_{10B}) y uno de 5 (C_5).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: cuatro funciones; hay que calcular el cambio $= \text{suma} - 40$ y repartirlo en monedas de 10 y 5. **Teoría:** Karnaugh (T3), tabla de verdad a partir de un texto.

1 Tabla de casos relevantes

La suma máxima es 65 céntimos. El producto se entrega con suma ≥ 40.

Monedas	Suma	P	Cambio	Dispensadores
25 + 25	50	1	10	C_{10A}
25 + 25 + 10	60	1	20	C_{10A}, C_{10B}
25 + 25 + 5	55	1	15	C_{10A}, C_5
25 + 25 + 10 + 5	65	1	25	C_{10A}, C_{10B}, C_5
25 + 10 + 5	40	1	0	ninguno
25 + 10	35	0	0	ninguno

Con una sola moneda de 25 y sin la de 10 y la de 5 juntas no se llega a 40.

2 Ecuaciones

$$P = E_{25B}E_{10}E_5 + E_{25A}E_{10}E_5 + E_{25A}E_{25B}$$

$$C_{10A} = E_{25A}E_{25B} \qquad C_{10B} = E_{25A}E_{25B}E_{10} \qquad C_5 = E_{25A}E_{25B}E_5$$

El cambio solo existe cuando se han metido **las dos** de 25 (50 céntimos): por eso las tres salidas contienen $E_{25A}E_{25B}$.

✅ Resultado final

Las cuatro ecuaciones anteriores, verificadas con las 16 combinaciones (suma máxima 65 céntimos).

`Hoja Karnaugh · K10` **K10 · Nota de 0 a 10 en un display de barras** ✔ verificado

Una nota de 0 a 10 llega en binario ($N_3N_2N_1N_0$). Se visualiza en un display de 7 segmentos como una **barra que crece**: de 0 a 4 solo se enciende g (un guion: suspenso); 5 enciende a; 6, a, b; 7 y 8, a, b, c; 9, a, b, c, d; y 10 enciende $a, \ldots, f$. Las combinaciones 11 a 15 son indiferentes.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: siete funciones con 5 indiferentes. **Teoría:** Karnaugh con indiferentes (T3).

1 Ecuaciones

$$a = N_2N_0 + N_2N_1 + N_3 \qquad b = N_2N_1 + N_3 \qquad c = N_2N_1N_0 + N_3$$

$$d = N_3N_0 + N_3N_1 \qquad e = N_3N_1 \qquad f = N_3N_1 \qquad g = \overline{N_3N_2} + \overline{N_3N_1N_0}$$

Fíjate en que e y f valen lo mismo: solo se encienden con la nota 10.

✅ Resultado final

Las siete ecuaciones anteriores, comprobadas para las notas 0 a 10.

Hoja Karnaugh · K17

K17 · Binario de 4 bits a dos dígitos BCD

✔ verificado

Un número binario $E_3E_2E_1E_0$ (0 a 15) se convierte a **dos dígitos BCD**: decenas D (un solo bit, vale 0 o 1) y unidades $U_3U_2U_1U_0$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: cinco funciones de cuatro variables. **Teoría:** BCD y binario (T2), Karnaugh (T3).

1 Idea

Para $E \leq 9$ el BCD coincide con el binario. Para $10 \leq E \leq 15$ las decenas valen 1 y las unidades son $E - 10$ (0 a 5). Por ejemplo, $13 = 0\,1101 \rightarrow D = 1$, unidades $= 0011$.

2 Ecuaciones

$$D = E_3E_1 + E_3E_2 \qquad U_0 = E_0$$

$$U_3 = E_3\overline{E_2E_1} \qquad U_2 = \overline{E_3}E_2 + E_2E_1 \qquad U_1 = \overline{E_3}E_1 + E_3E_2\overline{E_1}$$

$U_0 = E_0$: restar 10 no cambia la paridad.

✅ Resultado final

Las cinco ecuaciones anteriores, verificadas con las 16 combinaciones.

Hoja de problemas · Autómatas síncronos con biestables JK

8 problemas: tres o más unos o ceros, detectores de secuencia con solapamiento, sumador serie y contador reversible.

Hoja autómatas · A1

A1 · Tres o más unos seguidos

✔ verificado

Diseña un autómata síncrono (Mealy, biestables JK) con una entrada serie E y una salida S que valga 1 mientras se hayan recibido **tres o más unos consecutivos** (desde el tercer uno y mientras sigan llegando unos). Un cero reinicia la cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: diagrama de estados, codificación, tablas de excitación y ecuaciones J-K y de salida. **Teoría:** diseño de autómatas síncronos (T10), excitación del JK (T6).

1 Estados

S_0: ningún uno seguido. S_1: un uno. S_2: dos o más unos. Tres estados necesitan dos biestables: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$ (Q_1Q_0); el 11 es indiferente.

2 Transiciones (entrada E / salida S)

Estado	$E = 0$	$E = 1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_0/0$	$S_2/1$

En S_2 con $E = 1$ ya van tres o más unos: salida 1 y se queda.

3 Excitación y ecuaciones

Con la tabla de excitación del JK y Karnaugh con el estado 11 como indiferente:

$$J_1 = Q_0E \qquad K_1 = \overline{E}$$

$$J_0 = \overline{Q_1}E \qquad K_0 = 1$$

$$S = Q_1E$$

4 Comprobación

Simulado a nivel de puerta contra la definición («lleva tres o más unos seguidos») para **todas** las secuencias de hasta 12 bits.

✅ Resultado final

$J_1 = Q_0E$, $K_1 = \overline{E}$, $J_0 = \overline{Q_1}E$, $K_0 = 1$, $S = Q_1E$ (Mealy).

`Hoja autómatas · A2` **A2 · Tres o más ceros seguidos** ✔ verificado

El mismo problema que A1, pero con **tres o más ceros consecutivos**. Un uno reinicia la cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: el mismo diseño con la entrada negada. **Teoría:** T10, T6.

1 Cambio respecto de A1

Es A1 con E y $\overline{E}$ intercambiados: los mismos tres estados cuentan ceros en vez de unos.

2 Ecuaciones

$$J_1 = Q_0\overline{E} \qquad K_1 = E$$

$$J_0 = \overline{Q_1E} \qquad K_0 = 1$$

$$S = Q_1\overline{E}$$

✅ Resultado final

$J_1 = Q_0\overline{E}$, $K_1 = E$, $J_0 = \overline{Q_1E}$, $K_0 = 1$, $S = Q_1\overline{E}$. Verificado con todas las secuencias de hasta 12 bits.

Hoja autómatas · A3

A3 · Detector de la secuencia 1011 (con solapamiento)

✔ verificado

Diseña un detector de la secuencia **1011** para una entrada serie E. La salida S vale 1 en el ciclo en que llega el último bit de la secuencia. Las secuencias pueden **solaparse** (en 1011011 se detectan dos).

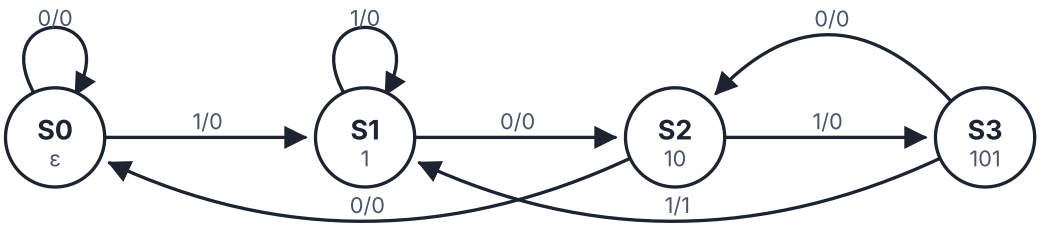
🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: autómata de Mealy con 4 estados y 2 biestables JK. **Teoría:** detectores de secuencia (T10).

1 Estados

Cada estado recuerda **el mayor prefijo de 1011** que es sufijo de lo recibido. S_0: ninguno. S_1: «1». S_2: «10». S_3: «101». Codificación $S_0 = 00,\ S_1 = 01,\ S_2 = 10,\ S_3 = 11$.

2 Diagrama



Detector de 1011 con solapamiento. Etiquetas: E / S

Desde S_3 con un 1 se detecta la secuencia y el último 1 pasa a ser el primero de la siguiente (S_1). Con un 0 se queda «10» (S_2).

3 Ecuaciones

$$J_1 = Q_0\overline{E} \qquad K_1 = \overline{Q_0 E} + Q_0 E$$

$$J_0 = E \qquad K_0 = \overline{E}$$

$$S = Q_1 Q_0 E$$

⚠️ **El solapamiento**

Sin solapamiento, S_3 con $E = 1$ volvería a S_0. Con solapamiento vuelve a S_1. Es el error más común en este tipo de problemas: **mira siempre qué parte del final de la secuencia vale como principio de la siguiente**.

✅ **Resultado final**

$J_1 = Q_0\overline{E}$, $K_1 = \overline{Q_0 E} + Q_0 E$, $J_0 = E$, $K_0 = \overline{E}$, $S = Q_1 Q_0 E$ (Mealy). Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A5a`

A5a · Detector de cuatro unos (1111)

✔ verificado

Diseña un detector de la secuencia **1111** con solapamiento (la salida vale 1 cada vez que los cuatro últimos bits recibidos son unos).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: Mealy con 4 estados. **Teoría:** T10.

1 Estados

Los estados cuentan los unos seguidos: S_0 (0), S_1 (1), S_2 (2), S_3 (3 o más). Con $E = 1$ en S_3 se activa la salida y se **permanece** en S_3 (solapamiento). Con $E = 0$ siempre se vuelve a S_0.

2 Ecuaciones

$$J_1 = Q_0 E \qquad K_1 = \overline{E}$$

$$J_0 = E \qquad K_0 = \overline{Q_1} + \overline{E}$$

$$S = Q_1 Q_0 E$$

✅ **Resultado final**

$J_1 = Q_0 E$, $K_1 = \overline{E}$, $J_0 = E$, $K_0 = \overline{Q_1} + \overline{E}$, $S = Q_1 Q_0 E$. Verificado con todas las secuencias de hasta 12 bits.

Hoja autómatas · A5b

A5b · Detector de la secuencia 11011

✔ verificado

Diseña un detector de la secuencia **11011** con solapamiento.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: Mealy con 5 estados, es decir 3 biestables. **Teoría:** T10.

1 Estados

Prefijos de 11011: S_0 (ninguno), S_1 («1»), S_2 («11»), S_3 («110»), S_4 («1101»). Cinco estados necesitan 3 biestables (000 a 100); los otros tres códigos son indiferentes.

2 Transiciones

Estado	$E = 0$	$E = 1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_3/0$	$S_2/0$
S_3	$S_0/0$	$S_4/0$
S_4	$S_0/0$	$S_2/1$

En S_2 con $E = 1$ (tres unos seguidos) se sigue en «11»: se queda en S_2. En S_4 con $E = 1$ se detecta la secuencia y los dos últimos unos valen como «11»: se pasa a S_2.

3 Ecuaciones

$$J_2 = Q_1 Q_0 E \qquad K_2 = 1$$

$$J_1 = Q_0 E + Q_2 E \qquad K_1 = Q_0$$

$$J_0 = \overline{Q_2 Q_1} E + Q_1 \overline{E} \qquad K_0 = 1$$

$$S = Q_2 E$$

✅ **Resultado final**

Las siete ecuaciones anteriores. Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A7` **A7 · Detector de 1010 o de 110** ✔ verificado

Diseña un autómata con una entrada serie E cuya salida vale 1 cuando los últimos bits recibidos son **1010 o 110** (con solapamiento).

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: un autómata que detecta **dos** secuencias a la vez. **Teoría:** T10.

1 Estados

Cada estado es el mayor sufijo de lo recibido que es prefijo de alguna de las dos secuencias. Los prefijos útiles son: ε, «1», «10», «101», «11». Son 5 estados ($S_\varepsilon, S_1, S_{10}, S_{11}, S_{101}$), 3 biestables. Codificación: $S_\varepsilon = 000$, $S_1 = 001$, $S_{10} = 010$, $S_{11} = 011$, $S_{101} = 100$.

2 Transiciones

Estado	$E = 0$	$E = 1$
S_ε	$S_\varepsilon / 0$	$S_1 / 0$
S_1	$S_{10} / 0$	$S_{11} / 0$
S_{10}	$S_\varepsilon / 0$	$S_{101} / 0$

Estado	$E=0$	$E=1$
S_{11}	$S_{10}/1$	$S_{11}/0$
S_{101}	$S_{10}/1$	$S_{11}/0$

Con 110 el último 0 deja «10» (vale como inicio de 1010). Con 1010 también queda «10».

3 Ecuaciones

$$J_2 = Q_1\overline{Q_0}E \qquad K_2 = 1$$

$$J_1 = Q_0 + Q_2 \qquad K_1 = \overline{Q_0}$$

$$J_0 = \overline{Q_1}E \qquad K_0 = \overline{E}$$

$$S = Q_1Q_0\overline{E} + Q_2\overline{E}$$

✅ **Resultado final**

Las siete ecuaciones anteriores. Verificado con todas las secuencias de hasta 12 bits.

`Hoja autómatas · A9` **A9 · Sumador serie** ✔ verificado

Diseña un **sumador serie**: dos números llegan bit a bit, empezando por el menos significativo, por las entradas E_1 y E_0. En cada ciclo la salida S da el bit correspondiente de la suma. Un biestable JK guarda el acarreo.

🧭 **Antes de empezar: qué piden y qué teoría usas**

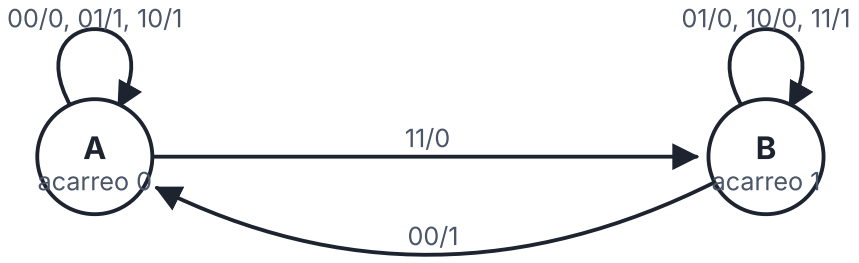
Qué piden: autómata de Mealy de dos estados (acarreo 0 o 1). **Teoría:** sumador completo (T5), autómatas (T10).

1 Estados

Dos estados: A (acarreo 0, $Q_0 = 0$) y B (acarreo 1, $Q_0 = 1$). Un solo biestable.

2 Transiciones

Con $t = E_1 + E_0 + \text{acarreo}$: salida $= t \bmod 2$ y nuevo acarreo $= \lfloor t/2 \rfloor$.



Sumador serie. Etiquetas: E_1E_0 / S

El estado recuerda el acarreo; la salida es el bit de suma.

3 Ecuaciones

$$J_0 = E_1 E_0 \qquad K_0 = \overline{E_1 E_0}$$

$$S = \overline{Q_0 E_1} E_0 + \overline{Q_0} E_1 \overline{E_0} + Q_0 \overline{E_1 E_0} + Q_0 E_1 E_0$$

El acarreo se activa solo si las dos entradas valen 1 ($J_0 = E_1 E_0$) y se desactiva solo si las dos valen 0 ($K_0 = \overline{E_1}\,\overline{E_0}$). En el resto de casos se mantiene. La salida es la **XOR de tres bits**: $S = Q_0 \oplus E_1 \oplus E_0$.

✅ Resultado final

$J_0 = E_1 E_0$, $K_0 = \overline{E_1 E_0}$, $S = Q_0 \oplus E_1 \oplus E_0$. Probado con 400 sumas aleatorias de hasta 10 bits.

Hoja autómatas · A11

A11 · Contador reversible módulo 4

✔ verificado

Diseña un contador módulo 4 **reversible** (autómata de Moore). La entrada E elige el sentido: $E = 1$ cuenta hacia arriba $(0, 1, 2, 3, 0 \ldots)$ y $E = 0$ hacia abajo $(0, 3, 2, 1, 0 \ldots)$. La salida es el valor de la cuenta en binario de dos bits.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: un contador de dos biestables JK con entrada de sentido. **Teoría:** contadores (T8), autómatas de Moore (T10).

1 Estados y salida

Cuatro estados $S_0, \ldots, S_3$ con codificación igual al valor $(00, 01, 10, 11)$. Como es Moore, **la salida depende solo del estado**: $S_1 S_0 = Q_1 Q_0$.

2 Ecuaciones

Para contar: Q_0 cambia siempre; Q_1 cambia cuando $Q_0 = 1$ si sube y cuando $Q_0 = 0$ si baja.

$$J_0 = K_0 = 1 \qquad J_1 = K_1 = \overline{Q_0 E} + Q_0 E$$

$J_1 = K_1 = \overline{Q_0 \oplus E}$: se activa cuando $Q_0 = E$ (con $E = 1$ sube y $Q_0 = 1$ acarrea; con $E = 0$ baja y $Q_0 = 0$ pide préstamo).

3 Comprobación

Con $E = 1, 1, 1, 1, 1, 0, 0, 0, 0$ desde el estado 0 las salidas, tras cada flanco de reloj, son [1, 2, 3, 0, 1, 0, 3, 2, 1].

✅ Resultado final

$J_0 = K_0 = 1$; $J_1 = K_1 = \overline{Q_0 E} + Q_0 E$; salidas $S_1 = Q_1$, $S_0 = Q_0$ (Moore).

EJERCICIOS RESUELTOS

Simulacros de examen

Dos exámenes de cuatro problemas con las soluciones ocultas. Cronométrate 2 horas.

Cómo usarlos

1. Prepara papel, calculadora y el formulario de una página (formulario). Nada más.
2. Pon un cronómetro de **2 horas** por simulacro. Son cuatro problemas de 2,5 puntos, como en los exámenes reales (examen).
3. Resuelve **sin abrir** las soluciones. Los desplegables están cerrados a propósito.
4. Corrige con la regla de los exámenes: se valoran la **simplicidad** y la **claridad**. Un cronograma sin explicación no cuenta.
5. Anota qué tipo de problema has fallado y vuelve a su tema en la guía (plan).

> 💡 **Todo está comprobado**
>
> Cada solución se ha simulado a nivel de puerta con todas las combinaciones posibles (o con todas las secuencias de entrada de hasta 8 a 12 bits, en los autómatas). Los problemas son originales de esta guía, pero del mismo tipo que los de examen.

Simulacro 1

Problema 1 · Código Gray a binario (2,5 puntos)

Un sensor de posición angular entrega su lectura en **código Gray de 3 bits** $G_2G_1G_0$. Diseña el circuito combinacional que la convierte a binario natural $B_2B_1B_0$: tabla de verdad, ecuaciones simplificadas y esquema con puertas.

Ver la solución del problema 1

Teoría: código Gray (solo cambia un bit entre valores consecutivos) y Karnaugh o XOR (T2, T3).

La secuencia Gray de 0 a 7 es ["000", "001", "011", "010", "110", "111", "101", "100"]. Se invierte la tabla: para cada valor Gray se busca el binario.

$G_2G_1G_0$	000	001	011	010	110	111	101	100
$B_2B_1B_0$	000	001	010	011	100	101	110	111

Ecuaciones simplificadas por Karnaugh:

$$B_2 = G_2 \qquad B_1 = \overline{G_2}G_1 + G_2\overline{G_1}$$

$$B_0 = \overline{G_2G_1}G_0 + \overline{G_2}G_1\overline{G_0} + G_2\overline{G_1G_0} + G_2G_1G_0$$

B_1 es una XOR y B_0 es la XOR de tres entradas. Con puertas XOR queda:

$$B_2 = G_2 \qquad B_1 = G_2 \oplus G_1 \qquad B_0 = G_2 \oplus G_1 \oplus G_0 = B_1 \oplus G_0$$

Son **dos XOR en cascada**: la forma más simple. Cada bit binario es la XOR de todos los bits Gray de igual o mayor peso.

Problema 2 · Sumador BCD de un dígito (2,5 puntos)

Diseña con bloques estándar un sumador de dos dígitos BCD (A y B, cada uno de 0 a 9) con acarreo de entrada. La salida debe ser un dígito BCD y un acarreo de salida. Explica cómo se detecta que hace falta corregir y por qué se suma 6.

Ver la solución del problema 2

Teoría: BCD y aritmética en BCD (T2), sumador de 4 bits (T5).

1. Un sumador de 4 bits suma $A + B + C_{in}$ en binario. El máximo es $9 + 9 + 1 = 19$, que cabe en 5 bits (acarreo K y suma $S_3S_2S_1S_0$).
2. Si la suma vale ≤ 9 ya es un dígito BCD. Si vale de 10 a 19, el resultado BCD correcto es **«1» y el dígito** $S - 10$. Pero el binario ha contado 16 por cada acarreo, no 10. Se **suma 6** (la diferencia entre 16 y 10) y se descartan los 16.
3. Hace falta corregir cuando la suma es mayor que 9: $K = 1$ (suma ≥ 16) o $S_3S_2 = 1$ o $S_3S_1 = 1$ (suma de 10 a 15):

$$\text{Corregir} = K + S_3S_2 + S_3S_1$$

1. Un segundo sumador de 4 bits suma $0\,(\text{Corregir})(\text{Corregir})0$, es decir 0110 o 0000, a $S_3S_2S_1S_0$. Su resultado es el dígito BCD. El acarreo de salida es «Corregir».

Ejemplos: $7 + 8 = 15 = 01111_2$; hay que corregir; $1111 + 0110 = 10101$: dígito 0101 y acarreo 1, es decir **15**. $9 + 9 + 1 = 19 = 10011_2$: el binario da $K = 1$ y 0011; $0011 + 0110 = 1001$ con acarreo 1: **19**. $4 + 3 = 0111$: no se corrige.

Se ha comprobado con las 200 combinaciones de (A, B, C_{in}).

Problema 3 · Contador síncrono 0, 2, 5, 7 (2,5 puntos)

Diseña con biestables JK un contador síncrono de 3 bits que repita la secuencia **0, 2, 5, 7, 0, 2, 5, 7...** (en decimal). Los estados 1, 3, 4 y 6 no se usan. Da las ecuaciones J y K de cada biestable y una señal F que valga 1 en el estado 7. Comenta qué pasa si el circuito arranca en un estado no usado.

Ver la solución del problema 3

Teoría: diseño de contadores con JK (T8, T6).

Tabla de transición (los estados no usados son indiferentes):

Estado actual $Q_2Q_1Q_0$	Siguiente
000	010
010	101
101	111
111	000

Con la tabla de excitación del JK y Karnaugh con indiferentes:

$$J_2 = K_2 = Q_1 \qquad J_1 = K_1 = 1 \qquad J_0 = K_0 = Q_1$$

$$F = Q_1 Q_0$$

Todos los biestables trabajan en modo **toggle** ($J = K$): Q_1 conmuta siempre, Q_2 y Q_0 conmutan cuando $Q_1 = 1$. Se ha simulado a nivel de puerta y repite $0, 2, 5, 7, 0, \ldots$

> ⚠️ **El circuito no se recupera solo**
>
> Los estados no usados forman un **ciclo cerrado**: $1 \to 3 \to 4 \to 6 \to 1$. Si al encender el circuito los biestables arrancan en uno de ellos, el contador **nunca entra** en la secuencia buena. Solución: poner un **reset** asíncrono (clear) que lleve a 000 al arrancar, o elegir las indiferentes para que los estados no usados desemboquen en la secuencia (autoarranque).

Problema 4 · Detector de la secuencia 0110 (2,5 puntos)

Diseña con biestables JK un autómata (Mealy) con una entrada serie E y una salida S que valga 1 cuando los últimos cuatro bits recibidos sean **0110**. Puede haber solapamiento. Da el diagrama de estados, la tabla de transición y las ecuaciones.

Ver la solución del problema 4

Teoría: autómatas síncronos (T10).

Estados: el mayor prefijo de 0110 que es sufijo de lo recibido. S_0: ninguno. S_1: «0». S_2: «01». S_3: «011». Codificación $00, 01, 10, 11$.

Estado	$E=0$	$E=1$
S_0	$S_1/0$	$S_0/0$
S_1	$S_1/0$	$S_2/0$
S_2	$S_1/0$	$S_3/0$
S_3	$S_1/1$	$S_0/0$

Con un 0 siempre se pasa a «0» (S_1), salvo en S_3 donde ese 0 completa 0110 (salida 1) y vale como inicio de la siguiente (S_1).

$$J_1 = Q_0 E \qquad K_1 = \overline{E} + Q_0$$

$$J_0 = \overline{E} + Q_1 \qquad K_0 = E$$

$$S = Q_1 Q_0 \overline{E}$$

Verificado con todas las secuencias de hasta 12 bits.

Simulacro 2

Problema 1 · Codificador de prioridad (2,5 puntos)

Diseña un codificador de prioridad de cuatro entradas $I_3 I_2 I_1 I_0$ (I_3 es la más prioritaria). Salidas: $A_1 A_0$ (número de la entrada activa más prioritaria) y V, que indica si hay alguna entrada activa. Da las ecuaciones simplificadas.

Ver la solución del problema 1

Teoría: codificadores de prioridad (T5), Karnaugh (T3).

Entradas activas	A_1A_0	V
ninguna	00	0
solo I_0	00	1
I_1 (y quizá I_0)	01	1
I_2 (y quizá menores)	10	1
I_3 (y cualquier otra)	11	1

Con Karnaugh sobre las 16 combinaciones:

$$A_1 = I_2 + I_3 \qquad A_0 = \overline{I_2} I_1 + I_3 \qquad V = I_0 + I_1 + I_2 + I_3$$

V es una OR de las cuatro entradas. Fíjate en que A_0 **no** es «$I_3 + I_1$»: si $I_2 = 1$ y $I_1 = 1$, gana I_2 y A_0 debe ser 0. Por eso aparece $\overline{I_2} I_1$.

Problema 2 · Multiplicar por 5 (2,5 puntos)

Un número X de 4 bits sin signo debe multiplicarse por 5. Diseña el circuito con un solo sumador y sin multiplicador. ¿Cuántos bits tiene el resultado?

Ver la solución del problema 2

Teoría: numeración y desplazamientos (T2), sumadores (T5).

$5X = 4X + X$. Multiplicar por 4 es **desplazar dos posiciones a la izquierda** (cableado, sin puertas): $X \ll 2$ es $X_3X_2X_1X_0\,00$ (6 bits). Un sumador de 7 bits suma $X \ll 2$ y X (ampliados con ceros).

El máximo es $5 \cdot 15 = 75$. Como $2^6 = 64 < 75 < 128 = 2^7$, hacen falta **7 bits**. Se ha comprobado con los 16 valores de X.

Problema 3 · Temporización con un reloj de 50 MHz (2,5 puntos)

Una placa tiene un reloj de 50 MHz. (a) Obtén una señal de 1 Hz con ciclo de trabajo del 50 %: ¿hasta qué valor cuenta el contador y cuántos bits tiene? (b) Un display de 4 dígitos multiplexado se refresca con un contador de 17 bits cuyos dos bits altos eligen el dígito. ¿Cuál es la frecuencia de barrido y la de refresco de cada dígito? ¿Se verá parpadear? (c) ¿Cuántos bits necesita un contador que mide 10 ms (antirrebote) con ese reloj?

Ver la solución del problema 3

Teoría: divisores de frecuencia, multiplexado de displays y antirrebote (T8, T12).

(a) Una señal de 1 Hz con ciclo del 50 % cambia cada **medio segundo**: $25\,000\,000$ ciclos de reloj. El contador cuenta de 0 a 24999999 y entonces invierte un biestable de salida. Como $2^{24} < 25\,000\,000 \leq 2^{25}$, necesita **25 bits**.

(b) El contador de 17 bits desborda cada 2^{17} ciclos: $50\,000\,000/2^{17} \approx 381.5$ Hz. Los dos bits altos recorren cuatro dígitos, así que cada dígito se enciende ≈ 95.4 Hz. Es más que 60 Hz: **no se ve parpadear**.

(c) 10 ms $= 0{,}01 \cdot 50\,000\,000 = 500\,000$ ciclos. Como $2^{18} = 262\,144 < 500\,000 \leq 524\,288 = 2^{19}$, hacen falta **19 bits**.

Problema 4 · Máquina de 30 céntimos con cambio (2,5 puntos)

Una máquina vende un producto de 30 céntimos y acepta monedas de 10 y de 20 céntimos, una por ciclo. Las entradas son $M_{20}M_{10}$: 00 (nada), 01 (moneda de 10), 10 (moneda de 20); 11 no se da. Cuando el total llega a 30 o más, la salida Z entrega el producto y se vuelve a empezar. Si el total es 40, la salida C devuelve 10 céntimos de cambio. Diseña el autómata de Mealy con JK.

Ver la solución del problema 4

Teoría: autómatas de Mealy (T10).

Estados: S_0 (0 céntimos), S_1 (10), S_2 (20). Codificación $00, 01, 10$; el 11 es indiferente, y la entrada 11 también.

Estado	$M_{20}M_{10} = 00$	01 (10 cts)	10 (20 cts)
S_0	$S_0/ZC = 00$	$S_1/00$	$S_2/00$
S_1	$S_1/00$	$S_2/00$	$S_0/Z = 1$ (30)
S_2	$S_2/00$	$S_0/Z = 1$ (30)	$S_0/Z = 1, C = 1$ (40)

$$J_1 = \overline{Q_0}M_{20} + Q_0M_{10} \qquad K_1 = M_{10} + M_{20}$$

$$J_0 = \overline{Q_1}M_{10} \qquad K_0 = M_{10} + M_{20}$$

$$Z = Q_0M_{20} + Q_1M_{10} + Q_1M_{20} \qquad C = Q_1M_{20}$$

Verificado con todas las secuencias de monedas de hasta 8 ciclos.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 1 · Códigos binarios

13 problemas resueltos: conversiones entre decimal, binario, hexadecimal, octal, BCD y Gray; rangos; complemento a 1, a 2 y a 10; sumas y desbordamiento.

💡 Cómo se usa esta sección

Son los problemas resueltos del capítulo 1 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras y resueltos de nuevo. Cada resultado se ha calculado con un programa y se ha comparado con el valor que da el libro. Si hay alguna diferencia, aparece una nota «Ojo». La teoría está en el tema 2.

Libro · Cap. 1 · P1.1

Binario a decimal y decimal a binario

✔ verificado

Pasa a decimal veinte números binarios sin signo y pasa a binario veinte números decimales.

🧭 Antes de empezar: qué piden y qué teoría usas

Método: de binario a decimal se suman los pesos 2^i de los bits que valen 1. De decimal a binario se divide entre 2 y se leen los restos de abajo arriba. Los ceros a la izquierda no cambian el valor.

1 Binario a decimal

- $01_2 = 1$
- $1101_2 = 13$
- $1100_2 = 12$
- $1111_2 = 15$
- $0011_2 = 3$
- $0101_2 = 5$
- $1010_2 = 10$
- $0000_2 = 0$
- $000000_2 = 0$
- $110011_2 = 51$
- $10101_2 = 21$
- $0100011_2 = 35$
- $01111000_2 = 120$

- $0110011_2 = 51$
- $000000011_2 = 3$
- $11011_2 = 27$
- $011_2 = 3$
- $1111111111_2 = 1023$
- $100000001_2 = 257$
- $00110_2 = 6$

2 Decimal a binario

- $1 = 1_2$
- $7 = 111_2$
- $9 = 1001_2$
- $12 = 1100_2$
- $34 = 100010_2$
- $32 = 100000_2$
- $33 = 100001_2$
- $1 = 1_2$
- $1 = 1_2$
- $100 = 1100100_2$
- $205 = 11001101_2$
- $63 = 111111_2$
- $64 = 1000000_2$
- $4 = 100_2$
- $56 = 111000_2$
- $62 = 111110_2$
- $1023 = 1111111111_2$
- $1024 = 10000000000_2$
- $0 = 0_2$
- $15 = 1111_2$

3 Ejemplo con el método de aproximación

Para 205: cabe 128 (resto 77), cabe 64 (resto 13), no cabe 32 ni 16, cabe 8 (resto 5), no cabe 2, cabe 4 y cabe 1: $205 = 128 + 64 + 8 + 4 + 1 = 11001101_2$.

✅ Resultado final

Todas las conversiones se han calculado con código y coinciden con el libro. Regla útil: $2^{10} - 1 = 1023$ es diez unos.

Libro · Cap. 1 · P1.2

Hexadecimal, octal, decimal y binario

✔ verificado

Dado un número escrito en una de las bases (hexadecimal, octal o decimal), escribe su valor en las otras tres bases y en binario. Una de las filas está mal escrita: detéctala.

🧭 Antes de empezar: qué piden y qué teoría usas

Atajo: el hexadecimal agrupa los bits de 4 en 4 y el octal de 3 en 3. Para ir de decimal a otra base, lo más cómodo es pasar antes por binario.

1 Tabla de resultados

Dato	Hex	Octal	Decimal	Binario
12A (hex)	12A	452	298	100101010
1700 (octal)	3C0	1700	960	1111000000
4 (dec)	4	4	4	100
FF (hex)	FF	377	255	11111111
6432 (octal)	D1A	6432	3354	110100011010
17 (dec)	11	21	17	10001
055 (hex)	55	125	85	1010101
034 (octal)	1C	34	28	11100
8 (dec)	8	10	8	1000
3F (hex)	3F	77	63	111111
25 (dec)	19	31	25	11001
178 (octal)	—	—	—	Error
38 (dec)	26	46	38	100110
0A1 (hex)	A1	241	161	10100001

Dato	Hex	Octal	Decimal	Binario
3154 (octal)	66C	3154	1644	11001101100
16 (dec)	10	20	16	10000
10 (octal)	8	10	8	1000
09 (hex)	9	11	9	1001

2 **La fila con error**

El dato 178_8 lleva el dígito 8, y en octal solo existen 0 a 7. No es un número octal válido: es una «falta de ortografía», no una operación mal hecha.

✅ **Resultado final**

$12A_{16} = 452_8 = 298_{10}$, $FF_{16} = 377_8 = 255_{10}$, $D1A_{16} = 6432_8 = 3354_{10}$ y $66C_{16} = 3154_8 = 1644_{10}$.

⚠️ **Errata del libro**

En la fila de $D1A_{16}$ el libro imprime 3348 en decimal. Con los otros tres datos de esa fila (6432_8, 110100011010_2) el valor correcto es 3354: $13 \cdot 256 + 1 \cdot 16 + 10 = 3354$.

`Libro · Cap. 1 · P1.3`

Leer una combinación de 4 bits con seis códigos decimales

✔ verificado

Para cada una de las 16 combinaciones de 4 bits, indica qué dígito decimal representa en BCD puro, BCD exceso 3, Aiken, BCD 3321, Gray exceso 3 y BCD 5421. Si la combinación no es válida en ese código, pon un guion.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: un código decimal usa solo 10 de las 16 combinaciones. Las otras 6 son «prohibidas». Aquí se parte de la definición de cada código.

- BCD puro: el dígito en binario.
- Exceso 3: el dígito $+3$.
- Aiken: 0 a 4 como en binario y 5 a 9 como 11 a 15.
- 3321 y 5421: los pesos de los bits valen $3, 3, 2, 1$ y $5, 4, 2, 1$.
- Gray exceso 3: el Gray de $d + 3$.

1 Cómo se construye cada columna

Para obtener las 10 palabras de cada código se aplica su definición a $d = 0 \ldots 9$ y se invierte la tabla: la palabra que no aparece se marca con un guion. Se han verificado los pesos del 3321 y del 5421, que Aiken y exceso 3 son autocomplementarios ($\bar{d} = 9 - d$ al invertir los 4 bits) y que el Gray exceso 3 cambia un solo bit entre dígitos consecutivos.

2 Tabla

Bits	BCD puro	BCD XS3	Aiken	BCD 3321	Gray XS3	BCD 5421
0000	0	—	0	0	—	0
0001	1	—	1	1	—	1
0010	2	—	2	2	0	2
0011	3	0	3	3	—	3
0100	4	1	4	—	4	4
0101	5	2	—	4	3	—
0110	6	3	—	5	1	—
0111	7	4	—	—	2	—
1000	8	5	—	—	—	5
1001	9	6	—	—	—	6
1010	—	7	—	—	9	7
1011	—	8	5	—	—	8

Bits	BCD puro	BCD XS3	Aiken	BCD 3321	Gray XS3	BCD 5421
1100	—	9	6	6	5	9
1101	—	—	7	7	6	—
1110	—	—	8	8	8	—
1111	—	—	9	9	7	—

✅ **Resultado final**

Cada columna tiene exactamente seis guiones. El código más ordenado y simétrico es el exceso 3, porque cada dígito y su complemento a 9 se obtienen invirtiendo los bits.

`Libro · Cap. 1 · P1.4` **De decimal a BCD, exceso 3 y Aiken** ✔ verificado

Escribe en BCD puro, BCD exceso 3 y BCD Aiken los números decimales de la tabla.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Regla: cada **dígito** decimal se codifica por separado en 4 bits. No se convierte el número entero a binario: 12 en BCD es $0001\ 0010$ y no 1100.

1 **Tabla**

Decimal	BCD puro	BCD XS3	Aiken
4	0100	0111	0100
32	00110010	01100101	00110010
99	10011001	11001100	11111111
63	01100011	10010110	11000011
0	0000	0011	0000
1	0001	0100	0001
55	01010101	10001000	10111011
00	00000000	00110011	00000000

Decimal	BCD puro	BCD XS3	Aiken
36	00110110	01101001	00111100
82	10000010	10110101	11100010
11	00010001	01000100	00010001
12	00010010	01000101	00010010
04	00000100	00110111	00000100
100	000100000000	010000110011	000100000000
44	01000100	01110111	01000100
56	01010110	10001001	10111100

2 **Dos comprobaciones**

Las 16 filas se han decodificado de vuelta y dan el mismo decimal. Además 99 en Aiken es $1111\ 1111$ y en exceso 3 es $1100\ 1100$.

✅ **Resultado final**

Cada dígito ocupa 4 bits: 100 ocupa 12 bits en BCD ($0001\ 0000\ 0000$) frente a 7 en binario puro. El BCD es más largo, pero se lee dígito a dígito.

`Libro · Cap. 1 · P1.5` **De binario a Gray y de Gray a binario** ✔ verificado

Convierte entre binario puro y código Gray las parejas de la tabla. En unas se conoce el binario y en otras el Gray.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Binario a Gray: $G_{n-1} = B_{n-1}$ y $G_i = B_{i+1} \oplus B_i$. **Gray a binario:** $B_{n-1} = G_{n-1}$ y $B_i = B_{i+1} \oplus G_i$ (se va de izquierda a derecha).

1 **Ejemplo:** 11000 **a Gray**

$G_4 = 1$; $G_3 = 1 \oplus 1 = 0$; $G_2 = 1 \oplus 0 = 1$; $G_1 = 0 \oplus 0 = 0$; $G_0 = 0 \oplus 0 = 0$. Resultado

10100.

2 **Ejemplo:** 1000_{Gray} **a binario**

$B_3 = 1$; $B_2 = 1 \oplus 0 = 1$; $B_1 = 1 \oplus 0 = 1$; $B_0 = 1 \oplus 0 = 1$. Resultado 1111.

3 **Tabla completa**

Binario	Gray
0011	0010
001	001
11000	10100
01010101	01111111
11110000	10001000
01	01
1111	1000
10000	11000
10111100	11100010
00001	00001
0101001111	0111101000
1000	1100
0010	0011
0110	0101
1001111	1101000
1100	1010

✅ **Resultado final**

Las 16 parejas se comprueban en los dos sentidos. En Gray dos números consecutivos se diferencian en un solo bit (comprobado para 0 a 63).

Libro · Cap. 1 · P1.6

Rango de valores según el número de bits

✔ verificado

Escribe el intervalo de valores que se puede representar con n bits (de 1 a 16) en binario puro sin signo, con bit de signo, en complemento a 1 y en complemento a 2. Después calcula cuántos bits necesitan varias magnitudes reales.

🧭 Antes de empezar: qué piden y qué teoría usas

Fórmulas: sin signo $[0, 2^n - 1]$; bit de signo y C1 $[-(2^{n-1} - 1),\ 2^{n-1} - 1]$; C2 $[-2^{n-1},\ 2^{n-1} - 1]$. Con un solo bit no cabe el signo: hacen falta al menos 2.

1 Tabla de rangos

n	Sin signo	Bit de signo	C1	C2
1	[0, 1]	al menos 2 bits	al menos 2 bits	al menos 2 bits
2	[0, 3]	[−1, +1]	[−1, +1]	[−2, +1]
3	[0, 7]	[−3, +3]	[−3, +3]	[−4, +3]
4	[0, 15]	[−7, +7]	[−7, +7]	[−8, +7]
5	[0, 31]	[−15, +15]	[−15, +15]	[−16, +15]
6	[0, 63]	[−31, +31]	[−31, +31]	[−32, +31]
7	[0, 127]	[−63, +63]	[−63, +63]	[−64, +63]
8	[0, 255]	[−127, +127]	[−127, +127]	[−128, +127]
9	[0, 511]	[−255, +255]	[−255, +255]	[−256, +255]
10	[0, 1023]	[−511, +511]	[−511, +511]	[−512, +511]
11	[0, 2047]	[−1023, +1023]	[−1023, +1023]	[−1024, +1023]
12	[0, 4095]	[−2047, +2047]	[−2047, +2047]	[−2048, +2047]
13	[0, 8191]	[−4095, +4095]	[−4095, +4095]	[−4096, +4095]

n	Sin signo	Bit de signo	C1	C2
14	[0, 16383]	[−8191, +8191]	[−8191, +8191]	[−8192, +8191]
15	[0, 32767]	[−16383, +16383]	[−16383, +16383]	[−16384, +16383]
16	[0, 65535]	[−32767, +32767]	[−32767, +32767]	[−32768, +32767]

2 Por qué C2 llega más lejos

Bit de signo y C1 tienen dos ceros ($+0$ y -0), así que pierden una combinación. C2 tiene un solo cero y la combinación sobrante es el -2^{n-1} (un 1 seguido de ceros).

3 Magnitudes reales

Aplicación	Rango	Bits
Temperatura ambiente	[-30, 50]	7 bits en BP con C2
Notas de 100 alumnos (cada una)	[0, 10]	4 bits en BP
Suma de las notas de 100 alumnos	[0, 1000]	10 bits en BP
Pesetas por cajetilla	[0, 500]	9 bits en BP
Velocidad de un coche	[0, 300]	9 bits en BP
Peso de una persona	[0, 200]	8 bits en BP
Suma de 100 pesos	[0, 20000]	15 bits en BP
Altura sobre/bajo el mar (avión incluido)	[-12000, 9000]	15 bits en BP con C2
Diferencia horaria	[-12, 12]	5 bits en BP con C2
Puntos de un equipo de baloncesto (supuesto 0 a 200)	[0, 200]	8 bits en BP

4 Lecturas útiles

Para sumar 100 notas hay que mirar el rango de la **suma** (0 a 1000 ⇒ 10 bits), no solo el de cada nota (4 bits). Con 7 bits en C2 se cubre -64 a $+63$ y los $-30 \ldots + 50$ °C caben.

✅ **Resultado final**

Se ha comprobado por enumeración de todas las combinaciones hasta 10 bits que los intervalos y el número de valores distintos son los de la tabla.

`Libro · Cap. 1 · P1.7`

Las 16 combinaciones de 4 bits leídas como sin signo, C1 y C2

✔ verificado

Escribe el valor decimal de cada combinación de 4 bits según tres lecturas: binario sin signo, binario con signo en C1 y binario con signo en C2.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Pista: si el primer bit es 0, el número es positivo y se lee como binario normal. Si es 1, se obtiene el complemento del número (a 1 o a 2) y se lee con signo menos.

1 **Tabla**

Bits	Sin signo	C1	C2
0000	0	+0	0
0001	1	+1	+1
0010	2	+2	+2
0011	3	+3	+3
0100	4	+4	+4
0101	5	+5	+5
0110	6	+6	+6
0111	7	+7	+7
1000	8	−7	−8
1001	9	−6	−7

Bits	Sin signo	C1	C2
1010	10	−5	−6
1011	11	−4	−5
1100	12	−3	−4
1101	13	−2	−3
1110	14	−1	−2
1111	15	−0	−1

2 **El caso 1000**

En C1: el complemento a 1 de 1000 es $0111 = +7$, luego vale -7. En C2: el complemento a 2 es $0111 + 1 = 1000$ otra vez. Es el único número que se «muerde la cola»: vale $-2^{n-1} = -8$. Conviene memorizarlo: **un 1 seguido de ceros es -2^{n-1} en C2**.

✅ **Resultado final**

En C1 existen dos ceros ($0000 = +0$ y $1111 = -0$). En C2 solo hay uno y por eso llega hasta -8.

`Libro · Cap. 1 · P1.8` **De decimal con signo a binario con signo** ✔ verificado

Escribe cada número decimal con signo en binario puro con bit de signo, en C1 y en C2, usando el menor número de bits posible en cada código.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Positivos: se escriben igual en los tres códigos (un 0 delante y la magnitud). **Negativos:** con bit de signo se pone un 1 y la magnitud. En C1 se invierten los bits del positivo. En C2 se invierten y se suma 1. **Anchura:** se cuentan los bits de la magnitud y se añade uno para el signo; en C2 el -2^{n-1} cabe en n bits.

1 **Ejemplo con -17**

El positivo es 010001. Bit de signo: 110001. C1: 101110. C2: $101110 + 1 = 101111$.

2 Tabla

Decimal	Bit de signo	C1	C2
+15	01111	01111	01111
+36	0100100	0100100	0100100
+8	01000	01000	01000
+1	01	01	01
0	00	00	00
−10	11010	10101	10110
−4	1100	1011	100
−9	11001	10110	10111
+17	010001	010001	010001
−17	110001	101110	101111
−1	11	10	11
−106	11101010	10010101	10010110
−64	11000000	10111111	1000000
−63	1111111	1000000	1000001
−44	1101100	1010011	1010100
−0	10	11	00
+99	01100011	01100011	01100011
−99	11100011	10011100	10011101
−1585	111000110001	100111001110	100111001111
+1260	010011101100	010011101100	010011101100

3 Cómo se ha comprobado

Cada palabra se ha vuelto a leer con una función independiente y da el decimal de partida. Además $C2 = C1 + 1$ para todos los negativos de 12 bits.

✅ **Resultado final**

Los positivos son iguales en los tres códigos. En un cero negativo (-0) el bit de signo da 10, el C1 da 11 y el C2 no lo distingue: 00.

Libro · Cap. 1 · P1.9

Leer una combinación con signo: bit de signo, C1 y C2

✔ verificado

Para cada secuencia binaria de la tabla, indica qué número decimal con signo representa si se lee como binario con bit de signo, como C1 y como C2.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Lectura: primer bit 0 → positivo, se lee normal. Primer bit 1 → negativo. Con bit de signo se lee la magnitud del resto. En C1 se invierten los bits y se lee. En C2 se invierten, se suma 1 y se lee (o se usa el peso -2^{n-1} del primer bit).

1 Ejemplo con 1011001

Bit de signo: -25 (magnitud 011001). C1: invierto $0100110 = 38$, luego -38. C2: $0100110 + 1 = 0100111 = 39$, luego -39.

2 Tabla

Secuencia	Bit de signo	C1	C2
100	−0	−3	−4
00111	+7	+7	+7
10100	−4	−11	−12
11000	−8	−7	−8
10011101	−29	−98	−99
1100	−4	−3	−4

Secuencia	Bit de signo	C1	C2
011111	+31	+31	+31
10000	−0	−15	−16
11111	−15	−0	−1
01111	+15	+15	+15
1111	−7	−0	−1
10010100	−20	−107	−108
11111111	−127	−0	−1
1011001	−25	−38	−39
100000	−0	−31	−32
0000001	+1	+1	+1

✅ **Resultado final**

La misma secuencia cambia de valor según el código: **primero hay que saber en qué código está escrita**. Cada fila se ha vuelto a codificar con el decimal obtenido y se recupera la secuencia original.

`Libro · Cap. 1 · P1.10` **Suma en binario sin signo y desbordamiento** ✔ verificado

Suma las parejas de números binarios sin signo de la tabla. Indica el resultado en decimal y si hay desbordamiento al usar tantos bits como tiene el sumando más largo.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Regla: se suma como en decimal, de derecha a izquierda, llevando acarreos. En binario sin signo hay desbordamiento cuando **sale un acarreo del bit más significativo**: el resultado no cabe en n bits. Si un sumando es más corto, se rellena con ceros a la izquierda.

1 **Tabla**

A	B	n	A+B	Decimal	Desborda
1100 (12)	0001 (1)	4	1101	13	no
101000 (40)	000001 (1)	6	101001	41	no
0001011001 (89)	1111001010 (970)	10	10000100011	1059	sí
1101001010 (842)	0000010011 (19)	10	1101011101	861	no
111111111 (511)	000000001 (1)	9	1000000000	512	sí
00110 (6)	00110 (6)	5	01100	12	no
11000 (24)	10011 (19)	5	101011	43	sí
00010 (2)	00010 (2)	5	00100	4	no
0100001 (33)	0000101 (5)	7	0100110	38	no
0011011010 (218)	1110000101 (901)	10	10001011111	1119	sí

2 Qué significa «desborda»

Con $n = 9$: $111111111 + 1 = 1\ 000000000$. El 1 de la izquierda no cabe y queda $000000000 = 0$ en 9 bits, pero el valor real es $511 + 1 = 512$. Un contador de 9 bits que pasa de 511 vuelve a 0 por el mismo motivo.

✅ Resultado final

Hay desbordamiento exactamente cuando la suma real $\geq 2^n$ (comprobado en las 10 sumas).

⚠️ Errata del libro

En la tercera fila el libro imprime $25 + 970$ y un resultado $1\ 0000100011$ (desbordamiento, valor 35). Pero $25 + 970 = 995$ cabe en 10 bits y no desborda. El resultado impreso corresponde a $89 + 970 = 1059 = 1\ 0000100011$, es decir, al sumando 0001011001. La tabla de arriba usa ese sumando.

Libro · Cap. 1 · P1.11

Suma con signo en C1 y en C2

✔ verificado

Suma las parejas de números binarios con signo, una vez interpretadas en C1 y otra en C2. Escribe el resultado en binario y en decimal y comprueba si hay desbordamiento.

🧭 Antes de empezar: qué piden y qué teoría usas

C2: se suma como si no tuvieran signo y se **tira** el acarreo final. **C1:** igual, pero el acarreo final se **recircula**: se vuelve a sumar al resultado. **Desbordamiento (ambos):** hay desbordamiento si el acarreo que entra en el bit de signo es distinto del que sale de él.

1 Ejemplo C1: $00101 + 11100$

Suma: 100001. El acarreo final es 1: se recircula, $00001 + 1 = 00010 = +2$. Decimal: $+5 + (-3) = +2$.

2 Resultados en C1

A	B	A+B	A (dec)	B (dec)	Resultado	Desborda
00101	11100	00010	+5	−3	+2	no
11010	00011	11101	−5	+3	−2	no
0001101	1101000	1110101	+13	−23	−10	no
1111001	0110000	0101010	−6	+48	+42	no
0011	1100	1111	+3	−3	−0	no

3 Resultados en C2

A	B	A+B	A (dec)	B (dec)	Resultado	Desborda
00101	11100	00001	+5	−4	+1	no
11010	00011	11101	−6	+3	−3	no
0001101	1101000	1110101	+13	−24	−11	no
1111001	0110000	0101001	−7	+48	+41	no

A	B	A+B	A (dec)	B (dec)	Resultado	Desborda
0011	1100	1111	+3	−4	−1	no

✅ **Resultado final**

En ninguna suma hay desbordamiento: el número de bits está bien elegido. Los resultados decimales coinciden con la suma de los decimales de partida.

`Libro · Cap. 1 · P1.12`

Sumar números decimales pasando antes a C1 o C2

✔ verificado

Codifica los dos sumandos decimales en C1 y en C2, súmalos en binario y di si hay desbordamiento. Usa el menor número de bits que permita escribir los dos sumandos.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Orden de trabajo: 1) elegir n; 2) codificar cada sumando (el negativo, como complemento del positivo); 3) sumar; 4) mirar el desbordamiento con los dos últimos acarreos; 5) leer el resultado.

1 Caso $(-12) + (-20)$

Necesita 6 bits. En C1: $110011 + 101011 = 1\ 011110$; recirculo y queda 011111: el resultado debería ser -32 y no cabe (el mínimo de C1 con 6 bits es -31). **Desbordamiento.** En C2: $110100 + 101100 = 1\ 100000$, que es -32 y sí cabe. **Sin desbordamiento.**

2 Caso $(+3) + (+3)$

Con el menor n (3 bits) hay desbordamiento en los dos códigos: $011 + 011 = 110$, que se leería como negativo. Con 4 bits ($0011 + 0011 = 0110 = +6$) el resultado es correcto. Elegir n bien evita el desbordamiento.

3 Todas las sumas

Código	Suma	n	A	B	A+B	Real	Leído	Desborda
C1	+5 + −3	4	0101	1100	0010	+2	+2	no

Código	Suma	n	A	B	A+B	Real	Leído	Desborda
C2	+5 + −3	4	0101	1101	0010	+2	+2	no
C1	−5 + +3	4	1010	0011	1101	−2	−2	no
C2	−5 + +3	4	1011	0011	1110	−2	−2	no
C1	0 + −12	5	00000	10011	10011	−12	−12	no
C2	0 + −12	5	00000	10100	10100	−12	−12	no
C1	−12 + −20	6	110011	101011	011111	−32	+31	sí
C2	−12 + −20	6	110100	101100	100000	−32	−32	no
C1	+3 + +3	3	011	011	110	+6	−1	sí
C2	+3 + +3	3	011	011	110	+6	−2	sí
C1	−37 + +37	7	1011010	0100101	1111111	0	−0	no
C2	−37 + +37	7	1011011	0100101	0000000	0	0	no
C1	+3 + +3	4	0011	0011	0110	+6	+6	no
C2	+3 + +3	4	0011	0011	0110	+6	+6	no

✅ **Resultado final**

La regla «acarreo que entra en el bit de signo distinto del que sale» coincide en todos los casos con «el resultado real no cabe en n bits».

⚠️ **Posible errata en el libro**

Para $(-37)+(+37)$ el libro marca «desborde». Aplicando la regla no lo hay: en C1 no hay ningún acarreo y el resultado es $1111111 = -0$; en C2 el acarreo que entra en el signo y el que sale son ambos 1 y el resultado es 0000000. El 0 es representable.

Libro · Cap. 1 · P1.13

Suma en decimal con signo en complemento a 10

✔ verificado

Suma en decimal con signo en complemento a 10 (cuatro dígitos) las parejas $99 + 103$, $-99 + 103$ y $99 - 103$. Indica los dos últimos acarreos para decidir el desbordamiento.

🧭 Antes de empezar: qué piden y qué teoría usas

C10: un positivo se escribe con un 0 delante y un negativo es el complemento a 10 del positivo (empieza por 9). Se suma dígito a dígito, se tira el acarreo final y se mira si el acarreo que entra en el dígito de signo coincide con el que sale.

1 Codificación

$+99 = 0099$, $+103 = 0103$, $-99 = 10^4 - 99 = 9901$, $-103 = 9897$.

2 Sumas

Operación	A	B	A+B	Decimal	Acarreos (entra y sale)	Desborda
+99 + +103	0099	0103	0202	+202	0 y 0	no
−99 + +103	9901	0103	0004	+4	1 y 1	no
+99 + −103	0099	9897	9996	−4	0 y 0	no

3 Ejemplo dígito a dígito: $9901 + 0103$

$1 + 3 = 4$; $0 + 0 = 0$; $9 + 1 = 10$ (llevo 1); $9 + 0 + 1 = 10$ (llevo 1, se tira). Resultado $0004 = +4$, con acarreos 1 y 1: no hay desbordamiento.

✅ Resultado final

Resultados: $+202$, $+4$ y -4. La regla de los dos acarreos se ha comprobado de forma exhaustiva para todas las sumas de 2 y 3 dígitos.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 2 · Manipulación booleana

12 problemas resueltos: tablas de verdad, teoremas, minitérminos y maxitérminos, De Morgan, implementación con NAND y con NOR y análisis de circuitos.

💡 Cómo se usa esta sección

Son los problemas resueltos del capítulo 2 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras y resueltos de nuevo. Cada igualdad y cada tabla se ha comprobado con un programa que recorre todas las combinaciones de entrada. Si el libro difiere, aparece una nota «Ojo». La teoría está en el tema 3.

Libro · Cap. 2 · P2.1

Tablas de verdad de ocho funciones de tres variables

✔ verificado

Obtén la columna de salida de ocho funciones de A, B, C, desde el simple producto ABC hasta expresiones con complementos anidados.

🧭 Antes de empezar: qué piden y qué teoría usas

Método: se sustituye cada combinación ABC en la expresión y se opera de dentro hacia fuera. Recuerda que $0 \cdot x = 0$ y $1 + x = 1$: muchas filas salen a simple vista.

1 Las expresiones

- $f_1 = ABC$
- $f_2 = A + B + C$
- $f_3 = A(B + \bar{C})$
- $f_4 = \overline{(A + B)(B + C)}$
- $f_5 = \bar{B}\,(A + C\,\overline{(A + B)})$
- $f_6 = \overline{(A + C) \cdot \overline{B(C + B)}}$
- $f_7 = (A + B)(\bar{A} + \bar{B})$
- $f_8 = A + B\,(A + C\,B(A + B))$

2 Un ejemplo paso a paso: $f_6(001)$

Con $A=0, B=0, C=1$: $A+C=1$; $B(C+B)=0\cdot 1=0$; su complemento es 1; el producto es $1\cdot 1=1$ y el complemento exterior da 0. Así que $f_6(001)=0$.

3 Tabla de verdad

ABC	f1	f2	f3	f4	f5	f6	f7	f8
000	0	0	0	1	0	1	0	0
001	0	1	0	1	1	0	0	0
010	0	1	0	0	0	1	1	0
011	0	1	0	0	0	1	1	1
100	0	1	1	1	1	0	1	1
101	0	1	0	0	1	0	1	1
110	0	1	1	0	0	1	0	1
111	1	1	1	0	0	1	0	1

4 Formas más cortas

Mirando la tabla se reconocen formas simples. Todas se han comprobado por código:

- $f_3 = A(B+\bar{C})$ no se puede abreviar más.
- $f_4 = \bar{A}\bar{B}+\bar{B}\bar{C}$
- $f_5 = \bar{B}\,(A+C)$
- $f_6 = B+\bar{A}\bar{C}$
- $f_7 = A\oplus B$
- $f_8 = A+BC$

✅ Resultado final

Las ocho columnas coinciden con las del libro. Las expresiones largas (f_5, f_6, f_8) se reducen mucho: la absorción hace que partes enteras sobren.

Libro · Cap. 2 · P2.2 Demostrar los teoremas básicos por inducción perfecta

✔ verificado

Demuestra con tablas de verdad (inducción perfecta) los teoremas de una, dos y tres variables: neutros, complementos, idempotencia, elementos nulos, absorción, De Morgan y distributivas.

🧭 Antes de empezar: qué piden y qué teoría usas

Inducción perfecta: un teorema booleano es cierto si las dos columnas que se comparan son idénticas en **todas** las combinaciones de entrada. Con n variables hay 2^n filas.

1 Resultado de cada comprobación

Teorema	Un lado	El otro lado	Filas	Iguales
Elemento neutro de la suma	$0 + X$	X	2	sí
Elemento neutro del producto	$1 \cdot X$	X	2	sí
Complemento de la suma	$X + \bar{X}$	1	2	sí
Complemento del producto	$X \cdot \bar{X}$	0	2	sí
Idempotencia de la suma	$X + X$	X	2	sí
Idempotencia del producto	$X \cdot X$	X	2	sí
Elemento nulo de la suma	$X + 1$	1	2	sí
Elemento nulo del producto	$X \cdot 0$	0	2	sí
Absorción (1)	$X + XY$	X	4	sí
Absorción (2)	$X(X + Y)$	X	4	sí
De Morgan de la suma	$\overline{X + Y}$	$\bar{X}\bar{Y}$	4	sí
De Morgan del producto	$\overline{XY}$	$\bar{X} + \bar{Y}$	4	sí
Distributiva del producto	$X(Y + Z)$	$XY + XZ$	8	sí
Distributiva de la suma	$X + YZ$	$(X + Y)(X + Z)$	8	sí
De Morgan extendido (suma)	$\overline{X + Y + Z}$	$\bar{X}\bar{Y}\bar{Z}$	8	sí

Teorema	Un lado	El otro lado	Filas	Iguales
De Morgan extendido (producto)	$\overline{XYZ}$	$\bar{X}+\bar{Y}+\bar{Z}$	8	sí

2 **Ejemplo: De Morgan con dos variables**

XY	$\overline{X+Y}$	$\bar{X}\bar{Y}$	$\overline{XY}$	$\bar{X}+\bar{Y}$
00	1	1	1	1
01	0	0	1	1
10	0	0	1	1
11	0	0	0	0

Las columnas 2 y 3 coinciden, y las columnas 4 y 5 también.

3 **Qué hacer con un teorema que no conoces**

Haz su tabla de verdad. Si las dos columnas son iguales, es un teorema, aunque no sepas por qué. Si difieren en una sola fila, es falso.

✅ **Resultado final**

Los 16 teoremas se cumplen en todas sus filas.

> ⚠️ **Errata del libro**
>
> En la tabla del elemento nulo, el libro imprime un 1 en las dos filas de la columna del valor constante 0 de $X \cdot 0$. La columna del valor constante debe valer 0, igual que el resultado de $X \cdot 0$. Es un error de impresión: el teorema es correcto.

`Libro · Cap. 2 · P2.3` **Minitérminos, maxitérminos, Σ y Π de cuatro funciones**

✔ verificado

Con la tabla de verdad de cuatro funciones de A, B, C, D (dos de ellas con combinaciones indiferentes), escribe cada una como suma de minitérminos y como producto de maxitérminos, en forma compacta Σ y Π, y cuenta las puertas de los dos circuitos.

🧭 Antes de empezar: qué piden y qué teoría usas

Minitérmino m_i: producto de las 4 variables que vale 1 solo en la fila i (la variable va sin negar si su bit es 1). **Maxitérmino M_i:** suma de las 4 variables que vale 0 solo en la fila i (la variable va negada si su bit es 1). Una función es la suma de los m_i de sus unos o el producto de los M_i de sus ceros. Una X no se escribe en ninguna de las dos formas.

1 Tabla de verdad con minitérminos y maxitérminos

i	ABCD	Minitérmino	Maxitérmino	f1	f2	f3	f4
0	0000	$\bar{A}\bar{B}\bar{C}\bar{D}$	$A+B+C+D$	0	1	1	X
1	0001	$\bar{A}\bar{B}\bar{C}D$	$A+B+C+\bar{D}$	1	1	0	X
2	0010	$\bar{A}\bar{B}C\bar{D}$	$A+B+\bar{C}+D$	1	0	1	X
3	0011	$\bar{A}\bar{B}CD$	$A+B+\bar{C}+\bar{D}$	1	1	1	0
4	0100	$\bar{A}B\bar{C}\bar{D}$	$A+\bar{B}+C+D$	0	1	X	0
5	0101	$\bar{A}B\bar{C}D$	$A+\bar{B}+C+\bar{D}$	1	1	0	0
6	0110	$\bar{A}BC\bar{D}$	$A+\bar{B}+\bar{C}+D$	0	0	X	1
7	0111	$\bar{A}BCD$	$A+\bar{B}+\bar{C}+\bar{D}$	0	0	1	1
8	1000	$A\bar{B}\bar{C}\bar{D}$	$\bar{A}+B+C+D$	1	0	X	1
9	1001	$A\bar{B}\bar{C}D$	$\bar{A}+B+C+\bar{D}$	0	1	X	0
10	1010	$A\bar{B}C\bar{D}$	$\bar{A}+B+\bar{C}+D$	0	1	X	X
11	1011	$A\bar{B}CD$	$\bar{A}+B+\bar{C}+\bar{D}$	0	1	0	1
12	1100	$AB\bar{C}\bar{D}$	$\bar{A}+\bar{B}+C+D$	0	1	0	1
13	1101	$AB\bar{C}D$	$\bar{A}+\bar{B}+C+\bar{D}$	0	1	0	1
14	1110	$ABC\bar{D}$	$\bar{A}+\bar{B}+\bar{C}+D$	0	0	1	X
15	1111	$ABCD$	$\bar{A}+\bar{B}+\bar{C}+\bar{D}$	1	1	1	0

2 Forma normal disyuntiva: suma de minitérminos

- $f1 = \Sigma(1\text{-}3, 5, 8, 15)$
- $f2 = \Sigma(0, 1, 3\text{-}5, 9\text{-}13, 15)$
- $f3 = \Sigma(0, 2, 3, 7, 14, 15)$ con indiferentes $d(4, 6, 8\text{-}10)$
- $f4 = \Sigma(6\text{-}8, 11\text{-}13)$ con indiferentes $d(0\text{-}2, 10, 14)$

3 Forma normal conjuntiva: producto de maxitérminos

- $f1 = \Pi(0, 4, 6, 7, 9\text{-}14)$
- $f2 = \Pi(2, 6\text{-}8, 14)$
- $f3 = \Pi(1, 5, 11\text{-}13)$ con indiferentes $d(4, 6, 8\text{-}10)$
- $f4 = \Pi(3\text{-}5, 9, 15)$ con indiferentes $d(0\text{-}2, 10, 14)$

4 Circuitos

Cada minitérmino es una puerta AND de 4 entradas y todas van a una OR. Cada maxitérmino es una OR de 4 entradas y todas van a una AND.

Función	AND de 4 entradas	OR final	OR de 4 entradas	AND final
f1	6	1	10	1
f2	11	1	5	1
f3	6	1	5	1
f4	6	1	5	1

Se necesitan además 4 inversores para tener $\bar{A}, \bar{B}, \bar{C}, \bar{D}$. Con indiferentes conviene usarlas luego al simplificar (tema de Karnaugh), pero en esta forma canónica no se escriben.

✅ Resultado final

Se han obtenido los cuatro Σ y los cuatro Π y coinciden con los del libro. Se ha comprobado que los minitérminos, los maxitérminos y las indiferentes de cada función forman justo las 16 filas.

`Libro · Cap. 2 · P2.4`

Teorema de De Morgan sobre sumas y productos

✔ verificado

Aplica el teorema de De Morgan a diez expresiones: un complemento exterior sobre una suma de productos se convierte en un producto de sumas, y al revés.

🧭 Antes de empezar: qué piden y qué teoría usas

Regla: para negar una expresión, cambia cada $+$ por $\cdot$, cada $\cdot$ por $+$, y niega cada variable. Cuida los paréntesis. Para fiarte de un resultado, compara las tablas de verdad de los dos lados.

1 Las diez igualdades

- $f1 = \overline{AB + CD} = (\bar{A} + \bar{B})(\bar{C} + \bar{D})$
- $f2 = \overline{\bar{A}\bar{B} + \bar{D}} = (A + B)\, D$
- $f3 = \overline{A + BC + BD + \bar{B}\bar{D} + \bar{A}\bar{B}} = \bar{A}(\bar{B} + \bar{C})(\bar{B} + \bar{D})(B + D)(A + B)$
- $f4 = \overline{ABCD + \bar{A}\bar{B}\bar{C}\bar{D}} = (\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + B + C + D)$
- $f5 = \overline{\bar{A} + \bar{B}\bar{C} + \bar{B}C\bar{D} + A\bar{C}\bar{D}} = A(B + C)(B + \bar{C} + D)(\bar{A} + C + D)$
- $f6 = \overline{AB + AC + A\bar{B}\bar{C}D + \bar{A}\bar{C}} = (\bar{A} + \bar{B})(\bar{A} + \bar{C})(\bar{A} + B + C + \bar{D})(A + C)$
- $f7 = \overline{ABCD} = \bar{A} + \bar{B} + \bar{C} + \bar{D}$
- $f8 = \overline{(A + B)(C + D)} = \bar{A}\bar{B} + \bar{C}\bar{D}$
- $f9 = \overline{(\bar{A} + \bar{C})(\bar{A} + D)(\bar{A} + C + \bar{D})(B + D)} = AC + A\bar{D} + A\bar{C}D + \bar{B}\bar{D}$
- $f10 = \overline{(B + D)\, A\, (A + \bar{C} + D)(\bar{A} + C + D)} = \bar{B}\bar{D} + \bar{A} + \bar{A}C\bar{D} + A\bar{C}\bar{D}$

2 Cómo se comprueba

Se ha recorrido por código toda la tabla de 16 filas de cada pareja de expresiones. En las diez, los dos lados dan la misma salida en todas las filas.

✅ Resultado final

Las diez igualdades son correctas.

⚠️ Errata del libro

En el apartado f_4 el libro escribe $(\bar{A} + \bar{B} + \bar{C} + \bar{C})(A + B + C + D)$ con la C negada dos veces. El factor correcto es $\bar{A} + \bar{B} + \bar{C} + \bar{D}$. Con la versión impresa las dos expresiones **no** son iguales (se comprueba mirando la fila $ABCD = 1110$).

Libro · Cap. 2 · P2.5

De Morgan con complementos anidados

✔ verificado

Aplica De Morgan a expresiones con paréntesis y varios niveles de complemento, de modo que al final solo queden variables negadas (nunca una barra sobre una suma o un producto).

🧭 Antes de empezar: qué piden y qué teoría usas

Truco: trabaja de fuera hacia dentro. Quita la barra exterior con la regla de De Morgan y repite el proceso con cada barra que quede dentro. Un complemento doble desaparece: $\bar{\bar{x}} = x$.

1 Los cinco primeros apartados

- $f_1 = \overline{AB + C(A + \bar{B}) + D(A + \bar{C})} = (\bar{A} + \bar{B})(\bar{C} + \bar{A}B)(\bar{D} + \bar{A}C)$
- $f_2 = \overline{B(C + D(A + \bar{B})) + \bar{A}\bar{B}} = (\bar{B} + \bar{C}(\bar{D} + \bar{A}B))(A + B)$
- $f_3 = \overline{BC + A\bar{D}\,(B + \overline{A(B + D)})} = (\bar{B} + \bar{C})(\bar{A} + D + \bar{B}A(B + D))$
- $f_4 = \overline{A + B(C + D(A + \bar{B}))} = \bar{A}(\bar{B} + \bar{C}(\bar{D} + \bar{A}B))$
- $f_5 = \overline{\overline{ABCD} + \overline{A + B + C + D}} = ABCD\,(A + B + C + D)$

2 Dos ejercicios propios del mismo estilo

- $g_1 = \overline{\overline{AB} + \bar{C}D} = AB\,(C + \bar{D})$
- $g_2 = \overline{\overline{A + B} \cdot \overline{C\bar{D}}} = A + B + C\bar{D}$

Sustituyen a los apartados f_6 y f_7 del libro, que encadenan tres complementos y cuya lectura en la copia que he usado no es fiable. La técnica es idéntica.

3 Una observación

En f_5 el resultado se reduce a $ABCD$: si las cuatro variables valen 1, la suma $A + B + C + D$ vale 1 también, y en cualquier otro caso $ABCD = 0$.

✅ Resultado final

Las siete igualdades se han comprobado en las 16 filas de la tabla.

Libro · Cap. 2 · P2.6

Implementar sumas de productos solo con puertas NAND

✔ verificado

Reescribe cinco sumas de productos de modo que solo hagan falta puertas NAND (dos niveles) y cuenta las puertas.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: si niegas dos veces una suma de productos y aplicas De Morgan una vez, queda un NAND de NANDs: $\overline{\overline{t_1}\cdot\overline{t_2}\cdots} = t_1 + t_2 + \cdots$. Cada producto de varias variables es una NAND de primer nivel. Un término de una sola variable entra directamente negado en la NAND de salida.

1 Las cinco funciones en forma NAND

- $f1 = AB + CD = \overline{\overline{AB}\,\overline{CD}}$
- $f2 = \bar{A}\bar{B} + \bar{A}BCD + B\bar{C}\bar{D} = \overline{\overline{\bar{A}\bar{B}}\,\overline{\bar{A}BCD}\,\overline{B\bar{C}\bar{D}}}$
- $f3 = A + BC + \bar{D} = \overline{\bar{A}\,\overline{BC}\,D}$
- $f4 = \bar{A}\bar{D} + ABCD + \bar{A}\bar{C} = \overline{\overline{\bar{A}\bar{D}}\,\overline{ABCD}\,\overline{\bar{A}\bar{C}}}$
- $f5 = ABCD + AB\bar{C} + \bar{A}C\bar{D} + B\bar{C}\bar{D} = \overline{\overline{ABCD}\,\overline{AB\bar{C}}\,\overline{\bar{A}C\bar{D}}\,\overline{B\bar{C}\bar{D}}}$

2 Puertas necesarias

Función	NAND (incluida la de salida)	Inversores
f1	3	0
f2	4	3
f3	2	1
f4	4	3
f5	5	3

Los inversores sirven para generar las variables negadas que usan las NAND de primer nivel. En f_3 solo hace falta negar A; $\bar{D}$ ya se compensa porque $\overline{\bar{D}} = D$ entra directamente.

3 Comprobación

Para cada función se ha evaluado la forma con NAND y la suma de productos en las 16 combinaciones: coinciden.

✅ **Resultado final**

Todas las implementaciones NAND son equivalentes a la suma de productos original.

`Libro · Cap. 2 · P2.7`

Implementar productos de sumas solo con puertas NOR

✔ verificado

Reescribe cinco productos de sumas de modo que solo hagan falta puertas NOR (dos niveles) y cuenta las puertas.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Idea: es el dual del problema anterior. Un producto de sumas $(s_1)(s_2)\cdots$ es igual a $\overline{\overline{s_1}+\overline{s_2}+\cdots}$. Cada suma de varias variables es una NOR de primer nivel; un factor de una sola variable entra negado en la NOR de salida.

1 Las cinco funciones en forma NOR

- $f1 = (A+B)(C+D) = \overline{\overline{A+B}+\overline{C+D}}$
- $f2 = (\bar{A}+\bar{D})(B+\bar{C}+\bar{D})(A+\bar{B}+\bar{C}+D) = \overline{\overline{\bar{A}+\bar{D}}+\overline{B+\bar{C}+\bar{D}}+\overline{A+\bar{B}+\bar{C}+D}}$
- $f3 = (\bar{B}+\bar{D})(\bar{B}+\bar{C})(\bar{A}+\bar{D})(B+D) = \overline{\overline{\bar{B}+\bar{D}}+\overline{\bar{B}+\bar{C}}+\overline{\bar{A}+\bar{D}}+\overline{B+D}}$
- $f4 = (\bar{B}+\bar{C}+D)A(B+\bar{D})B = \overline{\overline{\bar{B}+\bar{C}+D}+\bar{A}+\overline{B+\bar{D}}+\bar{B}}$
- $f5 = (\bar{A}+\bar{B}+\bar{D})(A+C)(B+\bar{D})(\bar{A}+\bar{D}) = \overline{\overline{\bar{A}+\bar{B}+\bar{D}}+\overline{A+C}+\overline{B+\bar{D}}+\overline{\bar{A}+\bar{D}}}$

2 Puertas necesarias

Función	NOR (incluida la de salida)	Inversores
f1	3	0
f2	4	4
f3	5	4
f4	3	4

Función	NOR (incluida la de salida)	Inversores
f5	5	3

3 Un detalle

En f_4 aparecen los factores sueltos A y B. Como $A\,B\,(\ldots) = 1$ exige $A = B = 1$, la función se reduce a $AB(\bar{C} + D)$ (se ha comprobado en las 16 filas). El circuito del enunciado funciona igual, pero está lejos de ser mínimo.

✅ Resultado final

Todas las implementaciones NOR son equivalentes al producto de sumas de partida.

`Libro · Cap. 2 · P2.8`

Analizar tres circuitos y obtener su tabla de verdad

✔ verificado

Obtén la tabla de verdad de tres circuitos de tres entradas: uno con inversores, tres puertas AND y una OR; otro con tres NAND y una NAND de salida; y un sumador completo con puertas XOR, AND y OR.

🧭 Antes de empezar: qué piden y qué teoría usas

Método: fija una combinación en las entradas, propaga el valor puerta a puerta hasta la salida y apúntalo. Repite para las 8 combinaciones. Es lento pero seguro.

1 Circuito AND–OR

Las tres puertas AND calculan $\bar{X}\bar{Y}$, XY y XZ y una OR las suma: $f_1 = \bar{X}\bar{Y} + XY + XZ$.

2 Circuito NAND–NAND

Con las mismas tres funciones en NAND de primer nivel y una NAND de salida, $f_2 = \overline{\overline{\bar{X}\bar{Y}} \cdot \overline{XY} \cdot \overline{XZ}}$. Por De Morgan es igual a f_1: la tabla es idéntica.

3 Sumador completo

Con sumandos A_i, B_i y acarreo de entrada C_{i-1}: $S_i = A_i \oplus B_i \oplus C_{i-1}$ y $C_i = A_i B_i + (A_i \oplus B_i)\, C_{i-1}$. Comprobación: $A_i + B_i + C_{i-1} = 2C_i + S_i$ en las 8 filas.

4 Tabla de verdad conjunta

XYZ = $A_i B_i C_{i-1}$	f_1	f_2	S_i	C_i
000	1	1	0	0
001	1	1	1	0
010	0	0	1	0
011	0	0	0	1
100	0	0	1	0
101	1	1	0	1
110	1	1	0	1
111	1	1	1	1

✅ **Resultado final**

$f_1 = f_2$, y el sumador completo da S_i = paridad de los tres bits y C_i = mayoría de los tres.

`Libro · Cap. 2 · P2.9`

Analizar un circuito de 4 entradas y 4 salidas

✔ verificado

Un circuito recibe un dígito BCD $B_3B_2B_1B_0$ y produce $X_3X_2X_1X_0$ con seis puertas AND, tres OR y un inversor directo. Obtén su tabla de verdad con las 16 combinaciones y di qué hace.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Método: primero escribe la función de cada salida leyendo las puertas, después evalúa. Es más rápido que propagar bit a bit las 16 filas.

1 Funciones de salida

- $X_3 = B_3 + B_2B_0$
- $X_2 = \bar{B}_2B_0 + B_2\bar{B}_1\bar{B}_0 + \bar{B}_2B_1$

- $X_1 = B_1B_0 + \bar{B}_1\bar{B}_0$ (es una XNOR)
- $X_0 = \bar{B}_0$

2 Tabla de verdad del circuito

$B_3B_2B_1B_0$	Salida $X_3X_2X_1X_0$	BCD exceso 3 ($d+3$)	Coincide
0000	0011	0011	ok
0001	0100	0100	ok
0010	0101	0101	ok
0011	0110	0110	ok
0100	0111	0111	ok
0101	1000	1000	ok
0110	0001	1001	error
0111	1010	1010	ok
1000	1011	1011	ok
1001	1100	1100	ok
1010	1101	—	sin uso
1011	1110	—	sin uso
1100	1111	—	sin uso
1101	1000	—	sin uso
1110	1001	—	sin uso
1111	1010	—	sin uso

3 Conclusión

Casi todas las filas válidas (0 a 9) coinciden con el código **exceso 3**: el circuito es un convertidor de BCD natural a BCD exceso 3, salvo en el dígito 6. Las filas 1010 a 1111 no son dígitos BCD: su salida no importa.

✅ **Resultado final**

Es un convertidor BCD → exceso 3 incompleto.

⚠️ **Errata del libro**

(1) Falta un término en X_3: el valor correcto es $X_3 = B_3 + B_2B_1 + B_2B_0$. Con el circuito del libro el dígito 6 (0110) da 0001 en lugar de 1001. El libro afirma que la tabla es la de un conversor a exceso 3 y no menciona este fallo. (2) En la fila 1100 la tabla del libro imprime 1011, pero el circuito da 1111 (la puerta de X_2 detecta $B_2\bar{B}_1\bar{B}_0$). Esta segunda diferencia no afecta a ningún dígito BCD válido.

`Libro · Cap. 2 · P2.10`

Un codificador de 8 a 3 construido con puertas OR

✔ verificado

Un bloque tiene 8 entradas $E_7 \ldots E_0$ y tres salidas, cada una con una OR de cuatro entradas. Obtén sus tablas con las ocho combinaciones de «un solo 1» y con algunas más con varios unos, y di qué hace.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Cómo se lee: cada salida es una OR de las entradas cuyo número tiene un 1 en esa posición. $S_2 = E_7 + E_6 + E_5 + E_4$, $S_1 = E_7 + E_6 + E_3 + E_2$, $S_0 = E_7 + E_5 + E_3 + E_1$.

1 **Tabla de verdad (selección)**

$E_7 \ldots E_0$	$S_2S_1S_0$
10000000	111
01000000	110
00100000	101
00010000	100
00001000	011
00000100	010
00000010	001

$E_7 \dots E_0$	$S_2 S_1 S_0$
00000001	000
00000000	000
00011100	111
10000001	111
11111111	111

2 Conclusión

Con un solo 1 la salida es el número binario de la entrada activa. Con varios unos la salida es la **OR bit a bit** de sus códigos (por ejemplo $00011100 \rightarrow 100 \,|\, 011 \,|\, 010 = 111$). Es un **codificador sin prioridad** 8:3: solo vale si se activa una entrada cada vez.

✅ **Resultado final**

Codificador 8:3 sin prioridad. Las 12 filas verificadas coinciden con las del libro.

`Libro · Cap. 2 · P2.11`

Un decodificador de 3 a 8 con habilitación

✔ verificado

Un circuito con entradas E_I, A_2, A_1, A_0 y ocho salidas $S_7 \dots S_0$ tiene ocho puertas AND de cuatro entradas. Obtén su tabla de verdad y di qué función realiza.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Lectura: la salida S_k es una AND de E_I con A_2, A_1, A_0 (cada una directa o negada) tal que vale 1 solo cuando $A_2 A_1 A_0$ es el número k en binario.

1 Salidas

$S_k = E_I \cdot m_k(A_2, A_1, A_0)$ para $k = 0 \dots 7$, donde m_k es el minitérmino de índice k.

2 Tabla de verdad

E_I	$A_2A_1A_0$	$S_7S_6S_5S_4S_3S_2S_1S_0$
0	000	00000000
0	001	00000000
0	010	00000000
0	011	00000000
0	100	00000000
0	101	00000000
0	110	00000000
0	111	00000000
1	000	00000001
1	001	00000010
1	010	00000100
1	011	00001000
1	100	00010000
1	101	00100000
1	110	01000000
1	111	10000000

✅ **Resultado final**

Con $E_I = 0$ todas las salidas valen 0; con $E_I = 1$ se activa solo la salida cuyo número es $A_2A_1A_0$. Es un **decodificador 3:8 con habilitación**.

`Libro · Cap. 2 · P2.12` **Un generador de paridad de 7 bits** ✔ verificado

Un circuito tiene 7 entradas $D_6 \ldots D_0$ y una salida BPI_1. Se forma con seis XOR en árbol (tres de primer nivel, dos de segundo y una de tercero que reciben D_6 directamente) y un inversor final. Obtén su tabla de verdad con varias combinaciones y di qué hace.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: la XOR de varios bits vale 1 si el número de unos es impar. Con el inversor al final, $BPI_1 = 1$ cuando el número de unos de $D_6 \dots D_0$ es **par**.

1 Tabla de verdad (selección)

$D_6 \dots D_0$	Unos	Salida calculada	Salida del libro	Estado
0000000	0	1	1	ok
1010101	4	1	1	ok
0101010	3	0	0	ok
1111000	4	1	1	ok
0000111	3	0	0	ok
0110100	3	0	0	ok
0000011	2	1	1	ok
1110001	4	1	1	ok
1111110	6	1	0	**error del libro**
1111100	5	0	0	ok
0000001	1	0	0	ok
1111111	7	0	0	ok

2 Conclusión

Si añades BPI_1 a los 7 bits, el total de unos siempre es **impar**. Se ha verificado en las 128 combinaciones. Es un **generador de paridad impar** (bit de paridad impar).

✅ Resultado final

Generador de paridad impar para 7 bits de datos.

> **⚠️ Errata del libro**
>
> En la fila 1111110 el libro imprime $BPI_1 = 0$. Tiene 6 unos (par), así que el circuito da 1. El resto de filas coinciden.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 3 · Mapas de Karnaugh

Dos problemas, con 28 mapas para dibujar y 60 funciones para simplificar: con unos, con indiferentes y como producto de sumas.

💡 Cómo se usa esta sección

Son los problemas resueltos del capítulo 3 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras. Cada simplificación se ha calculado con un minimizador y se ha comparado con la solución del libro: se comprueba que cubre todos los 1, que no cubre ningún 0 y que tiene el mismo coste que el mínimo. Los mapas de esta guía ponen AB en las filas y CD en las columnas. El libro pone A, C en las columnas y B, D en las filas: es el mismo mapa con los ejes cambiados (cada casilla lleva escrito su número de minitérmino). La teoría está en el tema 4.

`Libro · Cap. 3 · P3.1`

Dibujar 28 mapas de Karnaugh a partir de formas normales

✔ verificado

Dibuja el mapa de Karnaugh de 28 funciones de cuatro variables, dadas como suma de minitérminos Σ o como producto de maxitérminos Π, algunas con combinaciones indiferentes.

🧭 Antes de empezar: qué piden y qué teoría usas

Cómo se coloca cada minitérmino: el número $m = 8A + 4B + 2C + D$. Las filas y las columnas del mapa siguen el orden de Gray $(00, 01, 11, 10)$, de modo que dos casillas vecinas se diferencian en una sola variable. En un Σ se escribe 1 en las casillas de la lista; en un Π la lista son los **ceros** y el resto son unos; las X de $d(\dots)$ van aparte.

1 Funciones 1 a 10 (sumatorios sin indiferentes)

$f_1 = \Sigma(0, 2, 4, 6, 8, 10)$

AB\CD

	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4 **1**	5	7	6 **1**
11	12	13	15	14
10	8 **1**	9	11	10 **1**

f1

Los 1 son las casillas de Σ.

$f_2 = \Sigma(4, 8\text{-}12, 15)$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7	6
11	12 **1**	13	15 **1**	14
10	8 **1**	9 **1**	11 **1**	10 **1**

f2

Los 1 son las casillas de Σ.

$f_3 = \Sigma(1, 4, 8, 9)$

AB\CD

	00	01	11	10
00	0	1 **1**	3	2
01	4 **1**	5	7	6
11	12	13	15	14
10	8 **1**	9 **1**	11	10

f3

Los 1 son las casillas de Σ.

$f_4 = \Sigma(2, 3, 7, 8, 10)$

AB\CD

	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4	5	7 **1**	6
11	12	13	15	14
10	8 **1**	9	11	10 **1**

f4

Los 1 son las casillas de Σ.

$f_5 = \Sigma(1, 5, 9, 13)$

AB\CD	00	01	11	10
00	0	1 **1**	3	2
01	4	5 **1**	7	6
11	12	13 **1**	15	14
10	8	9 **1**	11	10

f5

Los 1 son las casillas de Σ.

$f_6 = \Sigma(8, 10, 12, 14)$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12 **1**	13	15	14 **1**
10	8 **1**	9	11	10 **1**

f6

Los 1 son las casillas de Σ.

$f_7 = \Sigma(1, 13\text{-}15)$

AB\CD

	00	01	11	10
00	0	1 **1**	3	2
01	4	5	7	6
11	12	13 **1**	15 **1**	14 **1**
10	8	9	11	10

f7

Los 1 son las casillas de Σ.

$f_8 = \Sigma(1\text{-}9, 12, 15)$

AB\CD

	00	01	11	10
00	0	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13	15 **1**	14
10	8 **1**	9 **1**	11	10

f8

Los 1 son las casillas de Σ.

$f_9 = \Sigma(1, 3, 5, 7, 9, 11, 13, 15)$

AB\CD

	00	01	11	10
00	0	1 **1**	3 **1**	2
01	4	5 **1**	7 **1**	6
11	12	13 **1**	15 **1**	14
10	8	9 **1**	11 **1**	10

f9

Los 1 son las casillas de Σ.

$f_{10} = \Sigma(0, 2, 4, 6, 8, 10, 12, 14)$

AB\CD

	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4 **1**	5	7	6 **1**
11	12 **1**	13	15	14 **1**
10	8 **1**	9	11	10 **1**

f10

Los 1 son las casillas de Σ.

2 Funciones 11 a 14 (sumatorios con indiferentes)

$f_{11} = \Sigma(1, 8, 10, 15)$ con indiferentes $d(0, 2, 6, 7)$

AB\CD

	00	01	11	10
00	0 ×	1 **1**	3	2 ×
01	4	5	7 ×	6 ×
11	12	13	15 **1**	14
10	8 **1**	9	11	10 **1**

f11

Los 1 son las casillas de Σ; × indiferente.

$f_{12} = \Sigma(3, 4, 13, 14)$ con indiferentes $d(7\text{-}12)$

AB\CD

AB \ CD	00	01	11	10
00	0	1	3 **1**	2
01	4 **1**	5	7 ×	6
11	12 ×	13 **1**	15	14 **1**
10	8 ×	9 ×	11 ×	10 ×

f12

Los 1 son las casillas de Σ; × indiferente.

$f_{13} = \Sigma(1, 2, 5, 12)$ con indiferentes $d(0, 4)$

AB\CD

	00	01	11	10
00	0 ×	1 **1**	3	2 **1**
01	4 ×	5 **1**	7	6
11	12 **1**	13	15	14
10	8	9	11	10

f13

Los 1 son las casillas de Σ; × indiferente.

$f_{14} = \Sigma(0\text{-}4, 8\text{-}12)$ con indiferentes $d(5\text{-}7)$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 ×	7 ×	6 ×
11	12 **1**	13	15	14
10	8 **1**	9 **1**	11 **1**	10 **1**

f14

Los 1 son las casillas de Σ; × indiferente.

3 Funciones 15 a 24 (productos de maxitérminos)

$f_{15} = \Pi(0, 4, 6, 10)$

AB\CD	00	01	11	10
00	0 **1**	1	3	2
01	4 **1**	5	7	6 **1**
11	12	13	15	14
10	8	9	11	10 **1**

f15

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{16} = \Pi(0, 2, 4, 5, 7, 11)$

AB\CD	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4 **1**	5 **1**	7 **1**	6
11	12	13	15	14
10	8	9	11 **1**	10

f16

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{17} = \Pi(13\text{-}15)$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13 **1**	15 **1**	14 **1**
10	8	9	11	10

f17

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{18} = \Pi(1, 4, 8, 9, 14, 15)$

AB\CD	00	01	11	10
00	0	1 **1**	3	2
01	4 **1**	5	7	6
11	12	13	15 **1**	14 **1**
10	8 **1**	9 **1**	11	10

f18

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{19} = \Pi(0, 2, 4, 10\text{-}15)$

AB\CD

	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4 **1**	5	7	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11 **1**	10 **1**

f19

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{20} = \Pi(2, 3, 6, 7, 10, 11, 14, 15)$

AB\CD

	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4	5	7 **1**	6 **1**
11	12	13	15 **1**	14 **1**
10	8	9	11 **1**	10 **1**

f20

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{21} = \Pi(2, 6, 7, 11, 15)$

AB\CD

	00	01	11	10
00	0	1	3	2 **1**
01	4	5	7 **1**	6 **1**
11	12	13	15 **1**	14
10	8	9	11 **1**	10

f21

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{22} = \Pi(0\text{-}7)$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12	13	15	14
10	8	9	11	10

f22

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{23} = \Pi(0\text{-}3, 8\text{-}11)$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4	5	7	6
11	12	13	15	14
10	8 **1**	9 **1**	11 **1**	10 **1**

f23

Se marcan con 1 los ceros de Π, que es lo que se rodea.

$f_{24} = \Pi(0, 1, 4, 5, 8, 9, 12, 13)$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3	2
01	4 **1**	5 **1**	7	6
11	12 **1**	13 **1**	15	14
10	8 **1**	9 **1**	11	10

f24

Se marcan con 1 los ceros de Π, que es lo que se rodea.

4 **Funciones 25 a 28 (productos con indiferentes)**

$f_{25} = \Pi(3, 4, 9, 10)$ con indiferentes $d(7, 8, 13, 14)$

AB\CD

	00	01	11	10
00	0	1	3 **1**	2
01	4 **1**	5	7 ×	6
11	12	13 ×	15	14 ×
10	8 ×	9 **1**	11	10 **1**

f25

Se marcan con 1 los ceros de Π, que es lo que se rodea; × indiferente.

$f_{26} = \Pi(4\text{-}9, 13\text{-}15)$ con indiferentes $d(0\text{-}2)$

AB\CD

	00	01	11	10
00	0 ×	1 ×	3	2 ×
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12	13 **1**	15 **1**	14 **1**
10	8 **1**	9 **1**	11	10

f26

Se marcan con 1 los ceros de Π, que es lo que se rodea; × indiferente.

$f_{27} = \Pi(4, 5, 14, 15)$ con indiferentes $d(0\text{-}3, 7\text{-}9)$

AB\CD

AB \ CD	00	01	11	10
00	0 ×	1 ×	3 ×	2 ×
01	4 **1**	5 **1**	7 ×	6
11	12	13	15 **1**	14 **1**
10	8 ×	9 ×	11	10

f27

Se marcan con 1 los ceros de Π, que es lo que se rodea; × indiferente.

$f_{28} = \Pi(1, 3, 5, 13)$ con indiferentes $d(7\text{-}12)$

AB\CD	00	01	11	10
00	0	1 **1**	3 **1**	2
01	4	5 **1**	7 ×	6
11	12 ×	13 **1**	15	14
10	8 ×	9 ×	11 ×	10 ×

f28

Se marcan con 1 los ceros de Π, que es lo que se rodea; × indiferente.

✅ **Resultado final**

Los 28 mapas se han generado de la lista, y se ha comprobado que los unos, los ceros y las indiferentes de cada función suman las 16 casillas. Los mapas impresos en el libro que se han contrastado coinciden.

`Libro · Cap. 3 · P3.2 (f1–f22)` **Simplificar 22 mapas de Karnaugh con unos**

✔ verificado

Simplifica 22 funciones de cuatro variables dadas por su mapa de Karnaugh (sin indiferentes) y escribe la suma de productos mínima.

🧭 Antes de empezar: qué piden y qué teoría usas

Método: rodea todos los 1 con el menor número de lazos y del mayor tamaño posible (de 1, 2, 4, 8 o 16 casillas). Los lazos pueden solaparse y pueden dar la vuelta al mapa. Cada lazo da un producto con las variables que **no cambian** dentro de él.

1 Funciones 1 a 6

f_1: $f_1 = \overline{A}B + \overline{A}D + C + A\overline{D}$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4	5 **1**	7 **1**	6 **1**
11	12 **1**	13	15 **1**	14 **1**
10	8 **1**	9	11 **1**	10 **1**

f1

Se rodean los 1.

f_2: $f_2 = \overline{A}B + B\overline{C} + A\overline{B}$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14
10	8 **1**	9 **1**	11 **1**	10 **1**

f2

Se rodean los 1.

f_3: $f_3 = \overline{ABC} + \overline{A}CD + AB$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2
01	4	5	7 **1**	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11	10

f3

Se rodean los 1.

f_4: $f_4 = \overline{AB} + B\overline{C}D$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4	5 **1**	7	6
11	12	13 **1**	15	14
10	8	9	11	10

f4

Se rodean los 1.

f_5: $f_5 = \overline{A}B + A\overline{B}$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12	13	15	14
10	8 **1**	9 **1**	11 **1**	10 **1**

f5

Se rodean los 1.

f_6: $f_6 = \overline{A}\overline{B}C + C\overline{D} + B\overline{C} + A\overline{D}$ Hay 2 soluciones mínimas equivalentes.

AB\CD

AB \ CD	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4 **1**	5 **1**	7	6 **1**
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9	11	10 **1**

f6

Se rodean los 1.

2 Funciones 7 a 12

f_7: $f_7 = \overline{A}BC + A\overline{C} + A\overline{D}$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11	10 **1**

f7

Se rodean los 1.

f_8: $f_8 = \overline{ABC} + \overline{AB}D + \overline{A}BC + A\overline{D}$ Hay 4 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2
01	4	5	7 **1**	6 **1**
11	12 **1**	13	15	14 **1**
10	8 **1**	9	11	10 **1**

f8

Se rodean los 1.

f_9: $f_9 = \overline{C}D + \overline{A}BD + BC\overline{D} + A\overline{BD}$ Hay 4 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1 **1**	3	2
01	4	5 **1**	7 **1**	6 **1**
11	12	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11	10 **1**

f9

Se rodean los 1.

f_{10}: $f_{10} = \overline{A}CD + B\overline{C}D + A\overline{B}D + ABC$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1	3	2
01	4 **1**	5 **1**	7	6
11	12	13 **1**	15 **1**	14 **1**
10	8 **1**	9	11	10 **1**

f10

Se rodean los 1.

f_{11}: $f_{11} = \overline{ABD} + \overline{B}CD + B\overline{CD} + ABC$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1	3 **1**	2 **1**
01	4 **1**	5	7	6
11	12 **1**	13	15 **1**	14 **1**
10	8	9	11 **1**	10

f11

Se rodean los 1.

f_{12}: $f_{12} = B\overline{C}D + BC\overline{D} + A\overline{C}\overline{D} + ACD$

AB\CD	00	01	11	10
00	0	1	3	2
01	4	5 **1**	7	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9	11 **1**	10

f12

Se rodean los 1.

3 Funciones 13 a 18

f_{13}: $f_{13} = \overline{AC}D + B\overline{D} + A\overline{B}D$

AB\CD

	00	01	11	10
00	0	1 **1**	3	2
01	4 **1**	5 **1**	7	6 **1**
11	12 **1**	13	15	14 **1**
10	8	9 **1**	11 **1**	10

f13

Se rodean los 1.

f_{14}: $f_{14} = \overline{A}CD + \overline{A}B\overline{D} + \overline{A}BC + AC\overline{D} + AB\overline{C}$ Hay 6 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3 **1**	2
01	4	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14 **1**
10	8	9	11	10 **1**

f14

Se rodean los 1.

f_{15}: $f_{15} = B\overline{C}D + \overline{A}BC\overline{D} + A\overline{BC} + ABD$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5 **1**	7	6 **1**
11	12	13 **1**	15 **1**	14
10	8 **1**	9 **1**	11	10

f15

Se rodean los 1.

f_{16}: $f_{16} = \overline{ABD} + CD + B\overline{C} + A\overline{C} + AB$ El libro da $\bar{C}\bar{D} + B\bar{C} + AB + AD + CD + \bar{A}\bar{B}C$: es correcta, pero tiene 6 términos y 13 literales. La mínima tiene 5 términos y 11 literales (se ha comprobado que cubre exactamente los mismos 1).

AB\CD

	00	01	11	10
00	0 **1**	1	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9 **1**	11 **1**	10

f16

Se rodean los 1.

f_{17}: $f_{17} = B\overline{C}D + \overline{A}BC\overline{D} + A\overline{BC} + ABD$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5 **1**	7	6 **1**
11	12	13 **1**	15 **1**	14
10	8 **1**	9 **1**	11	10

f17

Se rodean los 1.

f_{18}: $f_{18} = D + C + B + A$

AB\CD

	00	01	11	10
00	0	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9 **1**	11 **1**	10 **1**

f18

Se rodean los 1.

4 Funciones 19 a 22

f_{19}: $f_{19} = \overline{AB} + \overline{CD} + BD + AC$ Hay 6 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9	11 **1**	10 **1**

f19

Se rodean los 1.

f_{20}: $f_{20} = \overline{AB} + \overline{C}D + BC + A\overline{D}$ Hay 6 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9 **1**	11	10 **1**

f20

Se rodean los 1.

$f_{21}: f_{21} = 1$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 **1**	9 **1**	11 **1**	10 **1**

f21

Se rodean los 1.

f_{22}: $f_{22} = \overline{AD} + B\overline{D} + AD$ Hay 2 soluciones mínimas equivalentes.

AB\CD

AB \ CD	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4 **1**	5	7	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9 **1**	11 **1**	10

f22

Se rodean los 1.

✅ **Resultado final**

Las 22 soluciones del libro cubren exactamente los 1 y tienen el coste mínimo. Las funciones f_{19} y f_{20} tienen varias soluciones equivalentes (se han contado por código). f_{21} vale 1 siempre y $f_{18} = A + B + C + D$ vale 1 salvo en $ABCD = 0000$.

> ⚠️ **Nota sobre f_{16}**
>
> La solución del libro para f_{16} es correcta pero **no es mínima**: usa 6 términos y existe una de 5 (ver más arriba).

`Libro · Cap. 3 · P3.2 (f23-f38)`

Simplificar 16 mapas con combinaciones indiferentes

✔ verificado

Simplifica 16 funciones de cuatro variables con casillas indiferentes (×) y escribe la suma de productos mínima.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> **Las indiferentes:** una × puede valer 0 o 1 según convenga. Se usa para hacer lazos más grandes o menos lazos, pero **nunca se forma un lazo solo con ×** (no haría falta).

1 Funciones 23 a 28

f_{23}: $f_{23} = \overline{BCD} + AB\overline{C}$

Se rodean los 1 usando las ×.

f_{24}: $f_{24} = C\overline{D} + AB\overline{C}$

AB\CD

	00	01	11	10
00	0	1	3	2 **1**
01	4	5	7	6 ×
11	12 **1**	13 **1**	15	14 ×
10	8	9	11 ×	10 **1**

f24

Se rodean los 1 usando las ×.

f_{25}: $f_{25} = AC$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 ×	5 ×	7	6
11	12 ×	13 ×	15 **1**	14 **1**
10	8	9	11 **1**	10 **1**

f25

Se rodean los 1 usando las ×.

f_{26}: $f_{26} = \overline{D} + B + A$

AB\CD

	00	01	11	10
00	0 ×	1	3	2 **1**
01	4 ×	5 **1**	7 ×	6 **1**
11	12 **1**	13 ×	15 ×	14 ×
10	8 **1**	9 **1**	11 ×	10 **1**

f26

Se rodean los 1 usando las ×.

f_{27}: $f_{27} = B\overline{C} + A\overline{C} + A\overline{D} + AB$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7	6 ×
11	12 ×	13 **1**	15 **1**	14 ×
10	8 **1**	9 **1**	11	10 **1**

f27

Se rodean los 1 usando las ×.

f_{28}: $f_{28} = \overline{A}C + \overline{B}C + A\overline{C}$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3 ×	2 **1**
01	4	5	7 **1**	6 ×
11	12 ×	13 **1**	15	14
10	8 **1**	9 ×	11 **1**	10 ×

f28

Se rodean los 1 usando las ×.

2 Funciones 29 a 34

f_{29}: $f_{29} = C + B$

AB\CD

	00	01	11	10
00	0	1 ×	3 ×	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 ×	13 **1**	15 **1**	14 **1**
10	8	9 ×	11 ×	10 **1**

f29

Se rodean los 1 usando las ×.

f_{30}: $f_{30} = \overline{AC}D + ACD + AB\overline{D}$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1 **1**	3	2
01	4	5 **1**	7	6
11	12 **1**	13	15 **1**	14 **1**
10	8	9 ×	11 ×	10

f30

Se rodean los 1 usando las ×.

f_{31}: $f_{31} = \overline{D} + B + AC$

AB\CD

	00	01	11	10
00	0 ×	1	3	2 **1**
01	4 **1**	5 **1**	7 **1**	6 ×
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 ×	9	11 **1**	10 **1**

f31

Se rodean los 1 usando las ×.

f_{32}: $f_{32} = B\overline{CD} + BCD + A\overline{D} + AB$

AB\CD

	00	01	11	10
00	0	1	3	2 ×
01	4 **1**	5	7 **1**	6
11	12 ×	13 **1**	15 **1**	14 ×
10	8 **1**	9	11	10 ×

f32

Se rodean los 1 usando las ×.

f_{33}: $f_{33} = A\overline{BD} + ABC$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3	2
01	4	5	7	6 ×
11	12	13	15 **1**	14 ×
10	8 ×	9	11	10 **1**

f33

Se rodean los 1 usando las ×.

f_{34}: $f_{34} = 1$

AB\CD

	00	01	11	10
00	0 **1**	1 ×	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 ×	14 ×
10	8 **1**	9 ×	11 **1**	10 ×

f34

Se rodean los 1 usando las ×.

3 Funciones 35 a 38

f_{35}: $f_{35} = \overline{B}CD + B\overline{C} + AC$

AB\CD

	00	01	11	10
00	0	1	3 **1**	2
01	4 **1**	5 **1**	7	6 ×
11	12 **1**	13 ×	15 **1**	14 ×
10	8 ×	9	11 **1**	10 **1**

f35

Se rodean los 1 usando las ×.

f_{36}: $f_{36} = 1$

AB\CD

	00	01	11	10
00	0 ×	1 ×	3 ×	2 ×
01	4 ×	5 ×	7 ×	6 ×
11	12 ×	13 ×	15 ×	14 ×
10	8 ×	9 ×	11 ×	10 ×

f36

Se rodean los 1 usando las ×.

f_{37}: $f_{37} = 1$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 ×	2 ×
01	4 **1**	5 **1**	7 ×	6 ×
11	12 **1**	13 **1**	15 ×	14 ×
10	8 **1**	9 **1**	11 ×	10 ×

f37

Se rodean los 1 usando las ×.

f_{38}: $f_{38} = D$

f38

Se rodean los 1 usando las ×.

✅ **Resultado final**

Las 16 soluciones del libro son válidas y mínimas. Casos curiosos: f_{34} y f_{37} valen 1 (todas las × y unos forman un único lazo de 16); f_{36} solo tiene indiferentes y vale cualquier cosa (0, 1 o $AB + C$); $f_{38} = D$ usa las × de las filas con $D = 1$.

`Libro · Cap. 3 · P3.2 (f39–f60)` **Simplificar 22 mapas como producto de sumas**

✔ verificado

Simplifica 22 funciones de cuatro variables, representadas por los **ceros** de su mapa, y escribe el producto de sumas mínimo.

🧭 Antes de empezar: qué piden y qué teoría usas

Producto de sumas (POS): rodea los 0 como si fueran unos, lee cada lazo como un producto y niega el resultado con De Morgan. Cada lazo se convierte en una **suma** en la que cada variable aparece **negada si vale 1** en el lazo y sin negar si vale 0.

1 Funciones 39 a 44

f_{39}: $f_{39} = (A + B + \overline{C} + \overline{D})(\overline{B} + C + \overline{D})(\overline{A} + B + D)(\overline{A} + C + D)$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3 **1**	2
01	4	5 **1**	7	6
11	12 **1**	13 **1**	15	14
10	8 **1**	9	11	10 **1**

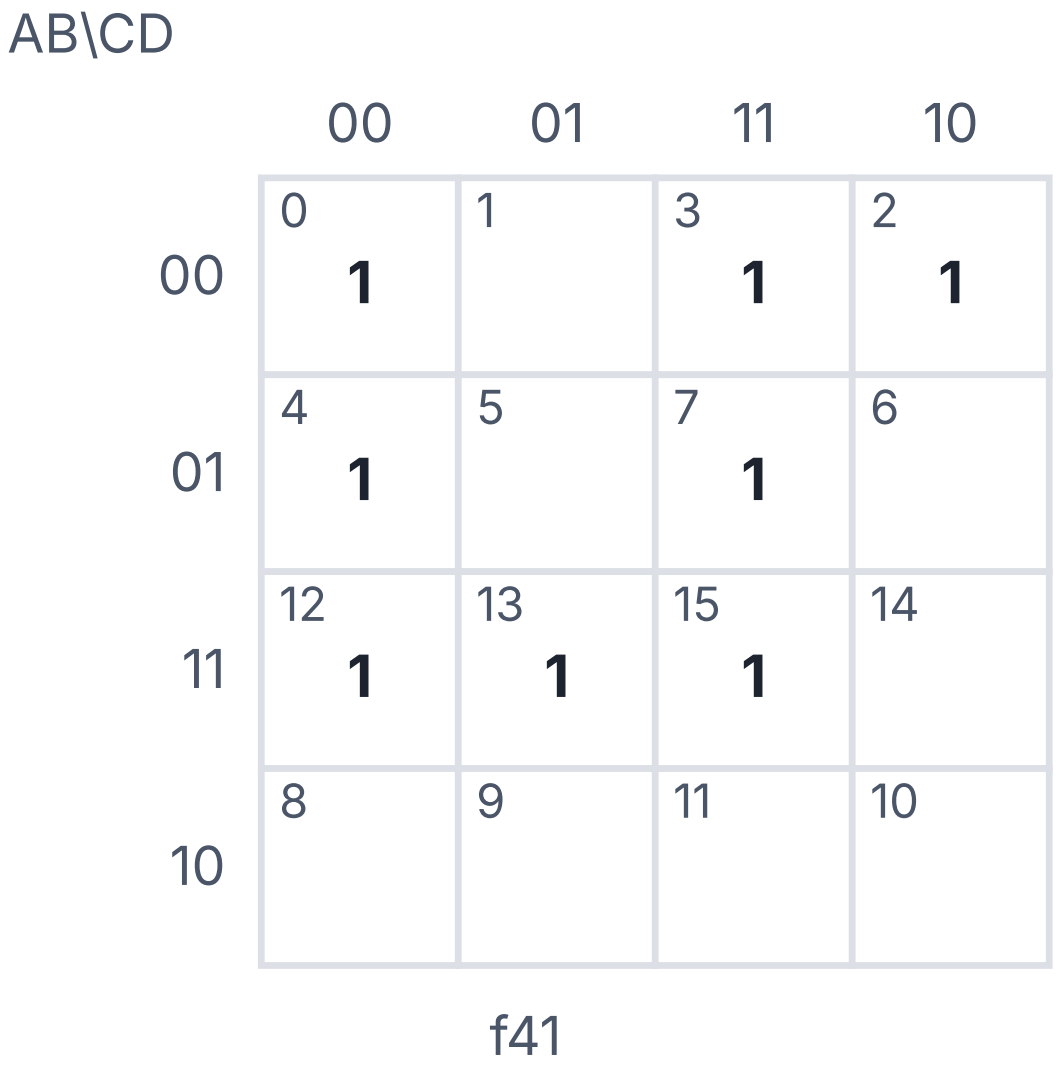
f39

Mapa de $\overline{f_{39}}$: los 1 son los ceros de f_{39}. Se rodean para leer el producto de sumas.

f_{40}: $f_{40} = 0$

Mapa de $\overline{f_{40}}$: los 1 son los ceros de f_{40}. Se rodean para leer el producto de sumas.

f_{41}: $f_{41} = (A + B + D)(A + \overline{C} + \overline{D})(\overline{B} + C + D)(\overline{A} + \overline{B} + \overline{D})$ Hay 2 soluciones mínimas equivalentes.



Mapa de $\overline{f_{41}}$: los 1 son los ceros de f_{41}. Se rodean para leer el producto de sumas.

f_{42}: $f_{42} = (A+B+C)(A+B+\overline{D})(A+\overline{B}+\overline{C}+D)(\overline{A}+B+D)(\overline{A}+\overline{C}+\overline{D})(\overline{A}+\overline{B}+C)$ En el mapa del libro hay un 0 aislado en la casilla $ABCD = 0110$ que no tiene lazo ni factor en la solución impresa: $(\bar{A}+B+D)(\bar{A}+\bar{B}+C)(\bar{A}+\bar{B}+\bar{D})(B+\bar{C}+\bar{D})(A+B+C)$. Si ese 0 es real, la solución del libro **no** lo cumple (valdría 1 en esa casilla). Si es una errata del mapa, la solución del libro es correcta. Aquí se resuelve el mapa tal como se ve.

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2
01	4	5	7	6 **1**
11	12 **1**	13 **1**	15 **1**	14
10	8 **1**	9	11 **1**	10 **1**

f42

Mapa de $\overline{f_{42}}$: los 1 son los ceros de f_{42}. Se rodean para leer el producto de sumas.

f_{43}: $f_{43} = (A + \overline{B})(\overline{B} + C)(\overline{A} + B + D)(\overline{A} + \overline{C} + \overline{D})$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14
10	8 **1**	9	11 **1**	10 **1**

f43

Mapa de $\overline{f_{43}}$: los 1 son los ceros de f_{43}. Se rodean para leer el producto de sumas.

f_{44}: $f_{44} = (A + D)(\overline{C} + D)(\overline{B} + \overline{D})(\overline{A} + B)$ Hay 6 soluciones mínimas equivalentes.

Mapa de $\overline{f_{44}}$: los 1 son los ceros de f_{44}. Se rodean para leer el producto de sumas.

2 Funciones 45 a 50

f_{45}: $f_{45} = (A + B + C)(A + \overline{B} + \overline{C} + D)(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C + D)$

AB\CD

	00	01	11	10
00	0 **1**	1 **1**	3	2
01	4	5	7	6 **1**
11	12 **1**	13	15	14
10	8	9	11 **1**	10 **1**

f45

Mapa de $\overline{f_{45}}$: los 1 son los ceros de f_{45}. Se rodean para leer el producto de sumas.

f_{46}: $f_{46} = (A + B + C)(A + \overline{C} + D)(\overline{A} + B + \overline{D})(\overline{A} + \overline{B} + \overline{C})$ Hay 2 soluciones mínimas equivalentes.

Mapa de $\overline{f_{46}}$: los 1 son los ceros de f_{46}. Se rodean para leer el producto de sumas.

f_{47}: $f_{47} = (A+B)(A+D)(B+\overline{C})(\overline{C}+\overline{D})(\overline{A}+\overline{B}+C)$ El libro da $(A+\bar{C})(A+B)(B+\bar{C})(\bar{B}+C+D)(\bar{A}+\bar{B}+\bar{D})$: es correcta, con 5 factores y 12 literales. La mínima tiene también 5 factores y un literal menos.

AB\CD	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14
10	8	9	11 **1**	10 **1**

f47

Mapa de $\overline{f_{47}}$: los 1 son los ceros de f_{47}. Se rodean para leer el producto de sumas.

f_{48}: $f_{48} = (A)(B)(C)(D)$

AB\CD	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11 **1**	10 **1**

f48

Mapa de $\overline{f_{48}}$: los 1 son los ceros de f_{48}. Se rodean para leer el producto de sumas.

f_{49}: $f_{49} = (A + \overline{B} + C)$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7 ×	6
11	12	13 ×	15 ×	14
10	8	9	11	10

f49

Mapa de $\overline{f_{49}}$: los 1 son los ceros de f_{49}. Se rodean para leer el producto de sumas.

f_{50}: $f_{50} = (D)(\overline{B})(\overline{A} + C)$

Mapa de $\overline{f_{50}}$: los 1 son los ceros de f_{50}. Se rodean para leer el producto de sumas.

3 Funciones 51 a 56

f_{51}: $f_{51} = (A + C + D)(A + B + \overline{C})$

AB\CD

	00	01	11	10
00	0 **1**	1	3 **1**	2 **1**
01	4 **1**	5 ×	7 ×	6
11	12 ×	13 ×	15 ×	14 ×
10	8	9	11	10

f51

Mapa de $\overline{f_{51}}$: los 1 son los ceros de f_{51}. Se rodean para leer el producto de sumas.

f_{52}: $f_{52} = 0$

AB\CD	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 ×
01	4 **1**	5 **1**	7 **1**	6 ×
11	12 **1**	13 **1**	15 **1**	14 ×
10	8 **1**	9 **1**	11 **1**	10 ×

f52

Mapa de $\overline{f_{52}}$: los 1 son los ceros de f_{52}. Se rodean para leer el producto de sumas.

f_{53}: $f_{53} = (\overline{C} + D)(\overline{B} + C)$

AB\CD

	00	01	11	10
00	0	1	3	2 **1**
01	4 **1**	5 **1**	7	6 **1**
11	12 ×	13 ×	15	14 **1**
10	8	9	11	10 **1**

f53

Mapa de $\overline{f_{53}}$: los 1 son los ceros de f_{53}. Se rodean para leer el producto de sumas.

f_{54}: $f_{54} = (B + C + D)(A + \overline{C} + D)(\overline{A} + \overline{B} + C)$

AB\CD

	00	01	11	10
00	0 **1**	1	3	2 **1**
01	4	5	7	6 **1**
11	12 **1**	13 **1**	15	14 ×
10	8 **1**	9	11	10

f54

Mapa de $\overline{f_{54}}$: los 1 son los ceros de f_{54}. Se rodean para leer el producto de sumas.

f_{55}: $f_{55} = (\overline{B} + \overline{C} + D)(\overline{A})$

Mapa de $\overline{f_{55}}$: los 1 son los ceros de f_{55}. Se rodean para leer el producto de sumas.

f_{56}: $f_{56} = (B + D)(\overline{C} + \overline{D})(\overline{B} + C)$ Hay 3 soluciones mínimas equivalentes.

AB\CD

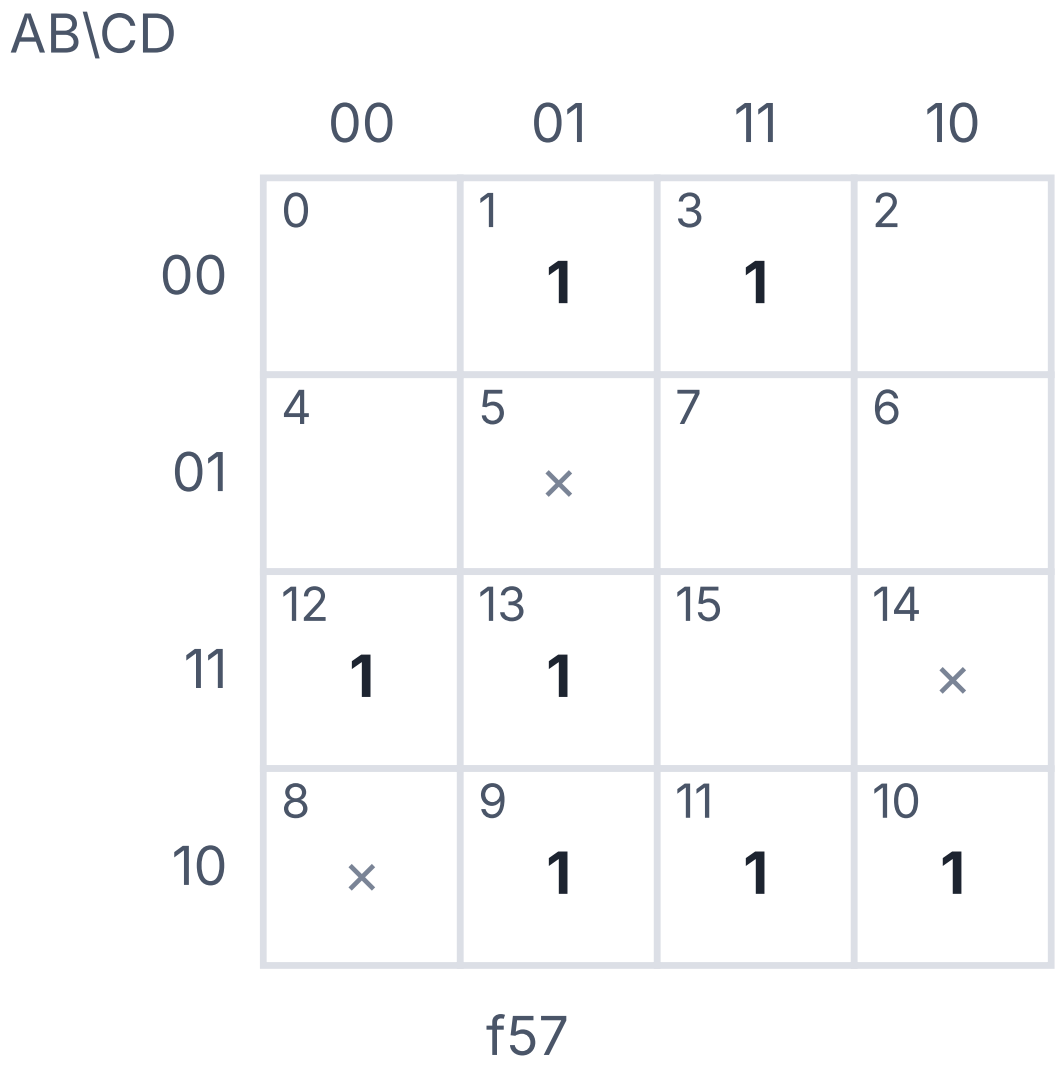
	00	01	11	10
00	0 ×	1	3 **1**	2 **1**
01	4 ×	5 **1**	7 **1**	6
11	12 **1**	13 **1**	15 ×	14 ×
10	8 **1**	9	11 ×	10 **1**

f56

Mapa de $\overline{f_{56}}$: los 1 son los ceros de f_{56}. Se rodean para leer el producto de sumas.

4 Funciones 57 a 60

f_{57}: $f_{57} = (B + \overline{D})(C + \overline{D})(\overline{A} + D)$ Hay 3 soluciones mínimas equivalentes.



Mapa de $\overline{f_{57}}$: los 1 son los ceros de f_{57}. Se rodean para leer el producto de sumas.

f_{58}: $f_{58} = (\overline{B} + D)(\overline{A} + \overline{B})$

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 ×	9	11	10 ×

f58

Mapa de $\overline{f_{58}}$: los 1 son los ceros de f_{58}. Se rodean para leer el producto de sumas.

f_{59}: $f_{59} = (\overline{B})$ Hay 2 soluciones mínimas equivalentes.

AB\CD

	00	01	11	10
00	0	1	3	2
01	4 ×	5 ×	7 ×	6 ×
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8 ×	9 ×	11 ×	10 ×

f59

Mapa de $\overline{f_{59}}$: los 1 son los ceros de f_{59}. Se rodean para leer el producto de sumas.

f_{60}: $f_{60} = 0$

Mapa de $\overline{f_{60}}$: los 1 son los ceros de f_{60}. Se rodean para leer el producto de sumas.

✅ **Resultado final**

Las 22 soluciones del libro valen 0 en los 0 y 1 en los 1 del mapa. Funciones especiales: $f_{40} = f_{52} = f_{60} = 0$ (el mapa entero son ceros e indiferentes), $f_{48} = ABCD$ (un solo 1), $f_{49} = A + \bar{B} + C$ y $f_{59} = \bar{A}$ (sin formar lazos de 8 ni de 4 con las indiferentes).

⚠️ **Notas sobre f_{42} y f_{47}**

En f_{42} el mapa tiene un 0 en $ABCD = 0110$ que la solución impresa no cubre: o sobra ese 0 en el mapa o falta un factor. En f_{47} la solución impresa es correcta pero tiene un literal de más.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 4 · Circuitos combinacionales con puertas

Problemas resueltos de diseño combinacional: de la tabla de verdad al circuito de puertas, con Karnaugh, SOP y POS.

💡 Cómo se usa esta sección

Son los problemas resueltos del capítulo 4 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras y resueltos de nuevo. El esquema del libro es siempre el mismo: **variables → tabla de verdad → simplificación → circuito**. Aquí cada tabla se genera por código, cada simplificación se calcula con un minimizador y se compara con la del libro (cubre los 1, no cubre ningún 0 y tiene el mismo coste), y cada circuito se describe por el número y el tipo de puertas, sin copiar el esquema. Los mapas de esta guía ponen las dos primeras variables en las filas y las otras dos en las columnas; el libro los dibuja con otros ejes, pero es el mismo mapa. Si el libro difiere, aparece una nota «Ojo». La teoría está en el tema 3 (Boole y Karnaugh) y el tema 5 (sumadores, comparadores, multiplexores).

`Libro · Cap. 4 · P4.1` **Voto mayoritario de tres sensores** ✔ verificado

Tres sensores binarios miden la misma señal. Diseña el circuito que dé a su salida el valor que tienen la mayoría de ellos.

🧭 Antes de empezar: qué piden y qué teoría usas

Método de diseño combinacional: (1) decide qué son entradas y qué salidas; (2) escribe la tabla de verdad; (3) pasa a suma de minitérminos; (4) simplifica con Karnaugh; (5) dibuja el circuito.

1 Entradas y salida

Entradas: S_2, S_1, S_0. Salida: S. Con tres sensores nunca hay empate, así que la mayoría siempre existe.

2 Tabla de verdad

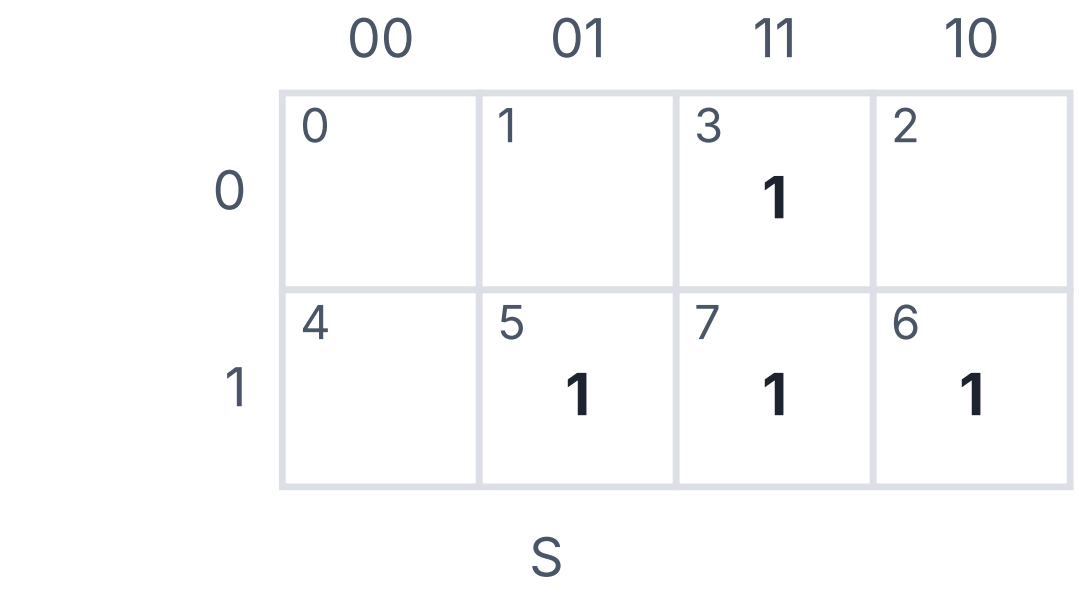
n	S2	S1	S0	S
0	0	0	0	0

n	S2	S1	S0	S
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$S = 1$ cuando hay **al menos dos** unos: filas 3, 5, 6 y 7. En la mitad de las filas gana el 0 y en la otra mitad el 1.

3 Simplificación

$S = \Sigma(3, 5\text{-}7)$. Simplificada: $S = S_1\,S_0 + S_2\,S_0 + S_2\,S_1$. Circuito de dos niveles: 3 AND de 2 entradas, 1 OR de 3 entradas. Coincide en coste con la solución del libro (misma expresión).



Casillas con el número de minitérmino; cada lazo da un producto.

Cada par de sensores que coinciden en 1 basta para activar la salida. Por eso hay tres términos: S_1S_0, S_2S_1 y S_2S_0.

4 Comprobación

El programa recorre las 8 combinaciones y comprueba que $S_1S_0 + S_2S_1 + S_2S_0$ vale 1 justo cuando hay dos o más unos.

✅ Resultado final

$S = S_1S_0 + S_2S_1 + S_2S_0$: tres AND de dos entradas y una OR de tres, sin inversores.

⚠️ Errata en el libro

En la fórmula final del libro el tercer término está escrito como S_2S_1, igual que el segundo. Debe ser S_2S_0: con la fórmula impresa, la combinación $S_2S_1S_0 = 101$ daría 0 en vez de 1. Se ha comprobado por código.

`Libro · Cap. 4 · P4.2`

Divisor de dos números de 2 bits

✔ verificado

Diseña el circuito más pequeño que divida dos números naturales de dos bits, A_1A_0 entre B_1B_0, y dé el cociente C_1C_0 y el resto R_1R_0.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: con números tan pequeños, la «división» es repartir enteros. Hay 16 combinaciones de entrada y 4 salidas, así que son cuatro funciones de cuatro variables.

1 Entradas y salidas

Entradas: A_1A_0 (dividendo) y B_1B_0 (divisor). Salidas: cociente C_1C_0 y resto R_1R_0.

2 El caso del divisor cero

Dividir entre 0 no tiene resultado. Hay que **elegir** qué hacer. Aquí, igual que en el libro, se toma cociente y resto iguales al dividendo (filas 0, 4, 8 y 12). Es una decisión de diseño: otra opción serían indiferentes, que dan un circuito más pequeño.

3 Tabla de verdad

A1	A0	B1	B0	C1	C0	R1	R0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	0	0
0	1	1	0	0	0	0	1
0	1	1	1	0	0	0	1
1	0	0	0	1	0	1	0
1	0	0	1	1	0	0	0
1	0	1	0	0	1	0	0
1	0	1	1	0	0	1	0
1	1	0	0	1	1	1	1
1	1	0	1	1	1	0	0
1	1	1	0	0	1	0	1
1	1	1	1	0	1	0	0

Ejemplos: $2 \div 1 = 2$ resto 0 (fila 9); $3 \div 2 = 1$ resto 1 (fila 14); $1 \div 3 = 0$ resto 1 (fila 7).

4 Cociente C_1

$C_1 = \Sigma(8, 9, 12, 13)$. Simplificada: $C_1 = A_1 \, \overline{B_1}$. Circuito de dos niveles: 1 AND de 2 entradas, 1 inversor (B_1). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Cociente** C_0

$C_0 = \Sigma(4, 5, 10, 12\text{-}15)$. Simplificada: $C_0 = A_0\,\overline{B_1} + A_1\,B_1\,\overline{B_0} + A_1\,A_0$. Circuito de dos niveles: 2 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 3 entradas, 2 inversores (B_0, B_1). Coincide en coste con la solución del libro (misma expresión).

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4 **1**	5 **1**	7	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11	10 **1**

C0

Casillas con el número de minitérmino; cada lazo da un producto.

6 Resto R_1

$R_1 = \Sigma(8, 11, 12)$. Simplificada: $R_1 = A_1\,\overline{B_1}\,\overline{B_0} + A_1\,\overline{A_0}\,B_1\,B_0$. Circuito de dos niveles: 1 AND de 3 entradas y 1 AND de 4 entradas, 1 OR de 2 entradas, 3 inversores (A_0, B_0, B_1). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Resto** R_0

$R_0 = \Sigma(4, 6, 7, 12, 14)$. Simplificada: $R_0 = A_0\,\overline{B_0} + \overline{A_1}\,A_0\,B_1$. Circuito de dos niveles: 1 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 2 entradas, 2 inversores (A_1, B_0). Coincide en coste con la solución del libro (misma expresión).

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7 **1**	6 **1**
11	12 **1**	13	15	14 **1**
10	8	9	11	10

R0

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las cuatro funciones del libro son correctas y mínimas. Coste total: C_1 (1 término), C_0 (3), R_1 (2), R_0 (2).

`Libro · Cap. 4 · P4.3` **Sumador/restador de 1 bit con acarreo previo** ✔ verificado

Diseña un circuito con entradas M, A_0, B_0, C_i y salida $S_2S_1S_0$ en complemento a dos: si $M = 0$ calcula $A_0 + B_0 + C_i$; si $M = 1$ calcula $A_0 - (B_0 + C_i)$.

🧭 Antes de empezar: qué piden y qué teoría usas

Ejercicio para practicar el complemento a dos. El resultado necesita 3 bits porque el rango va de -2 a $+3$ ($A_0 - (B_0 + C_i)$ llega a -2; $A_0 + B_0 + C_i$ llega a $+3$).

1 Entradas y salidas

Cuatro entradas (M elige la operación) y tres salidas $S_2S_1S_0$ en C2 de 3 bits, donde el rango es $-4 \ldots +3$.

2 Tabla de verdad

M	A0	B0	Ci	S2	S1	S0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	0	1	1	1	1
1	0	1	0	1	1	1
1	0	1	1	1	1	0
1	1	0	0	0	0	1
1	1	0	1	0	0	0
1	1	1	0	0	0	0
1	1	1	1	1	1	1

Ejemplos: fila 3: $0+1+1=+2=010$. Fila 7: $1+1+1=+3=011$. Fila 10: $0-(1+0)=-1=111$. Fila 14: $1-(1+0)=0=000$.

3 Salida S_2 (signo)

$S_2=\Sigma(9\text{-}11,15)$. Simplificada: $S_2=M\,\overline{A_0}\,\mathrm{Ci}+M\,\overline{A_0}\,B_0+M\,B_0\,\mathrm{Ci}$. Circuito de dos niveles: 3 AND de 3 entradas, 1 OR de 3 entradas, 1 inversor (A_0). Coincide en coste con la solución del libro (misma expresión).

MA0\B0Ci

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15 **1**	14
10	8	9 **1**	11 **1**	10 **1**

S2

Casillas con el número de minitérmino; cada lazo da un producto.

4 Salida S_1

$S_1=\Sigma(3,5\text{-}7,9\text{-}11,15)$. Simplificada: $S_1=B_0\,\mathrm{Ci}+\overline{M}\,A_0\,\mathrm{Ci}+\overline{M}\,A_0\,B_0+M\,\overline{A_0}\,\mathrm{Ci}+M\,\overline{A_0}\,B_0$. Circuito de dos niveles: 1 AND de 2 entradas y 4 AND de 3 entradas, 1 OR de 5 entradas, 2 inversores (A_0, M). Coincide en coste con la solución del libro (misma expresión).

MA0\B0Ci

	00	01	11	10
00	0	1	3 **1**	2
01	4	5 **1**	7 **1**	6 **1**
11	12	13	15 **1**	14
10	8	9 **1**	11 **1**	10 **1**

S1

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Salida** S_0

$S_0 = \Sigma(1, 2, 4, 7, 9, 10, 12, 15)$. Simplificada: $S_0 = \overline{A_0}\,\overline{B_0}\,\mathrm{Ci} + \overline{A_0}\,B_0\,\overline{\mathrm{Ci}} + A_0\,\overline{B_0}\,\overline{\mathrm{Ci}} + A_0\,B_0\,\mathrm{Ci}$. Circuito de dos niveles: 4 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (A_0, B_0, Ci). Coincide en coste con la solución del libro (misma expresión).

MA0\B0Ci

	00	01	11	10
00	0	1 **1**	3	2 **1**
01	4 **1**	5	7 **1**	6
11	12 **1**	13	15 **1**	14
10	8	9 **1**	11	10 **1**

S0

Casillas con el número de minitérmino; cada lazo da un producto.

Observa que $S_0 = A_0 \oplus B_0 \oplus C_i$: el bit menos significativo no depende de M (sumar o restar cambia la paridad igual).

✅ **Resultado final**

Las tres funciones del libro son correctas y mínimas como suma de productos. S_0 es un XOR de tres entradas: con puertas XOR cuesta menos que sus cuatro términos de tres literales.

`Libro · Cap. 4 · P4.4` **Multiplicador de dos números de 2 bits** ✔ verificado

Implementa el circuito más pequeño capaz de multiplicar dos números en binario puro de dos bits, A_1A_0 y B_1B_0.

🧭 Antes de empezar: qué piden y qué teoría usas

Por qué 4 bits de salida: el producto más grande es $3 \times 3 = 9 = 1001_2$, que necesita 4 bits. Aquí hay muchos ceros en las salidas, así que conviene trabajar con **productos de sumas** (maxitérminos).

1 Tabla de verdad

A1	A0	B1	B0	S3	S2	S1	S0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Ejemplos: fila 6: $01 \times 10 = 1 \times 2 = 0010$. Fila 12: $11 \times 00 = 0$. Fila 15: $11 \times 11 = 3 \times 3 = 9 = 1001$.

2 Salida S_3

$S3 = \Pi(0\text{-}14)$. Producto de sumas mínimo: $S3 = (A_1)(A_0)(B_1)(B_0)$. En suma de productos sería $A_1\ A_0\ B_1\ B_0$ (1 términos). Coincide en coste con el libro.

A1A0\B1B0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11 **1**	10 **1**

S3 (se marcan los ceros)

Se rodean los ceros: cada lazo da una suma.

3 Salida S_2

$S2 = \Pi(0\text{-}9, 12, 13, 15)$. Producto de sumas mínimo: $S2 = (A_1)(B_1)(\overline{A_0} + \overline{B_0})$. En suma de productos sería $A_1\ \overline{A_0}\ B_1 + A_1\ B_1\ \overline{B_0}$ (2 términos). Coincide en coste con el libro.

A1A0\B1B0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14
10	8 **1**	9 **1**	11	10

S2 (se marcan los ceros)

Se rodean los ceros: cada lazo da una suma.

4 **Salida** S_1

$S1 = \Pi(0\text{-}5, 8, 10, 12, 15)$. Producto de sumas mínimo: $S1 = (A_1 + A_0)(A_1 + B_1)(A_0 + B_0)(B_1 + B_0)(\overline{A_1} + \overline{A_0} + \overline{B_1} + \overline{B_0})$. En suma de productos sería $\overline{A_1}\, A_0\, B_1 + A_0\, B_1\, \overline{B_0} + A_1\, \overline{A_0}\, B_0 + A_1\, \overline{B_1}\, B_0$ (4 términos). Coincide en coste con el libro.

A1A0\B1B0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5 **1**	7	6
11	12 **1**	13	15 **1**	14
10	8 **1**	9	11	10 **1**

S1 (se marcan los ceros)

Se rodean los ceros: cada lazo da una suma.

5 **Salida** S_0

$S0 = \Pi(0\text{-}4, 6, 8\text{-}12, 14)$. Producto de sumas mínimo: $S0 = (A_0)(B_0)$. En suma de productos sería $A_0\,B_0$ (1 términos). Coincide en coste con el libro.

A1A0\B1B0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5	7	6 **1**
11	12 **1**	13	15	14 **1**
10	8 **1**	9 **1**	11 **1**	10 **1**

S0 (se marcan los ceros)

Se rodean los ceros: cada lazo da una suma.

Es el AND de los dos bits menos significativos: A_0B_0.

✅ **Resultado final**

Se comprueba que las cuatro salidas forman el producto correcto para las 16 combinaciones. $S_3 = A_1A_0B_1B_0$ y $S_0 = A_0B_0$ son simples ANDs; S_2 y S_1 son las que requieren simplificar.

`Libro · Cap. 4 · P4.5` **Complemento a dos de un número de 4 bits** ✔ verificado

Diseña el circuito más pequeño que calcule el complemento a dos de un número de cuatro bits $E_3E_2E_1E_0$, con salida $C_3C_2C_1C_0$.

🧭 Antes de empezar: qué piden y qué teoría usas

Regla: C2 $=$ invertir todos los bits $+1$. Hay una forma más rápida de leerlo: de derecha a izquierda se copian los bits hasta el primer 1 (incluido) y el resto se invierte.

1 Tabla de verdad

E3	E2	E1	E0	C3	C2	C1	C0
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

Ejemplos: $0010 \to 1101 + 1 = 1110$. $1000 \to 0111 + 1 = 1000$ (el -8 es su propio opuesto). $0000 \to 1111 + 1 = 0000$.

2 Salida C_3

$C_3 = \Sigma(1\text{-}8)$. Simplificada: $C_3 = \overline{E_3}\,E_0 + \overline{E_3}\,E_1 + \overline{E_3}\,E_2 + E_3\,\overline{E_2}\,\overline{E_1}\,\overline{E_0}$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 4 entradas, 1 OR de 4 entradas, 4 inversores (E_0, E_1, E_2, E_3). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

3 Salida C_2

$C_2 = \Sigma(1\text{-}4, 9\text{-}12)$. Simplificada: $C_2 = \overline{E_2}\,E_0 + \overline{E_2}\,E_1 + E_2\,\overline{E_1}\,\overline{E_0}$. Circuito de dos niveles: 2 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 3 entradas, 3 inversores (E_0, E_1, E_2). Coincide en coste con la solución del libro (misma expresión).

E3E2\E1E0

	00	01	11	10
00	0	1 **1**	3 **1**	2 **1**
01	4 **1**	5	7	6
11	12 **1**	13	15	14
10	8	9 **1**	11 **1**	10 **1**

C2

Casillas con el número de minitérmino; cada lazo da un producto.

4 **Salida** C_1

$C_1 = \Sigma(1, 2, 5, 6, 9, 10, 13, 14)$. Simplificada: $C_1 = \overline{E_1}\, E_0 + E_1\, \overline{E_0}$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas, 2 inversores (E_0, E_1). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

Es el XOR $E_1 \oplus E_0$.

5 Salida C_0

$C_0 = \Sigma(1, 3, 5, 7, 9, 11, 13, 15)$. Simplificada: $C_0 = E_0$. C_0 es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

El bit menos significativo no cambia: $C_0 = E_0$, un cable.

✅ **Resultado final**

Las cuatro funciones del libro son correctas y mínimas. Nota: el -8 (1000) no tiene opuesto en 4 bits y el circuito lo devuelve igual: es el desbordamiento típico del C2.

`Libro · Cap. 4 · P4.6` **Complemento a 9 de un dígito BCD** ✔ verificado

Diseña el circuito más pequeño que obtenga el complemento a 9 de un dígito BCD $B_3B_2B_1B_0$, con salida $C_{93}C_{92}C_{91}C_{90}$.

🧭 Antes de empezar: qué piden y qué teoría usas

Complemento a 9: para un dígito B es $9 - B$. Se usa para restar en decimal. Las combinaciones $1010 \ldots 1111$ no son dígitos BCD: ¿qué hacer con ellas?

1 Qué hacer con las seis combinaciones prohibidas

Hay tres opciones: (1) declarar la salida **indiferente** ($\times$), (2) forzar un valor, por ejemplo 0000, o (3) añadir una salida de **error** que valga 1. El libro elige la primera, porque da circuitos más pequeños. Si se quisiera la salida de error, sería $B_3B_2 + B_3B_1$ (comprobado por código).

2 Tabla de verdad

B3	B2	B1	B0	C93	C92	C91	C90
0	0	0	0	1	0	0	1
0	0	0	1	1	0	0	0
0	0	1	0	0	1	1	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	0	0
0	1	1	0	0	0	1	1
0	1	1	1	0	0	1	0
1	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0
1	0	1	0	×	×	×	×
1	0	1	1	×	×	×	×
1	1	0	0	×	×	×	×
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×

Ejemplos: $0 \to 9 = 1001$; $6 \to 3 = 0011$; $9 \to 0 = 0000$.

3 Salida C_{93}

$C_{93} = \Sigma(0, 1)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $C_{93} = \overline{B_3}\,\overline{B_2}\,\overline{B_1}$. Circuito de dos niveles: 1 AND de 3 entradas, 3 inversores (B_1, B_2, B_3). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Salida C_{92}

$C_{92} = \Sigma(2\text{-}5)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $C_{92} = \overline{B_2}\,B_1 + B_2\,\overline{B_1}$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas, 2 inversores (B_1, B_2). Coincide en coste con la solución del libro (misma expresión).

B3B2\B1B0

	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4 **1**	5 **1**	7	6
11	12 ×	13 ×	15 ×	14 ×
10	8	9	11 ×	10 ×

C92

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Salida** C_{91}

$C_{91} = \Sigma(2,3,6,7)$ con indiferentes $d(10,11,12,13,14,15)$. Simplificada: $C_{91} = B_1$. C_{91} es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

Fíjate: $C_{91} = B_1$ es un cable, no hace falta ninguna puerta.

6 Salida C_{90}

$C_{90} = \Sigma(0, 2, 4, 6, 8)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $C_{90} = \overline{B_0}$. Circuito de dos niveles: 1 inversor (B_0). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

Es un solo inversor.

✅ **Resultado final**

Con las indiferentes, las cuatro funciones son muy sencillas: un AND de tres entradas, dos AND y una OR, un cable y un inversor. Las cuatro soluciones del libro son correctas y mínimas.

`Libro · Cap. 4 · P4.7` **Control de calidad: cuatro piezas por lote** ✔ verificado

De cada lote se extraen 4 piezas y un sensor da 1 si la pieza es aprobada y 0 si no. Diseña el circuito que indique: todas aprobadas (TA), mayoría aprobada (MA), igual número de aprobadas y rechazadas (I) y mayoría de rechazadas (MR).

🧭 Antes de empezar: qué piden y qué teoría usas

Truco: el número de unos de la entrada decide la salida. Cuatro unos: TA. Tres: MA. Dos: I. Cero o uno: MR. Siempre se activa **una y solo una** de las cuatro salidas.

1 Tabla de verdad

M1	M2	M3	M4	TA	MA	I	MR
0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	1
0	0	1	1	0	0	1	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	0
0	1	1	1	0	1	0	0
1	0	0	0	0	0	0	1
1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0
1	0	1	1	0	1	0	0
1	1	0	0	0	0	1	0
1	1	0	1	0	1	0	0
1	1	1	0	0	1	0	0
1	1	1	1	1	0	0	0

Fila 0: todas rechazadas, $MR = 1$. Fila 6 (0110): dos aprobadas, $I = 1$. Fila 11 (1011): tres aprobadas, $MA = 1$. Fila 15: $TA = 1$ (y MA no se activa, aunque la mayoría también sea de aprobadas).

2 Salida TA

$\mathrm{TA} = \Sigma(15)$. Simplificada: $\mathrm{TA} = M_1\, M_2\, M_3\, M_4$. Circuito de dos niveles: 1 AND de 4 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

3 Salida MA

$\mathrm{MA} = \Sigma(7, 11, 13, 14)$. Simplificada: $\mathrm{MA} = \overline{M_1}\, M_2\, M_3\, M_4 + M_1\, \overline{M_2}\, M_3\, M_4 + M_1\, M_2\, \overline{M_3}\, M_4 + M_1\, M_2\, M_3\, \overline{M_4}$. Circuito de dos niveles: 4 AND de 4 entradas, 1 OR de 4 entradas, 4 inversores (M_1, M_2, M_3, M_4). Coincide en coste con la solución del libro (misma expresión).

M1M2\M3M4

	00	01	11	10
00	0	1	3	2
01	4	5	7 **1**	6
11	12	13 **1**	15	14 **1**
10	8	9	11 **1**	10

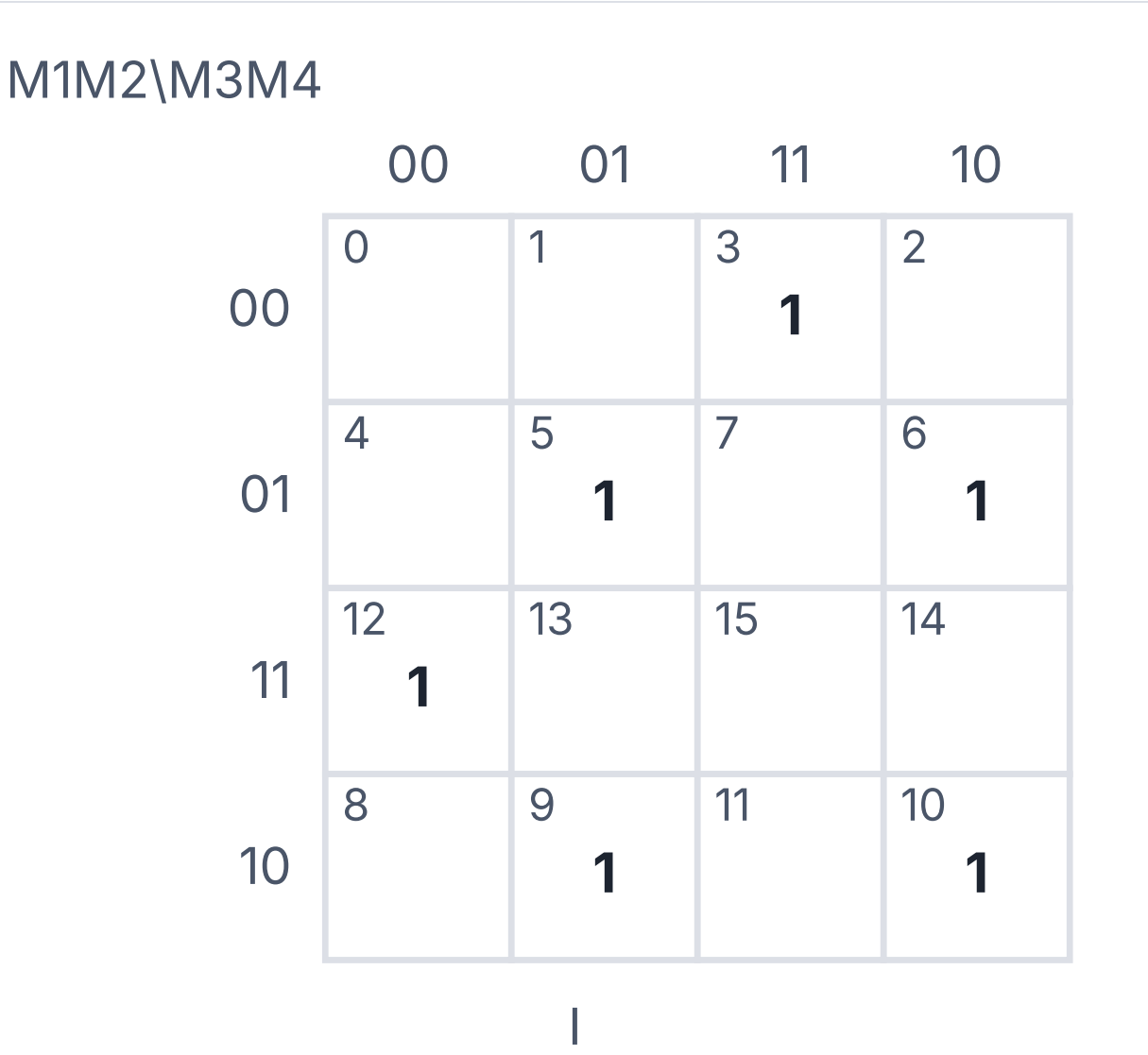
MA

Casillas con el número de minitérmino; cada lazo da un producto.

Los cuatro 1 del mapa están aislados (no son vecinos): no hay nada que agrupar y la suma de minitérminos ya es la mínima.

4 **Salida** I

$I = \Sigma(3, 5, 6, 9, 10, 12)$. Simplificada: $I = \overline{M_1}\,\overline{M_2}\,M_3\,M_4 + \overline{M_1}\,M_2\,\overline{M_3}\,M_4 + \overline{M_1}\,M_2\,M_3\,\overline{M_4} + M_1\,\overline{M_2}\,\overline{M_3}\,M_4 + M_1\,\overline{M_2}\,M_3\,\overline{M_4} + M_1\,M_2\,\overline{M_3}\,\overline{M_4}$. Circuito de dos niveles: 6 AND de 4 entradas, 1 OR de 6 entradas, 4 inversores (M_1, M_2, M_3, M_4). Coincide en coste con la solución del libro (misma expresión).



Casillas con el número de minitérmino; cada lazo da un producto.

Seis minitérminos aislados: es el patrón de «tablero de ajedrez» de los mapas, que no se simplifica. Son las combinaciones con exactamente dos unos.

5 Salida MR

$\mathrm{MR} = \Sigma(0\text{-}2, 4, 8)$. Simplificada: $\mathrm{MR} = \overline{M_1}\,\overline{M_2}\,\overline{M_3} + \overline{M_1}\,\overline{M_2}\,\overline{M_4} + \overline{M_1}\,\overline{M_3}\,\overline{M_4} + \overline{M_2}\,\overline{M_3}\,\overline{M_4}$. Circuito de dos niveles: 4 AND de 3 entradas, 1 OR de 4 entradas, 4 inversores (M_1, M_2, M_3, M_4). Coincide en coste con la solución del libro (misma expresión).

M1M2\M3M4

	00	01	11	10
00	0 **1**	1 **1**	3	2 **1**
01	4 **1**	5	7	6
11	12	13	15	14
10	8 **1**	9	11	10

MR

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Los cuatro resultados coinciden con los del libro y son mínimos. Observa el coste: MA e I necesitan 4 y 6 AND de 4 entradas. Es el precio de detectar «exactamente k unos».

`Libro · Cap. 4 · P4.8` **Rumbo de un barco en un display de 7 segmentos**

✔ verificado

El piloto automático indica el rumbo con cuatro señales N, S, E, O. Diseña el circuito más pequeño que lo dibuje en un 7 segmentos: norte activa a; sur, d; este, b, c; oeste, e, f; noreste, a, b; noroeste, a, f; sureste, c, d; suroeste, d, e.

🧭 Antes de empezar: qué piden y qué teoría usas

Dibujar con segmentos. Cada fila de la tabla es un rumbo y solo hay que «pintarlo». Un rumbo válido tiene una señal activa, o dos que sean vecinas (N+E, N+O, S+E, S+O). Norte con sur, o este con oeste, son imposibles.

1 Qué filas son válidas

Válidas (8): N=8, S=4, E=2, O=1, NE=10, NO=9, SE=6, SO=5. Imposibles (8): 0, 3, 7, 11, 12, 13, 14 y 15. El libro las deja como indiferentes, incluida la fila 0 (sin rumbo): un barco siempre lleva alguno.

2 Tabla de verdad

N	S	E	O	a	b	c	d	e	f	g
0	0	0	0	×	×	×	×	×	×	×
0	0	0	1	0	0	0	0	1	1	0
0	0	1	0	0	1	1	0	0	0	0
0	0	1	1	×	×	×	×	×	×	×
0	1	0	0	0	0	0	1	0	0	0
0	1	0	1	0	0	0	1	1	0	0
0	1	1	0	0	0	1	1	0	0	0
0	1	1	1	×	×	×	×	×	×	×
1	0	0	0	1	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	1	0
1	0	1	0	1	1	0	0	0	0	0
1	0	1	1	×	×	×	×	×	×	×
1	1	0	0	×	×	×	×	×	×	×
1	1	0	1	×	×	×	×	×	×	×
1	1	1	0	×	×	×	×	×	×	×
1	1	1	1	×	×	×	×	×	×	×

Ejemplos: fila 1 (oeste): se encienden e, f (el lado izquierdo). Fila 9 (noroeste): a, f. Fila 8 (norte): solo a.

3 Segmento a

$a = \Sigma(8\text{-}10)$ con indiferentes $d(0, 3, 7, 11, 12, 13, 14, 15)$. Simplificada: $a = N$. a es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Segmento b

$b = \Sigma(2, 10)$ con indiferentes $d(0, 3, 7, 11, 12, 13, 14, 15)$. Simplificada: $b = \overline{S}\,E$. Circuito de dos niveles: 1 AND de 2 entradas, 1 inversor (S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Segmento** c

$c = \Sigma(2,6)$ con indiferentes $d(0,3,7,11,12,13,14,15)$. Simplificada: $c = \overline{N}\,E$. Circuito de dos niveles: 1 AND de 2 entradas, 1 inversor (N). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

6 **Segmento** d

$d = \Sigma(4\text{-}6)$ con indiferentes $d(0, 3, 7, 11, 12, 13, 14, 15)$. Simplificada: $d = S$. d es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Segmento** e

$e = \Sigma(1,5)$ con indiferentes $d(0,3,7,11,12,13,14,15)$. Simplificada: $e = \overline{N}\,O$. Circuito de dos niveles: 1 AND de 2 entradas, 1 inversor (N). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

8 **Segmento** f

$f = \Sigma(1, 9)$ con indiferentes $d(0, 3, 7, 11, 12, 13, 14, 15)$. Simplificada: $f = \overline{S}\, O$. Circuito de dos niveles: 1 AND de 2 entradas, 1 inversor (S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

9 Segmento g

$g = \Sigma()$ con indiferentes $d(0, 3, 7, 11, 12, 13, 14, 15)$. Simplificada: $g = 0$. La salida g es constante. Coincide en coste con la solución del libro (misma expresión).

✅ Resultado final

Con tantas indiferentes, casi todo se reduce a una o dos variables: $a = N$, $d = S$, $b = \bar{S}E$, $c = \bar{N}E$, $e = \bar{N}O$, $f = \bar{S}O$ y $g = 0$ (el segmento central nunca se enciende). Todas coinciden con el libro.

Libro · Cap. 4 · P4.9

Rumbo del barco con alarma de datos erróneos

✔ verificado

Repite el ejercicio anterior, pero si las señales dan información imposible se debe encender **todo** el 7 segmentos.

🧭 Antes de empezar: qué piden y qué teoría usas

Cambio respecto al 4.8: las ocho combinaciones imposibles dejan de ser indiferentes y valen 1 en las siete salidas. Un display totalmente encendido avisa al piloto de que algo falla. A cambio, el circuito crece: perdemos las simplificaciones que daban las ×.

1 Tabla de verdad

N	S	E	O	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	1
0	0	0	1	0	0	0	0	1	1	0
0	0	1	0	0	1	1	0	0	0	0
0	0	1	1	1	1	1	1	1	1	1
0	1	0	0	0	0	0	1	0	0	0
0	1	0	1	0	0	0	1	1	0	0
0	1	1	0	0	0	1	1	0	0	0
0	1	1	1	1	1	1	1	1	1	1
1	0	0	0	1	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	1	0
1	0	1	0	1	1	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1

Es la tabla del 4.8 con las $\times$ convertidas en 1.

2 Segmento a

$a = \Sigma(0, 3, 7\text{-}15)$. Simplificada: $a = \overline{S}\,\overline{E}\,\overline{O} + E\,O + N$. Circuito de dos niveles: 1 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 3 entradas, 3 inversores (E, O, S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

3 Segmento b

$b = \Sigma(0, 2, 3, 7, 10\text{-}15)$. Simplificada: $b = \overline{N}\,\overline{S}\,\overline{O} + \overline{S}\,E + E\,O + N\,S$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (N, O, S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Segmento c

$c = \Sigma(0, 2, 3, 6, 7, 11\text{-}15)$. Simplificada: $c = \overline{N}\,\overline{S}\,\overline{O} + \overline{N}\,E + E\,O + N\,S$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (N, O, S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 Segmento d

$d = \Sigma(0, 3\text{-}7, 11\text{-}15)$. Simplificada: $d = \overline{N}\,\overline{E}\,\overline{O} + E\,O + S$. Circuito de dos niveles: 1 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 3 entradas, 3 inversores (E, N, O). Coincide en coste con la solución del libro (misma expresión).

NS\EO	00	01	11	10
00	0 **1**	1	3 **1**	2
01	4 **1**	5 **1**	7 **1**	6 **1**
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11 **1**	10

d

Casillas con el número de minitérmino; cada lazo da un producto.

6 Segmento e

$e = \Sigma(0, 1, 3, 5, 7, 11\text{-}15)$. Simplificada: $e = \overline{N}\,\overline{S}\,\overline{E} + \overline{N}\,O + E\,O + N\,S$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (E, N, S). Coincide en coste con la solución del libro (misma expresión).

NS\EO	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2
01	4	5 **1**	7 **1**	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11 **1**	10

e

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Segmento** f

$f = \Sigma(0, 1, 3, 7, 9, 11\text{-}15)$. Simplificada: $f = \overline{N}\,\overline{S}\,\overline{E} + \overline{S}\,O + E\,O + N\,S$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (E, N, S). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

8 Segmento g

$g = \Sigma(0, 3, 7, 11\text{-}15)$. Simplificada: $g = \overline{N}\,\overline{S}\,\overline{E}\,\overline{O} + E\,O + N\,S$. Circuito de dos niveles: 2 AND de 2 entradas y 1 AND de 4 entradas, 1 OR de 3 entradas, 4 inversores (E, N, O, S). Coincide en coste con la solución del libro (misma expresión).

NS\EO	00	01	11	10
00	0 **1**	1	3 **1**	2
01	4	5	7 **1**	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11 **1**	10

g

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las siete funciones del libro son mínimas, salvo una errata en b (ver nota). Comparado con el 4.8, el circuito pasa de unos pocos cables a 3–4 productos por salida: **saber qué pasa con lo imposible cambia mucho el coste**. El término EO (la combinación imposible «este y oeste») aparece en las siete salidas, y NS («norte y sur») en cinco.

⚠️ **Errata en el libro (función b)**

En la fórmula del libro para b, el primer término está escrito con $\bar{E}$ ($\bar{N}\bar{S}\bar{E}$). Ese término valdría 1 en la fila 1 (0001, oeste), pero $b = 0$ en esa fila. El término correcto es $\bar{N}\bar{S}\bar{O}$ (cubre las filas 0 y 2). Se ha comprobado por código.

Libro · Cap. 4 · P4.10

Decodificador de BCD a 7 segmentos

✔ verificado

Implementa el circuito más pequeño que convierta un dígito BCD $B_3B_2B_1B_0$ en las siete señales a, b, c, d, e, f, g de un display de 7 segmentos.

🧭 Antes de empezar: qué piden y qué teoría usas

Cómo se rellena la tabla: ten delante un 7 segmentos dibujado y apunta qué segmentos forman cada cifra. Las filas 10 a 15 no son BCD y se dejan como indiferentes. El 6 se dibuja sin el segmento superior ($a = 0$) y el 9 sin el inferior ($d = 0$); son formas de escribirlos, no la única posible.

1 Tabla de verdad

B3	B2	B1	B0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	0	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
1	0	1	0	×	×	×	×	×	×	×
1	0	1	1	×	×	×	×	×	×	×
1	1	0	0	×	×	×	×	×	×	×
1	1	0	1	×	×	×	×	×	×	×
1	1	1	0	×	×	×	×	×	×	×
1	1	1	1	×	×	×	×	×	×	×

Ejemplos: el 0 enciende todo menos g (1111110); el 1, b y c (0110000); el 2, a, b, d, e, g (1101101).

2 Segmento a

$a = \Sigma(0, 2, 3, 5, 7\text{-}9)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $a = \overline{B_2}\,\overline{B_0} + \overline{B_2}\,B_1 + B_2\,B_0 + B_3$. Circuito de dos niveles: 3 AND de 2 entradas, 1 OR de 4 entradas, 2 inversores (B_0, B_2). Coincide en coste con la solución del libro (misma expresión).

B3B2\B1B0

	00	01	11	10
00	0 **1**	1	3 **1**	2 **1**
01	4	5 **1**	7 **1**	6
11	12 ×	13 ×	15 ×	14 ×
10	8 **1**	9 **1**	11 ×	10 ×

a

Casillas con el número de minitérmino; cada lazo da un producto.

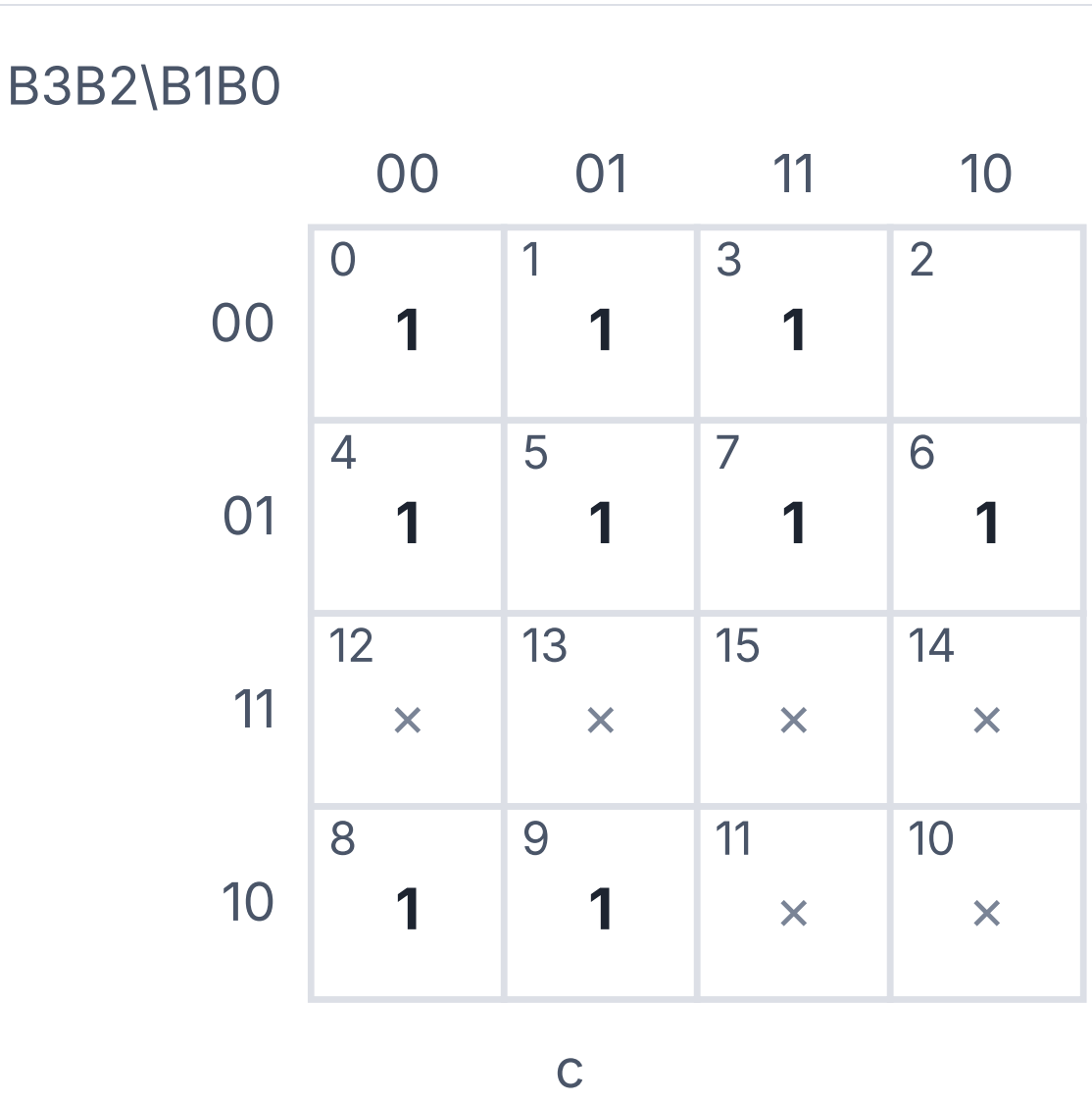
3 Segmento b

$b = \Sigma(0\text{-}4, 7\text{-}9)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $b = \overline{B_2} + \overline{B_1}\,\overline{B_0} + B_1\,B_0$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 3 entradas, 3 inversores (B_0, B_1, B_2). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 **Segmento** c

$c = \Sigma(0, 1, 3\text{-}9)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $c = \overline{B_1} + B_0 + B_2$. Circuito de dos niveles: 1 OR de 3 entradas, 1 inversor (B_1). Coincide en coste con la solución del libro (misma expresión).



Casillas con el número de minitérmino; cada lazo da un producto.

5 **Segmento** d

$d = \Sigma(0, 2, 3, 5, 6, 8)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $d = \overline{B_2}\,\overline{B_0} + \overline{B_2}\,B_1 + B_1\,\overline{B_0} + B_2\,\overline{B_1}\,B_0$. Circuito de dos niveles: 3 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 4 entradas, 3 inversores (B_0, B_1, B_2). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

6 Segmento e

$e = \Sigma(0, 2, 6, 8)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $e = \overline{B_2}\,\overline{B_0} + B_1\,\overline{B_0}$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas, 2 inversores (B_0, B_2). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Segmento** f

$f = \Sigma(0, 4\text{-}6, 8, 9)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $f = \overline{B_1}\,\overline{B_0} + B_2\,\overline{B_1} + B_2\,\overline{B_0} + B_3$. Circuito de dos niveles: 3 AND de 2 entradas, 1 OR de 4 entradas, 2 inversores (B_0, B_1). Coincide en coste con la solución del libro (misma expresión).

B3B2\B1B0

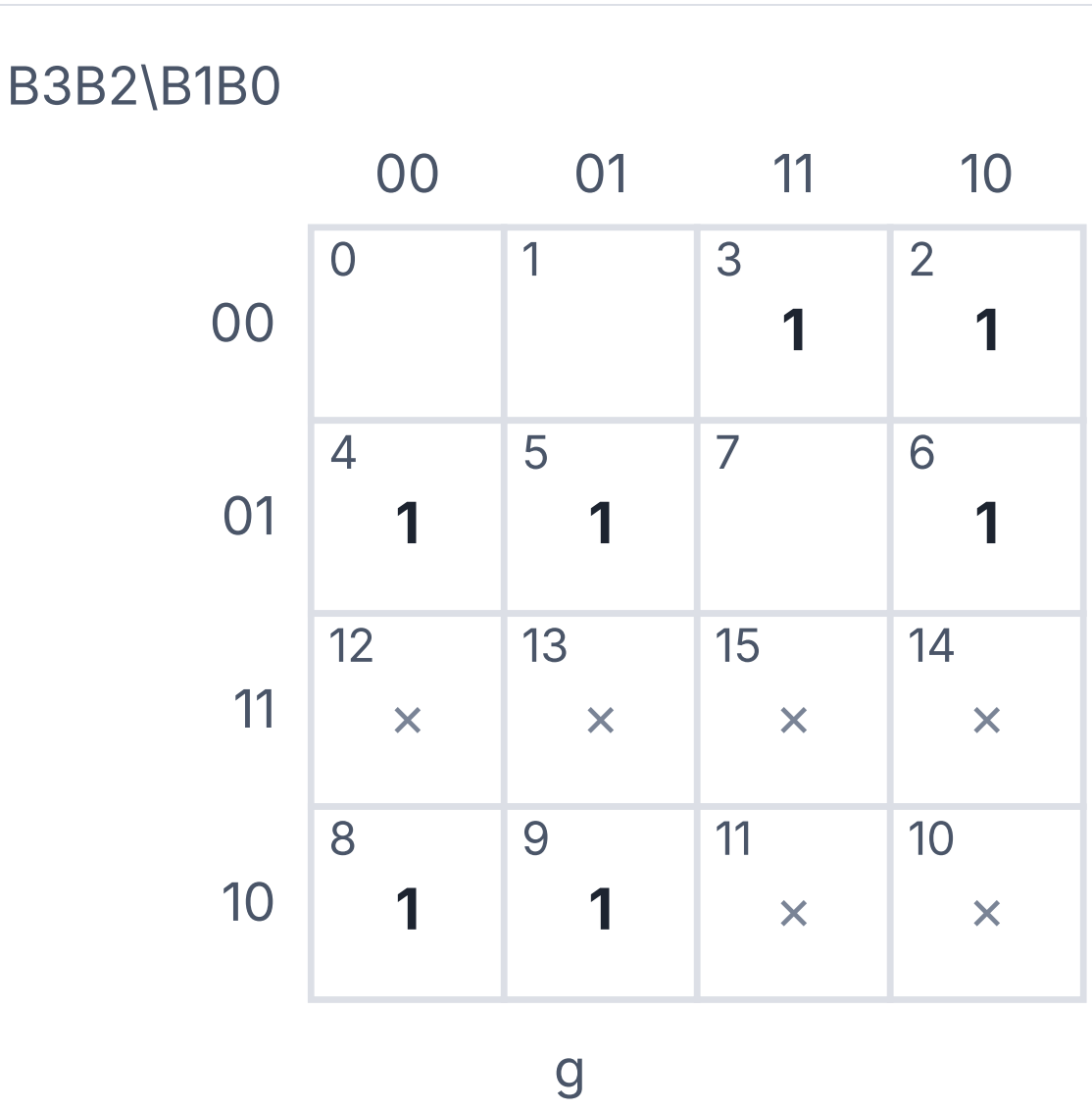
	00	01	11	10
00	0 **1**	1	3	2
01	4 **1**	5 **1**	7	6 **1**
11	12 ×	13 ×	15 ×	14 ×
10	8 **1**	9 **1**	11 ×	10 ×

f

Casillas con el número de minitérmino; cada lazo da un producto.

8 Segmento g

$g = \Sigma(2\text{-}6, 8, 9)$ con indiferentes $d(10, 11, 12, 13, 14, 15)$. Simplificada: $g = \overline{B_2}\, B_1 + B_1\, \overline{B_0} + B_2\, \overline{B_1} + B_3$. Circuito de dos niveles: 3 AND de 2 entradas, 1 OR de 4 entradas, 3 inversores (B_0, B_1, B_2). Coincide en coste con la solución del libro (otra forma equivalente, también mínima).



Casillas con el número de minitérmino; cada lazo da un producto.

9 **Variante: el 9 con segmento inferior**

Si se prefiere el 9 con el segmento d encendido, solo cambia d, que pasa a valer 1 también en la fila 9: $d = \overline{B_2}\,\overline{B_0} + \overline{B_2}\,B_1 + B_1\,\overline{B_0} + B_2\,\overline{B_1}\,B_0 + B_3$ (5 términos, frente a 4).

✅ **Resultado final**

Las siete simplificaciones del libro son correctas y de coste mínimo. Cada segmento necesita entre 1 y 4 productos. $c = \bar{B}_1 + B_0 + B_2$ es el más sencillo (solo es 0 en los números 2).

⚠️ Errata en el libro (tabla)

En la tabla del libro, la fila del 9 aparece con $d = 1$ y $f = 0$ (1111001), que es justo lo contrario de lo que dicen sus propios sumatorios ($d = \Sigma(0, 2, 3, 5, 6, 8)$ no incluye el 9 y $f = \Sigma(0, 4, 5, 6, 8, 9)$ sí). Aquí se sigue la versión de los sumatorios y de las expresiones finales (1110011).

Libro · Cap. 4 · P4.11

Comparador de dos números de 2 bits

✔ verificado

Diseña un comparador de dos números $A = A_1A_0$ y $B = B_1B_0$ con tres salidas: $A > B$, $A = B$ y $A < B$.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: se comparan las dos cifras y se activa **una sola** de las tres salidas. Es un ejercicio muy sistemático: la tabla se escribe leyendo A y B como números de 0 a 3.

1 Tabla de verdad

A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0

A1	A0	B1	B0	A>B	A=B	A<B
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

Fila 0: $0 = 0$. Fila 3: $0 < 3$. Fila 8: $2 > 0$.

2 Salida $A > B$

$(A > B) = \Sigma(4, 8, 9, 12\text{-}14)$. Simplificada: $G = A_0\,\overline{B_1}\,\overline{B_0} + A_1\,\overline{B_1} + A_1\,A_0\,\overline{B_0}$. Circuito de dos niveles: 1 AND de 2 entradas y 2 AND de 3 entradas, 1 OR de 3 entradas, 2 inversores (B_0, B_1). Coincide en coste con la solución del libro (misma expresión).

A1A0\B1B0

	00	01	11	10
00	0	1	3	2
01	4 **1**	5	7	6
11	12 **1**	13 **1**	15	14 **1**
10	8 **1**	9 **1**	11	10

G

Casillas con el número de minitérmino; cada lazo da un producto.

3 **Salida** $A = B$

$(A = B) = \Sigma(0, 5, 10, 15)$. Simplificada: $E = \overline{A_1}\,\overline{A_0}\,\overline{B_1}\,\overline{B_0} + \overline{A_1}\,A_0\,\overline{B_1}\,B_0 + A_1\,\overline{A_0}\,B_1\,\overline{B_0} + A_1\,A_0\,B_1\,B_0$. Circuito de dos niveles: 4 AND de 4 entradas, 1 OR de 4 entradas, 4 inversores (A_0, A_1, B_0, B_1). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

La igualdad es la diagonal del mapa y no se simplifica: cuatro minitérminos. Pero hay un truco: $A = B$ equivale a que **cada pareja de bits** coincida, es decir, $(A_1 \odot B_1) \cdot (A_0 \odot B_0)$ con dos puertas XNOR y un AND. Es la forma habitual de hacerlo (verificada por código).

4 Salida $A < B$

$(A < B) = \Sigma(1\text{-}3, 6, 7, 11)$. Simplificada: $L = \overline{A_1}\,\overline{A_0}\,B_0 + \overline{A_1}\,B_1 + \overline{A_0}\,B_1\,B_0$. Circuito de dos niveles: 1 AND de 2 entradas y 2 AND de 3 entradas, 1 OR de 3 entradas, 2 inversores (A_0, A_1). Coincide en coste con la solución del libro (misma expresión).

A1A0\B1B0

	00	01	11	10
00	0	1 **1**	3 **1**	2 **1**
01	4	5	7 **1**	6 **1**
11	12	13	15	14
10	8	9	11 **1**	10

L

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las tres funciones del libro son correctas y mínimas como suma de productos. La salida «$A = B$» es mucho más barata con XNOR que con cuatro AND de cuatro entradas.

`Libro · Cap. 4 · P4.12` **Máquina registradora con cambio** ✔ verificado

Una máquina tiene cuatro ranuras, de 25, 25, 10 y 5 ptas (en cada una cabe una moneda). Entrega el producto cuando lo introducido llega a 40 ptas o más y devuelve el cambio con monedas de 5 y de 10. Salidas: producto PR, vuelta de 5 (D_5) y dos vueltas de 10 (D_{10A}, D_{10B}).

🧭 Antes de empezar: qué piden y qué teoría usas

Sumar el dinero de cada fila: $25A + 25B + 10 + 5$. Si llega a 40, se da producto y el cambio es lo que sobra, repartido con monedas de 10 primero y de 5 si queda una.

1 Tabla de verdad

M25A	M25B	M10	M5	PR	D5	D10A	D10B
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	0	0	0	0	0
0	1	1	1	1	0	0	0
1	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0
1	0	1	0	0	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	1	0
1	1	0	1	1	1	1	0
1	1	1	0	1	0	1	1
1	1	1	1	1	1	1	1

Ejemplos: fila 7 ($25 + 10 + 5 = 40$): solo PR. Fila 12 ($25 + 25 = 50$): producto y 10 de vuelta (D_{10A}). Fila 13 (55): producto y 15 de vuelta (D_{10A} y D_5). Fila 15 (65): producto y 25 de vuelta (D_{10A}, D_{10B}, D_5).

2 Una decisión de diseño

Las filas intermedias (por ejemplo 10 ptas) no dan nada: la máquina está esperando más monedas. Además la fila 15 «no debería ocurrir», porque el producto ya habría salido antes. Aun así **no se dejan indiferentes**: un valor libre en la salida podría soltar monedas por error. Pon un valor seguro cuando un error cueste dinero.

3 Producto PR

$\mathrm{PR} = \Sigma(7, 11\text{-}15)$. Simplificada: $\mathrm{PR} = \mathrm{M25B}\, M_{10}\, M_5 + \mathrm{M25A}\, M_{10}\, M_5 + \mathrm{M25A}\, \mathrm{M25B}$. Circuito de dos niveles: 1 AND de 2 entradas y 2 AND de 3 entradas, 1 OR de 3 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Vuelta de 5 (D_5)

$D_5 = \Sigma(13, 15)$. Simplificada: $D_5 = \mathrm{M25A}\,\mathrm{M25B}\,M_5$. Circuito de dos niveles: 1 AND de 3 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 Vuelta de 10 (D_{10A})

$\mathrm{D10A} = \Sigma(12\text{-}15)$. Simplificada: $\mathrm{D10A} = \mathrm{M25A}\,\mathrm{M25B}$. Circuito de dos niveles: 1 AND de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

:5AM25B\M10M5

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12 **1**	13 **1**	15 **1**	14 **1**
10	8	9	11	10

D10A

Casillas con el número de minitérmino; cada lazo da un producto.

6 **Vuelta de 10 (** D_{10B} **)**

$\mathrm{D10B} = \Sigma(14, 15)$. Simplificada: $\mathrm{D10B} = \mathrm{M25A}\,\mathrm{M25B}\,M_{10}$. Circuito de dos niveles: 1 AND de 3 entradas. Coincide en coste con la solución del libro (misma expresión).

:5AM25B\M10M5

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15 **1**	14 **1**
10	8	9	11	10

D10B

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las cuatro funciones del libro son correctas y mínimas. D_{10A} y D_{10B} son simples ANDs: el cambio solo existe cuando se han metido las dos monedas de 25.

`Libro · Cap. 4 · P4.13` **Nota de un alumno en un 7 segmentos (solo NAND)**

✔ verificado

La nota (0 a 10) llega en binario puro $N_3N_2N_1N_0$. Diseña con puertas NAND el circuito que la muestre en un 7 segmentos de cátodo común: suspenso (0–4) enciende g; aprobado (5), a; bien (6), a, b; notable (7–8), a, b, c; sobresaliente (9), a, b, c, d; matrícula (10), todos menos g.

🧭 Antes de empezar: qué piden y qué teoría usas

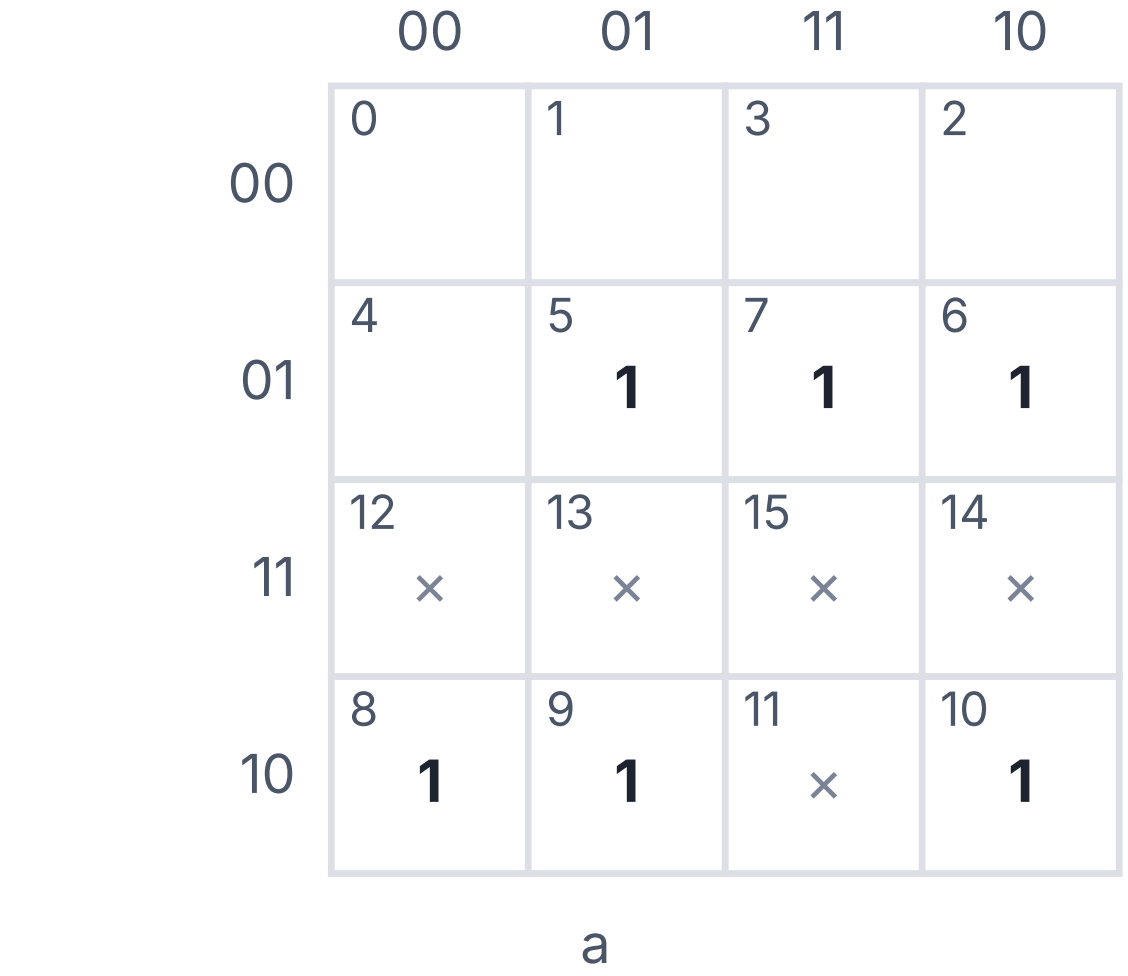
Dos pasos: (1) simplificar como suma de productos; (2) pasar a **NAND–NAND**: cada AND se sustituye por un NAND y la OR final por otro NAND. Por De Morgan el resultado no cambia, y una familia lógica solo necesita un tipo de puerta. Las filas 11 a 15 (notas imposibles) son indiferentes. Atención: aquí 10 es una nota válida (no confundir con BCD).

1 Tabla de verdad

N3	N2	N1	N0	a	b	c	d	e	f	g
0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1
0	1	0	1	1	0	0	0	0	0	0
0	1	1	0	1	1	0	0	0	0	0
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	0	0	0	0
1	0	0	1	1	1	1	1	0	0	0
1	0	1	0	1	1	1	1	1	1	0
1	0	1	1	×	×	×	×	×	×	×
1	1	0	0	×	×	×	×	×	×	×
1	1	0	1	×	×	×	×	×	×	×
1	1	1	0	×	×	×	×	×	×	×
1	1	1	1	×	×	×	×	×	×	×

2 Simplificación de las siete salidas

a. $a = \Sigma(5\text{-}10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $a = N_2\,N_0 + N_2\,N_1 + N_3$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 3 entradas. Coincide en coste con la solución del libro (misma expresión).



Casillas con el número de minitérmino; cada lazo da un producto.

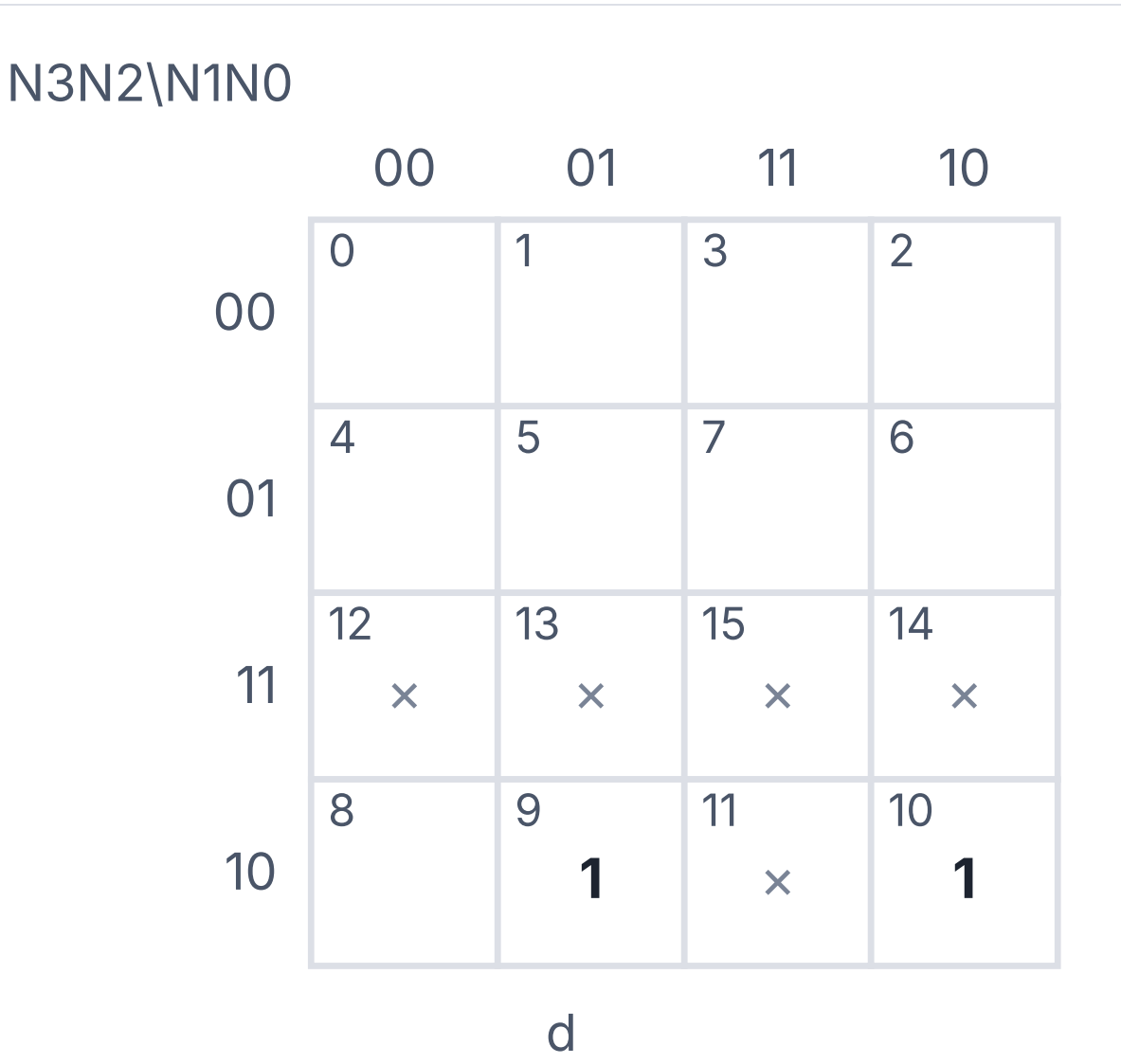
b. $b = \Sigma(6\text{-}10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $b = N_2\,N_1 + N_3$. Circuito de dos niveles: 1 AND de 2 entradas, 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

c. $c = \Sigma(7\text{-}10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $c = N_2\, N_1\, N_0 + N_3$. Circuito de dos niveles: 1 AND de 3 entradas, 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

d. $d = \Sigma(9, 10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $d = N_3\,N_0 + N_3\,N_1$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).



Casillas con el número de minitérmino; cada lazo da un producto.

e. $e = \Sigma(10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $e = N_3\, N_1$. Circuito de dos niveles: 1 AND de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

f. $f = \Sigma(10)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $f = N_3\, N_1$. Circuito de dos niveles: 1 AND de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

g. $g = \Sigma(0\text{-}4)$ con indiferentes $d(11, 12, 13, 14, 15)$. Simplificada: $g = \overline{N_3}\,\overline{N_2} + \overline{N_3}\,\overline{N_1}\,\overline{N_0}$. Circuito de dos niveles: 1 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 2 entradas, 4 inversores (N_0, N_1, N_2, N_3). Coincide en coste con la solución del libro (misma expresión).

N3N2\N1N0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5	7	6
11	12 ×	13 ×	15 ×	14 ×
10	8	9	11 ×	10

g

Casillas con el número de minitérmino; cada lazo da un producto.

3 Paso a NAND–NAND

Regla: $S = T_1 + T_2 + \cdots = \overline{\overline{T_1} \cdot \overline{T_2} \cdots}$. Cada producto T_i de dos o más literales sale de una NAND; la salida es una NAND de todas ellas. Un literal solo, como N_3, entra a la NAND de salida a través de un inversor (una NAND con las entradas unidas). Aquí, compartiendo términos que se repiten (N_2N_1, N_3N_1) y con $e = f$ iguales:

- 7 NAND de dos o tres entradas para los productos,
- 6 NAND de salida distintas (las salidas e y f son la misma señal),
- 4 inversores para los complementos de entrada ($\overline{N_0}, \overline{N_1}, \overline{N_2}, \overline{N_3}$) y 1 para el literal suelto N_3.

Se ha simulado por código el circuito NAND–NAND completo y da exactamente la tabla de verdad para las 11 notas válidas.

✅ **Resultado final**

Las siete funciones del libro son correctas y mínimas. Se ve la lógica de las notas: e y f solo valen 1 en la matrícula (N_3N_1), d en 9 y 10, y g en el suspenso (0 a 4). Fíjate en que N_3 solo vale 1 a partir de 8, y las indiferentes de 11 a 15 permiten escribir «N_3» en vez de un producto largo.

Libro · Cap. 4 · P4.14

Termómetro con dos leds (verde/rojo)

✔ verificado

Activa el led verde LV si la temperatura está entre -3 y $+4\,^{\circ}$C (ambas incluidas) y el rojo LR en el resto. La temperatura llega en complemento a 2 con 4 bits $T_3T_2T_1T_0$.

🧭 Antes de empezar: qué piden y qué teoría usas

Lectura en C2: con 4 bits se representan los valores de -8 a $+7$. Las filas $0\ldots7$ son $0\ldots7$ y las filas $8\ldots15$ son $-8\ldots-1$.

1 Tabla de verdad

T3	T2	T1	T0	grados	LV	LR
0	0	0	0	0	1	0
0	0	0	1	+1	1	0
0	0	1	0	+2	1	0
0	0	1	1	+3	1	0
0	1	0	0	+4	1	0
0	1	0	1	+5	0	1
0	1	1	0	+6	0	1
0	1	1	1	+7	0	1
1	0	0	0	-8	0	1
1	0	0	1	-7	0	1
1	0	1	0	-6	0	1
1	0	1	1	-5	0	1
1	1	0	0	-4	0	1

T3	T2	T1	T0	grados	LV	LR
1	1	0	1	-3	1	0
1	1	1	0	-2	1	0
1	1	1	1	-1	1	0

El rango verde es $0, 1, 2, 3, 4$ (filas 0–4) y $-3, -2, -1$ (filas 13–15): en total 8 de las 16 combinaciones.

2 Led verde LV

$\mathrm{LV} = \Sigma(0\text{-}4, 13\text{-}15)$. Simplificada: $\mathrm{LV} = \overline{T_3}\,\overline{T_2} + \overline{T_3}\,\overline{T_1}\,\overline{T_0} + T_3\,T_2\,T_0 + T_3\,T_2\,T_1$. Circuito de dos niveles: 1 AND de 2 entradas y 3 AND de 3 entradas, 1 OR de 4 entradas, 4 inversores (T_0, T_1, T_2, T_3). Coincide en coste con la solución del libro (misma expresión).

T3T2\T1T0

	00	01	11	10
00	0 **1**	1 **1**	3 **1**	2 **1**
01	4 **1**	5	7	6
11	12	13 **1**	15 **1**	14 **1**
10	8	9	11	10

LV

Casillas con el número de minitérmino; cada lazo da un producto.

3 Led rojo LR

$\mathrm{LR} = \Sigma(5\text{-}12)$. Simplificada: $\mathrm{LR} = \overline{T_3}\,T_2\,T_0 + \overline{T_3}\,T_2\,T_1 + T_3\,\overline{T_2} + T_3\,\overline{T_1}\,\overline{T_0}$. Circuito de dos niveles: 1 AND de 2 entradas y 3 AND de 3 entradas, 1 OR de 4 entradas, 4 inversores (T_0, T_1, T_2, T_3). Coincide en coste con la solución del libro (misma expresión).

T3T2\T1T0

	00	01	11	10
00	0	1	3	2
01	4	5 **1**	7 **1**	6 **1**
11	12 **1**	13	15	14
10	8 **1**	9 **1**	11 **1**	10 **1**

LR

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las dos funciones del libro son correctas y mínimas. Pero hay un atajo: $LR = \overline{LV}$ (comprobado por código), así que basta **un inversor** detrás de LV en vez de otros 4 AND y una OR.

`Libro · Cap. 4 · P4.15` **Cambio de pesetas a euros** ✔ verificado

Una máquina acepta por una única ranura monedas de 5, 10, 25 y 50 ptas y devuelve su valor en euros con monedas de 1, 2, 5, 10 y 20 céntimos (con redondeo). Cuatro señales $P_5, P_{10}, P_{25}, P_{50}$ indican qué moneda se ha introducido.

🧭 Antes de empezar: qué piden y qué teoría usas

Conversión: al cambio oficial de $166{,}386$ ptas por euro, 5 ptas son $3{,}005$ cént. (3), 10 ptas son 6, 25 ptas son 15 y 50 ptas son 30 (comprobado por código). Cada importe se reparte con las monedas disponibles: $3 = 2 + 1$, $6 = 5 + 1$, $15 = 10 + 5$, $30 = 20 + 10$.

1 Qué filas son válidas

Como la ranura es única, solo puede haber **una** moneda a la vez: son válidas la fila 0 (ninguna) y las filas con un único 1 (1, 2, 4 y 8). Las otras 11 filas son indiferentes.

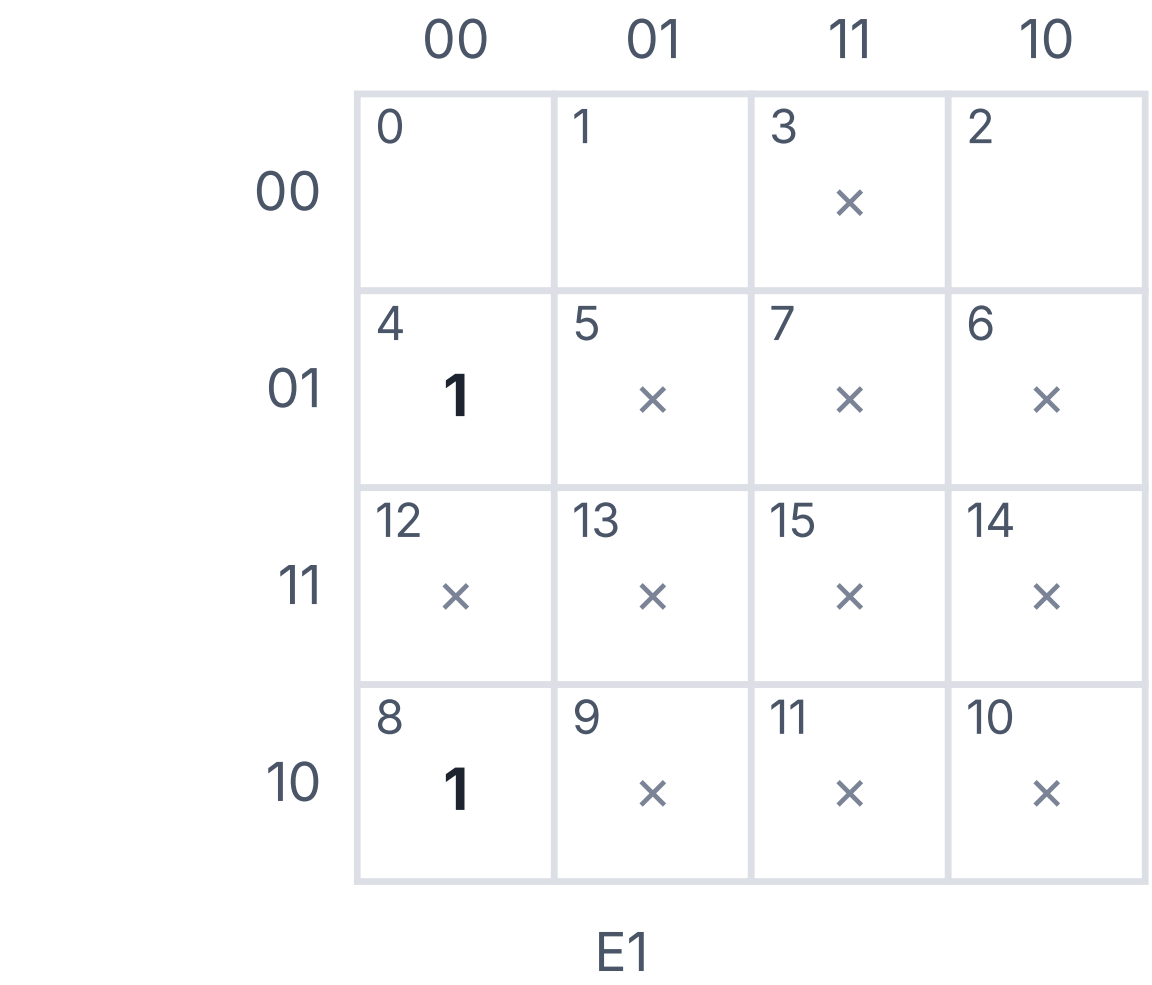
2 Tabla de verdad

P5	P10	P25	P50	E1	E2	E5	E10	E20
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0
0	0	1	1	×	×	×	×	×
0	1	0	0	1	0	1	0	0
0	1	0	1	×	×	×	×	×
0	1	1	0	×	×	×	×	×
0	1	1	1	×	×	×	×	×
1	0	0	0	1	1	0	0	0
1	0	0	1	×	×	×	×	×
1	0	1	0	×	×	×	×	×
1	0	1	1	×	×	×	×	×
1	1	0	0	×	×	×	×	×
1	1	0	1	×	×	×	×	×
1	1	1	0	×	×	×	×	×
1	1	1	1	×	×	×	×	×

Ejemplos: fila 2 (25 ptas): 15 cént. $= 10 + 5$, salidas 00110. Fila 8 (5 ptas): 3 cént. $= 2 + 1$, salidas 11000.

3 Salida $E1$

$E_1 = \Sigma(4, 8)$ con indiferentes $d(3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15)$. Simplificada: $E_1 = P_{10} + P_5$. Circuito de dos niveles: 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

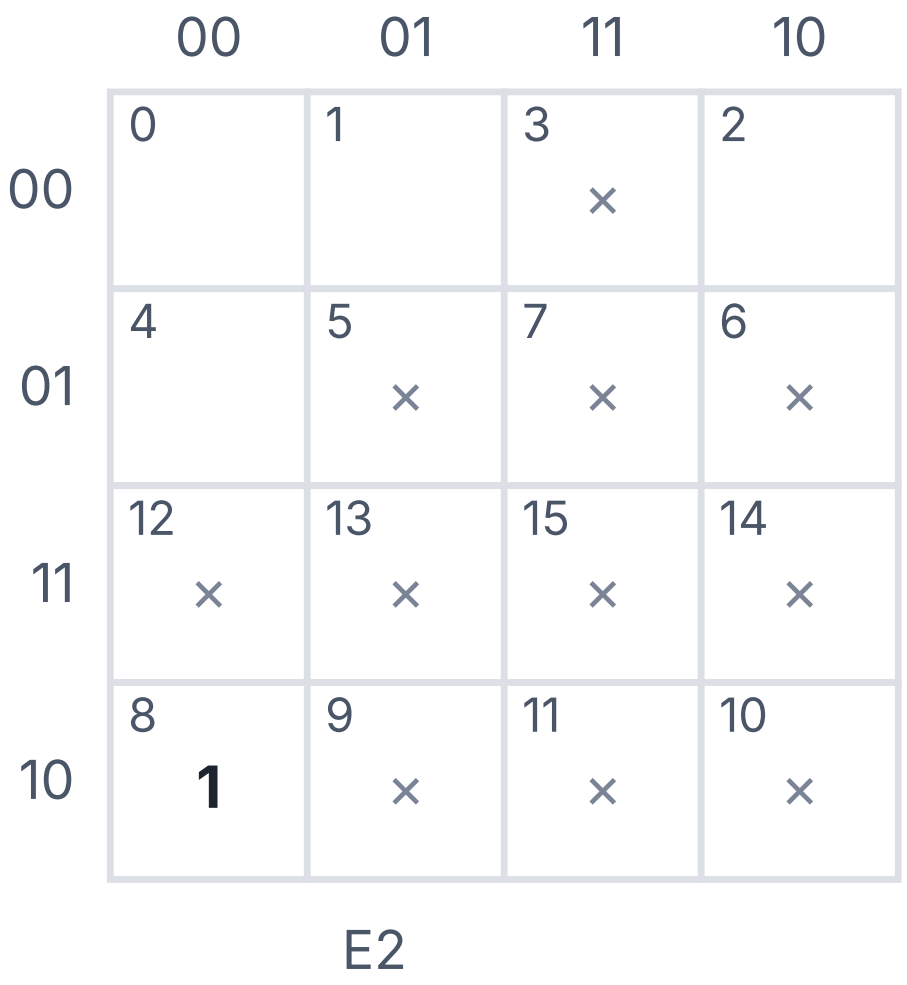


Casillas con el número de minitérmino; cada lazo da un producto.

4 Salida $E2$

$E_2 = \Sigma(8)$ con indiferentes $d(3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15)$. Simplificada: $E_2 = P_5$. E_2 es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

P5P10\P25P50



E2

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Salida** $E5$

$E_5 = \Sigma(2,4)$ con indiferentes $d(3,5,6,7,9,10,11,12,13,14,15)$. Simplificada: $E_5 = P_{25} + P_{10}$. Circuito de dos niveles: 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

P5P10\P25P50

	00	01	11	10
00	0	1	3 ×	2 **1**
01	4 **1**	5 ×	7 ×	6 ×
11	12 ×	13 ×	15 ×	14 ×
10	8	9 ×	11 ×	10 ×

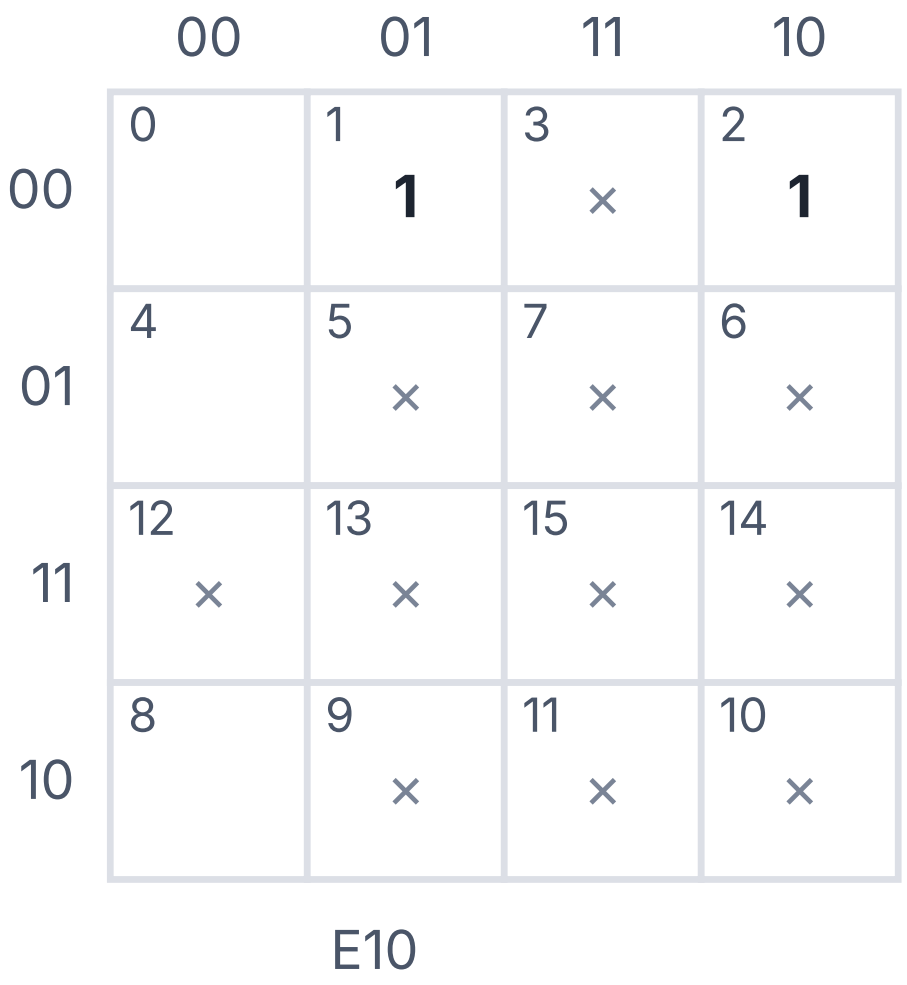
E5

Casillas con el número de minitérmino; cada lazo da un producto.

6 **Salida** $E10$

$E_{10} = \Sigma(1,2)$ con indiferentes $d(3,5,6,7,9,10,11,12,13,14,15)$. Simplificada: $E_{10} = P_{50} + P_{25}$. Circuito de dos niveles: 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

P5P10\P25P50



E10

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Salida** $E20$

$E_{20} = \Sigma(1)$ con indiferentes $d(3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15)$. Simplificada: $E_{20} = P_{50}$. E_{20} es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Con tantas indiferentes todo se reduce a cables y OR de dos entradas: $E_1 = P_5 + P_{10}$, $E_2 = P_5$, $E_5 = P_{25} + P_{10}$, $E_{10} = P_{50} + P_{25}$, $E_{20} = P_{50}$. Coinciden con el libro y son mínimas. Si se quisiera permitir varias monedas a la vez, habría que rehacer la tabla (el propio libro lo deja como ejercicio).

`Libro · Cap. 4 · P4.16` **Semáforos de una rotonda** ✔ verificado

Una rotonda tiene cuatro calles de entrada y cuatro de salida. Por la entrada A llegan 5 coches/min, por B 15, por C 25 y por D 30. Cuatro sensores indican por qué calles circulan coches. Las salidas S_A, S_B, S_C, S_D pueden recoger 5, 10, 20 y 40 coches/min. Como mucho hay coches en dos calles de entrada a la vez. Activa los semáforos de salida para que salgan tantos coches como entran.

🧭 Antes de empezar: qué piden y qué teoría usas

Es un convertidor de código: divide el flujo total entre 5 y escríbelo en binario con pesos $1, 2, 4, 8$ (que corresponden a 5, 10, 20 y 40). Por ejemplo, $35 = 7 \cdot 5$, y $7 = 0111$: se abren las salidas de 5, 10 y 20.

1 Qué filas son válidas

Como mucho dos calles activas: son indiferentes las filas con tres o cuatro unos: 7, 11, 13, 14 y 15. El libro avisa de que poner $\times$ aquí es algo arriesgado y que también se podrían fijar a 0 o a 1.

2 Tabla de verdad

A	B	C	D	SA	SB	SC	SD
0	0	0	0	0	0	0	0
0	0	0	1	0	1	1	0
0	0	1	0	1	0	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	0	1
0	1	1	0	0	0	0	1
0	1	1	1	×	×	×	×
1	0	0	0	1	0	0	0
1	0	0	1	1	1	1	0
1	0	1	0	0	1	1	0
1	0	1	1	×	×	×	×
1	1	0	0	0	0	1	0
1	1	0	1	×	×	×	×
1	1	1	0	×	×	×	×
1	1	1	1	×	×	×	×

Ejemplos: fila 2 (entran 25): se abren S_A y S_C $(5+20)$. Fila 6 (entran 40): solo S_D. Fila 9 (35): S_A, S_B, S_C. Fila 3 (55): S_A, S_B, S_D $(5+10+40)$.

3 Salida SA

$\mathrm{SA} = \Sigma(2\text{-}5, 8, 9)$ con indiferentes $d(7, 11, 13, 14, 15)$. Simplificada: $\mathrm{SA} = \overline{A}\,\overline{B}\,C + \overline{A}\,B\,\overline{C} + A\,\overline{B}\,\overline{C}$. Circuito de dos niveles: 3 AND de 3 entradas, 1 OR de 3 entradas, 3 inversores (A, B, C). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Salida SB

$\mathrm{SB} = \Sigma(1, 3, 4, 9, 10)$ con indiferentes $d(7, 11, 13, 14, 15)$. Simplificada: $\mathrm{SB} = \overline{B}\,D + \overline{A}\,B\,\overline{C}\,\overline{D} + A\,C$. Circuito de dos niveles: 2 AND de 2 entradas y 1 AND de 4 entradas, 1 OR de 3 entradas, 4 inversores (A, B, C, D). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Salida** SC

$\mathrm{SC} = \Sigma(1, 2, 9, 10, 12)$ con indiferentes $d(7, 11, 13, 14, 15)$. Simplificada: $\mathrm{SC} = \overline{B}\,\overline{C}\,D + \overline{B}\,C\,\overline{D} + A\,B$. Circuito de dos niveles: 1 AND de 2 entradas y 2 AND de 3 entradas, 1 OR de 3 entradas, 3 inversores (B, C, D). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

6 **Salida** SD

$\mathrm{SD} = \Sigma(3, 5, 6)$ con indiferentes $d(7, 11, 13, 14, 15)$. Simplificada: $\mathrm{SD} = C\,D + B\,D + B\,C$. Circuito de dos niveles: 3 AND de 2 entradas, 1 OR de 3 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

✅ **Resultado final**

Las cuatro funciones del libro son correctas y mínimas. Se comprueba por código que los semáforos abiertos suman exactamente el flujo de entrada en las 11 filas válidas. $S_D = CD + BD + BC$ es una función **mayoría** de tres variables.

`Libro · Cap. 4 · P4.17` **Transcodificador de binario a dos dígitos BCD** ✔ verificado

Implementa un transcodificador que pase un número en binario puro $E_3E_2E_1E_0$ (0 a 15) a dos dígitos BCD: decenas $A_3A_2A_1A_0$ y unidades $B_3B_2B_1B_0$.

🧭 Antes de empezar: qué piden y qué teoría usas

Basta pasar a decimal y escribir en BCD. Hasta 9 solo hay unidades. De 10 a 15 hay una decena (1) y las unidades son el número menos 10. Las tres cifras más significativas de las decenas valen siempre 0.

1 Tabla de verdad

E3	E2	E1	E0	A3	A2	A1	A0	B3	B2	B1	B0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	1	0
0	0	1	1	0	0	0	0	0	0	1	1
0	1	0	0	0	0	0	0	0	1	0	0
0	1	0	1	0	0	0	0	0	1	0	1
0	1	1	0	0	0	0	0	0	1	1	0
0	1	1	1	0	0	0	0	0	1	1	1
1	0	0	0	0	0	0	0	1	0	0	0
1	0	0	1	0	0	0	0	1	0	0	1
1	0	1	0	0	0	0	1	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	1
1	1	0	0	0	0	0	1	0	0	1	0
1	1	0	1	0	0	0	1	0	0	1	1
1	1	1	0	0	0	0	1	0	1	0	0
1	1	1	1	0	0	0	1	0	1	0	1

Ejemplos: $9 \to 0000\,1001$; $10 \to 0001\,0000$; $15 \to 0001\,0101$.

2 Decenas: $A_3 = A_2 = A_1 = 0$

Para 4 bits de entrada la decena solo puede ser 0 o 1, así que esas tres salidas son **la constante 0** (se pueden conectar a masa; el libro las deja por coherencia con el BCD).

3 Decenas: A_0

$A_0 = \Sigma(10\text{-}15)$. Simplificada: $A_0 = E_3\,E_1 + E_3\,E_2$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas. Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

4 Unidades: B_3

$B_3 = \Sigma(8, 9)$. Simplificada: $B_3 = E_3\,\overline{E_2}\,\overline{E_1}$. Circuito de dos niveles: 1 AND de 3 entradas, 2 inversores (E_1, E_2). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

5 **Unidades:** B_2

$B_2 = \Sigma(4\text{-}7, 14, 15)$. Simplificada: $B_2 = \overline{E_3}\, E_2 + E_2\, E_1$. Circuito de dos niveles: 2 AND de 2 entradas, 1 OR de 2 entradas, 1 inversor (E_3). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

6 **Unidades:** B_1

$B_1 = \Sigma(2,3,6,7,12,13)$. Simplificada: $B_1 = \overline{E_3}\,E_1 + E_3\,E_2\,\overline{E_1}$. Circuito de dos niveles: 1 AND de 2 entradas y 1 AND de 3 entradas, 1 OR de 2 entradas, 2 inversores (E_1, E_3). Coincide en coste con la solución del libro (misma expresión).

E3E2\E1E0

	00	01	11	10
00	0	1	3 **1**	2 **1**
01	4	5	7 **1**	6 **1**
11	12 **1**	13 **1**	15	14
10	8	9	11	10

B1

Casillas con el número de minitérmino; cada lazo da un producto.

7 **Unidades:** B_0

$B_0 = \Sigma(1,3,5,7,9,11,13,15)$. Simplificada: $B_0 = E_0$. B_0 es un simple cable (o una sola variable). Coincide en coste con la solución del libro (misma expresión).

Casillas con el número de minitérmino; cada lazo da un producto.

El bit menos significativo no cambia: $B_0 = E_0$, un cable.

✅ **Resultado final**

Se comprueba que $10 \cdot (\text{decenas}) + \text{unidades}$ vale la entrada en las 16 filas. Las soluciones del libro son correctas y mínimas salvo una errata en B_2.

⚠️ **Errata en el libro (B_2)**

En la fórmula del libro $B_2 = \bar{E}_3 E_2 + E_2 E_0$. El segundo término debe ser $E_2 E_1$: con $E_2 E_0$ la fila 13 (1101) daría $B_2 = 1$ (el 13 es $0001\,0011$, y $B_2 = 0$) y la fila 14 (1110, que es $0001\,0100$) daría 0 en vez de 1. Se ha comprobado por código.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 5 · Bloques MSI

Veinte problemas con codificadores, decodificadores, multiplexores, comparadores y sumadores: cada bloque se simula y se comprueba con todas las entradas.

💡 Cómo se usa esta sección

Son los problemas resueltos del capítulo 5 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras. Aquí no se diseña con puertas sino con **bloques funcionales**: sumadores, comparadores, multiplexores, codificadores y decodificadores BCD-7 segmentos (teoría en el tema 5). Cada bloque está programado como una función, el circuito entero se simula con **todas** las combinaciones de entrada y se compara con lo que debería hacer. Los esquemas no se copian: cada circuito se describe en texto, bloque a bloque. Si el libro tiene un fallo, aparece en un recuadro de errata.

`Libro · Cap. 5 · P5.1`

Codificador decimal a BCD exceso 3 sin prioridad

✔ verificado

Completa la tabla de un codificador de 10 líneas a 4 bits que entregue el dígito decimal en BCD exceso 3, con entrada de habilitación EI, salida de habilitación EO y señal GS de «hay alguna tecla». No tiene prioridad.

🧭 Antes de empezar: qué piden y qué teoría usas

Dos ideas: (1) *sin prioridad* significa que el diseño solo garantiza el resultado cuando **una** línea está activa; (2) el exceso 3 de un dígito d es simplemente $d + 3$ en binario: $0 \to 0011$, $9 \to 1100$.

1 Qué hace cada señal

EI habilita el bloque: si vale 0, todo queda en 0. GS $(= 1)$ indica que hay alguna entrada activa. EO vale 1 solo cuando el bloque está habilitado y **no** hay ninguna entrada activa: sirve para encadenar otro codificador detrás.

2 Tabla

EI	Entrada activa	EO	S3S2S1S0	GS
1	E9	0	1100	1
1	E8	0	1011	1
1	E7	0	1010	1
1	E6	0	1001	1
1	E5	0	1000	1
1	E4	0	0111	1
1	E3	0	0110	1
1	E2	0	0101	1
1	E1	0	0100	1
1	E0	0	0011	1
1	ninguna	1	0000	0
0	(cualquiera)	0	0000	0

3 Ecuaciones

Cada bit de salida es un OR de las entradas cuyo código lo tiene a 1, todo AND con EI: $S_3 = (E_9 + E_8 + E_7 + E_6 + E_5)\,EI$, $S_2 = (E_9 + E_4 + E_3 + E_2 + E_1)\,EI$, $S_1 = (E_8 + E_7 + E_4 + E_3 + E_0)\,EI$, $S_0 = (E_8 + E_6 + E_4 + E_2 + E_0)\,EI$. Además $GS = (E_9 + \cdots + E_0)\,EI$ y $EO = \overline{E_9} \cdots \overline{E_0}\,EI$. El programa simula el bloque con las 11 combinaciones de una tecla (o ninguna) y con $EI = 0$ y $EI = 1$: las ecuaciones coinciden con la tabla.

4 Qué pasa con dos teclas a la vez

Como los ORs mezclan los códigos, pulsar E_9 y E_0 da $1100 \,|\, 0011 = 1111$, que no es ningún dígito válido. Para evitarlo hace falta un codificador **con** prioridad.

✅ Resultado final

La tabla anterior y las seis ecuaciones describen el bloque. Todas se han verificado por simulación.

Libro · Cap. 5 · P5.2

Decodificador BCD a 7 segmentos con LT, RBI y RBO

✔ verificado

Completa la tabla de un decodificador BCD a 7 segmentos cuyas entradas y salidas son todas activas a nivel bajo. Tiene prueba de lámparas (LT), supresión de ceros a la izquierda (RBI) y una línea RBO/BI que es salida de supresión y a la vez entrada de borrado.

🧭 Antes de empezar: qué piden y qué teoría usas

Pistas para leer la tabla. «Activa a nivel bajo» significa que un 0 *activa*: una salida $\bar{a} = 0$ **enciende** el segmento a, y una entrada $\bar{E}_3 \ldots \bar{E}_0$ se lee invertida (el código 1100 en las entradas es el dígito $0011 = 3$). La tabla se lee en dos direcciones: arriba, de entradas a salidas; abajo, al revés (dadas las salidas, ¿qué entradas las producen?).

1 Reglas del bloque

1) Si $BI = 0$ entra por la línea RBO/BI, todo se apaga y $RBO = 0$. 2) Si $\overline{LT} = 0$, se encienden los siete segmentos (prueba de lámparas). 3) Si $\overline{RBI} = 0$ y el dígito es 0, se apaga y $RBO = 0$: así un 0 a la izquierda no se escribe y avisa al siguiente dígito. 4) En cualquier otro caso se dibuja el dígito y $RBO = 1$.

2 Parte superior (entradas → salidas)

LT	RBI	E3	E2	E1	E0	RBO/BI	a	b	c	d	e	f	g	Qué se ve
0	0	0	0	0	0	1	0	0	0	0	0	0	0	todo encendido
1	1	1	1	0	0	1	0	0	0	0	1	1	0	3
1	1	0	1	1	1	1	0	0	0	0	0	0	0	todo encendido
1	0	1	0	0	0	1	0	0	0	1	1	1	1	7
1	1	1	1	1	1	1	0	0	0	0	0	0	1	0
1	0	1	1	1	1	0	1	1	1	1	1	1	1	apagado

LT	RBI	E3	E2	E1	E0	RBO/BI	a	b	c	d	e	f	g	Qué se ve
0	1	1	1	1	1	1	0	0	0	0	0	0	0	todo encendido
1	1	1	0	0	1	1	1	1	0	0	0	0	0	6 (sin trazo superior)
1	1	0	0	1	1	1	¿	¿	¿	¿	¿	¿	¿	código no BCD
1	0	1	1	1	0	1	1	0	0	1	1	1	1	1

La fila del código 0011 invertido (dígito 12) no es BCD: el programa confirma que el modelo no define ninguna salida y por eso se deja como «¿».

3 Parte inferior (salidas → entradas)

Se busca qué entradas dan la salida pedida. El programa recorre las 64 combinaciones de $LT, RBI, E_3..E_0$ y comprueba que cada fila del libro es cierta **para todas las combinaciones que cubren sus «X»**. Casos con dos respuestas posibles: (a) salida «todo encendido» con $RBO = 1$: $LT = 0$ y lo demás indiferente, **o** $LT = 1$ con el dígito 8 (entradas 0111); (b) «todo apagado» con $RBO = 0$: $LT = 1, RBI = 0$ y dígito 0, **o** $BI = 0$ forzado desde fuera (cualquier entrada). Las filas de los dígitos 6, 3, 1, 0, 5 y 7 salen de las entradas 1001, 1100, 1110, 1111, 1010 y 1000 respectivamente.

4 La fila «¿»

El patrón a, d, g encendidos (0110110) no coincide con ningún dígito 0–9, ni siquiera con las variantes del 6 y del 9 con o sin trazo. Es el patrón que un fabricante da a un código **no BCD** (10–15); cuál es depende del integrado, así que el ejercicio solo pide reconocer que no hay dígito.

✅ Resultado final

Las dos mitades de la tabla son coherentes con las cuatro reglas de arriba, salvo una excepción que anotamos.

⚠️ Errata en el libro

El dígito 6 aparece de dos formas: en la mitad superior y en la primera fila de la inferior sale **sin** el trazo superior ($\bar{a} = 1$, salidas 1100000), pero en la penúltima fila del bloque inferior el mismo código 1001 da **con** trazo superior (0100000). Un decodificador no puede dar dos salidas distintas para la misma entrada: una de las dos filas es una errata de copia. Aquí se ha tomado la versión sin trazo, que es la que aparece dos veces.

`Libro · Cap. 5 · P5.3`

Qué hace una red de ANDs y sumadores completos

✔ verificado

Un circuito tiene nueve puertas AND y seis sumadores completos conectados en dos filas. Rellena una tabla de funcionamiento con cinco pares de entradas $A_{2\text{-}0}$, $B_{2\text{-}0}$ y averigua qué operación realiza.

🧭 Antes de empezar: qué piden y qué teoría usas

Cómo se analiza: se pone un valor en cada entrada y se propaga hacia la salida bloque a bloque. Cada AND forma un producto parcial $A_i B_j$; cada sumador completo (FA) suma tres bits y deja un bit de suma y un acarreo.

1 Producto parcial por filas

Las tres ANDs de cada fila multiplican **un bit de** A por los tres de B. La fila de A_0 está sin desplazar, la de A_1 desplazada una posición y la de A_2 dos. Las dos filas de sumadores van acumulando esas tres filas columna a columna, y el acarreo de cada FA pasa al siguiente.

2 Tabla

$A_{2\text{-}0}$	$B_{2\text{-}0}$	$P_{5\text{-}0}$	decimal
010 (2)	110 (6)	001100	12
111 (7)	111 (7)	110001	49
000 (0)	000 (0)	000000	0
110 (6)	110 (6)	100100	36
001 (1)	010 (2)	000010	2

Ejemplo 010×110: $A_0 = 0$ da la fila 000, $A_1 = 1$ da 110 (desplazada), $A_2 = 0$ da 000. Suma: $1100 = 12 = 2 \times 6$.

3 Qué operación es

Los resultados son $2 \cdot 6 = 12$, $7 \cdot 7 = 49$, 0, $6 \cdot 6 = 36$ y $1 \cdot 2 = 2$: es un **multiplicador de 3×3 bits**. El programa modela las nueve ANDs y los seis FA y comprueba que da $A \times B$ en las 64 combinaciones de entrada.

✅ Resultado final

El circuito es un multiplicador binario sin signo de 3×3 bits con salida de 6 bits. Verificado con las 64 combinaciones.

`Libro · Cap. 5 · P5.4`

Mostrar el mayor de dos números decimales

✔ verificado

Dos selectores giratorios dan los dígitos decimales A y B. Un 7 segmentos debe mostrar solo el mayor; si son iguales no se ve nada. Diseña el circuito con bloques MSI.

🧭 Antes de empezar: qué piden y qué teoría usas

Receta general: *codificar → comparar → elegir → decodificar*. Cada selector entrega una línea de 10 (una por dígito), así que lo primero es pasarla a binario.

1 Codificar

Un codificador 10:4 por selector convierte «la línea k activa» en el número k en BCD (4 bits).

2 Comparar

Un comparador de 4 bits da $A > B$ y $A = B$.

3 Elegir

Un cuádruple multiplexor 2:1, mandado por $A > B$, deja pasar A si $A > B = 1$ y B en caso contrario. Cuando son iguales da igual cuál pase, porque el display se borrará.

4 Mostrar

La salida del multiplexor va a un decodificador BCD-7 segmentos. La señal $A = B$ se conecta a su entrada de borrado BI: si $A = B$, el display queda apagado.

5 Comprobación

El programa prueba los 100 pares (A, B): muestra $\max(A, B)$ en todos los casos con $A \neq B$ y apagado en los 10 con $A = B$.

✅ Resultado final

Coste: 2 codificadores 10:4, 1 comparador de 4 bits, 1 cuádruple mux 2:1 y 1 decodificador BCD-7 segmentos con entrada de borrado. Verificado con los 100 pares.

`Libro · Cap. 5 · P5.5`

Restar dos dígitos y mostrar solo si es positivo

✔ verificado

Dos selectores dan los dígitos decimales A y B. Un 7 segmentos debe mostrar $A - B$ solo si el resultado es positivo o cero; si es negativo, no debe mostrarse nada.

🧭 Antes de empezar: qué piden y qué teoría usas

Restar en binario es sumar el opuesto: $A - B = A + (-B)$, y en complemento a dos $-B = \overline{B} + 1$. Con 4 bits se pueden representar los números $-8 \ldots +7$.

1 Diseño del libro

1) Un sumador 4+4 calcula $-B = \overline{B} + 1$ (cuatro inversores y la constante 0001). 2) Otro sumador 4+4 hace $A + (-B)$ y da $R_3R_2R_1R_0$. 3) El resultado va al decodificador BCD-7 segmentos, y el bit R_3 se lleva a su entrada de borrado: $R_3 = 1$ ⇒ «resta negativa» ⇒ display apagado. Hace falta un sumador 4+4 (no uno 1+1) porque el $+1$ puede generar acarreos entre bits.

2 Qué comprueba el programa

Se simulan los 100 pares (A, B) de dígitos y se compara con el resultado esperado (mostrar $A - B$ si $A \geq B$, apagado si $A < B$). El diseño del libro acierta en 96 pares y **falla en cuatro**:

3 Los cuatro pares que fallan

A	B	A−B	R3R2R1R0	Debería mostrar	Muestra
0	9	-9	0111	nada	7
8	0	8	1000	8	nada
9	0	9	1001	9	nada
9	1	8	1000	8	nada

La resta de dos dígitos va de -9 a $+9$, pero con 4 bits en complemento a dos solo caben $-8 \ldots +7$. Si la diferencia es **8 o 9** (1000 y 1001, con $R_3 = 1$), el circuito la toma por negativa y **borra** un resultado que debía verse. Si la diferencia es -9 ($0 - 9$), el resultado se desborda a $+7$ (0111, $R_3 = 0$) y el circuito **muestra un 7** en lugar de apagarse.

4 Diseño corregido

Se usa **un solo sumador 4+4** con A, $\overline{B}$ y $C_{in} = 1$. Su acarreo de salida C_{out} vale 1 si y solo si $A \geq B$ (comprobado para los 256 pares de 4 bits). Se conecta $\overline{C_{out}}$ a BI. Las cuatro salidas $S_3S_2S_1S_0$ son directamente el resultado, hasta 9. El programa confirma que acierta en los 100 pares.

✅ Resultado final

La estructura del libro sirve cuando $-8 \leq A - B \leq 7$, pero falla en $8 - 0$, $9 - 0$ y $9 - 1$ (borra) y en $0 - 9$ (muestra un 7).

> **⚠️ Errata en el libro**
>
> Con R_3 como indicador de signo falla en cuatro pares de dígitos: con diferencia 8 o 9 (tres pares) borra un resultado válido, y con $0 - 9$ el desbordamiento da $+7$ y lo muestra. La solución más simple es usar el acarreo de salida de un único sumador $A + \overline{B} + 1$ y negarlo hacia BI. El libro avisa de que no hay que comparar con 0000 con un comparador, lo cual es cierto; pero tampoco basta con mirar el bit de signo.

Libro · Cap. 5 · P5.6 Mostrar el mayor de tres números

✔ verificado

Tres selectores dan los dígitos decimales A, B y C. Un 7 segmentos debe mostrar el mayor de los tres.

🧭 Antes de empezar: qué piden y qué teoría usas

Torneo de eliminación: el mayor de tres se obtiene en dos rondas, igual que en el problema de dos números.

1 Ronda 1

Un comparador $A > B$ gobierna un cuádruple mux 2:1 que da R': si $A > B$, $R' = A$; si no, $R' = B$ (da igual cuál se tome cuando son iguales).

2 Ronda 2

Otro comparador compara $R' > C$ y manda otro cuádruple mux 2:1: si $R' > C$ el resultado es $R = R'$, si no, $R = C$.

3 Mostrar

R entra en un decodificador BCD-7 segmentos.

4 Comprobación

Se simulan las $16^3 = 4096$ ternas de números de 4 bits (incluidos los empates) y el resultado es siempre el máximo. En los empates da igual qué rama se escoja porque el valor es el mismo.

✅ Resultado final

Dos comparadores de 4 bits, dos cuádruples multiplexores 2:1 y un decodificador. Verificado con las 4096 ternas.

`Libro · Cap. 5 · P5.7`

Mostrar los dos mayores de tres números

✔ verificado

Tres selectores dan los dígitos decimales A, B y C. Dos 7 segmentos deben mostrar los dos mayores.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: tres comparaciones ($A > B$, $A > C$, $B > C$) determinan el orden completo de los tres números. Esas tres señales sirven de código de selección de dos multiplexores de 8 entradas.

1 Código de selección

Con $x = A > B$ (peso 4), $y = A > C$ (peso 2) y $z = B > C$ (peso 1) hay 8 combinaciones, pero **dos son imposibles**: $x = 1, z = 1, y = 0$ ($A > B > C$ y $A \leq C$) y $x = 0, z = 0, y = 1$. Corresponden a los índices 5 y 2, y se dejan sin conectar (N.C.).

2 Entradas de los multiplexores

x y z	Orden	M1	M2
000	$A \leq B \leq C$	B	C
001	$A \leq C < B$	B	C
011	$C < A \leq B$	A	B
100	$B < A \leq C$	A	C
110	$A > C \geq B$	A	C
111	$A > B > C$	A	B

En cada fila M1 y M2 son los **dos mayores**, sin garantizar quién es el primero: por ejemplo en $x\,y\,z = 001$ el mayor es B y el segundo C, y el libro los saca por M1 y M2 en ese orden, mientras que en 011 el mayor es B y sale por M2.

3 Comprobación

Con las $16^3 = 4096$ ternas el par $\{M_1, M_2\}$ coincide siempre con los dos mayores, **incluidos los empates**, y nunca se llega a los índices 2 y 5. El libro deja como ejercicio saber cuál es el mayor.

4 Variante: M1 el mayor y M2 el segundo

Basta con cambiar las entradas de los multiplexores: M1: $0 \to C$, $1 \to B$, $3 \to B$, $4 \to C$, $6 \to A$, $7 \to A$; M2: $0 \to B$, $1 \to C$, $3 \to A$, $4 \to A$, $6 \to C$, $7 \to B$. El programa comprueba que así M1 es siempre el mayor y M2 el segundo en las 4096 ternas.

✅ Resultado final

Tres comparadores y dos multiplexores 4×8:1 (cuádruples) más dos decodificadores. Todo

comprobado, incluidos los empates.

Libro · Cap. 5 · P5.8

Valor absoluto de un número en complemento a dos

✔ verificado

Diseña un circuito que obtenga el valor absoluto de un número de cuatro bits en complemento a dos.

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: el valor absoluto es A si es positivo y $-A$ si es negativo. El signo está en el bit más significativo, A_3.

1 Calcular $-A$

Cuatro inversores y un sumador 4+4 con la constante 0001 dan $\overline{A}+1$.

2 Elegir

Un cuádruple multiplexor 2:1, mandado por A_3, deja pasar A si $A_3=0$ y $-A$ si $A_3=1$.

3 Tabla de comprobación

A	valor		A	calculado	valor
0000	0	0000	0		
0001	1	0001	1		
0010	2	0010	2		
0011	3	0011	3		
0100	4	0100	4		
0101	5	0101	5		
0110	6	0110	6		
0111	7	0111	7		

A	valor		A	calculado	valor
1000	-8	1000	-8		
1001	-7	0111	7		
1010	-6	0110	6		
1011	-5	0101	5		
1100	-4	0100	4		
1101	-3	0011	3		
1110	-2	0010	2		
1111	-1	0001	1		

El programa simula las 16 entradas: en 15 de ellas el resultado es correcto.

✅ **Resultado final**

Un sumador 4+4, cuatro inversores y un cuádruple mux 2:1.

⚠️ **Errata en el libro**

Con 4 bits el número 1000 (-8) no tiene opuesto representable: el circuito devuelve 1000 , que sigue valiendo -8 y no $+8$. Es el único caso en el que el valor absoluto no es correcto. El libro no lo menciona; se ha detectado simulando las 16 entradas.

`Libro · Cap. 5 · P5.9`

Nota final: media por defecto, por exceso, mínima o máxima

✔ verificado

Se tienen dos notas N_1 y N_0 (cada una un dígito BCD). Con dos líneas de control C_1C_0 se elige la nota final: 00 media redondeada por defecto, 01 nota mínima, 10 nota máxima, 11 media redondeada por exceso. Muéstrala en un 7 segmentos.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Dividir entre 2 en binario es desplazar un bit a la derecha: el bit que se pierde es el resto. Si el resto es 1, redondear por exceso es sumar 1.

1 Media por defecto

Un sumador 4+4 da $A + B = C_{out} S_3 S_2 S_1 S_0$ (5 bits). Se descarta S_0: queda $C_{out} S_3 S_2 S_1$, el cociente de dividir entre 2.

2 Media por exceso

Se suma S_0 al cociente anterior: con $S_0 = 0$ se suma 0 y con $S_0 = 1$ se suma 1. Así sale $\lceil (A + B)/2 \rceil$. Un segundo sumador 4+4 hace esa suma.

3 Mínimo y máximo

Un comparador de 4 bits y dos cuádruples mux 2:1 eligen el menor y el mayor.

4 Elegir

Un cuádruple mux 4:1, mandado por $C_1 C_0$, escoge entre las cuatro opciones (en el orden de la tabla del enunciado) y el decodificador BCD-7 segmentos las muestra.

5 Ejemplos

N_1	N_0	suma	por defecto	por exceso
3	4	7	3	4
3	5	8	4	4
8	9	17	8	9

Se ha comprobado con las 100 parejas de dígitos que las cuatro opciones coinciden con $\lfloor (a + b)/2 \rfloor$, $\min$, $\max$ y $\lceil (a + b)/2 \rceil$.

✅ Resultado final

Dos sumadores 4+4, un comparador de 4 bits, dos cuádruples mux 2:1 (mínimo y máximo) y un cuádruple mux 4:1. Verificado con las 100 parejas.

Libro · Cap. 5 · P5.10

Sensor de temperatura entre −5 °C y +4 °C

✔ verificado

Un sensor entrega la temperatura en 4 bits, en complemento a dos. La salida debe activarse cuando la temperatura esté entre -5 °C y $+4$ °C, ambos incluidos.

🧭 Antes de empezar: qué piden y qué teoría usas

Truco: los comparadores MSI comparan en binario puro, no en complemento a dos. Así que conviene mirar el intervalo como números **sin signo**.

1 Cómo se ven los valores

En 4 bits: $0000 \ldots 0100$ son $0 \ldots +4$ y $1011 \ldots 1111$ son $-5 \ldots -1$. Sin signo, los primeros son $0 \ldots 4$ y los segundos $11 \ldots 15$: dos tramos separados.

2 Dos comparadores

Un comparador da $T < 0101$ (menor que 5 ⇒ valores $0 \ldots 4$, o sea $0 \ldots +4$ °C). Otro da $T > 1010$ (mayor que 10 ⇒ $11 \ldots 15$, o sea $-5 \ldots -1$ °C).

3 Salida

Una puerta OR une las dos salidas: $Salida = (T < 5) + (T > 10)$. Se prueban los 16 valores y la salida vale 1 exactamente cuando $-5 \leq T \leq 4$.

✅ Resultado final

Dos comparadores de 4 bits con constantes 0101 y 1010, y una OR. Verificado con los 16 códigos.

Libro · Cap. 5 · P5.11

Expendedor de golosinas: dinero que falta o que sobra

✔ verificado

Un expendedor recibe el dinero introducido (4 bits, en céntimos de euro, binario puro) y el producto pedido mediante cuatro microinterruptores G, Ch, R y P (gominolas, chicle, regaliz y pipas), que cuestan 2, 3, 5 y 6 céntimos. Un 7 segmentos muestra cuánto dinero falta o sobra, y dos leds indican si debe entregarse el producto (LP) o si el usuario debe completar el dinero (LD).

🧭 Antes de empezar: qué piden y qué teoría usas

Idea: la resta *dinero* − *precio* da la información que se pide. Si es ≥ 0, el producto se entrega y la cifra es lo que sobra. Si es negativa, falta dinero y la cifra es su valor absoluto.

1 Elegir el precio

Un codificador 4:2 pasa la tecla pulsada a un código de 2 bits y un cuádruple multiplexor 4:1 saca **el precio ya negado** en complemento a dos: $-2 = 1110$, $-3 = 1101$, $-5 = 1011$, $-6 = 1010$.

2 Restar

Un sumador 4+4 hace $D + (-\text{precio}) = R$.

3 Valor absoluto de R

Cuatro inversores y otro sumador 4+4 dan $-R = \overline{R} + 1$. Un mux 2:1 gobernado por R_3 elige R o $-R$.

4 Salidas

El 7 segmentos muestra S. $LP = \overline{R_3}$ (el producto sale si $R_3 = 0$) y $LD = R_3$ (el decodificador 1:2 de las dos señales es un simple inversor).

5 Qué ocurre si sobra mucho

Se han simulado los 16 importes con los cuatro productos. El diseño del libro acierta siempre que la diferencia $D - \text{precio}$ sea **como mucho 7**; falla cuando es 8 o más: 16 casos, p. ej. $D = 15$ con gominolas ($15 - 2 = 13 = 1101$, que parece negativo porque $R_3 = 1$, y el circuito dice que falta dinero: $LD = 1$ y muestra 3).

6 Diseño corregido

Se usa el **acarreo** del primer sumador: $C_{out} = 1$ exactamente cuando $D \geq$ precio. Entonces $LP = C_{out}$, $LD = \overline{C_{out}}$ y el mux elige R si $C_{out} = 1$ y $-R$ si no. El programa lo comprueba con los 64 casos.

✅ Resultado final

El diseño del libro es correcto para sobras de hasta 7 céntimos; con el acarreo como señal de «llega el dinero» lo es en todo el rango.

> **⚠️ Errata en el libro**
>
> 1) Con R_3 como indicador de signo falla cuando la diferencia es ≥ 8 (por ejemplo, 15 céntimos para unas gominolas de 2). 2) En el texto la lista de códigos del producto (gominolas 00, chicle 01, regaliz 10, pipas 11) está invertida respecto al esquema, donde G entra por la línea 3 del codificador 4:2 y selecciona el precio -2. El esquema es el coherente.

`Libro · Cap. 5 · P5.12`

Votación en un consejo con voto de calidad, trampa y veto

✔ verificado

Cuatro países, A, B, C y D, votan en un consejo. Un 7 segmentos muestra el resultado de la votación. Si hay empate decide el voto de calidad de A. Además, el país anfitrión tiene un interruptor de trampa que da la vuelta al resultado. Después hay que rediseñar el circuito para seis países en los que los votos de A y B valen doble, y añadir el derecho de veto de un país.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Dos ideas:** (1) contar votos = **sumar bits**; (2) el resultado depende de si la suma es mayor, igual o menor que la mitad. Con cuatro votos, «mitad» es 2.

1 Primera solución: contar con sumadores y comparar

Dos sumadores 1+1 dan $S' = A + B$ y $S'' = C + D$ (2 bits cada uno), y otro $S = S' + S''$ de 3 bits (0–4). Un comparador de 3 bits compara S con 010: $S > 2$ ⇒ voto 1; $S = 2$ ⇒ voto de A; $S < 2$ ⇒ voto 0. Un multiplexor 4:1 gobernado por $(S > 2,\ S = 2)$ elige entre 0, A y 1 (la combinación «mayor y a la vez igual» es imposible y queda sin conectar). Para la trampa un mux 2:1 elige entre V y $\overline{V}$ según el interruptor. El resultado (V_0) y tres ceros forman el dígito BCD $000V_0$ que va al decodificador: muestra 0 o 1.

2 Segunda solución: fuerza bruta

Un solo multiplexor 16:1 con A, B, C, D como selección y constantes en las entradas. Las 16 posiciones se rellenan a mano, porque se sabe qué vale el resultado en cada caso:

A B C D	votos a favor	resultado (sin trampa)
0000	0	0
0001	1	0
0010	1	0
0011	2	0
0100	1	0
0101	2	0
0110	2	0
0111	3	1
1000	1	0
1001	2	1
1010	2	1
1011	3	1
1100	2	1
1101	3	1
1110	3	1
1111	4	1

Con exactamente dos votos a favor, el resultado es el voto de A: es 1 en las posiciones 9, 10 y 12 ($A = 1$) y 0 en la 3, 5 y 6. Las demás se rellenan por mayoría. Lo demás del circuito es igual (trampa y decodificador).

3 Tercera solución: seis países, A y B valen doble

Los votos pesan $2A + 2B + C + D + E + F$, con total 8; la mayoría es > 4 y el empate es $= 4$. Un sumador 2+2 suma los números de dos bits AC y BD (donde A y B ocupan el bit de peso 2, así valen doble) y E entra por su acarreo de entrada. Su resultado de 3 bits se suma con 000 y con F como acarreo de entrada en un sumador 3+3. Así los pesos no se mezclan; no valdría un sumador 2+2 *sin* agrupar, porque dos países valdrían doble por la posición equivocada. La salida de 4 bits se compara con 0100. Un mux 4:1 elige 0, A o 1 y otro 2:1 la

trampa. La línea de **veto** va a la entrada de borrado del decodificador: si algún país veta, la pantalla se apaga.

4 Comprobación

Con las 16 combinaciones de 4 países y las dos posiciones de la trampa, las soluciones 1 y 2 dan lo mismo que una cuenta directa (empate para A). Con las 64 combinaciones de 6 países, las 2 de la trampa y las 2 del veto (256 casos) la solución de seis países coincide con $2A + 2B + C + D + E + F$ comparado con 4.

✅ Resultado final

Tres diseños verificados: dos sumadores 1+1 + sumador + comparador + mux (elegante), un mux 16:1 (fuerza bruta) y el rediseño de seis países con veto.

`Libro · Cap. 5 · P5.13`

Votación por bloques: cuatro países en A y tres en B

✔ verificado

En una votación los países se agrupan en dos bloques: cuatro en el bloque A y tres en el B. Un 7 segmentos muestra el resultado: si hay mayoría en A, ése es el resultado; si hay empate en A, decide el bloque B. Diseña el circuito.

🧭 Antes de empezar: qué piden y qué teoría usas

Cómo se razona: hay que saber **cuántos** votos a favor hay en A (0 a 4) y en B (0 a 3). La regla es: $a > 2 \Rightarrow 1$; $a < 2 \Rightarrow 0$; $a = 2 \Rightarrow$ el resultado lo da B, y gana el 1 si en B hay al menos dos votos.

1 Primera solución

Cuatro sumadores 1+1 para el bloque A en dos capas (primero $A_3 + A_2$ y $A_1 + A_0$, luego se suman sus resultados de 2 bits columna a columna, $S_{A2..0}$) y un sumador 1+1 para B (suma 3 bits, uno por el acarreo de entrada, y da $S_{B1}S_{B0}$). Un comparador de 3 bits compara S_A con 010 ($A > 2$ y $A = 2$, la señal C). Un comparador de 2 bits compara S_B con 01 ($B > 1$). Un mux 2:1 gobernado por C elige entre $(A > 2)$ si $C = 0$ y $(B > 1)$ si $C = 1$. Se visualiza el resultado con $000V$ en el decodificador.

2 Segunda solución

Muy parecida, pero con sumadores 4+4 en lugar de 1+1: se rellenan con ceros las entradas libres. Funciona igual y no mezcla pesos porque la suma se hace en dos niveles.

3 Tercera solución: con puertas

Tres sumadores 1+1 para A (resultado $S_{A2}S_{A1}S_{A0}$) y uno para B ($S_{B1}S_{B0}$). El voto vale 1 si: $S_{A2} = 1$ (cuatro votos), $S_{A1}S_{A0} = 11$ (tres), o $S_{A1}S_{A0} = 10$ (dos) y además $S_{B1} = 1$. O sea $V = S_{A2} + S_{A1}S_{A0} + S_{A1}\overline{S_{A0}}S_{B1}$: una AND y una OR detectan «3 ó 4», otra AND resuelve el caso de 2 con B. Se sustituyen los comparadores y el multiplexor por tres puertas.

4 Comprobación

Las tres soluciones se han simulado con las $2^7 = 128$ combinaciones de votos y las tres coinciden con la regla $a > 2$ ó $(a = 2$ y $b \geq 2)$.

✅ Resultado final

Las tres soluciones son equivalentes. La tercera es la más barata; las dos primeras son más fáciles de ampliar si cambia el número de países.

`Libro · Cap. 5 · P5.14`

Reloj con alarma: cuatro displays y cuatro vistas

✔ verificado

Cuatro 7 segmentos deben mostrar, según dos señales de control P_1 y P_2, el mes, el día, la hora, los minutos, los segundos y las décimas de segundo del día en curso, y también la hora y los minutos de la alarma. Diseña el circuito.

🧭 Antes de empezar: qué piden y qué teoría usas

Un multiplexor por display. Cada display SG_k necesita elegir su dígito entre cuatro fuentes, y la elección la hacen las mismas dos líneas P_1P_2 para los cuatro. Es el uso clásico de un **cuádruple multiplexor 4:1**: 4 bits de dato por entrada, 4 entradas.

1 Qué muestra cada combinación

P_1P_2	Se ve	SG_3	SG_2	SG_1	SG_0
00	HH MM	HD	HU	MD	MU

P_1P_2	Se ve	SG_3	SG_2	SG_1	SG_0
01	SS DD	SD	SU	DD	DU
10	HH MM de alarma	HAD	HAU	MAD	MAU
11	MM DD (mes y día)	MD	MU	DD	DU

Siglas: HD/HU decenas/unidades de hora; MD/MU de minutos; SD/SU de segundos; HAD/HAU y MAD/MAU de hora y minutos de alarma; DD/DU de día.

2 **Estructura**

Cuatro multiplexores 4:1 de 4 bits (uno por display), con P_1P_2 como selección común; la salida de cada uno va a un decodificador BCD-7 segmentos. Total: 4 cuádruples mux 4:1 y 4 decodificadores.

3 **Comprobación**

El programa recorre las cuatro combinaciones P_1P_2 y verifica que cada display recibe exactamente su fuente de la tabla.

✅ **Resultado final**

Con esta estructura, para mostrar otra «vista» basta añadir una entrada a cada multiplexor.

⚠️ **Errata en el libro**

En el esquema las entradas 1 de los displays SG_1 y SG_0 (vista «segundos y décimas») se llaman DD y DU, igual que las del día (entrada 3). Como las décimas de segundo no son el día del mes, deben ser señales distintas, por ejemplo dc_D y dc_U.

`Libro · Cap. 5 · P5.15` **Sumador/restador en C-1 o C-2** ✔ verificado

Diseña un sumador/restador en binario puro con signo en complemento a uno o a dos. Dos líneas de control: $COD = 0$ elige C-1 y $COD = 1$ elige C-2; $OP = 0$ suma y $OP = 1$ resta.

🧭 Antes de empezar: qué piden y qué teoría usas

Recordatorio: restar es sumar el opuesto. El opuesto en C-1 es invertir todos los bits ($\overline{B}$). En C-2 es invertir y sumar 1. La suma en C-1 necesita además el **acarreo circular**: el acarreo de salida se vuelve a sumar al bit menos significativo.

1 Negar B si hay resta

Cuatro puertas XOR: $B' = B \oplus OP$. Si $OP = 1$ invierten B; si $OP = 0$ lo dejan igual.

2 Sumar

Un sumador 4+4 hace $A + B' + C_{in}$.

3 Elegir C_{in}

Un mux 4:1 gobernado por $OP\,COD$ da: 00 (suma C-1) ⇒ $C_{in} = C_{out}$ (acarreo circular); 01 (suma C-2) ⇒ 0; 10 (resta C-1) ⇒ C_{out}; 11 (resta C-2) ⇒ 1 (el $+1$ del complemento a dos).

4 Comprobación

Para los 256 pares de operandos y las dos operaciones, el programa calcula el resultado de cada modo y lo compara con el valor aritmético: en C-2 sale bien siempre que el resultado esté en $-8 \ldots 7$, y en C-1 siempre que esté en $-7 \ldots 7$ (fuera de esos rangos hay desbordamiento y el circuito no lo detecta).

✅ Resultado final

El circuito funciona. Un único sumador 4+4 más 4 XOR y un mux 4:1.

⚠️ Errata en el libro

En los modos C-1 el acarreo de salida alimenta la entrada de acarreo del mismo sumador, es decir, hay un **bucle combinacional**. Estable no siempre es único: cuando $A + B' = 15$ (1111) hay dos soluciones consistentes (1111 con $C_{in} = 0$ y 0000 con $C_{in} = 1$). Las dos son «cero» en C-1, así que el valor es correcto, pero el circuito físico puede quedarse en cualquiera. Un diseño sin bucle usa dos sumadores en cascada (el segundo suma el acarreo del primero).

Libro · Cap. 5 · P5.16

Comparadores para C-1, C-2 y signo-magnitud con un comparador de 4 bits

✔ verificado

A partir de un comparador de cuatro bits, diseña tres comparadores: uno para binario puro con complemento a 1, otro para binario puro con complemento a 2 y otro para binario con bit de signo.

🧭 Antes de empezar: qué piden y qué teoría usas

El problema: el comparador MSI trata sus 4 bits como un número **sin signo**. Si ambos operandos tienen el mismo signo, el orden es el mismo con o sin signo; si los signos difieren, el positivo siempre gana.

1 C-1 y C-2: el mismo circuito

Se usan A_3 y B_3 para gobernar dos multiplexores 4:1, uno para $A > B$ y otro para $A < B$: con $A_3B_3 = 00$ (ambos positivos) y 11 (ambos negativos) se deja pasar la salida del comparador, porque en ambos casos el orden sin signo coincide con el orden con signo (p. ej. 1100 vs 1001: sin signo $12 > 9$, con signo $-4 > -7$). Con $A_3B_3 = 01$ (A positivo, B negativo): $A > B = 1$ y $A < B = 0$. Con 10: $A > B = 0$ y $A < B = 1$. La salida $A = B$ pasa directa.

2 Comparador con bit de signo

Antes del comparador se transcodifica cada operando a C-1 con cuatro XOR: si $A_3 = 1$ se invierte la magnitud $A_{2\text{-}0}$ ($A' = A_{2\text{-}0} \oplus A_3$) y si $A_3 = 0$ se deja igual. Después se usa el mismo circuito que antes. Ejemplo: 1100 (-4 en signo y magnitud) y 1001 (-1) se transcodifican a 1011 y 1110. Ambos son negativos, el comparador sin signo da $11 < 14$ y en efecto $-4 < -1$.

3 Comprobación

Con los 256 pares de números en C-2, el circuito da exactamente la comparación aritmética. Con C-1 y con signo-magnitud, coincide en todos salvo en dos pares ordenados: el cero con cero negativo (en C-1: 0000 y 1111; con signo: 0000 y 1000), donde el circuito dice que el positivo es mayor en lugar de «iguales».

✅ Resultado final

Un comparador de 4 bits y dos multiplexores 4:1 sirven para C-1 y C-2; para signo-magnitud se añaden dos cuartetos de XOR.

> **⚠️ Errata en el libro**
>
> Con C-1 y con signo y magnitud hay **dos representaciones del cero** ($+0$ y -0). El circuito del libro las ordena ($+0 > -0$) en vez de declararlas iguales; el libro no lo advierte. Si hace falta distinguirlo, hay que añadir una comprobación específica (por ejemplo, si los dos números son cero).

`Libro · Cap. 5 · P5.17`

Sumador BCD exceso 3 con sumadores binarios

✔ verificado

Diseña un sumador de un dígito en BCD exceso 3 utilizando sumadores 4+4 en binario puro. Compruébalo con al menos tres ejemplos. ¿Es más sencillo un sumador BCD natural o uno en exceso 3? ¿Por qué?

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Exceso 3:** el dígito d se guarda como $d + 3$. Al sumar dos dígitos se suman dos «+3», así que el resultado queda con un **exceso 6**. Hay que corregirlo para dejarlo con exceso 3 y, si la suma pasa de 9, pasar a la siguiente cifra.

1 Diseño

1) Un sumador 4+4 calcula $S' = A + B$. 2) Si $C'_{out} = 0$ no hubo acarreo del dígito (S' sigue en exceso 6 y la suma decimal es ≤ 9): hay que restar 3, es decir, sumar 1101. 3) Si $C'_{out} = 1$ la suma decimal es ≥ 10, se genera acarreo y el resultado debe quedar en exceso 3: se suma 0011. 4) La constante se obtiene con un inversor y un 1: $B_3 = B_2 = \overline{C'_{out}}$, $B_1 = C'_{out}$, $B_0 = 1$. 5) Un segundo sumador 4+4 hace $S = S' + \text{constante}$. El acarreo de salida es C'_{out}.

2 Tres ejemplos

A	B	$S' = A + B$	C'	se suma	S	resultado
0011 (0)	0011 (0)	0110	0	1101	0011	0, acarreo 0
0110 (3)	1010 (7)	0000	1	0011	0011	0, acarreo 1 (10)
1100 (9)	1100 (9)	1000	1	0011	1011	8, acarreo 1 (18)

El programa comprueba los 100 pares de dígitos.

3 ¿Cuál es más sencillo?

El de **exceso 3**: solo necesita un inversor para generar la constante. El BCD natural necesita detectar «suma > 9 o acarreo» ($C'_{out} + S_3S_2 + S_3S_1$, es decir, 2 AND y 1 OR de tres entradas) y además hacer la corrección $+0110$. La razón es que el exceso de 6 de la primera suma coincide con los 6 códigos que sobran en 4 bits: el acarreo binario es directamente el acarreo decimal y no hace falta detectar «>9». El programa simula ambos sumadores con los 100 pares.

✅ Resultado final

Dos sumadores 4+4 y un inversor. El BCD natural cuesta además 2 AND y una OR de tres entradas.

`Libro · Cap. 5 · P5.18`

Binario de 5 bits a dos dígitos BCD

✔ verificado

Diseña un circuito que pase un número de cinco bits en binario puro a dos dígitos BCD.

🧭 Antes de empezar: qué piden y qué teoría usas

Dividir entre 10: un número entre 0 y 31 tiene como máximo 3 decenas. Cada vez que se pasa de 9 hay que «llevar» una decena, y como un dígito BCD usa 4 bits (16 combinaciones) para 10 valores, hay que sumar **6** para saltarse los códigos sobrantes.

1 Paso 1: los cuatro bits bajos

Se detecta si $A_{3\text{-}0} \geq 10$ con $A_3A_2 + A_3A_1$ (2 AND y una OR). Si es así se suma 0110 con un sumador 4+4: $S' = A_{3\text{-}0} + (0 \text{ ó } 6)$; el acarreo $C_{out}S'$ vale 1 exactamente en ese caso (es la decena que hay que llevar).

2 Paso 2: el bit de peso 16

El bit A_4 vale 16, que es «1 decena y 6 unidades». Se le suma 6 al dígito de unidades: $RU = S' + (0 \text{ ó } 6)$ con A_4 como control. Esto debe hacerse con un **sumador BCD**, porque la suma puede pasar de 9 (por ejemplo $5 + 6 = 11$ ⇒ unidad 1 y acarreo 1).

3 Paso 3: decenas

$RD_{1\text{-}0} = A_4 + C_{out}S' + C_{out}RU$ con un sumador 1+1 (valores 0 a 3).

4 Ejemplos

$A_{4\text{-}0}$	S'	RU	RD
00111 (7)	0111	0111 (7)	0
01100 (12)	1100+0110 = 0010	0010 (2)	0+1+0 = 1
11111 (31)	1111+0110 = 0101	0101+0110 = 0001 (1)	1+1+1 = 3

Se han simulado las 32 entradas y siempre sale el cociente y el resto de dividir entre 10.

✅ Resultado final

Dos sumadores 4+4 (el segundo en BCD), un sumador 1+1 y una detección con 2 AND y 1 OR. Verificado con los 32 valores.

> **⚠️ Errata en el libro**
>
> En el texto el criterio de corrección se enuncia como «$A_{3\text{-}0}$ mayor que 10», pero el circuito (que usa $A_3A_2 + A_3A_1$) detecta ≥ 10, que es lo correcto: el valor 10 también necesita corrección.

`Libro · Cap. 5 · P5.19`

Recinto con dos puertas y claves ligadas

✔ verificado

Para entrar en un recinto hay que pasar dos puertas, P1 y P2. En la P1 se introduce un dígito BCD mayor o igual que 6. En la P2, un número que esté a no más de tres unidades de la mitad del introducido en la P1 (si se introduce 8, hay que meter un número entre 1 —que es 4−3— y 7 —que es 4+3—). Diseña el circuito.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Dos condiciones independientes:** abrir P1 (un comparador) y abrir P2 (un intervalo, que son dos comparaciones). La puerta se abre si se cumplen las dos.

1 Puerta 1

Se necesita $P_1 \geq 6$, es decir, $P_1 > 5$: un comparador de 4 bits con la constante 0101.

2 Puerta 2: dividir entre 2

Dividir entre 2 en binario puro es quitar el bit de menos peso: $P = P_{1\,3\text{-}1}$ (3 bits).

3 Puerta 2: los dos extremos

Un sumador 4+4 calcula $P + 3$ ($P + 0011$) y otro $P - 3$, que se obtiene sumando 13 (1101), porque $-3 \equiv 13 \pmod{16}$. Dos comparadores de 4 bits comparan P_2 con cada extremo: $P_2 \leq P + 3$ (salidas «menor» o «igual») y $P_2 \geq P - 3$ (salidas «mayor» o «igual»). La puerta 2 se abre si se cumplen las dos: 2 OR y 1 AND.

4 Rango útil

Como $P_1 \geq 6$, P vale 3 o 4, así que $P - 3$ es 0 o 1 y nunca «da la vuelta» a números negativos. Por eso basta con comparar en binario puro.

5 Comprobación

El programa recorre $P_1 \in 0..9$ y $P_2 \in 0..15$ y confirma las dos condiciones.

✅ Resultado final

Tres comparadores, dos sumadores 4+4, 3 puertas OR/AND. Con la corrección indicada abajo el circuito cumple el enunciado en todo el rango.

> **⚠️ Errata en el libro**
>
> En el esquema el comparador de la puerta 1 tiene la constante 0101 en la entrada A y la clave P_1 en la B, y se toma la salida $A > B$. Eso da «$5 > P_1$», o sea, abriría la puerta para $P_1 < 5$, justo al revés de lo que dice el enunciado ($P_1 \geq 6$). La solución es tomar la salida $A < B$ ($5 < P_1$) o intercambiar las entradas. Comprobado por código.

`Libro · Cap. 5 · P5.20`

Tres personas y cuatro leds según cuántos números coinciden

✔ verificado

Tres personas codifican con microinterruptores, cada una en dos bits, un número de 0 a 3. Diseña el circuito que active uno de los cuatro leds: L3 si los tres números son iguales, L2 si al menos dos son iguales, L2S si solo dos son iguales y L1 en el resto de casos.

🧭 Antes de empezar: qué piden y qué teoría usas

Tres parejas posibles: (A,B), (A,C) y (B,C). Si se comparan las tres parejas con comparadores de igualdad de 2 bits, cada led es una combinación sencilla de los tres resultados e_{AB}, e_{AC}, e_{BC}.

1 Comparar

Tres comparadores de 2 bits, usando solo la salida $A = B$.

2 L3 y L2

$L3 = e_{AB}\, e_{BC}$ (si $A = B$ y $B = C$, los tres son iguales). $L2 = e_{AB} + e_{AC} + e_{BC}$ (al menos una pareja coincide).

3 L2S

Solo una pareja coincide: $L2S = e_{AB}\overline{e_{AC}} + e_{AC}\overline{e_{BC}} + e_{BC}\overline{e_{AC}}$ (3 AND, un OR y 2 inversores). Es más fácil de leer como «hay exactamente una pareja que coincide».

4 L1

$L1 = \overline{L3 + L2 + L2S}$, una NOR de tres entradas (o 3 inversores y 1 AND). Pero $L3$ y $L2S$ implican $L2$, así que $L1 = \overline{L2}$ basta: **un solo inversor**.

5 Comprobación

Con las $4^3 = 64$ ternas, las cuatro salidas coinciden con la cuenta de cuántos valores distintos hay (1, 2 o 3).

✅ Resultado final

Tres comparadores de 2 bits y unas pocas puertas. Verificado con las 64 ternas, y se ha

comprobado que $L1 = \overline{L2}$.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 6 · Biestables

Quince problemas de cronogramas y análisis de biestables R-S, J-K, D, T, maestro/esclavo y latch: cada cronograma sale de una simulación con puertas.

💡 Cómo se leen los cronogramas de esta sección

Son los problemas resueltos del capítulo 6 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras y con entradas dibujadas sobre una **cuadrícula de ranuras de tiempo** (cada línea vertical está separada cuatro ranuras; un semiperíodo de reloj son 4 ranuras). La teoría está en el tema 6.

Convenio de los dibujos:

- **Banda gris clara**: valor desconocido a priori (la salida se ha parado, pero no se sabe en qué valor).
- **Banda gris oscura**: carrera (la salida bascula sin parar mientras dura la condición).
- **Trazo grueso**: entrada prohibida en ese momento.
- En un flanco se toma el valor de las entradas **justo antes** de él.

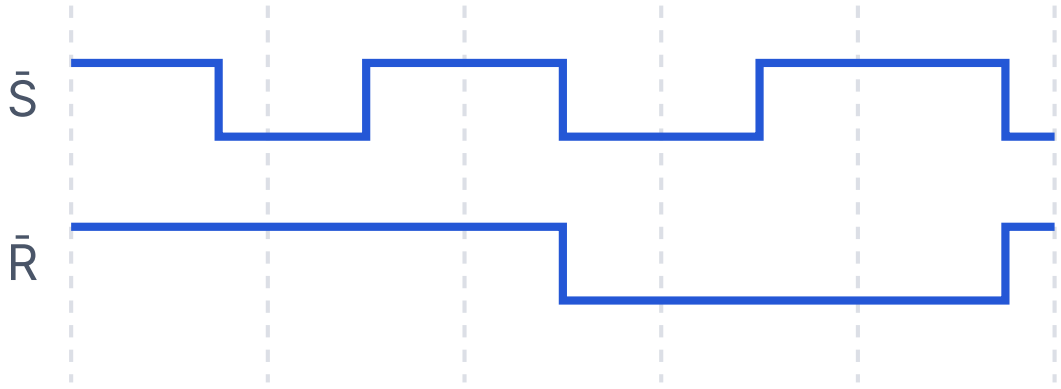
Cada latch (R-S, J-K, D, y los maestros/esclavos) se ha simulado como una red de puertas NAND: las puertas se disparan en cualquier orden y, si el resultado final depende de quién gana, la salida se marca como desconocida. Las carreras se detectan cuando la combinación J=K=1 realimenta la salida mientras el nivel está activo. Los flip-flops por flanco se simulan con su tabla de excitación. Así los cronogramas no se dibujan a mano: salen del simulador y se contrastan con el razonamiento. Si el libro tiene un fallo, aparece en un recuadro de errata.

Libro · Cap. 6 · P6.1

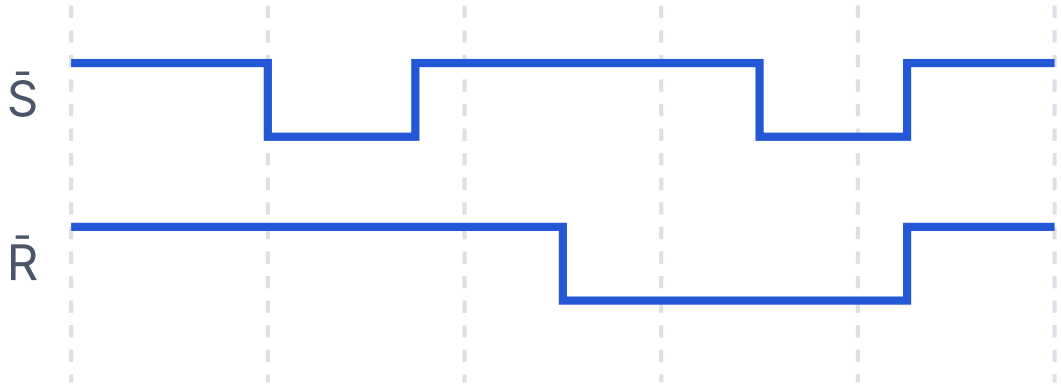
Báscula R-S asíncrona con puertas NAND

✔ verificado

Completa los dos cronogramas de una báscula R-S asíncrona hecha con dos puertas NAND (entradas activas en bajo, $\overline{S}$ y $\overline{R}$). Marca con trazo grueso cuándo la entrada está prohibida y en gris cuándo no se puede saber el valor de salida.



Primer cronograma: entradas



Segundo cronograma: entradas

🧭 Antes de empezar: qué piden y qué teoría usas

Una tabla de cuatro filas basta. Con $Q = \overline{\overline{S} \cdot \overline{Q}}$ y $\overline{Q} = \overline{\overline{R} \cdot Q}$: $\overline{S}\,\overline{R} = 11$ mantiene; 01 pone Q a 1; 10 pone Q a 0; 00 es la combinación prohibida y deja **las dos salidas a 1**.

1 Reglas

$\overline{S}$	$\overline{R}$	Operación	Q	$\overline{Q}$
1	1	mantiene	Q	$\overline{Q}$
0	1	puesta a 1	1	0
1	0	puesta a 0	0	1
0	0	prohibida	1	1

2 Primer cronograma

Q se pone a 1 con el primer pulso bajo de $\overline{S}$ y se mantiene. Cuando $\overline{R}$ baja mientras $\overline{S}$ también está bajo, la entrada es prohibida y las dos salidas valen 1 (trazo grueso). En cuanto $\overline{S}$ sube y $\overline{R}$ sigue bajo, Q pasa a 0: sale de la prohibida por **un solo lado**, y el resultado es único. Al final, $\overline{R}$ sube y $\overline{S}$ baja: Q vuelve a 1.

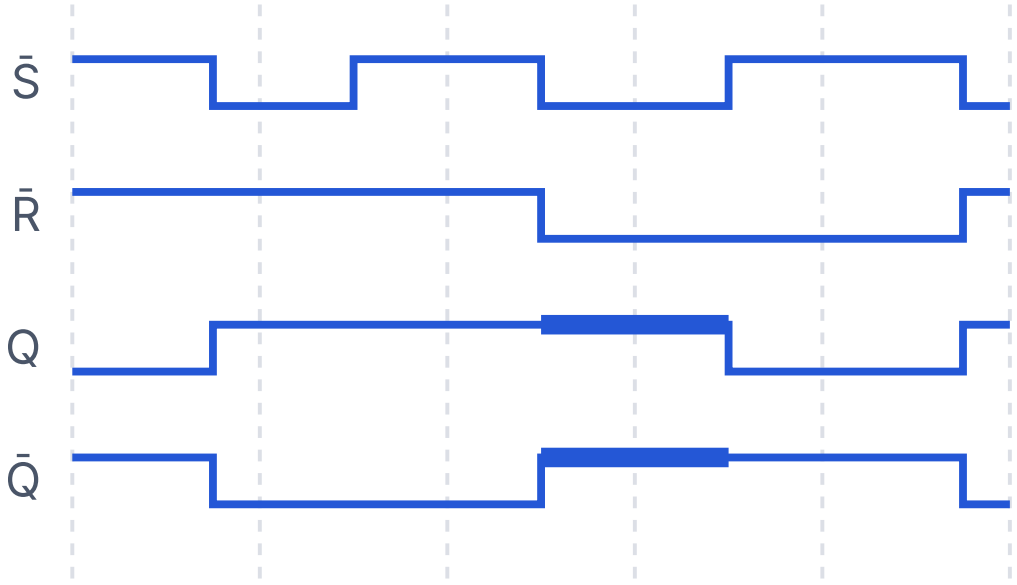
3 Segundo cronograma

Tras la puesta a 1 y la puesta a 0, las dos entradas bajan **a la vez** (prohibida: $Q = \overline{Q} = 1$) y después suben también **a la vez**. Las dos puertas ven ahora «1, 1» y compiten: la que gane se queda a 0 y la otra a 1, pero no se sabe cuál. La salida queda desconocida (banda gris) hasta la siguiente puesta a 0 o a 1.

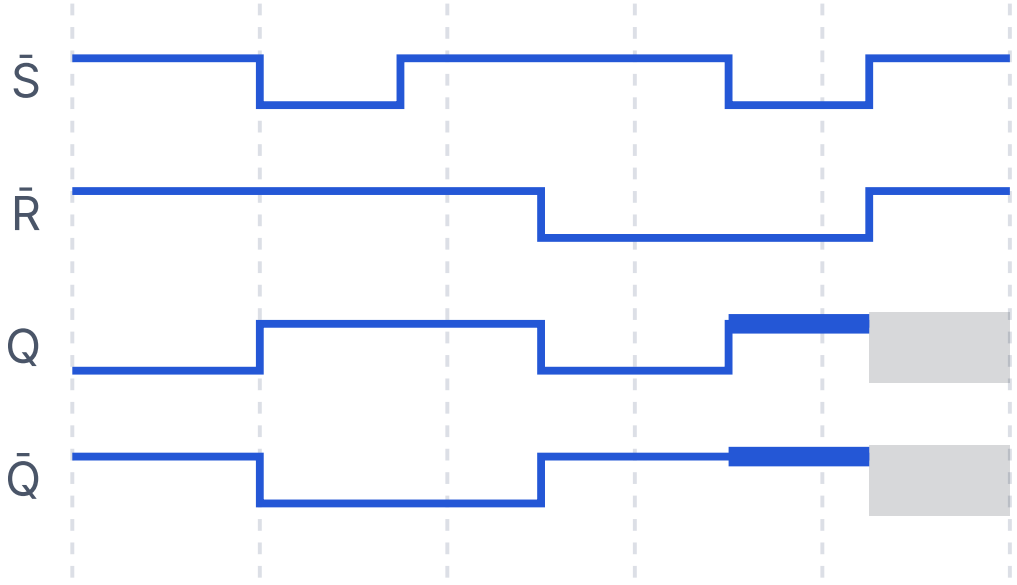
4 Qué enseña

Pasar de 00 a 11 por un solo lado es seguro; hacerlo por los dos a la vez deja la báscula en un estado imprevisible. Sea cual sea el estropicio (prohibida, desconocida), una orden clara de puesta a 0 o a 1 devuelve la báscula a la normalidad.

✅ Resultado final



Cronograma 1



Cronograma 2

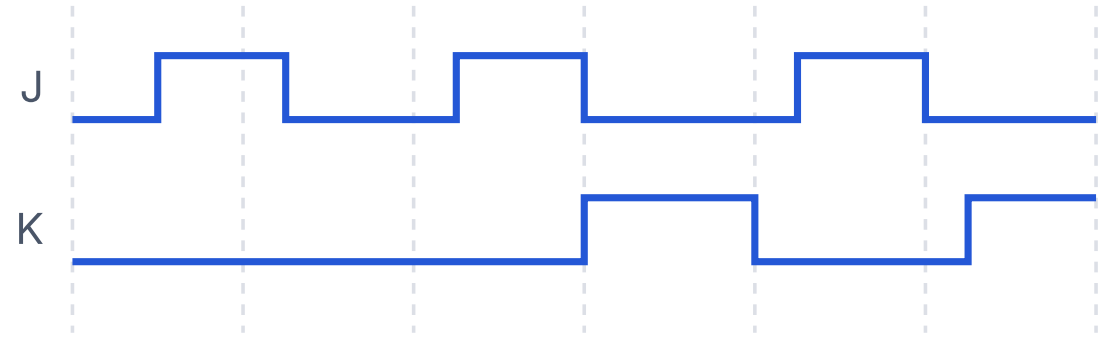
En el primero toda la evolución es determinista (con una fase prohibida); en el segundo, la salida queda desconocida tras salir de la prohibida por los dos lados a la vez.

Libro · Cap. 6 · P6.2

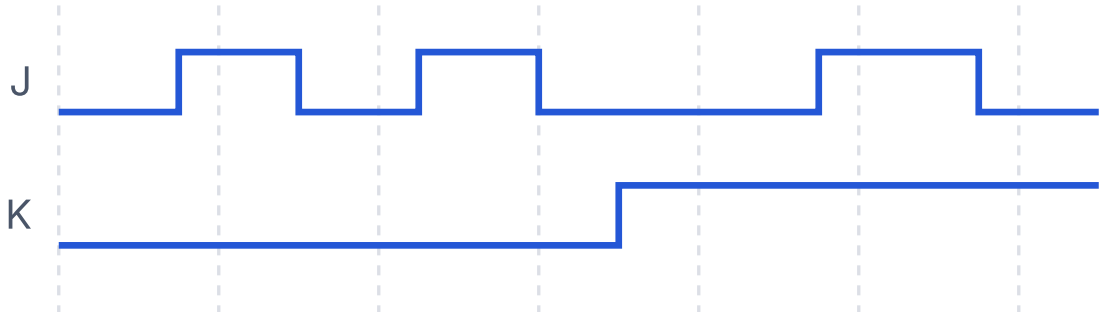
Báscula J-K asíncrona: normal, con carreras y con indeterminación

✔ verificado

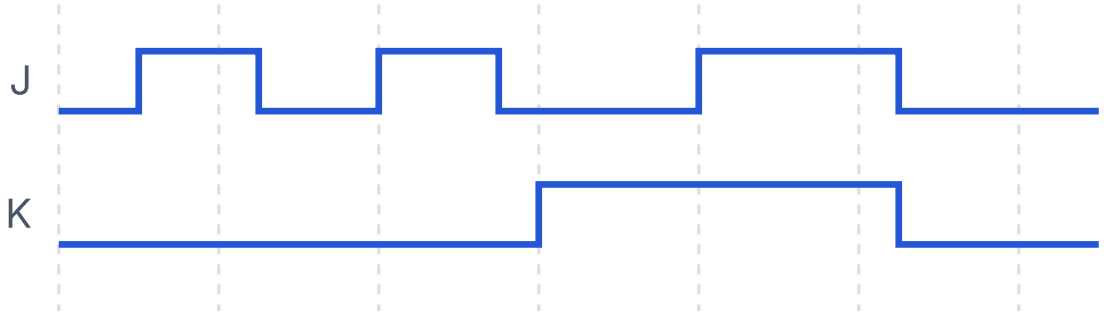
Completa los tres cronogramas de una báscula J-K **asíncrona** (sin reloj: J y K actúan directamente sobre la salida). Señala dónde hay carreras (gris oscuro) y dónde la salida queda desconocida (gris claro).



Cronograma 1: entradas



Cronograma 2: entradas



Cronograma 3: entradas

🧭 Antes de empezar: qué piden y qué teoría usas

Sin reloj no hay nada que la proteja: un J-K asíncrono con $J = K = 1$ realimenta su propia salida y bascula sin parar. Eso es una carrera. Cuando termina, si lo hace en 00, la salida queda parada en un valor desconocido; si termina en 10 o 01, el valor es el de esa orden.

1 Reglas

J	K	Efecto
0	0	mantiene
1	0	Q = 1
0	1	Q = 0
1	1	carrera: Q bascula continuamente

2 Cronograma 1

Nunca coinciden J y K: la evolución es normal. Cada pulso de J pone Q a 1 y cada pulso de K lo pone a 0; los pulsos de J con Q ya a 1 no cambian nada.

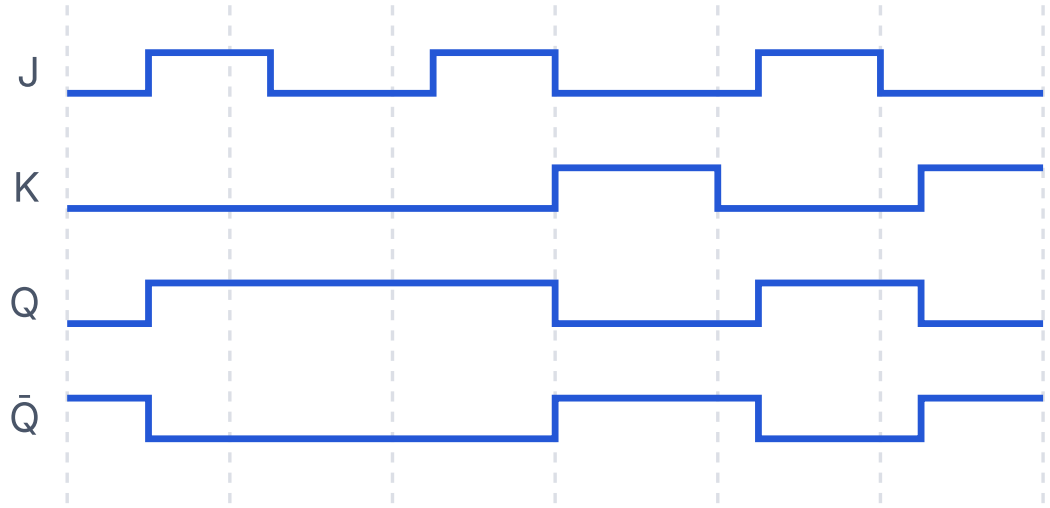
3 Cronograma 2

En la parte final J sube mientras K sigue a 1: $JK = 11$ y la salida entra en carreras. Cuando J baja queda $JK = 01$ y Q se queda en **0**, sin ambigüedad: se sale de la carrera por un solo lado. En el dibujo el basculeo se ve lento y ordenado, pero en realidad se produce a una frecuencia de megahercios, con efectos muy malos sobre el resto del circuito.

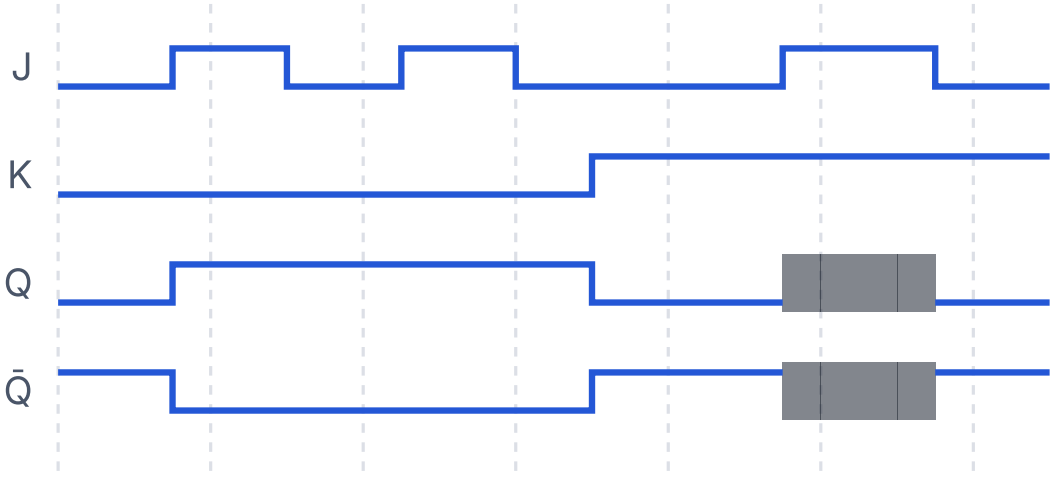
4 Cronograma 3

Se repite la carrera, pero ahora J y K bajan **a la vez** (de 11 a 00). No se sabe en qué momento exacto se paró la oscilación, así que la salida queda parada en un valor desconocido hasta la siguiente orden. Evolución completa: normal, mala (carreras), peor (indeterminación) y, con una orden nueva, otra vez normal.

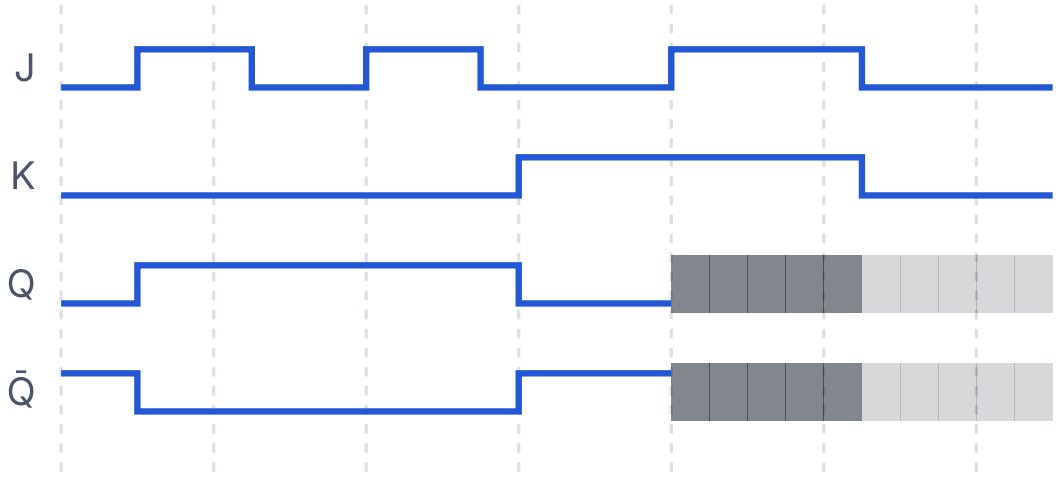
✅ Resultado final



Cronograma 1



Cronograma 2



Cronograma 3

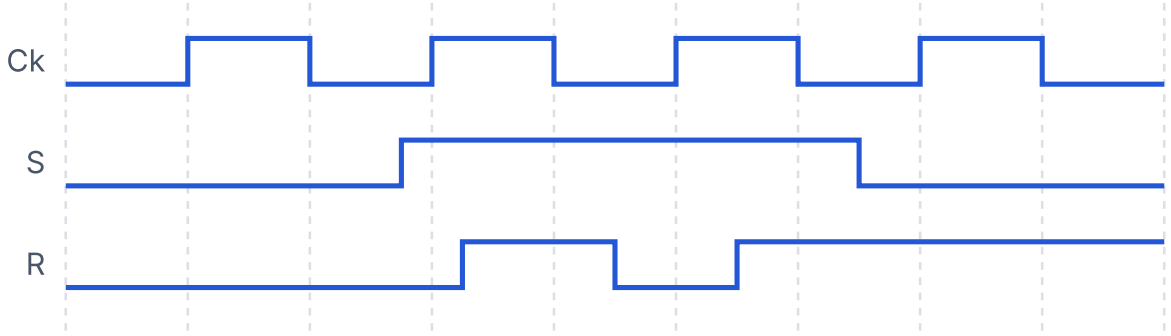
Las dos salidas son siempre complementarias salvo en las bandas grises, donde ninguna de las dos tiene un valor utilizable.

Libro · Cap. 6 · P6.3

R-S síncrona por nivel alto y por nivel bajo

✔ verificado

Completa el cronograma de dos biestables R-S síncronos: Q_{NA}, activo con el reloj a nivel alto, y Q_{NB}, activo con el reloj a nivel bajo.



Entradas

🧭 Antes de empezar: qué piden y qué teoría usas

Un síncrono por nivel es un asíncrono con una puerta que se abre y se cierra. Mientras el reloj está en su nivel activo, S y R actúan como en la báscula asíncrona; fuera de él, la salida se mantiene. Se usa la realización clásica con NAND: con $S = R = 1$ activos las dos salidas valen 1 (prohibido).

1 Nivel alto (Q_{NA})

Primer pulso: $S = 0$, $R = 0$, se mantiene (0). Segundo pulso: S pasa a 1 y R a 1 un poco después: primero Q=1 y luego **prohibido** (trazo grueso). Al bajar el reloj, la puerta se cierra con $S = R = 1$: las dos salidas quedan desconocidas (gris). El tercer pulso empieza con $S = 1$, $R = 0$: Q=1; de nuevo R sube y sale otra prohibida, que al cerrar el reloj deja otra vez la salida desconocida. En el cuarto pulso $S = 0$, $R = 1$ y Q se pone a 0.

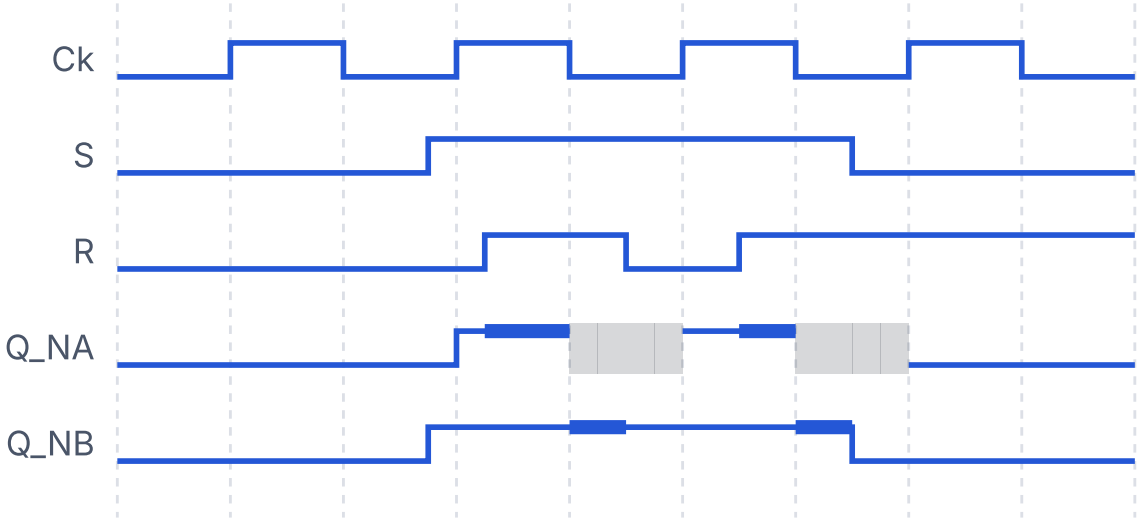
2 Nivel bajo (Q_{NB})

Es lo mismo con el reloj invertido. Q pasa a 1 cuando $S = 1$ y $R = 0$ con el reloj a nivel bajo. Cuando R sube aparece una prohibida breve (las dos salidas a 1) que se deshace por un solo lado (R baja) y Q sigue en 1. Después, con el reloj bajo, vuelve a haber $S = R = 1$, y termina por un solo lado (cae S): Q pasa a 0 sin ambigüedad. Como las dos prohibidas **terminan por un solo lado**, la salida no queda desconocida.

3 Qué enseña

Añadir sincronismo por nivel a un R-S no lo mejora, lo empeora: hay más ocasiones de situaciones prohibidas y de indeterminación a la salida.

✅ Resultado final



R-S por nivel

Trazo grueso: entrada prohibida con la puerta abierta. Gris claro: salida desconocida a priori.

Libro · Cap. 6 · P6.4

R-S síncrona por flanco ascendente y por flanco descendente

✔ verificado

Con las mismas entradas que el problema anterior, completa el cronograma de dos biestables R-S síncronos por flanco: $Q_{\uparrow}$ (flanco de subida) y $Q_{\downarrow}$ (flanco de bajada).

🧭 Antes de empezar: qué piden y qué teoría usas

En un flanco solo importa una cosa: el valor de S y R justo antes de él. Si es 00, mantiene; 10 pone a 1; 01 pone a 0; 11 deja la salida **desconocida** hasta el siguiente flanco que dé una orden clara.

1 Flanco de subida

Los cuatro flancos de subida encuentran $SR = 00$ (mantiene), 10 (pone a 1), 10 (sigue a 1) y 01 (pone a 0). Ninguno coincide con 11, así que la evolución es normal.

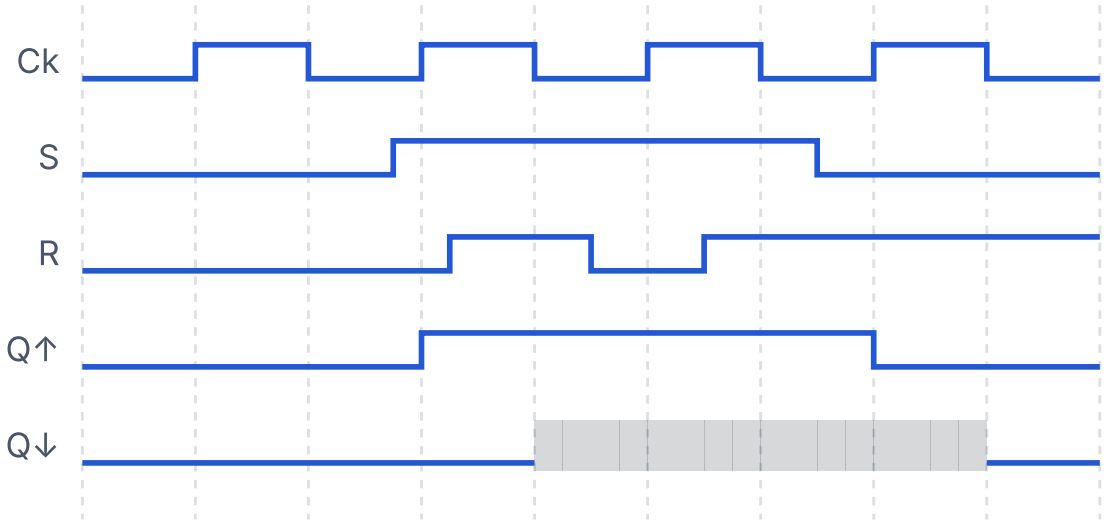
2 Flanco de bajada

Los cuatro flancos de bajada encuentran $SR = 00$ (mantiene), 11, 11 y 01. Con el primer 11 la salida queda desconocida y con el segundo sigue desconocida; solo se aclara con el último flanco, que trae $S = 0$, $R = 1$ y pone Q a 0.

3 Qué enseña

Al añadir el sincronismo por flanco a un R-S, toda situación prohibida genera directamente una salida indeterminada. El sincronismo ha **degradado** al R-S.

✅ Resultado final



R-S por flanco

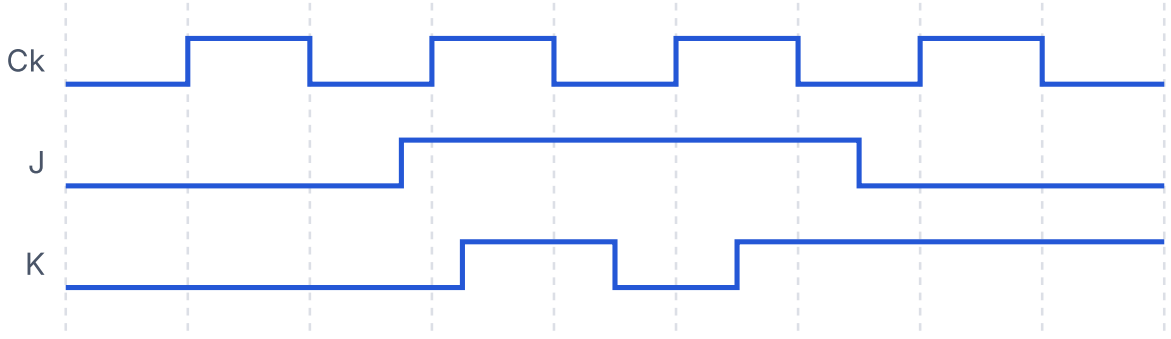
Gris claro: salida desconocida a priori. Los flancos de bajada de este ejemplo son justo los que caen sobre $SR = 11$.

Libro · Cap. 6 · P6.5

J-K síncrona por nivel alto y por nivel bajo

✔ verificado

Con las mismas entradas (J en lugar de S y K en lugar de R), completa el cronograma de dos J-K síncronos por nivel: Q_{NA} (nivel alto) y Q_{NB} (nivel bajo).



Entradas

🧭 Antes de empezar: qué piden y qué teoría usas

$JK = 11$ **ya no es «prohibida» sino «bascula».** Pero si el nivel activo dura más que el retardo de las puertas, la salida cambia más de una vez: carrera. Al terminar el nivel, la salida queda parada en un valor imposible de predecir.

1 Nivel alto (Q_{NA})

Con $J = 1$, $K = 0$ al principio del segundo pulso, Q pasa a 1; cuando K sube se da $JK = 11$ con el reloj alto: **carrera** durante el resto del nivel; al bajar el reloj, la salida queda desconocida. En el siguiente pulso, $J = 1$, $K = 0$ pone Q a 1 de forma determinista y poco después K sube otra vez: otra carrera y otra indeterminación. Al final, con $J = 0$, $K = 1$, Q a 0.

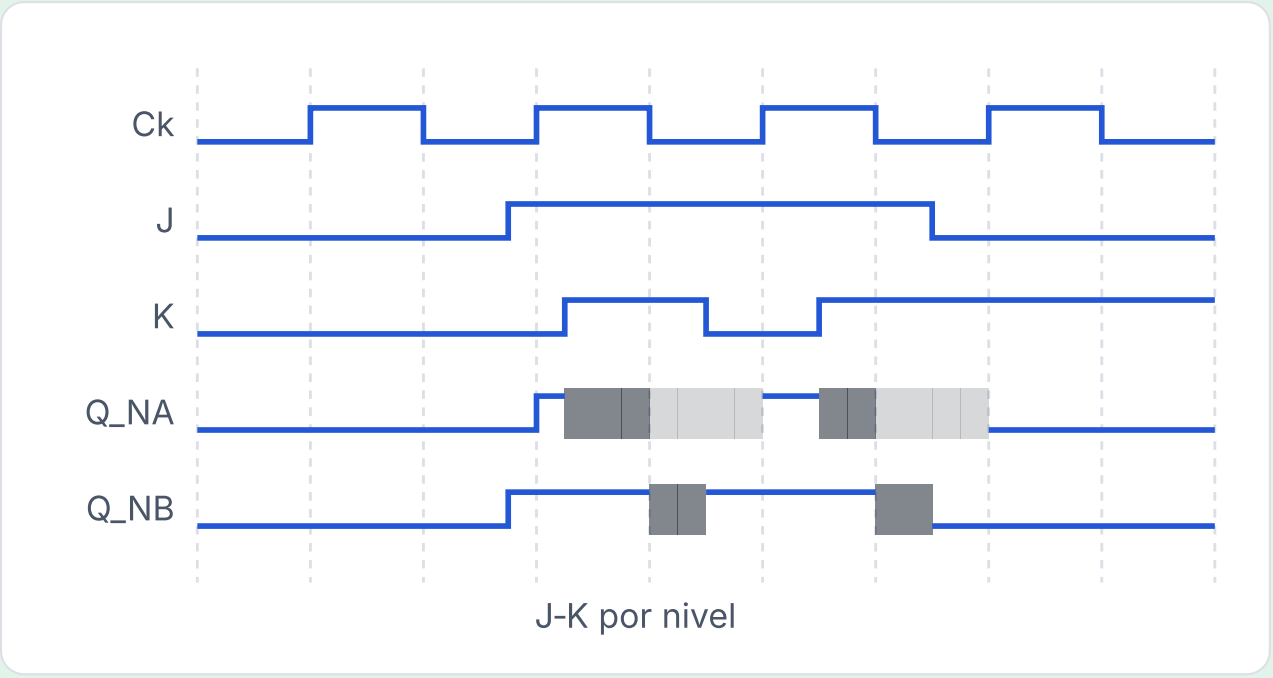
2 Nivel bajo (Q_{NB})

La primera vez que coinciden $J = K = 1$ con el reloj bajo (carrera breve) se sale con $J = 1$, $K = 0$ y Q queda a 1. La segunda vez termina con $J = 0$, $K = 1$ y Q se pone a 0: las dos carreras se resuelven por un solo lado.

3 Qué enseña

Igual que con el R-S, el sincronismo por nivel degrada al J-K: aparece la indeterminación con mucha más facilidad.

✅ Resultado final



J-K por nivel

Gris oscuro: carreras. Gris claro: salida desconocida a priori.

Libro · Cap. 6 · P6.6

J-K síncrona por flanco ascendente y descendente

✔ verificado

Completa el cronograma de dos J-K síncronos por flanco: $Q_{\uparrow}$ (flanco de subida) y $Q_{\downarrow}$ (flanco de bajada), con las mismas entradas que el problema anterior.

🧭 Antes de empezar: qué piden y qué teoría usas

Un flanco es un instante, y un instante no puede tener carrera. En cada flanco se mira JK y se aplica la tabla (00 mantiene, 10 pone a 1, 01 pone a 0, 11 invierte). Entre flancos, la salida no se entera de nada.

1 Flanco de subida

Los cuatro flancos de subida encuentran $JK = 00$, 10, 10 y 01: nada, Q a 1, Q sigue a 1 y Q a 0. La salida sube en el primer flanco con $J = 1$ y baja en el último.

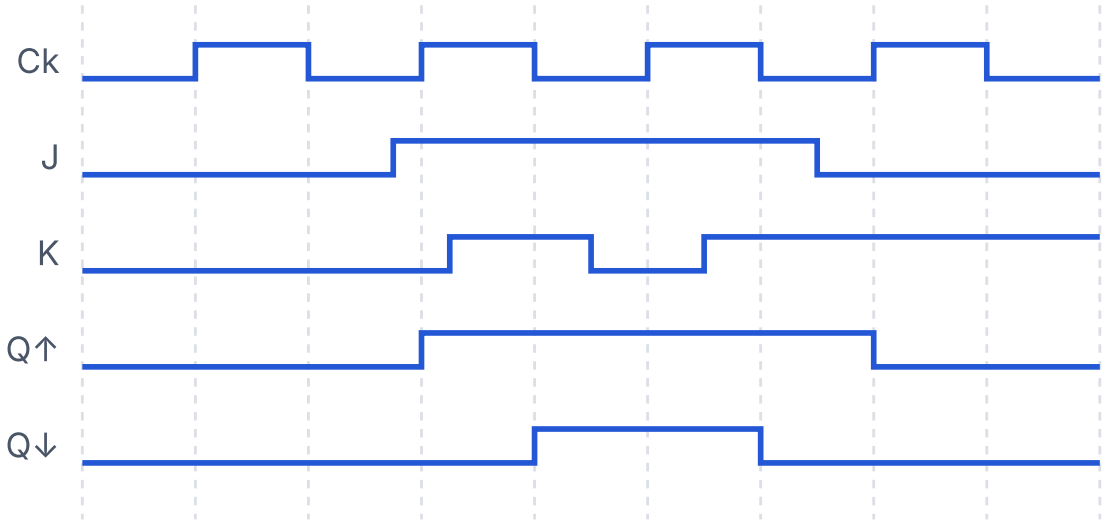
2 Flanco de bajada

Aquí llegan $JK = 00$, 11, 11 y 01: nada, **inversión** (Q pasa a 1), **inversión** (Q vuelve a 0, de ahí el pulso corto) y 0. Es la única vez en el capítulo en que el 11 hace algo con sentido.

3 Qué enseña

El sincronismo por flanco le ha sentado bien al J-K: **no hay carreras ni indeterminaciones**, solo una inversión por flanco con $JK = 11$. Es el J-K que se usa en la práctica; el R-S y el J-K por nivel y el R-S por flanco solo tienen interés teórico.

✅ Resultado final



J-K por flanco

> **⚠️ Errata en el libro**
>
> El primer comentario del libro afirma que «el J-K síncrono por nivel no presenta situaciones ni de carreras ni indeterminadas». Es al revés: el J-K por nivel del problema anterior sí las presenta; lo que no las tiene es el J-K **por flanco**. Es una errata de palabra.

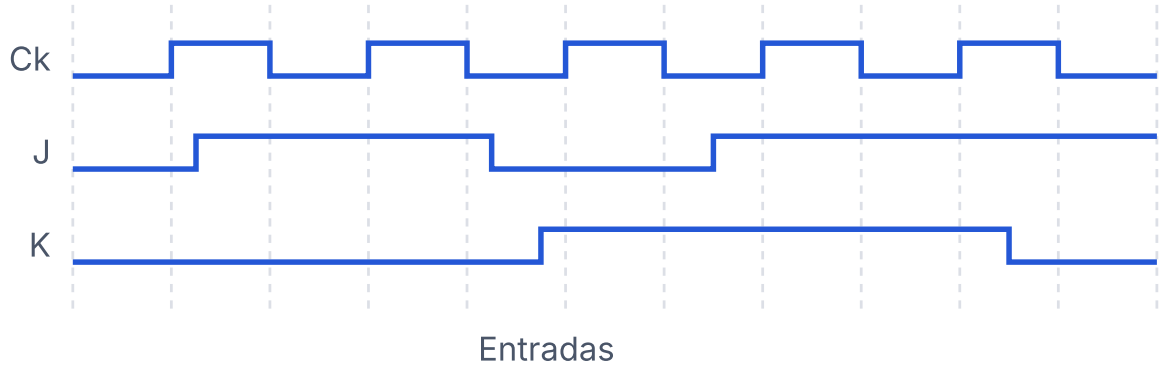
Libro · Cap. 6 · P6.7

Maestro/esclavo J-K en sus dos versiones

✔ verificado

Completa el cronograma de dos biestables J-K maestro/esclavo. En el primero (versión A) el maestro es activo con el reloj a nivel alto y el esclavo a nivel bajo; en el segundo (versión B), al revés.

El maestro es un R-S síncrono por nivel cuyas entradas son $S_M = J \cdot \overline{Q_S}$ y $R_M = K \cdot Q_S$; el esclavo es otro R-S síncrono por nivel, con el nivel contrario, cuyas entradas son $S_S = Q_M$ y $R_S = \overline{Q_M}$. La salida del conjunto es Q_S.



Entradas

🧭 Antes de empezar: qué piden y qué teoría usas

El maestro piensa y el esclavo copia. En cada semiperíodo solo uno de los dos está abierto: mientras el maestro mira a J y K, el esclavo duerme, y al revés. Las dos tareas se separan en el tiempo, y por eso no hay carreras ni indeterminaciones.

1 Qué hace el maestro

Con $JK = 00$ mantiene Q_M. Con $JK = 11$ **toma el opuesto del esclavo**: $Q_M = \overline{Q_S}$ (por eso se realimenta Q_S). Con 10 y 01 se pone a 1 o a 0 si el esclavo no lo tiene ya así.

2 Versión A

El maestro mira durante el nivel alto; el esclavo copia durante el nivel bajo siguiente. Q_M sube en el primer pulso (con $J = 1$), el esclavo lo copia al bajar el reloj, Q_M baja con $K = 1$, el esclavo lo copia, etc. La salida Q_S cambia en los flancos de bajada.

3 Versión B

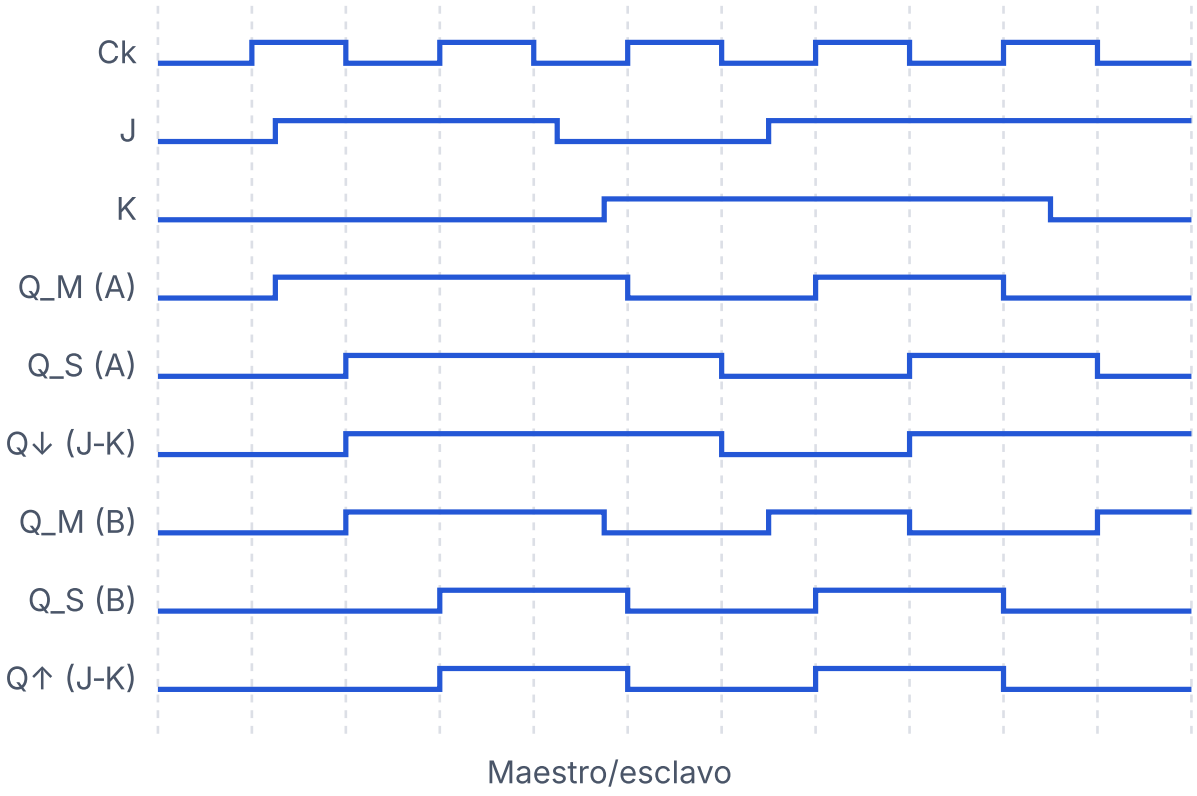
Igual con los niveles intercambiados. La salida Q_S cambia en los flancos de subida y es **idéntica** a la de un J-K por flanco ascendente. En el dibujo se ha superpuesto $Q_{\uparrow}$ para

comprobarlo.

4 El caso difícil

En el último pulso alto de la versión A cambian J y K **durante el nivel activo del maestro**: primero $JK = 11$ (el maestro toma el opuesto de $Q_S = 1$, es decir, 0) y luego $JK = 10$. Con $JK = 10$: $S_M = 1 \cdot \overline{Q_S} = 0$ y $R_M = 0 \cdot Q_S = 0$, el maestro **mantiene su 0**, y por eso Q_S copia 0, no 1. Un J-K por flanco de bajada habría visto $JK = 10$ en el flanco y habría puesto 1: ahí **no coinciden**. La equivalencia M/S ≡ J-K por flanco solo es obligada si J y K no cambian durante el nivel activo del maestro.

✅ Resultado final



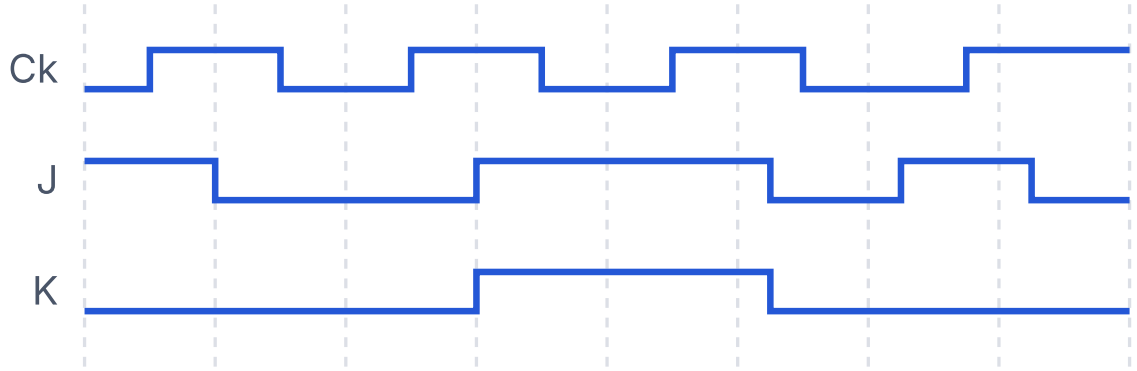
La salida del conjunto es la del esclavo. En la versión A, Q_S y $Q_{\downarrow}$ solo difieren en el último tramo (el caso difícil); en la versión B coinciden siempre.

Libro · Cap. 6 · P6.8

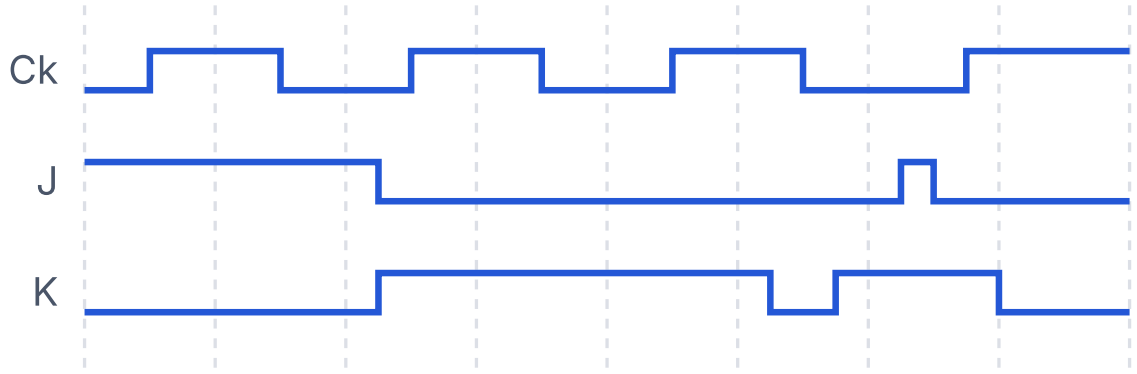
Dos maestros/esclavos J-K y sus equivalentes por flanco

✔ verificado

Completa los dos cronogramas con cuidado. En el primero el maestro es activo con el reloj a nivel bajo y el esclavo a nivel alto; en el segundo, al revés. Añade en cada uno la salida del J-K síncrono por flanco que corresponda.



Cronograma 1: entradas



Cronograma 2: entradas

🧭 Antes de empezar: qué piden y qué teoría usas

Hay que ir paso a paso porque el maestro depende de la salida del esclavo y el esclavo del maestro: $S_M = J\overline{Q_S}$, $R_M = K\,Q_S$. No se resuelve primero todo el maestro y luego todo el esclavo, sino a la vez, semiperíodo a semiperíodo.

1 Cronograma 1

El maestro (nivel bajo) ve a $J = 1$ al principio y pone $Q_M = 1$; el esclavo lo copia en el primer nivel alto. Con $JK = 11$ en el nivel bajo siguiente, el maestro toma el opuesto del esclavo (0), y el esclavo lo copia después. Luego queda a 0 hasta que J sube con $K = 0$. La salida Q_S cambia **en los flancos de subida**.

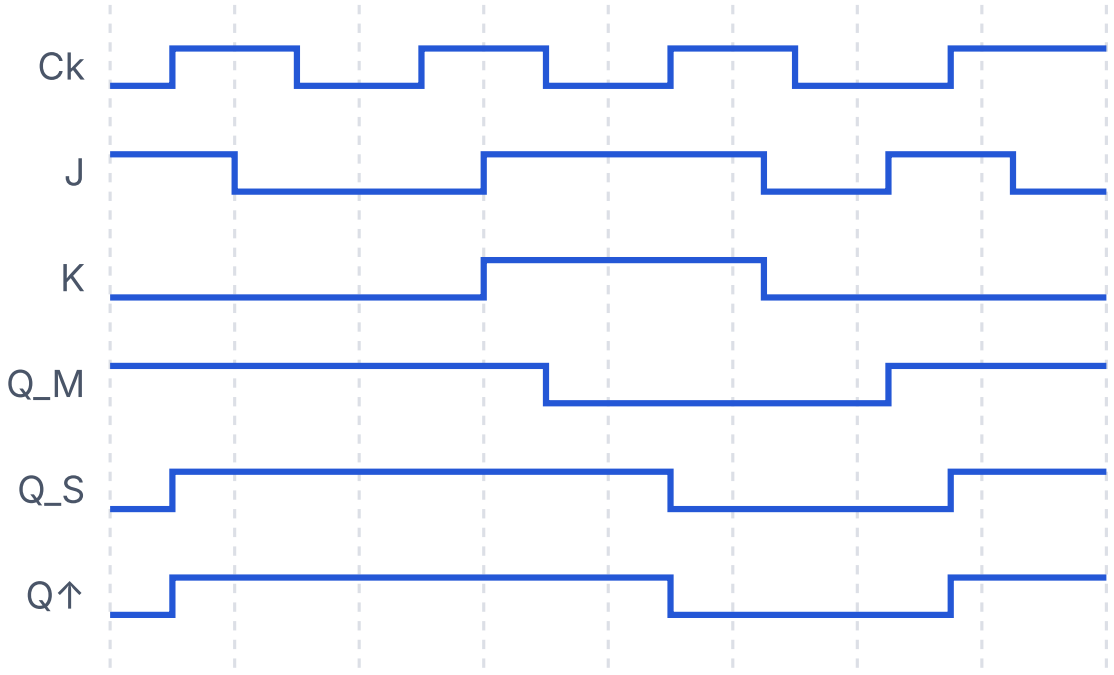
2 Cronograma 2

Maestro de nivel alto. Pone $Q_M = 1$ mientras J vale 1 y K 0, y se pone a 0 cuando K sube. El esclavo copia en el nivel bajo siguiente; los pequeños pulsos de J y K que caen con el maestro cerrado no tienen ningún efecto. Q_S cambia **en los flancos de bajada**.

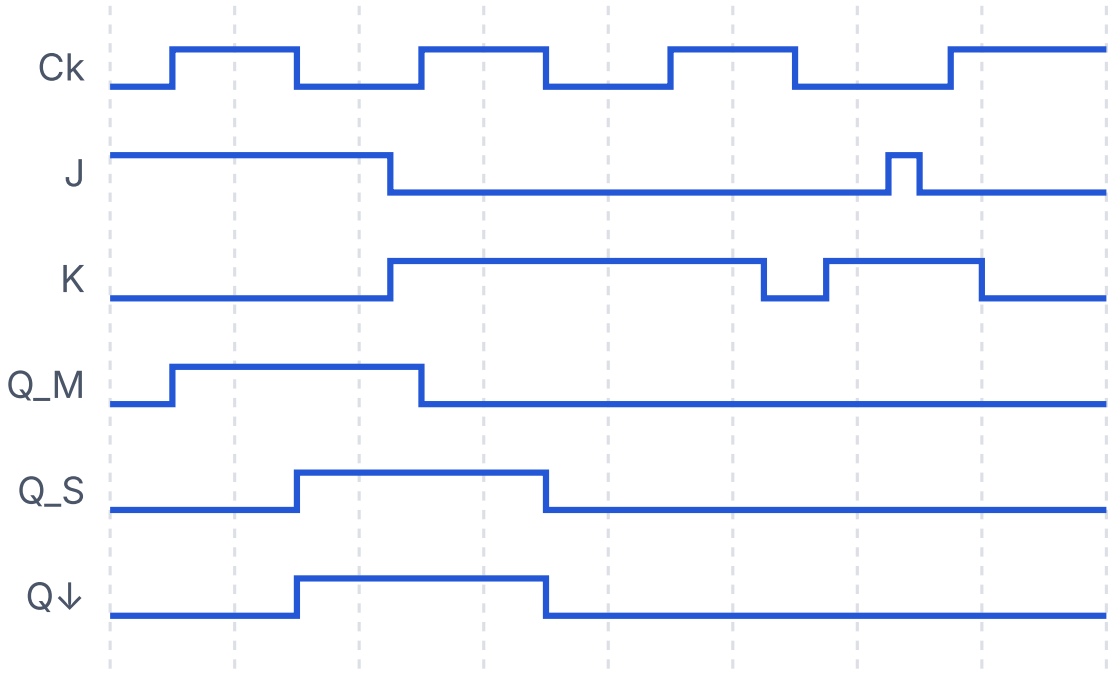
3 Equivalencias

Un maestro/esclavo con maestro de nivel bajo equivale a un J-K por flanco ascendente, y con maestro de nivel alto a un J-K por flanco descendente: en los dos cronogramas la salida del esclavo y la del J-K por flanco coinciden. Pero eso solo vale mientras J y K no cambien durante el nivel activo del maestro.

✅ Resultado final



Cronograma 1: maestro bajo, esclavo alto



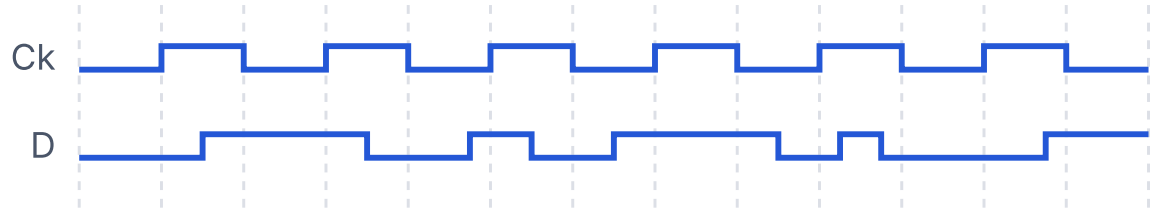
Cronograma 2: maestro alto, esclavo bajo

Libro · Cap. 6 · P6.9

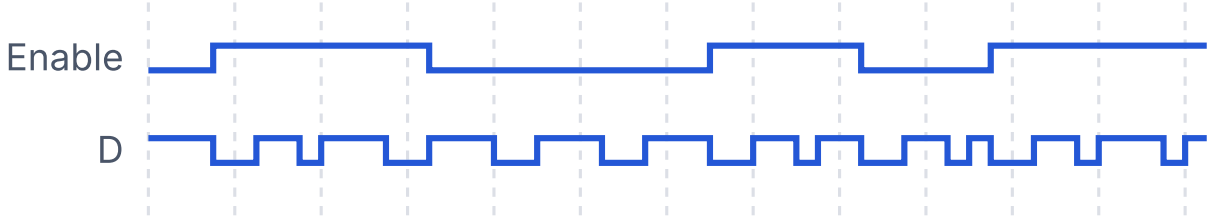
Báscula D por flanco y latch D por nivel

✔ verificado

Completa los cronogramas de una báscula D síncrona por flanco (de subida, $Q_{\uparrow}$, y de bajada, $Q_{\downarrow}$) y de una báscula D síncrona por nivel (un latch o cerrojo, Q_{NA}) cuya señal de reloj es una línea ENABLE aperiódica.



Por flanco: entradas



Latch: entradas

🧭 Antes de empezar: qué piden y qué teoría usas

Una báscula D no puede tener situaciones prohibidas ni indeterminadas sea cual sea el sincronismo: solo tiene una entrada. Por flanco copia a D en ese instante; por nivel copia a D mientras esté abierta y lo guarda cuando se cierra.

1 Por flanco

Se anota el valor de D justo antes de cada flanco: en $Q_{\uparrow}$ sale 0, 1, 1, 1, **0** (D baja justo antes del quinto flanco) y 0; en $Q_{\downarrow}$ sale 1, 0, 0, 1, 0, 1. Los cambios de D entre flancos no se ven.

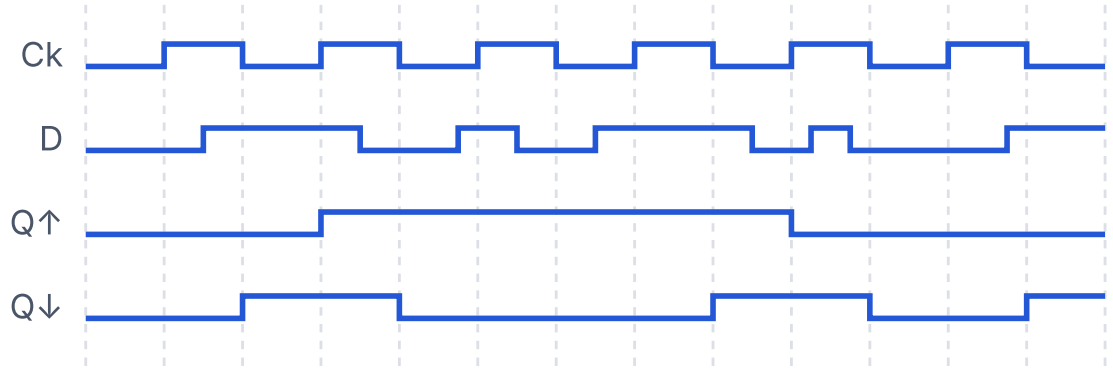
2 Latch

Mientras ENABLE vale 1 la salida **sigue** a D, con todos sus pulsos cortos; cuando ENABLE baja, la salida se queda con el último valor de D. Se parece a la anterior solo en el nombre: aquí cada glitch de D con ENABLE alto llega a la salida.

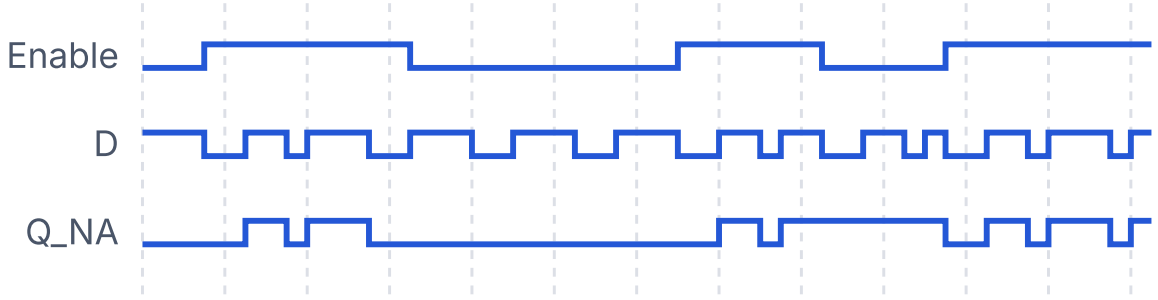
3 Comprobación

El latch D se simuló también como un R-S con puertas NAND con $S = D$, $R = \overline{D}$ y una puerta de habilitación: da la misma salida. Y no hay indeterminaciones, porque S y R nunca valen 1 a la vez.

✅ Resultado final



D por flanco



Latch D (ENABLE aperiódica)

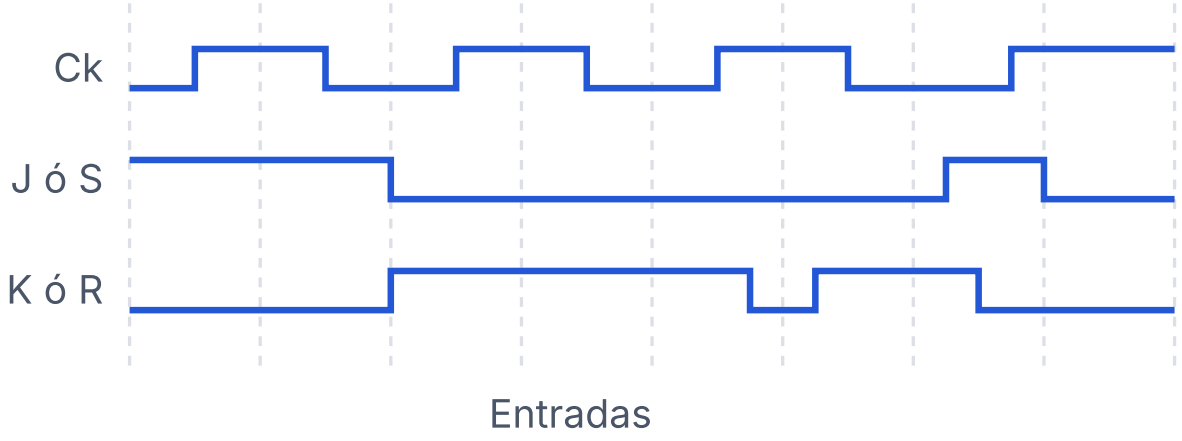
Una línea ENABLE aperiódica y no un reloj: por eso se habla de latch o cerrojo, y no de biestable.

Libro · Cap. 6 · P6.10

Un mismo cronograma para J-K y para R-S de todos los sincronismos

✔ verificado

Completa el cronograma dado. Las dos entradas se aplican a un J-K (J, K) y a un R-S (S=J, R=K). Para cada uno hay que dibujar la salida de las versiones por nivel alto (Q_{NA}), por nivel bajo (Q_{NB}), por flanco de subida ($Q_{\uparrow}$) y por flanco de bajada ($Q_{\downarrow}$); en el R-S, además, las de maestro/esclavo con maestro a nivel alto ($Q_{MS(NA)}$) y bajo ($Q_{MS(NB)}$).



🧭 Antes de empezar: qué piden y qué teoría usas

Es el mismo método ocho veces. Para cada dispositivo se aplica la regla que corresponde: por nivel, lo que valgan las entradas mientras esté abierto; por flanco, lo que valgan justo antes del flanco; maestro/esclavo, el nivel activo del maestro y luego la copia del esclavo.

1 J-K

Solo coinciden $J = K = 1$ una vez, con el reloj a nivel bajo: el J-K de nivel bajo entra en carrera una ranura (de ahí la banda oscura) y, como J se queda en 1 después, se resuelve poniéndose a 1. Los demás dispositivos no ven nunca $JK = 11$.

2 R-S

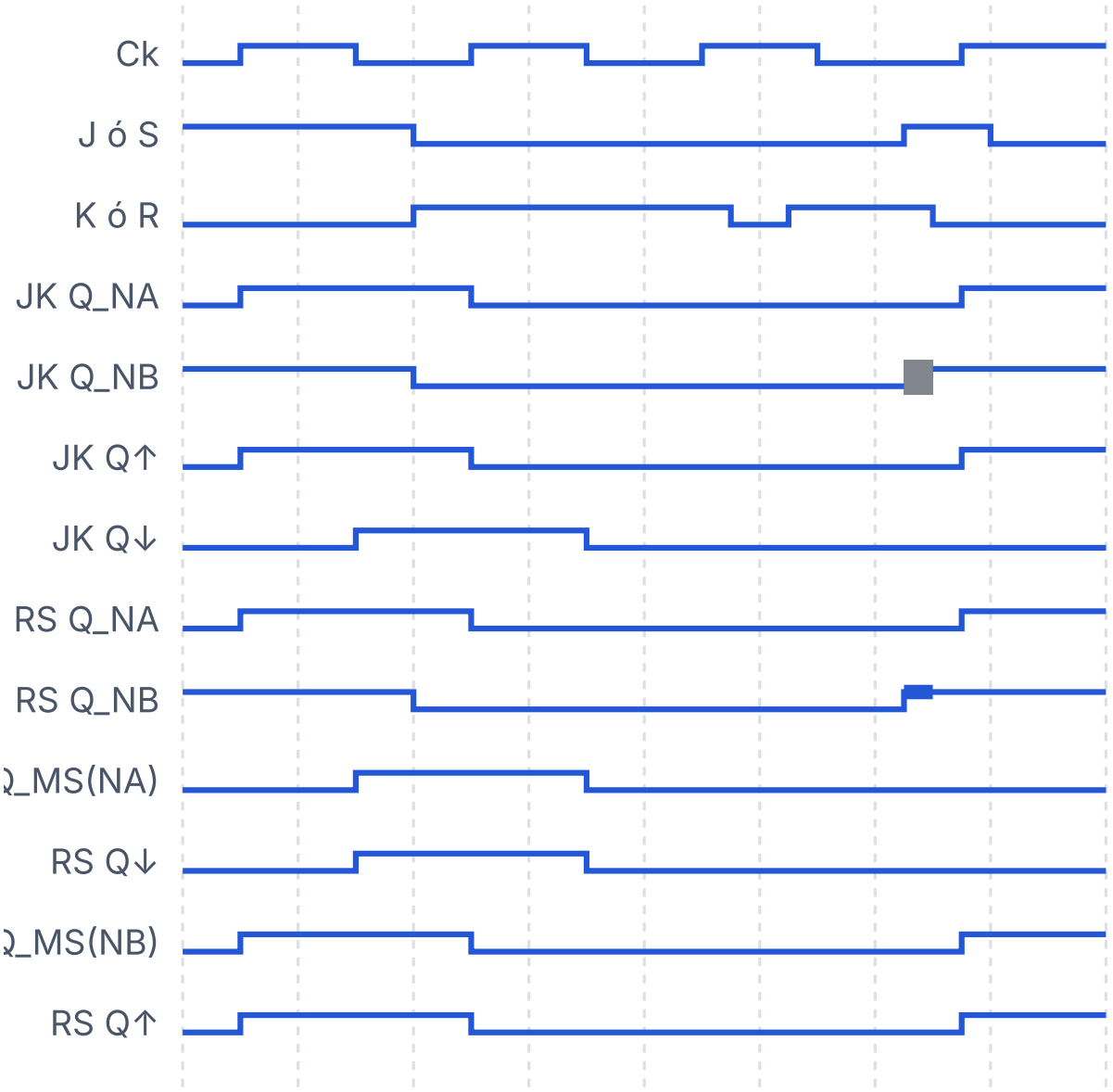
La misma situación $S = R = 1$ con el R-S de nivel bajo es una combinación prohibida (las dos salidas valen 1) y también se sale por un solo lado: Q queda a 1 sin ambigüedad, igual que en el J-K. El resto de dispositivos coincide con sus equivalentes de J-K.

3 Equivalencias

La salida del R-S maestro/esclavo con maestro a nivel alto coincide con la del R-S por flanco de bajada ($Q_{\downarrow}$), y con maestro a nivel bajo con la de flanco de subida ($Q_{\uparrow}$). Las versiones por nivel

alto de J-K y R-S coinciden aquí con $Q_{\uparrow}$ de J-K porque en los niveles altos no se dan ni $JK = 11$ ni $SR = 11$.

✅ **Resultado final**



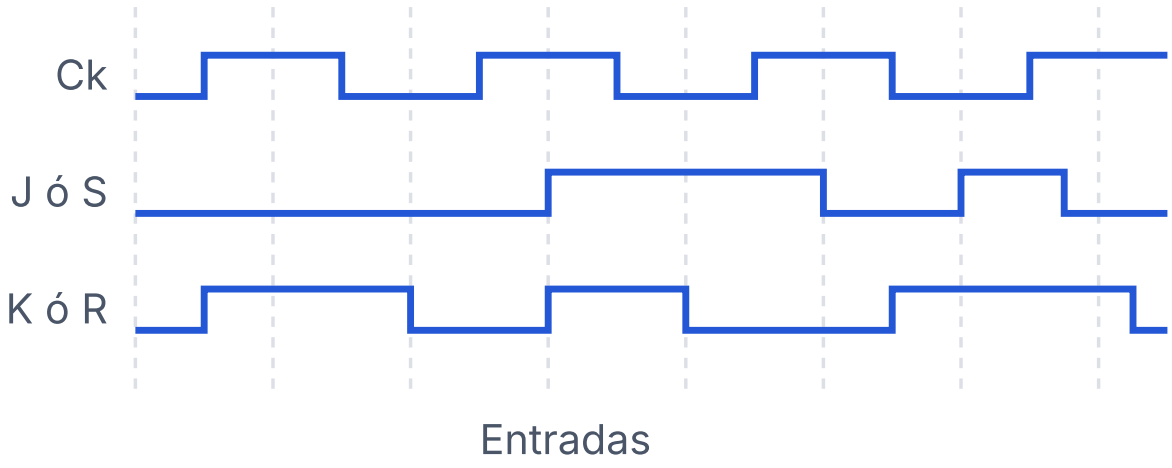
Todos los sincronismos

Gris oscuro: carreras. Trazo grueso: entrada prohibida. Los R-S maestro/esclavo y por flanco dan la misma salida cuando las entradas no cambian durante el nivel activo del maestro.

`Libro · Cap. 6 · P6.11` **R-S y J-K con prohibidas, carreras e indeterminaciones**

✔ verificado

Completa el cronograma para biestables R-S (S=J, R=K) y J-K. Señala con trazo grueso lo prohibido, con gris oscuro las carreras y con gris claro lo desconocido. Hay que dibujar las salidas de nivel alto (Q_{NA}), nivel bajo (Q_{NB}), flanco de subida ($Q_{\uparrow}$) y, para el R-S, flanco de bajada ($Q_{\downarrow}$).



🧭 Antes de empezar: qué piden y qué teoría usas

Es el cronograma más completo del capítulo: en él aparecen las prohibidas del R-S, las carreras del J-K por nivel y las indeterminaciones de los dos. Se resuelve igual que los anteriores, fila a fila y con cuidado, porque un fallo se arrastra a lo largo de todo el cronograma.

1 R-S por nivel alto

En el segundo pulso S y R suben a la vez con el reloj alto: prohibida (las dos salidas a 1). Al cerrarse el reloj con $SR = 11$ la salida queda **desconocida**. El tercer pulso trae $S = 1$, $R = 0$ y Q=1. En el cuarto vuelve a darse $SR = 11$ (prohibida) y a continuación $S = 0$, $R = 1$: Q pasa a 0.

2 R-S por nivel bajo y por flanco

En nivel bajo hay dos prohibidas: la primera se resuelve por un solo lado (R baja y Q sigue en 1); luego, con $S = 0$, $R = 1$, Q pasa a 0, y viene otra prohibida que, al subir el reloj (la puerta se cierra con $SR = 11$), deja la salida desconocida. Por flanco de subida el R-S queda desconocido solo en el último flanco (llega con $SR = 11$); por flanco de bajada, el primer flanco pone Q a 0, el segundo llega con $SR = 11$ y la salida ya no se aclara.

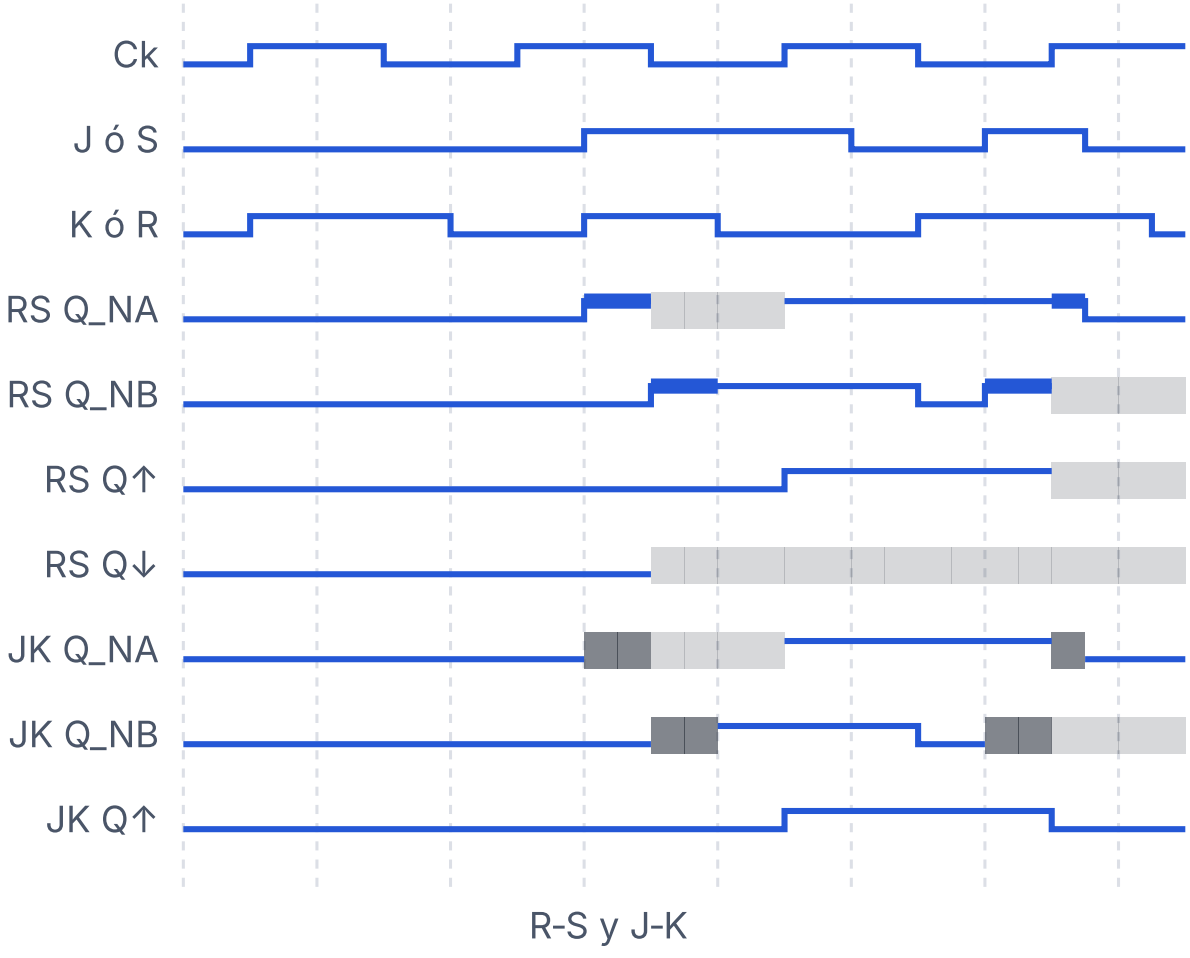
3 J-K

Por nivel alto: carrera (J y K suben a la vez), desconocida (el reloj baja), set, carrera breve y reset. Por nivel bajo: carrera, set, reset, carrera y desconocida. Por flanco de subida el J-K no tiene ningún problema: el flanco que llega con $JK = 11$ solo invierte la salida.

4 Qué enseña

Con el mismo patrón de entradas, el J-K por flanco es un dispositivo limpio y todos los demás tienen alguna situación indefinida. El sincronismo por flanco ha degradado al R-S, pero ha sentado bien al J-K.

✅ Resultado final



Trazo grueso: prohibido. Gris oscuro: carreras. Gris claro: valor desconocido a priori.

> **⚠️ Errata en el libro**
>
> En el dibujo del libro, la salida del J-K de nivel alto (Q_{NA}) queda a 1 después de la última carrera, pero ahí J ya vale 0 y K vale 1 con el reloj aún en nivel alto: la salida debe bajar a 0, como en el R-S. Es un fallo de dibujo.

`Libro · Cap. 6 · P6.12` **Tres circuitos con puertas: ¿qué biestable es?** ✔ verificado

Completa las tablas de verdad de tres circuitos con puertas y realimentación de la salida. Indica a qué biestable corresponde cada uno y qué señales son A, B y C.

- **Circuito 1:** $Q = A + \overline{B} \cdot Q$ (una AND de $\overline{B}$ con la propia salida, y una OR con A).
- **Circuito 2:** $Q = A \cdot B + \overline{A} \cdot Q$ (una AND de A y B, otra de $\overline{A}$ con la propia salida, y una OR).
- **Circuito 3:** $Q = A \cdot B + \overline{A} \cdot Q + \overline{C} \cdot Q$ (tres AND y una OR de tres entradas).

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Un circuito con realimentación se analiza poniendo una entrada, siguiendo su efecto y anotando la salida;** si hace falta, se supone un valor inicial para Q_{t-1}. Si el resultado depende de él, la operación es de mantenimiento.

1 Circuito 1

A	B	Q(t)	Operación
0	0	Q_{t-1}	mantiene
0	1	0	puesta a 0
1	0	1	puesta a 1
1	1	1	prohibida

R-S asíncrono: $A = S$, $B = R$. Con $A = B = 1$, la OR da 1: $Q = 1$ (prohibida).

2 Circuito 2

A	B	Q(t)	Operación
0	0	Q_{t-1}	mantiene

A	B	Q(t)	Operación
0	1	Q_{t-1}	mantiene
1	0	0	puesta a 0
1	1	1	puesta a 1

D síncrono por nivel (latch D): $A = Ck$ (habilita) y $B = D$. Con $A = 0$ mantiene y con $A = 1$ copia B.

3 Circuito 3

A	B	C	Q(t)	Operación
0	0	0	Q_{t-1}	mantiene
0	0	1	Q_{t-1}	mantiene
0	1	0	Q_{t-1}	mantiene
0	1	1	Q_{t-1}	mantiene
1	0	0	Q_{t-1}	mantiene
1	0	1	0	puesta a 0
1	1	0	1	puesta a 1
1	1	1	1	prohibida

R-S síncrono por nivel: $A = Ck$, $B = S$, $C = R$. La fila 111 es la prohibida (queda $Q = 1$).

4 Comprobación

Las tres tablas se han calculado resolviendo la ecuación de cada circuito para $Q_{t-1} = 0$ y para $Q_{t-1} = 1$: si salen valores distintos la fila es «mantiene».

✅ Resultado final

Circuito 1: R-S asíncrono. Circuito 2: latch D. Circuito 3: R-S síncrono por nivel.

> **⚠️ Errata en el libro**
>
> El libro dice que el circuito 3 es «un biestable R-S o J-K síncrono por nivel». Pero su tabla tiene una fila prohibida (111, que da $Q = 1$), y un J-K no tiene ninguna combinación prohibida (con $J = K = 1$ invertiría la salida). Es un R-S; la mención al J-K sobra.

`Libro · Cap. 6 · P6.13`

Un T y un D con realimentación: ¿qué biestable resulta?

✔ verificado

Completa las tablas de verdad de dos circuitos formados por un flip-flop por flanco de subida, dos AND, una OR y la realimentación de sus salidas. Indica qué biestable resulta y qué señales son A y B.

- **Circuito 1:** un T con $T = A \cdot \overline{Q} + B \cdot Q$.
- **Circuito 2:** un D con $D = A \cdot \overline{Q} + \overline{B} \cdot Q$.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Basta con excitar el circuito con una entrada y anotar la salida.** Para cada fila (A, B, Q_{t-1}) se calcula la entrada del flip-flop y, con ella y con la tabla del flip-flop, la salida tras el flanco.

1 Circuito 1

A	B	Q_{t-1}	T	Q(t)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	1	0
1	0	0	1	1
1	0	1	0	1
1	1	0	1	1
1	1	1	1	0

T vale 1 cuando $A = 1$ y $Q = 0$ (hay que subir), o cuando $B = 1$ y $Q = 1$ (hay que bajar). Con $T = 1$ invierte y con $T = 0$ mantiene.

2 Circuito 2

A	B	Q_{t-1}	D	Q(t)
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

D vale 1 si hay que subir ($A = 1, Q = 0$) o si hay que mantener a 1 ($B = 0, Q = 1$).

3 Qué biestable es

Los dos son un **J-K por flanco de subida**, con $A = J$ y $B = K$: $Q_t = J\overline{Q_{t-1}} + \overline{K}\,Q_{t-1}$ en los dos. Un T o un D con dos puertas y realimentación forma un J-K. Estos circuitos son clásicos en los libros de teoría.

4 Fuera del flanco

Con el reloj en nivel alto o bajo (sin flanco de subida) la salida **mantiene** su valor: en las dos tablas del libro la última fila es «todo X, $Q_t = Q_{t-1}$».

✅ Resultado final

Circuitos 1 y 2: J-K por flanco de subida, con $A = J$ y $B = K$.

> ⚠️ **Errata en el libro**
>
> En la tabla del circuito 2, la fila con $A = 0$, $B = 0$, $Q_{t-1} = 1$ lleva $D = 0$ pero $Q(t) = 1$, lo que es incoherente: con $B = 0$ y $Q = 1$ la puerta da $\overline{B} \cdot Q = 1$, luego $D = 1$.

`Libro · Cap. 6 · P6.14`

Tabla de verdad de un D por flanco hecho con seis NAND

✔ verificado

Completa la tabla de verdad de un circuito con seis puertas NAND (1 a 6) que forma un biestable. Se anotan los valores internos Q_1 (salida de la 1), Q_4 (salida de la 4), $\overline{S}$ (salida de la 2) y $\overline{R}$ (salida de la 3), y las salidas Q y $\overline{Q}$ (puertas 5 y 6).

- Puertas 1 y 2 forman un primer latch R-S de NAND: $Q_1 = \overline{\overline{S} \cdot Q_4}$, $\overline{S} = \overline{Ck \cdot Q_1}$.
- Puerta 3: $\overline{R} = \overline{Ck \cdot \overline{S} \cdot Q_4}$. Puerta 4: $Q_4 = \overline{\overline{R} \cdot D}$.
- Puertas 5 y 6 forman el latch de salida: $Q = \overline{\overline{S} \cdot \overline{Q}}$, $\overline{Q} = \overline{\overline{R} \cdot Q}$.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> **El circuito es complicado de analizar: solo vale la paciencia y escribir sobre un dibujo.** Hay que empezar por la primera fila y seguir los cambios de entrada junto con sus efectos en la salida. Se puede ver como **tres latches R-S asíncronos**: (1,2), (3,4) y (5,6).

1 Con Ck = 0

Las puertas 2 y 3 valen 1 ($\overline{S} = \overline{R} = 1$): el latch de salida mantiene. Q4 = $\overline{D}$ y $Q_1 = D$ por la puerta 1: el circuito **sigue a D por dentro** pero sin tocar la salida.

2 En el flanco de subida

Con $Ck = 1$, si $D = 1$ ($Q_4 = 0$, $Q_1 = 1$): $\overline{S} = 0$ y $\overline{R} = 1$: se pone Q a 1. Si $D = 0$ ($Q_4 = 1$, $Q_1 = 0$): $\overline{S} = 1$ y $\overline{R} = 0$: Q a 0. En los dos casos, una de las dos puertas 2 o 3 se activa.

3 Con Ck = 1 después del flanco

Si se capturó un 1, $\overline{S} = 0$ mantiene $Q_1 = 1$ y **bloquea** a la puerta 3 ($\overline{R}$ no puede bajar aunque Q_4 cambie). Si se capturó un 0, $\overline{R} = 0$ fija $Q_4 = 1$ sea cual sea D. En los dos casos los cambios de D ya no llegan al latch de salida. Por eso el último comportamiento de la tabla es el interesante: D pasa a 0 pero la salida sigue en 1.

4 Tabla

Ck	D	Q_{t-1}	Q1	Q4	$\overline{S}$	$\overline{R}$	Q(t)
0	1	0	1	0	1	1	0
↑	1		1	0	0	1	1
1	1		1	0	0	1	1
0	1	1	1	0	1	1	1
↑	1		1	0	0	1	1
1	1		1	0	0	1	1
0	0	0	0	1	1	1	0
↑	0		0	1	1	0	0
1	0		0	1	1	0	0
0	0	1	0	1	1	1	1
↑	0		0	1	1	0	0
1	0		0	1	1	0	0
0	1	0	1	0	1	1	0
↑	1		1	0	0	1	1
1	0		1	1	0	1	1

✅ Resultado final

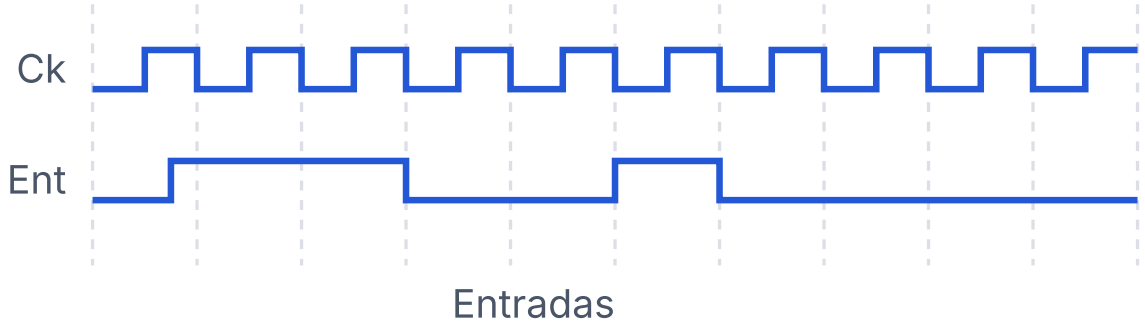
El circuito es un **biestable D síncrono por flanco de subida**: solo captura el valor de D en el flanco; los cambios posteriores de D no llegan a la salida. Hay cinco comportamientos (uno por flanco): los cuatro primeros son los normales; el quinto es el que muestra que el circuito ignora a D después del flanco. Comprobado con 3000 cambios aleatorios de reloj y de D contra un D por flanco ideal.

Libro · Cap. 6 · P6.15

Un circuito con dos D: detector de flancos descendentes

✔ verificado

Completa el cronograma del circuito siguiente y di cuál es su cometido. Son dos biestables D por flanco de subida con el mismo reloj: el primero recibe Ent en D; el segundo recibe Q_1 en D; y una puerta AND da la salida $Sal = \overline{Q_1} \cdot Q_2$.



🧭 Antes de empezar: qué piden y qué teoría usas

Dos biestables en cadena retrasan la entrada dos relojes: Q_1 es Ent retrasada un flanco y Q_2 es Q_1 retrasada otro. La salida mira la diferencia entre ellos.

1 Q_1 y Q_2

En cada flanco de subida Q_1 copia a Ent y Q_2 copia el valor **anterior** de Q_1. Cada uno repite la entrada con un flanco de reloj de retraso respecto al anterior.

2 La salida

$Sal = \overline{Q_1} \cdot Q_2$ vale 1 solo cuando Q_1 ya ha bajado (la entrada fue 0 en el último flanco) pero Q_2 aún recuerda un 1: es decir, **justo el periodo de reloj siguiente a que la entrada baje**. Dura exactamente un periodo T.

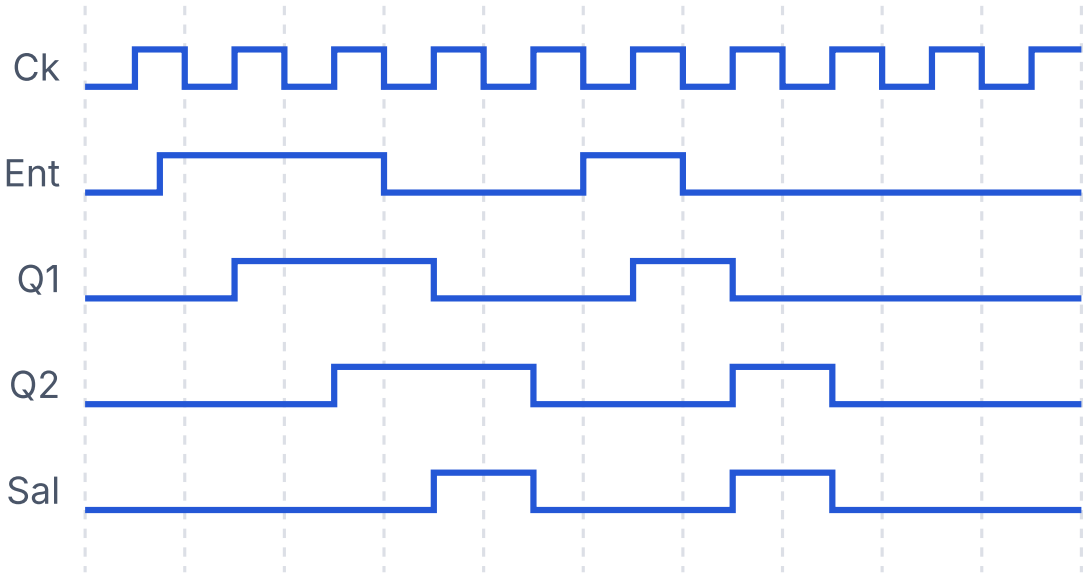
3 Cometido

El circuito **detecta los flancos descendentes** de la entrada y los convierte en un pulso de duración T (un periodo de reloj). Un pulso largo de duración t de la entrada se transforma en uno corto de duración T. El pulso de salida se da cuando la entrada baja a 0, no cuando sube.

4 Variante

Para detectar flancos **ascendentes** (pulso al principio) basta con usar $Sal = Q_1 \cdot \overline{Q_2}$ (se toma Q_1 y $\overline{Q_2}$ en la puerta).

✅ **Resultado final**



Detector de flanco descendente

Sal vale 1 durante exactamente un periodo de reloj tras cada flanco descendente de Ent.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 7 · Registros y contadores

Veintidós problemas de registros, contadores síncronos y asíncronos, 74160/74163 y aplicaciones: cada circuito se ha simulado y comprobado.

💡 Cómo leer esta sección

Son los problemas resueltos del capítulo 7 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras. Los esquemas del libro **no se copian**: cada circuito se describe en texto (qué entradas lleva cada puerta o multiplexor) y se simula con un modelo propio; los cronogramas y las tablas salen de esa simulación. La teoría está en el tema 7.

Modelo de los contadores integrados que usa el capítulo (cuenta en cada flanco de subida del reloj):

CLEAR	LOAD	ENP·ENT	Efecto en el flanco	RCO
0	×	×	borra a 0000 (sin esperar al reloj)	0
1	0	×	carga $D_3..D_0$	—
1	1	0	mantiene	—
1	1	1	cuenta $+1$ (74160: de 9 pasa a 0; 74163: de 15 pasa a 0)	1 si el valor es el último (9 ó 15) y ENT=1

El 74160 es un contador **BCD** (módulo 10) y el 74163 es un contador **binario** de cuatro bits (módulo 16). El borrado se trata como asíncrono, la carga es síncrona (necesita flanco) y las entradas de habilitación ENP y ENT se usan juntas como una sola.

`Libro · Cap. 7 · P7.1` **Registro universal de 4 bits con multiplexores 4:1**

✔ verificado

Dibuja un registro de 4 bits con biestables D cuya operación se elige con dos líneas de control S_1S_0: 11 carga en paralelo la entrada $E_3..E_0$, 10 invierte el contenido, 01 lo pone a cero de forma **síncrona** y 00 lo deja como está. Añade una línea EO (enable de salida) que ponga las salidas en alta impedancia mediante buffers triestado. Hay también una entrada de borrado asíncrono $\overline{INICIO}$.

🧭 Antes de empezar: qué piden y qué teoría usas

Un multiplexor delante de cada D. El biestable D copia lo que le llegue; si lo que le llega es una de cuatro posibilidades elegidas por S_1S_0, se consigue cada operación sin tocar el biestable.

1 Qué lleva cada entrada del multiplexor

Para el bit i, el mux 4:1 recibe: entrada 0 → Q_i; entrada 1 → el valor constante 0; entrada 2 → $\overline{Q_i}$; entrada 3 → E_i. Las líneas S_1S_0 van a las entradas de selección de **los cuatro multiplexores a la vez**.

2 Por qué funciona

$S_1S_0 = 00$: $D_i = Q_i$, y el flanco vuelve a escribir lo mismo (**inhibición**). 01: $D_i = 0$ (**borrado síncrono**, distinto del asíncrono $\overline{INICIO}$, que no espera al reloj). 10: $D_i = \overline{Q_i}$ (**volteo**). 11: $D_i = E_i$ (**carga**).

3 Salida con triestados

Cada Q_i pasa por un buffer triestado gobernado por EO: con $EO = 1$ la salida muestra el valor y con $EO = 0$ queda desconectada (Z), de modo que varios registros pueden compartir los mismos hilos de un bus.

✅ Resultado final

S_1S_0	Entrada del mux que se elige	D_i	Operación
00	0	Q_i	inhibición
01	1	0	borrado síncrono
10	2	$\overline{Q_i}$	volteo
11	3	E_i	carga en paralelo

Comprobado de forma exhaustiva: las $16 \cdot 16 \cdot 4 = 1024$ combinaciones de estado, entrada y selección coinciden con la descripción funcional (conservar, 0, $n \oplus 1111$, E).

`Libro · Cap. 7 · P7.2`

Registro de 4 bits con desplazamientos circulares

✔ verificado

Implementa un registro de 4 bits con biestables D y cuatro operaciones elegidas por S_1S_0: 11 carga en paralelo; 10 desplaza una posición **a la izquierda con recirculación** (el bit que sale por arriba entra por abajo); 01 desplaza **a la derecha con recirculación**; 00 inhibe.

🧭 Antes de empezar: qué piden y qué teoría usas

Cada bit toma el valor de un vecino. Desplazar es que D_i valga el bit vecino de antes del flanco; recircular es que los extremos se cierren en anillo.

1 Qué lleva cada multiplexor

Para el bit i (los índices se toman módulo 4): entrada 0 → Q_i; entrada 1 → Q_{i+1} (el vecino de más peso: **derecha**); entrada 2 → Q_{i-1} (el de menos peso: **izquierda**); entrada 3 → E_i. El bit 3 recibe Q_0 en la entrada 1 y el bit 0 recibe Q_3 en la entrada 2, y eso es la recirculación.

2 Lectura de las operaciones

01: cada $D_i = Q_{i+1}$, el contenido se mueve hacia el bit 0 y el bit Q_0 vuelve al bit 3 (rotación a la derecha). 10: cada $D_i = Q_{i-1}$ y Q_3 vuelve a Q_0 (rotación a la izquierda).

3 Propiedades comprobadas

Cuatro rotaciones seguidas devuelven el valor inicial. Rotar a la izquierda equivale a **multiplicar por 2 módulo 15** (con el 15 como punto fijo): por ejemplo $1011 \to 0111 \to 1110 \to 1101 \to 1011$ ($11 \to 7 \to 14 \to 13 \to 11$).

✅ Resultado final

Rotación a la izquierda de 1011 en cuatro flancos: 1011 → 0111 → 1110 → 1101 → 1011.

S_1S_0	Mux del bit i elige	Operación
00	Q_i	inhibición
01	Q_{i+1}	rotación a la derecha
10	Q_{i-1}	rotación a la izquierda
11	E_i	carga en paralelo

Verificado con las 1024 combinaciones de estado, entrada y selección.

Libro · Cap. 7 · P7.3

Registro con dos buses bidireccionales

✔ verificado

Diseña un registro de 4 bits con dos buses de datos bidireccionales, $A_3..A_0$ y $B_3..B_0$. El registro se carga desde uno de los buses y se lee por el otro. Una línea C fija el sentido: con $C = 1$ la transferencia es de A a B y con $C = 0$ de B a A.

🧭 Antes de empezar: qué piden y qué teoría usas

Un par de triestados por cada cara del biestable. Se necesitan dos buffers que lleven datos hacia la entrada D (uno desde A y otro desde B) y otros dos que saquen Q hacia cada bus. Se habilitan por parejas para que nunca dos salidas empujen el mismo hilo.

1 Cableado por bit

Entrada D: un buffer desde A_i habilitado con $C = 1$ y otro desde B_i habilitado con $C = 0$ (la línea C llega invertida al segundo). Salida Q_i: un buffer hacia B_i habilitado con $C = 1$ y otro hacia A_i habilitado con $C = 0$.

2 Con $C = 1$

El registro captura A en el flanco y Q se pone en B: transferencia $A \rightarrow B$. El buffer $Q \rightarrow A$ está en alta impedancia y por eso A puede ser conducido desde fuera sin conflicto.

3 Con $C = 0$

Es la imagen en espejo: se captura B y se muestra en A.

4 Qué puede salir mal

El diseño **supone que quien conduce el bus cumple el sentido**. Si con $C = 1$ otro dispositivo conduce B con un valor distinto del que presenta Q, hay un cortocircuito lógico (conflicto). El modelo con valores 0, 1, Z y X lo detecta en los dos casos posibles.

✅ Resultado final

C	Se captura	Se muestra	Buffers activos
1	A	Q en B	$A \to D$ y $Q \to B$
0	B	Q en A	$B \to D$ y $Q \to A$

Simulado con las ocho combinaciones de C, valor previo de Q y dato externo: el dato llega siempre al registro y aparece en el otro bus, sin conflictos mientras se respete el sentido.

`Libro · Cap. 7 · P7.4`

Registro con un solo bus de datos

✔ verificado

Diseña un registro de 4 bits con **un único** bus de datos $D_3..D_0$ que sirve a la vez para escribirlo y para leerlo. La línea $E/\overline{L}$ decide: a 1 el registro se carga desde el bus y a 0 vuelca su contenido sobre el bus.

🧭 Antes de empezar: qué piden y qué teoría usas

Es el caso anterior con un solo bus: cada bit del bus se conecta a la vez a la entrada del biestable (a través de un triestado) y a su salida (a través de otro).

1 Por bit

El buffer de escritura va del bus a la entrada D y se habilita con $E/\overline{L} = 1$. El buffer de lectura va de Q al bus y se habilita con $E/\overline{L} = 0$ (llega invertido). Nunca están activos a la vez.

2 Escritura

Con $E/\overline{L} = 1$ el bus es conducido por quien escribe y en el flanco el registro lo captura; Q no se envía al bus.

3 Lectura

Con $E/\overline{L} = 0$ el registro conduce el bus con su contenido y nadie más debe conducirlo. Durante la lectura también llega un flanco al reloj, pero el biestable captura lo que él mismo está poniendo en el bus, y por tanto no cambia.

✅ Resultado final

$E/\overline{L}$	Operación	Quién conduce el bus
1	se carga $D_i \rightarrow Q_i$	el exterior
0	se descarga $Q_i \rightarrow D_i$	el registro

La lectura es idempotente: si el registro conduce el bus, lo que captura es lo que ya contenía. Comprobado para los dos valores de Q y de dato externo, incluyendo el caso de conflicto (alguien más conduce el bus durante la lectura).

Libro · Cap. 7 · P7.5

Contador de Johnson de tres bits

✔ verificado

Tres biestables D con el mismo reloj están enlazados así: $D_0 = \overline{Q_2}$, $D_1 = Q_0$, $D_2 = Q_1$. Completa el cronograma partiendo de $Q_2Q_1Q_0 = 000$ e indica de qué contador se trata.

🧭 Antes de empezar: qué piden y qué teoría usas

Un registro de desplazamiento realimentado con el último bit negado. Cada bit copia al anterior y el primero recibe lo contrario del último, de modo que el registro se va llenando de unos y después de ceros.

1 Cada flanco

El contenido se desplaza una posición hacia Q_2 y entra por Q_0 el complemento de lo que salía por Q_2. Partiendo de 000 los unos entran uno a uno (001, 011, 111) y a continuación los ceros (110, 100, 000).

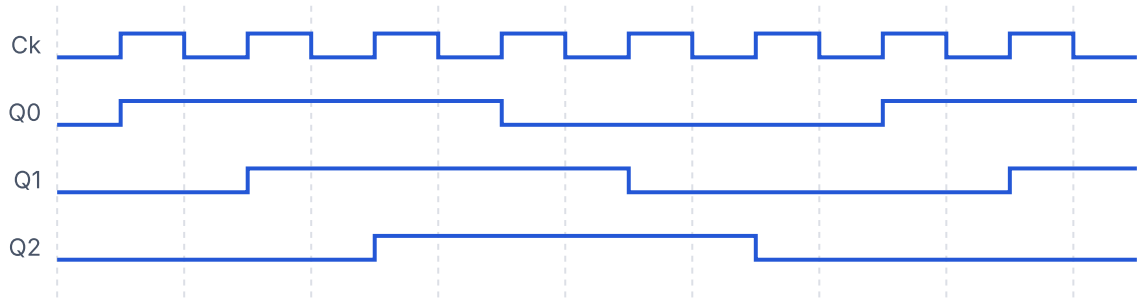
2 Qué contador es

Es un **contador de Johnson** (o de anillo retorcido) de 3 bits: $2 \cdot 3 = 6$ estados de los $2^3 = 8$ posibles, **módulo 6**. Entre dos estados consecutivos cambia un solo bit.

3 Estados sobrantes

Los estados 010 y 101 no pertenecen a la secuencia y forman un ciclo aparte (010 → 101 → 010). Si el contador arranca en uno de ellos no entra solo en la secuencia útil; hay que inicializarlo.

✅ Resultado final



Contador de Johnson de 3 bits

Secuencia $Q_2Q_1Q_0$: 000 → 001 → 011 → 111 → 110 → 100 → 000.

Cada estado se identifica con una puerta de **solo dos entradas** (por ser un código con un único cambio entre estados):

Estado	Decodificador
000	$\overline{Q_0}\cdot\overline{Q_2}$
001	$Q_0\cdot\overline{Q_1}$
011	$Q_1\cdot\overline{Q_2}$
111	$Q_0\cdot Q_2$
110	$\overline{Q_0}\cdot Q_1$
100	$\overline{Q_1}\cdot Q_2$

`Libro · Cap. 7 · P7.6`

Dos contadores con biestables D: uno asíncrono y otro «aberrante»

✔ verificado

Completa los cronogramas de dos circuitos de dos biestables D e indica qué secuencia generan. **Circuito 1:** cada biestable tiene $D = \overline{Q}$ (se carga con lo contrario de lo que tenía); el primero recibe Ck y el segundo recibe Q_0 como reloj, ambos activos por flanco de subida. **Circuito 2:** el primero es igual (D $= \overline{Q_0}$, flanco de subida de Ck); el segundo tiene $D = Q_0$ y es activo por **flanco de bajada** de Ck.

🧭 Antes de empezar: qué piden y qué teoría usas

Hay que preguntar a cada biestable qué hace y cuándo lo hace. Lo primero lo dice el valor de D; lo segundo, la señal que mueve su reloj.

1 Circuito 1 (asíncrono)

Q_0 cambia en cada flanco de subida de Ck. Q_1 cambia en cada flanco de subida de Q_0, que no es el reloj general: por eso el circuito es asíncrono. Como los flancos de subida de Q_0 llegan cada dos relojes, Q_1 cambia más despacio.

2 Secuencia del circuito 1

En el primer flanco Q_0 pasa de 0 a 1 y arrastra a Q_1, que también pasa a 1: se va de 00 a **11**. Después 10, 01, 00: es un contador **descendente de módulo 4** (Q_1Q_0: 00, 11, 10, 01, 00...).

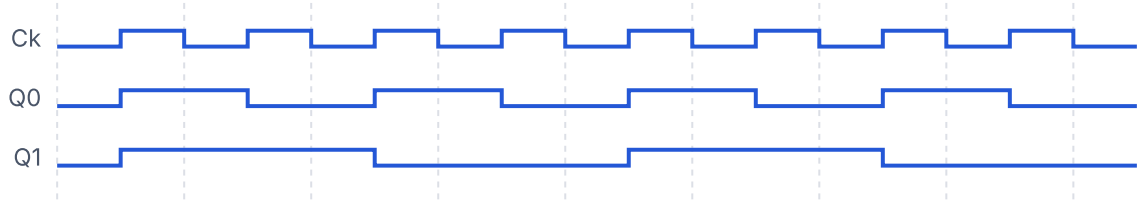
3 Circuito 2

Q_0 cambia en cada flanco de subida; Q_1 copia a Q_0 en cada flanco de **bajada**. Con medio periodo de desfase, la secuencia es 00, 01, 11, 10, 00...: el **código Gray** de dos bits. Cada cambio de estado tiene lugar en un flanco distinto, subida o bajada.

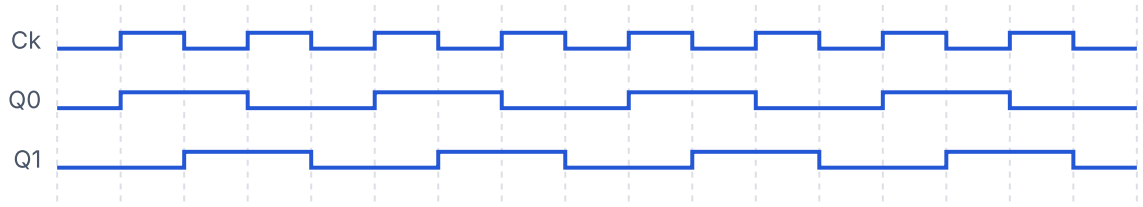
4 Valoración

Los biestables del circuito 2 son síncronos pero respecto de **flancos diferentes** del mismo reloj. El circuito funciona, pero es el tipo de truco que se evita en un diseño síncrono: su interés es teórico.

✅ Resultado final



Circuito 1: contador descendente asíncrono



Circuito 2: secuencia Gray de dos bits

Secuencia Q_1Q_0 del circuito 1: 00 → 11 → 10 → 01 → 00. Del circuito 2 (un estado por cada medio periodo): 00 → 01 → 11 → 10 → 00.

Libro · Cap. 7 · P7.7

Contador asíncrono con una línea TIPO

✔ verificado

Tres biestables J-K activos por flanco de bajada están encadenados: el primero recibe Ck, el segundo recibe Q_0 y el tercero recibe Q_1 como reloj. Las entradas $J_1 = K_1 = J_2 = K_2 = 1$, mientras que $J_0 = K_0 = TIPO$. Completa el cronograma para $TIPO = 1$ y para $TIPO = 0$ e indica la secuencia en cada caso.

🧭 Antes de empezar: qué piden y qué teoría usas

Con $J = K = 1$ un J-K voltea en cada flanco; con $J = K = 0$ no hace nada. El primer biestable manda: si no voltea, tampoco lo harán los siguientes, porque no recibirán flancos.

1 $TIPO = 1$

Los tres biestables voltean. Q_0 cambia en cada bajada de Ck; Q_1 lo hace en cada bajada de Q_0 y Q_2 en cada bajada de Q_1. Al ser flancos de bajada encadenados el conjunto cuenta **hacia arriba**: 000, 001, 010, ..., 111, 000. Es un **contador asíncrono ascendente de módulo 8**.

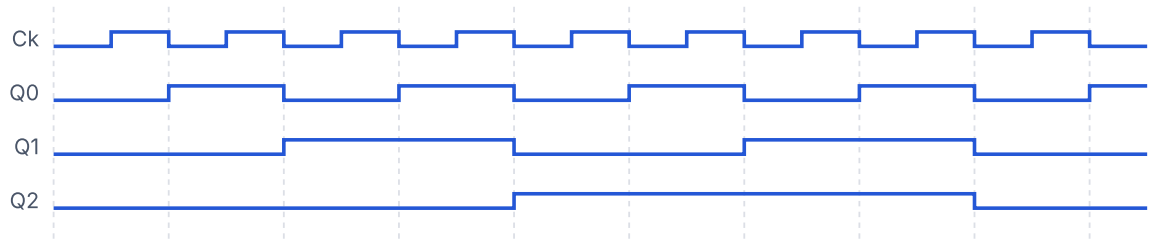
2 $TIPO = 0$

$J_0 = K_0 = 0$: el primero **mantiene** su valor, Q_0 no cambia y el segundo no recibe flancos. Como tampoco los recibe el tercero, el contador **se queda donde estaba** (aquí, en 000).

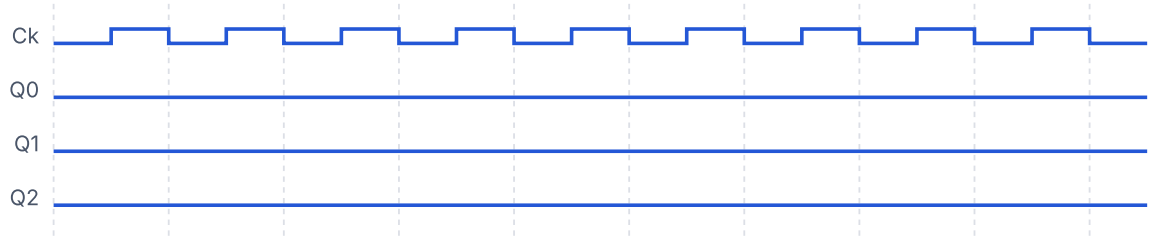
3 Resumen

La línea TIPO funciona como un **enable** de la cuenta: con TIPO a 0 el contador está parado y con TIPO a 1 cuenta en binario natural.

✅ Resultado final



TIPO = 1: cuenta ascendente módulo 8



TIPO = 0: el contador no cuenta

Libro · Cap. 7 · P7.8

Contador BCD síncrono ascendente con biestables J-K

✔ verificado

Diseña un contador síncrono ascendente BCD (0, 1, ..., 9, 0, ...) con biestables J-K activos por flanco de subida.

🧭 Antes de empezar: qué piden y qué teoría usas

Método de diseño síncrono: se escribe la tabla de estados (estado actual → siguiente), se traduce a las entradas J y K que hacen falta en cada biestable mediante la tabla de excitación, se simplifica con Karnaugh y se cablean las funciones. Los seis estados 1010...1111 no existen en BCD y se toman como indiferentes.

1 Tabla de excitación del J-K

Para cada biestable: $0 \to 0$ pide $J = 0, K = \times$; $0 \to 1$ pide $J = 1, K = \times$; $1 \to 0$ pide $J = \times, K = 1$; $1 \to 1$ pide $J = \times, K = 0$.

2 Tabla del contador

Se calcula J y K de cada bit para cada una de las diez filas del BCD (tabla del resultado). Las

seis filas restantes valen × en todas las entradas.

3 Simplificación

Con Karnaugh y las indiferencias: $J_3 = Q_2Q_1Q_0$, $K_3 = Q_0$, $J_2 = K_2 = Q_1Q_0$, $J_1 = \overline{Q_3}Q_0$, $K_1 = Q_0$, $J_0 = K_0 = 1$. El bit 0 siempre voltea y los demás voltean cuando todos los de menor peso valen 1, con la salvedad de que Q_1 no debe subir cuando $Q_3 = 1$ (para no pasar de 9 a 10).

4 Comprobación

Se ha simulado el contador con esas ecuaciones: 0, 1, ..., 9, 0. Además se ha comprobado qué pasa en los estados no usados: **todos entran solos en la secuencia** en un máximo de dos ciclos (10 → 11 → 4; 12 → 13 → 4; 14 → 15 → 0): el contador es **autocorrector**. Es una propiedad que el libro no analiza.

✅ Resultado final

$Q_3Q_2Q_1Q_0$	J_3	K_3	J_2	K_2	J_1	K_1	J_0	K_0
0000	0	X	0	X	0	X	1	X
0001	0	X	0	X	1	X	X	1
0010	0	X	0	X	X	0	1	X
0011	0	X	1	X	X	1	X	1
0100	0	X	X	0	0	X	1	X
0101	0	X	X	0	1	X	X	1
0110	0	X	X	0	X	0	1	X
0111	1	X	X	1	X	1	X	1
1000	X	0	0	X	0	X	1	X
1001	X	1	0	X	0	X	X	1

Funciones: $J_3 = Q_2Q_1Q_0$; $K_3 = Q_0$; $J_2 = K_2 = Q_1Q_0$; $J_1 = \overline{Q_3}\,Q_0$; $K_1 = Q_0$; $J_0 = K_0 = 1$.

Estados no usados: 10 → 11 → 4; 11 → 4; 12 → 13 → 4; 13 → 4; 14 → 15 → 0; 15 → 0.

> **⚠️ Errata en el libro**
>
> En las formas canónicas el libro escribe dos veces «J3»: la segunda lista ($\Sigma(3)$) corresponde a J_2, como indican la tabla y el diagrama de Karnaugh.

`Libro · Cap. 7 · P7.9`

Contador Gray síncrono de módulo 16

✔ verificado

Diseña un contador síncrono ascendente de módulo 16 que recorra los estados en código Gray, con biestables J-K.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Mismo método que el contador BCD, con otra tabla de estados.** Cada estado pasa al siguiente código Gray (0000, 0001, 0011, 0010, 0110, ..., 1000, 0000). Como hay 16 estados, no queda ninguna indiferencia en las filas: solo las que aporta la tabla de excitación del J-K.

1 Tabla de estados y excitación

Se asigna a cada estado el siguiente de la secuencia Gray y se calculan J y K de cada bit con la misma tabla de excitación del problema anterior.

2 Funciones

Las formas canónicas (Σ de las filas, numeradas por el valor binario del estado) son $J_3 = \Sigma(4)$, $K_3 = \Sigma(8)$, $J_2 = \Sigma(2)$, $K_2 = \Sigma(14)$, $J_1 = \Sigma(1, 13)$, $K_1 = \Sigma(7, 11)$, $J_0 = \Sigma(0, 6, 10, 12)$ y $K_0 = \Sigma(3, 5, 9, 15)$. Con las indiferencias salen: $J_3 = Q_2\overline{Q_1}\,\overline{Q_0}$, $K_3 = \overline{Q_2}\,\overline{Q_1}\,\overline{Q_0}$, $J_2 = \overline{Q_3}Q_1\overline{Q_0}$, $K_2 = Q_3Q_1\overline{Q_0}$, $J_1 = Q_3Q_2Q_0 + \overline{Q_3}\,\overline{Q_2}Q_0$, $K_1 = \overline{Q_3}Q_2Q_0 + Q_3\overline{Q_2}Q_0$ y los de J_0 y K_0 con cuatro términos cada uno.

3 Una simplificación más

Los cuatro términos de J_0 y de K_0 son los de la **paridad** de $Q_3Q_2Q_1$: $K_0 = Q_3 \oplus Q_2 \oplus Q_1$ y $J_0 = \overline{K_0}$. Es lógico: en una secuencia Gray el bit menos significativo cambia siempre que haga falta mantener la paridad total, y se comprueba al simular. (Es una observación propia, no del libro.)

4 Comprobación

Con las funciones simplificadas el contador recorre los 16 estados en el orden Gray y vuelve al inicio; entre dos estados consecutivos cambia exactamente un bit.

✅ **Resultado final**

$Q_3..Q_0$ (t)	$Q_3..Q_0$ (t+1)	J_3	K_3	J_2	K_2	J_1	K_1	J_0	K_0
0000	0001	0	X	0	X	0	X	1	X
0001	0011	0	X	0	X	1	X	X	0
0011	0010	0	X	0	X	X	0	X	1
0010	0110	0	X	1	X	X	0	0	X
0110	0111	0	X	X	0	X	0	1	X
0111	0101	0	X	X	0	X	1	X	0
0101	0100	0	X	X	0	0	X	X	1
0100	1100	1	X	X	0	0	X	0	X
1100	1101	X	0	X	0	0	X	1	X
1101	1111	X	0	X	0	1	X	X	0
1111	1110	X	0	X	0	X	0	X	1
1110	1010	X	0	X	1	X	0	0	X
1010	1011	X	0	0	X	X	0	1	X
1011	1001	X	0	0	X	X	1	X	0
1001	1000	X	0	0	X	0	X	X	1
1000	0000	X	1	0	X	0	X	0	X

Secuencia: 0000 → 0001 → 0011 → 0010 → 0110 → 0111 → 0101 → 0100 → 1100 → 1101 → 1111 → 1110 → 1010 → 1011 → 1001 → 1000 → 0000.

`Libro · Cap. 7 · P7.10`

Contadores módulo 60, módulo 1000 y módulo 12 con circuitos integrados

✔ verificado

Con contadores síncronos BCD 74160 diseña un contador de **módulo 60**; haz lo mismo para uno de **módulo 1000** y otro de **módulo 12** en BCD. Para el módulo 12 estudia también una versión con un 74163 binario y un convertidor.

🧭 Antes de empezar: qué piden y qué teoría usas

Se encadenan contadores de modo que cada uno cuente solo cuando todos los anteriores están en su último valor (conexión de RCO a ENP·ENT), y se recorta la cuenta cargando 0 cuando se llega al valor que se quiere evitar.

1 Módulo 60

Dos 74160: **unidades** (0 a 9) y **decenas**. Las decenas cuentan solo cuando las unidades valen 9 (RCO a ENP·ENT). Hay que impedir que lleguen a 6: una NAND de Q_2 y Q_0 de las decenas (valor 5) y del RCO de las unidades (valor 9) detecta 59 y se lleva a LOAD de las decenas, que carga 0 con el siguiente flanco. Así después de 59 se pasa a 00, y no a 60.

2 Qué pasa si falta el RCO en la NAND

Si la puerta solo mirase el valor 5 de las decenas, el contador **no llegaría nunca a 51**: al llegar a 50 las decenas cargarían 0 en el flanco siguiente mientras las unidades siguen contando, y la secuencia pasaría de 50 a **01** (la cuenta quedaría en 1…50). El libro dice «de 50 a 00»; la simulación da 01. El RCO asegura que se espera a que las unidades lleguen a 9.

3 Módulo 1000

Tres 74160 en serie y sin recorte (todos son módulo 10): unidades siempre habilitadas; decenas con RCO de unidades; **centenas con RCO·RCO**. No hace falta usar LOAD. Una propiedad curiosa del cronograma: el valor del contador coincide con el **número de flancos recibidos**, porque al llegar el primer flanco el contador ya vale 000 y pasa a 001.

4 Módulo 12, opción A

Dos 74160 con una NAND de Q_0 de decenas y Q_0 de unidades (valor 11). Su salida va a los dos LOAD (con $D = 0000$). Las decenas cuentan solo con el RCO de las unidades. Secuencia 00, 01, …, 09, 10, 11, 00.

5 Módulo 12, opción B

Un 74163 binario con una NAND de $Q_3 Q_1 Q_0$ (valor 1011) que carga 0. Solo hace falta un integrado, pero la salida está en **binario** (0000 a 1011) y no en BCD. Para visualizarla en dos dígitos hay que convertirla con un conversor binario-BCD (74185): para valores hasta 11 el bit menos significativo pasa sin cambios, porque 10 es par.

✅ Resultado final

Contador	Integrados	Recorte	Secuencia
módulo 60	2 × 74160	NAND (Q_2, Q_0 decenas, RCO unidades) → LOAD decenas	00 … 59, 00
módulo 1000	3 × 74160	ninguno	000 … 999, 000
módulo 12 (A)	2 × 74160	NAND (Q_0 dec., Q_0 uni.) → LOAD ambos	00 … 11, 00
módulo 12 (B)	74163 + conversor	NAND (Q_3, Q_1, Q_0) → LOAD	0000 … 1011, 0000

Simulados los cuatro circuitos durante varios ciclos (130 flancos el de 60, 2100 el de 1000, 40 los de 12): el valor coincide en cada flanco con el contador teórico módulo N.

`Libro · Cap. 7 · P7.11`

Reloj digital de 12 horas con un reloj de 1 kHz

✔ verificado

Diseña un reloj partiendo de una señal de 1 kHz y visualiza horas, minutos y segundos en displays de siete segmentos. Añade una indicación AM/PM.

🧭 Antes de empezar: qué piden y qué teoría usas

Una cadena de divisores: cada contador cuenta solo cuando el anterior ha llegado a su último valor y ahí se pone a cero para empezar de nuevo. Todos los contadores comparten el mismo reloj de 1 kHz y se encadenan con los **enables**, no con relojes distintos (diseño síncrono).

1 Milésimas

Tres 74160 en serie forman un contador de **módulo 1000**: 1000 flancos a 1 kHz son 1 s. Su fin

de cuenta $RCO \cdot RCO \cdot RCO$ vale 1 una vez por segundo y es el enable de los segundos.

2 Segundos y minutos

Cada pareja de 74160 es un **módulo 60** como el del problema anterior: unidades hasta 9; decenas hasta 5. Para que una pareja cuente deben cumplirse a la vez el enable de la anterior, el fin de las unidades (9) y las decenas (5) cuando toca saltar.

3 Una misma puerta, dos usos

La puerta AND que detecta «decenas = 5, unidades = 9 y enable de la etapa anterior» sirve dos veces: da el **enable** de la etapa siguiente (minutos suman uno más) y, invertida, el **LOAD** de las decenas propias, que cargan 0 en ese mismo flanco. Es decir, cada contador se pone a cero justo cuando el siguiente cuenta.

4 Horas y AM/PM

Para las horas se decodifica el valor 12 (decenas Q_0 y unidades Q_1) y se lleva, invertido, a los LOAD de las dos cifras. Es el recorte a módulo 12: el valor 12 solo existe durante un flanco y se pasa a 00. Esa misma señal se lleva a un **biestable T** cuyo estado es la indicación PM, que cambia cada 12 horas.

5 Comprobación

Se ha simulado el circuito completo (9 contadores y el biestable T) en los instantes críticos precargando justo antes de cada desbordamiento: milésimas, segundos, minutos, horas, y paso por las 12 en los dos sentidos (AM→PM y PM→AM). Además, un día entero con reloj de 1 Hz (86 400 flancos) sale exacto.

✅ Resultado final

Etapa	Contador	Cuenta cuando…	Recorte
milésimas	3 × 74160	siempre	ninguno (mod 1000)
segundos	2 × 74160	RCO·RCO·RCO de milésimas	59 → 00

Etapa	Contador	Cuenta cuando...	Recorte
minutos	2 × 74160	fin de segundos (59 s y 999 ms)	59 → 00
horas	2 × 74160	fin de minutos	12 → 00 (módulo 12) y toggle de PM

Ejemplo de paso por mediodía (milésimas en la última columna, de $11{:}59{:}59.995$ AM): 11:59:59.999 AM · 12:00:00.000 AM · 00:00:00.001 PM · 00:00:00.002 PM · 00:00:00.003 PM.

Se observa que el 12 aparece un solo milisegundo antes de volver a 00; la indicación PM cambia en ese flanco y no se vuelve a tocar durante 12 horas.

`Libro · Cap. 7 · P7.12`

Contador módulo 10 que cuenta con el reloj o con un pulsador

✔ verificado

Diseña un contador síncrono módulo 10 que cuente al ritmo de un reloj o bien una vez por cada pulsación de un pulsador. Con el interruptor $I = 0$ debe contar con el reloj; con $I = 1$ debe contar al ritmo del pulsador, una vez por pulsación sea cual sea su duración.

🧭 Antes de empezar: qué piden y qué teoría usas

Hay dos formas de hacerlo y la buena no toca el reloj. Se puede escoger la señal de reloj del contador con un multiplexor (reloj o pulsador), o se puede dejar siempre el mismo reloj y escoger si el contador está **habilitado**.

1 Primera solución: multiplexor en el reloj

Un mux 2:1 elige entre Ck ($I = 0$) y el pulsador ($I = 1$) como reloj de un 74160. Funciona solo si el pulsador está **filtrado** (red RC, báscula R-S...): un pulsador real rebota y cada rebote es un flanco. Con un pulso de entrada sucio de cuatro subidas el contador salta de cuatro en cuatro (simulado). Y la solución tiene un defecto de método: la señal de reloj ya no es siempre Ck.

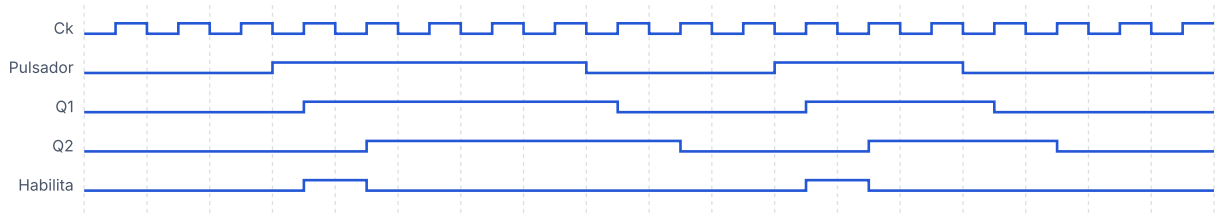
2 Segunda solución: reloj único y detector de flancos

Dos biestables D con el reloj Ck copian el pulsador: Q_1 el valor actual y Q_2 el anterior. La salida $Q_1 \cdot \overline{Q_2}$ vale 1 **exactamente un periodo** después de cada subida del pulsador, sea cual sea el tiempo que se mantenga pulsado. Un mux 2:1 escoge ENP·ENT: con $I = 0$ vale 1 (cuenta con el reloj) y con $I = 1$ vale ese pulso.

3 Comprobación

Con $I = 1$ se han simulado 300 secuencias aleatorias de pulsaciones (de 2 a 12 relojes de duración): el contador avanza **una unidad por pulsación**. Con $I = 0$ avanza una por reloj. Las dos soluciones requieren que el pulsador esté filtrado de rebotes; la segunda además cumple que la única señal de reloj es Ck.

✅ Resultado final



Solución 2 con I = 1 (cada pulsación suma uno)

La señal «Habilita» ($Q_1 \cdot \overline{Q_2}$) dura exactamente un periodo de reloj y aparece **dos veces** para las dos pulsaciones: la primera, aunque se mantenga pulsado durante cinco relojes, y la segunda de tres. El contador suma uno en cada uno de esos dos periodos.

`Libro · Cap. 7 · P7.13`

Cerradura que se abre con la secuencia 1011

✔ verificado

Una puerta se abre automáticamente cuando recibe, de forma síncrona por una línea de entrada, la secuencia 1, 0, 1, 1: debe darse en los **cuatro últimos flancos de reloj**. Diséñala con un registro de desplazamiento.

🧭 Antes de empezar: qué piden y qué teoría usas

Un registro de 4 bits «recuerda» los cuatro últimos bits. Si el bit nuevo entra por Q_0, en Q_3 está el más antiguo y en Q_0 el más reciente. La clave, leída en el orden de llegada $1, 0, 1, 1$, corresponde al contenido $Q_3Q_2Q_1Q_0 = 1011$.

1 Opción A: puerta AND

Una AND de cuatro entradas con Q_2 negado: $Abre = Q_3 \cdot \overline{Q_2} \cdot Q_1 \cdot Q_0$. Requiere un inversor y una AND.

2 Opción B: comparador

Un comparador de cuatro bits (tipo 7485) con una entrada fija a 1011 y la otra conectada a $Q_3..Q_0$: la salida «igual» abre la puerta. Es más potente porque **cambiar la clave** es solo cambiar interruptores.

3 Comparación

La opción A es más sencilla pero menos clara; la B es más versátil. En coste no hay una ventaja grande: un 7485 frente a un 7408 y un 7404.

4 Solapamiento

La puerta se abre **cada vez** que los cuatro últimos bits valen 1011, aunque se solapen con una secuencia anterior: con la entrada 0 1 0 1 1 0 1 1 1 0 1 1 se abre en el cuarto, en el séptimo y en el último flanco (la secuencia 1011 y la siguiente comparten los dos últimos bits).

✅ Resultado final

Comprobado con 400 secuencias aleatorias de longitud 4 a 40: las dos opciones dan la misma salida y se abren exactamente cuando los cuatro últimos bits son 1011.

Flanco	Bit	$Q_3Q_2Q_1Q_0$	Abre
1	0	0000	0
2	1	0001	0
3	0	0010	0
4	1	0101	0
5	1	1011	1
6	0	0110	0
7	1	1101	0
8	1	1011	1
9	1	0111	0
10	0	1110	0
11	1	1101	0

Flanco	Bit	$Q_3Q_2Q_1Q_0$	Abre
12	1	1011	1

Libro · Cap. 7 · P7.14

Visualizar un dígito BCD recibido en serie

✔ verificado

Visualiza en un display de siete segmentos el dígito BCD que llega **en serie** y sincronizado con un reloj. Si el dígito recibido es erróneo (no es BCD), no debe visualizarse.

🧭 Antes de empezar: qué piden y qué teoría usas

Registro de desplazamiento + decodificador BCD-7 segmentos + comparador. El registro convierte los cuatro bits de la entrada serie en un número de cuatro bits; el decodificador lo muestra; y un comparador vigila que no pase de 9.

1 Cableado

Las salidas $Q_3..Q_0$ del registro van a las entradas 8-4-2-1 del decodificador y a la entrada A de un comparador cuya entrada B vale 1001 (9). La salida $A > B$ se lleva a la línea de **apagado** del decodificador (BI), que es activa en bajo: la señal debe llegar invertida (o usarse la salida complementaria) para que el display se apague cuando el número es mayor que 9.

2 Funcionamiento

Si el número recibido es mayor que 9 se activa BI y el display queda en blanco; si es ≤ 9 se ve el dígito. Se ha comprobado con los 16 valores posibles.

3 Un matiz

Como el display está conectado al registro, durante la recepción **se ven valores intermedios**: con la entrada 1 0 0 1 (el 9, bit más significativo primero) se verá sucesivamente 1 → 2 → 4 → 9. Solo después del cuarto flanco hay un valor válido. Para un montaje serio convendría un registro de captura en la salida.

✅ Resultado final

Flanco	Bit recibido	$Q_3Q_2Q_1Q_0$	Display
1	1	0001	1

Flanco	Bit recibido	$Q_3Q_2Q_1Q_0$	Display
2	0	0010	2
3	0	0100	4
4	1	1001	9

Tabla completa: los valores 0000 a 1001 se muestran; 1010 a 1111 dejan el display en blanco.

`Libro · Cap. 7 · P7.15`

Nota de un alumno a partir de tres partes de examen

✔ verificado

Un circuito recibe **en serie** bloques de tres bits: cada bit dice si el alumno ha superado la primera, la segunda y la tercera parte de un examen. Debe encenderse uno solo de tres diodos (o ninguno): NOTA5 si solo se ha superado la primera parte, NOTA7 si se han superado la primera y la segunda, y NOTA10 si se han superado las tres.

🧭 Antes de empezar: qué piden y qué teoría usas

Registro de desplazamiento y tres puertas AND. El registro guarda los tres últimos bits recibidos. Si el bit nuevo entra por Q_0, tras tres flancos Q_2 guarda la primera parte, Q_1 la segunda y Q_0 la tercera.

1 Funciones

NOTA10 $= Q_2Q_1Q_0$; NOTA7 $= Q_2Q_1\overline{Q_0}$; NOTA5 $= Q_2\overline{Q_1}\,\overline{Q_0}$. Los inversores no se pueden eliminar: sin ellos, un alumno con las tres partes superadas encendería los tres diodos a la vez.

2 Defecto de la primera versión

El registro se evalúa en **todos** los flancos, y en los dos que siguen a cada alumno contiene bits de dos alumnos distintos. Con la entrada 0 1 1 | 1 0 0 (el primero no obtiene nota; el segundo aprueba solo la primera parte) los diodos que se encienden son NOTA10 en el flanco 4, NOTA7 en el flanco 5, NOTA5 en el flanco 6: las dos primeras notas son **falsas**, mezcla de dos alumnos.

3 Solución con contador

Un 74160 cargado con 0001 por LOAD cuenta 1, 2, 3, 1, 2, 3…; una AND de Q_1 y Q_0 da 1 solo cuando vale 3, es decir, **una vez cada tres flancos**. Esa señal habilita las tres puertas (que pasan a ser de cuatro entradas) y, invertida, activa el LOAD del contador que recarga 1.

4 Las inicializaciones

Tras INICIO el contador está a **0**, no a 1, y aun así el montaje es correcto: de 0 a 3 hay tres flancos, igual que de 1 a 3 tras la recarga (2 flancos de cuenta más el de carga). Se ha comprobado con 40 alumnos aleatorios: la nota correcta sale una sola vez por alumno y solo en el tercer flanco del bloque.

✅ Resultado final

Flanco	Bit recibido	Contenido $Q_2Q_1Q_0$	Diodo encendido (versión sin contador)
1	0	000	—
2	1	001	—
3	1	011	—
4	1	111	NOTA10
5	0	110	NOTA7
6	0	100	NOTA5

Con el contador, la secuencia de valores del 74160 es $0, 1, 2, 3, 1, 2, 3, \ldots$ y la nota solo se muestra cuando vale 3.

⚠️ Errata en el libro

En el esquema del libro los rótulos $P1, P2, P3$ de la salida del registro aparecen en un orden que no coincide con el cableado de las puertas (según se lee el dibujo, las puertas de NOTA7 y NOTA5 invierten la línea rotulada $P1$, cuando el enunciado exige invertir la **última** parte). La solución de arriba toma la convención coherente: Q_2 es la primera parte, y reproduce la tabla de notas del enunciado.

Libro · Cap. 7 · P7.16 Multiplexado de cuatro displays de siete segmentos

✔ verificado

Diseña un circuito que multiplexe sobre cuatro displays de siete segmentos el contenido de cuatro dígitos BCD. Para los cuatro displays hay solo siete líneas de segmentos y cuatro líneas de enable, una por display. El reloj disponible es de 200 Hz.

🧭 Antes de empezar: qué piden y qué teoría usas

Un solo decodificador BCD-7 segmentos para los cuatro dígitos, repartido en el tiempo. Cada T segundos se enciende un display distinto con el dígito que le corresponde; si el cambio es suficientemente rápido el ojo ve los cuatro encendidos a la vez.

1 Contador de barrido

Un 74160 cuenta 0, 1, 2, 3, 0... porque una NAND de Q_1 y Q_0 (valor 3) carga 0 con LOAD. Solo se usan sus dos bits bajos.

2 Selección del dígito

Los cuatro dígitos pasan por **buffers triestado** hacia el decodificador común. Un decodificador 2:4 con Q_1Q_0 activa un único enable: el del buffer del dígito que toca y, a la vez, el del transistor que alimenta a su display. El triestado puede sustituirse por un multiplexor 4:1 de cuatro bits.

3 Frecuencias

Cada display se enciende uno de cada cuatro ciclos: $200\ Hz/4 = 50\ Hz$ de refresco **por display**, con $T = 5$ ms por dígito. No conviene subir mucho la frecuencia de barrido (1 kHz, 1 MHz): los transistores y el propio display no responden. Entre 50 y 100 Hz por display es lo adecuado, por lo que 200 Hz es una buena elección. Es el mismo principio con el que se barre una pantalla de televisión.

✅ Resultado final

Flanco	Contador	Display activo	Dígito mostrado
0	0	UNI	4
1	1	DEC	3
2	2	CEN	2

Flanco	Contador	Display activo	Dígito mostrado
3	3	MIL	1
4	0	UNI	4
5	1	DEC	3
6	2	CEN	2
7	3	MIL	1

Con el número 1234 de ejemplo (unidades = 4, decenas = 3, ...): en cada flanco hay **exactamente un enable activo** (simulado durante 16 flancos) y el patrón se repite cada cuatro.

`Libro · Cap. 7 · P7.17`

Divisor entre 1, 2, 4 u 8 con un registro universal

✔ verificado

Diseña un divisor cuyo dividendo es un número binario de 4 bits cargado en un registro y cuyo divisor vale 1, 2, 4 u 8 según cuál de cuatro microinterruptores esté activado. El divisor tiene un interruptor I para marcar el inicio de la división.

🧭 Antes de empezar: qué piden y qué teoría usas

Dividir por 2^k es desplazar k posiciones hacia el bit de menor peso. Se usa un registro universal 74179 (que carga en paralelo, desplaza e inhibe) y un contador que cuenta los desplazamientos hechos; un comparador dice si ya se han hecho tantos como pide el divisor.

1 Codificar el divisor

Los cuatro microinterruptores (1, 2, 4, 8) pasan por un codificador 4:2 que da k en dos bits: 1 → 00, 2 → 01, 4 → 10, 8 → 11.

2 Procedimiento

Se activa INICIO (contador a 0), se pone el dividendo en los cuatro microinterruptores y se da un pulso a CARGA DIVIDENDO. Después se pone $I = 1$: mientras el contador es **menor** que k (salida m) el registro desplaza y el contador suma uno; al llegar a k se detiene. El cociente queda estable en las salidas $Q_3..Q_0$. Para otra división hay que volver a $I = 0$ e INICIO.

3 Control del registro

Con C = carga, I = división y m = «contador menor que k», las líneas S_1S_0 valen: $C = 1$ → 01 (carga); $C = 0, I = 1, m = 1$ → 11 (desplaza); $C = 0, I = 1, m = 0$ → 10 (inhibe, final); $C = 0, I = 0$ → 10 (no divide). Por Karnaugh: $S_1 = \overline{C}$ y $S_0 = C + I \cdot m$, con $ENP \cdot ENT = I \cdot m$.

4 Comprobación

Se han probado las $16 \cdot 4 = 64$ divisiones posibles: el cociente es siempre $\lfloor dividendo/divisor \rfloor$ y se hacen exactamente k desplazamientos. Los bits que salen por el extremo son el resto, que este circuito **no conserva**.

✅ Resultado final

C	I	m	S_1S_0	Operación
1	×	×	01	carga el dividendo
0	1	1	11	desplaza y cuenta
0	1	0	10	final: inhibe
0	0	×	10	no divide

Ejemplo: dividendo 1101 (13) entre 4 ($k = 2$): dos desplazamientos dan $1101 \rightarrow 0110 \rightarrow 0011$, es decir, 3 = $\lfloor 13/4 \rfloor$ (el resto, 1, sale por la derecha).

⚠️ Errata en el libro

Dos detalles del libro no concuerdan entre sí: el comentario dice que durante la división $S_1 = 1$ y $S_0 = 0$, pero la tabla del propio ejercicio da $S_1S_0 = 11$ (y $S_0 = C + I \cdot m$). Además, la salida del comparador que se usa debe ser «código mayor que contador» (con A = código y B = contador, la salida **M**), mientras que el texto habla de «contador menor» (m). Lo importante es que el registro desplace hacia el bit de menor peso mientras queden desplazamientos por hacer.

Libro · Cap. 7 · P7.18

Alarma de exceso de velocidad de una rueda

✔ verificado

Un microbot lleva en una rueda cuatro marcas ópticas. Hay que activar una alarma cuando la rueda dé **más de tres vueltas por segundo**. El reloj es de 100 Hz. Diseña el circuito.

🧭 Antes de empezar: qué piden y qué teoría usas

Medir una frecuencia es contar pulsos durante un intervalo conocido. Tres vueltas por segundo con cuatro marcas son 12 pulsos por segundo; la alarma debe saltar si en una ventana de 1 s se cuentan más de 12.

1 Base de tiempos

Dos 74160 en cascada forman un contador de módulo 100 que cuenta los relojes de 100 Hz: la ventana dura 100 relojes, es decir, **1 s**. Cuando los dos valen 9 (99 relojes) una NAND de sus fin-de-cuenta borra el contador de pulsos.

2 Contador de pulsos

Un 74163 (binario de 4 bits) cuenta los pulsos de la rueda. Para que cada marca cuente **una vez** aunque sea larga, la señal de la rueda pasa antes por dos biestables D y una AND ($Q_1 \cdot \overline{Q_2}$), el detector de flancos del problema 7.12.

3 Alarma

Un comparador compara la cuenta con 1100 (12) y su salida «mayor que» enciende la alarma: se activa en cuanto se supera el 12 y se apaga con el borrado de la siguiente ventana.

4 Límites

Un pulso de rueda debe durar al menos un periodo de reloj (0,01 s) para ser detectado; si dura menos hay que subir la frecuencia base (1 kHz o más). Como el 74163 es de módulo 16, a partir de 16 pulsos por segundo (4 vueltas por segundo) la cuenta **da la vuelta**: la alarma se enciende al pasar por 13, 14 y 15, pero se apaga cuando el contador vuelve a 0. Para medir velocidades mayores hay que ampliar el módulo conectando más 74163 en cascada.

✅ Resultado final

Vueltas por segundo	Pulsos por segundo	Cuenta máxima en la ventana	Alarma
1	4	4	no
2	8	8	no
2.5	10	10	no
3.5	14	14	sí
4	16	15	sí
5	20	15	sí

Simulación a nivel de reloj durante cuatro ventanas por velocidad (se descarta la primera, de arranque). A 4 y 5 vueltas por segundo el contador ya ha dado la vuelta: la alarma se enciende un instante, pero no queda fija. Con 3 vueltas por segundo la cuenta está justo en el límite (12 pulsos, ±1 por el desfase entre ventana y rueda).

`Libro · Cap. 7 · P7.19`

Marcador de tenis con dos pulsadores

✔ verificado

Diseña un marcador de tenis controlado por dos pulsadores, uno por jugador. En principio basta con un marcador de puntos que tenga en cuenta el empate (iguales) y la ventaja. Los números pueden visualizarse como 0, 1, 2, 3... en lugar de 0, 15, 30, 40.

🧭 Antes de empezar: qué piden y qué teoría usas

Un contador de puntos por jugador y un circuito que decide cuándo termina el juego. Un juego se gana al alcanzar **al menos cuatro puntos con una diferencia de al menos dos**.

1 Entradas

Cada pulsador pasa por un detector de flancos de dos biestables D y una AND, de modo que cada pulsación suma **un único** punto aunque se mantenga pulsada. Cada jugador tiene un 74163; se supone que un juego nunca pasa de 15 puntos.

2 Quién gana y por cuánto

Un comparador dice cuál de los dos contadores es mayor y dos multiplexores 4×2:1 colocan el mayor como **minuendo** y el menor como **sustraendo**. Un restador calcula la diferencia.

3 Fin de juego

Un comparador contra 0011 da $MD = 1$ si el mayor supera 3 (tiene al menos 4) y otro contra 0001 da $MR = 1$ si la diferencia supera 1 (es al menos 2). $JUEGO = MD \cdot MR$.

4 Qué hace JUEGO

Activa, invertida, la línea LOAD de los dos contadores (con $D = 0$) para ponerlos a 0; se lleva a un demultiplexor 1:2 que enciende el led del ganador (la señal de «A mayor» elige el led) y podría habilitar un contador de juegos ganados por cada jugador. Todo el circuito comparte la misma línea de reloj.

5 Comprobación

La lógica combinacional coincide con la regla del tenis para las $16 \cdot 16 = 256$ combinaciones de puntos. Además se han simulado 500 partidos aleatorios de 4 a 40 puntos y los juegos (quién gana y con qué tanteo) coinciden con una simulación directa de la regla.

✅ Resultado final

Puntos A	Puntos B	Mayor	Diferencia	MD	MR	JUEGO
3	0	3	3	0	1	0
4	0	4	4	1	1	1
4	2	4	2	1	1	1
4	3	4	1	1	0	0
3	3	3	0	0	0	0
5	3	5	2	1	1	1
5	4	5	1	1	0	0
6	4	6	2	1	1	1

Ejemplo con los puntos A A B B A A A B B B B A A B B B (A y B alternan): juegos terminados gana A por 4-2, gana B por 1-4.

Libro · Cap. 7 · P7.20

Contador asíncrono de 0 a 5 con puesta a cero

✔ verificado

Diseña un contador **asíncrono** activo por flanco de bajada cuya secuencia sea 0, 1, 2, 3, 4, 5, 0, 1... Completa el cronograma sin tener en cuenta los retardos.

🧭 Antes de empezar: qué piden y qué teoría usas

Se deja contar libremente hasta el primer estado que no se quiere y se obliga a volver a cero. Con tres biestables J-K con $J = K = 1$ se tiene un contador asíncrono de módulo 8; el estado 6 (110) es el primero no deseado.

1 Contador base

Tres J-K con $J = K = 1$: Ck al primero y cada salida Q al reloj del siguiente. Con flancos de bajada así encadenados la cuenta es ascendente de módulo 8.

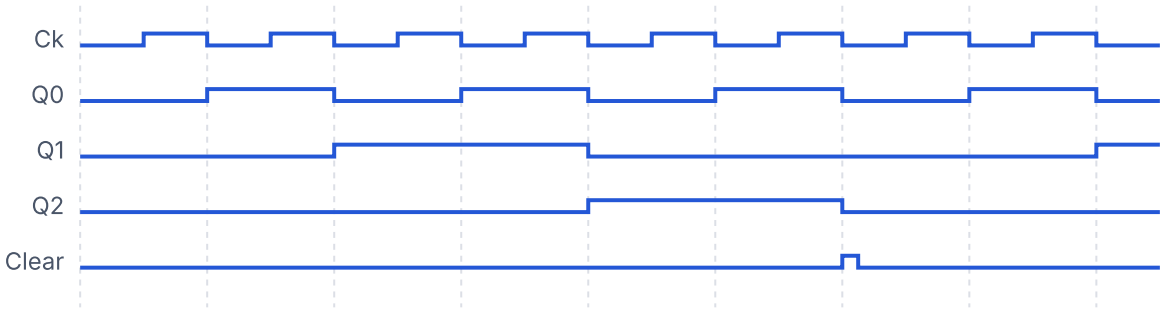
2 Recorte

Una puerta NAND decodifica 110 ($Q_2 \cdot Q_1 \cdot \overline{Q_0}$) y su salida, activa en bajo, se lleva a los **CLEAR** de los tres biestables. En cuanto el contador llega a 6 se pone a 0 de forma asíncrona. La NAND no necesita $\overline{Q_0}$: basta $Q_2 \cdot Q_1$, porque el 7 nunca se alcanza (se comprobó que la secuencia es la misma).

3 El «glitch»

En el flanco que debería llevar el contador de 5 a 6, el contador **llega a tomar el valor 6** durante un breve lapso, el que tarda la NAND y el borrado en actuar. Es un estado transitorio no deseado (un *glitch*) que puede alterar los circuitos que lean las salidas. Por eso estos contadores tienen interés sobre todo teórico.

✅ Resultado final



Cuenta 0-5 (el pulso «Clear» está exagerado en anchura)

Secuencia: 0 → 1 → 2 → 3 → 4 → 5 → 0. En cada vuelta aparece el estado transitorio 6 (110) antes de volver a 0.

Libro · Cap. 7 · P7.21

Contador BCD asíncrono con flancos de bajada

✔ verificado

Diseña un contador ascendente BCD **asíncrono** activo por flanco de bajada y completa el cronograma, sin tener en cuenta los retardos.

🧭 Antes de empezar: qué piden y qué teoría usas

Mismo método que el problema anterior con cuatro biestables. Un contador asíncrono ascendente de cuatro J-K con $J = K = 1$ es de módulo 16; se decodifica el primer estado que no es BCD, 1010, y se lleva a los CLEAR.

1 Decodificar el 10

NAND de $Q_3 \cdot \overline{Q_2} \cdot Q_1 \cdot \overline{Q_0}$ hacia los cuatro CLEAR. Basta con $Q_3 \cdot Q_1$ (en una cuenta de 0 a 9 solo el 10 las cumple primero); el libro lo señala y la simulación lo confirma.

2 Resultado

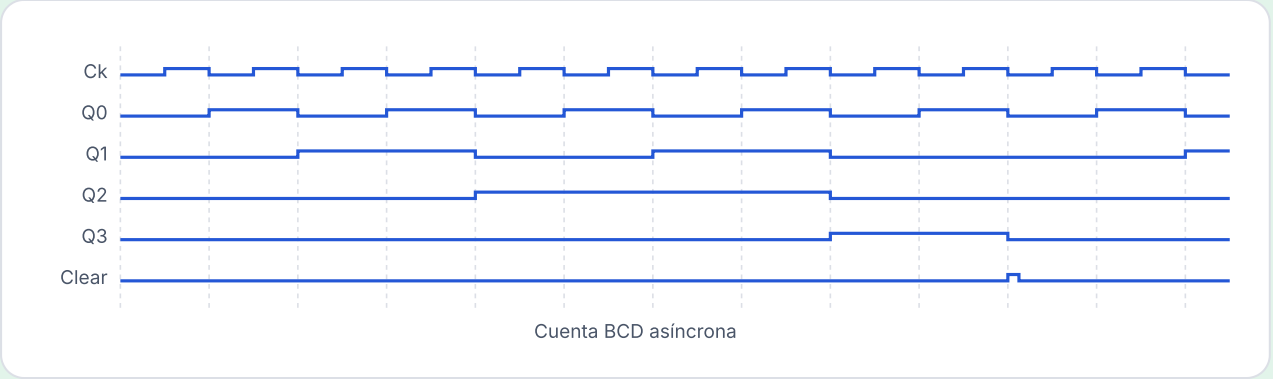
El contador pasa por 0, 1, ..., 9 y, al llegar a 10, vuelve a 0. Como en el caso anterior hay un instante en que el contador vale 10 antes del borrado (glitch).

3 Comprobación

Se simula el encadenamiento de flancos con retardo cero entre biestables: se comprueba que durante los rizados no se cumple la condición de borrado en ningún estado intermedio salvo en

el 10.

✅ **Resultado final**



Secuencia: 0 → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 0.

`Libro · Cap. 7 · P7.22`

Secuencia de un contador asíncrono con preset

✔ verificado

Tres biestables J-K con $J = K = 1$ están conectados así: el reloj Ck va al primero, la salida Q_0 al reloj del segundo y Q_1 al reloj del tercero, todos activos por flanco de **subida**. Una puerta NAND recibe $\overline{Q_2}$, Q_1 y $\overline{Q_0}$ y su salida va a los **PRESET** de los tres. Indica la secuencia que genera y completa el cronograma, sin tener en cuenta los retardos.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Es un circuito secuencial asíncrono, un contador. Hay que decidir si cuenta hacia arriba o hacia abajo, cuál es su módulo máximo y qué hace la puerta.

1 Sentido

Cada Q va al reloj del siguiente y el reloj es de subida: el contador es **descendente** (la salida del primero sube cuando le toca bajar al segundo: 7, 6, 5, ...).

2 Máximo

Con tres biestables el módulo máximo es 8.

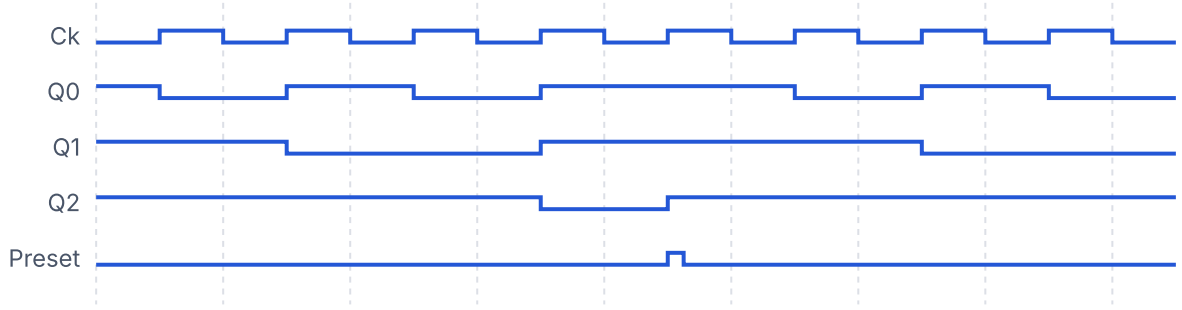
3 La puerta

La NAND decodifica $\overline{Q_2}\,Q_1\,\overline{Q_0} = 010$, el valor **2**, que es el primer estado no deseado de la cuenta descendente. Su salida va a los PRESET: fuerza **111 = 7**.

4 Resultado

El contador se recorta a 5 estados y genera 7, 6, 5, 4, 3, 7, 6, ... Es un contador asíncrono descendente de **módulo 5**. En cada vuelta aparece brevemente el 2 antes del preset (glitch).

✅ Resultado final



Cuenta asíncrona descendente módulo 5

Secuencia: 7 → 6 → 5 → 4 → 3 → 7 (y se repite).

⚠️ Errata en el libro

En el último punto del comentario el libro llama a este contador «ascendente de módulo 5», pero la secuencia que escribe a continuación (7, 6, 5, 4, 3, 7) es **descendente**, como dice él mismo unas líneas antes.

EJERCICIOS RESUELTOS

Libro de problemas · Capítulo 8 · Autómatas

Treinta y cinco problemas de diseño de autómatas de Moore y de Mealy: detectores de secuencias, aritmética serie, contadores, registros y automatismos industriales. Cada máquina se ha simulado contra un modelo de referencia y cada ecuación se ha comprobado.

💡 Cómo leer esta sección

Son los problemas resueltos del capítulo 8 del libro *Problemas resueltos de electrónica digital* (ITES-Paraninfo), reformulados con otras palabras. Los diagramas del libro **no se copian**: cada autómata se da como tabla de transición y, cuando es pequeño, como diagrama dibujado aquí. Las ecuaciones J-K y D que imprime el libro se comprueban **una a una** contra la tabla de estados, y las máquinas se simulan frente a un modelo de referencia con miles de entradas aleatorias. Si algo del libro no cuadra, se anota en un cuadro de *Errata*. La teoría está en el tema 10.

Método común de todos los problemas (flujo de diseño):

1. **Diagrama de transición de estados**: qué hay que recordar y cómo cambia con cada entrada (Mealy: salida en la transición; Moore: salida en el estado).
2. **Tabla de transición** y reducción de estados (estados equivalentes se fusionan).
3. **Codificación** de estados con n biestables, $2^n \geq$ número de estados; los códigos sobrantes y las entradas imposibles son **indiferentes** ($\times$).
4. **Tabla de excitación** del biestable elegido. Para el J-K: $0 \to 0$ pide $J = 0, K = \times$; $0 \to 1$ pide $J = 1, K = \times$; $1 \to 0$ pide $J = \times, K = 1$; $1 \to 1$ pide $J = \times, K = 0$. Para el D: $D = Q^+$.
5. **Minimización** (Karnaugh o Quine-McCluskey) de J, K (o D) y de la salida, y **circuito**.

En este capítulo el orden de las variables es siempre *entradas primero, estado después* ($E\,q_1 q_0$), y el estado inicial lo fija el RESET asíncrono de los biestables.

`Libro · Cap. 8 · P8.1` **Detector de tres unos seguidos (Mealy)** ✔ verificado

Diseña un autómata de **Mealy** que vigile una línea serie E_0 y ponga su salida a 1 cuando lleguen **tres o más unos consecutivos**: la salida vale 1 en cada uno que completa o prolonga una racha de al menos tres. Impleméntalo con biestables J-K.

🧭 Antes de empezar: qué piden y qué teoría usas

Un autómata recuerda sólo lo imprescindible del pasado. Aquí basta saber cuántos unos seguidos acaban de llegar: ninguno, uno, o dos o más. Eso son **tres estados**. Como la salida depende de la entrada actual (el tercer 1 se detecta *al llegar*), es natural un Mealy.

1 Estados

Q_0: la racha de unos es de longitud 0. Q_1: acaba de llegar un 1. Q_2: llevamos dos o más unos seguidos.

2 Transiciones

Un 0 rompe la racha desde cualquier estado y devuelve a Q_0. Un 1 avanza: $Q_0 \to Q_1 \to Q_2$ y Q_2 se queda en Q_2. La salida vale 1 sólo en la transición $Q_2 \xrightarrow{1} Q_2$ (es el tercer 1 o posterior). Se permiten solapamientos: 1111 da dos detecciones.

3 Codificación

Con tres estados hacen falta dos biestables: $Q_0 = 00$, $Q_1 = 01$, $Q_2 = 10$. El código 11 no se usa y es indiferente.

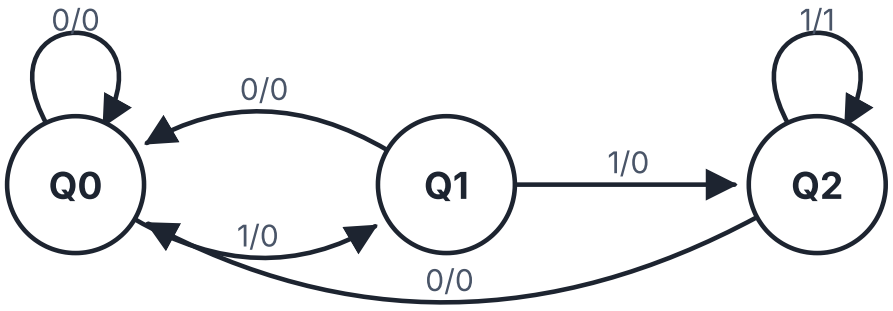
4 Tabla de excitación y minimización

Para cada fila $(E_0, q_1 q_0)$ se anota qué pide cada biestable J-K según el cambio $q \to Q$. Minimizando con los códigos libres como indiferentes sale el resultado de abajo.

5 Circuito

Dos biestables J-K con el mismo reloj. J_1 es una AND de E_0 y q_0; K_1 es $\overline{E_0}$ (un inversor); J_0 es una AND de E_0 y $\overline{q_1}$; K_0 está a nivel alto fijo; la salida es una AND de E_0 y q_1.

✅ Resultado final



Detector de 111 (Mealy). Etiquetas: E0/S

Estado	0	1
Q0	Q0/0	Q1/0
Q1	Q0/0	Q2/0
Q2	Q0/0	Q2/1

Estado	Código q_1q_0
Q0	00
Q1	01
Q2	10

Función	Ecuación
J_1	$E_0 \cdot q_0$
K_1	$\overline{E_0}$
J_0	$E_0 \cdot \overline{q_1}$
K_0	1
S	$E_0 \cdot q_1$

Las ecuaciones impresas en el libro se han comprobado en **las seis transiciones definidas** y reproducen la tabla. Mi propia minimización coincide con ellas término a término.

Coste de la lógica combinacional: 3 puertas AND, 0 OR y 2 inversores (sin contar los biestables).

Simulación: 200 flujos aleatorios de 60 bits comparados con el modelo «los tres últimos bits son 111»; coinciden en todos.

Libro · Cap. 8 · P8.2

Detector de tres ceros seguidos (Mealy, con J-K y con D)

✔ verificado

Igual que el problema anterior pero detectando **tres o más ceros seguidos**. Impleméntalo con biestables J-K y después con biestables D.

🧭 Antes de empezar: qué piden y qué teoría usas

Es el mismo autómata con los papeles de 0 y 1 intercambiados. Sólo cambian las ecuaciones: donde antes aparecía E_0 ahora aparece $\overline{E_0}$. Lo interesante es comparar las dos tecnologías de biestable.

1 Autómata

Tres estados que cuentan los ceros seguidos (0, 1, 2 o más). Un 1 devuelve a Q_0. La salida es 1 en $Q_2 \xrightarrow{0} Q_2$.

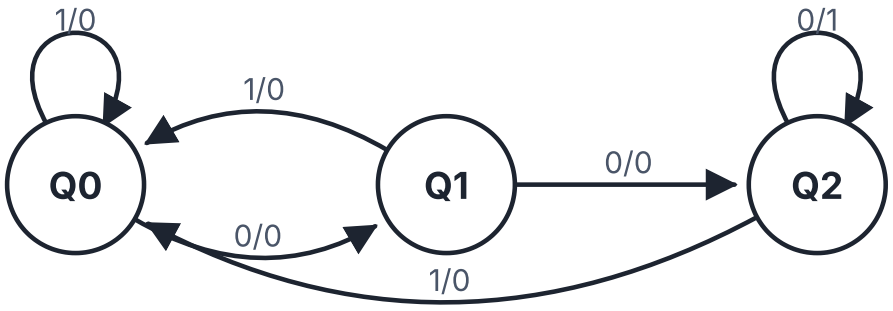
2 Con biestables D

Como $D = Q^+$, basta mirar en qué filas el código siguiente tiene cada bit a 1: D_0 sólo vale 1 al pasar de Q_0 a Q_1 con un 0, y D_1 vale 1 al ir a Q_2.

3 Con biestables J-K

Las ecuaciones tienen más términos de partida (hay dos funciones por biestable) pero se simplifican mucho gracias a los indiferentes: $K_0 = 1$ siempre.

✅ Resultado final



Detector de 000 (Mealy). Etiquetas: E0/S

Estado	0	1
Q0	Q1/0	Q0/0
Q1	Q2/0	Q0/0
Q2	Q2/1	Q0/0

Con biestables J-K

Función	Ecuación
J_1	$\overline{E_0} \cdot q_0$
K_1	E_0
J_0	$\overline{E_0} \cdot \overline{q_1}$
K_0	1
S	$\overline{E_0} \cdot q_1$

Con biestables D

Función	Ecuación
D_1	$\overline{E_0} \cdot q_0 + \overline{E_0} \cdot q_1$
D_0	$\overline{E_0} \cdot \overline{q_1} \cdot \overline{q_0}$
S	$\overline{E_0} \cdot q_1$

Ambos juegos de ecuaciones, los del libro y los míos, se han comprobado contra la tabla. Con D se necesitan dos funciones y con J-K cuatro, pero las del J-K son mucho más cortas: es el compromiso habitual (el J-K simplifica la lógica a cambio de más hilos).

Libro · Cap. 8 · P8.3

Detector de la secuencia 1011 con solapamiento

✔ verificado

Diseña un autómata de Mealy que detecte la secuencia **1011** en una línea serie, permitiendo que dos apariciones se solapen (el último 1 de una puede ser el primero de la siguiente).

🧭 Antes de empezar: qué piden y qué teoría usas

Cada estado representa «lo que llevo escrito del patrón»: nada (Q_0), «1» (Q_1), «10» (Q_2) y «101» (Q_3). Lo difícil son las transiciones que *no* avanzan: hay que volver al **sufijo más largo que siga siendo prefijo del patrón**.

1 Transiciones que avanzan

$Q_0 \xrightarrow{1} Q_1$, $Q_1 \xrightarrow{0} Q_2$, $Q_2 \xrightarrow{1} Q_3$ y $Q_3 \xrightarrow{1}$ (se completa 1011: salida 1).

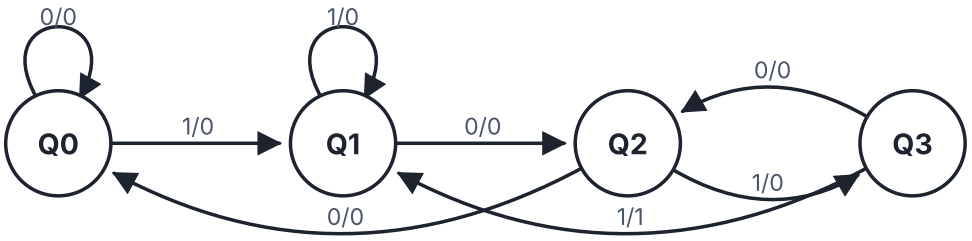
2 Transiciones que retroceden

En Q_1 («1») un nuevo 1 deja «1»: se queda en Q_1. En Q_2 («10») un 0 da «100»: sin prefijo útil, a Q_0. En Q_3 («101») un 0 da «1010» y su sufijo «10» es prefijo del patrón: a Q_2. Tras detectar (1011) el último 1 queda como primer 1 de otra posible aparición: a Q_1 (solapamiento).

3 Codificación y ecuaciones

Cuatro estados, dos biestables, código natural. Al estar los cuatro códigos usados no hay indiferentes por códigos libres.

✅ Resultado final



Detector de 1011. Etiquetas: E0/S

Estado	0	1
Q0	Q0/0	Q1/0
Q1	Q2/0	Q1/0
Q2	Q0/0	Q3/0
Q3	Q2/0	Q1/1

Función	Ecuación
J_1	$\overline{E_0} \cdot q_0$
K_1	$\overline{E_0} \cdot \overline{q_0} + E_0 \cdot q_0$
J_0	E_0
K_0	$\overline{E_0}$
S	$E_0 \cdot q_1 \cdot q_0$

Comprobadas las ocho transiciones y simulados 200 flujos aleatorios de 60 bits contra «los cuatro últimos bits son 1011».

`Libro · Cap. 8 · P8.4`

Tres bits iguales seguidos, ceros o unos (Mealy, 5 estados)

✔ verificado

Diseña un autómata de Mealy que active su salida cuando en la línea E_0 lleguen **tres o más bits iguales seguidos**, sean tres ceros o tres unos.

🧭 Antes de empezar: qué piden y qué teoría usas

Ahora el estado debe recordar dos cosas: cuál fue el último bit y cuántas veces seguidas se repite. Hay que distinguir «un 0 suelto», «dos ceros», «tres o más ceros» y lo mismo para los unos; pero el primer bit de la secuencia, que aún no tiene antecesor, es el estado inicial Q_0. Total: **cinco estados**.

1 Estados

Q_0 inicial. Q_1: un 0 (racha de ceros de longitud 1). Q_2: dos o más ceros. Q_3: un 1. Q_4: dos o más unos.

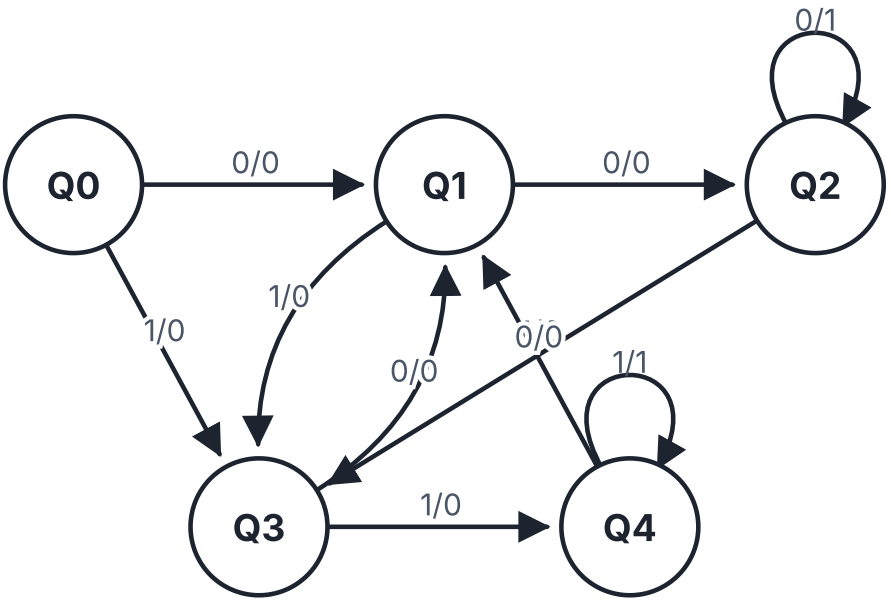
2 Transiciones

Un bit igual al de la racha la alarga (y si ya era de longitud 2 o más, la salida es 1); un bit distinto empieza una racha nueva de longitud 1 (de Q_1, Q_2 o Q_4 se pasa a Q_3 con un 1; de Q_3 o Q_4 a Q_1 con un 0).

3 Tres biestables

Cinco estados necesitan $\lceil \log_2 5 \rceil = 3$ biestables. Los códigos 101, 110 y 111 son indiferentes: eso permite simplificar mucho las ecuaciones.

✅ Resultado final



Tres bits iguales seguidos (Mealy)

Estado	0	1
Q0	Q1/0	Q3/0
Q1	Q2/0	Q3/0
Q2	Q2/1	Q3/0
Q3	Q1/0	Q4/0
Q4	Q1/0	Q4/1

Función	Ecuación
J_2	$E_0 \cdot q_1 \cdot q_0$
K_2	$\overline{E_0}$
J_1	$q_0 + E_0 \cdot \overline{q_2}$
K_1	q_0
J_0	$E_0 \cdot \overline{q_2} + \overline{E_0} \cdot \overline{q_1}$

Función	Ecuación
K_0	$\overline{E_0} \cdot \overline{q_1} + E_0 \cdot q_1$
S	$\overline{E_0} \cdot q_1 \cdot \overline{q_0} + E_0 \cdot q_2$

Se han comprobado las ecuaciones del libro en las 10 transiciones definidas (las combinaciones con códigos 101-111 son indiferentes) y se ha simulado contra el modelo «los tres últimos bits son iguales». Reducción de estados: el algoritmo de particiones no fusiona ninguno, así que cinco es el mínimo.

Libro · Cap. 8 · P8.5

Detectar 1111 y 11011 con solapamiento (Moore y Mealy)

✔ verificado

Diseña, como autómata de **Mealy** y como autómata de **Moore**, un detector de **dos** secuencias a la vez: 1111 y 11011. Se permite que las apariciones se solapen y que se pase de una secuencia a otra sin volver al principio.

🧭 Antes de empezar: qué piden y qué teoría usas

El estado recuerda el sufijo más largo de lo recibido que todavía puede convertirse en alguna de las dos secuencias. Los prefijos propios de los patrones son ε, 1, 11, 111, 110 y 1101: **seis estados**. El Moore necesita dos más porque «detecté ahora» debe vivir en el estado y no en la transición.

1 Mealy

Se construye por «longest suffix»: desde el prefijo p y la entrada b se forma pb; si acaba en algún patrón completo la salida vale 1; el estado siguiente es el sufijo más largo de pb que sea prefijo propio de algún patrón. Por ejemplo, de «111» con un 1 se completa 1111 y queda «111» (solapamiento).

2 Moore

Cada estado Mealy se **duplica** si se puede llegar a él tanto con salida 0 como con salida 1. Aquí le pasa a «111» (tras 1111) y a «11» (tras 11011): 6 + 2 = **8 estados**, sin fusiones posibles.

3 Comprobación

Ambas máquinas se simulan contra «los cuatro últimos bits son 1111 o los cinco últimos 11011» con 200 flujos aleatorios de 60 bits.

✅ **Resultado final**

Mealy (6 estados) — estado = sufijo útil

Estado	Sufijo
Q0	ε
Q1	1
Q2	11
Q3	110
Q4	111
Q5	1101

Estado	0	1
Q0	Q0/0	Q1/0
Q1	Q0/0	Q2/0
Q2	Q3/0	Q4/0
Q3	Q0/0	Q5/0
Q4	Q3/0	Q4/1
Q5	Q0/0	Q2/1

Moore (8 estados)

Estado	0	1	Salida
Q0	Q0	Q1	0
Q1	Q0	Q2	0
Q2	Q3	Q4	0
Q3	Q0	Q5	0

Estado	0	1	Salida
Q4	Q3	Q6	0
Q5	Q0	Q7	0
Q6	Q3	Q6	1
Q7	Q3	Q4	1

⚠️ Errata en el libro

En el dibujo del libro del Mealy hay una flecha desde Q_5 con entrada 1 cuyo destino es ambiguo en la figura (acaba en un nudo de trazos). Por eso aquí se construye el autómata a partir de los sufijos y se verifica por simulación, en lugar de copiar esa flecha. El número de estados (6 y 8) sí coincide con el del libro.

`Libro · Cap. 8 · P8.6`

Recordar cuál fue la última secuencia detectada: 010 o 110

✔ verificado

Una línea serie E_0 puede contener las secuencias 010 y 110. Diseña un autómata cuya salida valga **1 si la última secuencia detectada fue 010** y **0 si fue 110** (al empezar vale 0). Hazlo de Moore y de Mealy.

🧭 Antes de empezar: qué piden y qué teoría usas

La salida es una memoria de un bit (cuál fue la última detección), y además hay que seguir buscando los patrones en la línea. El estado debe contener ambas cosas: el valor de la salida y la parte útil de los dos últimos bits.

1 Modelo de referencia

Se parte de un modelo que lleva la salida o y los dos últimos bits: si los tres últimos son 010, $o = 1$; si son 110, $o = 0$; en otro caso o no cambia.

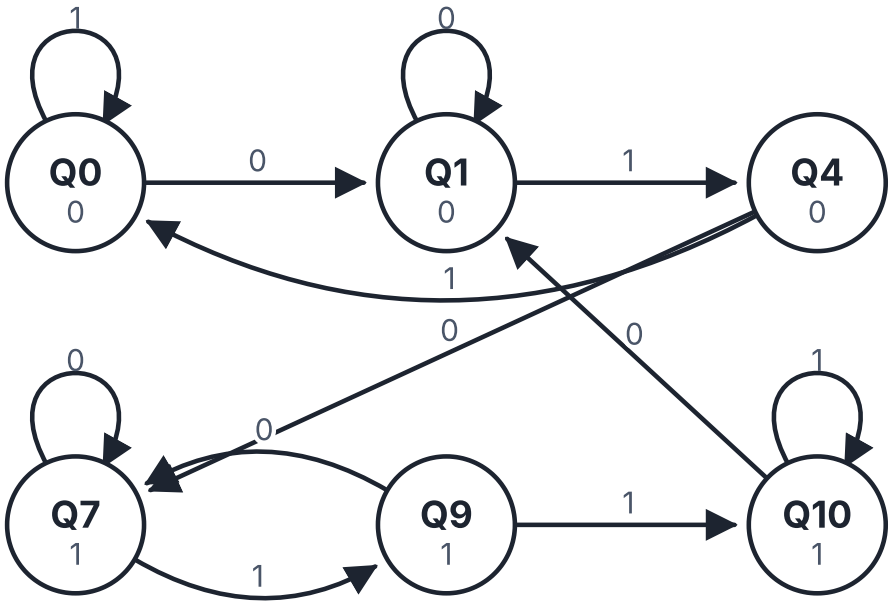
2 Reducción

El modelo tiene 14 estados posibles (2 valores de o por 7 contextos). El algoritmo de equivalencia de estados lo reduce a **6 estados**, que son los de la tabla.

3 Cuidado con el solapamiento

Después de detectar 110, el 0 final **ya es el primer símbolo** de un posible 010 (110 10 → detecta 010). Un autómata que, tras 110, vuelva al estado de «nada recibido» pierde esa detección.

✅ Resultado final



Última secuencia: 010→1, 110→0 (Moore)

Estado	0	1	Salida
Q0	Q1	Q0	0
Q1	Q1	Q4	0
Q4	Q7	Q0	0
Q7	Q7	Q9	1
Q9	Q7	Q10	1
Q10	Q1	Q10	1

Simulados 200 flujos aleatorios contra el modelo de referencia: coinciden. La versión de Mealy se obtiene con la misma tabla pasando la salida a las transiciones.

⚠️ **Errata en el libro**

Según se lee en el diagrama del libro, desde el estado que ya había visto 110 con salida 1 (en la figura Q_5) el cero final lleva a Q_0 («nada»). Con entrada 0 1 0 1 1 0 1 0 el autómata así leído **no** detecta el último 010 (da 0, y debería dar 1), mientras que si ese cero lleva a Q_1 («un 0») sí lo detecta. Ambas versiones se han simulado.

`Libro · Cap. 8 · P8.7`

Detectar 1010 o 110 y reducir estados (Mealy)

✔ verificado

Diseña un autómata de Mealy que detecte las secuencias **1010** y **110** con solapamiento. Después comprueba si se pueden **fusionar estados** y analiza a mano el comportamiento con las entradas 1010, 110 y 110101000110.

🧭 **Antes de empezar: qué piden y qué teoría usas**

El primer diseño es el de los sufijos útiles, y sale con cinco estados. La pregunta clave es si dos estados son **equivalentes**: mismas salidas y, para cada entrada, siguientes equivalentes. Si lo son, se fusionan sin cambiar el comportamiento externo.

1 Cinco estados

Q_0 (ε), Q_1 (1), Q_2 (10), Q_3 (101) y Q_4 (11). En Q_3 y en Q_4 un 0 da salida 1 y lleva a Q_2 (1010 o 110); y con un 1 los dos van a Q_4.

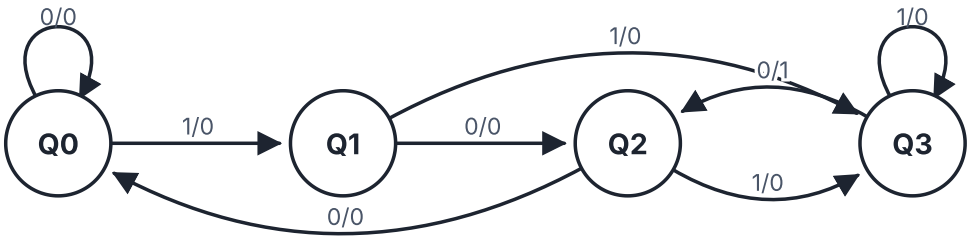
2 Fusión

Q_3 y Q_4 **son equivalentes**: ante un 0 ambos emiten 1 y pasan a Q_2; ante un 1 ambos emiten 0 y pasan a Q_4 (equivalente a sí mismo). El algoritmo de particiones lo confirma. Queda un autómata de **cuatro estados**, con dos biestables.

3 Análisis a mano

Con 1010 el autómata recorre $Q_0 \to Q_1 \to Q_2 \to Q_3 \to Q_2$ y emite 1 en el último paso. Con 110 recorre $Q_0 \to Q_1 \to Q_3 \to Q_2$ y emite 1 también al final. La secuencia larga 110101000110 produce **cuatro** detecciones (en los símbolos 3, 5, 7 y 12).

✅ **Resultado final**



Detector de 1010 o 110 (4 estados)

Estado	0	1
Q0	Q0/0	Q1/0
Q1	Q2/0	Q3/0
Q2	Q0/0	Q3/0
Q3	Q2/1	Q3/0

Función	Ecuación
J_1	q_0
K_1	$\overline{E_0} \cdot \overline{q_0}$
J_0	E_0
K_0	$\overline{E_0}$
S	$\overline{E_0} \cdot q_1 \cdot q_0$

Recorrido del libro, comprobado con la máquina de 4 estados:

Entrada	Estados visitados	Detecciones
1010	Q0 → Q1 → Q2 → Q3 → Q2	1
110	Q0 → Q1 → Q3 → Q2	1
110101000110	Q0 → Q1 → Q3 → Q2 → Q3 → Q2 → Q3 → Q2 → Q0 → Q0 → Q1 → Q3 → Q2	4

Libro · Cap. 8 · P8.8

Detección de errores en código Manchester

✔ verificado

Un mensaje llega por una línea serie E_0 codificado en **Manchester**: cada bit se envía como dos símbolos, el 0 como 01 y el 1 como 10. Diseña un autómata que active una salida ERROR si en algún par llegan dos símbolos iguales (00 u 11). Sólo se sale del error con el RESET asíncrono. Después rehaz el diseño sustituyendo el RESET por un pulsador **P** de inicio/reinicio, que además marca el primer símbolo de cada par.

🧭 Antes de empezar: qué piden y qué teoría usas

Es un detector de secuencias prohibidas (00 y 11) alineado por parejas. El autómata sólo necesita saber en qué mitad del par está y cuál fue el primer símbolo. Una vez en ERROR no hay vuelta atrás: la entrada deja de importar (la «X» de la flecha).

1 Versión con RESET

Seis estados si se quiere saber qué bit llegó (Q0, Q1, Q2, R0, R1, ERROR): R_0 y R_1 significan «acabo de recibir un 0 / un 1». Como la única salida es ERROR, R_0, R_1 y Q_0 son **equivalentes** (mismas transiciones y misma salida): quedan **cuatro estados**.

2 Versión con pulsador P

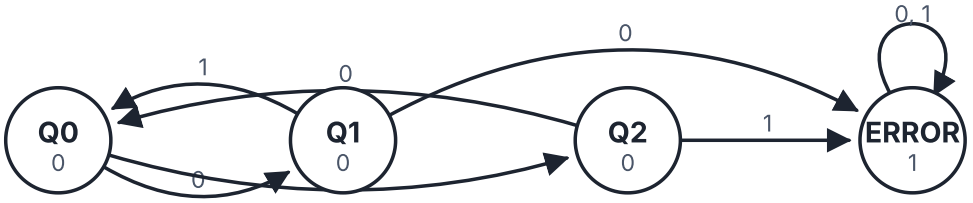
Entradas ordenadas $P\,E_0$. Con $P = 0$ en INICI el autómata espera. Con $P = 1$ la entrada E_0 es el primer símbolo: va a S_0 si es 0 y a S_1 si es 1. En S_0 se espera un 1 (vuelve a INICI) y un 0 es error; en S_1 al revés. Para recibir el segundo símbolo no hace falta mantener P. De ERROR sólo se sale con $P = 1$.

3 Códigos libres y condiciones libres

Con cuatro estados y dos biestables no hay códigos libres. En cambio, con P y E_0 las transiciones tienen muchos «no importa» sobre la entrada (X), que se escriben como una sola flecha para varias combinaciones.

✅ Resultado final

Versión con RESET

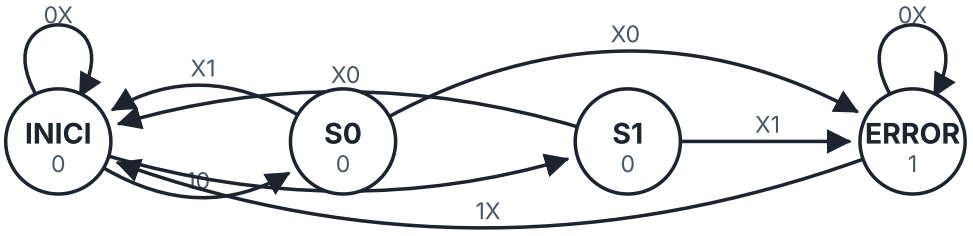


Manchester con RESET (4 estados)

Estado	0	1	Salida
Q0	Q1	Q2	0
Q1	ERROR	Q0	0
Q2	Q0	ERROR	0
ERROR	ERROR	ERROR	1

Q0, R0 y R1 se han fusionado en Q0. Simulados 300 flujos aleatorios contra «hay algún par igual en las posiciones pares» y cien mensajes Manchester correctos (ninguno da error).

Versión con pulsador P (entradas $P\,E_0$)



Manchester con pulsador P

Estado	00	01	10	11	Salida
INICI	INICI	INICI	S0	S1	0
S0	ERROR	INICI	ERROR	INICI	0
S1	INICI	ERROR	INICI	ERROR	0
ERROR	ERROR	ERROR	INICI	INICI	1

Estado	Código $q_1 q_0$	Salida
INICI	00	0
S0	01	0
S1	10	0
ERROR	11	1

Función	Ecuación
J_1	$P \cdot E_0 \cdot \overline{q_0} + \overline{E_0} \cdot q_0$
K_1	$\overline{E_0} \cdot \overline{q_0} + P \cdot q_0$
J_0	$E_0 \cdot q_1 + P \cdot \overline{E_0} \cdot \overline{q_1}$
K_0	$E_0 \cdot \overline{q_1} + P \cdot q_1$
S	$q_1 \cdot q_0$

Las ecuaciones del libro se han comprobado en las 16 transiciones; además se han verificado propiedades por simulación: un mensaje correcto no da error, un par repetido sí, el error permanece con $P = 0$ y se limpia con $P = 1$.

Libro · Cap. 8 · P8.9

Sumador serie con biestable de acarreo (Mealy)

✔ verificado

Dos números llegan **en serie**, bit a bit y empezando por el menos significativo, por las líneas E_1 y E_0. Diseña el autómata que entrega en serie la suma, un bit por ciclo de reloj. Impleméntalo con un biestable J-K y con uno D.

🧭 Antes de empezar: qué piden y qué teoría usas

Sumar en serie es repetir un sumador completo con una memoria de un bit: el acarreo. Basta un estado por cada valor del acarreo (0 ó 1): **dos estados y un solo biestable**. La salida es la suma de los tres bits E_1, E_0 y acarreo, y depende de la entrada actual, por lo que se trata de un Mealy.

1 Estados

C_0: el acarreo pendiente es 0. C_1: el acarreo pendiente es 1.

2 Transiciones y salida

La suma de tres bits $E_1 + E_0 + q$ da salida $S = E_1 \oplus E_0 \oplus q$ y nuevo acarreo $= E_1E_0 + E_1q + E_0q$ (mayoría). Con $q = 0$ sólo hay acarreo si ambos bits son 1. Con $q = 1$ lo hay salvo que ambos sean 0.

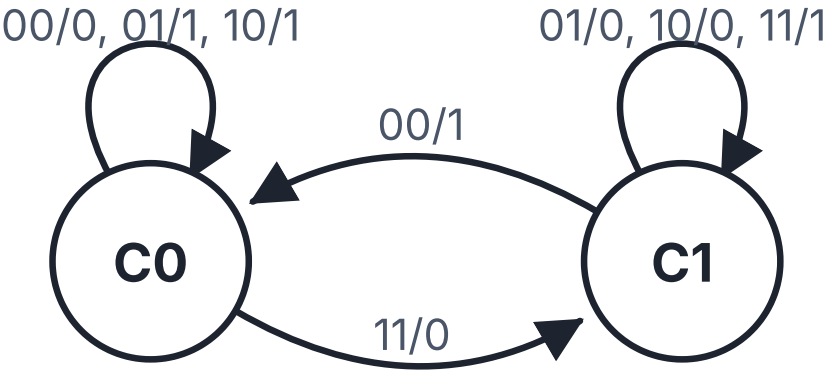
3 Con J-K

El acarreo se pone a 1 cuando $E_1E_0 = 11$ ($J_0 = E_1E_0$) y se pone a 0 cuando $E_1E_0 = 00$ ($K_0 = \overline{E_1}\,\overline{E_0}$); en los demás casos se mantiene.

4 Con D

D_0 es la función mayoría de E_1, E_0 y q_0, la misma de la etapa de acarreo de un sumador completo.

✅ Resultado final



Sumador serie. Etiquetas: E1E0/S

Estado	00	01	10	11
C0	C0/0	C0/1	C0/1	C1/0
C1	C0/1	C1/0	C1/0	C1/1

J-K: $J_0 = E_1E_0$, $K_0 = \overline{E_1}\,\overline{E_0}$. **D:** $D_0 = E_0q_0 + E_1q_0 + E_1E_0$. **Salida:** $S = E_1 \oplus E_0 \oplus q_0$ (3 puertas XOR en cadena de dos).

Ejemplo $13 + 11$ (01101 y 01011, bit menos significativo primero):

Ciclo	E_1	E_0	Acarreo previo	Salida S	Acarreo nuevo
0	1	1	0	0	1
1	0	1	1	0	1
2	1	0	1	0	1
3	1	1	1	1	1
4	0	0	1	1	0
5	0	0	0	0	0

La salida leída de abajo arriba es 11000 = 24, y el ciclo extra con entradas a 0 saca el acarreo final. Comprobado con 300 pares aleatorios de 8 bits.

Libro · Cap. 8 · P8.10

Sumador serie como autómata de Moore

✔ verificado

Rehaz el sumador serie del problema anterior como autómata de **Moore**: la salida debe depender sólo del estado. Los operandos llegan por E_1 y E_0, bit menos significativo primero. Impleméntalo con biestables J-K y analiza si se pueden reducir estados y qué diferencia hay con la versión de Mealy.

🧭 Antes de empezar: qué piden y qué teoría usas

En un Moore la salida es parte del estado, así que el estado tiene que recordar, además del acarreo, el último bit de suma que se calculó. Las combinaciones (acarreo, suma) son cuatro: C_0S_0, C_0S_1, C_1S_0 y C_1S_1.

1 Transiciones

Desde un estado con acarreo c y entradas E_1E_0 se calcula la suma $s = E_1 \oplus E_0 \oplus c$ y el acarreo nuevo $c' = E_1E_0 + E_1c + E_0c$; el estado siguiente es $C_{c'}S_s$. La transición **no depende del bit de suma almacenado**: C_0S_0 y C_0S_1 tienen las mismas filas, y lo mismo C_1S_0 y C_1S_1.

2 ¿Se pueden fusionar?

A pesar de esa simetría **no son equivalentes**: tienen distinta salida (el bit S). Un Moore necesita pues cuatro estados y dos biestables, frente a los dos estados y un biestable del Mealy. El algoritmo de particiones confirma que cuatro es el mínimo.

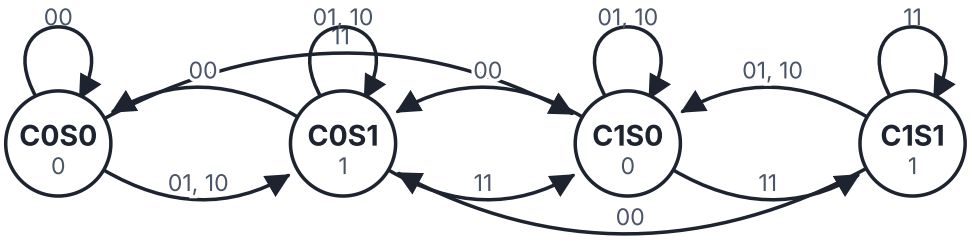
3 Codificación y ecuaciones

Con el código $q_1q_0 = (\text{acarreo}, \text{suma})$ la salida es $S = q_0$. El biestable q_1 hace de acarreo exactamente como en el Mealy ($J_1 = E_1E_0$, $K_1 = \overline{E_1}\,\overline{E_0}$) y q_0 guarda la suma: $J_0 = E_1 \oplus E_0 \oplus q_1$ y K_0 es su negación.

4 Retardo

En un circuito real la suma aparece **un ciclo más tarde** que en el Mealy (se lee en el estado, tras el flanco). Por eso, después de los n bits de los operandos, hace falta un ciclo extra con entradas 00 para sacar el último bit.

✅ Resultado final



Sumador serie de Moore. Cada estado: C (acarreo) y S (suma)

Estado	00	01	10	11	Salida
C0S0	C0S0	C0S1	C0S1	C1S0	0
C0S1	C0S0	C0S1	C0S1	C1S0	1

Estado	00	01	10	11	Salida
C1S0	C0S1	C1S0	C1S0	C1S1	0
C1S1	C0S1	C1S0	C1S0	C1S1	1

Estado	Código $q_1 q_0$	Salida
C0S0	00	0
C0S1	01	1
C1S0	10	0
C1S1	11	1

Función	Ecuación (libro, corregida)
J_1	$E_1 \cdot E_0$
K_1	$\overline{E_1} \cdot \overline{E_0}$
J_0	$E_1 \oplus E_0 \oplus q_1$
K_0	$\overline{E_1} \cdot \overline{E_0} \cdot \overline{q_1} + \overline{E_1} \cdot E_0 \cdot q_1 + E_1 \cdot \overline{E_0} \cdot q_1 + E_1 \cdot E_0 \cdot \overline{q_1}$
S	q_0

Mi propia minimización coincide con ellas como función.

Simulados 300 pares de números de 8 bits: la salida da $a + b$ y es, ciclo a ciclo, idéntica a la del Mealy.

> ⚠️ **Errata en el libro**
>
> En la ecuación de K_0 el primer sumando aparece impreso como $\overline{E_1}\,\overline{E_0}\,q_1$, pero debe ser $\overline{E_1}\,\overline{E_0}\,\overline{q_1}$. $K_0 = \overline{E_1 \oplus E_0 \oplus q_1}$ vale 1 cuando entre E_1, E_0 y q_1 hay un número **par** de unos: para 000 sí, para 001 no. Con el término impreso fallan exactamente dos transiciones, ambas con entrada 00: en C_0S_1 el biestable q_0 no se pone a 0 (debería, pues la suma es 0) y en C_1S_1 sí se pone a 0 (debería quedarse en 1, pues la suma es 1). Se ha comprobado contra la tabla.

`Libro · Cap. 8 · P8.11` **Contador módulo 4 reversible (Moore, 8 estados)**

✔ verificado

Diseña un contador de Moore **módulo 4 reversible**: con $E_0 = 1$ cuenta hacia arriba (0, 1, 2, 3, 0, ...) y con $E_0 = 0$ hacia abajo (3, 2, 1, 0, 3, ...). La salida son dos bits con el valor de la cuenta. Al cambiar el sentido, la nueva secuencia **empieza por su primer valor** (por ejemplo, al pasar de subir a bajar se sale con el 3).

🧭 Antes de empezar: qué piden y qué teoría usas

El valor de la cuenta no basta para decidir el siguiente: si la salida es 2, lo que viene depende de si se estaba subiendo (2→3) o bajando (2→1). Hay que recordar el sentido, y por eso se necesitan **dos copias de cada valor**: ocho estados.

1 Estados

$A_0 \dots A_3$: cuenta ascendente (salida 0...3). $D_3 \dots D_0$: cuenta descendente (salida 3...0). El estado inicial es A_0.

2 Transiciones

Dentro de una secuencia se avanza con el mismo valor de E_0 (en A_k, $E_0 = 1$ da A_{k+1}; en D_k, $E_0 = 0$ da D_{k-1}, con vuelta módulo 4). Al cambiar de sentido: desde cualquier A_k con $E_0 = 0$ se salta a D_3, y desde cualquier D_k con $E_0 = 1$ se salta a A_0.

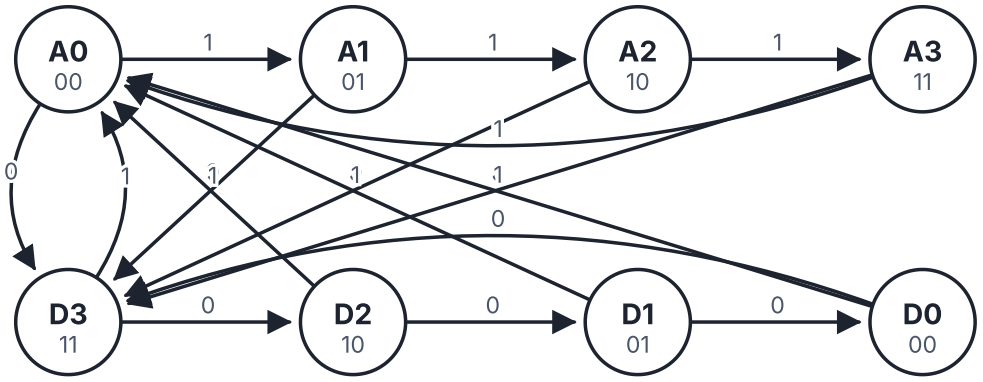
3 ¿Bastarían cuatro estados?

No. El algoritmo de particiones deja los ocho estados separados: por ejemplo A_2 y D_2 tienen la misma salida pero llevan a salidas distintas (3 ó 3 frente a 1 ó 0).

4 Codificación

Se codifica con q_2 = sentido (0 ascendente, 1 descendente) y $q_1 q_0$ = el valor en las ascendentes; en las descendentes se numeran $D_3 = 100$, $D_2 = 101$, $D_1 = 110$, $D_0 = 111$. Con esa elección las salidas son sencillas: $S_1 = q_2 \oplus q_1$, $S_0 = q_2 \oplus q_0$ (una XOR convierte los códigos descendentes en el valor de la cuenta).

✅ Resultado final



Contador reversible mod 4 (Moore). Estado: salida

Estado	0	1	Salida
A0	D3	A1	00
A1	D3	A2	01
A2	D3	A3	10
A3	D3	A0	11
D3	D2	A0	11
D2	D1	A0	10
D1	D0	A0	01
D0	D3	A0	00

Estado	Código $q_2q_1q_0$	Salida
A0	000	00
A1	001	01
A2	010	10
A3	011	11
D3	100	11

Estado	Código $q_2q_1q_0$	Salida
D2	101	10
D1	110	01
D0	111	00

Función	Ecuación
J_2	$\overline{E_0}$
K_2	E_0
J_1	$E_0 \cdot \overline{q_2} \cdot q_0 + \overline{E_0} \cdot q_2 \cdot q_0$
K_1	$\overline{E_0} \cdot \overline{q_2} + q_0 + E_0 \cdot q_2$
J_0	$E_0 \cdot \overline{q_2} + \overline{E_0} \cdot q_2$
K_0	1
S_1	$q_2 \oplus q_1$
S_0	$q_2 \oplus q_0$

Las ecuaciones del libro se han comprobado en las 16 transiciones (los ocho códigos están usados, no hay indiferentes). Simulados 300 flujos aleatorios de 40 entradas contra un modelo que lleva la cuenta y el sentido.

Libro · Cap. 8 · P8.12

Contador módulo 8 con habilitación (Moore)

✔ verificado

Diseña un contador de Moore **módulo 8** con una entrada de habilitación E_0: con $E_0 = 1$ cuenta un valor en cada flanco de reloj y con $E_0 = 0$ se mantiene. La salida son tres bits con el valor de la cuenta.

🧭 Antes de empezar: qué piden y qué teoría usas

Ocho estados, uno por valor, y la salida es el propio código del estado, así que no hace falta lógica de salida. Todo el trabajo está en las funciones de excitación, y con J-K sale una estructura muy regular.

1 Autómata

$Q_n \xrightarrow{E_0=0} Q_n$ y $Q_n \xrightarrow{E_0=1} Q_{(n+1) \bmod 8}$. Es un anillo de ocho estados con un lazo en cada uno.

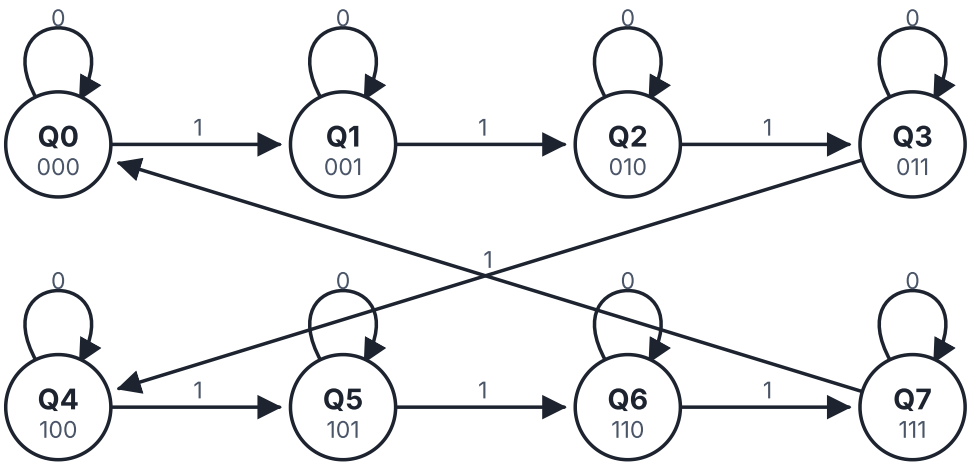
2 Codificación

Se asigna a Q_n el binario de n en $q_2q_1q_0$; así $S_2S_1S_0 = q_2q_1q_0$.

3 Excitación

Un bit cambia de valor cuando el contador está habilitado **y todos los bits inferiores valen 1** (acarreo del incremento). Para ese bit se necesita conmutar: $J = K = 1$; en caso contrario, mantener: $J = K = 0$. De ahí $J_0 = K_0 = E_0$, $J_1 = K_1 = E_0q_0$ y $J_2 = K_2 = E_0q_1q_0$: es un contador con biestables que **conmutan** (tipo T).

✅ Resultado final



Contador módulo 8 con habilitación. Etiqueta: E0

Estado	0	1	Salida
Q0	Q0	Q1	000
Q1	Q1	Q2	001
Q2	Q2	Q3	010
Q3	Q3	Q4	011

Estado	0	1	Salida
Q4	Q4	Q5	100
Q5	Q5	Q6	101
Q6	Q6	Q7	110
Q7	Q7	Q0	111

Función	Ecuación
J_2	$E_0 \cdot q_1 \cdot q_0$
K_2	$E_0 \cdot q_1 \cdot q_0$
J_1	$E_0 \cdot q_0$
K_1	$E_0 \cdot q_0$
J_0	E_0
K_0	E_0
S_2	q_2
S_1	q_1
S_0	q_0

Mi minimización coincide con las del libro como función. Simulados 200 flujos de 50 entradas: la salida es siempre «número de unos recibidos módulo 8». Coste: dos AND de dos y tres entradas (E_0q_0 y $E_0q_1q_0$; la AND de tres puede hacerse en cascada con la anterior).

Libro · Cap. 8 · P8.13

Registro de desplazamiento de 3 bits visto como autómata (Moore)

✔ verificado

Un registro de desplazamiento de tres bits, con entrada serie E_0 y desplazamiento hacia la izquierda, es también un autómata. Diséñalo como autómata de Moore (salida = contenido del registro) con biestables J-K y con biestables D.

🧭 Antes de empezar: qué piden y qué teoría usas

Ejercicio de cambio de punto de vista: el estado es el contenido del registro, así que hay ocho estados Q_n (n = valor binario). Al desplazar a la izquierda el valor se duplica y entra E_0 por la derecha. Ya se sabe cómo debe quedar el circuito; la gracia es que el método general lo reconstruye.

1 Transiciones

$Q_n \xrightarrow{0} Q_{2n \bmod 8}$ y $Q_n \xrightarrow{1} Q_{(2n+1) \bmod 8}$. El bit más significativo se pierde (sale del registro) y el nuevo bit entra por el menos significativo.

2 Con biestables D

$D = Q^+$: el bit q_2 pasa a valer el que tenía q_1, q_1 el de q_0 y q_0 el de la entrada: $D_2 = q_1$, $D_1 = q_0$, $D_0 = E_0$. Son cables directos, sin puertas.

3 Con biestables J-K

El método da $J_2 = q_1$, $K_2 = \overline{q_1}$, $J_1 = q_0$, $K_1 = \overline{q_0}$, $J_0 = E_0$, $K_0 = \overline{E_0}$: cada J-K se comporta como un D (con un inversor entre J y K). Es la forma de convertir un J-K en D.

✅ Resultado final

Estado	0	1	Salida
Q0	Q0	Q1	000
Q1	Q2	Q3	001
Q2	Q4	Q5	010
Q3	Q6	Q7	011
Q4	Q0	Q1	100
Q5	Q2	Q3	101
Q6	Q4	Q5	110
Q7	Q6	Q7	111

J-K

Función	Ecuación
J_2	q_1
K_2	$\overline{q_1}$
J_1	q_0
K_1	$\overline{q_0}$
J_0	E_0
K_0	$\overline{E_0}$
S_2	q_2
S_1	q_1
S_0	q_0

D

Función	Ecuación
D_2	q_1
D_1	q_0
D_0	E_0
S_2	q_2
S_1	q_1
S_0	q_0

Las ecuaciones del libro coinciden con las mías como funciones y reproducen las 16 transiciones. Simulados 200 flujos: la salida es siempre «los tres últimos bits recibidos» (con ceros al principio). El diagrama completo tiene 16 flechas y se omite aquí por ser poco legible.

`Libro · Cap. 8 · P8.14`

Bit de paridad par de un mensaje de 3 bits (Mealy, 7→5 estados)

✔ verificado

Un mensaje de **tres bits** llega en serie por E_0, sin pausas, y se repite sin parar. Diseña un autómata de Mealy que ponga su salida a 0 en los dos primeros bits de cada mensaje y, en el tercero, entregue el **bit de paridad par** (1 si el número de unos del mensaje es impar). Después de cada tercer bit, el autómata vuelve al principio.

🧭 Antes de empezar: qué piden y qué teoría usas

El primer diseño es un árbol de decisión de tres niveles: tras el primer bit (2 casos), tras el segundo (4 casos) y tras el tercero se vuelve a Q_0. Salen siete estados. Pero para calcular la paridad **no importa qué bits llegaron, sólo cuántos unos**, y eso permite reducir.

1 Diseño en árbol (7 estados)

Q_1/Q_2: el primer bit fue 0/1. $Q_3, \ldots, Q_6$: los dos primeros bits fueron 00, 01, 10 y 11. En el tercer bit la salida es el OR exclusivo de los tres. Desde $Q_3 \ldots Q_6$ se vuelve a Q_0.

2 Reducción

El algoritmo de particiones fusiona Q_3 con Q_6 (ambos con paridad par hasta ahora: salida 0 con un 0 y 1 con un 1) y Q_4 con Q_5 (paridad impar). Quedan **cinco estados**: Q_0, dos para «primer bit visto» y dos para «paridad par/impar».

3 Estados del diseño reducido

Q_0 inicio. Q_1 primer bit 1 y Q_2 primer bit 0 (el nombre es de la tabla de abajo). Q_3: paridad par tras dos bits. Q_4: paridad impar tras dos bits. En el tercer bit: Q_3 emite lo que llega y Q_4 lo invierte.

4 Mealy y Moore

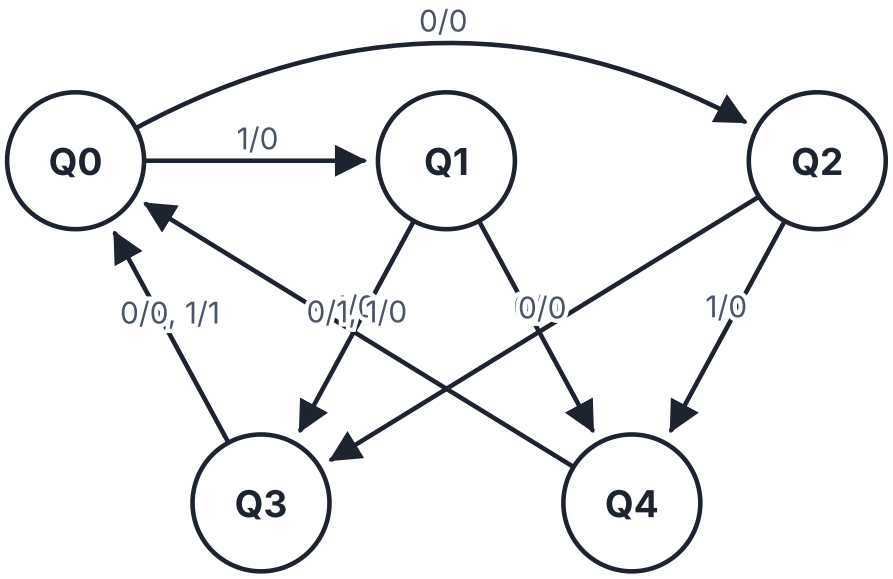
En el Mealy la paridad sale **durante** el tercer ciclo (depende de la entrada actual); en el Moore equivalente sale tras el tercer flanco. Pasar a Moore exige duplicar Q_0 según lleguemos con salida 0 ó 1: **6 estados** y ninguna fusión posible.

✅ Resultado final

Árbol de 7 estados

Estado	0	1
Q0	Q1/0	Q2/0
Q1	Q3/0	Q4/0
Q2	Q5/0	Q6/0
Q3	Q0/0	Q0/1
Q4	Q0/1	Q0/0
Q5	Q0/1	Q0/0
Q6	Q0/0	Q0/1

Reducido a 5 estados (los estados se renombran)



Paridad de 3 bits, 5 estados. E0/S

Estado	0	1
Q0	Q2/0	Q1/0
Q1	Q4/0	Q3/0
Q2	Q3/0	Q4/0

Estado	0	1
Q3	Q0/0	Q0/1
Q4	Q0/1	Q0/0

Estado	Código $q_2 q_1 q_0$
Q0	000
Q1	001
Q2	010
Q3	011
Q4	100

Función	Ecuación
J_2	$\overline{E_0} \cdot \overline{q_1} \cdot q_0 + E_0 \cdot q_1 \cdot \overline{q_0}$
K_2	1
J_1	$\overline{E_0} \cdot \overline{q_2} \cdot \overline{q_0} + E_0 \cdot q_0$
K_1	$q_0 + E_0$
J_0	$E_0 \cdot \overline{q_2} \cdot \overline{q_1} + \overline{E_0} \cdot q_1$
K_0	$\overline{E_0} + q_1$
S	$E_0 \cdot q_1 \cdot q_0 + \overline{E_0} \cdot q_2$

Las ecuaciones del libro se han comprobado en las 10 transiciones (los códigos 101, 110 y 111 son indiferentes). Las dos versiones, la de 7 y la de 5 estados, se han simulado con 200 flujos aleatorios de 60 bits contra el modelo «cada tres bits, el OR exclusivo de los tres». Mi propia minimización coincide con ellas como función.

Libro · Cap. 8 · P8.15

Codificador de prioridad 4:2 serie (Mealy)

✔ verificado

Las cuatro entradas d_0, d_1, d_2, d_3 de un codificador de prioridad 4:2 llegan **en serie** por E_0, en ese orden (primero d_0), repitiéndose en grupos de cuatro bits. Diseña un autómata de Mealy que, al llegar el cuarto bit, entregue por dos salidas S_1S_0 el código de la entrada activa de mayor índice (d_0 no importa: si sólo está activa d_0 o ninguna, sale 00). En los otros ciclos las salidas valen 00. Estudia después si tiene ventaja recibir los bits en orden inverso.

🧭 Antes de empezar: qué piden y qué teoría usas

La prioridad manda sobre el orden de llegada: la entrada de mayor índice llega la última, así que cuando se recibe d_3 ya se conoce todo lo anterior. Cada estado debe resumir «cuál es la entrada activa más alta de las vistas hasta ahora». Con un contador de posición implícito (que sólo avanza) los estados forman un árbol que se reúne en Q_0.

1 Primer bit

d_0 no influye en el resultado: de Q_0 se pasa a Q_1 con cualquier entrada (la «X» de la flecha).

2 Segundo y tercer bit

$Q_1 \to Q_2$ si $d_1 = 0$ y $Q_1 \to Q_3$ si $d_1 = 1$. Después, $d_2 = 1$ lleva a Q_5 («la mayor vista es d_2», que domina a d_1) en cualquiera de los dos casos; con $d_2 = 0$ se llega a Q_4 (ninguna activa: 00) o a Q_6 (sólo d_1: 01).

3 Cuarto bit

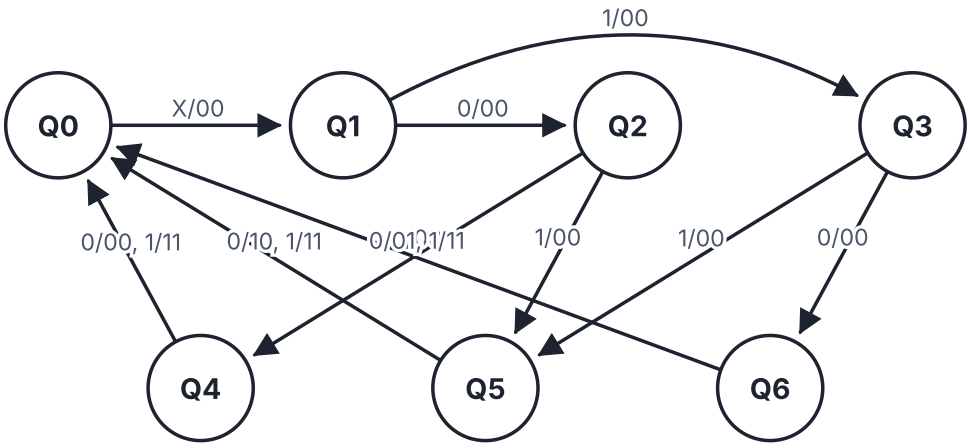
Desde Q_4, Q_5 y Q_6 se vuelve a Q_0 y se emite el código: con $d_3 = 1$ siempre 11; con $d_3 = 0$, 00, 10 ó 01 según el estado.

4 Orden inverso (el más significativo primero)

Se puede, pero **no es más sencillo**: hay que decidir ya con d_3 y seguir contando los bits restantes sin cambiar de opinión, con cadenas de estados «X» hasta el final del grupo. Salen **10 estados** (el más significativo primero, la prioridad no deja ahorrar ninguno); el algoritmo de particiones confirma que son todos distintos.

✅ Resultado final

Orden d_0 primero (7 estados)



Codificador 4:2 serie (d0 primero). E0/S1S0

Estado	0	1
Q0	Q1/00	Q1/00
Q1	Q2/00	Q3/00
Q2	Q4/00	Q5/00
Q3	Q6/00	Q5/00
Q4	Q0/00	Q0/11
Q5	Q0/10	Q0/11
Q6	Q0/01	Q0/11

Estado	Código $q_2q_1q_0$
Q0	000
Q1	001
Q2	010
Q3	011
Q4	100
Q5	101

Estado	Código $q_2q_1q_0$
Q6	110

Función	Ecuación
J_2	q_1
K_2	1
J_1	$\overline{q_2} \cdot q_0$
K_1	$\overline{q_0} + E_0$
J_0	$\overline{q_2} \cdot \overline{q_1} + E_0 \cdot \overline{q_2}$
K_0	$\overline{E_0} + q_2$
S_1	$q_2 \cdot q_0 + E_0 \cdot q_2$
S_0	$q_2 \cdot q_1 + E_0 \cdot q_2$

Las ecuaciones del libro reproducen las 14 transiciones (código 111 indiferente). Simulados 300 flujos de 48 bits contra el modelo de prioridad: coinciden las dos versiones.

Orden inverso (10 estados)

Estado	0	1
Q0	Q2/00	Q1/00
Q1	Q3/00	Q3/00
Q2	Q5/00	Q4/00
Q3	Q6/00	Q6/00
Q4	Q7/00	Q7/00
Q5	Q9/00	Q8/00
Q6	Q0/11	Q0/11
Q7	Q0/10	Q0/10
Q8	Q0/01	Q0/01
Q9	Q0/00	Q0/00

Mi propia minimización coincide con ellas como función.

Libro · Cap. 8 · P8.16

Comparador serie de dos números (Moore)

✔ verificado

Dos números llegan en serie por E_1 (A) y E_0 (B), **empezando por el bit menos significativo**. Diseña un autómata de Moore que indique en tres salidas $S_2S_1S_0$ si, con los bits recibidos hasta el momento, $A > B$, $A = B$ o $A < B$. Impleméntalo con J-K. Después piensa qué cambia si los bits llegan empezando por el más significativo.

🧭 Antes de empezar: qué piden y qué teoría usas

Con el menos significativo primero manda siempre el último bit que difiere: cada bit nuevo es más significativo que todos los anteriores, así que si $E_1 \neq E_0$ decide él; si son iguales, se mantiene el veredicto anterior. Tres estados: IGUAL, MAYOR y MENOR.

1 Transiciones (LSB primero)

En cualquier estado, $E_1E_0 = 10$ lleva a MAYOR y 01 a MENOR (el bit nuevo prevalece). Con 00 ó 11 se queda donde estaba. Desde IGUAL, 00 y 11 mantienen IGUAL.

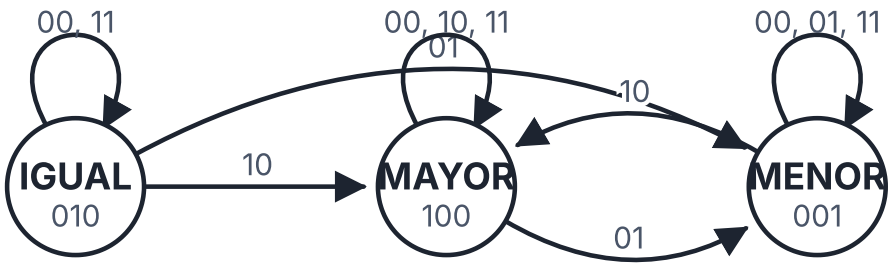
2 Codificación y ecuaciones

IGUAL $= 00$, MAYOR $= 01$, MENOR $= 10$ (el 11 es indiferente). Las salidas son simples: $S_2 = q_0$ (mayor), $S_1 = \overline{q_1}\,\overline{q_0}$ (igual), $S_0 = q_1$ (menor).

3 Orden inverso (MSB primero)

Ahora el **primer bit que difiere** decide para siempre: MAYOR y MENOR se vuelven estados absorbentes (se quedan con cualquier entrada) y sólo IGUAL puede abandonarse. Las ecuaciones se simplifican: $K_1 = K_0 = 0$, $J_1 = \overline{E_1}E_0\overline{q_0}$, $J_0 = E_1\overline{E_0}\,\overline{q_1}$. Esa versión necesita un reinicio para empezar otra comparación.

✅ Resultado final



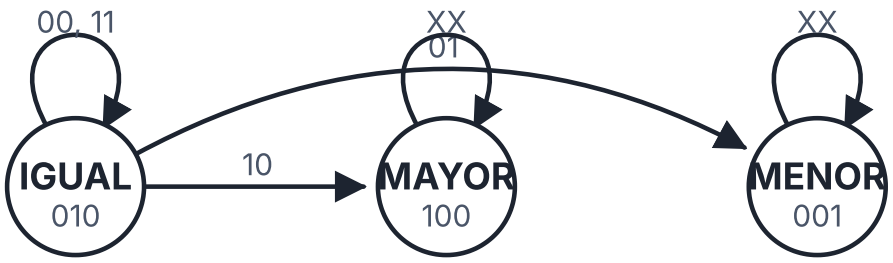
Comparador serie, LSB primero (Moore). Estado: S2S1S0

Estado	00	01	10	11	Salida
IGUAL	IGUAL	MENOR	MAYOR	IGUAL	010
MAYOR	MAYOR	MENOR	MAYOR	MAYOR	100
MENOR	MENOR	MENOR	MAYOR	MENOR	001

Estado	Código $q_1 q_0$	Salida
IGUAL	00	010
MAYOR	01	100
MENOR	10	001

Función	Ecuación
J_1	$\overline{E_1} \cdot E_0$
K_1	$E_1 \cdot \overline{E_0}$
J_0	$E_1 \cdot \overline{E_0}$
K_0	$\overline{E_1} \cdot E_0$
S_2	q_0
S_1	$\overline{q_1} \cdot \overline{q_0}$
S_0	q_1

Versión MSB primero



Comparador serie, MSB primero

Función	Ecuación
J_1	$\overline{E_1} \cdot E_0 \cdot \overline{q_0}$
K_1	0
J_0	$E_1 \cdot \overline{E_0} \cdot \overline{q_1}$
K_0	0
S_2	q_0
S_1	$\overline{q_1} \cdot \overline{q_0}$
S_0	q_1

Simulados 300 pares de 8 bits en los dos órdenes, comprobando la salida **en cada ciclo** (en LSB primero, comparando los bits bajos recibidos; en MSB primero, los bits altos). Ejemplo con $A = 001110_2 = 14$ y $B = 100010_2 = 34$, LSB primero:

Ciclo	E_1E_0	Estado	Salida
1	00	IGUAL	010
2	11	IGUAL	010
3	10	MAYOR	100
4	10	MAYOR	100
5	00	MAYOR	100
6	01	MENOR	001

Tras 5 bits ($A = 01110 = 14$, $B = 00010 = 2$) el resultado es MAYOR; el sexto bit, $A_5 = 0$ y $B_5 = 1$, lo cambia a MENOR, porque B pasa a valer 34.

> **⚠️ Errata en el libro**
>
> En el ejemplo numérico del libro el segundo número, 100010, aparece con valor decimal 18. Es 34 ($32 + 2$). La conclusión ($A < B$) no cambia.

`Libro · Cap. 8 · P8.17`

Puerta automática con pulsador y topes (autómata «industrial»)

✔ verificado

Una puerta se abre al activar un pulsador P. Se abre hasta que el sensor A detecta el tope de apertura; entonces empieza a cerrarse hasta que el sensor C detecta el tope de cierre, y se para. Dos salidas $S_1 S_0$ controlan el motor: 00 para, 01 cierra, 10 abre. Las entradas se ordenan PAC. Diseña el autómata.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Los estados de un automatismo coinciden con los estados del aparato**: QUIETO, ABRIR y CERRAR (infinitivos o gerundios: una buena pista). La salida es el propio estado, así que es un Moore. Además, las entradas son **asíncronas** (las genera una persona o un sensor): antes de llegar al autómata se pasan por biestables D con el mismo reloj, que las sincronizan. El reloj despierta al autómata en cada flanco, y si va rápido es casi como si fuera continuo.

1 Método de las historias

En estos problemas se empieza **contando historias** (secuencias típicas de funcionamiento) y se escribe una tabla paso a paso. Después se completan las transiciones que faltan. Dos consejos del libro: en una historia, dos entradas consecutivas deben diferir en un solo bit (de 010 a 000 a 001, no de 101 a 000), y la transición de llegada a un estado debe **repetirse como lazo** en el estado de destino (si 010 lleva a CERRAR, 010 también mantiene CERRAR).

2 Entradas imposibles

Con tres entradas hay 8 combinaciones por estado, pero A y C no pueden valer 1 a la vez (la puerta no está abierta y cerrada a la vez): se descartan 011 y 111. Quedan **seis** transiciones por estado.

3 Completar los estados

QUIETO: con 001 se queda; con P (100, 101, 110) se pone a abrir; con 000 ó 010 (en reposo pero sin el sensor de cerrado) pasa a cerrar, para dejar la puerta bien cerrada. ABRIR: con 001 sigue abriendo (extraño pero razonable); con 010 empieza a cerrar (tope de apertura); con 110 (pulsan y ya está abierta) también cierra. CERRAR: con 001 y con 101 llega al tope y queda quieto; con 100 (pulsan mientras cierra) vuelve a abrir; en el resto sigue cerrando.

✅ **Resultado final**

Estado	000	001	010	011	100	101	110	111
QUIETO	CERRAR	QUIETO	CERRAR	×	ABRIR	ABRIR	ABRIR	×
ABRIR	ABRIR	ABRIR	CERRAR	×	ABRIR	ABRIR	CERRAR	×
CERRAR	CERRAR	QUIETO	CERRAR	×	ABRIR	QUIETO	CERRAR	×

Historia 1 (apertura y cierre completos)

Paso	Entrada PAC	Estado	Salida S_1S_0
1	001	QUIETO	00
2	101	ABRIR	10
3	100	ABRIR	10
4	000	ABRIR	10
5	010	CERRAR	01
6	000	CERRAR	01
7	001	QUIETO	00

Historia 2 (se pulsa P mientras se cierra: vuelve a abrir)

Paso	Entrada PAC	Estado	Salida S_1S_0
1	001	QUIETO	00
2	101	ABRIR	10
3	100	ABRIR	10
4	000	ABRIR	10

Paso	Entrada PAC	Estado	Salida $S_1 S_0$
5	010	CERRAR	01
6	000	CERRAR	01
7	100	ABRIR	10
8	000	ABRIR	10
9	010	CERRAR	01
10	000	CERRAR	01
11	001	QUIETO	00

Estado	Código $q_1 q_0$	Salida
QUIETO	00	00
ABRIR	10	10
CERRAR	01	01

Función	Ecuación (mía, mínima)
J_1	$P \cdot \overline{A} \cdot \overline{C} + P \cdot \overline{q_0}$
K_1	A
J_0	$\overline{P} \cdot \overline{C} \cdot \overline{q_1} + A \cdot q_1$
K_0	$C + P \cdot \overline{A}$
S_1	q_1
S_0	q_0

Las dos historias del libro se han reproducido exactamente con la tabla completada, y 300 ciclos aleatorios (abrir, esperar, cerrar, esperar) terminan siempre en QUIETO. El libro no desarrolla las ecuaciones de este ejemplo, que aquí se derivan por el método general, con el código 11 y las entradas imposibles como indiferentes; los resultados se han comprobado contra la tabla.

Libro · Cap. 8 · P8.18

Limpiaparabrisas con pulsadores de marcha y paro

✔ verificado

Un limpiaparabrisas tiene un pulsador de marcha M y otro de paro P. Al pulsar M se pone en marcha y sigue funcionando al soltarlo; al pulsar P se para. Si se pulsan los dos a la vez, **prevalece el paro**. Diseña el autómata con un biestable J-K.

🧭 Antes de empezar: qué piden y qué teoría usas

Dos estados, PARO y MARCHA, y la salida es el estado: el motor está encendido sólo en MARCHA. La prioridad del paro se refleja en la fila 11 (los dos pulsadores): lleva a PARO desde cualquier estado.

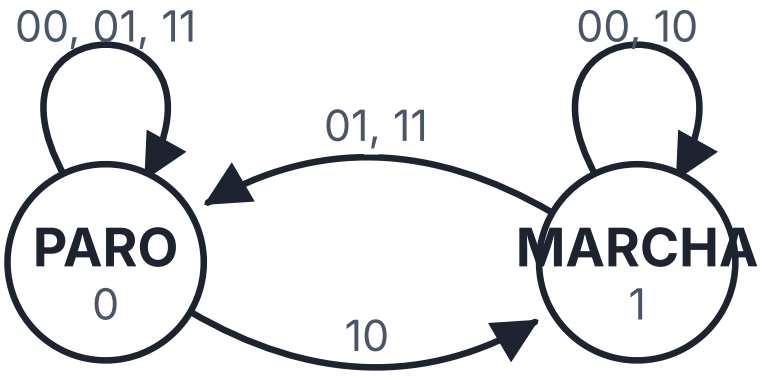
1 Tabla de transición

Entradas ordenadas MP. En PARO: sólo 10 lleva a MARCHA. En MARCHA: sólo 01 y 11 llevan a PARO. En todo lo demás se mantiene el estado.

2 Ecuaciones

Con PARO $= 0$ y MARCHA $= 1$: J_0 debe valer 1 sólo desde PARO con $M = 1, P = 0$, y K_0 vale 1 siempre que $P = 1$ (el paro manda). Resultado: $J_0 = M\overline{P}$, $K_0 = P$, $S_0 = q_0$.

✅ Resultado final



Limpiaparabrisas (entradas MP)

Estado	00	01	10	11	Salida
PARO	PARO	PARO	MARCHA	PARO	0
MARCHA	MARCHA	PARO	MARCHA	PARO	1

Función	Ecuación
J_0	$M \cdot \overline{P}$
K_0	P
S	q_0

Comprobadas las ecuaciones del libro en las ocho transiciones, y simulados 200 flujos aleatorios de 40 pares MP contra «el estado es PARO si P, si no MARCHA si M, si no el anterior».

⚠️ Errata en el libro

El enunciado del libro pide un autómata de Mealy, pero la solución que se desarrolla tiene la salida en el estado y se rotula «resultados del autómata de Moore». Aquí se presenta como Moore, que es lo que de verdad se diseña (la salida es q_0 y no depende de las entradas).

`Libro · Cap. 8 · P8.19`

Limpiaparabrisas con detector de posición de reposo (3 estados)

✔ verificado

Se mejora el limpiaparabrisas anterior con un detector de reposo R (vale 1 cuando la escobilla está en su posición inicial). Entradas ordenadas MPR. Al pulsar P la escobilla **no se detiene donde esté**, sino que termina el recorrido y se para al llegar al reposo. Diseña el autómata con salida de Moore.

🧭 Antes de empezar: qué piden y qué teoría usas

Ahora aparece un estado transitorio: MAPA («marcha para paro»), en el que la escobilla sigue moviéndose pero ya se ha ordenado parar. MARCHA y PARO son estacionarios; MAPA dura hasta que $R = 1$. El motor está activo en MARCHA y en MAPA.

1 **Por qué no bastan dos estados**

Con la entrada 001 (reposo, nada pulsado) la escobilla debe **seguir** si estaba en MARCHA (pasa por el reposo en cada ciclo) pero debe **parar** si estaba en MAPA (acaba de llegar al reposo). La misma entrada pide acciones distintas: hace falta memoria, o sea un tercer estado. El algoritmo de particiones lo confirma: MARCHA y MAPA tienen la misma salida pero no son equivalentes.

2 Transiciones

MARCHA: P con $R = 0$ (010) pasa a MAPA; P con $R = 1$ (011) a PARO; el resto sigue en MARCHA. MAPA: llegar al reposo sin M (001, 011) lleva a PARO; si se vuelve a pulsar M (100, 101) vuelve a MARCHA; si no, sigue. PARO: M lleva a MARCHA; si la escobilla no está en reposo (000, 010) pasa a MAPA para volver a él; si lo está, se queda. Las combinaciones 110 y 111 (M y P a la vez) se consideran imposibles.

3 Codificación

PARO $= 00$, MARCHA $= 01$, MAPA $= 11$ (el 10 es indiferente). Con esa elección la salida es $S = q_0$.

✅ Resultado final

Estado	000	001	010	011	100	101	110	111
PARO	MAPA	PARO	MAPA	PARO	MARCHA	MARCHA	×	×
MARCHA	MARCHA	MARCHA	MAPA	PARO	MARCHA	MARCHA	×	×
MAPA	MAPA	PARO	MAPA	PARO	MARCHA	MARCHA	×	×

Estado	Código $q_1 q_0$	Salida
PARO	00	0
MARCHA	01	1
MAPA	11	1

Función	Ecuación (mía, mínima)
J_1	$\overline{M} \cdot \overline{R} \cdot \overline{q_0} + P \cdot \overline{R}$
K_1	$R + M$

Función	Ecuación (mía, mínima)
J_0	$\overline{R} + M$
K_0	$\overline{M} \cdot R \cdot q_1 + P \cdot R$
S	q_0

Historia izquierda (marcha, pasa por el reposo sin parar, se pulsa P y se detiene al llegar al reposo)

Paso	Entrada MPR	Estado	Salida
1	001	PARO	0
2	101	MARCHA	1
3	100	MARCHA	1
4	000	MARCHA	1
5	001	MARCHA	1
6	000	MARCHA	1
7	010	MAPA	1
8	000	MAPA	1
9	001	PARO	0

Historia derecha (marcha y paro pulsando P hasta el reposo)

Paso	Entrada MPR	Estado	Salida
1	100	MARCHA	1
2	000	MARCHA	1
3	010	MAPA	1
4	011	PARO	0

Simulados 300 casos aleatorios: sin pulsar P la escobilla no se detiene nunca, y tras pulsar P siempre acaba parada al llegar al reposo. El libro no incluye ecuaciones de este ejemplo; las de arriba son las mínimas que salen del método y se han comprobado contra la tabla.

> **⚠️ Errata en el libro**
>
> En la historia de la izquierda, el paso 5 («tope») aparece con la entrada 010; debe ser 001: la escobilla **pasa por el reposo sin parar**, y con 010 iría a MAPA, lo que choca con el paso 7. En la de la derecha, «seguir pulsando P» y «reposo con P pulsado» aparecen como 100 y 110; con el orden MPR que usa el libro son 010 y 011. Por último, el estado del paso 9 se rotula QUIETO y es PARO.

`Libro · Cap. 8 · P8.20`

Motor que se activa al soltar el pulsador (Moore, 4 estados)

✔ verificado

Un motor se controla con un pulsador de marcha M y otro de paro P, pero la orden **sólo se ejecuta al soltar** el pulsador (en el paso de 1 a 0). Diseña el autómata. Después hazlo con un **único pulsador** que alterne marcha y paro, también al soltar.

> **🧭 Antes de empezar: qué piden y qué teoría usas**
>
> **Hay que distinguir «pulsado» de «ya soltado».** Con la orden al soltar, cada pulsación produce dos situaciones distintas: mientras se mantiene el pulsador (el motor aún no ha cambiado) y después de soltarlo (ya cambió). Cuatro estados: P (parado), PM (parado, M pulsado y pendiente de soltar), M (en marcha) y MP (en marcha, P pulsado y pendiente).

1 Transiciones

P: con M pulsado (10) pasa a PM; con P (01) no hace nada. PM: mientras M siga pulsado se queda; al soltarlo (00) pasa a M. M: con P (01) pasa a MP; MP: al soltar P (00) vuelve a P. Dos combinaciones, 11 y las que supondrían soltar uno y pulsar el otro en el mismo ciclo, no pueden ocurrir (indiferentes).

2 Salida

El motor está activo en M y en MP, y parado en P y PM. Con el código P $= 00$, PM $= 01$, M $= 10$, MP $= 11$ la salida es sencillamente $S_0 = q_1$.

3 Un solo pulsador

Con el mismo cuarteto de estados, pero con una entrada: PM y MP **se alternan** cada vez que se pulsa. La tabla codificada es la de un contador que cambia con el flanco de bajada, y las

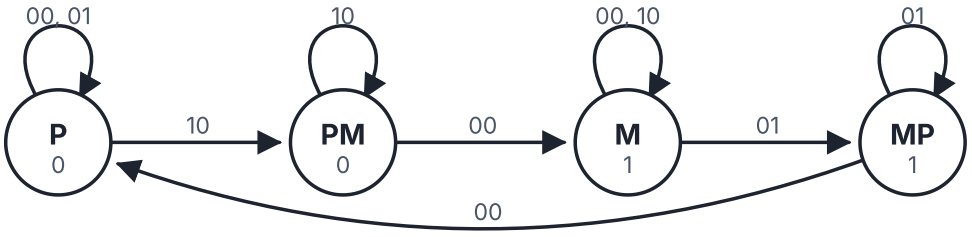
ecuaciones tienen $J_1 = K_1$ (el bit alto conmuta).

4 ¿Y con dos estados?

Con sólo «parado/marcha», la entrada 1 debería llevar de un estado al otro; pero entonces, mientras el pulsador siga a 1, el estado **conmutaría en cada flanco de reloj** (oscilaría). Hace falta recordar que «esta pulsación ya se ha contado», y eso son los otros dos estados.

✅ Resultado final

Dos pulsadores (entradas MP; «—» = combinación imposible)



Motor con dos pulsadores. Estado: salida

Estado	00	01	10	11	Salida
P	P	P	PM	×	0
PM	M	—	PM	×	0
M	M	MP	M	×	1
MP	P	MP	—	×	1

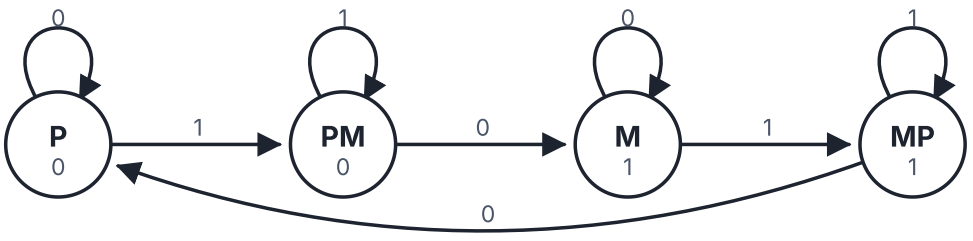
Estado	Código $q_1 q_0$	Salida
P	00	0
PM	01	0
M	10	1
MP	11	1

Función	Ecuación
J_1	$\overline{M} \cdot q_0$
K_1	$\overline{P} \cdot q_0$
J_0	$P \cdot q_1 + M \cdot \overline{q_1}$
K_0	$\overline{M} \cdot \overline{P}$
S	q_1

Historia completa (reposo, pulso de M, soltar, pulso de P, soltar):

Paso	Entrada MP	Estado	Salida
1	00	P	0
2	10	PM	0
3	00	M	1
4	01	MP	1
5	00	P	0

Un solo pulsador (entrada P)



Motor con un pulsador (alterna al soltar)

Estado	0	1	Salida
P	P	PM	0
PM	M	PM	0
M	M	MP	1
MP	P	MP	1

Estado	Código $q_1 q_0$	Salida
P	00	0
PM	01	0
M	10	1
MP	11	1

Función	Ecuación
J_1	$\overline{P} \cdot q_0$
K_1	$\overline{P} \cdot q_0$
J_0	P
K_0	$\overline{P}$
S	q_1

Historia:

Paso	Entrada P	Estado	Salida
1	0	P	0
2	1	PM	0
3	0	M	1
4	1	MP	1
5	0	P	0

Las ecuaciones del libro se han comprobado en todas las transiciones definidas de las dos versiones.

⚠️ Errata en el libro

En la tabla de historias del problema con un solo pulsador, el paso 4 anota el estado MP con salida 0 y el paso 5 anota la entrada «soltar» como 1. El estado MP tiene salida 1 (el motor sigue en marcha mientras se mantiene el pulsador) y «soltar» es la entrada 0. La tabla de arriba es la correcta y reproduce el funcionamiento.

Libro · Cap. 8 · P8.21

Depósito con dos bombas que se turnan

✔ verificado

Un depósito se llena con dos bombas, B_1 y B_2, y tiene dos sensores de nivel: I (inferior) y S (superior), con las entradas ordenadas IS (la combinación 01 es imposible: nivel alto sin nivel bajo). Con el nivel por encima de S las bombas están paradas. Al bajar de S se pone en marcha **una** bomba; si el nivel sigue bajando hasta quedar por debajo de I, se conectan **las dos**. Las bombas deben **turnarse** para desgastarse por igual. Diseña el autómata.

🧭 Antes de empezar: qué piden y qué teoría usas

Para turnarse hay que recordar cuál fue la última en trabajar. Eso duplica los estados «paradas» y «las dos en marcha»: hay que saber qué bomba toca después. Salen seis estados de Moore.

1 Estados

UB1 y UB2: bombas paradas con el depósito lleno; la siguiente bomba que se use será B_1 o B_2. B1 y B2: sólo trabaja esa bomba. B12a y B12b: trabajan las dos; hay dos copias porque al subir el nivel se debe continuar con la bomba **distinta** de la última usada.

2 Funcionamiento

De UB1 con $IS = 10$ (nivel intermedio) se pasa a B1; desde B1, al llenarse (11), se pasa a UB2, de modo que la próxima vez toque B_2. Si con B1 el nivel baja de I (00), se pasa a B12a; al volver a 10 sigue sólo B_2 (la que descansó) y al llenarse se vuelve a UB1.

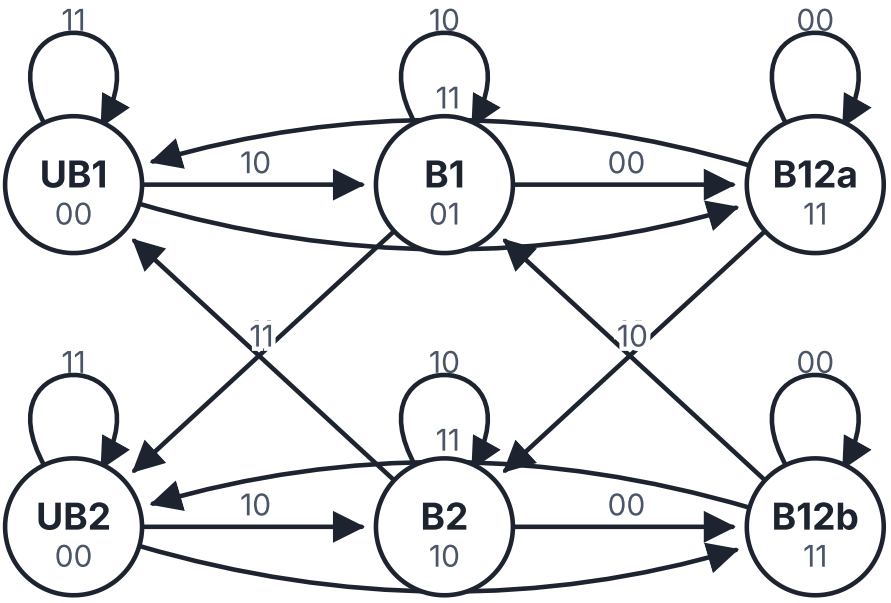
3 Salidas

Las dos salidas B_2B_1 son 00 en UB1/UB2, 01 en B1, 10 en B2 y 11 en B12a/B12b.

4 ¿Bastan cuatro estados?

No. Con sólo «paradas», «B_1», «B_2» y «las dos» no se sabe cuál fue la última bomba en trabajar, y no se podría asegurar el turno. El algoritmo de particiones separa los seis estados.

✅ Resultado final



Depósito con dos bombas (entradas IS). Estado: B2B1

Estado	00	01	10	11	Salida
UB1	B12a	×	B1	UB1	00
B1	B12a	×	B1	UB2	01
B12a	B12a	×	B2	UB1	11
UB2	B12b	×	B2	UB2	00
B2	B12b	×	B2	UB1	10
B12b	B12b	×	B1	UB2	11

Estado	Código $q_2q_1q_0$	Salida
UB1	000	00
B1	001	01
B12a	011	11
UB2	100	00

Estado	Código $q_2q_1q_0$	Salida
B2	010	10
B12b	111	11

Función	Ecuación (mía, mínima)
J_2	$\overline{I}\cdot q_1\cdot\overline{q_0}+S\cdot\overline{q_1}\cdot q_0$
K_2	$I\cdot\overline{S}$
J_1	$\overline{I}+\overline{S}\cdot q_2$
K_1	$S+I\cdot q_2$
J_0	$\overline{I}+\overline{S}\cdot\overline{q_2}\cdot\overline{q_1}$
K_0	$S+I\cdot\overline{q_2}\cdot q_1$
S_1	q_1
S_0	q_0

Se reproducen las dos historias del libro: (izquierda) UB1, B1, UB2, UB2, B2, B12b, B1, UB2; (derecha) B1, B12a, B2, B12b, B1, UB2. Además, se ha simulado el depósito con un nivel que sube y baja de uno en uno, durante 200 trayectorias de 80 pasos, y se verifica que (i) las dos bombas sólo trabajan juntas con el nivel por debajo de I, (ii) están paradas sólo con el nivel lleno y (iii) la bomba que se usa al bajar de S es siempre distinta de la última que trabajó sola. El libro desarrolla este ejemplo con el diagrama; las ecuaciones de aquí son las del método.

Libro · Cap. 8 · P8.22

Puertas de cristal de un edificio (Moore, entradas PCA)

✔ verificado

Una puerta de cristal tiene un detector de persona P y dos topes: abierta A y cerrada C (entradas ordenadas PCA). Debe cumplir: si llega una persona, se abre; si ha empezado a abrirse, **completa la apertura**; mientras haya gente, **permanece abierta**; si se está cerrando y aparece una persona, **vuelve a abrirse**. Sale una orden de dos bits (cerrando, abriendo).

🧭 Antes de empezar: qué piden y qué teoría usas

Cuatro estados de la puerta: CDA (cerrada, parada), ABDO (abriéndose), ABTA (abierta, parada) y CDO (cerrándose). Las salidas valen 00, 01, 00 y 10: CDA y ABTA son estacionarios y no se mueve el motor. $C = A = 1$ es imposible (011 y 111 se descartan).

1 Lo que dice el enunciado

CDA con persona (100, 110) → ABDO. ABDO se mantiene con cualquier combinación sin A (completa la apertura) y pasa a ABTA con $A = 1$ y gente (101) o a CDO con $A = 1$ sin gente (001). ABTA con gente y abierta (101) se queda; sin gente (001) pasa a CDO. CDO termina en CDA al llegar a C (010); con persona vuelve a ABDO (100, 110) o a ABTA (101).

2 Lo que hay que completar

El dibujo no cubre todas las combinaciones. Las completo así: en CDA, sin C (000) o con A (001), la puerta no está donde debe: pasa a CDO (cerrar) como en el problema 8.17; en ABTA, si A cae a 0 con gente (100, 110) se reabre (ABDO) y sin gente (000, 010) se cierra (CDO); CDO con 110 reabre (ABDO).

✅ Resultado final

Estado	000	001	010	011	100	101	110	111	Salida
CDA	CDO	CDO	CDA	×	ABDO	ABTA	ABDO	×	00
ABDO	ABDO	CDO	ABDO	×	ABDO	ABTA	ABDO	×	01
ABTA	CDO	CDO	CDO	×	ABDO	ABTA	ABDO	×	00
CDO	CDO	CDO	CDA	×	ABDO	ABTA	ABDO	×	10

Estado	Código $q_1 q_0$	Salida
CDA	00	00
ABDO	01	01
ABTA	11	00
CDO	10	10

Función	Ecuación (mía, mínima)
J_1	$\overline{P} \cdot \overline{C} \cdot \overline{q_0} + A$
K_1	$C \cdot \overline{q_0} + P \cdot \overline{A}$
J_0	P
K_0	$\overline{P} \cdot q_1 + \overline{P} \cdot A$
S_1	$q_1 \cdot \overline{q_0}$
S_0	$\overline{q_1} \cdot q_0$

Historias

Historia	Entradas PCA (desde el estado inicial)	Estados
Ciclo completo	010, 110, 100, 000, 001, 000, 010	CDA → ABDO → ABDO → ABDO → CDO → CDO → CDA
Con gente al final	100, 100, 101, 001, 000, 010	ABDO → ABDO → ABTA → CDO → CDO → CDA
Alguien aparece al cerrar	100, 101, 001, 100, 101, 001, 010	ABDO → ABTA → CDO → ABDO → ABTA → CDO → CDA

El libro presenta este ejemplo sólo con el diagrama de estados y sus comentarios (sin ecuaciones); la tabla completa y las ecuaciones son las del método, y se han comprobado frente a las historias y a tres propiedades simuladas (abriendo no se detiene, cerrando con persona reabre, abierta con gente permanece abierta).

`Libro · Cap. 8 · P8.23`

Desvío de trenes: un tren corto o uno largo

✔ verificado

En una vía hay un desvío con dos ramales: A (corto) y B (largo). Dos sensores, E_1 a la entrada y E_0 a la salida del tramo de aguja (entradas ordenadas E_1E_0), dicen por dónde va el tren: si **sólo** E_1 está activo y luego se libera, el tren es corto y pasa por A; si E_1 **y** E_0 **están activos a la vez**, el tren es largo y debe ir por B. La salida AB posiciona la aguja: 10 es el ramal A y 01 el B. Por defecto, la aguja está en A.

🧭 Antes de empezar: qué piden y qué teoría usas

Dos soluciones: la «natural» y la mínima. La primera sigue el recorrido de un tren y tiene cinco estados; pero en realidad lo único que importa es **dónde está la aguja**, y eso sólo necesita saber si hay un tren largo cruzando. Esa idea da dos estados y un único biestable.

1 Primera solución: cinco estados

NADA (sin trenes), ENTRA (el tren acaba de entrar), VIAA (tren corto, sale por A), SALEA (saliendo) y VIAB (tren largo). Las dos historias del libro se recorren como sigue: corto, NADA→ENTRA→VIAA→SALEA→NADA; largo, NADA→ENTRA→VIAB→VIAB→NADA. Los saltos no previstos («—») no pueden ocurrir físicamente y son indiferentes.

2 Segunda solución: dos estados

VIAA (salida 10) y VIAB (01). Sólo se pasa a VIAB cuando se ven **los dos sensores a la vez** (11); se vuelve a VIAA cuando ya no hay tren (00). Con un biestable q_0 (0 = VIAA, 1 = VIAB): $J_0 = E_1 E_0$ y $K_0 = \overline{E_0}$, y las salidas $A = \overline{q_0}$, $B = q_0$.

3 Equivalencia

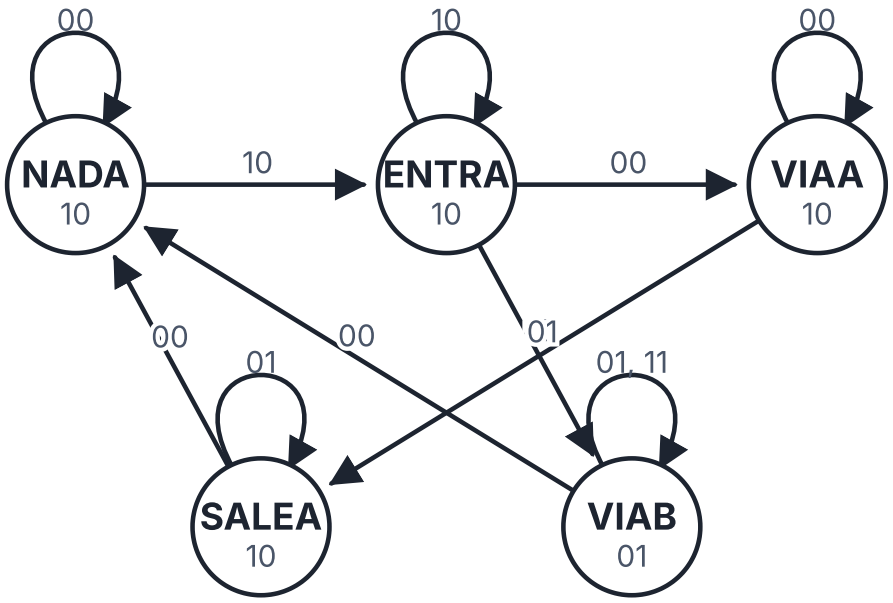
Para todos los pasos de trenes posibles (cortos y largos, con pausas) la aguja se coloca **exactamente igual** en las dos soluciones: se han comparado 300 secuencias aleatorias de hasta cinco trenes.

4 Una pega del diseño

Tras pasar un tren largo, en cuanto los dos sensores quedan libres la aguja vuelve a A, aunque el tren aún pudiera estar saliendo del ramal. El libro sugiere dejar la salida en B un poco más; es un ajuste de diseño, no un fallo.

✅ Resultado final

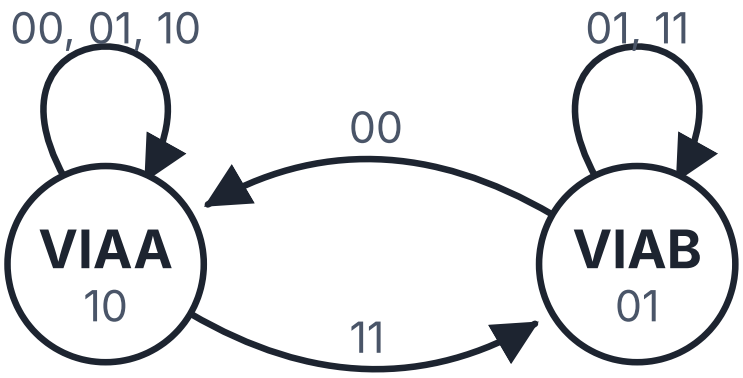
Cinco estados (« — »: no puede ocurrir)



Desvío: cinco estados. Estado: salida AB

Estado	00	01	10	11	Salida
NADA	NADA	—	ENTRA	—	10
ENTRA	VIAA	—	ENTRA	VIAB	10
VIAA	VIAA	SALEA	—	—	10
SALEA	NADA	SALEA	—	—	10
VIAB	NADA	VIAB	—	VIAB	01

Dos estados



Desvío: dos estados

Estado	00	01	10	11	Salida
VIAA	VIAA	VIAA	VIAA	VIAB	10
VIAB	VIAA	VIAB	—	VIAB	01

Función	Ecuación
J_0	$E_1 \cdot E_0$
K_0	$\overline{E_0}$
S_1	$\overline{q_0}$
S_0	q_0

En ambas el comportamiento del libro (historias de arriba) se reproduce.

⚠️ Errata en el libro

Las ecuaciones de salida impresas en el libro son $A = q_0$ y $B = \overline{q_0}$. Eso contradice la propia codificación (VIAA $= 0$ debe dar $AB = 10$, es decir $A = \overline{q_0}$ y $B = q_0$). Las ecuaciones de excitación J_0 y K_0 son correctas; las salidas están intercambiadas. Con las salidas impresas, el sistema colocaría la aguja en B cuando no hay tren.

Libro · Cap. 8 · P8.24

Secador de manos con pulsador y detector de presencia

✔ verificado

Un secador de manos arranca cuando se pulsa un pulsador P **y** el detector de presencia D ve unas manos, y sigue funcionando mientras D siga detectando. Entradas ordenadas PD. Diseña el autómata con un biestable J-K.

🧭 Antes de empezar: qué piden y qué teoría usas

Dos estados, PARO y MARCHA, y el motor es el estado. La condición para arrancar es un AND ($P \cdot D$) y la de seguir es sólo D: el pulsador deja de importar una vez en marcha.

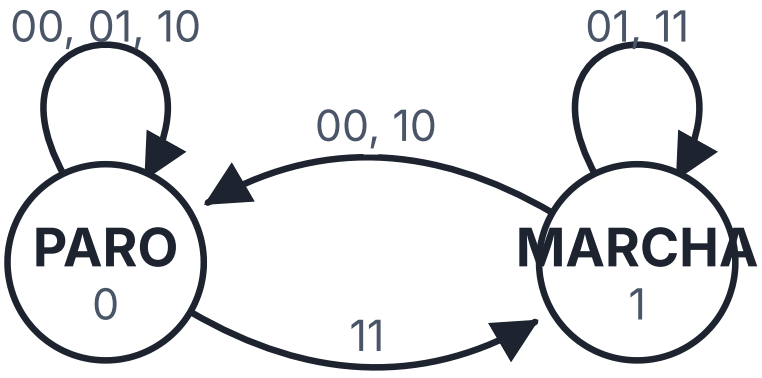
1 Tabla

En PARO sólo $PD = 11$ lleva a MARCHA. En MARCHA se continúa con $D = 1$ (01 y 11) y se para con $D = 0$ (00 y 10), aunque se esté pulsando.

2 Ecuaciones

$J_0 = P \cdot D$ (arranque), $K_0 = \overline{D}$ (se apaga al dejar de detectar), $S_0 = q_0$.

✅ Resultado final



Secador (entradas PD)

Estado	00	01	10	11	Salida
PARO	PARO	PARO	PARO	MARCHA	0
MARCHA	PARO	MARCHA	PARO	MARCHA	1

Función	Ecuación
J_0	$P \cdot D$
K_0	$\overline{D}$
S	q_0

Las dos historias del libro (arranque y parada normales; salir sin soltar el pulsador) se reproducen. Simulados 200 flujos de 40 pares PD contra «el estado siguiente es $D \cdot (P + \text{estado})$».

Libro · Cap. 8 · P8.25

Contador módulo 4 que cuenta al pulsar (8 estados)

✔ verificado

Un contador módulo 4 debe avanzar **un solo paso cada vez que se pulsa** un pulsador E_0 (en el flanco de subida de la pulsación), por mucho que dure la pulsación. La salida son dos bits con la cuenta. Diseña el autómata de Moore.

🧭 Antes de empezar: qué piden y qué teoría usas

Un pulsador humano dura muchos ciclos de reloj, así que hay que distinguir «he contado ya esta pulsación» de «aún no». Por cada valor de la cuenta hay un estado estacionario C_n (pulsador suelto) y otro transitorio C_nT (pulsador mantenido tras haber contado). Son ocho estados.

1 Transiciones

$C_n \xrightarrow{0} C_n$; $C_n \xrightarrow{1} C_{n+1}T$ (cuenta al pulsar y pasa a esperar). $C_nT \xrightarrow{1} C_nT$ (mantiene) y $C_nT \xrightarrow{0} C_n$ (se suelta).

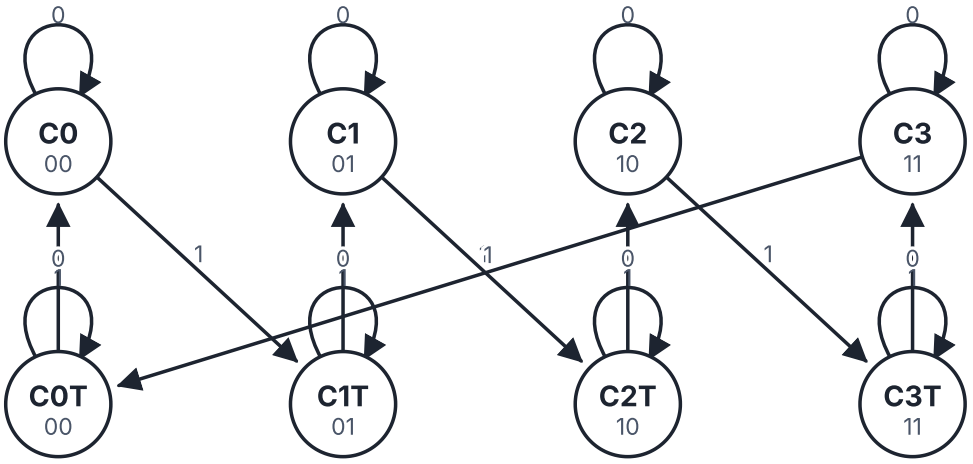
2 Salida

Tanto C_n como C_nT dan el valor n.

3 Codificación y ecuaciones

Los estacionarios son 000 a 011 y los transitorios 100 a 111 (q_2 = «ya contado»). $J_2 = E_0$, $K_2 = \overline{E_0}$ (es una copia retrasada del pulsador). Los otros dos biestables **conmutan** sólo en el instante de la pulsación, es decir cuando $E_0 = 1$ y $q_2 = 0$: $J_0 = K_0 = E_0\overline{q_2}$ y $J_1 = K_1 = E_0\overline{q_2}q_0$ (el bit alto cambia si el bajo es 1).

✅ Resultado final



Contador con pulsador. Etiqueta: E0; estado: cuenta

Estado	0	1	Salida
C0	C0	C1T	00
C1	C1	C2T	01
C2	C2	C3T	10
C3	C3	C0T	11
C0T	C0	C0T	00
C1T	C1	C1T	01
C2T	C2	C2T	10
C3T	C3	C3T	11

Estado	Código $q_2q_1q_0$	Salida
C0	000	00
C1	001	01
C2	010	10
C3	011	11
C0T	100	00
C1T	101	01
C2T	110	10
C3T	111	11

Función	Ecuación
J_2	E_0
K_2	$\overline{E_0}$
J_1	$E_0 \cdot \overline{q_2} \cdot q_0$
K_1	$E_0 \cdot \overline{q_2} \cdot q_0$
J_0	$E_0 \cdot \overline{q_2}$
K_0	$E_0 \cdot \overline{q_2}$
S_1	q_1
S_0	q_0

Mi minimización coincide con la del libro como función. Simulados 300 flujos aleatorios de 40 entradas contra «número de flancos de subida, módulo 4».

> ⚠️ **Errata en el libro**
>
> En el diagrama del libro el estado C_2T aparece rotulado con la salida 01; debe ser 10, como indican la tabla de estados y el funcionamiento (cuenta 2). Se ha comprobado que con 01 el contador daría 1 en vez de 2 en ese estado.

Libro · Cap. 8 · P8.26

Caja fuerte con combinación 1-1-0 y alarma

✔ verificado

Una caja fuerte tiene dos pulsadores, P_1 y P_0 (entradas ordenadas P_1P_0, nivel alto al pulsar). Se abre al introducir la **combinación 1, 1, 0** (pulsar P_1, pulsar P_1, pulsar P_0; entre una pulsación y la siguiente hay que soltar). Cualquier pulsación equivocada dispara la **alarma**. Salidas CA: C abre la caja, A activa la alarma. Un RESET asíncrono la devuelve al estado inicial. Pulsar los dos a la vez es imposible.

🧭 Antes de empezar: qué piden y qué teoría usas

Cada pulsación correcta es un par de estados: «el botón está apretado» (esperando soltar) y «ya se ha aceptado» (esperando el siguiente). Eso da una cadena $Q_0 \to Q_1 \to S_1 \to Q_3 \to S_{11} \to Q_6 \to S_{110}$, y de cada estado de espera un botón equivocado lleva a ALARM. Tanto la apertura como la alarma son **absorbentes**: sólo se sale con el RESET.

1 Cadena de aciertos

En Q_0 se espera un 1 (10 → Q_1); mientras se mantiene (10) se queda; al soltar (00) pasa a S_1 («un 1 aceptado»). Lo mismo para el segundo 1 ($S_1 \to Q_3 \to S_{11}$). En S_{11} se espera el 0 (01 → Q_6), y al soltar se llega a S_{110} (abierta, salida 10).

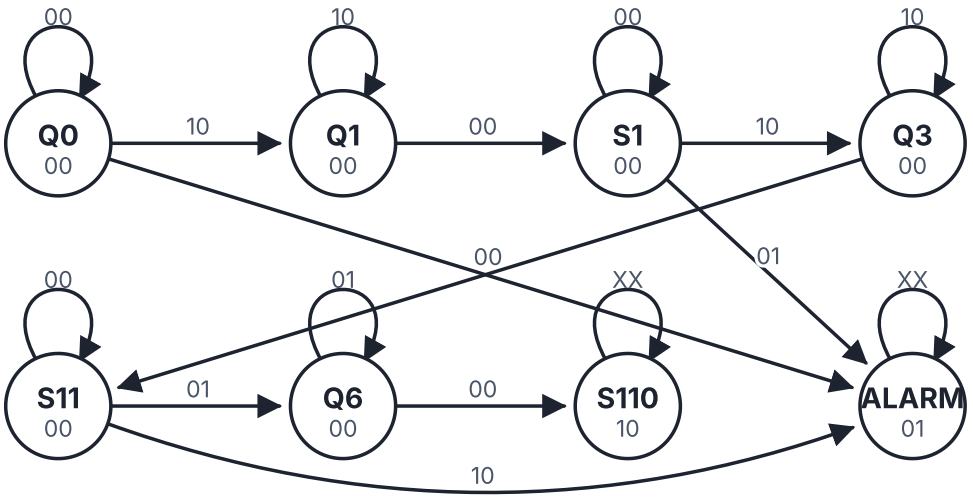
2 Errores

En Q_0 y en S_1 una pulsación de P_0 (01) es un error; en S_{11} lo es una de P_1 (10). Esos tres saltos van a ALARM (salida 01).

3 Estados de la cadena

Los estados Q_1, Q_3 y Q_6 son los de «botón apretado». Con las entradas posibles, no se puede pasar de 10 a 01 de golpe (habría que soltar primero), por lo que algunas casillas quedan como indiferentes.

✅ Resultado final



Caja fuerte (entradas P1P0). Estado: salida CA

Estado	00	01	10	11	Salida
Q0	Q0	ALARM	Q1	×	00
Q1	S1	—	Q1	×	00
S1	S1	ALARM	Q3	×	00
Q3	S11	—	Q3	×	00
S11	S11	Q6	ALARM	×	00
Q6	S110	Q6	—	×	00
S110	S110	S110	S110	×	10
ALARM	ALARM	ALARM	ALARM	×	01

Estado	Código $q_2q_1q_0$	Salida
Q0	000	00
Q1	001	00
S1	010	00
Q3	011	00
S11	100	00

Estado	Código $q_2q_1q_0$	Salida
Q6	101	00
S110	110	10
ALARM	111	01

Función	Ecuación (mía, mínima)
J_2	$\overline{P_1} \cdot q_1 \cdot q_0 + P_0$
K_2	0
J_1	$\overline{P_1} \cdot \overline{P_0} \cdot q_0 + P_0 \cdot \overline{q_2} + P_1 \cdot q_2$
K_1	$\overline{P_1} \cdot \overline{q_2} \cdot q_0$
J_0	$P_0 \cdot \overline{q_2} + P_0 \cdot \overline{q_1} + P_1 \cdot \overline{q_2} + P_1 \cdot \overline{q_1}$
K_0	$\overline{P_1} \cdot \overline{P_0} \cdot \overline{q_1} + \overline{P_1} \cdot \overline{q_2}$
S_1	$q_2 \cdot q_1 \cdot \overline{q_0}$
S_0	$q_2 \cdot q_1 \cdot q_0$

Historia de apertura correcta: pulsar 1, soltar, pulsar 1, soltar, pulsar 0, soltar → $Q_1, S_1, Q_3, S_{11}, Q_6, S_{110}$. Simulados 400 intentos aleatorios (combinación correcta, incompleta, con errores y con pulsaciones de más): el resultado siempre es «abierta» sólo con 1-1-0 y «alarma» con cualquier otro primer error. El libro presenta este ejemplo con el diagrama y las historias; las ecuaciones de arriba son las del método.

Libro · Cap. 8 · P8.27

Juego de reflejos entre dos jugadores (con juez)

✔ verificado

Dos jugadores, A y B, tienen un pulsador cada uno (E_1 y E_0). Gana el que pulse primero; si pulsan a la vez, hay empate. Salidas S_1S_0: 10 gana A, 01 gana B, 11 empate. Diseña el autómata. Después mejóralo: un **juez** da la orden de salida con un pulsador J (hay una luz L_J encendida durante la partida), quien pulse **antes** de la orden pierde, y un pulsador de **inicio** I prepara una nueva partida.

🧭 Antes de empezar: qué piden y qué teoría usas

Versión simple: cuatro estados. INICIO y tres estados absorbentes (GA, GB, GT) que registran el resultado. Su problema es doble: sólo se reinicia con RESET y se puede hacer trampa pulsando antes de empezar. La versión con juez lo resuelve añadiendo estados.

1 Versión simple

INICIO/00: con 00 espera; con 10 gana A, con 01 gana B, con 11 empate. Una vez decidido, el resultado se mantiene (la «X» de la flecha: cualquier entrada). Con el código INICIO $= 00$, GB $= 01$, GA $= 10$, GT $= 11$ las salidas son $S = q$ (sin lógica de salida).

2 Versión con juez

Entradas $I\ J\ E_1E_0$ y salidas $L_JS_1S_0$. Estados: REPOS (espera el primer inicio), INICIO (preparado, sin luz), JUGAN (juez ha dado la orden, luz encendida), PIERA y PIERB (un jugador pulsó antes de la orden y **pierde**: gana el otro), PIERT (los dos pulsaron antes), y GANA, GANB y GANT (resultado válido).

3 Reglas

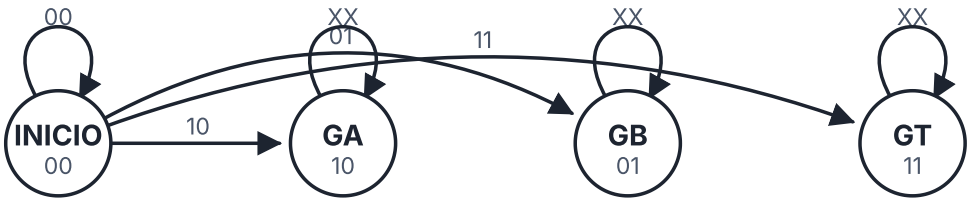
En INICIO: sólo $J = 1$ solo (0100) pasa a JUGAN; si un jugador pulsa sin que el juez haya dado la orden, pierde (PIERA con 0010, PIERB con 0001, PIERT con 0011); si la orden y la pulsación coinciden se acepta la pulsación (0110, 0101, 0111). En JUGAN el primero que pulse gana (0X10, 0X01, 0X11). Cualquier estado de resultado se mantiene hasta que $I = 1$ lo devuelve a INICIO.

4 Sobre REPOS

REPOS sólo se usa después del RESET: se sale con $I = 1$ y no se vuelve a entrar, porque los reinicios llevan a INICIO. El libro comenta que podría eliminarse.

✅ Resultado final

Versión simple



Juego simple (entradas E1E0)

Estado	00	01	10	11	Salida
INICIO	INICIO	GB	GA	GT	00
GA	GA	GA	GA	GA	10
GB	GB	GB	GB	GB	01
GT	GT	GT	GT	GT	11

Estado	Código $q_1 q_0$	Salida
INICIO	00	00
GA	10	10
GB	01	01
GT	11	11

Función	Ecuación (mía)
J_1	$E_1 \cdot \overline{q_0}$
K_1	0
J_0	$E_0 \cdot \overline{q_1}$
K_0	0
S_1	q_1
S_0	q_0

Versión con juez (9 estados; entradas $I\ J\ E_1 E_0$; estado: salida $L_J S_1 S_0$)

Estado	0000	0001	0010	0011	0100	0101	0110	0111
REPOS	REPOS	REPOS	REPOS	REPOS	REPOS	REPOS	REPOS	REPOS
INICIO	INICIO	PIERB	PIERA	PIERT	JUGAN	GANB	GANA	GANT
JUGAN	JUGAN	GANB	GANA	GANT	JUGAN	GANB	GANA	GANT
PIERA	PIERA	PIERA	PIERA	PIERA	PIERA	PIERA	PIERA	PIERA
PIERB	PIERB	PIERB	PIERB	PIERB	PIERB	PIERB	PIERB	PIERB
PIERT	PIERT	PIERT	PIERT	PIERT	PIERT	PIERT	PIERT	PIERT
GANA	GANA	GANA	GANA	GANA	GANA	GANA	GANA	GANA
GANB	GANB	GANB	GANB	GANB	GANB	GANB	GANB	GANB
GANT	GANT	GANT	GANT	GANT	GANT	GANT	GANT	GANT

Historia completa del libro: REPOS → INICIO → INICIO → JUGAN → JUGAN → GANA → GANA → INICIO, con salidas 000, 000, 000, 100, 100, 110, 110, 000. Gana B: 000, 000, 000, 100, 100, 101, 000. Pulsa A antes de la orden: PIERA con salida 001. Empate: GANT con salida 111. Se ha comprobado que desde cualquier estado $I = 1$ lleva a INICIO. El libro no desarrolla ecuaciones de la versión con juez (cuatro biestables), que quedan como ejercicio.

> ⚠️ **Errata en el libro**
>
> En las historias y en el diagrama de la versión con juez hay varias erratas: (1) la historia anota «Gana B» con salida 110; la salida de GANB es 101. (2) El diagrama rotula PIER A con salida 010 y PIER B con 001, al revés que la historia y el texto: si A se adelanta, gana B, y el código es 001 para PIERA. (3) La última historia anota el reinicio con entrada 0000; debe ser 1000 (el pulsador de inicio). Aquí se usan las salidas correctas.

`Libro · Cap. 8 · P8.28`

Alarma de sobrecarga de un ascensor (peso en serie… de tres bits)

✔ verificado

Un sensor de peso entrega el peso del ascensor en tres bits $E_2E_1E_0$ (valor de 0 a 7). Debe sonar una **alarma** si el peso es **5 durante al menos 4 pulsos de reloj consecutivos**, **6 durante al menos 2** o **7 durante al menos 1**. Cualquier otro valor (0 a 4) borra la cuenta. Diseña el autómata de Moore.

🧭 Antes de empezar: qué piden y qué teoría usas

El autómata cuenta cuántos pulsos seguidos lleva el mismo peso crítico. Para el 5 se cuenta hasta 4, para el 6 hasta 2 y el 7 se detecta ya a la primera, así que hay ocho estados: INICI (nada), P51 a P54 (lleva 1, 2, 3 y 4 o más pulsos con peso 5), P61 y P62 (1 y 2 o más con peso 6) y P7. La alarma está en P54, P62 y P7.

1 Transiciones

Con peso de 0 a 4 se vuelve siempre a INICI. Con peso 5 se avanza P51→P52→P53→P54 (que se mantiene); desde cualquier otro estado (incluidos P6x y P7) se empieza una cuenta nueva en P51. Con peso 6 se avanza P61→P62 y desde el resto se entra en P61. Con peso 7 se va a P7 desde cualquier estado.

2 Qué significa «seguidos»

Es la cuenta de **pulsos consecutivos con exactamente el mismo peso**: si entre dos pesos 5 aparece un 6, la cuenta de 5 se pierde (y al volver el 5 se parte de 1). Es una decisión de diseño razonable, pero conviene saberla: tres pesos 5 seguidos de un 6 no dan alarma por ninguno de los dos.

3 Codificación

Natural: INICI $= 000$, P51 $= 001$, ..., P54 $= 100$, P61 $= 101$, P62 $= 110$, P7 $= 111$. La salida es $S_0 = q_2\overline{q_0} + q_2 q_1$.

✅ Resultado final

Estado	000	001	010	011	100	101	110	111	Salida
INICI	INICI	INICI	INICI	INICI	INICI	P51	P61	P7	0
P51	INICI	INICI	INICI	INICI	INICI	P52	P61	P7	0
P52	INICI	INICI	INICI	INICI	INICI	P53	P61	P7	0
P53	INICI	INICI	INICI	INICI	INICI	P54	P61	P7	0
P54	INICI	INICI	INICI	INICI	INICI	P54	P61	P7	1
P61	INICI	INICI	INICI	INICI	INICI	P51	P62	P7	0
P62	INICI	INICI	INICI	INICI	INICI	P51	P62	P7	1

Estado	000	001	010	011	100	101	110	111	Salida
P7	INICI	INICI	INICI	INICI	INICI	P51	P61	P7	1

Función	Ecuación (libro)
J_2	$E_2 \cdot E_0 \cdot q_1 \cdot q_0 + E_2 \cdot E_1$
K_2	$\overline{E_1} \cdot \overline{E_0} + \overline{E_1} \cdot q_0 + \overline{E_1} \cdot q_1 + \overline{E_2}$
J_1	$E_2 \cdot E_0 \cdot \overline{q_2} \cdot q_0 + E_2 \cdot E_1 \cdot q_2 \cdot q_0 + E_2 \cdot E_1 \cdot E_0$
K_1	$\overline{E_0} \cdot \overline{q_2} + \overline{E_1} \cdot q_0 + \overline{E_1} \cdot q_2 + \overline{E_2} + \overline{E_0} \cdot q_0$
J_0	$E_2 \cdot E_0 \cdot \overline{q_2} + E_2 \cdot E_0 \cdot q_1 + E_2 \cdot E_1 \cdot \overline{q_1} + E_2 \cdot E_1 \cdot \overline{q_2}$
K_0	$\overline{E_0} \cdot q_2 \cdot \overline{q_1} + \overline{E_1} \cdot \overline{q_2} + \overline{E_1} \cdot \overline{E_0} + \overline{E_2}$
S	$q_2 \cdot \overline{q_0} + q_2 \cdot q_1$

Salida: $S_0 = q_2\overline{q_0} + q_2q_1$. Mi propia minimización coincide con ellas como función.

Las ecuaciones del libro (suma de productos) se han comprobado en las 64 transiciones: todas reproducen la tabla. Se ha simulado con 400 secuencias de 40 pesos, muchas de ellos críticos, contra el modelo directo «longitud de la racha de pesos iguales». También se reproduce la historia del libro: tres pesos bajos, luego cinco veces el 5 (con alarma al cuarto) y vuelta a INICI con el 2.

Libro · Cap. 8 · P8.29

Dispensador de leche con calentador (Moore, 10 estados)

✔ verificado

Una máquina dispensa leche cuando se pulsa un pulsador P, **al soltarlo**. Durante cuatro segundos sale la leche y se recarga el depósito; después, durante otros cuatro segundos, se calienta la leche. En las ocho fases la máquina está ocupada y **no atiende nuevas pulsaciones**. Cuatro salidas $D\,R\,C\,L_R$: dispensador, recargador, calentador y led rojo de ocupado (L_R, a 1 durante las ocho fases). Hay un reloj de 1 Hz. Diseña el autómata de Moore.

🧭 Antes de empezar: qué piden y qué teoría usas

Es una secuencia temporizada disparada por un pulsador: una espera, una confirmación de que se soltó y ocho pasos de reloj fijos. Con un reloj de 1 Hz cada paso dura un segundo, así que se necesitan 4 estados para la fase de dispensar y 4 para la de calentar. Más los dos de espera: **10 estados**.

1 Esperar y confirmar

INICIO (todo a 0): con $P = 1$ pasa a TRANS; TRANS espera a que se suelte ($P = 0$) y entonces arranca el ciclo. Esto evita arrancar al pulsar y permite pulsaciones largas.

2 Dispensar y calentar

D1, D2, D3 y D4 con salidas 1101 ($D = R = L_R = 1$), y C1, C2, C3 y C4 con salidas 0011 ($C = L_R = 1$). En cada uno se pasa al siguiente con cualquier entrada (la «X»: no se atiende el pulsador durante el ciclo). Tras C4 se vuelve a INICIO.

3 Reducción

No se puede fusionar ningún par: los ocho estados del ciclo están a distancias distintas del final, y INICIO y TRANS difieren en cómo reaccionan a P. El algoritmo de particiones confirma que el autómata es mínimo.

✅ **Resultado final**

Estado	0	1	Salida
INICIO	INICIO	TRANS	0000
TRANS	D1	TRANS	0000
D1	D2	D2	1101
D2	D3	D3	1101
D3	D4	D4	1101
D4	C1	C1	1101
C1	C2	C2	0011
C2	C3	C3	0011
C3	C4	C4	0011
C4	INICIO	INICIO	0011

El libro presenta sólo el diagrama y los comentarios, sin ecuaciones (cuatro biestables para 10 estados). Se ha simulado el funcionamiento: tras soltar el pulsador salen exactamente cuatro

ciclos 1101, luego cuatro 0011 y se vuelve a 0000, sin importar lo que valga P durante esos ocho ciclos (200 casos aleatorios).

Libro · Cap. 8 · P8.30

Barrera de un garaje (Moore, entradas SE SS SA SC)

✔ verificado

Una barrera de garaje tiene cuatro sensores, ordenados $S_E\,S_S\,S_A\,S_C$: coche en la **entrada** (antes de la barrera), coche en la **salida** (ya pasó), barrera **abierta** y barrera **cerrada**. La orden al motor son dos bits M_1M_0: 00 parado, 01 abre, 10 cierra. Debe abrirse cuando llega un coche, esperar abierta hasta que haya pasado y cerrarse. S_E y S_S no pueden valer 1 a la vez, ni S_A y S_C.

🧭 Antes de empezar: qué piden y qué teoría usas

Cuatro estados de la barrera: REPOS (cerrada y parada), ABDO (abriéndose), ABTO (abierta, parada, esperando que el coche pase) y CDO (cerrándose). De las 16 entradas quedan 9 posibles: se descartan 11XX y XX11.

1 Transiciones que dibuja el libro

REPOS con $S_E = 1$ pasa a ABDO; ABDO se mantiene (1001, 1000, 0000) hasta que se completa la apertura (1010 → ABTO) o el coche ya ha pasado (0100, 0110 → CDO); ABTO se mantiene con 1010; CDO se mantiene (0010, 0110, 0100) y llega a REPOS con 0001.

2 Cómo se completan

ABDO: si $S_S = 1$ (el coche ya pasó) cierra; si no, con $S_A = 1$ queda abierta. ABTO: sale a CDO cuando el coche pasa ($S_S = 1$). CDO: con $S_C = 1$ llega a REPOS. REPOS: sólo S_E la saca de ahí. Así quedan cubiertas las nueve combinaciones posibles de cada estado y se reproducen las dos historias del libro.

✅ Resultado final

Estado	0000	0001	0010	0011	0100	0101	0110	0111	100
REPOS	REPOS	REPOS	REPOS	×	REPOS	REPOS	REPOS	×	ABI
ABDO	ABDO	ABDO	ABTO	×	CDO	CDO	CDO	×	ABI
ABTO	ABTO	ABTO	ABTO	×	CDO	CDO	CDO	×	ABT

Estado	0000	0001	0010	0011	0100	0101	0110	0111	100
CDO	CDO	REPOS	CDO	×	CDO	REPOS	CDO	×	CD(

Estado	Código $q_1 q_0$	Salida
REPOS	00	00
ABDO	01	01
ABTO	11	00
CDO	10	10

Función	Ecuación (mía, mínima)
J_1	$\mathrm{SA} \cdot q_0 + \mathrm{SS} \cdot q_0$
K_1	$\mathrm{SC} \cdot \overline{q_0}$
J_0	$\mathrm{SE} \cdot \overline{q_1}$
K_0	SS
S_1	$q_1 \cdot \overline{q_0}$
S_0	$\overline{q_1} \cdot q_0$

Historia 1: REPOS, ABDO, ABDO, ABDO, CDO, CDO, CDO, REPOS. Historia 2 (coche que se adelanta con la barrera a medias, abierta del todo, pasa y cierra): REPOS, ABDO, ABDO, ABDO, ABDO, ABDO, ABDO, ABTO, CDO, CDO, CDO, REPOS.

El libro no da ecuaciones de este problema (sólo diagrama y comentarios). Las de arriba se derivan con el método general y se han comprobado.

> ⚠️ **Errata en el libro**
>
> Las flechas del diagrama no son coherentes con las historias del propio libro: el dibujo rotula con 1010 la salida de ABDO a ABTO y con 0010 la de ABTO a CDO, pero en las historias se llega a ABTO con 0010 (paso 10) y se pasa a CDO con 0110 (paso 11). Aquí se usa la tabla que cumple las historias (la que sale arriba).

`Libro · Cap. 8 · P8.31` **Calificaciones recibidas en serie (Moore, 10 → 9 estados)**

✔ verificado

Un sistema de corrección entrega la nota de cada examen en **tres bits seguidos** por una línea serie E: el primero dice si se supera la parte básica (BAS), el segundo si se llega a notable (NOT) y el tercero si se llega a sobresaliente (SOB). Interpretación: $0XX$ suspenso, $10X$ aprobado, 110 notable y 111 sobresaliente. Hay tres luces $L_A L_N L_S$ (aprobado, notable, sobresaliente) que se encienden **al llegar el tercer bit**. Diseña el autómata de Moore y reduce sus estados.

🧭 Antes de empezar: qué piden y qué teoría usas

Hay que contar tres bits aunque no importen. En un suspenso, el primer bit (0) decide, pero hay que dejar pasar los otros dos antes de aceptar el examen siguiente; por eso aparecen cadenas de estados «de relleno» (SUSP1, SUSP2, SUSP3 y APRO1, APRO2).

1 Estados

Suspenso: INICIO, SUSP1, SUSP2, SUSP3 (cuentan 3 bits sin salida). Aprobado: EMPIE, APRO1, APRO2 (luz L_A). Notable/sobresaliente: SIGUE (ha llegado «11»), NOTAB (L_N) y SOBRE (L_S). Desde los estados finales (SUSP3, APRO2, NOTAB, SOBRE) el siguiente bit es el primero de otro examen.

2 Reducción

INICIO y SUSP3 son **equivalentes**: la misma salida (000) y los mismos siguientes (0→SUSP1, 1→EMPIE). Se fusionan y quedan **nueve estados** (cuatro biestables). El algoritmo de particiones no encuentra más.

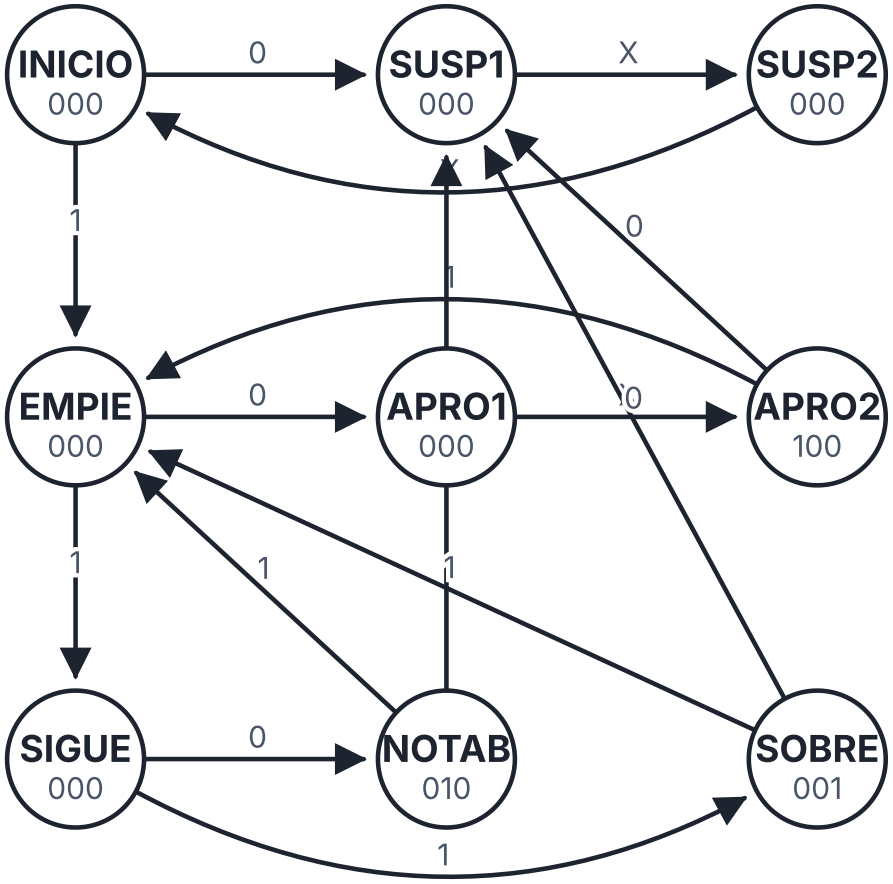
✅ Resultado final

Con diez estados

Estado	0	1	Salida
INICIO	SUSP1	EMPIE	000
SUSP1	SUSP2	SUSP2	000
SUSP2	SUSP3	SUSP3	000
SUSP3	SUSP1	EMPIE	000
EMPIE	APRO1	SIGUE	000
APRO1	APRO2	APRO2	000

Estado	0	1	Salida
APRO2	SUSP1	EMPIE	100
SIGUE	NOTAB	SOBRE	000
NOTAB	SUSP1	EMPIE	010
SOBRE	SUSP1	EMPIE	001

Reducido a nueve (INICIO y SUSP3 fusionados)



Calificaciones (9 estados). Etiqueta: E; estado: LALNLS

Estado	0	1	Salida
INICIO	SUSP1	EMPIE	000

Estado	0	1	Salida
SUSP1	SUSP2	SUSP2	000
SUSP2	INICIO	INICIO	000
EMPIE	APRO1	SIGUE	000
APRO1	APRO2	APRO2	000
APRO2	SUSP1	EMPIE	100
SIGUE	NOTAB	SOBRE	000
NOTAB	SUSP1	EMPIE	010
SOBRE	SUSP1	EMPIE	001

Simulados 300 flujos de entre 1 y 6 exámenes de tres bits aleatorios: las luces se encienden sólo tras el tercer bit y con el resultado correcto; las dos versiones se comportan igual. El libro desarrolla este ejemplo con diagrama y sin ecuaciones.

`Libro · Cap. 8 · P8.32`

Generar un reloj CK′ cuyo primer flanco tras CLEAR sea ascendente

✔ verificado

Se quiere generar a partir de un reloj CK y una señal $CLEAR$ un reloj CK' que, al activarse $CLEAR$ (nivel alto), se ponga a 0 y, tras liberarse, **empiece siempre con un flanco ascendente**: nunca debe aparecer un flanco o un pulso incompleto aunque $CLEAR$ se suelte con $CK = 1$. Entradas $CLEAR\,CK$, salida CK'. Diseña el autómata.

🧭 Antes de empezar: qué piden y qué teoría usas

Hay que esperar a ver un 0 de CK antes de seguir al reloj. Si $CLEAR$ se libera cuando CK vale 1 y se copiara CK sin más, saldría un pulso recortado. Basta un estado de espera (ACTIV) que no copie CK hasta que $CK = 0$.

1 Estados

INICIO: sin activar (sólo $CLEAR = 1$ lo saca de ahí). ACTIV: $CLEAR$ aplicado; espera un 0 de CK (con $CK = 1$ se queda). CK0 y CK1: reproducen CK (salida 0 y 1).

2 Transiciones

ACTIV con $CLEAR = 0$ y $CK = 0$ pasa a CK0. CK0 pasa a CK1 con $CK = 1$ y CK1 a CK0 con $CK = 0$. En CK0, CK1 y ACTIV, $CLEAR = 1$ lleva (y mantiene) en ACTIV, con la salida a 0.

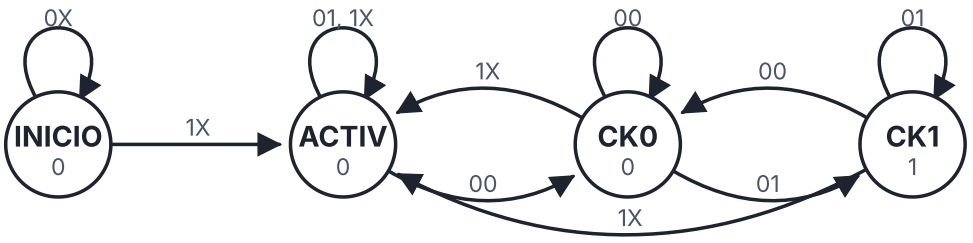
3 Garantía

Como se llega a CK0 antes que a CK1, el primer cambio de la salida es siempre $0 \to 1$, un flanco ascendente. Se ha comprobado en 300 secuencias aleatorias.

4 Ojo con el uso

Una señal generada así no debería usarse como reloj de otros biestables: es una señal derivada de un autómata y rompe el principio de no mezclar relojes. Es preferible usar el CK original con una entrada de habilitación (**ENABLE**).

✅ Resultado final



Generador de CK′ (entradas CLEAR CK). Estado: salida CK'

Estado	00	01	10	11	Salida
INICIO	INICIO	INICIO	ACTIV	ACTIV	0
ACTIV	CK0	ACTIV	ACTIV	ACTIV	0
CK0	CK0	CK1	ACTIV	ACTIV	0
CK1	CK0	CK1	ACTIV	ACTIV	1

Simulados 400 flujos de 40 pares $CLEARCK$ contra el modelo «0 hasta que haya habido un CLEAR y después un 0 de CK; a partir de ahí, la salida es CK». El libro presenta sólo el diagrama y los comentarios.

Libro · Cap. 8 · P8.33

Un pulso de un ciclo de reloj por cada pulso de entrada (3 variantes)

✔ verificado

Una señal E_0, manual y mucho más lenta que el reloj (cada nivel dura **más de un periodo** de reloj), debe generar **un pulso de exactamente un ciclo de reloj**. Diséñalo de tres maneras: (1) que el pulso aparezca al **subir** E_0, (2) al **bajar**, (3) en **los dos** flancos (dos pulsos por cada pulso de entrada). Autómatas de Moore con J-K.

🧭 Antes de empezar: qué piden y qué teoría usas

Es un detector de flancos síncrono. Un flanco no es un nivel: es un cambio. El autómata debe recordar dos cosas: que ya vio el flanco (para no repetir el pulso aunque E_0 siga a 1) y, en los dos últimos casos, qué nivel tenía E_0.

1 Opción 1: flanco de subida

INICIO (espera un 1) → PULSO (salida 1, dura un ciclo) → FINAL (espera a que E_0 baje, sin repetir). Código 00, 01, 10 y $S_0 = q_0$. Ecuaciones: $J_1 = q_0$, $K_1 = \overline{E_0}$, $J_0 = E_0\overline{q_1}$, $K_0 = 1$.

2 Opción 2: flanco de bajada

INICIO → EMPIE (hay un 1; espera que baje) → PULSO (salida 1) → INICIO. Código 00, 01, 10 y $S_0 = q_1$. Ecuaciones: $J_1 = \overline{E_0}q_0$, $K_1 = 1$, $J_0 = E_0\overline{q_1}$, $K_0 = \overline{E_0}$.

3 Opción 3: los dos flancos

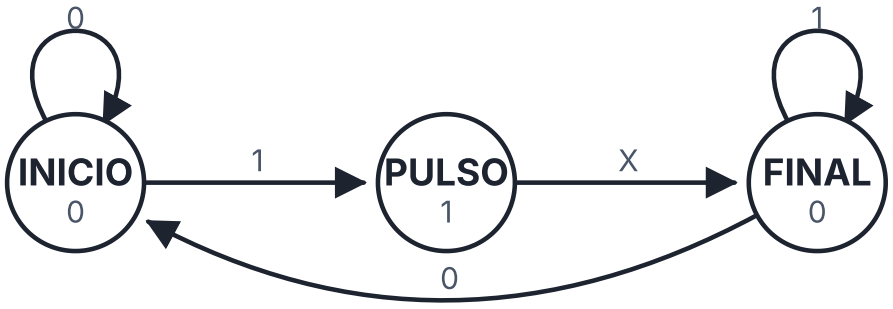
INICIO → PULS1 (salida 1) → MANT (mantiene el nivel alto) → PULS0 (salida 1, tras caer) → INICIO. Código 00, 01, 10 y 11, con $S_0 = q_0$. Ecuaciones: $J_1 = K_1 = q_0$, $J_0 = E_0\overline{q_1} + \overline{E_0}q_1$, $K_0 = 1$.

4 Por qué se exige un nivel mayor que un periodo

El autómata sólo mira E_0 una vez por ciclo. Si el pulso de entrada durara menos de un ciclo, podría pasar inadvertido; y si los niveles duran un solo ciclo, las transiciones de espera (FINAL, PULSO, ...) no llegan a ver el cambio siguiente. Con niveles de al menos dos ciclos funcionan las tres.

✅ Resultado final

Opción 1: flanco de subida



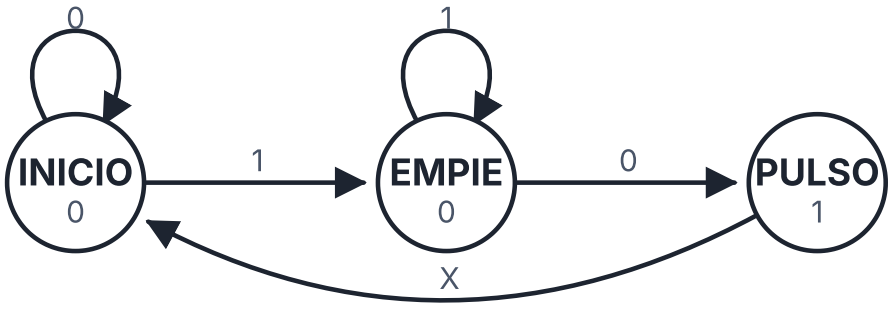
Pulso en el flanco de subida

Estado	0	1	Salida
INICIO	INICIO	PULSO	0
PULSO	FINAL	FINAL	1
FINAL	INICIO	FINAL	0

Estado	Código $q_1 q_0$	Salida
INICIO	00	0
PULSO	01	1
FINAL	10	0

Función	Ecuación
J_1	q_0
K_1	$\overline{E_0}$
J_0	$E_0 \cdot \overline{q_1}$
K_0	1
S	q_0

Opción 2: flanco de bajada

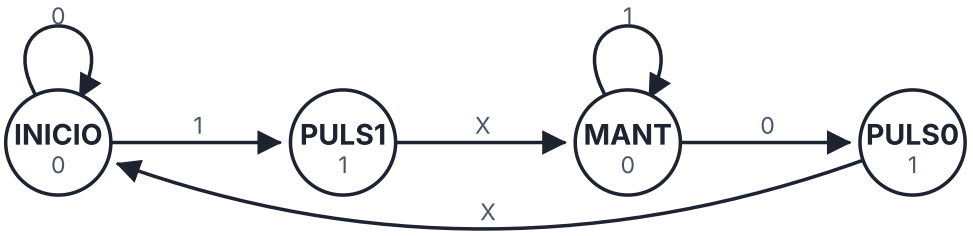


Pulso en el flanco de bajada

Estado	0	1	Salida
INICIO	INICIO	EMPIE	0
EMPIE	PULSO	EMPIE	0
PULSO	INICIO	INICIO	1

Función	Ecuación
J_1	$\overline{E_0} \cdot q_0$
K_1	1
J_0	$E_0 \cdot \overline{q_1}$
K_0	$\overline{E_0}$
S	q_1

Opción 3: los dos flancos

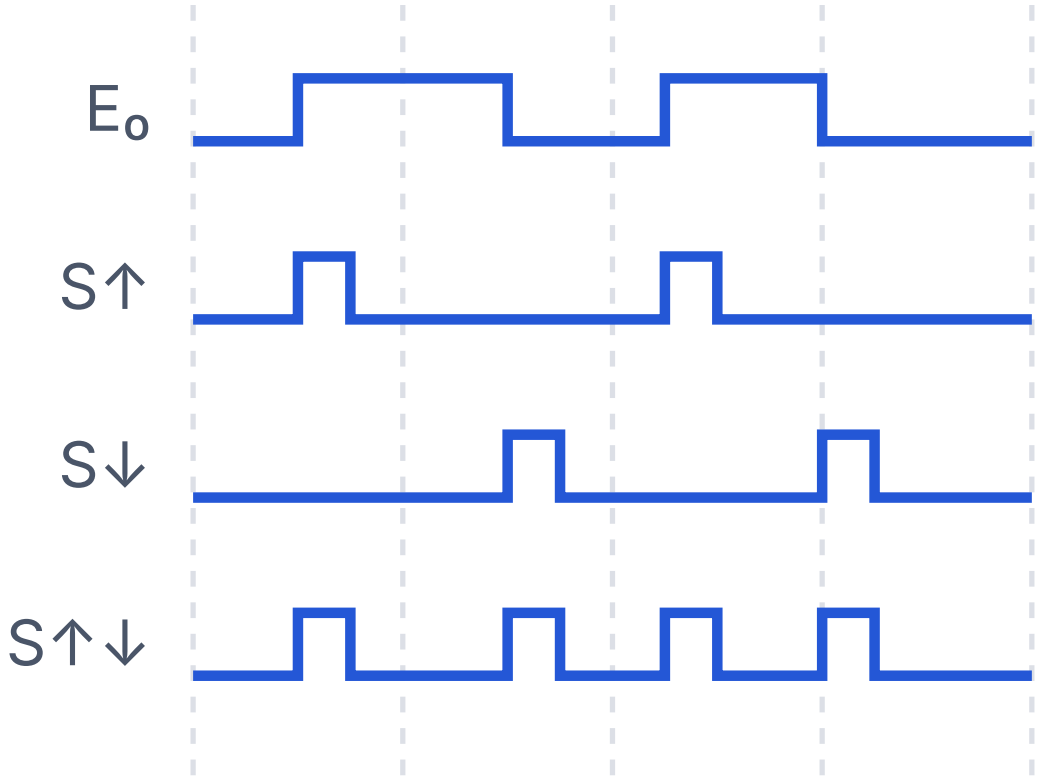


Pulso en los dos flancos

Estado	0	1	Salida
INICIO	INICIO	PULS1	0
PULS1	MANT	MANT	1
MANT	PULS0	MANT	0
PULS0	INICIO	INICIO	1

Función	Ecuación
J_1	q_0
K_1	q_0
J_0	$E_0 \cdot \overline{q_1} + \overline{E_0} \cdot q_1$
K_0	1
S	q_0

Cronograma de las tres opciones con la misma entrada



Las ecuaciones del libro se han comprobado en todas las transiciones de las tres variantes, y se han simulado 400 secuencias con niveles de 2 a 4 ciclos contra «el estado actual difiere del anterior» (y sus dos casos particulares).

> ⚠️ **Errata en el libro**
>
> En el diagrama del libro de la opción 3, el estado PULS1 aparece con salida 0. Debe ser 1: sin ella no habría pulso en el flanco de subida, y contradice la tabla de estados codificada y el cronograma ($S_0 = q_0$).

`Libro · Cap. 8 · P8.34`

Transmisor serie de dos bits con arranque y parada

✔ verificado

Un transmisor envía por una línea serie S los dos bits E_1E_0 que tiene en sus entradas cuando se acciona un pulsador P (entradas $P\,E_1\,E_0$). La línea en reposo está a 1. Cada transmisión es una trama de **cuatro símbolos**: un bit de arranque a 0, el bit E_1, el bit E_0 y un bit final a 0; después la línea vuelve a 1. Debe transmitirse al **soltar** el pulsador. Diseña el autómata de Moore.

> 🧭 **Antes de empezar: qué piden y qué teoría usas**
>
> **El autómata es un secuenciador con ramificaciones sólo para copiar los datos.** Hay una espera, una confirmación de soltado y luego tres pasos fijos. En los pasos de datos hay que elegir entre dos estados según el bit que se copia (UNO0/UNO1 y DOS0/DOS1) porque la salida es parte del estado.

1 Esperar y arrancar

INICIO (línea a 1): con $P = 1$ pasa a TRANS; TRANS espera a que P se suelte; al soltar, EMPIE saca el bit de arranque (0).

2 Copiar los datos

De EMPIE se pasa a UNO0 o UNO1 según E_1 (salida igual al bit copiado); de ahí a DOS0 o DOS1 según E_0; y de DOS siempre a FINAL (salida 0).

3 Fin

En FINAL, si P vuelve a estar pulsado se encadena otra trama (TRANS); si no, se vuelve a

INICIO con la línea a 1. La salida recorre pues 1, 0, E_1, E_0, 0, 1.

✅ **Resultado final**

Estado	000	001	010	011	100	101	110	111
INICIO	INICIO	INICIO	INICIO	INICIO	TRANS	TRANS	TRANS	TRANS
TRANS	EMPIE	EMPIE	EMPIE	EMPIE	TRANS	TRANS	TRANS	TRANS
EMPIE	UNO0	UNO0	UNO1	UNO1	UNO0	UNO0	UNO1	UNO1
UNO0	DOS0	DOS1	DOS0	DOS1	DOS0	DOS1	DOS0	DOS1
UNO1	DOS0	DOS1	DOS0	DOS1	DOS0	DOS1	DOS0	DOS1
DOS0	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL
DOS1	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL	FINAL
FINAL	INICIO	INICIO	INICIO	INICIO	TRANS	TRANS	TRANS	TRANS

Simulado con 300 tramas de datos aleatorios y pulsaciones de longitud variable: la secuencia de salida tras soltar P es siempre 0, E_1, E_0, 0, 1. El libro lo presenta sólo con diagrama y comentarios.

`Libro · Cap. 8 · P8.35`

Receptor serie de dos bits con detección de error

✔ verificado

Un receptor recibe por una línea serie E las tramas del problema anterior: la línea en reposo está a 1 y cada trama es 0 (arranque), dos bits de datos, 0 (final). Salidas $I_1 I_0$ AL: los dos bits recibidos y una alarma AL que se activa si el bit final no vale 0. Diseña el autómata de Moore. Después modifícalo para que **mantenga los datos** en las salidas hasta que empiece la trama siguiente.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Es el recíproco del transmisor: hay que recordar los dos bits conforme llegan. Los estados que recuerdan el primer bit (UNO0/UNO1) se ramifican otra vez con el segundo (DOS0, DOS1, TRES0, TRES1), y en todos ellos se comprueba el bit final. Si vale 1, el receptor cae en ERROR, que es **absorbente**.

1 **Primer autómata**

INICI espera el 0 de arranque; EMPIE lo ha recibido; UNO0/UNO1 recuerdan el primer dato (I_1 ya vale ese bit); DOS0, DOS1, TRES0 y TRES1 recuerdan los dos y presentan $I_1 I_0$ (00, 01, 10 y 11) con $AL = 0$. Con 0 de cierre se pasa a BIEN y con 1 a ERROR ($AL = 1$).

2 Después de BIEN

Con 1 (reposo) a INICI; con 0, la trama siguiente empieza ya: a EMPIE.

3 Autómata que mantiene el dato

En lugar de BIEN se tienen cuatro estados FIJO0 a FIJO3, que **conservan los mismos valores de salida** durante el reposo (1 se queda; 0 es el arranque de la trama siguiente: EMPIE). Así los datos se mantienen estables hasta el siguiente 0.

✅ Resultado final

Primer autómata (10 estados; las «X» de salida se toman como 0)

Estado	0	1	Salida
INICI	EMPIE	INICI	000
EMPIE	UNO0	UNO1	000
UNO0	DOS0	DOS1	000
UNO1	TRES0	TRES1	100
DOS0	BIEN	ERROR	000
DOS1	BIEN	ERROR	010
TRES0	BIEN	ERROR	100
TRES1	BIEN	ERROR	110
BIEN	EMPIE	INICI	000
ERROR	ERROR	ERROR	001

Autómata que mantiene el dato (13 estados)

Estado	0	1	Salida
INICI	EMPIE	INICI	000
EMPIE	UNO0	UNO1	000
UNO0	DOS0	DOS1	000
UNO1	TRES0	TRES1	100
DOS0	FIJO0	ERROR	000
DOS1	FIJO1	ERROR	010
TRES0	FIJO2	ERROR	100
TRES1	FIJO3	ERROR	110
ERROR	ERROR	ERROR	001
FIJO0	EMPIE	FIJO0	000
FIJO1	EMPIE	FIJO1	010
FIJO2	EMPIE	FIJO2	100
FIJO3	EMPIE	FIJO3	110

Simuladas 300 secuencias de entre una y cinco tramas con datos aleatorios y algún final erróneo: los datos aparecen en el ciclo en que se recibe el segundo bit, la alarma se activa y permanece tras un final erróneo, y en el segundo autómata el dato se mantiene durante el reposo. El libro no incluye ecuaciones (diagramas y comentarios).

> ⚠️ **Errata en el libro**
>
> En el primer diagrama el estado DOS1 (los dos bits recibidos son 0 y 1) aparece con salida 110. Debe ser 010: $I_1 = 0$ e $I_0 = 1$, que es además lo que dice el segundo autómata para el estado equivalente FIJO1.

DEMOSTRACIONES

Demostraciones

Por qué son ciertas las reglas que usas: De Morgan, consenso, C2, sumador, universalidad de NAND, JK, contadores, Johnson y LFSR.

Aquí se demuestran, paso a paso, las reglas que en los temas se usan sin discutir. En el examen no suelen pedir la demostración entera, pero **entender el porqué** te permite reconstruir la regla si la olvidas. Todos los enunciados se han comprobado además por código con todos los casos posibles.

Demostración D1

D1 · Leyes de De Morgan

✔ verificado

Demuestra que $\overline{A+B} = \overline{A} \cdot \overline{B}$ y que $\overline{A \cdot B} = \overline{A} + \overline{B}$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: probar una igualdad entre funciones lógicas. **Idea:** dos funciones son iguales si valen lo mismo para **todas** las combinaciones de entradas (inducción perfecta).

1 Tabla de verdad

A	B	$\overline{A+B}$	$\overline{A}\,\overline{B}$	$\overline{AB}$	$\overline{A}+\overline{B}$
0	0	1	1	1	1
0	1	0	0	1	1
1	0	0	0	1	1
1	1	0	0	0	0

Las columnas 3 y 4 coinciden; las 5 y 6 también. Con dos variables hay solo 4 filas: la prueba es completa.

2 La idea en palabras

$\overline{A+B}$ vale 1 solo si **ninguna** de las dos vale 1, es decir, si $A = 0$ **y** $B = 0$. $\overline{AB}$ vale 0 solo si las dos valen 1, es decir, vale 1 si **alguna** vale 0.

3 Generalización

Se aplica a n variables: $\overline{A+B+C}=\overline{A}\,\overline{B}\,\overline{C}$. La regla práctica: **cambia + por · (y al revés) y niega cada término**.

✅ **Resultado final**

Demostrado por inducción perfecta (y comprobado también con tres variables).

Demostración D2

D2 · Adyacencia, absorción y consenso

✔ verificado

Demuestra: (a) $AB+A\overline{B}=A$; (b) $A+AB=A$; (c) $AB+\overline{A}C+BC=AB+\overline{A}C$.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: tres identidades que justifican los grupos de Karnaugh. **Idea:** sacar factor común y usar $X+\overline{X}=1$.

1 (a) Adyacencia

$AB+A\overline{B}=A(B+\overline{B})=A\cdot 1=A$. Es la regla que hace que **dos casillas vecinas** del mapa se simplifiquen: la variable que cambia desaparece.

2 (b) Absorción

$A+AB=A(1+B)=A\cdot 1=A$. Si $A=1$ la suma ya vale 1, y si $A=0$ el término AB también vale 0.

3 (c) Consenso

Se multiplica el término BC por $(A+\overline{A})=1$:

$$BC=BC(A+\overline{A})=ABC+\overline{A}BC.$$

Entonces $AB+\overline{A}C+ABC+\overline{A}BC$. Pero ABC está absorbido por AB y $\overline{A}BC$ por $\overline{A}C$ (absorción). Queda $AB+\overline{A}C$.

⚠️ Para qué sirve el consenso

En un circuito con **aleas** (fallos momentáneos) a veces se **añade** el término de consenso a propósito. En el diseño con mínimos se **quita**.

✅ Resultado final

Las tres identidades se demuestran con factor común y $X + \overline{X} = 1$.

Demostración D3

D3 · Por qué invertir y sumar 1 da el opuesto en C2

✔ verificado

Demuestra que en complemento a 2 de n bits el opuesto de N es $\overline{N} + 1$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar la regla de «invertir y sumar 1». **Idea:** el C2 de N se define como $2^n - N$. Sumar N y su C2 da 2^n, que en n bits es 0.

1 Un número más su inverso

Si inviertes todos los bits de N, cada posición suma 1 con la de N: $N + \overline{N} = 11\ldots1 = 2^n - 1$.

2 Despejar el opuesto

$$-N \equiv 2^n - N = (2^n - 1 - N) + 1 = \overline{N} + 1.$$

Eso es exactamente «invertir y sumar 1». En n bits, 2^n se descarta (es el acarreo final), así que $N + (\overline{N} + 1) = 0$.

3 Ejemplo

Con 4 bits, $N = 0101 = 5$: $\overline{N} = 1010$, $+1 = 1011$. Comprobación: $0101 + 1011 = 1\,0000$; el acarreo se descarta y queda 0000. Entonces $1011 = -5$.

✅ Resultado final

$-N = \overline{N} + 1$ porque $N + \overline{N} = 2^n - 1$. Comprobado con todos los números de 3, 4, 5 y 8 bits.

Demostración D4

D4 · Por qué restar es sumar el inverso y un 1

✔ verificado

Demuestra que $A - B = A + \overline{B} + 1$ en un sumador de n bits (módulo 2^n).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar que un sumador sirve también como restador. **Teoría:** D3.

1 Aplicar D3

Por D3, $-B = \overline{B} + 1$ módulo 2^n.

2 Sumar

$A - B = A + (-B) = A + \overline{B} + 1$. El «+1» se introduce como **acarreo de entrada** $C_{in} = 1$, sin gastar otro sumador.

✅ Resultado final

Un sumador con B invertida y $C_{in} = 1$ calcula $A - B$. Comprobado con las 2^8 combinaciones de 4 bits y con 8 bits.

Demostración D5

D5 · Ecuaciones del sumador completo

✔ verificado

Demuestra que un sumador completo tiene $S = A \oplus B \oplus C_{in}$ y $C_{out} = AB + C_{in}(A \oplus B)$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar de «suma de tres bits» a ecuaciones lógicas. **Idea:** la suma $A + B + C_{in}$ va de 0 a 3 y se escribe $C_{out}S$ en binario.

1 Tabla

$A+B+C_{in}$	0	1	2	3
$C_{out}S$	00	01	10	11

S vale 1 cuando el número de unos es **impar**: es la XOR de tres entradas. C_{out} vale 1 cuando hay **dos o más** unos: es la mayoría de tres.

2 Mayoría de tres

La mayoría de tres es $AB + AC_{in} + BC_{in}$ (ver K1 en las hojas). Se agrupa: $AB + C_{in}(A + B)$. Y como AB ya cubre el caso $A = B = 1$, se puede cambiar $A + B$ por $A \oplus B$: $C_{out} = AB + C_{in}(A \oplus B)$. También es válida $AB + C_{in}(A + B)$.

✅ **Resultado final**

$S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$. Comprobadas las 8 combinaciones.

Demostración D6

D6 · NAND y NOR son puertas universales

✔ verificado

Demuestra que con solo NAND se puede construir NOT, AND y OR (y por tanto cualquier función). Haz lo mismo con NOR.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: construir las tres puertas básicas a partir de una sola. **Teoría:** De Morgan (D1).

1 NOT

$\text{NAND}(A, A) = \overline{A \cdot A} = \overline{A}$. Se unen las dos entradas.

2 AND

$\text{NAND}(A, B) = \overline{AB}$. Se niega con una NAND con las entradas unidas: $\overline{\overline{AB}} = AB$.

3 OR

Por De Morgan, $A + B = \overline{\overline{A} \cdot \overline{B}} = \text{NAND}(\overline{A}, \overline{B})$: se niegan A y B con NAND y luego otra

NAND.

4 NOR

Por dualidad: $\mathrm{NOR}(A,A)=\overline{A}$; $A+B=\overline{\mathrm{NOR}(A,B)}$; $AB=\mathrm{NOR}(\overline{A},\overline{B})$.

✅ Resultado final

Con NAND (o con NOR) se obtiene NOT, AND y OR, y con ellas cualquier función. Comprobado con todas las combinaciones.

Demostración D7

D7 · Ecuación característica del JK y tabla de excitación

✔ verificado

Demuestra que el JK cumple $Q^+=J\overline{Q}+\overline{K}Q$ y deduce la tabla de excitación.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: pasar del comportamiento del JK a una ecuación y de ahí al diseño. **Teoría:** T6.

1 La ecuación

El JK **pone a 1** (J) cuando $Q=0$ y **mantiene** el 1 si $K=0$ cuando $Q=1$:

- $Q=0$: sube solo si $J=1$. Término $J\overline{Q}$.
- $Q=1$: se queda en 1 solo si $K=0$. Término $\overline{K}Q$.

Sumando: $Q^+=J\overline{Q}+\overline{K}Q$.

2 Tabla de excitación

Se despejan J,K para cada transición $Q\to Q^+$:

Q	Q^+	J	K
0	0	0	X
0	1	1	X
1	0	X	1

Q	Q^+	J	K
1	1	X	0

Si $Q = 0$ la K no influye; si $Q = 1$ la J no influye. Esas X son los indiferentes que simplifican las ecuaciones.

✅ **Resultado final**

$Q^+ = J\overline{Q} + \overline{K}Q$ y la tabla de arriba. Comprobada con las 8 combinaciones (Q, J, K) y con las 4 transiciones.

Demostración D8

D8 · Un contador módulo N divide la frecuencia entre N

✔ verificado

Demuestra que la salida de un contador módulo N (con una señal que vale 1 en un estado) tiene frecuencia f_{clk}/N y que se necesitan $n = \lceil \log_2 N \rceil$ biestables.

🧭 **Antes de empezar: qué piden y qué teoría usas**

Qué piden: relacionar cuenta, periodo y número de biestables. **Teoría:** T8.

1 Frecuencia

El contador pasa por N estados y vuelve al inicial **una vez cada N flancos**. Si una señal vale 1 en un solo estado, esa señal se repite cada N ciclos de reloj: periodo $N\,T_{clk}$, frecuencia f_{clk}/N. Un biestable T con $T = 1$ es el caso $N = 2$.

2 Número de biestables

Con n biestables hay 2^n combinaciones. Hacen falta N estados distintos: $2^n \geq N$, es decir $n \geq \log_2 N$. El mínimo entero es $n = \lceil \log_2 N \rceil$. Si N no es potencia de 2 sobran $2^n - N$ estados: hay que decidir adónde van (ver el **bloqueo** en los simulacros).

✅ **Resultado final**

$f_{out} = f_{clk}/N$ y $n = \lceil \log_2 N \rceil$. Comprobado para $N = 3, 5, 6, 10$.

Demostración D9

D9 · Contador síncrono binario: $T_i = Q_{i-1} \cdots Q_0$

✔ verificado

Demuestra que un contador binario síncrono de n bits con biestables T usa $T_i = Q_{i-1}Q_{i-2} \cdots Q_0$ (con $T_0 = 1$).

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: justificar cuándo conmuta cada bit al sumar 1. **Teoría:** T8.

1 Sumar 1 en binario

Al sumar 1 el bit 0 siempre cambia. El bit i cambia solo si **todos los inferiores valían 1** (hay acarreo hasta él), como en $0111 + 1 = 1000$.

2 Biestable T

Un T conmuta cuando $T = 1$. Así que $T_i = Q_{i-1} \cdots Q_0$, una AND de todos los bits menores. $T_0 = 1$ (AND vacía).

✅ Resultado final

$T_0 = 1, T_1 = Q_0, T_2 = Q_1Q_0, T_3 = Q_2Q_1Q_0$, … Comprobado con los 16 valores de un contador de 4 bits.

Demostración D10

D10 · Estados de los registros anillo, Johnson y LFSR

✔ verificado

Demuestra: (a) un registro en anillo de n biestables tiene n estados; (b) un contador Johnson tiene $2n$; (c) un LFSR de n bits tiene **como máximo** $2^n - 1$.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: contar los estados que visita cada registro. **Teoría:** T9.

1 (a) Anillo

Un único 1 que circula: hay n posiciones posibles y tras n desplazamientos vuelve al inicio. n estados.

2 (b) Johnson

La salida se realimenta **negada**. Empezando en $0\ldots0$ se van llenando de unos desde un lado (n estados) y luego se van vaciando (n estados más). Total $2n$.

3 (c) LFSR

La realimentación es una XOR de algunos bits. Con n bits hay 2^n valores, pero el estado $00\ldots0$ **se queda atascado** (la XOR de ceros es 0). Los demás 2^n-1 valores son el máximo posible. Se alcanza solo con ciertas posiciones de realimentación (polinomios primitivos).

✅ Resultado final

Anillo n, Johnson $2n$, LFSR $\leq 2^n-1$. Comprobados por simulación para $n=1\ldots8$ (anillo y Johnson) y con $n=3$ a 7 (LFSR, ciclo máximo de 2^n-1).

Demostración D11

D11 · Cuántas funciones hay de n variables y por qué un MUX universal

✔ verificado

Demuestra que hay 2^{2^n} funciones de n variables y que un multiplexor $2^k{:}1$ puede implementar cualquier función de $k+1$ variables.

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: dos conteos con el mismo truco: tabla de verdad. **Teoría:** T3, T5.

1 Número de funciones

La tabla de verdad tiene 2^n filas y en cada una la salida es 0 o 1, de forma independiente. Total $2\cdot2\cdots2=2^{2^n}$. Con $n=2$: $2^4=16$; con $n=4$: $2^{16}=65\,536$.

2 MUX 4:1 y tres variables

Se eligen A, B como selección. Para cada valor de (A,B) la función es una función de **solo** C, y solo hay cuatro: 0, 1, C y $\overline{C}$. Basta cablear una de ellas a cada entrada de datos.

3 Generalización

Un MUX $2^k{:}1$ usa k variables como selección. Para cada valor, la función restante depende de **una** variable más y solo puede ser 0, 1, x o $\overline{x}$.

✅ Resultado final

2^{2^n} funciones; MUX $2^k{:}1$ implementa cualquier función de $k+1$ variables (comprobado con las 256 funciones de 3 variables).

Demostración D12

D12 · Número de chips de memoria

✔ verificado

Una memoria de $16\text{K} \times 16$ se construye con chips de $4\text{K} \times 8$. ¿Cuántos chips hacen falta y cómo se conectan?

🧭 Antes de empezar: qué piden y qué teoría usas

Qué piden: ampliar palabras y ampliar profundidad. **Teoría:** T7.

1 Anchura

Se necesitan 16 bits de ancho con chips de 8: $16/8 = 2$ chips **en paralelo** (comparten dirección, aportan 8 bits cada uno).

2 Profundidad

$16\text{K}/4\text{K} = 4$ bloques. Se unen los 12 bits bajos de dirección a todos los chips y los 2 bits altos entran en un **decodificador 2:4** que activa la selección de un bloque.

3 Total

$4 \cdot 2 = 8$ chips. Equivale a $\dfrac{2^{14} \cdot 16}{2^{12} \cdot 8} = 8$.

✅ Resultado final

8 chips: 4 bloques de profundidad con 2 chips en paralelo cada uno. Dirección: 14 bits (12 a los chips + 2 al decodificador).

REPASO

Autoevaluación

Test por temas con explicación y tarjetas para memorizar definiciones y fórmulas.

Test por temas

Responde sin mirar la teoría. Al elegir una opción se marca la correcta y se muestra la **explicación**: léela aunque hayas acertado. Si fallas dos preguntas de un tema, vuelve a él antes de seguir.

🎯 **Tu puntuación**

0 correctas de 0 respondidas

Tema 2 · Sistemas de numeración, operaciones y códigos (4 preguntas)

¿Cuál es el rango de un entero en complemento a dos con 6 bits?

A de -31 a $+32$

B de -32 a $+32$

C de 0 a 63

D de -32 a $+31$

Con $n = 6$: $-2^5 = -32$ hasta $2^5 - 1 = 31$.

En C2 de 8 bits sumas $01100100 + 00110010$. ¿Qué ocurre?

A Hay un error de paridad

B Se produce acarreo y el resultado es correcto

C Hay desbordamiento: dos positivos dan un resultado con MSB igual a 1

D No hay problema: 150 cabe en 8 bits

$100 + 50 = 150 > 127$. El resultado 10010110 se lee como -106.

$0100\,0111$ en BCD representa:

- **A** 47
- **B** 0x47 en decimal, es decir, 71
- **C** 71
- **D** 4,7

Cada grupo de 4 bits es una cifra decimal: 4 y 7.

Para 3 bits, ¿qué código de Gray sigue a 011?

- **A** 000
- **B** 100
- **C** 010
- **D** 111

La secuencia es 000, 001, 011, 010, 110, 111, 101, 100.

Tema 3 · Álgebra de Boole y funciones lógicas (3 preguntas)

Según De Morgan, $\overline{A \cdot B + C}$ es igual a:

- **A** $\overline{A} + \overline{B} + C$
- **B** $(\overline{A} + \overline{B})\,\overline{C}$
- **C** $(\overline{A} + \overline{B}) + \overline{C}$
- **D** $\overline{A}\,\overline{B} + \overline{C}$

Se cambia + por · y se niega cada término: $\overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B})\overline{C}$.

En un mapa de Karnaugh, ¿cuántas casillas puede tener un grupo válido?

- **A** 1, 2, 4, 8...
- **B** Solo números pares
- **C** Cualquier número
- **D** Hasta un máximo de 4

Los grupos son potencias de 2.

¿Cuál es la forma simplificada de $AB + A\overline{B} + \overline{A}B$?

- **A** $\overline{A} + \overline{B}$
- **B** $A \oplus B$
- **C** $A + B$
- **D** AB

$AB + A\overline{B} = A$, y queda $A + \overline{A}B = A + B$.

Tema 4 · Familias lógicas (3 preguntas)

Una puerta tiene $V_{OH} = 2{,}7$ V y la siguiente exige $V_{IH} = 2{,}0$ V. El margen de ruido en alto es:

- **A** 0,7 V
- **B** 4,7 V
- **C** 0,3 V
- **D** 2,0 V

$NM_H = V_{OH} - V_{IH} = 0{,}7$ V.

Una salida da $I_{OL} = 16$ mA y $I_{OH} = 0{,}8$ mA; cada entrada pide $I_{IL} = 0{,}8$ mA e $I_{IH} = 40$ μA. El fan-out es:

A 40

B 1

C 16

D 20

$16/0{,}8 = 20$ y $0{,}8\ \text{mA}/40\ \mu\text{A} = 20$. El mínimo es 20.

Una puerta CMOS conmuta a 2 MHz con $C = 10$ pF y $V_{DD} = 3$ V. Su potencia dinámica es:

A 0,18 mW

B 1,8 mW

C 0,9 mW

D 0,06 mW

$P = 10 \cdot 10^{-12} \cdot 9 \cdot 2 \cdot 10^{6} = 1{,}8 \cdot 10^{-4}$ W $= 0{,}18$ mW.

Tema 5 · Funciones de la lógica combinacional (3 preguntas)

Para implementar una función de 4 variables con un solo multiplexor, ¿cuál es el mínimo?

A Imposible con un solo MUX

B MUX 4:1

C MUX 16:1 y nada más

D MUX 8:1

Con $2^{n-1} = 8$ entradas de datos y 3 variables en selección; la cuarta va a los datos.

Un decodificador 3:8 tiene la entrada 101. ¿Qué salida se activa?

A S_6

B S_1

C S_5

D S_3

$101_2 = 5$. Se activa la salida S_5.

Un sumador de 8 bits con acarreo en cascada tiene un retardo de 15 ns por etapa. El retardo del acarreo es:

A 120 ns

B 23 ns

C 15 ns

D 8 ns

$8 \cdot 15 = 120$ ns: el acarreo atraviesa las 8 etapas.

Tema 6 · Biestables (flip-flops) y dispositivos relacionados (3 preguntas)

Un biestable JK tiene $Q = 1$ y recibe $J = 0$, $K = 0$. Tras el flanco:

A $Q = 1$ y luego conmuta

B $Q = 1$ (mantiene)

C $Q = 0$

D Estado prohibido

$J = K = 0$ mantiene. Con $Q^+ = J\overline{Q} + \overline{K}Q = 1$.

Quieres pasar de $Q = 1$ a $Q^+ = 0$ con un biestable T. ¿Qué pones en T?

A $T = X$

B $T = 1$

C $T = 0$

D No es posible

Para cambiar de estado, $T = 1$.

Un biestable T con $T = 1$ recibe un reloj de 8 MHz. La salida tiene frecuencia:

A 2 MHz

B 8 MHz

C 4 MHz

D 16 MHz

Conmuta en cada flanco: la frecuencia se divide por 2.

Tema 7 · Memorias y circuitos lógicos programables (3 preguntas)

¿Cuántas líneas de dirección tiene una memoria de 32K×8?

A 15

B 32

C 5

D 8

$32\text{K} = 2^5 \cdot 2^{10} = 2^{15}$.

¿Cuántos chips de 4K×4 hacen falta para una memoria de 8K×8?

A 2

B 16

C 4

D 8

A lo ancho, $8/4 = 2$; a lo largo, $8\text{K}/4\text{K} = 2$. Total $2 \cdot 2 = 4$.

Una ROM de 8 posiciones y 2 bits puede implementar:

A Un sumador completo (3 entradas, 2 salidas)

B Un contador de 8 bits

C Un biestable

D Cualquier función de 4 entradas

Tiene 3 líneas de dirección (entradas) y 2 bits de dato (salidas).

Tema 8 · Contadores (3 preguntas)

¿Cuántos biestables necesita un contador de módulo 100?

A 100

B 10

C 6

D 7

$2^6 = 64 < 100 \leq 2^7 = 128$.

Un contador binario síncrono de 4 bits tiene $Q_3Q_2Q_1Q_0 = 0111$. Tras el siguiente flanco:

A 1111

B 0110

C 1000

D 0000

$T_3 = Q_2Q_1Q_0 = 1, T_2 = Q_1Q_0 = 1, T_1 = Q_0 = 1, T_0 = 1$: todos conmutan.

Un contador asíncrono de 5 bits con $t_{pd} = 10$ ns por etapa: ¿cuánto tarda en estabilizarse como máximo?

A 5 ns

B 32 ns

C 10 ns

D 50 ns

$5 \cdot 10 = 50$ ns.

Tema 9 · Registros de desplazamiento (2 preguntas)

Un contador Johnson de 5 bits tiene:

A 5 estados

B 10 estados

C 32 estados

D 25 estados

$2n = 10.$

Un registro de desplazamiento de 8 bits recibe un dato serie. ¿Cuántos flancos de reloj hacen falta para cargar una palabra completa?

A 3

B 8

C 16

D 1

Entra un bit por flanco.

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos (4 preguntas)

Un autómata tiene 6 estados. ¿Cuántos biestables necesita como mínimo?

A 3

B 6

C 4

D 2

$2^2 = 4 < 6 \leq 2^3 = 8.$

En una máquina de Mealy, la salida:

A Depende solo del estado

B Depende solo de la entrada

C Depende del estado y de la entrada actual

D Es siempre síncrona

Esa es la definición; en Moore depende solo del estado.

Con biestables D, ¿cuál es la entrada D de un biestable que debe pasar de 0 a 1?

A $D = \overline{Q}$

B $D = 0$

C $D = X$

D $D = 1$

En un biestable D, $D = Q^{+}$.

Un pulsador rebota 10 ms. Con reloj de 100 MHz, ¿cuántos ciclos debe esperar el filtro para ser fiable?

A Al menos 10^{6}

B 10^{9}

C 100

D 10^{3}

$10 \cdot 10^{-3}\ \mathrm{s} \cdot 100 \cdot 10^{6}\ \mathrm{Hz} = 10^{6}$ ciclos.

Tema 11 · Descripciones funcionales y estructurales (2 preguntas)

Un archivo VHDL que describe un sumador con la expresión `s <= a + b;` es una descripción:

- A Física
- B De transistores
- C Estructural
- D Funcional (de comportamiento)

Dice qué calcula, no con qué puertas.

¿Qué paso del flujo de diseño comprueba que el circuito hace lo que debe, sin tener en cuenta los retardos?

- A La simulación funcional
- B La síntesis
- C La implementación
- D La programación de la placa

La simulación funcional ignora los retardos reales; la temporal los incluye.

Tema 12 · Introducción a los dispositivos lógicos programables (2 preguntas)

Una LUT de 5 entradas almacena:

- A 16 bits
- B 32 bits
- C 5 bits
- D 25 bits

$2^5 = 32$ posiciones de un bit.

¿Qué hace falta para que un diseño aparezca en los LEDs de la placa?

A Solo simularlo

B Asignar las señales a los pines de la FPGA

C Cambiar el reloj de la placa

D Nada más que sintetizarlo

Sin las restricciones de pines, la herramienta coloca las señales donde quiere.

Tema 13 · Introducción al diseño digital con VHDL (3 preguntas)

En VHDL, `y <= a and b;` describe:

A Una puerta AND siempre presente

B Un proceso secuencial

C Una variable temporal

D Una instrucción que se ejecuta una sola vez

Es una asignación concurrente: hardware que existe de forma permanente.

¿Qué provoca un `if` sin `else` dentro de un proceso combinacional?

A Una puerta NOT

B Un error de sintaxis siempre

C Un latch inferido

D Un contador

La salida mantiene su valor en la rama no cubierta, y eso es memoria.

Un contador módulo 10 en VHDL necesita como mínimo una señal de:

A 5 bits

B 10 bits

C 3 bits

D 4 bits

$\lceil \log_2 10 \rceil = 4$.

Tarjetas para memorizar

Pulsa una tarjeta para darle la vuelta. Intenta contestar **en voz alta** antes de girarla.

¿Rango de un número en C2 de n bits?
-2^{n-1} a $2^{n-1} - 1$.

¿Cómo se niega un número en C2?
Invertir todos los bits y sumar 1.

¿Cuándo hay desbordamiento en C2?
Cuando sumas dos números del mismo signo y el resultado tiene signo contrario.

¿Corrección en la suma BCD?
Si la cifra pasa de 9 (o hay acarreo), sumar 0110.

¿Qué caracteriza al código Gray?
Entre dos códigos consecutivos cambia un solo bit.

¿Enuncia De Morgan?
$\overline{A + B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.

¿Cuántas funciones lógicas distintas hay de n variables?
2^{2^n}.

¿Qué son puertas universales?
NAND y NOR: con una sola se construye cualquier función.

¿Por qué las columnas de Karnaugh van en orden 00,01,11,10?

Para que las casillas vecinas difieran en un solo bit (código Gray).

¿Cómo se usan las casillas indiferentes?

Solo si permiten hacer un grupo mayor; no hay que cubrirlas.

¿Margen de ruido en nivel alto?

$NM_H = V_{OH} - V_{IH}$.

¿Cómo se calcula el fan-out?

El menor de I_{OL}/I_{IL} y I_{OH}/I_{IH}.

¿Para qué sirve el colector abierto?

Permite unir varias salidas en un cable (AND cableada) usando una resistencia pull-up.

¿Qué es el estado Z?

Alta impedancia: la salida queda desconectada. Permite compartir un bus.

¿De qué depende la potencia dinámica de CMOS?

De la capacidad, de la tensión al cuadrado y de la frecuencia.

¿Ecuación del acarreo de un sumador completo?

$C_{out} = AB + C_{in}(A \oplus B)$.

¿Cómo se resta con un sumador?

$A + \overline{B} + 1$: se invierte B y se pone $C_{in} = 1$.

¿Qué número de selección necesita un MUX de 8 a 1?

3 bits ($2^3 = 8$).

¿Qué es cada salida de un decodificador?

Un minitérmino de las entradas.

¿Qué hace un codificador de prioridad?

Entrega el código de la entrada activa de mayor índice cuando hay varias.

¿Ecuación característica del JK?

$Q^+ = J\overline{Q} + \overline{K}Q$.

¿Qué hace un JK con J=K=1?

Conmuta en cada flanco.

¿Entradas del JK para pasar de 1 a 0?

$J = X$ (cualquiera) y $K = 1$.

¿Qué es el tiempo de setup?

Tiempo que la entrada debe estar estable antes del flanco de reloj.

¿Qué diferencia hay entre latch y flip-flop?

El latch es transparente mientras está habilitado; el flip-flop muestrea solo en el flanco.

¿Cuántos bits tiene una memoria de $2^k \times m$?

$m \cdot 2^k$ bits.

¿Cuántas líneas de dirección tiene una memoria de 16K posiciones?

14 ($2^{14} = 16384$).

¿Qué diferencia hay entre ROM y RAM?

La ROM solo se lee y no se borra al apagar; la RAM se lee y escribe y (volátil) se borra.

¿Qué plano es fijo en una PAL?

El plano OR.

¿Cuántos biestables necesita un contador de módulo 60?

6 ($2^5 = 32 < 60 \leq 64$).

¿Ecuación de T_2 en un contador síncrono binario?

$T_2 = Q_1 Q_0$.

¿Ventaja del contador síncrono?

No acumula retardos de propagación y no genera glitches.

¿Qué es un glitch?

Un valor erróneo fugaz mientras los biestables de un contador asíncrono cambian en cadena.

¿Frecuencia de salida de un divisor por N?

f_{clk}/N.

¿Cuántos estados tiene un contador Johnson de n bits?

$2n$.

¿Qué entra en el primer biestable de un contador en anillo?

La salida del último.

¿Qué es SIPO?

Registro serie-entrada, paralelo-salida.

¿Periodo máximo de un LFSR de n bits?

$2^n - 1$.

¿Diferencia entre Moore y Mealy?

En Moore la salida solo depende del estado; en Mealy, del estado y de la entrada.

¿Cuántos biestables para 9 estados?

4 ($2^3 = 8 < 9 \leq 16$).

¿Entradas D para un estado destino Q^+?

$D = Q^+$.

¿Entrada T para pasar de Q a Q^+?

$T = Q \oplus Q^+$.

¿Qué haces con las combinaciones de código sin usar?

Las marcas como indiferentes (X) en Karnaugh.

¿Qué es una descripción estructural?

Dice de qué componentes consta el circuito y cómo se conectan.

¿Qué es una descripción funcional?

Dice qué hace el circuito sin indicar su estructura interna.

¿Qué hace la síntesis?

Convierte la descripción en puertas y biestables.

¿Qué almacena una LUT de 4 entradas?

16 bits: la tabla de verdad de una función de 4 variables.

¿Qué ventaja tiene la FPGA sobre un microcontrolador?

Paralelismo real: cada bloque es hardware que funciona a la vez.

¿Qué reloj trae la placa Basys?

50 MHz.

¿Qué pasa con una FPGA SRAM al apagarse?

Pierde la configuración y hay que volver a cargarla.

¿Qué es una entity?

La interfaz del módulo: nombre y puertos.

¿Qué es una architecture?

La descripción de lo que hay dentro del módulo.

¿Qué infiere VHDL si no asignas una salida en todas las ramas?

Un latch.

¿Cuándo toma efecto una asignación a una signal?

Al terminar el proceso (no al instante).

¿Qué sentencia detecta un flanco de subida?

`rising_edge(clk)`.

REPASO

Test del libro de problemas

Preguntas tipo test sobre los capítulos 1 a 8 del libro de problemas (códigos, álgebra de Boole, Karnaugh, diseño combinacional, bloques MSI, biestables, registros, contadores y autómatas). Cada respuesta está comprobada con código.

Responde sin mirar la solución. Al elegir una opción se marca la correcta y se explica por qué. Cada bloque corresponde a un capítulo de la sección *Libro de problemas resueltos*; si fallas dos preguntas de uno, repasa sus problemas.

Capítulo 1 · Sistemas de numeración y códigos

El número decimal 37 escrito en binario puro es:

- A 010101
- B 101001
- C 100101
- D 100110

$37 = 32 + 4 + 1$, es decir, 100101_2. Las otras opciones valen 41, 38 y 21.

El hexadecimal $B7_{16}$ vale en decimal:

- A 187
- B 171
- C 247
- D 183

$B7_{16} = 11 \cdot 16 + 7 = 183$. 171 es AB_{16} y 247 es $F7_{16}$.

El número decimal 59 en BCD natural es:

A 1001 0101

B 0101 1001

C 0011 1011

D 0101 1010

BCD codifica **cada dígito** por separado: $5 \to 0101$ y $9 \to 1001$. $0011\,1011$ sería el binario puro de 59 cortado en dos; $1001\,0101$ es el 95; 1010 no es un dígito BCD.

El dígito 5 en código exceso 3 es:

A 0101

B 1000

C 0010

D 1011

Exceso 3 es el BCD del dígito más 3: $5 + 3 = 8 = 1000$. 0101 sería el BCD natural y 1011 el Aiken.

En el código Aiken (2421) el dígito 7 se escribe:

A 1100

B 1101

C 1110

D 1011

Con pesos $2, 4, 2, 1$ de izquierda a derecha, $1101 = 2 + 4 + 0 + 1 = 7$. En Aiken los dígitos 0–4 coinciden con el binario y los 5–9 son el complemento de los 4–0; 1011 es el 5, 1100 el 6 y 1110 el 8.

El código Gray de la combinación binaria 1011 es:

A 1101

B 0110

C 1110

D 1010

El bit de mayor peso se copia y cada bit siguiente es el XOR de dos bits consecutivos del binario: $1,\ 1 \oplus 0 = 1,\ 0 \oplus 1 = 1,\ 1 \oplus 1 = 0$, es decir 1110.

El rango de números enteros con signo que se pueden representar en complemento a 2 con 6 bits es:

A 0 a 63

B -31 a $+31$

C -32 a $+31$

D -32 a $+32$

Con n bits en C2: de -2^{n-1} a $2^{n-1}-1$. Con 6 bits, de -32 a $+31$. El rango simétrico -31 a $+31$ es el de signo y magnitud o el de C1.

La combinación de 4 bits 1011 leída en complemento a 2 vale:

A $+11$

B -4

C -3

D -5

Con signo negativo: $1011 \rightarrow$ se invierte y se suma 1: $0100 + 1 = 0101 = 5$, luego -5. Como C1 valdría -4, como signo y magnitud -3, y sin signo 11.

Se suman en C2 con 4 bits $0101 + 0100$ ($5 + 4$). Resulta:

A Desbordamiento: dos positivos dan 1001, que se lee como negativo

B Resultado correcto $+9$

C Resultado correcto -7

D No hay desbordamiento porque no sale acarreo

El resultado 1001 no se puede leer como $+9$ (el máximo en C2 de 4 bits es $+7$): **el signo del resultado es distinto del de los sumandos**, y eso indica desbordamiento aunque no salga acarreo.

En binario sin signo con 4 bits se suma $1011 + 0111$ ($11 + 7$). Resulta:

A 1010 sin desbordamiento

B 0010 con acarreo de salida: desbordamiento

C 0010 sin desbordamiento

D 10010 cabe en 4 bits

$11 + 7 = 18$ no cabe en 4 bits (máximo 15). Se obtiene 10010_2: se queda 0010 y el acarreo de salida vale 1, lo que en sin signo indica **desbordamiento**.

El complemento a 10 del número decimal 374 (con 3 dígitos) es:

A 626

B 1626

C 625

D 636

C10 $= 10^3 - 374 = 626$. El complemento a 9 sería 625 (a cada dígito se le resta de 9) y C10 es el C9 más 1.

El complemento a 2 de 0110 $(+6)$ con 4 bits es:

A 1110

B 0110

C 1001

D 1010

Se invierten los bits (1001) y se suma 1: 1010, que en C2 vale -6. 1001 sería el complemento a 1.

Capítulo 2 · Álgebra de Boole y funciones lógicas

Aplicando De Morgan, $\overline{A + B \cdot C}$ es igual a:

A $\overline{A} \cdot \overline{B} + \overline{C}$

B $\overline{A} \cdot (\overline{B} + \overline{C})$

C $\overline{A} \cdot \overline{B} \cdot \overline{C}$

D $\overline{A} + \overline{B} \cdot \overline{C}$

La suma exterior pasa a producto y la parte interior $\overline{B \cdot C} = \overline{B} + \overline{C}$. Se verifica con las 8 combinaciones.

La función de tres variables $f(A, B, C) = \sum m(1, 3, 5, 7)$ (con A el bit de más peso) es igual a:

A $\overline{C}$

B A

C $A + B$

D C

Los minitérminos 1, 3, 5 y 7 son los de índice impar: es decir, los que tienen $C = 1$. La función es $f = C$.

¿Cuántas puertas NAND de 2 entradas necesita una función $f = AB + CD$ en dos niveles si solo hay NAND?

A 3

B 4

C 5

D 2

$f = \overline{\overline{AB} \cdot \overline{CD}}$: una NAND por cada producto y una NAND final que los combina. Las entradas se usan sin negar.

Con solo puertas NOR, el producto de sumas $(A + B)(C + D)$ se implementa con:

A Tres NOR: dos primeras y una que las combina

B Tres NAND: dos primeras y una que las combina

C Cuatro NOR en cuatro niveles

D Una sola NOR de cuatro entradas

$(A + B)(C + D) = \overline{\overline{A + B} + \overline{C + D}}$: dos NOR y una tercera NOR. Es el dual del caso de la suma de productos con NAND.

En un codificador de 8 a 3 hecho con puertas OR, el bit de menos peso A_0 vale:

A $D_0 + D_2 + D_4 + D_6$

B $D_4 + D_5 + D_6 + D_7$

C $D_2 + D_3 + D_6 + D_7$

D $D_1 + D_3 + D_5 + D_7$

$A_0 = 1$ para las entradas cuyo índice en binario termina en 1: 1, 3, 5 y 7. $D_4 + D_5 + D_6 + D_7$ es A_2 y $D_2 + D_3 + D_6 + D_7$ es A_1.

Un decodificador de 3 a 8 con habilitación activa recibe la entrada 101. Las salidas que se activan son:

A Ninguna, porque 101 es impar

B Las salidas 0 a 5

C Solo la salida 5

D Las salidas 1 y 4

Un decodificador activa **exactamente una** salida: la que corresponde al número de entrada ($101_2 = 5$). Si la habilitación estuviese inactiva no se activaría ninguna.

Un generador de paridad par para el dato de 7 bits 1101001 añade un bit de paridad igual a:

A El que corresponda al bit de más peso

B 0

C No se puede calcular con menos de 8 bits

D 1

El dato tiene 4 unos (par), luego el bit añadido vale 0 para que el total siga siendo par. El bit es el **XOR de los 7 bits**: $1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0$.

El teorema de absorción dice que $A + A \cdot B$ vale:

A $A \cdot B$

B B

C A

D $A + B$

Si $A = 1$ la suma da 1; si $A = 0$ el producto $AB = 0$ y la suma da 0. En los dos casos coincide con A.

Si $f(A, B, C) = \sum m(0, 2, 5)$, su forma en producto de maxitérminos es:

A $\prod M(1, 3, 5, 7)$

B $\prod M(4, 6, 7)$

C $\prod M(1, 3, 4, 6, 7)$

D $\prod M(0, 2, 5)$

Los maxitérminos son los índices donde la función vale 0, es decir, el resto de índices de 0 a 7: 1, 3, 4, 6 y 7.

La expresión $\overline{A} \cdot \overline{B}$ es equivalente a la salida de una puerta:

A XNOR de A y B

B NAND de A y B

C NOR de A y B

D OR de $\overline{A}$ y $\overline{B}$

Por De Morgan, $\overline{A}\,\overline{B} = \overline{A + B}$, que es la NOR. $\overline{A} + \overline{B}$ sería la NAND.

En un mapa de Karnaugh de 4 variables, un grupo de 8 unos adyacentes produce un término con:

- **A** Un solo literal
- **B** Ninguno: la función vale siempre 1
- **C** Tres literales
- **D** Dos literales

Un grupo de 2^k casillas elimina k variables. Con $k = 3$ se eliminan tres de las cuatro y queda un literal.

En tres variables, $f = \sum m(0, 1, 2, 3)$ (con A el bit de más peso) se simplifica a:

- **A** $\overline{B}$
- **B** A
- **C** 1
- **D** $\overline{A}$

Los minitérminos 0–3 son los que tienen $A = 0$: cuatro casillas adyacentes, un grupo que elimina B y C y deja $\overline{A}$.

Con combinaciones indiferentes en un mapa de Karnaugh, se incluyen en un grupo:

- **A** Ninguna: no se pueden usar
- **B** Solo las que están más cerca del origen
- **C** Solo las que permiten hacer el grupo más grande
- **D** Todas, siempre

Las X se toman como 1 **si ayudan** a formar un grupo mayor (menos literales) y se ignoran en caso contrario. No hay obligación de cubrirlas.

En producto de sumas, un mapa de tres variables con ceros solo en m_0 y m_1 da:

A $A \cdot B$

B $\overline{A} + \overline{B}$

C $A + B$

D $\overline{A} \cdot \overline{B}$

Se agrupan los ceros: $m_0 = 000$ y $m_1 = 001$ forman un grupo con $A = 0$ y $B = 0$. En una **suma**, las variables que valen 0 aparecen **sin negar**: $A + B$.

En un mapa de Karnaugh de 4 variables, las cuatro esquinas:

A Son adyacentes entre sí y forman un grupo posible

B Solo se agrupan si son indiferentes

C No se pueden agrupar

D Forman un grupo de 2 casillas

El mapa es un toroide: la primera y la última fila, y la primera y la última columna, son adyacentes. Las cuatro esquinas difieren solo en dos variables y forman un grupo de 4.

Las filas y las columnas de un mapa de Karnaugh se ordenan $00, 01, 11, 10$ porque:

A Casillas contiguas deben diferir solo en una variable (orden Gray)

B Es el orden del binario natural

C Así se simplifica siempre a una sola puerta

D Lo exige la forma producto de sumas

Con el orden Gray, dos casillas vecinas son combinaciones adyacentes, que se pueden agrupar y eliminar la variable que cambia.

En 4 variables ($ABCD$, A el de más peso), $f = \sum m(0,1,2,3,8,9,10,11)$ vale:

A $\overline{B}$

B $\overline{C}$

C $\overline{B}\,\overline{D}$

D $\overline{A}$

Los ocho minitérminos son los que tienen $B = 0$: en el mapa forman dos filas contiguas por el borde (un grupo de 8) y se simplifica a $\overline{B}$.

Un implicante primo es esencial cuando:

A Cubre alguna combinación indiferente

B Cubre algún 1 que ningún otro implicante primo cubre

C Tiene el mayor número de casillas del mapa

D No comparte casillas con ningún otro

Los implicantes primos esenciales hay que incluirlos siempre: son los únicos que cubren cierto 1 del mapa. El resto se elige para cubrir lo que falte con el menor coste.

Para obtener la forma mínima como producto de sumas se agrupan en el mapa:

A Los ceros

B Los unos

C Las combinaciones indiferentes

D Las filas completas

El producto de sumas se obtiene agrupando los ceros: cada grupo da una suma y la función es el producto de todas ellas. Los unos darían una suma de productos.

Capítulo 4 · Diseño combinacional con puertas

En el voto mayoritario de tres sensores, la salida vale 1 en:

- A 3 de las 8
- B 5 de las 8
- C Solo en la combinación 111
- D 4 de las 8 combinaciones

Vale 1 si hay al menos dos unos: $011, 101, 110, 111$. Es $f = AB + AC + BC$.

En el complemento a 2 de un número de 4 bits, el bit de menor peso de la salida es:

- A El opuesto al de la entrada
- B Siempre 1
- C Siempre 0
- D Igual que el de la entrada

Al invertir y sumar 1, el bit de menos peso se invierte y vuelve a invertirse con el +1: queda igual. En cambio, los demás bits dependen de los de menos peso.

En un circuito que da el complemento a 9 de un dígito BCD, la entrada 0100 (4) produce la salida:

- A 0101
- B 1011
- C 0100

D 0110

El complemento a 9 es $9 - d$: $9 - 4 = 5 = 0101$. Las entradas 1010 a 1111 no son dígitos BCD y se toman como indiferentes en el diseño.

Un multiplicador de dos números de 2 bits necesita una salida de:

A 5 bits

B 3 bits

C 2 bits

D 4 bits

El producto máximo es $3 \cdot 3 = 9 = 1001_2$, que necesita 4 bits.

En un comparador de dos números de 2 bits (A_1A_0 y B_1B_0), la salida «A es igual a B» es:

A $(A_1 \odot B_1) + (A_0 \odot B_0)$

B $(A_1 \odot B_1) \cdot (A_0 \odot B_0)$

C $A_1B_1 + A_0B_0$

D $(A_1 \oplus B_1) \cdot (A_0 \oplus B_0)$

Son iguales si **cada bit** lo es: XNOR bit a bit y un AND de los resultados. Con XOR se detectaría que todos los bits son distintos.

En un sumador completo de 1 bit, el acarreo de salida es:

A $C_{out} = A \cdot B \cdot C_{in}$

B $C_{out} = A \oplus B \oplus C_{in}$

C $C_{out} = A + B + C_{in}$

D $C_{out} = AB + C_{in}(A \oplus B)$

Hay acarreo si A y B valen 1, o si uno de los dos vale 1 y viene acarreo de entrada. La suma $A \oplus B \oplus C_{in}$ es la salida S, no el acarreo.

Con solo puertas NAND, una AND de dos entradas necesita:

A 1 NAND

B 3 NAND

C No se puede hacer

D 2 NAND

$A \cdot B = \overline{\overline{AB}}$: una NAND y otra NAND con las dos entradas unidas, que hace de inversor.

En un divisor de dos números de 2 bits hay una combinación que hay que tratar aparte. Es:

A Divisor igual a 00 (división por cero)

B Dividendo 11 y divisor 11

C Dividendo igual al divisor

D Dividendo igual a 00

Dividir por 0 no está definido: la salida correspondiente suele marcarse como indiferente o con un indicador de error. El resto de combinaciones tienen cociente y resto definidos.

En un decodificador de BCD a 7 segmentos, el dígito 1 enciende los segmentos:

A e y f

B b y c

C a y d

D a, b y c

Los segmentos b y c son los dos verticales de la derecha, que dibujan el 1. El dígito 7 es el que enciende a, b y c.

Capítulo 5 · Bloques combinacionales MSI

Un sumador de 4 bits recibe $A = 0111$, $B = 0101$ y $C_{in} = 0$. Su salida es:

A $S = 1100$ y $C_{out} = 1$

B $S = 1100$ y $C_{out} = 0$

C $S = 1110$ y $C_{out} = 0$

D $S = 0010$ y $C_{out} = 1$

$7 + 5 = 12 = 1100_2$, que cabe en 4 bits: no hay acarreo de salida.

Para calcular $A - B$ (sin signo, 4 bits) con un sumador se hace:

A $A + \overline{B} + 1$, y el acarreo de salida vale 1 si $A \geq B$

B $A + B + 1$, y el acarreo vale 1 si $A \geq B$

C $\overline{A} + B + 1$, y el acarreo vale 1 si $A \geq B$

D $A + \overline{B}$, y el acarreo vale 1 si $A < B$

Restar es sumar el complemento a 2: $\overline{B} + 1$. Si el acarreo de salida vale 1 el resultado es positivo o cero ($A \geq B$); si vale 0, no hay «llevada» y es negativo.

Para implementar una función cualquiera de 4 variables con un multiplexor 8:1 se llevan a las líneas de selección:

A 2 variables y las otras 2 a los datos

B 3 variables, y la cuarta (o 0, 1 o su complemento) a las entradas de datos

C Las 4 variables a la selección

D Solo se puede con un 16:1

Un 8:1 tiene 3 selecciones. Se eligen 3 variables como selección y, para cada una de las 8 combinaciones, la función restante depende solo de la cuarta: vale 0, 1, la variable o su complemento.

Un codificador decimal a exceso 3 sin prioridad recibe a la vez las líneas 9 y 0. La salida es:

A 0000

B 0011

C 1100

D 1111

Cada línea activa pone a 1 los bits de su código: el 9 da 1100 y el 0 da 0011. Sin prioridad, las salidas se combinan con OR: 1111, que ni siquiera es un código válido.

Se suman dos dígitos BCD, $7 + 8$, con un sumador binario. Para obtener el resultado BCD hay que:

A Sumar 3 a cada dígito

B Restar 6

C Sumar 6: $1111 + 0110$ da $0001\,0101$

D No hace falta corrección: 1111 es BCD

La suma binaria es 1111 (15), que no es un dígito BCD. Sumando 6 se salta de 1001 a 0000 con acarreo: $1111 + 0110 = 0001\,0101$, que es 15 en BCD.

En una suma en C2, hay desbordamiento cuando:

A El acarreo que entra al bit de signo y el que sale son distintos

B Siempre que $C_{out} = 1$

C Siempre que $C_{out} = 0$

D Solo si los dos sumandos son negativos

Se comprueba para las 256 sumas de 4 bits: coincide con $C_{in,3} \neq C_{out}$. Que salga acarreo no basta: $-1 + (-1)$ da acarreo y es correcto.

Un circuito de valor absoluto de 4 bits (C2) recibe 1000 (-8). La salida es:

A 0000

B 1000, que sigue valiendo -8

C 0111

D 1000, que ahora vale $+8$

El $+8$ no es representable en 4 bits C2 (máximo $+7$). Al negar 1000 resulta 1000 otra vez: es el único caso en que el valor absoluto es incorrecto.

Un multiplexor 4:1 tiene:

A 4 de datos y 4 de selección

B 2 entradas de datos y 4 de selección

C 4 de datos y 1 de selección

D 4 entradas de datos, 2 de selección y 1 salida

Para elegir entre 4 entradas hacen falta $\log_2 4 = 2$ líneas de selección.

Un decodificador de BCD a 7 segmentos solo define su salida para:

A Solo 0000 a 0111

B Las 16 entradas por igual

C Solo las entradas pares

D Las entradas 0000 a 1001; las otras seis son combinaciones no usadas

Hay 10 dígitos BCD. Las entradas 1010 a 1111 nunca se dan y en el diseño se toman como indiferentes (o se usan para mostrar símbolos o apagar el display).

Un sumador de 4 bits recibe $A = 1111$ y $B = 0001$ con $C_{in} = 0$. La salida es:

A $S = 0000$ y $C_{out} = 1$

B $S = 1111$ y $C_{out} = 0$

C $S = 0000$ y $C_{out} = 0$

D $S = 1110$ y $C_{out} = 1$

$15 + 1 = 16 = 10000_2$: los cuatro bits bajos valen 0 y se genera el acarreo de salida.

Capítulo 6 · Biestables y cronogramas

En una báscula R-S asíncrona con puertas NAND (entradas activas en bajo), la combinación $\overline{S} = \overline{R} = 0$ da:

A Las dos salidas a 1: situación prohibida

B $Q = 0$ y $\overline{Q} = 1$

C $Q = 1$ y $\overline{Q} = 0$

D Mantiene el estado anterior

Cada NAND con una entrada a 0 da 1: $Q = \overline{Q} = 1$. Es prohibida porque las dos salidas dejan de ser complementarias.

En la báscula R-S de NAND, las dos entradas pasan a la vez de $\overline{S}\,\overline{R} = 00$ a 11. La salida:

A Se mantiene en el valor anterior a la prohibida

B Queda en un valor desconocido a priori

C Queda siempre a 1

D Queda siempre a 0

Las dos puertas ven «1,1» a la vez y compiten: la que gane se queda a 0 y la otra a 1, sin que se pueda saber cuál. Si solo cambia una entrada, el resultado sí es único.

Una báscula J-K asíncrona con $J = K = 1$ mantenido produce:

A Q a 1

B Carreras: la salida bascula continuamente

C Q a 0

D Q invertida una sola vez

Sin reloj que acote el instante, la salida se realimenta y cambia una y otra vez mientras dure $JK = 11$. Por eso se necesita el sincronismo por flanco o el maestro/esclavo.

Un R-S síncrono por flanco encuentra $S = R = 1$ justo antes del flanco activo. Su salida es:

A Q invertida

B Q a 0

C Q a 1

D Indeterminada hasta una nueva orden clara

Es la situación prohibida del R-S. Con sincronismo por flanco no hay una puerta que «acabe» antes: la salida queda desconocida hasta que un flanco posterior dé 10 o 01.

Un J-K por flanco de subida con $J = K = 1$ antes del flanco:

A Mantiene la salida

B Entra en carreras

C Invierte la salida una vez en ese flanco

D Queda indeterminado

En un J-K por flanco, $JK = 11$ es la orden de **conmutar**: $Q^{+} = \overline{Q}$. Al ser un instante, no hay carreras.

Un maestro/esclavo J-K equivale a un J-K por flanco si:

A J y K no cambian durante el nivel activo del maestro

B Solo si el reloj es simétrico

C Solo si $J = K$

D Siempre, sea cual sea el cronograma

El maestro «piensa» durante su nivel activo y puede ser sensible a cambios de J y K en ese tiempo, mientras que el J-K por flanco solo mira un instante. En el problema del libro, un $JK = 11$ seguido de 10 da 0 en el M/S y 1 por flanco.

En el maestro de un J-K maestro/esclavo, las entradas del R-S interno son:

A $S_M = J \cdot Q_S$ y $R_M = K \cdot \overline{Q_S}$

B $S_M = J$ y $R_M = K$

C $S_M = J + \overline{Q_S}$ y $R_M = K + Q_S$

D $S_M = J \cdot \overline{Q_S}$ y $R_M = K \cdot Q_S$

Realimentar la salida del esclavo impide que S_M y R_M valgan 1 a la vez: solo se puede **subir** si $Q_S = 0$ y **bajar** si $Q_S = 1$. Con $JK = 11$ el maestro toma el opuesto del esclavo.

Un latch D con línea ENABLE a 1 y D con pulsos cortos:

A La salida ignora D mientras ENABLE vale 1

B La salida sigue a D, incluidos los pulsos cortos

C La salida solo copia D en los flancos de ENABLE

D La salida queda indeterminada

Por nivel, el latch D es **transparente** mientras ENABLE está activo y guarda el último valor al cerrarse. No hay situaciones prohibidas porque solo tiene una entrada.

Dos biestables D por flanco de subida en cadena (Q_1 copia a Ent, Q_2 copia a Q_1) con una AND $Sal = \overline{Q_1} \cdot Q_2$ forman:

A Un detector de flancos descendentes: pulso de un periodo de reloj

B Un detector de flancos ascendentes

C Un divisor de frecuencia por 2

D Un contador de pulsos

$Sal = 1$ solo cuando Q_1 ya ha bajado y Q_2 aún recuerda el 1: durante exactamente un periodo T tras cada flanco de bajada de Ent.

Con los mismos dos D, para detectar flancos ascendentes se usa la salida:

A $Sal = \overline{Q_1} \cdot Q_2$

B $Sal = Q_1 \cdot Q_2$

C $Sal = Q_1 \oplus Q_2$ con un solo D

D $Sal = Q_1 \cdot \overline{Q_2}$

Tras un flanco de subida de Ent, $Q_1 = 1$ pero Q_2 aún vale 0 durante un periodo. $Q_1 \oplus Q_2$ detectaría los dos tipos de flanco.

Un flip-flop T por flanco con $T = A \cdot \overline{Q} + B \cdot Q$ se comporta como:

A Un T con $T = A + B$

B Un R-S con $S = A$ y $R = B$

C Un J-K con $J = A$ y $K = B$

D Un D con $D = A$

Si $A = 1$ y $Q = 0$ hay que subir (invertir); si $B = 1$ y $Q = 1$ hay que bajar (invertir). Se comprueba con las 8 filas: $Q^+ = J\overline{Q} + \overline{K}Q$.

El circuito $Q = A \cdot B + \overline{A} \cdot Q$ (con Q realimentada) es:

A Un latch D por nivel con $A = Ck$ y $B = D$

B Un R-S asíncrono

C Un J-K síncrono por nivel

D Un T por flanco

Con $A = 0$ mantiene ($Q^+ = Q$) y con $A = 1$ copia B. No tiene combinación prohibida.

El D por flanco hecho con seis puertas NAND puede verse como:

A Un contador de 3 bits

B Tres latches R-S asíncronos de NAND

C Un maestro/esclavo de dos latches D

D Un J-K con dos puertas extra

Las puertas 1-2 y 3-4 forman dos latches de entrada que «capturan» D en el flanco, y las puertas 5-6 forman el latch de salida. Después del flanco, los cambios de D ya no llegan a la salida.

Capítulo 7 · Registros y contadores

Un registro de 4 bits con recirculación contiene 1011 y se desplaza un lugar a la izquierda con recirculación. El nuevo contenido es:

A 0111

B 0101

C 1101

D 0110

Cada bit pasa a la posición de más peso y el que sale por la izquierda entra por la derecha: $1011 \to 0111$. Sin recirculación habría entrado un 0 (0110). 1101 es el desplazamiento a la derecha con recirculación ($1011 \to 1101$).

Un contador de Johnson de n biestables (el último, negado, realimenta al primero) tiene módulo:

A $2^n - 1$

B n

C 2^n

D $2n$

Primero entran n unos y luego n ceros: $2n$ estados (6 con 3 bits, 8 con 4). Es la ventaja de que cambie un solo bit por flanco, aunque se aproveche una parte pequeña de los 2^n estados.

En el contador de Johnson de 3 bits, el estado 010 (que no pertenece a la secuencia útil) evoluciona hacia:

A 000

B Otro ciclo aparte: $010 \rightarrow 101 \rightarrow 010$

C La secuencia útil: 011

D Se queda en 010

Con $D_0 = \overline{Q_2}$, $D_1 = Q_0$, $D_2 = Q_1$, el estado 010 va a 101 y el 101 vuelve a 010: es un ciclo no deseado. Hay que inicializar el contador para empezar en la secuencia buena.

En un contador asíncrono con J-K ($J = K = 1$) donde cada Q es el reloj del siguiente biestable, el sentido de la cuenta depende de:

A El flanco activo: bajada → ascendente, subida → descendente

B Siempre es ascendente

C El valor de J y K

D La frecuencia del reloj

Con flancos de bajada, el siguiente biestable cambia cuando el anterior pasa de 1 a 0, como el acarreo de una suma (0,1,2,3...). Con flancos de subida cambia cuando el anterior pasa de 0 a 1, es decir, cuando «pide prestado»: 7, 6, 5... (comprobado por simulación).

Dos biestables D: el primero tiene $D = \overline{Q_0}$ y flanco de subida de Ck; el segundo tiene $D = Q_0$ y flanco de bajada de Ck. La secuencia Q_1Q_0 (un estado por medio periodo) es:

A 00, 01, 10, 11, 00...

B 00, 01, 11, 10, 00...

C 00, 10, 11, 01, 00...

D 00, 11, 10, 01, 00...

Q0 cambia en las subidas y Q1 copia a Q0 en las bajadas: 00 → (subida) 01 → (bajada) 11 → (subida) 10 → (bajada) 00. Es el código Gray de 2 bits; el circuito mezcla flancos distintos del mismo reloj.

En el contador BCD síncrono con J-K ($J_3 = Q_2Q_1Q_0$, $K_3 = Q_0$, $J_2 = K_2 = Q_1Q_0$, $J_1 = \overline{Q_3}Q_0$, $K_1 = Q_0$, $J_0 = K_0 = 1$), si arranca por error en 1110 (14):

A Pasa a 1001

B Pasa a 0000 directamente

C Pasa a 1111 y después a 0000: se autocorrige

D Se queda en 1110

El J-K simplificado usa las indiferencias de los estados no BCD, pero la simulación de las ecuaciones muestra que 10→11→4, 12→13→4 y 14→15→0: el contador siempre vuelve a la secuencia en como mucho dos flancos.

En el contador síncrono en código Gray de módulo 16 con J-K, la entrada K_0 (el bit menos significativo baja de 1 a 0) vale 1 cuando $Q_0 = 1$ y además:

A $Q_3 \oplus Q_2 \oplus Q_1 = 1$ (paridad impar)

B $Q_3 \oplus Q_2 = 0$

C $Q_2 \cdot Q_1 = 1$

D $Q_3 = 1$

Se comprueba con las 16 transiciones: el bit Q_0 cambia según la paridad de los otros tres bits. $K_0 = Q_3 \oplus Q_2 \oplus Q_1$ y $J_0 = \overline{K_0}$ (observación propia, que simplifica los cuatro términos).

En el módulo 60 con dos 74160, si la NAND que carga 0 en las decenas *solo* mira «decenas = 5» (olvida el RCO de las unidades), la cuenta hace:

A Se bloquea en 50

B $\ldots, 49, 50, 01, 02, \ldots$ (de 50 salta a 01 y nunca llega a 59)

C $\ldots, 59, 60, 00, \ldots$

D $\ldots, 58, 59, 00, \ldots$ como debe

Al llegar a 50 la NAND ya está activa y el siguiente flanco carga 0 en las decenas mientras las unidades siguen contando (50 → 01), antes de que lleguen a 9. Con el RCO se espera a 59 (decenas 5 y unidades 9).

En un contador módulo 1000 con tres 74160, las *centenas* cuentan cuando:

A Solo las unidades valen 9

B Las unidades y las decenas valen 9 a la vez (RCO·RCO)

C Siempre

D Solo las decenas valen 9

Para pasar de 099 a 100 deben desbordarse unidades **y** decenas. Con solo el RCO de las decenas, 090 → 100 saltaría (decenas = 9 sin que las unidades lo estén).

Un 74163 (binario, 4 bits) forma un contador módulo 12 con una NAND que lleva a LOAD (datos $= 0000$) la decodificación de:

A 1010 (10)

B 1100 (12): $Q_3 \cdot Q_2$

C 1011 (11): $Q_3 \cdot Q_1 \cdot Q_0$

D 1111 (15)

La carga es **síncrona**: se detecta el último estado deseado (11) y en el siguiente flanco se carga 0. Si se detectase el 12, el contador pasaría por 12 un ciclo.

Para obtener un pulso de 1 s a partir de un reloj de 1 kHz con 74160 en cascada hacen falta:

A Tres contadores BCD (módulo 1000)

B Cuatro contadores BCD (módulo 10 000)

C Un 74163 (módulo 16)

D Dos contadores BCD (módulo 100)

$1\ kHz/1000 = 1\ Hz$. Tres contadores de módulo 10 dan 1000. La señal de fin de cuenta es $RCO \cdot RCO \cdot RCO$.

Un registro de desplazamiento de 4 bits con una AND detecta la secuencia 1011 en los últimos cuatro flancos. Si la entrada es $1, 0, 1, 1, 0, 1, 1$, la puerta se abre:

A 3 veces

B 2 veces (las dos ventanas de 4 bits solapadas)

C Ninguna

D 1 vez

Las ventanas 1011 de los flancos 1-4 y 4-7 comparten un bit. Con una AND en $Q_3\overline{Q_2}Q_1Q_0$ el solapamiento se detecta de forma natural.

Para separar bloques de tres bits se usa un 74160 con $D = 0001$, y una AND de Q_1 y Q_0 que activa LOAD y habilita la salida. La secuencia de valores del contador es:

A $0, 1, 2, 3, 0, 1, 2, 3, \ldots$

B $1, 2, 3, 1, 2, 3, \ldots$ (tras INICIO empieza en 0, pero también sirve)

C $3, 2, 1, 3, 2, 1, \ldots$

D $0, 1, 2, 0, 1, 2, \ldots$

Cargar 1 hace que el periodo sea 3 flancos (de 1 a 3 y la carga). Tras INICIO el contador vale 0 y tarda también tres flancos en llegar a 3, así que la primera ventana sale bien.

Para multiplexar cuatro displays con un reloj de 200 Hz (un display por flanco), la frecuencia de refresco de cada display es:

A 200 Hz

B 800 Hz

C 50 Hz

D 25 Hz

Cada display se enciende una de cada cuatro veces: $200/4 = 50$ Hz, dentro del margen recomendable de 50-100 Hz.

En el divisor con registro desplazable, 1101 (13) dividido entre 4 se obtiene con:

A Cuatro desplazamientos

B Dos desplazamientos hacia el bit de menor peso: 0011 (3)

C Un desplazamiento: 0110

D Dos desplazamientos hacia el de mayor peso: 0100

Dividir entre 2^k es desplazar k lugares hacia el bit menos significativo: $1101 \rightarrow 0110 \rightarrow 0011$. El cociente es $\lfloor 13/4 \rfloor = 3$; el resto sale por el extremo.

Una rueda con 4 marcas ópticas, medida con un 74163 de 4 bits durante 1 s, alcanza como máximo una velocidad medible de:

A 4 vueltas por segundo (16 pulsos)

B 3 vueltas por segundo (12 pulsos)

C 8 vueltas por segundo

D 16 vueltas por segundo

El 74163 es de módulo 16: a partir de 16 pulsos la cuenta da la vuelta y la medida deja de ser fiable. 16 pulsos con 4 marcas por vuelta son 4 vueltas por segundo. Para más hay que ampliar el módulo con más contadores.

En el marcador de tenis, un juego termina cuando:

A El mayor de los dos tiene al menos 4 puntos y la diferencia es al menos 2

B Alguno llega a 4 puntos

C El mayor tiene exactamente 4 puntos

D La diferencia es 2 o más

Se implementa con dos comparadores: MD (mayor > 3) y MR (diferencia > 1), y $JUEGO = MD \cdot MR$. Con 4-3 o 5-4 el juego sigue (ventaja); con 4-2 o 6-4 termina.

En un contador asíncrono de 0 a 5 con J-K y puesta a cero asíncrona, la NAND de borrado decodifica:

A 101 (5), el último deseado

B 111 (7)

C 110 (6), el primer estado no deseado

D 000

Se deja contar hasta el primer valor que no se quiere (6). El contador llega a tomarlo durante un breve instante (glitch) hasta que el CLEAR actúa. Como el 7 nunca se alcanza, $Q_2 \cdot Q_1$ ya basta.

En el contador BCD asíncrono, la condición mínima ($Q_3 \cdot Q_1$) detecta el primer estado no BCD, que es:

A 1000 (8)

B 0110 (6)

C 1010 (10)

D 1100 (12)

De 0 a 9 solo el 10 es el primer estado con $Q_3 = 1$ y $Q_1 = 1$ a la vez (el 11 y siguientes nunca se alcanzan). Es un recorte a módulo 10 con un instante en 10 (glitch).

Tres J-K ($J = K = 1$) con flanco de subida y cada Q como reloj del siguiente, con una NAND de $\overline{Q_2}Q_1\overline{Q_0}$ a los PRESET, generan la secuencia:

A $0, 1, 2, 3, 4, 0, \ldots$

B $7, 6, 5, 4, 3, 7, \ldots$ (descendente, módulo 5)

C $7, 6, 5, 4, 3, 2, 1, 0$

D $2, 3, 4, 5, 6, 7, 2, \ldots$

La cadena de flancos de subida cuenta hacia abajo. Se decodifica el 2 (primer valor no deseado) y se fuerza 111 = 7: quedan los cinco estados 7-6-5-4-3.

Capítulo 8 · Autómatas finitos (Moore y Mealy)

En un autómata de Mealy la salida depende de:

A Sólo de las entradas anteriores

B Del estado siguiente, no del actual

C El estado y de las entradas actuales

D Sólo del estado

En el Mealy la salida se asocia a la **transición**: se calcula con el estado y la entrada presentes, y puede cambiar sin que cambie el estado. En el Moore depende sólo del estado y cambia sólo con el reloj.

El detector de la secuencia 1011 con solapamiento necesita 4 estados como autómata de Mealy. Como autómata de Moore necesita:

A 6 estados

B 4 estados

C 3 estados

D 5 estados

En el Moore la salida «he detectado» debe ser propia de un estado. El estado que sigue a un 1 se alcanza unas veces con salida 0 y otras (tras 1011) con salida 1, así que se **duplica**: $4 + 1 = 5$. El algoritmo de equivalencia confirma que no hay más fusiones.

Un detector de «tres unos seguidos» (Mealy, con solapamiento) recibe la entrada $1\,1\,1\,1\,0\,1\,1\,1$. Su salida es:

A $0\,0\,1\,0\,0\,0\,0\,1$

B $0\,0\,1\,1\,0\,0\,0\,1$

C $0\,0\,0\,1\,0\,0\,0\,1$

D $0\,0\,1\,1\,0\,0\,1\,1$

Con solapamiento, el cuarto 1 también cierra una racha de tres (los tres últimos son 111). La salida es 1 en los pasos 3, 4 y 8. Sin solapamiento (reiniciar tras detectar) la salida del paso 4 sería 0.

En la tabla de excitación del J-K, para pasar de $Q = 1$ a $Q^+ = 0$ lo *imprescindible* es:

A $K = 0$ y J indiferente

B $K = 1$ y J indiferente

C $J = 0$ y $K = 0$

D $J = 1$ y K indiferente

$J = 0, K = 1$ pone a 0 y $J = 1, K = 1$ conmuta (1→0): ambos valen. Por eso J es indiferente y $K = 1$ es obligatorio. $J = K = 0$ mantiene, $K = 0$ no puede bajar la salida.

En el sumador serie de Mealy (bits menos significativos primero) con un J-K para el acarreo, $J_0 = E_1 E_0$. La entrada K_0 es:

A $E_1 \oplus E_0$

B $E_1 + E_0$

C $E_1 E_0$

D $\overline{E_1}\,\overline{E_0}$

El acarreo sólo debe borrarse cuando **no puede generarse más**: con $E_1 = E_0 = 0$ ($\overline{E_1}\,\overline{E_0}$). Con $E_1 \oplus E_0 = 1$ el acarreo pendiente se mantiene (ni J ni K), y con 11 se pone o se mantiene a 1 (comprobado en las ocho transiciones).

El sumador serie de Mealy tiene 2 estados (acarreo 0 y 1). La versión de *Moore* necesita:

A 4 estados (acarreo y último bit de suma)

B 3 estados

C 2 estados, igual

D 8 estados

La salida S debe estar en el estado, que ha de recordar también la suma calculada: 2 acarreos × 2 valores de S = 4 estados, no fusionables porque tienen salidas distintas. Cuesta dos biestables en lugar de uno y la suma sale un ciclo más tarde.

Un contador de Moore módulo 4 reversible (con E_0 = sentido), que al cambiar de sentido empieza por el primer valor de la nueva secuencia, necesita como mínimo:

A 8 estados

B 4 estados

C 6 estados

D 5 estados

Una misma salida (por ejemplo 2) lleva a salidas distintas según se estuviera subiendo o bajando, así que cada valor tiene dos estados: $A_0 \dots A_3$ y $D_3 \dots D_0$. El algoritmo de equivalencia no fusiona ninguno.

En el contador módulo 8 con habilitación E_0 y J-K (con $J = K$ en cada biestable), el bit q_2 debe conmutar cuando:

A $E_0(q_1 + q_0) = 1$

B $q_1 q_0 = 1$, sin mirar la habilitación

C $E_0\, q_1\, q_0 = 1$ (habilitado y acarreo de los dos bits bajos)

D $E_0\, q_2 = 1$

q_2 cambia cuando el incremento acarrea desde $q_1 q_0 = 11$ y el contador está habilitado: $J_2 = K_2 = E_0 q_1 q_0$. Sin E_0 contaría siempre; con una OR contaría de más (ya desde 001).

El autómata de Mealy que da el bit de paridad de un mensaje de 3 bits tiene en su primer diseño (árbol) 7 estados. Al reducirlo quedan:

- **A** 5 estados
- **B** 3 estados
- **C** 6 estados
- **D** 4 estados

Se fusionan los dos estados «los dos primeros bits tienen paridad par» (00 y 11) y los dos de paridad impar (01 y 10): $7 - 2 = 5$. Para la paridad sólo importa **cuántos unos**, no cuáles bits fueron.

El codificador de prioridad 4:2 serie, recibiendo primero el bit menos significativo, necesita 7 estados. Recibiendo primero el más significativo necesita:

- **A** 4 estados
- **B** 5 estados
- **C** 10 estados (más, no menos)
- **D** 7 estados

Con el más significativo primero hay que decidir ya, pero luego se deben dejar pasar los bits restantes: se forman cadenas de estados «de relleno» (X) y el total sube a 10; el algoritmo de equivalencia no puede fusionarlos.

Un comparador serie de Moore recibe primero el bit menos significativo. Con $A = 0110$ (6) y $B = 0101$ (5), tras los cuatro ciclos el estado es:

- **A** MAYOR
- **B** MENOR
- **C** IGUAL

D No se puede saber

Los pares E_1E_0 en orden son 01, 10, 11, 00. Manda siempre **el último bit que difiere**: el primero da MENOR, el segundo MAYOR, y los dos siguientes (11 y 00) no cambian nada: MAYOR.

En el comparador serie que recibe primero el bit más significativo, con $A = 1010$ (10) y $B = 1001$ (9), el estado tras el cuarto bit es:

A MAYOR (el primer bit que difiere decide para siempre)

B IGUAL

C MENOR (el último bit decide)

D Depende del biestable

Los pares son 11, 00, 10, 01. Con el más significativo primero, el primer par que difiere (10) fija MAYOR, y MAYOR/MENOR son estados **absorbentes**: el 01 final no lo cambia. Con el orden inverso sí mandaría el último.

En los autómatas «industriales», las entradas (pulsadores, sensores) son asíncronas. Antes de llegar al autómata se las:

A Codifica con un decodificador

B Retrasa con puertas lógicas

C Cuenta con un contador

D Sincroniza con biestables D que usan el reloj del autómata

Un biestable D por flanco muestrea la entrada en el flanco de reloj y entrega una señal sincronizada. El autómata sólo «mira» las entradas en cada flanco; con un reloj rápido es casi como si fuera continuo.

En el autómata de la puerta (entradas PAC: pulsador, sensor de abierta, sensor de cerrada), las combinaciones 011 y 111 se tratan como imposibles porque:

A La puerta no puede estar abierta y cerrada a la vez ($A = C = 1$)

B El pulsador no puede estar activo con un sensor

C Dos biestables no admiten esas combinaciones

D Producirían un estado sin codificar

Esas dos filas tienen $A = C = 1$. Quedan seis entradas posibles por estado, y las combinaciones imposibles se usan como **indiferentes** al minimizar.

En el limpiaparabrisas con detector de reposo R (entradas MPR), con 001 la escobilla en MARCHA sigue y en MAPA se detiene. Eso demuestra que:

A Hacen falta tres estados: la misma entrada exige respuestas distintas según el pasado

B La entrada 001 es imposible

C Dos estados bastan si se añade una salida

D El detector R sobra

MARCHA y MAPA tienen la misma salida (motor encendido) pero **no son equivalentes**: ante 001 uno sigue y el otro pasa a PARO. Esa diferencia de comportamiento es lo que obliga a distinguirlos con un estado propio.

Un motor que se pone en marcha o se para al soltar su pulsador (activación en el paso de 1 a 0) necesita, como mínimo:

A 2 estados

B 3 estados

C 8 estados

D 4 estados (parado, pulsado-esperando, en marcha, pulsado-esperando)

Hay que distinguir «he pulsado y aún no he soltado» de «ya ha actuado». Con dos estados, mientras el pulsador siguiera a 1 el motor conmutaría en cada ciclo de reloj. Los dos estados de espera evitan esa oscilación.

Un detector de flanco de subida síncrono (salida de un ciclo de reloj) necesita 3 estados (INICIO, PULSO y FINAL). El que genera un pulso en los dos flancos necesita:

- **A** 4 estados
- **B** 3 estados, igual
- **C** 6 estados
- **D** 2 estados

Hay que recordar además que la entrada se mantiene a 1 antes de poder detectar la bajada: INICIO, PULS1, MANT y PULS0. Los dos estados de pulso son los que tienen salida 1.

El sensor de peso da 5, 5, 5, 6, 5, 5, 5, 5 en pulsos de reloj consecutivos. La alarma (peso 5 durante al menos 4 pulsos seguidos) se activa:

- **A** En el cuarto pulso
- **B** En el octavo pulso
- **C** En el séptimo pulso
- **D** Nunca

El 6 interrumpe la cuenta de cincos (se parte de 1 en el quinto pulso) y los cuatro últimos son cincos seguidos: sólo el octavo pulso cierra una racha de cuatro. Es un autómata con estados P51...P54.

En la caja fuerte de combinación 1-1-0, el usuario pulsa P_1, suelta, y a continuación pulsa P_0. El autómata:

- **A** Abre la caja

B Espera al tercer dígito antes de dar la alarma

C Vuelve al estado inicial

D Pasa a ALARM en cuanto se pulsa P_0

Tras un 1 correcto se espera otro 1: el estado S_1 con la entrada 01 (pulsar P_0) va directamente a ALARM, que es absorbente. Se sale de ahí sólo con el RESET.

El transmisor serie envía, tras soltar el pulsador, una trama con arranque 0, el bit E_1, el bit E_0 y un 0 final, con la línea en reposo a 1. Con $E_1E_0 = 10$ la salida tras soltar P es:

A $0, 0, 1, 0$

B $0, 1, 0, 0$ y luego 1 (reposo)

C $1, 0, 1, 0$

D $0, 1, 0, 1$

El arranque es 0, después $E_1 = 1$, $E_0 = 0$ y un 0 final; a continuación la línea vuelve a 1. Cada uno de esos símbolos es un estado de Moore (EMPIE, UNO1, DOS0, FINAL).

En el autómata de calificaciones (tres bits por examen, 10 estados), INICIO y SUSP3 se pueden fusionar porque:

A Tienen la misma salida y los mismos estados siguientes ante 0 y ante 1

B Los dos están antes de EMPIE

C Tienen el mismo nombre de estado

D Ambos son estados con salida 1

Dos estados son equivalentes si dan las mismas salidas y, para cada entrada, llevan a estados equivalentes: INICIO y SUSP3 hacen 000 y van a SUSP1 con 0 y a EMPIE con 1. Quedan **9 estados**.

CONSULTA

Formulario por temas

Las fórmulas y reglas que hay que saber de memoria, tema a tema. Al final, la versión de una página para imprimir.

Esto es lo que debes **saber sin mirar**. Si una fórmula te suena a chino, vuelve a la teoría del tema: aprender una fórmula sin saber de dónde sale falla en el examen.

Tema 2 · Sistemas de numeración, operaciones y códigos

- N combinaciones $\Rightarrow n = \lceil \log_2 N \rceil$ bits.
- C2 de n bits: $-2^{n-1} \le N \le 2^{n-1} - 1$.
- $-N = \overline{N} + 1$ (invertir y sumar 1).
- Gray: $g = b \oplus (b \gg 1)$.
- Suma BCD: si cifra > 9, sumar 0110.

Tema 3 · Álgebra de Boole y funciones lógicas

- $\overline{A + B} = \overline{A}\,\overline{B}$ y $\overline{AB} = \overline{A} + \overline{B}$.
- $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$.
- $A \oplus B = A\overline{B} + \overline{A}B$.
- n variables: 2^n filas y 2^{2^n} funciones distintas (con 4 variables, 65536).

Tema 4 · Familias lógicas

- $NM_H = V_{OH} - V_{IH}$, $NM_L = V_{IL} - V_{OL}$.
- fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$.
- $P_{din} = C\,V_{DD}^2\,f$.
- Retardo de una ruta: suma de los t_{pd} de sus puertas.

Tema 5 · Funciones de la lógica combinacional

- $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$.
- $A - B = A + \overline{B} + 1$.
- MUX $2^k : 1$ implementa cualquier función de $k + 1$ variables.
- Salida i de un decodificador $= m_i$ (minitérmino).

Tema 6 · Biestables (flip-flops) y dispositivos relacionados

- $Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$.

- $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
- Divisor por 2: $f_{out} = f_{clk}/2$.

Tema 7 · Memorias y circuitos lógicos programables

- bits totales $= 2^k \cdot m$; $k = \log_2 N$.
- Chips necesarios $= \dfrac{N_{total}}{N_{chip}} \cdot \dfrac{m_{total}}{m_{chip}}$.
- Una ROM $2^n \times m$ implementa m funciones de n variables.

Tema 8 · Contadores

- $n = \lceil \log_2 N \rceil$; estados no usados: $2^n - N$.
- $f_{out} = f_{clk}/N$.
- Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$.
- Asíncrono: $t_{total} = n \cdot t_{pd}$.

Tema 9 · Registros de desplazamiento

- Anillo: n estados; Johnson: $2n$; LFSR máximo: $2^n - 1$.
- Serie → paralelo de n bits: n ciclos de reloj.

Tema 10 · Análisis y síntesis de circuitos secuenciales síncronos

- $n = \lceil \log_2(\#\text{estados}) \rceil$.
- Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$.
- D: $D = Q^+$. T: $T = Q \oplus Q^+$. JK: usar tabla de excitación.

Tema 13 · Introducción al diseño digital con VHDL

- Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos.
- Bits de un contador: $\lceil \log_2 N \rceil$.
- Una `case` completa en un proceso combinatorio: añade siempre `when others`.

T5 · Bloques combinacionales (tabla de decisión)

Quiero...	Uso...
Sumar dos números de n bits	n sumadores completos en cascada (acarreo de uno al siguiente)
Restar $A - B$	Sumador con B invertida y $C_{in} = 1$

Quiero...	Uso...
Comparar	Comparador: salidas $A > B$, $A = B$, $A < B$
Elegir una entre 2^k	Multiplexor con k bits de selección
Activar una salida según un código	Decodificador n a 2^n
Mostrar un dígito	Decodificador BCD a 7 segmentos
Dividir/multiplicar por 2^k	Desplazar k posiciones (sin puertas)

T11 y T12 · Descripciones y lógica programable

- Un mismo circuito se describe con **ecuaciones**, **tabla de verdad**, **diagrama de estados**, **bloques** o **código** (VHDL). Los distintos niveles de abstracción son comportamiento, flujo de datos y estructura.
- LUT de k entradas: memoria de 2^k bits; implementa cualquier función de k variables.
- Una función de más de k variables se reparte entre varias LUT.

La versión de **una sola página**, para imprimir, está justo debajo y en el PDF `formulario.pdf`.
Comprueba en las normas de tu convocatoria si puedes llevar un formulario al examen.

CONSULTA

Formulario de una página

Todo lo esencial en una hoja: numeración, Boole, bloques, biestables, contadores, autómatas y VHDL.

Tema	Lo imprescindible
T2 Numeración	$n = \lceil \log_2 N \rceil$ bits. C2 de n bits: $-2^{n-1} \dots 2^{n-1}-1$; $-N = \overline{N}+1$. Desbordamiento: acarreo que entra al signo $\neq$ acarreo que sale. Gray: $g = b \oplus (b \gg 1)$. BCD: cifra > 9 suma 0110.
T3 Boole	$\overline{A+B} = \overline{A}\,\overline{B}$; $\overline{AB} = \overline{A}+\overline{B}$. $A + AB = A$; $AB + A\overline{B} = A$; $AB + \overline{A}C + BC = AB + \overline{A}C$. $A \oplus B = A\overline{B} + \overline{A}B$. Karnaugh: grupos de 2^k unos adyacentes (también en los bordes); las X solo se usan si ayudan. NAND-NAND = suma de productos.
T4 Familias	$NM_H = V_{OH} - V_{IH}$; $NM_L = V_{IL} - V_{OL}$. Fan-out $= \min(I_{OL}/I_{IL},\ I_{OH}/I_{IH})$. $P_{din} = CV_{DD}^2 f$.
T5 Combinacional	Sumador completo: $S = A \oplus B \oplus C_{in}$; $C_{out} = AB + C_{in}(A \oplus B)$. $A - B = A + \overline{B} + 1$. MUX 2^k:1 implementa funciones de $k+1$ variables. Decodificador: salida $i = m_i$.
T6 Biestables	$Q^+_{JK} = J\overline{Q} + \overline{K}Q$; $Q^+_D = D$; $Q^+_T = T \oplus Q$. Excitación JK: $0 \to 0$: $J = 0, K = X$; $0 \to 1$: $J = 1, K = X$; $1 \to 0$: $J = X, K = 1$; $1 \to 1$: $J = X, K = 0$. $T_{clk} \geq t_{cq} + t_{pd} + t_{su}$.
T7 Memorias	bits $= 2^k \cdot m$. Chips $= \frac{N}{N_c} \cdot \frac{m}{m_c}$. ROM $2^n \times m$ = m funciones de n variables.
T8 Contadores	$n = \lceil \log_2 N \rceil$; $f_{out} = f_{clk}/N$. Síncrono binario: $T_i = Q_{i-1} \cdots Q_0$. Asíncrono: $t = n\,t_{pd}$. Estados no usados: $2^n - N$ (¡cuidado con el bloqueo!).
T9 Registros	Anillo: n estados. Johnson: $2n$. LFSR máximo: $2^n - 1$. Serie a paralelo de n bits: n ciclos.
T10 Autómatas	Estados $\to n = \lceil \log_2(\#) \rceil$ biestables. Moore: $Z = f(Q)$. Mealy: $Z = f(Q, X)$. D: $D = Q^+$. T: $T = Q \oplus Q^+$. Detector con solapamiento: el estado es el mayor prefijo que es sufijo.

Tema	Lo imprescindible
T12–T13 FPGA y VHDL	LUT de k entradas: 2^k bits. Divisor: $DIV = f_{in}/f_{out}$; conmuta cada $DIV/2$ ciclos. `case` completo: `when others`. Lo concurrente (fuera de `process`) ocurre a la vez.

CONSULTA

Glosario

Los términos de la asignatura, en orden alfabético y con una definición corta.

Úsalo cuando un enunciado de examen use una palabra que no recuerdes. Las definiciones son cortas a propósito: para entender el concepto, ve al tema indicado en el índice.

Acarreo anticipado

Técnica que calcula los acarreos en paralelo para acelerar un sumador.

Alta impedancia (Z)

Estado de una salida tri-state en el que está eléctricamente desconectada.

Architecture

Parte del código VHDL que define el funcionamiento interno.

Autómata síncrono

Circuito secuencial cuyos biestables cambian todos con el mismo reloj.

BCD

Decimal codificado en binario: cada cifra decimal en 4 bits.

Biestable

Circuito de dos estados estables que almacena un bit.

Carga paralela

Entrada que fuerza al contador a un valor dado.

CLB

Bloque lógico configurable de una FPGA (LUT y biestables).

Codificación de estados

Asignación de un código binario a cada estado.

Codificador de prioridad

Da el código de la línea activa de mayor prioridad.

Colector abierto

Salida que solo fuerza el 0; el 1 lo pone una resistencia externa.

Complemento a dos

Representación de enteros con signo: el MSB pesa -2^{n-1}.

Contador asíncrono

Contador en el que cada biestable usa como reloj la salida del anterior.

Contador en anillo

Registro de desplazamiento realimentado con un único 1 que circula.

Contador Johnson

Registro de desplazamiento que realimenta la salida negada; $2n$ estados.

Contador módulo N

Contador que recorre N estados y vuelve al inicial.

Contador síncrono

Contador en el que todos los biestables reciben el mismo reloj.

Código Gray

Código en el que dos palabras consecutivas difieren en un solo bit.

Decodificador

Activa una única salida según el código binario de entrada.

Desbordamiento

El resultado de una operación no cabe en el número de bits disponible.

Descripción estructural

Descripción por componentes e interconexiones.

Descripción funcional

Descripción por comportamiento.

Diagrama de estados

Grafo con un nodo por estado y una flecha por transición.

Distancia de Hamming

Número de bits en que difieren dos palabras de un código.

EDA

Herramientas de diseño electrónico asistido por ordenador.

Entity

Parte del código VHDL que define las entradas y salidas de un módulo.

Estado

Resumen de la historia del circuito que basta para decidir qué hará.

Fan-out

Número máximo de entradas que una salida puede alimentar.

Fin de cuenta

Salida que avisa de que el contador llega a su último estado, para encadenar contadores.

Flanco

Instante en que la señal de reloj pasa de 0 a 1 (subida) o de 1 a 0 (bajada).

FPGA

Dispositivo lógico reprogramable formado por bloques lógicos y una red de interconexión.

Implicante primo

Grupo de unos que no se puede ampliar más en el mapa de Karnaugh.

Indiferente (X)

Combinación de entradas que no ocurre o cuya salida da igual.

Latch inferido

Memoria no deseada que aparece cuando una señal no se asigna en todos los casos.

LFSR

Registro de desplazamiento con realimentación lineal (XOR) que genera secuencias pseudoaleatorias.

LUT

Tabla de consulta: memoria que almacena la tabla de verdad de una función de pocas variables.

Margen de ruido

Cuánto ruido tolera una conexión sin que se interprete mal el nivel lógico.

Maxitérmino

OR con todas las variables que vale 0 en una sola fila de la tabla.

Metaestabilidad

Estado intermedio e impredecible tras violar los tiempos de setup o hold.

Minitérmino

AND con todas las variables que vale 1 en una sola fila de la tabla.

MSI

Integración de media escala: bloques como sumadores, multiplexores o decodificadores.

Multiplexor

Selector que copia a la salida una de varias entradas según un código de selección.

PAL

Matriz con plano AND programable y plano OR fijo.

Paridad

Bit extra que hace par (o impar) el número de unos de una palabra.

PLA

Matriz lógica programable con plano AND y plano OR programables.

Proceso

Bloque `process` de VHDL que describe comportamiento.

Puerta universal

Puerta con la que se puede construir cualquier función lógica (NAND o NOR).

RAM

Memoria de lectura y escritura, volátil.

Rebote

Oscilaciones breves de un contacto mecánico al abrir o cerrar.

Registro de desplazamiento

Cadena de biestables en la que cada uno toma el valor del anterior en cada flanco.

Restricciones (UCF)

Fichero que asocia las señales del diseño con los pines físicos de la FPGA.

Retardo de propagación

Tiempo que tarda la salida en reaccionar a un cambio de la entrada.

ROM

Memoria de solo lectura: guarda una tabla fija.

Setup / hold

Tiempos de preparación y mantenimiento de las entradas alrededor del flanco.

Síntesis

Transformación de una descripción de alto nivel en una red de puertas y biestables.

Síntesis

Conversión de la descripción en puertas y biestables.

Tabla de excitación

Dice qué entradas del biestable hacen falta para ir de Q a Q^+.

VHDL

Lenguaje de descripción de hardware.

Volátil

Que pierde su contenido al quitar la alimentación.

CONSULTA

Plan de estudio

Un plan de 4 semanas y uno de 7 días, con qué hacer cada sesión y cómo saber si vas bien.

Ningún plan sustituye a hacer problemas con lápiz. La regla de oro: **cada hora de teoría va seguida de media hora de ejercicios** de ese tema.

> 💡 **Antes de empezar**
>
> No hay fechas aquí a propósito: usa la de tu convocatoria. Cuenta hacia atrás desde ella y reparte las semanas. Marca lo hecho en la lista de progreso de El examen y cómo estudiar.

Plan de 4 semanas (1–2 horas al día)

Semana	Temas y bloques	Qué haces	Cómo sabes que vas bien		
1 · Base	[[t02	T2]] numeración y [[t03	T3]] Boole y Karnaugh	Teoría + tests de autoeval + hoja de Karnaugh K1–K5	Haces un mapa de 4 variables con indiferentes sin dudar
2 · Combinacional	[[t04	T4]] familias y [[t05	T5]] bloques	Sumador, comparador, MUX y displays; hoja K6–K17; primer examen resuelto (examenes)	Diseñas el recinto o la media de cuatro números sin mirar
3 · Secuencial	[[t06	T6]]–[[t09	T9]] biestables, memorias, contadores y registros	Tablas de excitación; contadores con JK; simulacro 1	Tienes la tabla de excitación del JK de memoria

Semana	Temas y bloques	Qué haces	Cómo sabes que vas bien			
4 · Autómatas y VHDL	[[t10	T10]], [[t11	T11]], [[t12	T12]], [[t13	T13]] y las prácticas	Hoja de autóma detecto con solapar revisar simulac

Plan exprés de 7 días

Día	Qué haces		
1	[[t02	T2]] y [[t03	T3]]: numeración, C2, BCD, Boole y Karnaugh. 10 mapas.
2	[[t05	T5]]: sumador, comparador, MUX, decodificador y display. Examen de diciembre completo (examenes).	
3	[[t06	T6]] y [[t08	T8]]: biestables y contadores; tabla de excitación JK. Hoja de

Día	Qué haces						
			autómatas A1, A2.				
4	[[t10	T10]]: autómatas. Detectores A3, A5 y A7; sumador serie A9.					
5	[[t07	T7]], [[t09	T9]], [[t12	T12]] y [[t13	T13]]: memorias, registros, FPGA y VHDL. Repasa las prácticas ([[p1	Práctica 1]] a [[p567	Prácticas 5–7]]).
6	**Simulacro 1** en 2 horas (simulacros) y corrección. Repite los fallos.						
7	**Simulacro 2**, formulario de una página y glosario. Descansa pronto.						

Cómo repartir el tiempo según los exámenes

El recuento de problemas por tipo está en El examen y cómo estudiar. Con solo tres exámenes no es una regla, pero orienta: **bloques MSI**, **aritmética y códigos** y **autómatas** aparecen en los tres; los contadores con biestables, en uno. Dedica la mitad del tiempo a esos tres grupos.

El día del examen

- Lee **todos** los enunciados y empieza por el que más seguro estés.
- En cada problema, **dibuja lo que te piden** (esquema, diagrama de estados, tabla) y explica en una frase por qué.
- Un cronograma sin explicación no cuenta: escribe qué ocurre en cada flanco.
- Guarda **10 minutos** para revisar: ¿cabe el resultado en los bits que has elegido? ¿Has cubierto el solapamiento? ¿Has puesto reset?

Guía no oficial de Electrónica Digital (UPCT) · Última actualización: 9 de noviembre de 2026. Los resultados numéricos y las ecuaciones se han comprobado con código.